Java Handbook

From first program to modern Java 25 · OOP · collections · streams · concurrency & virtual threads · JVM internals · testing · interview prep

21 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Java and the JVM**
   - 1.1 What Java is
   - 1.2 The Java ecosystem: SE, EE, OpenJDK and vendors
   - 1.3 JDK, JRE and JVM
   - 1.4 Installing a JDK and checking it works
   - 1.5 Write → compile → run
   - 1.6 Bytecode and class files
   - 1.7 How the JVM runs your code: class loading, interpreter, JIT
   - 1.8 Memory and garbage collection — an overview
   - 1.9 jshell — the Java REPL
   - 1.10 Launching source files directly
   - 1.11 IDEs, editors and build tools
   - 1.12 Java releases, LTS and JEPs
   - Chapter TL;DR
2. **Chapter 2 — Basics: Types, Variables and Operators**
   - 2.1 Statements, identifiers, keywords and comments
   - 2.2 Variables
   - 2.3 Primitive types
   - 2.4 Literals
   - 2.5 Reference types and `null`
   - 2.6 `var` — local variable type inference (Java 10)
   - 2.7 Type conversion: widening, narrowing, casting, promotion
   - 2.8 Operators
   - 2.9 Operator precedence and associativity
   - 2.10 Integer overflow
   - 2.11 Floating-point numbers
   - 2.12 Strings and text blocks
   - 2.13 Console output and input
   - Chapter TL;DR
3. **Chapter 3 — Control Flow**
   - 3.1 `if`, `else if`, `else`
   - 3.2 The classic `switch` statement
   - 3.3 Arrow labels and `switch` expressions (Java 14)
   - 3.4 Choosing between `if`, ternary and `switch`
   - 3.5 Pattern matching for `instanceof` (Java 16)
   - 3.6 Pattern matching for `switch` (Java 21)
   - 3.7 Record patterns (Java 21) and unnamed patterns (Java 22)
   - 3.8 Loops
   - 3.9 `break`, `continue` and labels
   - 3.10 Loop pitfalls
   - Chapter TL;DR
4. **Chapter 4 — Methods and Arrays**
   - 4.1 Methods — declaring and calling
   - 4.2 Parameters — Java is always pass-by-value
   - 4.3 Overloading
   - 4.4 Varargs
   - 4.5 Recursion
   - 4.6 Arrays
   - 4.7 Multidimensional and jagged arrays
   - 4.8 The `java.util.Arrays` toolbox
   - 4.9 `main`, command-line arguments, and simpler `main` (Java 25)
   - Chapter TL;DR
5. **Chapter 5 — OOP: Classes and Objects**
   - 5.1 Classes and objects
   - 5.2 Fields, instance methods and `this`
   - 5.3 Constructors
   - 5.4 Packages and imports
   - 5.5 Static members
   - 5.6 Access modifiers
   - 5.7 Encapsulation
   - 5.8 The `final` keyword
   - 5.9 Immutability basics
   - 5.10 Initialisation order
   - Chapter TL;DR
6. **Chapter 6 — OOP: Inheritance, Interfaces and More**
   - 6.1 Inheritance and `super`
   - 6.2 What a subclass gets — members, hiding and `Object`
   - 6.3 Overriding
   - 6.4 Polymorphism and casting
   - 6.5 Abstract classes
   - 6.6 Interfaces
   - 6.7 Sealed classes and interfaces (Java 17)
   - 6.8 Records
   - 6.9 Enums
   - 6.10 Nested classes: static nested and inner
   - 6.11 Local classes, anonymous classes and lambdas
   - 6.12 Choosing: abstract class, interface, record, enum or plain class
   - 6.13 Object methods: equals, hashCode and toString
   - Chapter TL;DR
7. **Chapter 7 — Core APIs: Strings, Numbers, Dates and Randomness**
   - 7.1 Strings: immutability, the pool and memory
   - 7.2 The `String` API toolkit
   - 7.3 `StringBuilder`, `StringBuffer` and `StringJoiner`
   - 7.4 Formatting text and numbers
   - 7.5 `Math`
   - 7.6 Wrapper classes and autoboxing
   - 7.7 `BigInteger` and `BigDecimal`
   - 7.8 The `java.time` model

- 7.9 Local dates, times, adjusters and `Period`
- 7.10 Zones, instants, `Duration` and daylight saving
- 7.11 Formatting and parsing dates
- 7.12 Random numbers
- Chapter TL;DR

8. **Chapter 8 — Exceptions**
   - 8.1 What an exception is
   - 8.2 The `Throwable` hierarchy: checked, unchecked and errors
   - 8.3 Handle or declare: checked exceptions and `throws`
   - 8.4 `try` / `catch` / `finally` and control flow
   - 8.5 Multi-catch, catch order and rethrowing
   - 8.6 try-with-resources
   - 8.7 Throwing exceptions and writing custom ones
   - 8.8 Chaining and exception translation
   - 8.9 Reading stack traces
   - 8.10 Best practices and anti-patterns
   - Chapter TL;DR
9. **Chapter 9 — Generics**
   - 9.1 Why generics
   - 9.2 Generic classes and records
   - 9.3 Generic interfaces
   - 9.4 Generic methods and type inference
   - 9.5 Bounded type parameters
   - 9.6 Invariance, wildcards and PECS
   - 9.7 Type erasure: how generics really work
   - 9.8 Restrictions and workarounds (consequences of erasure)
   - 9.9 Designing generic APIs
   - Chapter TL;DR
10. **Chapter 10 — Collections**
    - 10.1 The Collections Framework and its hierarchy
    - 10.2 Lists: `ArrayList` and `LinkedList`
    - 10.3 Sets: `HashSet`, `LinkedHashSet`, `TreeSet`
    - 10.4 Maps: `HashMap`, `LinkedHashMap`, `TreeMap`
    - 10.5 `HashMap` internals
    - 10.6 Queues, deques and priority queues
    - 10.7 Iterators and fail-fast behaviour
    - 10.8 Ordering: `Comparable` and `Comparator`
    - 10.9 The `Collections` utility class
    - 10.10 Unmodifiable and immutable collections
    - 10.11 Sequenced collections (Java 21)
    - 10.12 Big-O cheat sheet and a measured demo
    - 10.13 Choosing a collection
    - Chapter TL;DR
11. **Chapter 11 — Lambdas and Functional Programming**
    - 11.1 Why lambdas: passing behaviour
    - 11.2 Functional interfaces and lambda syntax
    - 11.3 The built-in functional interfaces (`java.util.function`)
    - 11.4 Method references
    - 11.5 Variable capture and "effectively final"
    - 11.6 Composing functions
    - 11.7 `Optional`
    - 11.8 Functional style in everyday Java
    - Chapter TL;DR
12. **Chapter 12 — Streams**
    - 12.1 What a stream is
    - 12.2 Laziness: how a pipeline really executes
    - 12.3 Intermediate operations
    - 12.4 Terminal operations and `reduce`
    - 12.5 Collectors
    - 12.6 Primitive streams
    - 12.7 Streams, `Optional` and nulls
    - 12.8 Parallel streams
    - 12.9 Stream gatherers (Java 24)
    - 12.10 Common mistakes
    - 12.11 Streams vs loops, and a real-world pipeline
    - Chapter TL;DR
13. **Chapter 13 — I/O and NIO**
    - 13.1 The big picture: `java.io` vs NIO.2
    - 13.2 `Path`: naming files and directories
    - 13.3 `Files`: managing files and directories
    - 13.4 Reading and writing whole files
    - 13.5 Byte streams and character streams
    - 13.6 Buffering
    - 13.7 Charsets and encodings
    - 13.8 Console and standard streams
    - 13.9 Binary data, channels and buffers
    - 13.10 Serialization (and why to avoid it)
    - 13.11 Properties files
    - 13.12 Working with JSON
    - 13.13 Best practices and common mistakes (checklist)
    - Chapter TL;DR
14. **Chapter 14 — Concurrency I: Threads, Locks and the `java.util.concurrent` Toolkit**
    - 14.1 Concurrency basics
    - 14.2 Creating and starting threads
    - 14.3 Thread lifecycle
    - 14.4 Interruption: cooperative cancellation
    - 14.5 Race conditions
    - 14.6 `synchronized` and intrinsic locks
    - 14.7 `volatile` and visibility
    - 14.8 `wait` / `notify`: guarded blocks
    - 14.9 Explicit locks: `ReentrantLock`, `Condition`, `ReadWriteLock`, `StampedLock`
    - 14.10 Atomic variables
    - 14.11 Executors and thread pools
    - 14.12 Concurrent collections
    - 14.13 Synchronizers: `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `Phaser`
    - 14.14 Designing thread-safe classes
    - Chapter TL;DR

15. **Chapter 15 — Concurrency II: Modern Concurrency**
    - 15.1 `Future` and its limits
    - 15.2 `CompletableFuture`
    - 15.3 Fork/Join
    - 15.4 Virtual threads
    - 15.5 Pinning and JEP 491
    - 15.6 `ThreadLocal` and scoped values
    - 15.7 Structured concurrency (preview)
    - 15.8 The Java Memory Model and happens-before
    - 15.9 Liveness: deadlock, livelock, starvation
    - 15.10 Best practices and choosing a concurrency model
    - Chapter TL;DR
16. **Chapter 16 — JVM Internals and Performance**
    - 16.1 The JVM's architecture
    - 16.2 Class loading in depth
    - 16.3 Runtime memory areas
    - 16.4 Object layout basics
    - 16.5 Garbage-collection algorithms
    - 16.6 Collectors compared on a real workload
    - 16.7 GC tuning basics
    - 16.8 JIT compilation, inlining and escape analysis
    - 16.9 Benchmarking: why naive timing lies, and JMH
    - 16.10 Profiling and diagnostic tools
    - 16.11 Memory leaks
    - 16.12 Modules (JPMS) basics
    - 16.13 Operating-system fundamentals for engineers
    - Chapter TL;DR
17. **Chapter 17 — Modern Java: 17 to 25**
    - 17.1 Reading a feature's status
    - 17.2 The version table
    - 17.3 The Java 17 baseline
    - 17.4 Java 18
    - 17.5 Java 19 and 20: the preview years
    - 17.6 Java 21 LTS
    - 17.7 Java 22
    - 17.8 Java 23
    - 17.9 Java 24
    - 17.10 Java 25 LTS
    - 17.11 Still preview or incubating in Java 25 — and what came next
    - 17.12 Upgrading 17 → 21 → 25: removals and deprecations
    - 17.13 In the interview
    - Chapter TL;DR
18. **Chapter 18 — Testing, Build and Tooling**
    - 18.1 The shape of a real Java project
    - 18.2 Maven
    - 18.3 Gradle
    - 18.4 JUnit: your first automated tests
    - 18.5 Parameterized, repeated and dynamic tests
    - 18.6 Test doubles and Mockito
    - 18.7 AssertJ fluent assertions
    - 18.8 Reading a failing test report
    - 18.9 Test design
    - 18.10 Logging
    - 18.11 JDBC basics
    - 18.12 Packaging: JAR files
    - 18.13 Other everyday tools
    - 18.14 Where next: Spring Boot and friends
    - Chapter TL;DR
19. **Chapter 19 — Java for Coding Interviews**
20. **Part 1 — The Java DSA toolkit**
    - 19.1 Testing your solutions: `assert` is off, use a helper
    - 19.2 Arrays, strings and char arithmetic
    - 19.3 `HashMap` and `HashSet` idioms
    - 19.4 `ArrayDeque`, `PriorityQueue` and `TreeMap`
    - 19.5 Sorting with comparators
    - 19.6 Numeric and equality pitfalls
    - 19.7 Complexity cheat table
21. **Part 2 — Twenty patterns**
    - 19.8 Two pointers
    - 19.9 Sliding window
    - 19.10 Hashing
    - 19.11 Prefix sums
    - 19.12 Stacks and monotonic stacks
    - 19.13 Heaps and top-k
    - 19.14 Binary search (and binary search on the answer)
    - 19.15 Intervals
    - 19.16 BFS and DFS: trees, grids and graphs
    - 19.17 Topological sort
    - 19.18 Union-find (disjoint set union)
    - 19.19 Dynamic programming
    - 19.20 Backtracking
    - 19.21 Tries
    - 19.22 Linked lists and fast/slow pointers
    - 19.23 Trees: recursion templates, LCA and serialisation
    - 19.24 Greedy
    - 19.25 Sort and search: quicksort, quickselect, Dutch flag, merge sort on a list
    - 19.26 Bit manipulation
    - 19.27 Math and geometry
    - 19.28 Talking through a solution in Java
22. **Part 3 — Java interview questions**
    - 19.29 Java interview Q&A
    - Chapter TL;DR
23. **Chapter 20 — Cheat Sheets and Glossary**

- 20.1 Syntax, types and primitive ranges
- 20.2 Operators and precedence
- 20.3 Control flow, `switch` and patterns
- 20.4 Methods and arrays
- 20.5 OOP: classes, interfaces, records, enums and sealed types
- 20.6 Core APIs: String, numbers, dates
- 20.7 Exceptions
- 20.8 Generics and PECS
- 20.9 Collections: big-O and which to choose
- 20.10 Functional interfaces, lambdas and `Optional`
- 20.11 Streams and collectors
- 20.12 I/O and NIO.2
- 20.13 Concurrency
- 20.14 The JVM: memory, GC, flags and tools
- 20.15 Modern Java by version: 17 to 25
- 20.16 Testing, build and tooling commands
- 20.17 Interview question index
- 20.18 Where next
- 20.19 Glossary (A–Z)
- Chapter TL;DR

24. **Chapter 21 — Interview Question Bank**
    - How to use this bank
    - 1. Platform and language basics
    - 2. OOP: classes, inheritance and polymorphism
    - 3. Strings
    - 4. equals, hashCode, immutability and cloning
    - 5. Exceptions
    - 6. Generics
    - 7. Collections
    - 8. Lambdas, functional interfaces and Optional
    - 9. Streams
    - 10. Modern Java (9 → 25)
    - 11. Concurrency
    - 12. JVM, memory and garbage collection
    - 13. I/O, serialization, reflection and annotations
    - 14. Testing and tooling
    - 15. Scenario and senior-level questions
    - 16. "What does this print?" output puzzles
    - 17. Coding tasks asked in Java rounds
    - 18. DSA problems in Java rounds
    - Sources of the question lists

Chapter 1 — Java and the JVM

This chapter explains what Java is and what actually happens between typing `javac` (the Java compiler) and seeing output. You will set up a JDK (Java Development Kit — the compiler plus the other developer tools), write, **compile** (translate human-written source code into a form a machine can execute) and run programs three different ways, look inside a `.class` file, watch the Just-In-Time compiler (the part of the JVM that turns frequently run code into fast machine code while the program is running) and the garbage collector (the automatic cleaner that frees memory your program no longer uses) at work, experiment in `jshell` (an interactive prompt for typing Java line by line), and learn how Java versions and long-term-support releases (versions that vendors keep patching for many years) work, so you can choose the right JDK for a project.

What you'll learn

#	Topic	You will be able to…
1.1	What Java is	explain Java, its history and where it is used in one minute
1.2	The Java ecosystem	tell Java SE, Jakarta EE, OpenJDK and vendor builds apart
1.3	JDK, JRE and JVM	say what each one contains and which one you install
1.4	Installing a JDK	install Java 25, set `JAVA_HOME`, verify the tools
1.5	Write → compile → run	build `HelloWorld`, read compiler errors, package a JAR
1.6	Bytecode and class files	read `javap -c` output and class-file versions
1.7	How the JVM runs code	describe class loading, the interpreter, JIT tiers and AOT caching
1.8	Memory and garbage collection	explain stack vs heap, reachability, generations and the collectors
1.9	jshell	try code instantly in the REPL
1.10	Launching source files	run `.java` files directly, including compact source files (Java 25)
1.11	IDEs and build tools	pick an editor and know where Maven/Gradle fit
1.12	Releases, LTS and JEPs	choose a Java version and read a feature's status

All code lives in `code/`. It was compiled with `javac --release 25` (the book targets **Java 25 LTS**) and run on the JDK 26 installed on the author's machine; where output depends on the machine (timings, memory sizes) that is stated. For exam-level detail and traps on this material (class structure, `main` signatures, packages, garbage-collection eligibility), see the OCP repo's Chapter 1 — Building Blocks.

1.1 What Java is

What it is

Java is a **general-purpose, statically typed, object-oriented programming language** together with a **runtime platform** (the Java Virtual Machine) that runs compiled Java programs. A **runtime** is the software that must be present while a program is running to execute it and provide services such as memory management.

- *General-purpose*: not tied to one domain — the same language builds banking back-ends, Android apps (historically), big-data engines and command-line tools.
- *Statically typed*: every variable has a type known at compile time, so the compiler rejects `int n = "five";` before the program ever runs.
- *Object-oriented*: code is organised into **classes** (blueprints) and **objects** (instances built from them). Modern Java also supports functional style (lambdas — short, unnamed functions you can pass around as values; streams — pipelines that filter and transform collections of data — Chapters 11–12).

Analogy: Java source is like a recipe written in one standard language; the JVM is a professional kitchen installed in every restaurant (Windows, macOS, Linux, containers — lightweight, isolated packages such as Docker images that bundle an app with everything it needs to run). Write the recipe once and any kitchen can cook it. That is the origin of the slogan *"write once, run anywhere"*.

A short history

Year	Event
1991	James Gosling and a small team at Sun Microsystems start the "Green Project" for consumer devices. The language is first called **Oak**.
1995	Renamed **Java** and announced publicly (May 1995); browser "applets" (small Java programs embedded in web pages) make it famous.
1996	JDK 1.0 released (January 1996).
2004	J2SE 5.0: generics, enhanced `for`, autoboxing, enums, annotations, varargs — the biggest language update until Java 8.
2006–07	Sun open-sources the JDK as **OpenJDK** (GPL v2 — a free, open-source licence — with the Classpath Exception).
2010	Oracle acquires Sun and becomes Java's steward.
2014	Java 8: lambdas, streams, `java.time`. Still widely deployed.
2017	Java 9: the module system (JPMS — groups packages into named, sealed-off modules) and `jshell`.
2018	Java 10 starts the **six-month release cadence** (a new feature release every March and September).
2021	Java 17 LTS; long-term-support releases now every **two years**.
2023	Java 21 LTS: virtual threads, pattern matching for `switch`, record patterns.
2025	**Java 25 LTS** (16 Sept 2025): compact source files and instance `main`, flexible constructor bodies, module imports, scoped values.
2026	Java 26 (March) and Java 27 (15 Sept 2026) — non-LTS feature releases.

Don't worry if feature names in this table (generics, enums, lambdas, virtual threads, records, pattern matching …) are new to you — each one is explained in its own later chapter.

Design traits

Trait	What it means in practice	Honest caveat
Platform independence	Compiled bytecode (the JVM's portable instruction format) runs on any OS/CPU that has a JVM.	You still need a JVM (or a native-image build — a program pre-compiled into a stand-alone executable) on the target.
Static typing	Type errors are caught by the compiler; IDEs (code editors with built-in tools) can refactor (restructure code without changing behaviour) safely.	More declarations than Python or JavaScript (softened by `var`, records).
Automatic memory management	A garbage collector frees unused objects; no `free()` / `delete`.	GC uses CPU and memory; you can still leak by keeping references (handles that point to objects).
Backward compatibility	Code compiled for Java 8 usually runs unchanged on Java 25.	Old APIs (the classes and methods a library exposes for you to call) linger (e.g. `Date`, `Vector`) and old habits persist.
Rich standard library (ready-made code shipped with Java)	Collections, concurrency (running several tasks at once), networking, HTTP client, `java.time`, crypto — all built in.	Large; learning which API is the modern one takes time.
Performance	The JIT compiler turns hot code (code that runs very often) into optimised machine code (instructions the CPU executes directly); long-running servers are fast.	Start-up and warm-up are slower than native languages (mitigated by AOT — ahead-of-time — caches, native images).
Safety	No pointer arithmetic (computing raw memory addresses by hand), array bounds are checked, bytecode is verified.	Checks have a (small) cost.

Where Java is used (real-world examples)

- **Enterprise back-ends** (the server-side part of an application that holds the business logic and data): banks, insurers, airlines and retailers run large Spring Boot / Jakarta EE services (popular **frameworks** — big reusable libraries that give an application its structure).
- **Big data and streaming**: Apache Kafka (moves streams of events between systems), Apache Cassandra (a distributed database), Elasticsearch (a search engine), Apache Hadoop and much of Apache Flink (large-scale data processing) run on the JVM (written in Java and Scala).
- **Developer tools**: IntelliJ IDEA, Eclipse, Maven, Gradle, Jenkins.
- **Android**: apps were historically written in Java (Android uses the Java *language* and much of its library, but its own runtime, ART, not the JVM); Google has recommended Kotlin first since 2019.
- **Games**: Minecraft: Java Edition.
- **Embedded and smart cards**: Java Card runs on SIM and bank cards.

Other languages also compile to JVM bytecode — **Kotlin, Scala, Groovy, Clojure** — and interoperate with Java libraries. Learning the JVM helps with all of them.

> **TL;DR**

Java = a statically typed, object-oriented language + a portable runtime (the JVM). It is mature (1996), backward compatible, fast after warm-up, and dominant in server-side and data infrastructure.

1.2 The Java ecosystem: SE, EE, OpenJDK and vendors

What it is

"Java" can mean several things. This table untangles them.

Name	What it is
Java SE (Standard Edition)	The *specification* of the language, JVM and core library (`java.lang`, `java.util`, …). Each version (SE 25) is defined through the Java Community Process (the formal process through which companies and individuals agree on changes to Java).
JDK (Java Development Kit)	A concrete *implementation* of Java SE plus tools (`javac`, `jshell`, `jar`, `javadoc`, `jlink`…).
OpenJDK	The open-source (source code public and free to use and modify) project where the reference JDK is developed (openjdk.org). Almost every JDK you can download is built from it.
Vendor builds	Oracle JDK, Eclipse Temurin (Adoptium), Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK, Red Hat build of OpenJDK, BellSoft Liberica, SAP Machine. Same code base, different support/licensing and extras.
Jakarta EE (formerly Java EE)	Specifications for enterprise APIs (servlets — handle web requests; JPA persistence — save objects to databases; CDI — wires objects together…) run on top of Java SE. Moved from Oracle to the Eclipse Foundation (2017) and renamed.
Java ME / Java Card	Small-device and smart-card editions.
GraalVM	A JDK distribution from Oracle with the Graal compiler and **Native Image**, which compiles Java ahead of time into a standalone native executable (a program file the OS runs directly, with no JVM needed).

Which vendor should I install?

For learning, any OpenJDK build of version 25 is fine; **Eclipse Temurin** is a common neutral choice. Companies pick a vendor by support contract, cloud (Corretto on AWS, Microsoft's build on Azure) or licensing. Oracle JDK 17 and later are released under the *Oracle No-Fee Terms and Conditions*, which allow free **production** use (running the real, live system that users depend on) for a limited period (for JDK 25, updates are free until September 2028), after which later updates need a paid licence; OpenJDK builds from other vendors are GPL-licensed.

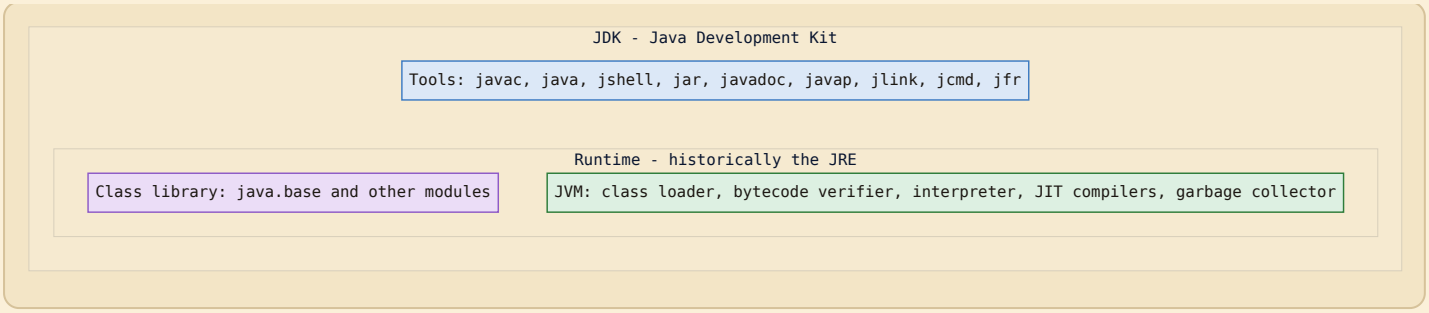
Common mistakes

- Thinking "OpenJDK" is lower quality than "Oracle JDK" — they are built from the same source.
- Confusing Java with **JavaScript**; they share only a name and C-like syntax. (See the JavaScript Handbook.)

1.3 JDK, JRE and JVM

What it is

Three nested layers:



Layer	Contains	Who needs it
JVM (Java Virtual Machine)	An abstract machine that loads bytecode, verifies it and executes it (interpreting and JIT-compiling), and manages memory. **HotSpot** is the JVM inside OpenJDK builds.	Every Java program at run time
JRE (Java Runtime Environment)	JVM + standard class library — enough to *run* programs, not compile them.	End users (historically)
JDK	JRE + development tools (`javac` compiler, `jshell`, `jar`, `javadoc`, debuggers, profilers).	Developers — **install this**

Virtual machine here means a program that pretends to be a CPU with its own instruction set (bytecode). It is not a VM like VirtualBox that emulates a whole computer.

Why the JRE "disappeared"

Since JDK 11, Oracle no longer ships a separate JRE download. Java 9's module system introduced `jlink`, which builds a **custom runtime** containing only the modules (named groups of packages, such as `java.base`) your application needs. Some vendors (for example Temurin) still publish JRE-only builds for convenience, mainly for containers.

A real example from this machine — a runtime containing only `java.base` is enough to run `HelloWorld`:

```
jlink --add-modules java.base --strip-debug --no-header-files --no-man-pages --output rt
du -sh rt                       # 44M   (the full JDK folder here is 377M)
rt/bin/java -cp out ch01.hello.HelloWorld
```

Output:

```
Hello, World!
Arguments received: 0
```

In the interview

Q: Difference between JDK, JRE and JVM? The JVM executes bytecode and manages memory; the JRE is the JVM plus the standard library (enough to run programs); the JDK is the JRE plus development tools like `javac`. Developers install a JDK; production images often use a `jlink`-trimmed runtime.

- *Is the JVM platform-independent?* No — the **bytecode** is portable; each OS/CPU needs its own JVM build.
- *What is HotSpot?* The JVM implementation in OpenJDK, named for its strategy of finding "hot" code and compiling it.

Self-check

1. You want to compile code. Is a JRE enough? → No, you need the JDK (`javac`).
2. What makes a Java program run on both Linux and Windows? → The same bytecode runs on each platform's own JVM.
3. What does `jlink` produce? → A trimmed runtime image containing only the modules you list.

1.4 Installing a JDK and checking it works

How it works

1. **Download** an OpenJDK 25 build (e.g. Temurin from adoptium.net) or use a version manager such as **SDKMAN!** (`sdk list java`, then `sdk install java <identifier>`) on macOS/Linux, or your OS package manager (a tool such as `apt` or Homebrew that installs software for you).
2. **Set** `JAVA_HOME` to the JDK folder and put `$JAVA_HOME/bin` on your `PATH`. Build tools (Maven, Gradle — programs that automate compiling, downloading libraries, testing and packaging) and IDEs read `JAVA_HOME`.
   - An **environment variable** is a named setting (like `JAVA_HOME=/path/to/jdk`) that the operating system passes to every program it starts. `PATH` is the environment variable listing the folders the shell searches when you type a command such as `java`.
   - macOS: `export JAVA_HOME=$(/usr/libexec/java_home -v 25)`
   - Linux: `export JAVA_HOME=/usr/lib/jvm/<jdk-folder>`
   - Windows: *System Properties → Environment Variables*, then add `%JAVA_HOME%\bin` to `Path`.
3. **Verify** in a new terminal (the command-line window where you type commands):

```
java -version
javac -version
```

Output (on the machine used for this book):

```
openjdk version "26" 2026-03-17
OpenJDK Runtime Environment (build 26+35-2893)
OpenJDK 64-Bit Server VM (build 26+35-2893, mixed mode, sharing)
javac 26
```

"mixed mode" means the JVM both interprets and JIT-compiles; *"sharing"* means it uses Class Data Sharing (a pre-parsed archive of core classes that speeds start-up).

Compiling for an older version: `--release`

A newer JDK can produce class files for an older Java version: `javac --release 25 …` checks your code against the Java 25 language **and** the Java 25 API, so you cannot accidentally use a Java 26 method. That is how this book's code targets Java 25 while running on JDK 26. (The older `-source`/`-target` pair does not check the API — prefer `--release`.)

Common mistakes

- Having several JDKs installed and `java` on `PATH` pointing at a different one than `javac` — check both versions.
- Setting `JAVA_HOME` to the `bin` folder instead of the JDK root.
- Installing only a JRE and then wondering why `javac` is "command not found".

1.5 Write → compile → run

What it is

Java is **compiled in two stages**: `javac` turns source code (`.java`) into platform-neutral **bytecode** (`.class` files); then the `java` launcher starts a JVM that loads and runs that bytecode.

The program

`code/HelloWorld.java`:

```java
package ch01.hello;

/** The classic first program: one public class, one entry point. */
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        System.out.println("Arguments received: " + args.length);
    }
}
```

Part	Meaning
`package ch01.hello;`	The **package** (namespace) this class belongs to. It must be the first statement. The fully qualified class name becomes `ch01.hello.HelloWorld`. Packages are covered in Chapter 5.
`/** … */`	A **Javadoc comment**; the `javadoc` tool turns these into HTML API docs. `//` is a line comment, `/* … */` a block comment.
`public class HelloWorld`	Declares a class. A file may contain at most one **public** top-level class, and the file must be named after it (`HelloWorld.java`).
`public static void main(String[] args)`	The **entry point** the launcher calls. `public` = callable from anywhere; `static` = no object needed; `void` = returns nothing; `args` = command-line arguments (Chapter 4).
`System.out.println(...)`	Prints a line to **standard output** (the program's normal text output — the terminal, by default). `System` is a class in `java.lang` (imported automatically — an *import* lets you use a class by its short name), `out` is its static `PrintStream` field (a variable belonging to the class itself, holding an object that writes text).
`;` and `{ }`	Every statement ends with a semicolon; braces group statements into blocks. Java ignores indentation, but you should not.

Since Java 25 a class may also declare a simpler `void main()` (no `static`, no parameters) — see 1.10. Traditional `public static void main(String[] args)` remains the standard for applications and works in every version.

How to compile and run

```
javac -d out HelloWorld.java            # compile; -d puts classes in out/ch01/hello/
java -cp out ch01.hello.HelloWorld       # run: class path + fully qualified class name
java -cp out ch01.hello.HelloWorld a b c
```

Output:

```
Hello, World!
Arguments received: 0
Hello, World!
Arguments received: 3
```

1. `javac` parses the file (reads the text and works out its grammatical structure), checks types and writes `out/ch01/hello/HelloWorld.class` (the folder structure mirrors the package).
2. `java` starts a JVM; `-cp` (class path) tells it where to look for classes.
3. The JVM loads `ch01.hello.HelloWorld`, finds `main`, and calls it with the arguments.

Reading errors: compile-time vs run-time

Compile-time errors are reported by `javac` with file, line and a caret. Real messages:

```
Hello.java:3: error: ';' expected
        System.out.println("hi")
                                ^
```

```
Hello.java:1: error: class Greeting is public, should be declared in a file named Greeting.java
```

```
Hello.java:3: error: incompatible types: String cannot be converted to int
        int n = "five";
                ^
```

Launcher errors happen when `java` cannot find the class:

Command	Error	Fix
`java hello`	`NoClassDefFoundError: hello (wrong name: Hello)` on macOS/Windows (case-insensitive file systems); `ClassNotFoundException: hello` on Linux	Class names are case-sensitive.
`java Hello.class`	`ClassNotFoundException: Hello.class`	Pass the class **name**, not the file name.
`java HelloWorld` (class is in a package)	`Could not find or load main class HelloWorld`	Use the fully qualified name and the right `-cp`.

Packaging a JAR

A **JAR** (Java ARchive) is a ZIP file of classes plus a manifest (a small text file of metadata, i.e. data describing the archive). With a `Main-Class` entry it becomes runnable:

```
jar --create --file hello.jar --main-class ch01.hello.HelloWorld -C out .
java -jar hello.jar one two
```

Output:

```
Hello, World!
Arguments received: 2
```

Real projects let Maven or Gradle build JARs (Chapter 18).

Common mistakes

- File name ≠ public class name (case matters: `helloworld.java` fails for `HelloWorld`).
- Forgetting `-cp`/wrong directory, or running from inside the package folder.
- Misspelling `main`'s signature (e.g. `String args` without `[]`) in a traditional class — it compiles, but the launcher reports no suitable main method.

Self-check

1. What does `javac` produce? → `.class` files containing bytecode.
2. Why does `java Hello.class` fail? → The launcher wants a class name, not a file path.
3. Where does the compiled `ch01.hello.HelloWorld` go with `-d out`? → `out/ch01/hello/HelloWorld.class`.

TL;DR

`javac` → bytecode; `java -cp <dir> <package.Class>` → JVM runs it. One public top-level class per file, named like the file.

1.6 Bytecode and class files

What it is

Bytecode is the instruction set of the JVM: compact, one-byte opcodes (operation codes — numbered instructions such as `iload`, `iadd`, `invokevirtual` …) for a **stack machine** — instructions push values onto an operand stack and pop them to compute. A `.class` file holds one class's bytecode plus a **constant pool** (names, strings, method references) and metadata.

Why it matters

- It is the unit of portability: the same `.class` runs on every JVM.
- The JVM verifies it before running (no forged pointers, no operand-stack overflow or underflow, no jumps into the middle of an instruction from malformed code).
- Tools (profilers — measure where time and memory go; mocking libraries — create fake objects for tests; frameworks like Spring and Hibernate) read or generate bytecode at run time.

Looking inside with `javap`

`code/Calculator.java`:

```
package ch01.bytecode;

public class Calculator {
    static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int total = add(2, 3);
        System.out.println("2 + 3 = " + total);
    }
}
```

`javap -c -cp out ch01.bytecode.Calculator` (a disassembler — turns bytecode back into readable instructions — shipped with the JDK):

```
public class ch01.bytecode.Calculator {
  public ch01.bytecode.Calculator();
    Code:
         0: aload_0
         1: invokespecial #1                  // Method java/lang/Object."<init>":()V
         4: return

  static int add(int, int);
    Code:
         0: iload_0
         1: iload_1
         2: iadd
         3: ireturn

  public static void main(java.lang.String[]);
    Code:
         0: iconst_2
         1: iconst_3
         2: invokestatic  #7                  // Method add:(II)I
         5: istore_1
         6: getstatic     #13                 // Field java/lang/System.out:Ljava/io/PrintStream;
         9: iload_1
        10: invokedynamic #19,  0             // InvokeDynamic #0:makeConcatWithConstants:(I)Ljava/lang/String;
        15: invokevirtual #23                 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
        18: return
}
```

How to read it:

1. `javac` added a **default constructor** (a constructor is the special method that sets up a new object; this one is `Calculator()`) that calls `Object`'s constructor — you did not write it.
2. `add`: `iload_0`/`iload_1` push the two `int` parameters, `iadd` pops both and pushes the sum, `ireturn` returns it. (`i` = int.)
3. `main`: pushes constants 2 and 3, calls `add` (`(II)I` = takes two ints, returns int), stores the result in local slot 1.
4. String concatenation compiles to `invokedynamic … makeConcatWithConstants` (since Java 9), which lets the JVM choose the fastest concatenation strategy at run time.

Class-file version

Every class file starts with the magic number `CAFEBABE` (fixed bytes that identify the file type), then a **major version** that says which Java produced it: major = 44 + Java version.

Java	8	11	17	21	25	26
Major version	52	55	61	65	**69**	70

```
javap -v -cp out ch01.bytecode.Calculator | grep major   # major version: 69
xxd out/ch01/bytecode/Calculator.class | head -1          # cafe babe 0000 0045 ...  (0x45 = 69)
```

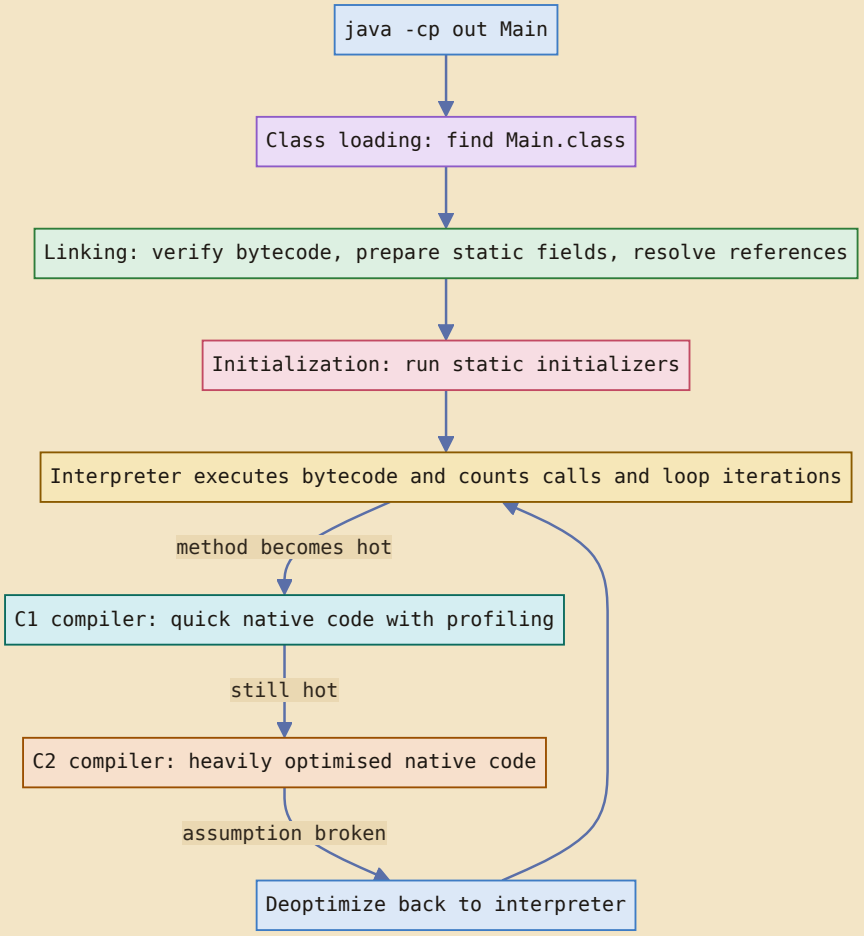
An older JVM refuses newer class files with `UnsupportedClassVersionError` ("compiled by a more recent version of the Java Runtime (class file version 69.0), this version … only recognizes class file versions up to 65.0"). Fix: run on a newer JVM or compile with `--release <older>`.

Self-check

1. What kind of machine does bytecode target? → A stack-based virtual machine.
2. You get `UnsupportedClassVersionError … 69.0`. Which Java compiled it? → Java 25.
3. Who wrote the `Calculator()` constructor? → The compiler (default constructor).

1.7 How the JVM runs your code: class loading, interpreter, JIT

How it works



1. **Class loading.** *Class loaders* find `.class` bytes and turn them into classes, lazily (the first time a class is used). There are three built-in loaders: **bootstrap** (core `java.base` classes), **platform** (other JDK modules) and **application/system** (your class path). Each asks its parent first (*parent delegation*), so you cannot replace `java.lang.String` with your own.
2. **Linking.** *Verification* checks the bytecode is type-safe and well-formed; *preparation* allocates static fields with default values; *resolution* turns symbolic names (e.g. `java/io/PrintStream.println`) into direct references (done lazily).
3. **Initialization.** Static field initializers and `static { }` blocks run, once per class.
4. **Execution.** The **interpreter** runs bytecode instruction by instruction — starts instantly but is slow. It keeps counters of how often each method and loop runs.
5. **JIT compilation.** *Just-In-Time* compilation converts **hot** methods into native machine code while the program runs. HotSpot uses **tiered compilation** (default since Java 8):

Tier	Who	What
0	Interpreter	Runs bytecode, gathers counts
1–3	**C1** ("client") compiler	Compiles fast with light optimisation; tier 3 adds profiling (which types and branches actually occur)
4	**C2** ("server") compiler	Uses the profile for aggressive optimisation: inlining, loop unrolling, escape analysis, dead-code removal

Those optimisations in plain words: **inlining** copies a small method's body into its caller to skip the call; **loop unrolling** repeats a loop body several times per iteration to cut loop overhead; **escape**

analysis spots objects that never leave a method so they need not live on the heap; **dead-code removal** deletes work whose result is never used.

6. **Speculation and deoptimization.** C2 optimises for what it has *seen* (e.g. "this call always hits `ArrayList`"). If that assumption later breaks, the JVM throws away the compiled code ("uncommon trap") and falls back to the interpreter, then may recompile.

Watching the JIT

`code/HotLoop.java` calls a tiny method five million times:

```
static long square(long x) { return x * x; }

public static void main(String[] args) {
    long sum = 0;
    for (long i = 0; i < 5_000_000; i++) {
        sum += square(i % 1000);
    }
    System.out.println("sum = " + sum);
}
```

`java -XX:+PrintCompilation -cp out ch01.jit.HotLoop | grep -E "HotLoop|sum ="` (real output; timestamps vary by machine):

```
15    9       3       ch01.jit.HotLoop::square (4 bytes)
15   10       4       ch01.jit.HotLoop::square (4 bytes)
15    9       3       ch01.jit.HotLoop::square (4 bytes)   made not entrant: not used
16   11 %     3       ch01.jit.HotLoop::main @ 4 (43 bytes)
16   12       3       ch01.jit.HotLoop::main (43 bytes)
16   13 %     4       ch01.jit.HotLoop::main @ 4 (43 bytes)
16   11 %     3       ch01.jit.HotLoop::main @ 4 (43 bytes)   made not entrant: OSR invalidation of lower level
19   13 %     4       ch01.jit.HotLoop::main @ 4 (43 bytes)   made not entrant: uncommon trap
sum = 1664167500000
```

Columns: milliseconds since start, compile ID, flags, **tier**, method. `square` was compiled by C1 (tier 3) and within the same millisecond by C2 (tier 4), and the C1 version was discarded ("made not entrant"). `%` marks **on-stack replacement (OSR)**: `main` runs only once, but its loop is hot, so the JVM compiles the loop and switches to it *while the method is still running*. The final line is a deoptimization.

Start-up and warm-up: AOT caching

Because the JVM starts in the interpreter, a Java service is slower for its first seconds (**warm-up**). Recent JDKs attack this with an **ahead-of-time (AOT) cache** recorded from a training run:

- JDK 24 (JEP 483 — a *JEP* is a numbered JDK Enhancement Proposal, see 1.12): classes can be loaded and linked ahead of time into a cache.
- JDK 25 (JEP 514): one-step cache creation, `java -XX:AOTCacheOutput=app.aot …` on a training run, then `java -XX:AOTCache=app.aot …` in production.
- JDK 25 (JEP 515): the cache also stores method profiles, so the JIT can start optimising sooner.

Separately, **GraalVM Native Image** compiles the whole application to a native executable before it runs: very fast start-up and small memory, at the cost of long builds and restrictions on reflection (code inspecting classes and calling methods by name at run time) and dynamic class loading.

Trade-offs: interpreted vs JIT vs AOT

Approach	Start-up	Peak speed	Portability	Example
Pure interpretation	Instant	Slow	High	Early JVMs, CPython (the standard Python interpreter)
JIT (HotSpot)	Slower (warm-up)	Very high — uses live profiles	High (bytecode)	Normal `java`
AOT cache (JDK 24+)	Faster	Same as JIT	High	`-XX:AOTCache`
Native image	Very fast	Good, often a bit lower without profile data	One binary per OS/CPU	GraalVM `native-image`

In the interview

Q: Is Java compiled or interpreted? Both. `javac` compiles source to bytecode ahead of time; the JVM then interprets that bytecode and JIT-compiles hot methods to native code at run time, using profiling data. That is why long-running Java services get faster after warm-up.

- *What is tiered compilation?* Interpreter → C1 (fast compile, profiling) → C2 (optimising compile using the profile).
- *What is deoptimization?* Discarding compiled code whose speculative assumption became false and returning to the interpreter.
- *Why is a microbenchmark with a simple timing loop misleading?* It mixes interpreted, C1 and C2 phases and the JIT may remove "useless" work; use JMH (Java Microbenchmark Harness, a tool for accurate small-scale timing; Chapter 16).

Self-check

1. When is a class loaded? → Lazily, the first time it is actively used.
2. What does `%` mean in `PrintCompilation` output? → On-stack replacement of a hot loop.
3. Why can a JIT beat a static compiler sometimes? → It optimises for the types and branches actually seen at run time.

1.8 Memory and garbage collection — an overview

This section gives the picture you need now; algorithms, tuning and profiling are in Chapter 16.

What it is

The JVM divides memory into areas:

> **Key term — thread:** a thread is one independent sequence of instructions running inside a program. A program can have many threads running at the same time (for example, one per incoming web request), and each has its own call stack but shares the heap with the others. Threads are covered properly in the concurrency chapters.

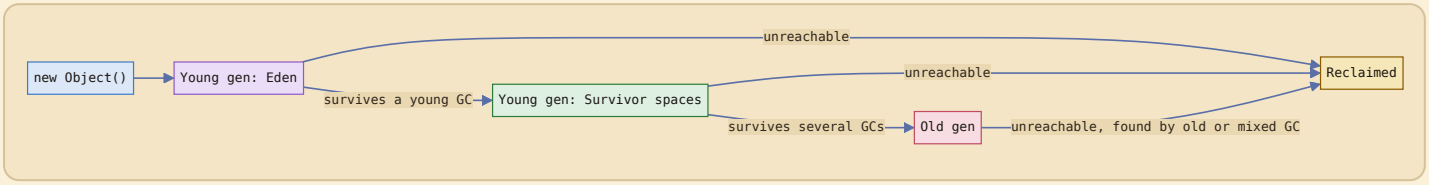
Area	Holds	Lifetime
Stack (one per thread)	*Frames* for each method call: local variables, parameters, partial results. Primitive locals live here; for objects, only the **reference** (a pointer-like handle) lives here.	Frame disappears when the method returns
Heap (shared)	All objects and arrays (`new …`).	Until the garbage collector reclaims them
Metaspace	Class metadata (methods, field layouts), in native memory (memory the JVM takes from the OS outside the Java heap).	While the class loader is alive
Code cache	JIT-compiled machine code.	Until evicted/deoptimized

Garbage collection (GC) is automatic reclamation of heap objects that are no longer **reachable**. An object is reachable if a chain of references leads to it from a **GC root** — a local variable in an active frame, a static field, an active thread and a few JVM-internal references. Anything not reachable is garbage, even if it still points to other objects (so reference cycles are collected too).

Analogy: a stadium cleaner removes any seat cushion no ticket-holder is still attached to — they do not ask the cushion whether it is needed.

How it works (the common core)

1. **Mark**: starting from the roots, follow references and mark every reachable object.
2. **Sweep / compact / copy**: free the unmarked memory; many collectors also move live objects together to avoid fragmentation (free memory scattered in many small gaps that are too small to use).
3. **Generations**: most objects die young (the *weak generational hypothesis* — think temporary strings in a request). Generational collectors therefore split the heap into a **young generation**, collected often and cheaply, and an **old generation** for survivors, collected rarely.
4. **Pauses**: some work needs application threads stopped (**stop-the-world** pause); modern collectors do most work **concurrently** (at the same time as) with your threads.



The collectors in JDK 25

Collector	Flag	Design	Good for
G1 (Garbage-First)	`-XX:+UseG1GC`	Generational, region-based (heap split into many equal-sized chunks), mostly concurrent marking; aims for a pause-time goal (default 200 ms)	**The default** — general-purpose servers and desktops
ZGC	`-XX:+UseZGC`	Concurrent, generational (the only mode since JDK 24); pauses typically under a millisecond regardless of heap size	Large heaps, latency-sensitive services (where the time each request takes matters)
Shenandoah	`-XX:+UseShenandoahGC`	Concurrent compacting; generational mode became a product feature in JDK 25	Low latency; included in many OpenJDK builds (not Oracle JDK)
Parallel	`-XX:+UseParallelGC`	Stop-the-world, many threads; maximises throughput (total work done per second)	Batch jobs where pauses do not matter
Serial	`-XX:+UseSerialGC`	Single-threaded, stop-the-world	Tiny heaps, small containers
Epsilon	experimental	Allocates, never collects	Testing and very short-lived jobs

The JVM picks G1 unless the machine looks "small" (fewer than 2 CPUs or under 1792 MB of memory), where it chooses Serial; JDK 27 (JEP 523) makes G1 the default everywhere. By default the maximum heap is ¼ of physical memory — `WhereAmI` below reports 12 288 MiB on a 48 GiB machine (MiB/GiB are binary units: 1 MiB = 1024 × 1024 bytes, 1 GiB = 1024 MiB). **Latency** and **throughput** are system-design terms you will meet again; see System Design Fundamentals.

Watching the GC

`code/GarbageMaker.java` creates 200 000 short-lived 1 KiB arrays and keeps ten 1 MiB arrays:

```
List<byte[]> keep = new ArrayList<>();          // long-lived: stays reachable
for (int i = 0; i < 200_000; i++) {
    byte[] temp = new byte[1024];                // short-lived: garbage after this iteration
    temp[0] = (byte) i;
    created++;
    if (i % 20_000 == 0) keep.add(new byte[1024 * 1024]);
}
```

`java -Xlog:gc -Xmx256m -cp out ch01.gc.GarbageMaker` (selected lines from one real run; timings and the exact sequence of collections vary from run to run):

```
[0.004s][info][gc] Using G1
[0.017s][info][gc] GC(0) Pause Young (Concurrent Start) (G1 Humongous Allocation) 1M->1M(10M) 0.407ms
[0.017s][info][gc] GC(1) Concurrent Mark Cycle
[0.018s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 6M->3M(10M) 0.195ms
[0.022s][info][gc] GC(4) Pause Young (Prepare Mixed) (G1 Evacuation Pause) 22M->5M(30M) 0.580ms
[0.024s][info][gc] GC(5) Pause Young (Mixed) (G1 Evacuation Pause) 22M->5M(30M) 0.305ms
...
temporary arrays created: 200000
arrays still reachable  : 10
```

Reading a line: `22M->5M(30M) 0.580ms` = heap in use before → after the collection (current heap size), and the pause length. Young collections free most memory because the temporary arrays are already dead. "Humongous" means an object at least half a G1 region in size (the 1 MiB arrays) that G1 allocates specially. With `-XX:+UseZGC` the first line reads `Using The Z Garbage Collector`.

Ask the JVM about itself with `code/WhereAmI.java`:

```
Runtime.Version v = Runtime.version();
System.out.println("Feature release : " + v.feature());
System.out.println("class file ver. : " + System.getProperty("java.class.version"));
System.out.println("Max heap (MiB)  : " + Runtime.getRuntime().maxMemory() / (1024 * 1024));
```

Output (this machine; yours will differ):

```
Feature release : 26
java.vendor     : Oracle Corporation
java.vm.name    : OpenJDK 64-Bit Server VM
class file ver. : 70.0
os.name         : Mac OS X
CPU cores seen  : 14
Max heap (MiB)  : 12288
```

Note the program was *compiled* for Java 25 but *runs* on JDK 26 — newer JVMs run older class files.

Common mistakes

- Believing Java cannot leak memory. A static `Map` cache that only grows keeps every entry reachable forever — a **memory leak**.
- Calling `System.gc()` to "free memory": it is only a request, and usually hurts.
- Relying on `finalize()` for cleanup: deprecated for removal (JEP 421, Java 18). Use try-with-resources (Chapter 8).
- Setting `-Xmx` (the flag for the maximum heap size) larger than a container's memory limit (the JVM reads container limits automatically; let it, or size explicitly).

In the interview

Q: How does Java garbage collection decide what to free? By reachability, not reference counting (keeping a count of how many references point at each object): anything not reachable from GC roots (thread stacks, static fields, JNI handles — references held by native C code via the Java Native Interface) is garbage, so cycles are collected. Most collectors are generational because most objects die young.

- *Default GC?* G1 since Java 9 on server-class machines.
- *When would you use ZGC?* Large heaps or strict latency targets, where sub-millisecond pauses matter more than a little throughput.
- *Stack vs heap?* Stack: per-thread frames with locals, freed on return. Heap: shared objects, freed by GC.

Self-check

1. Two objects reference each other but nothing else references them. Are they collected? → Yes; they are unreachable.
2. Where does `int x = 5;` inside a method live? Where does `new int[5]` live? → `x` on the stack; the array on the heap (the reference to it on the stack).
3. Why are young collections cheap? → Most young objects are dead, and the collector only copies the few live ones.

TL;DR

Stack = method frames; heap = objects. The GC frees unreachable objects; generational collectors exploit "most objects die young". Default is G1; ZGC/Shenandoah for low latency; Parallel for throughput.

1.9 jshell — the Java REPL

What it is

`jshell` (Java 9, JEP 222) is a **REPL** — *Read-Evaluate-Print Loop*: type a snippet, see the result immediately, no class or `main` needed. Ideal for trying an API or checking an operator's behaviour.

A session

Real session (lines after `jshell>` are what was typed):

```
|  Welcome to JShell -- Version 26
|  For an introduction type: /help intro

jshell> int x = 21
x ==> 21

jshell> x * 2
$2 ==> 42

jshell> "Java".length()
$3 ==> 4

jshell> String shout(String s) { return s.toUpperCase() + "!"; }
|  created method shout(String)

jshell> shout("hi")
$5 ==> "HI!"

jshell> /vars
|    int x = 21
|    int $2 = 42
|    int $3 = 4
|    String $5 = "HI!"
```

- Expressions not assigned to a variable get scratch names (`$2`, `$3`…) you can reuse.
- Semicolons are optional for single statements.
- Methods can be declared at the top level and redefined.

Command	Does
`/help`, `/help intro`	Help
`/vars`, `/methods`, `/types`, `/imports`	List what you have defined/imported
`/list`	Show your snippets
`/edit <name>`	Open a snippet in an editor
`/save file.jsh`, `/open file.jsh`	Save/load a session
`/reset`, `/exit`	Clear state, quit
Tab	Completion and method signatures

On the JDK 26 used here, `/imports` shows `import java.base`, so every public class in the `java.base` module is usable without imports.

When not to use it

For anything you want to keep, test or share, write a source file. jshell is a scratchpad.

1.10 Launching source files directly

Single-file and multi-file source launch

Since **Java 11** (JEP 330) the launcher can compile a single `.java` file in memory and run it: `java Hello.java`. No `.class` files are written. Since **Java 22** (JEP 458) the launched program may use other source files in the same directory tree; they are compiled on demand.

`code/launch/Main.java` uses `code/launch/Greeter.java`:

```
// Main.java
public class Main {
    public static void main(String[] args) {
        System.out.println(Greeter.greet("launcher"));
    }
}
// Greeter.java
public class Greeter {
    static String greet(String who) {
        return "Hello from " + who + ", compiled in memory";
    }
}
```

```
cd code/launch && java Main.java
```

Output:

```
Hello from launcher, compiled in memory
```

On Linux/macOS a single-file program can even start with a **shebang** line (a first line starting with `#!` that tells Unix which program should run the file) (`#!/usr/bin/env -S java --source 25`) in a file *without* the `.java` extension and be run as a script.

Compact source files and instance `main` — Java 25

JEP 512 (final in **Java 25**, after previews — opt-in trial versions, see 1.12 — in 21–24) removes the ceremony for small programs:

- A **compact source file** may declare methods and fields with no enclosing class; the compiler wraps them in an implicit, unnamed-to-you `final` class (a class that cannot be extended).
- `main` may be an **instance method** (a method that runs on an object rather than on the class itself) with no parameters: `void main()`.
- A compact source file automatically imports every public class of the `java.base` module (as if it began with `import module java.base;`), so `List`, `Map`, `Path` … need no imports.
- The new class `java.lang.IO` offers `IO.println(…)`, `IO.print(…)`, `IO.println()` and `IO.readln(prompt)`/`IO.readln()` for simple console I/O (input/output — reading what the user types and printing text; built on `System.out` and `System.in`).

`code/compact/HelloCompact.java` — the whole file:

```
void main() {
    List<String> langs = List.of("Java", "Kotlin", "Scala");   // java.util.List without an import
    IO.println("Hello from a compact source file!");
    IO.println("JVM languages: " + langs);
}
```

```
cd code/compact && java HelloCompact.java
```

Output:

```
Hello from a compact source file!
JVM languages: [Java, Kotlin, Scala]
```

How the launcher chooses `main`: if the class has a `main(String[])`, that is used; otherwise a no-argument `main()`. If the chosen method is not `static`, the launcher first creates an instance with the no-argument constructor.

Trade-offs

Style	Best for	Limits
`javac` + `java`	Real applications, anything with packages or dependencies	Two steps
`java File.java`	Scripts, examples, teaching, small tools	Compiles every run; no build-tool dependency management
Compact source file	First programs, quick utilities	No `package`; cannot be referenced by name from other code; grows into a normal class as the program grows

Common mistakes

- Adding a `package` declaration to a compact source file (not allowed).
- Assuming instance `main` works on Java 21 without `--enable-preview` — it was preview until Java 25.

1.11 IDEs, editors and build tools

What they are

An **IDE** (Integrated Development Environment) combines an editor, compiler integration, debugger (a tool that pauses a running program so you can step through it and inspect variables), refactoring and test runner.

Tool	Notes
IntelliJ IDEA (JetBrains)	The most popular Java IDE; strong refactoring and inspections. Free and paid tiers.
Eclipse IDE	Free, open source, long-standing; its own incremental compiler.
VS Code + Microsoft's *Extension Pack for Java*	Lightweight; good for mixed-language work.
Apache NetBeans	Free, open source.

Build tools automate compiling, dependency download, testing and packaging: **Maven** (XML `pom.xml`, fixed lifecycle) and **Gradle** (Groovy/Kotlin DSL — a small domain-specific language for writing build scripts — flexible and incremental, i.e. rebuilds only what changed). A **dependency** is an external library your project uses; build tools download the right versions for you. Both read `JAVA_HOME` and support *toolchains* to pin a JDK version. Details: Chapter 18.

Recommended starting workflow

1. Chapters 1–3: plain `javac`/`java`, `java File.java` and `jshell` — you learn what the tools really do.
2. From Chapter 5 on (multiple classes, packages): an IDE.
3. Anything with dependencies: Maven or Gradle.

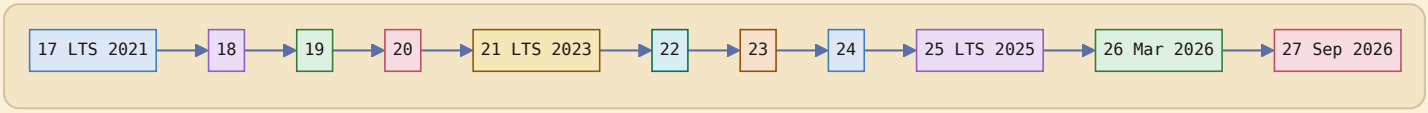
Coding conventions (used throughout this book)

Kind	Convention	Example
Class, interface, record, enum	`UpperCamelCase` noun	`OrderService`, `Shape`
Method, variable	`lowerCamelCase`; methods are verbs	`computeTotal()`, `itemCount`
Constant (`static final`)	`UPPER_SNAKE_CASE`	`MAX_RETRIES`
Package	all lowercase, reversed domain	`com.example.billing`
Indentation	4 spaces; opening brace on the same line	`if (x) {`

1.12 Java releases, LTS and JEPs

How the release model works

- A **feature release** ships every six months: **March** and **September** (since Java 10, 2018). Version numbers are just counters: 25, 26, 27…
- Each feature release gets a couple of quarterly **update releases** (e.g. 25.0.1, 25.0.2) from Oracle's OpenJDK builds; vendors then continue patching the **LTS** ones.
- **Long-Term Support (LTS)** releases are the ones vendors support for years: **8, 11, 17, 21, 25** — every two years since 17. The next is expected in September 2027 (Java 29). Oracle offers at least eight years of support for an LTS (five years Premier + three Extended).
- Non-LTS releases (22, 23, 24, 26, 27…) are production-quality but supported only until the next release. Use them to try features early.



JEPs and feature maturity

New features are proposed in **JEPs** (JDK Enhancement Proposals, openjdk.org/jeps) — numbered documents such as *JEP 512: Compact Source Files and Instance Main Methods*. A feature often goes through stages:

Stage	Meaning	How to use it
Incubator	An experimental API in a `jdk.incubator.*` module (e.g. the Vector API, still incubating in 27)	`--add-modules jdk.incubator.vector`
Preview	A complete language/VM/API feature that may still change	`javac --release 25 --enable-preview …` and `java --enable-preview …`
Final / permanent	Standard, stable part of Java	Just use it
Withdrawn	Removed before becoming final (e.g. *string templates*, previewed in 21–22, dropped in 23)	—

This book marks every modern feature with the version that **finalised** it (e.g. *records — Java 16*, *pattern matching for `switch` — Java 21*). A full tour is in Chapter 17.

Which version should I use?

Situation	Pick
New project, or learning	**Java 25 LTS**
Existing Java 17/21 service	Stay on it while supported; plan an upgrade to 25 (mostly smooth — run tests with the new JDK first)
Legacy Java 8	Upgrade path 8 → 11 → 17 → 21/25 needs attention to removed Java EE modules and strong encapsulation of JDK internals (your code may no longer reach the JDK's private implementation classes)
Library you publish	Compile with `--release` of the *oldest* Java you support

In the interview

Q: What is an LTS release and why does it matter? A feature release that vendors support with updates for many years (8, 11, 17, 21, 25); companies standardise on them so they are not forced to upgrade every six months.

- *How often does Java release?* Every six months (March/September); LTS every two years.
- *What's a preview feature?* A finished-but-not-final feature you must opt into with `--enable-preview`; it may change or be withdrawn.
- *Name three Java 21 features.* Virtual threads, pattern matching for `switch`, record patterns, sequenced collections.

Self-check

1. Is Java 26 an LTS? → No. 25 is the current LTS; 29 is the next expected.
2. What do you need to compile a preview feature on JDK 25? → `--release 25 --enable-preview` on both `javac` and `java`.
3. Where are new Java features proposed? → In JEPs on openjdk.org.

Chapter TL;DR

- **Java** = statically typed, object-oriented language + portable runtime. Current LTS: **Java 25** (Sept 2025); releases every March/September, LTS every two years.
- **JDK ⊃ runtime (JRE) ⊃ JVM.** Install a JDK; ship a `jlink` runtime or container image.
- `javac` compiles `.java` → `.class` bytecode (major version 69 = Java 25); `java` runs it on the JVM. `--release N` targets an older Java safely.

- The JVM **loads lazily, verifies, interprets, then JIT-compiles** hot code (C1 → C2) using profiles; AOT caches (JDK 24–25) and native images cut warm-up.
- **Stack** holds frames and locals; **heap** holds objects; the **GC** frees unreachable objects. Default G1; ZGC/Shenandoah for low latency.
- **jshell** for experiments; `java File.java` to run sources directly (multi-file since 22); **compact source files +** `void main()` **+** `IO` are final in **Java 25**.
- Exam-trap depth on this chapter's basics: OCP Chapter 1. Next: Chapter 2 — Basics: types, variables and operators.

Chapter 2 — Basics: Types, Variables and Operators

Every Java program is built from the same small parts: **variables** that hold **values** of some **type** (the kind of data — whole number, decimal, text, true/false — which decides what you can do with it), **operators** that combine them, and **statements** that tell the computer what to do in order. This chapter covers those parts precisely — including the places where Java silently does something you might not expect (integer overflow — a number too big for its type silently wrapping round; integer division — whole-number division that drops the remainder; floating-point rounding — decimals stored only approximately; `==` on strings). It ends with text blocks (multi-line strings), `String` basics and reading/writing the console (the text terminal the program runs in).

What you'll learn

#	Topic	You will be able to…
2.1	Statements, identifiers, keywords, comments	read and name things legally
2.2	Variables	declare, initialise, scope and `final` variables
2.3	Primitive types	pick the right primitive and know its range
2.4	Literals	write numbers, characters and booleans in every legal form
2.5	Reference types and `null`	tell primitives from references
2.6	`var`	use local type inference well (and know its limits)
2.7	Conversions and casting	predict widening, narrowing, promotion and compound-assignment casts
2.8	Operators	use arithmetic, relational, logical, bitwise, ternary operators
2.9	Precedence and associativity	evaluate any expression by hand
2.10	Integer overflow	detect and prevent silent wrap-around
2.11	Floating point	explain `0.1 + 0.2`, NaN, infinity, and when to use `BigDecimal`
2.12	Strings and text blocks	use the core `String` operations and multi-line literals
2.13	Console output and input	`println`/`printf`, `Scanner`, `BufferedReader`, `Console`, `IO`

Code: `code/`, compiled with `javac --release 25`, output pasted from real runs. Exam-level traps on this material live in the OCP repo: Chapter 1 — Building Blocks (types, literals, `var`, text blocks, scope) and Chapter 2 — Operators (promotion, casting, precedence).

2.1 Statements, identifiers, keywords and comments

What it is

- A **statement** is one complete instruction, ending in `;` (e.g. `int x = 5;`). A **block** `{ … }` groups statements and can appear wherever a statement can.
- An **expression** is code that produces a value (`x + 1`, `name.length()`, `a > b`). An *expression statement* is an expression used as a statement (`x++;`, `print(...)`).
- An **identifier** is a name you choose for a variable, method (a named block of code you can call), class (a blueprint describing a kind of object) or package (a named group of related classes, like a folder).

Identifier rules

Rule	Legal	Illegal
Starts with a letter, `$` or `_`	`total`, `$price`, `_temp`	`2fast`
Then letters, digits, `$`, `_`	`item2`, `max_value`	`my-name`, `a b`
Not a reserved keyword (a word the language uses itself) or the literals `true`, `false`, `null`	`klass`, `clazz`	`class`, `null`
A single `_` is not an identifier (it is a keyword; see *unnamed variables*, Chapter 3)	`__`	`_`
Case-sensitive	`count` ≠ `Count`	—

Unicode (the worldwide standard that gives every character in every script a number) letters are allowed (`größe`), and `$` is legal but by convention reserved for generated code.

Keywords

Java 25 has **51 reserved keywords** that can never be identifiers — e.g. `class`, `int`, `if`, `for`, `new`, `static`, `final`, `return`, `try`, plus `const` and `goto` (reserved but unused), `strictfp` (obsolete) and `_`. It also has **17 contextual keywords** that are keywords only in certain positions, so old code using them as names still compiles: `var`, `yield`, `record`, `sealed`, `non-sealed`, `permits`, `when`, and the module words `module`, `open`, `opens`, `exports`, `requires`, `transitive`, `uses`, `provides`, `to`, `with`. `true`, `false` and `null` are *literals* (values written directly in the code), not keywords, but are still unusable as names.

Comments

```
// line comment: to the end of the line
/* block comment:
   can span lines (cannot be nested) */
/** Javadoc comment: documents the next class, method or field. @param, @return tags… */
```

Write comments that explain **why**, not what the code already says.

2.2 Variables

What it is

A **variable** is a named storage location with a fixed **type**. Think of a labelled box: the label is the name, the box's shape (type) decides what may go in.

How it works

1. **Declare**: `type name;` — reserves the name and type.
2. **Initialise**: give it a first value (`name = value;`), often together: `int age = 30;`.
3. **Use/reassign**: read it in expressions, assign a new value of the same type.
4. The variable exists within its **scope** — the block `{ }` it was declared in, from the declaration to the closing brace.

`code/Variables.java`:

```java
static int counter;          // field: gets a default value (0) automatically

public static void main(String[] args) {
    int age;                 // declaration
    age = 30;                // initialisation (assignment)
    int year = 2025, month = 9;   // declare + initialise several of the same type
    age = age + 1;           // reassignment
    final double TAX_RATE = 0.18; // final: can be assigned exactly once
    // TAX_RATE = 0.2;       // compile error: cannot assign a value to final variable TAX_RATE

    System.out.println("age=" + age + " year=" + year + " month=" + month);
    System.out.println("TAX_RATE=" + TAX_RATE + " counter (field default)=" + counter);

    {                                // an inner block
        int inner = 5;               // exists only inside these braces
        System.out.println("inner=" + inner + ", age visible here too: " + age);
    }
    // System.out.println(inner);      // compile error: cannot find symbol

    int count;
    if (age > 18) count = 1; else count = 0;   // definitely assigned on every path
    System.out.println("count=" + count);
}
```

Output:

```
age=31 year=2025 month=9
TAX_RATE=0.18 counter (field default)=0
inner=5, age visible here too: 31
count=1
```

Local variables vs fields

	Local variable (inside a method/block)	Field (inside a class)
Default value	**None** — must be assigned before use, or compile error *"variable x might not have been initialized"*	Yes: `0`, `0.0`, `false`, `'\u0000'`, or `null` for references
Lifetime	Until the block ends	As long as the object (instance field — one copy per object) or class (`static` field — one copy shared by the whole class)
Modifiers	Only `final`	Access modifiers (`public`, `private` … — who may use it), `static`, `final`, … (Chapter 5)

The compiler's **definite assignment** analysis checks that every path assigns a local before it is read — that is why `count` above compiles: both branches assign it.

`final`

`final` means *assigned once*. For a primitive, the value can never change; for a reference, the variable cannot point elsewhere but the **object itself can still change** (`final List<String> xs = new ArrayList<>(); xs.add("ok");` is legal — `List`/`ArrayList` is a growable list of items, covered in Chapter 10). Use `final` for constants (`static final int MAX_RETRIES = 3;`) and wherever it clarifies intent.

Common mistakes

- Reading a local before assigning it in every branch.
- Redeclaring a name already in scope (`int x = 1; { int x = 2; }` is a compile error — locals cannot shadow locals, i.e. hide an outer variable by reusing its name).
- Thinking `final` makes an object immutable (impossible to change after it is created).

Self-check

1. Does `int n; System.out.println(n);` compile inside a method? → No — locals have no default value.

2. Can a `final List` have elements added? → Yes; only the reference is fixed.
3. What value does an uninitialised `boolean` field have? → `false`.

2.3 Primitive types

What it is

Java has exactly **eight primitive types** — simple values stored directly (not as objects). Everything else is a reference type (objects). Sizes and ranges are fixed by the language on every platform, unlike C. Sizes are measured in **bits** — a bit is a single binary digit, 0 or 1; 8 bits make one **byte**.

`code/Primitives.java` prints the real values from the wrapper classes (object versions of each primitive, see 2.5):

```
System.out.printf("%-8s %5d  %-22d %d%n", "int", Integer.SIZE, Integer.MIN_VALUE, Integer.MAX_VALUE);
// ... one line per type
```

Output:

```
type      bits  min                   max
byte         8  -128                  127
short       16  -32768                 32767
int         32  -2147483648            2147483647
long        64  -9223372036854775808   9223372036854775807
char        16  0                     65535
float       32  1.4E-45               3.4028235E38
double      64  4.9E-324              1.7976931348623157E308
boolean  true or false (size not specified by the language)
```

Type	Kind	Size	Range / precision	Default (fields)	Use it for
`byte`	signed integer	8 bits	−128 … 127	`0`	raw binary data, I/O buffers
`short`	signed integer	16 bits	−32 768 … 32 767	`0`	rarely — large arrays of small numbers
`int`	signed integer	32 bits	about ±2.1 billion (-2^{31} … $2^{31}-1$)	`0`	**the default integer** — counts, indexes
`long`	signed integer	64 bits	about ±9.2 × 10^{18}	`0L`	timestamps (ms since 1970), IDs, money in cents, big counters
`float`	IEEE 754 binary floating point	32 bits	~6–7 significant decimal digits, up to ±3.4 × 10^{38}	`0.0f`	graphics, large arrays where memory matters
`double`	IEEE 754 binary floating point	64 bits	~15–16 significant digits, up to ±1.8 × 10^{308}	`0.0`	**the default decimal** — measurements, science
`char`	unsigned UTF-16 code unit	16 bits	`'\u0000'` … `'\uFFFF'` (0 … 65 535)	`'\u0000'`	single characters
`boolean`	logical	not specified	`true`, `false`	`false`	conditions, flags

Notes:

- *Signed* means the type can hold negative numbers; *unsigned* means zero and positives only. **IEEE 754** is the international standard for how computers store numbers with a decimal point. **UTF-16** is a text encoding that stores characters as 16-bit units. An *I/O buffer* is a block of memory used to

hold data being read or written. *Epoch milliseconds* count milliseconds since 1 January 1970 UTC, the usual way computers store a point in time.

- `float` / `double` `MIN_VALUE` is the smallest **positive** value, not the most negative; the most negative is `-Double.MAX_VALUE`.
- A `char` is a UTF-16 *code unit*. Characters outside the Basic Multilingual Plane (the first 65 536 Unicode characters; many emoji lie beyond it) need two `char`s (a *surrogate pair*); see Chapter 7.
- `boolean` is not a number: `if (1)` does not compile, and you cannot cast between `boolean` and `int`.

Choosing a type

Need	Use
Whole numbers that fit in ±2 billion	`int`
Bigger whole numbers, epoch milliseconds, file sizes	`long`
Decimal measurements where tiny rounding is fine	`double`
Money, exact decimal arithmetic	`long` cents or `BigDecimal` (Java's exact decimal-number class) — never `double` (2.11)
Arbitrary-size integers	`BigInteger` (Chapter 7)
Yes/no	`boolean`

Self-check

1. Largest `int`? → 2 147 483 647 (`Integer.MAX_VALUE`, $2^{31}-1$).
2. Is `char` signed? → No, it is unsigned 16-bit (0–65 535).
3. Why is `Double.MIN_VALUE` positive? → It is the smallest positive (subnormal — a tiny value below the normal range, stored with fewer digits of precision) double, not the most negative.

2.4 Literals

What it is

A **literal** is a value written directly in source code. Its form decides its type. Numbers can be written in several bases: **hexadecimal** (base 16, digits 0–9 and A–F, prefix `0x`), **octal** (base 8, prefix `0`) and **binary** (base 2, prefix `0b`).

`code/Literals.java`:

```
int decimal = 255, hex = 0xFF, octal = 0377, binary = 0b1111_1111;
long big = 9_000_000_000L;              // L suffix: needed beyond int range
float price = 19.99f;                   // f suffix: literal is double otherwise
double sci = 6.022e23, plain = 3.14;
char letter = 'A', unicode = 'é', tab = '\t';
boolean ready = true;
String text = "He said \"hi\"\\n";      // escaped quote and backslash

System.out.println(decimal + " " + hex + " " + octal + " " + binary);
System.out.println(big + " " + price + " " + sci + " " + plain);
System.out.println(letter + " " + unicode + " [" + tab + "] " + ready);
System.out.println(text);
System.out.println("char 'A' as a number: " + (int) letter + ", 'A' + 1 = " + (letter + 1)
        + ", (char) ('A' + 1) = " + (char) (letter + 1));
```

Output:

```
255 255 255 255
9000000000 19.99 6.022E23 3.14
A é [   ] true
He said "hi"\n
char 'A' as a number: 65, 'A' + 1 = 66, (char) ('A' + 1) = B
Integer.toBinaryString(10) = 1010, toHexString(255) = ff
```

Literal	Form	Type
Decimal integer	`42`, `1_000_000`	`int`
Long	`42L` (use capital `L`; `l` looks like `1`)	`long`
Hexadecimal	`0xFF`, `0xCAFE_BABE`	`int` (or `long` with `L`)
Octal	leading `0`: `0377` = 255	`int` — beware: `010` is **8**
Binary	`0b1010` (Java 7)	`int`
Floating point	`3.14`, `1e-9`, `.5`, `2.`	`double`
Float	`3.14f`	`float`
Character	`'A'`, `'\n'`, `'é'`	`char`
Boolean	`true`, `false`	`boolean`
String	`"text"`, text blocks `"""…"""`	`String`
Null	`null`	the null type (assignable to any reference)

Underscores (Java 7) may separate digits for readability — `1_000_000`, `0b1111_0000` — but not at the start or end of the digits, next to the decimal point, next to the `L`/`f` suffix, or directly after `0x`/`0b`.

Escape sequences in `char` and `String` literals: `\n` newline, `\t` tab, `\r` carriage return, `\b` backspace, `\f` form feed, `\"`, `\'`, `\\`, `\s` space (Java 15), octal `\0`–`\377`, and Unicode `\uXXXX`.

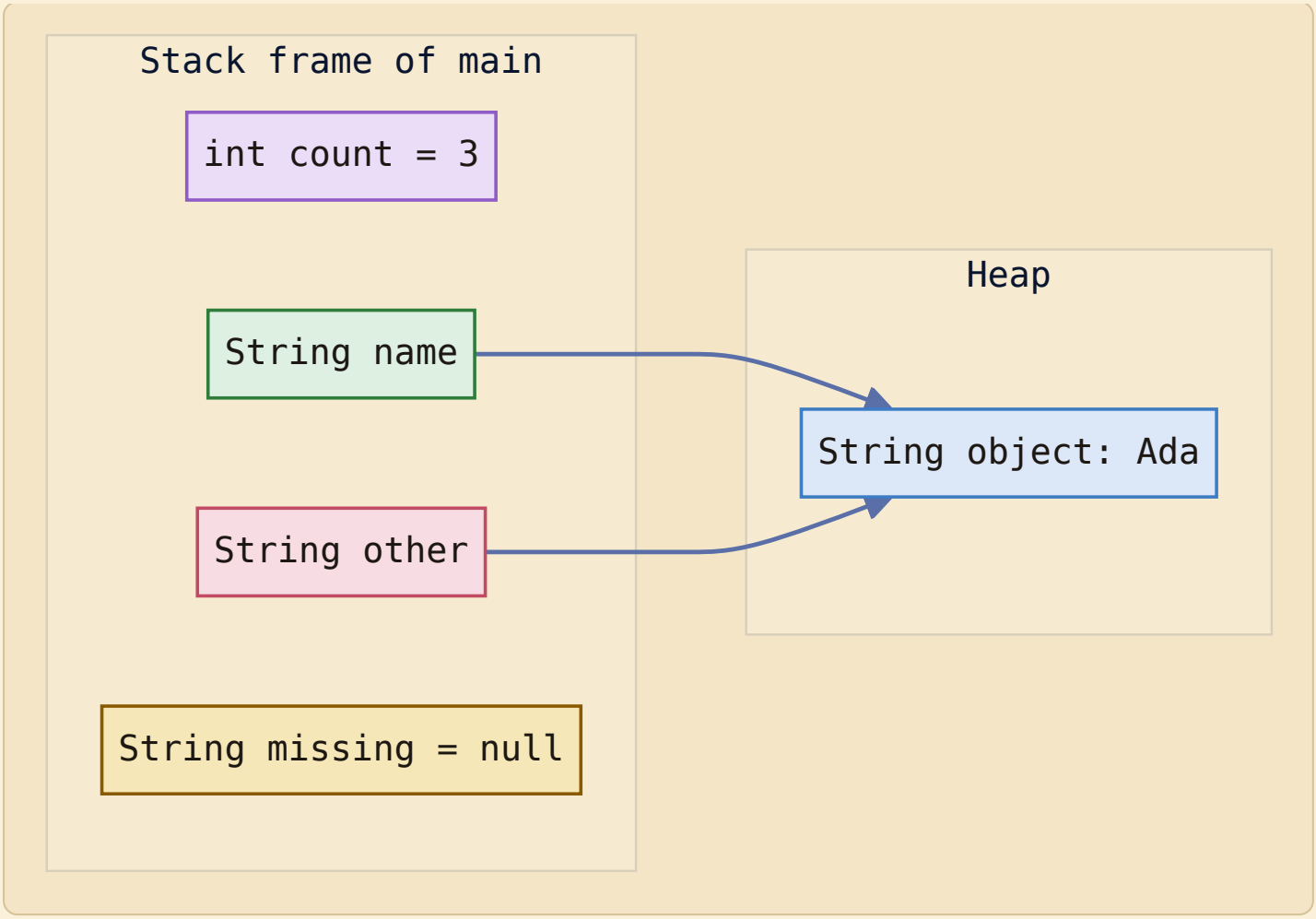
Common mistakes

- `long big = 9000000000;` → compile error *"integer number too large"*: the literal is an `int` until you add `L`.
- `float f = 1.5;` → compile error (*possible lossy conversion from double to float*); write `1.5f`.
- Leading zeros for alignment: `int code = 010;` is 8.

2.5 Reference types and `null`

What it is

Any type that is not one of the eight primitives is a **reference type**: classes (`String`, `Scanner`), interfaces (`List`), arrays (`int[]`), enums and records. (Interfaces, arrays, enums and records are each explained in Chapters 4–6; for now, just know they are all reference types.) A variable of a reference type holds a **reference** — a handle that points to an object on the heap (the memory area where all objects live; see 1.8) — or the special value `null`, meaning "points to nothing".



- Assigning a reference (`other = name`) copies the **arrow**, not the object: both variables now point to the same `String`.
- Calling a method through `null` throws `NullPointerException` at run time. To *throw* an **exception** is to signal an error by creating an error object that stops normal execution unless some code catches it (Chapter 8). Since Java 14 (JEP 358, on by default from Java 15), these *helpful NullPointerException* messages name exactly what was null, e.g. *Cannot invoke "String.length()" because "name" is null*.

	Primitive	**Reference**
Holds	the value itself	a pointer to an object, or `null`
Default (field)	`0` / `false` / `'\u0000'`	`null`
`==` compares	values	**identity** (same object?)
Has methods	no	yes (`name.length()`)
Name style	lowercase keyword (`int`)	a type name (`String`, `Integer`)

Every primitive has a **wrapper class** (`Integer`, `Double`, `Character`, `Boolean`…) so it can be used where an object is required, e.g. in `List<Integer>` (a list of `Integer` objects; the `<…>` part is a *generic* type argument, Chapter 9). Java converts automatically (*autoboxing/unboxing*). The details — and the `Integer` cache trap — are in Chapter 7.

2.6 `var` — local variable type inference (Java 10)

What it is

`var` (Java 10, JEP 286) lets the compiler **infer** (work out by itself) a local variable's type from its initialiser (the `= value` part of the declaration). The variable is still statically typed — the type is fixed at compile time; you just do not write it.

`code/VarDemo.java`:

```
var count = 10;                          // int
var ratio = 10 / 4.0;                    // double
var name = "Ada";                        // String
var scores = new HashMap<String, List<Integer>>();   // HashMap<String, List<Integer>>
scores.put(name, new ArrayList<>(List.of(90, 85)));
for (var entry : scores.entrySet()) {    // Map.Entry<String, List<Integer>>
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
// count = "ten";                        // compile error: the type is fixed at int
var items = new ArrayList<>();           // careful: ArrayList<Object>
items.add(1); items.add("two");
System.out.println(count + " " + ratio + " " + items);
```

Output:

```
Ada -> [90, 85]
10 2.5 [1, two]
{a=1}
```

Where `var` is allowed

Allowed	Not allowed
Local variables with an initialiser	Fields, method parameters, return types
`for` loop variables and for-each variables	Without initialiser: `var x;`
try-with-resources variables (auto-closing resources, Chapter 8)	`var x = null;` (no type to infer)
Lambda (short inline function) parameters (Java 11): `(var a, var b) -> a + b` — useful to add annotations (`@`-tags such as `@Override` that attach extra information)	Array initialiser alone: `var a = {1, 2};`
	Multiple variables: `var a = 1, b = 2;`

`var` is a contextual keyword: `var var = 1;` compiles, but you cannot name a *type* `var`.

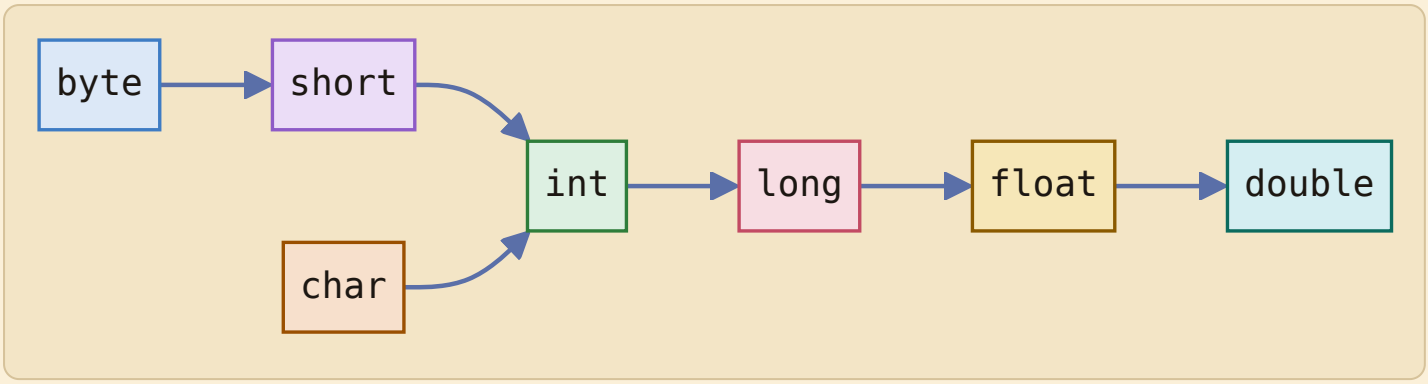
Trade-offs and style

Use `var` when the type is obvious from the right-hand side (`new`, a well-named factory — a method that creates and returns objects) or long and noisy (generic types such as `Map<String, List<Integer>>`). Avoid it when the reader cannot see the type (`var result = service.process();`) or when the inferred type differs from what you want (`var list = new ArrayList<>()` becomes `ArrayList<Object>`; `var x = 1;` is `int`, not `long`). The OpenJDK team's *Local Variable Type Inference Style Guidelines* say the same: choose clarity for the reader.

2.7 Type conversion: widening, narrowing, casting, promotion

What it is

Converting a value from one type to another. Java does **safe** conversions automatically and requires an explicit **cast** — `(type) value` — for risky ones.



Arrows show **widening primitive conversions**, done automatically (`int i = 'A'; long l = i; double d = l;`). Going against the arrows is **narrowing** and needs a cast.

How it works

1. **Widening** (automatic): the target type can hold every value… *almost*. `int → float`, `long → float` and `long → double` can lose **precision** (exact digits), not magnitude (overall size): a `float` has 24 bits of mantissa (the part of a floating-point number that stores its significant digits).
2. **Narrowing** (needs a cast): `(int) 3.99` truncates (chops off the fraction) toward zero to `3`; `(byte) 300` keeps the low 8 bits (300 − 256 = 44). Floating → integer conversion clamps out-of-range values to MIN/MAX and turns NaN ("Not a Number", the result of undefined maths such as 0.0/0) into 0.
3. **Numeric promotion** in expressions: before a binary (two-input) arithmetic operator runs, `byte`, `short` and `char` operands (the values an operator works on) become `int`; if either operand is `long`, `float` or `double`, both are converted to the wider of these. So `byte + byte` is an `int`.
4. **Compile-time constants** (values the compiler can work out itself, such as literals or `final` variables set from literals) may be assigned to a narrower type if they fit: `byte b = 10 + 20;` is fine; `byte b = x + y;` (variables) is not.
5. **Compound assignment** (`+=`, `-=`, `*=`…) includes a hidden cast: `x += 5` means `x = (T) (x + 5)` where `T` is `x`'s type.

`code/Conversions.java` (key lines):

```
double pi = 3.99;
int truncated = (int) pi;          // narrowing needs a cast; drops the fraction
long rounded = Math.round(pi);     // rounding is a method, not a cast

byte b = (byte) 300;               // keeps only the low 8 bits: 300 - 256 = 44
int precise = 123_456_789;
float f = precise;                 // widening but LOSSY: float has 24 bits of precision

byte x = 10, y = 20;
// byte sum = x + y;               // compile error: x + y is an int
byte sum = (byte) (x + y);
x += 5;                            // compound assignment hides a cast: x = (byte) (x + 5)

long wrong = 1_000_000 * 1_000_000;       // int overflow BEFORE widening to long
long right = 1_000_000L * 1_000_000;
```

Output:

```
widened: 100 100.0
(int) 3.99 = 3, Math.round(3.99) = 4
(byte) 300 = 44, (byte) 200 = -56
(int) 1e20 = 2147483647, (int) Double.NaN = 0
int 123456789 -> float -> int: 123456792
sum=30 sum2=30 x=15
constant=30 'a'+1=98 7/2=3 7/2.0=3.5
wrong=-727379968 right=1000000000000
```

The last line is the classic bug: the multiplication happens in `int` (both operands are `int`), overflows, and only then is the wrong result widened to `long`. Put `L` on the **first** operand.

Common mistakes

- Expecting `(int)` to round. It truncates toward zero: `(int) -3.99` is `-3`. Use `Math.round`, `Math.floor`, `Math.ceil`.
- `7 / 2` is `3`: both operands are `int`, so the division is integer division. Write `7 / 2.0` or `(double) 7 / 2`. Note `(double) (7 / 2)` is `3.0` — the cast came too late.
- Assuming widening is always lossless: `int → float` can change the value.

In the interview

Q: Why doesn't `short s = 1; s = s + 1;` compile, but `s += 1;` does? `s + 1` is promoted to `int`, and assigning an `int` to a `short` needs a cast. Compound assignment includes that cast implicitly.

- *What is `(byte) 200`?* −56: the low 8 bits `11001000` read as a signed byte.
- *Is `long → double` safe?* Magnitude yes, precision no: doubles have 53 bits of mantissa, longs 64.

Self-check

1. `char c = 'a'; c + 1` — type and value? → `int`, 98.
2. `(int) 7.9 + (int) -7.9` → 7 + (−7) = 0.
3. Fix `long ms = 24 * 60 * 60 * 1000 * 365;` → `24L * 60 * 60 * 1000 * 365`.

2.8 Operators

What it is

An **operator** is a symbol that computes a value from one (*unary*), two (*binary*) or three (*ternary*) operands.

Arithmetic: `+ - * / %` and `++ --`

`code/Arithmetic.java`:

```
System.out.println("17 / 5 = " + 17 / 5 + ", 17 % 5 = " + 17 % 5);
System.out.println("-17 / 5 = " + -17 / 5 + ", -17 % 5 = " + -17 % 5
        + ", Math.floorMod(-17, 5) = " + Math.floorMod(-17, 5));
System.out.println("17.0 / 5 = " + 17.0 / 5 + ", 5.5 % 2 = " + 5.5 % 2);
System.out.println("1.0 / 0 = " + 1.0 / 0 + ", 0.0 / 0 = " + 0.0 / 0);
try { System.out.println(1 / 0); }
catch (ArithmeticException e) { System.out.println("1 / 0 -> ArithmeticException: " + e.getMessage()); }

int a = 5;
int b = a++;     // post-increment: b gets the old value, then a becomes 6
int c = ++a;     // pre-increment: a becomes 7, then c gets 7
int n = 3;
int result = n++ + ++n;   // 3 + 5
```

Output:

```
17 / 5 = 3, 17 % 5 = 2
-17 / 5 = -3, -17 % 5 = -2, Math.floorMod(-17, 5) = 3
17.0 / 5 = 3.4, 5.5 % 2 = 1.5
1.0 / 0 = Infinity, 0.0 / 0 = NaN
1 / 0 -> ArithmeticException: / by zero
a=7 b=5 c=7
n++ + ++n with n=3 -> 8, n=5
compound ops -> x=1
-(-5) = 5, +3 = 3
```

Rule	Detail
Integer `/` truncates toward zero	`-17 / 5` is `-3`, not `-4`
`%` takes the sign of the **dividend** (the number being divided)	`-17 % 5` is `-2`. For a always-non-negative result (e.g. circular buffers — fixed-size arrays that wrap round to the start, "day of week"), use `Math.floorMod`
Integer division by zero	throws `ArithmeticException`
Floating division by zero	no exception: `Infinity`, `-Infinity` or `NaN`
`x++` vs `++x`	both add 1 to `x`; the *expression's value* is the old value (postfix) or the new value (prefix)
`+` with a `String` operand	means concatenation (joining text end to end) (2.12)

Style: use `++`/`--` as standalone statements (`i++;`). Expressions like `n++ + ++n` are legal and deterministic (always give the same result; Java evaluates operands left to right) but unreadable.

Assignment operators

`=`, and compound `+= -= *= /= %= &= |= ^= <<= >>= >>>=`. Assignment is itself an expression whose value is the assigned value, and it is right-associative: `a = b = c = 7` assigns 7 to all three.

Relational and equality: `< <= > >= == !=`, `instanceof`

All return `boolean`. For **primitives**, `==` compares values. For **references**, `==` compares **identity** — whether both sides point to the *same object*. To compare contents, call `equals` (a method every object has for content comparison).

`code/Comparison.java`:

```
String s1 = "java";
String s2 = new String("java");
System.out.println("s1 == s2: " + (s1 == s2) + ", s1.equals(s2): " + s1.equals(s2));
```

Output:

```
s1 == s2: false, s1.equals(s2): true
```

`x instanceof Type` tests whether a reference points to an object of that type (false for `null`). The pattern form `x instanceof String s` is in Chapter 3.

Logical: `&& || !` and `& | ^`

Operator	Name	Evaluates right side?
`&&`	conditional AND	only if left is `true` (**short-circuit**)
`||`	conditional OR	only if left is `false` (short-circuit)
`!`	NOT	—
`&`, `|`	logical AND / OR on booleans	**always** both sides
`^`	XOR (exactly one true)	always both

```
static boolean check(String label, boolean value) {
    System.out.println("  evaluated " + label);
    return value;
}
boolean r1 = check("left", false) && check("right", true);
boolean r2 = check("left", false) & check("right", true);

String name = null;
if (name != null && name.length() > 3) System.out.println("long name");   // safe: && stops at null
```

Output:

```
&& short-circuits:
  evaluated left
& evaluates both:
  evaluated left
  evaluated right
r1=false r2=false true ^ true=false !true=false
null-safe check with && worked
```

Short-circuiting is what makes the *null guard* idiom (a common coding pattern) `x != null && x.foo()` safe. With `&` it would throw `NullPointerException`.

Ternary `?:`

`condition ? valueIfTrue : valueIfFalse` is an **expression** (it produces a value), unlike `if`:

```
String category = age < 13 ? "child" : age < 20 ? "teen" : "adult";   // "adult" for 20
```

Keep it to one level; nested ternaries become a puzzle — use `if` or a `switch` expression (Chapter 3).

Bitwise and shift: `& | ^ ~ << >> >>>`

On integer types these work on individual bits. A *flag* is a yes/no setting stored in one bit; a *mask* is a value used with `&` to pick out particular bits. `code/Bitwise.java`:

```
static final int READ = 1, WRITE = 2, EXEC = 4;   // 001, 010, 100
int a = 0b1100, b = 0b1010;
System.out.println("a & b = " + bin(a & b) + ", a | b = " + bin(a | b)
        + ", a ^ b = " + bin(a ^ b) + ", ~a = " + (~a));
System.out.println("1 << 4 = " + (1 << 4) + ", 64 >> 2 = " + (64 >> 2));
System.out.println("-16 >> 2 = " + (-16 >> 2) + ", -16 >>> 28 = " + (-16 >>> 28));
int perms = READ | WRITE;                // set two flags
perms &= ~WRITE;                         // clear a flag
```

Output:

```
a & b = 1000, a | b = 1110, a ^ b = 110, ~a = -13
1 << 4 = 16, 64 >> 2 = 16
-16 >> 2 = -4, -16 >>> 28 = 15
1 << 32 = 1  (shift distance is taken mod 32 for int)
can write? true, can exec? false
after clearing WRITE: 1
is 12 even? true
```

Operator	Meaning	Example
`&`	1 where both bits are 1	mask/test a flag: `(perms & WRITE) != 0`
`|`	1 where either is 1	set a flag
`^`	1 where bits differ	toggle; `a ^ a == 0`
`~`	flip every bit; `~x == -x - 1`	clear with `& ~FLAG`
`<<`	shift left, fill with 0; `x << n` = $x \cdot 2^n$ (until overflow)	`1 << k` = k-th bit
`>>`	*arithmetic* shift right, copies the sign bit (the top bit: 1 means negative)	`-16 >> 2 = -4`
`>>>`	*logical* shift right, fills with 0	unsigned view of negatives

The shift distance uses only the low 5 bits for `int` (6 for `long`), so `1 << 32 == 1`. Real uses: permission flags, hash functions (functions that turn a value into a number used to decide where to store it; `HashMap`, Java's key→value lookup table, mixes `h ^ (h >>> 16)`), `EnumSet`'s bit vector, network protocols, and bit tricks in interviews (Chapter 19).

Self-check

1. `-7 / 2` and `-7 % 2`? → `-3` and `-1`.
2. Why is `if (s != null & s.isEmpty())` dangerous? → `&` evaluates both sides, so it throws when `s` is null.
3. `int x = 5; int y = x++ * 2;` → `y = 10`, `x = 6`.
4. Two strings print the same but `==` is false. Why? → They are different objects; use `equals`.

2.9 Operator precedence and associativity

What it is

Precedence decides which operator binds first (`*` before `+`); **associativity** decides the order among equal-precedence operators (most are left-to-right; assignment and the ternary are right-to-left). Operands themselves are always **evaluated left to right**.

Level (high → low)	Operators	Associativity
1 Postfix	`expr++` `expr--`	left
2 Unary	`++expr` `--expr` `+` `-` `~` `!` `(type)` cast	right
3 Multiplicative	`*` `/` `%`	left
4 Additive	`+` `-` (and string `+`)	left
5 Shift	`<<` `>>` `>>>`	left
6 Relational	`<` `>` `<=` `>=` `instanceof`	left
7 Equality	`==` `!=`	left
8 Bitwise AND	`&`	left
9 Bitwise XOR	`^`	left
10 Bitwise OR	`|`	left
11 Conditional AND	`&&`	left
12 Conditional OR	`||`	left
13 Ternary	`? :`	right
14 Assignment, lambda	`=` `+=` `-=` … `->`	right

`code/Precedence.java`:

```
System.out.println("2 + 3 * 4 = " + (2 + 3 * 4));
System.out.println("(2 + 3) * 4 = " + ((2 + 3) * 4));
System.out.println("10 - 4 - 3 = " + (10 - 4 - 3) + "  (left to right)");
int a, b, c;
a = b = c = 7;                                  // assignment is right-associative
System.out.println("a=" + a + " b=" + b + " c=" + c);
System.out.println(1 + 2 + "3" + 4 + 5);        // left to right: 3, then "33", "334", "3345"
System.out.println("true || false && false = " + (true || false && false));
int x = 6;
System.out.println("x & 1 == 0 needs parentheses: " + ((x & 1) == 0));
```

Output:

```
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
10 - 4 - 3 = 3  (left to right)
a=7 b=7 c=7
3345
true || false && false = true
5 > 3 == true: true
x & 1 == 0 needs parentheses: true
```

- `&&` binds tighter than `||`, so `true || false && false` is `true || (false && false)`.
- `==` binds tighter than `&`, so `x & 1 == 0` means `x & (1 == 0)` — a compile error for `int x`. Always parenthesise bitwise tests.
- `+` is left-associative, so `1 + 2 + "3"` adds first, then concatenates.

Rule of thumb: remember "unary → multiplicative → additive → comparison → equality → logical → ternary → assignment", and add parentheses whenever a reader might hesitate.

2.10 Integer overflow

What it is

Integer arithmetic in Java is done modulo 2^{32} (`int`) or 2^{64} (`long`) — that is, only the remainder after dividing by 2^{32} or 2^{64} is kept. When a result exceeds the range it silently **wraps around** — no exception, no warning. This is **overflow**.

Analogy: a car odometer with six digits rolls from 999 999 to 000 000.

> **Key term — two's complement:** the way computers store negative integers. The top bit is the sign bit, and a negative number −n is stored as the bit pattern of $2^{32} - n$ (for `int`). This lets the same adding circuit handle positive and negative numbers, and it is why adding 1 to the largest `int` produces the most negative one.

Why it matters

Overflow bugs are real and expensive: the binary-search (finding an item in sorted data by repeatedly halving the range) midpoint bug `(low + high) / 2` sat in the JDK's own `Arrays.binarySearch` for years (fixed in 2006), and timestamps multiplied in `int` produce negative durations.

`code/Overflow.java`:

```
int max = Integer.MAX_VALUE;
System.out.println("MAX_VALUE + 1 = " + (max + 1));
System.out.println("Math.abs(Integer.MIN_VALUE) = " + Math.abs(Integer.MIN_VALUE));

int low = 2_000_000_000, high = 2_100_000_000;
System.out.println("(low + high) / 2 = " + (low + high) / 2 + "   <- overflow bug");
System.out.println("low + (high - low) / 2 = " + (low + (high - low) / 2));
System.out.println("(low + high) >>> 1 = " + ((low + high) >>> 1));

try { Math.addExact(max, 1); }
catch (ArithmeticException e) { System.out.println("Math.addExact(MAX, 1) -> " + e.getMessage()); }
long seconds = 30L * 24 * 60 * 60 * 1000;     // L on the first operand: all long math
```

Output:

```
MAX_VALUE + 1 = -2147483648
Math.abs(Integer.MIN_VALUE) = -2147483648
-Integer.MIN_VALUE = -2147483648
(low + high) / 2 = -97483648   <- overflow bug
low + (high - low) / 2 = 2050000000
(low + high) >>> 1 = 2050000000
Math.addExact(MAX, 1) -> integer overflow
Math.toIntExact(3e9) -> integer overflow
30 days in ms (long) = 2592000000, as int math = -1702967296
```

How to defend

Technique	Example
Use a wider type early	`long total = (long) a * b;` or `30L * …`
Throw instead of wrapping	`Math.addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact`, `toIntExact` (Java 8), `absExact` (Java 15)
Rearrange the formula	`low + (high - low) / 2`
Unlimited size	`BigInteger` (Chapter 7)

Note that `Math.abs(Integer.MIN_VALUE)` is still negative, because +2 147 483 648 does not fit in an `int`.

In the interview

Q: What happens when an `int` overflows in Java? It wraps around silently using two's-complement arithmetic (`MAX_VALUE + 1 == MIN_VALUE`). Use `long`, `Math.*Exact` methods (which throw `ArithmeticException`), or `BigInteger`.

- *Where have you seen overflow in practice?* Binary-search midpoints, hash-code arithmetic (intentional), millisecond calculations, summing large arrays of `int`.

2.11 Floating-point numbers

What it is

`float` and `double` follow **IEEE 754** binary floating point: a number is stored as sign × binary fraction (mantissa) × 2^exponent. Most decimal fractions (0.1, 0.2) have no exact binary form — just as 1/3 has no exact decimal form — so they are stored as the nearest representable value.

`code/FloatingPoint.java`:

```
System.out.println("0.1 + 0.2 = " + (0.1 + 0.2));
System.out.println("0.1 + 0.2 == 0.3: " + (0.1 + 0.2 == 0.3));
System.out.println("|diff| < 1e-9: " + (Math.abs((0.1 + 0.2) - 0.3) < 1e-9));
double nan = 0.0 / 0;
System.out.println("NaN == NaN: " + (nan == nan) + ", Double.isNaN: " + Double.isNaN(nan));
System.out.println("-0.0 == 0.0: " + (-0.0 == 0.0) + ", Double.compare(-0.0, 0.0): " + Double.compare(-0.0, 0.0));
double total = 0;
for (int i = 0; i < 10; i++) total += 0.1;
BigDecimal exact = new BigDecimal("0.1").add(new BigDecimal("0.2"));
```

Output:

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: false
|diff| < 1e-9: true
0.1f + 0.2f = 0.3
1.0 - 0.9 = 0.09999999999999998
NaN == NaN: false, Double.isNaN: true
-0.0 == 0.0: true, Double.compare(-0.0, 0.0): -1
1e308 * 10 = Infinity, Math.sqrt(-1) = NaN
16777217 as float = 1.6777216E7  (2^24 + 1 not representable)
0.1 added ten times = 0.9999999999999999
BigDecimal 0.1 + 0.2 = 0.3, new BigDecimal(0.1) = 0.1000000000000000055511151231257827021181583404541015625
Math.round(2.5)=3 Math.round(-2.5)=-2 Math.floor(-2.5)=-3.0 Math.ceil(-2.5)=-2.0
```

(`0.1f + 0.2f` *prints* as `0.3` only because `float`'s shorter printed form hides the error.)

Special values

Value	Produced by	Test with
`Infinity`, `-Infinity`	overflow (`1e308 * 10`), `1.0 / 0`	`Double.isInfinite(x)`
`NaN` (Not a Number)	`0.0 / 0`, `Math.sqrt(-1)`	`Double.isNaN(x)` — **never** `x == Double.NaN` (NaN is not equal to anything, even itself)
`-0.0`	`-1.0 / Double.POSITIVE_INFINITY`, rounding tiny negatives	equals `0.0` with `==`, but `Double.compare` and `Double.equals` distinguish it

How to handle it

1. **Compare with a tolerance**, not `==`: `Math.abs(a - b) < 1e-9` (choose the tolerance for your domain).
2. **Money**: use `long` minor units (cents) or `BigDecimal` constructed from a **String** (`new BigDecimal("0.1")`) or `BigDecimal.valueOf(0.1)` — `new BigDecimal(0.1)` captures the binary error exactly, as shown above.

3. **Rounding**: `Math.round` rounds half **up toward positive infinity** (`-2.5 → -2`); `Math.floor`/`ceil` return `double`. For decimal rounding modes (half-even — round a trailing 5 to the nearest even digit — as banks use), use `BigDecimal.setScale`.
4. **Loops**: never use a `double` as a loop counter compared with `==` — after ten additions of 0.1 you have 0.9999999999999999, not 1.0.
5. Since Java 17 (JEP 306) all floating-point arithmetic is strict IEEE 754 on every platform, so results are reproducible; the `strictfp` modifier is obsolete.

In the interview

Q: Why is `0.1 + 0.2 != 0.3` in Java? 0.1 and 0.2 cannot be represented exactly in binary floating point; the stored approximations sum to 0.30000000000000004. Compare with a tolerance, or use `BigDecimal` for exact decimal arithmetic such as money.

- *How do you check for NaN?* `Double.isNaN(x)`; `x != x` also works because NaN is the only value not equal to itself.
- *Why not `new BigDecimal(0.1)`?* It converts the already-inexact double; use the String constructor or `BigDecimal.valueOf`.

Self-check

1. What does `1 / 0.0` give? And `1 / 0`? → `Infinity`; `ArithmeticException`.
2. Best type for a bank balance? → `BigDecimal` (or `long` cents).
3. `Math.round(-0.5)`? → `0`.

2.12 Strings and text blocks

What it is

A `String` is an object holding an immutable sequence of characters. It is a reference type with special support: literals (`"hi"`), the `+` concatenation operator, and a pool of shared literals. This section covers what you need for the next chapters; the full API, `StringBuilder` and formatting in depth are in Chapter 7.

`code/StringBasics.java`:

```
String message = "Hello" + ", " + "Ada" + "!";
System.out.println(message + " length=" + message.length());
System.out.println("charAt(0)=" + message.charAt(0) + " indexOf(\"Ada\")=" + message.indexOf("Ada")
        + " substring(7, 10)=" + message.substring(7, 10));

String s = "java";
s.toUpperCase();                        // result thrown away: Strings are immutable
System.out.println("after s.toUpperCase(): " + s);
s = s.toUpperCase();                    // reassign the variable to the new String
System.out.println("after s = s.toUpperCase(): " + s);

String a = "hi", b = "hi", c = new String("hi");
System.out.println("a == b: " + (a == b) + " (same pooled literal), a == c: " + (a == c)
        + ", a.equals(c): " + a.equals(c) + ", \"HI\".equalsIgnoreCase(a): " + "HI".equalsIgnoreCase(a));
System.out.println("String.valueOf(42) + 1 = " + String.valueOf(42) + 1
        + ", Integer.parseInt(\"42\") + 1 = " + (Integer.parseInt("42") + 1));
System.out.println("'a' + 'b' = " + ('a' + 'b') + ", \"\" + 'a' + 'b' = " + ("" + 'a' + 'b'));
```

Output:

```
Hello, Ada! length=11
charAt(0)=H indexOf("Ada")=7 substring(7, 10)=Ada
upper=HELLO, ADA! contains Ada? true
after s.toUpperCase(): java
after s = s.toUpperCase(): JAVA
a == b: true (same pooled literal), a == c: false, a.equals(c): true, "HI".equalsIgnoreCase(a): true
compareTo: -1, isBlank: true, strip: [pad], repeat: abababab
String.valueOf(42) + 1 = 421, Integer.parseInt("42") + 1 = 43
'a' + 'b' = 195, "" + 'a' + 'b' = ab
null concatenated: null
```

Key facts

Fact	Consequence
Immutable — no method changes a `String`	Methods return a *new* string; assign the result. Safe to share between threads (independent paths of execution running at the same time) and as `HashMap` keys (the lookup values in a map).
String pool — identical literals share one object	`"hi" == "hi"` is `true`, but `new String("hi")` makes a distinct object. Never rely on `==`; use `equals`.
Indexes are **0-based** (the first character is at index 0); `substring(begin, end)` excludes `end`	`"Hello".substring(1, 3)` is `"el"`; out-of-range throws `StringIndexOutOfBoundsException`.
`+` with any `String` operand concatenates	Numbers are converted with `String.valueOf`; `null` becomes `"null"`. Adding two `char`s is **numeric** (`'a' + 'b' == 195`).
Concatenating in a loop creates many objects	Use `StringBuilder` (a changeable, growable string) in loops (Chapter 7).
Converting	`String.valueOf(x)` / `"" + x` to text; `Integer.parseInt`, `Double.parseDouble` from text (throw `NumberFormatException` on bad input).

Most-used methods (Java version for newer ones): `length`, `charAt`, `indexOf`, `substring`, `contains`, `startsWith`/`endsWith`, `toUpperCase`/`toLowerCase`, `trim`, `strip` (11), `isEmpty`, `isBlank` (11), `repeat` (11), `lines` (11), `replace`, `split`, `equals`, `equalsIgnoreCase`, `compareTo`, `String.join`, `formatted` (15).

Text blocks (Java 15)

A **text block** (JEP 378, final in Java 15) is a multi-line string literal opened by `"""` followed by a line break and closed by `"""`. It is still an ordinary `String`.

`code/TextBlocks.java`:

```java
String json = """
        {
          "name": "Ada",
          "langs": ["Java", "SQL"]
        }
        """;
System.out.print(json);

String sql = """
    SELECT id, name
    FROM users
    WHERE active = true\
    """;                                        // \ at line end: no newline there
System.out.println("[" + sql + "]");

String spaces = """
    trailing kept:\s
    quote: \""" inside
    """;
System.out.print(spaces.replace(' ', '.'));

String html = """
    <p>Hello, %s!</p>
    """.formatted("World");
```

Output:

```
{
  "name": "Ada",
  "langs": ["Java", "SQL"]
}
[SELECT id, name
FROM users
WHERE active = true]
trailing.kept:.
quote:."""inside
<p>Hello, World!</p>
lines in json: 4
```

How the compiler processes a text block:

1. The content starts on the line **after** the opening `"""` (text on the same line is a compile error).
2. **Incidental indentation** is removed: the compiler finds the smallest indentation among non-blank lines *and the closing `"""` line*, and strips that much from every line. Move the closing delimiter (the `"""` marker) left to keep more indentation.
3. Trailing spaces on each line are stripped; line endings become `\n` whatever the OS.
4. Escapes are processed last. Two escapes were added for text blocks: `\<line-end>` joins lines (no newline), and `\s` is a single space that survives stripping.
5. If the closing `"""` is on its own line, the string ends with `\n`; if it follows text directly, it doesn't.

Use text blocks for embedded SQL (database query language), JSON (a text format for structured data), HTML (web-page markup) and test fixtures (fixed sample data used by tests). For exam-level indentation traps, see the OCP repo's text blocks section.

Common mistakes

- `if (input == "yes")` — compares identity; use `"yes".equals(input)` (also null-safe).
- Calling `s.trim();` without assigning the result.
- `"Total: " + a + b` when `a`, `b` are numbers → concatenates both (`"Total: 12"`); write `"Total: " + (a + b)`.

Self-check

1. `"abc".substring(1)`? → `"bc"`.

2. Why is `"yes".equals(input)` better than `input.equals("yes")`? → It does not throw if `input` is `null`.
3. Does a text block with the closing `"""` on its own line end with a newline? → Yes.

2.13 Console output and input

Output: `print`, `println`, `printf`

`System.out` is a `PrintStream` (Java's class for writing text) connected to **standard output** (the program's normal output, the terminal by default); `System.err` to **standard error** (a separate stream — a channel that data flows through — shown in red by many IDEs, useful for diagnostics that should not mix with data output).

`code/Formatting.java`:

```
System.out.print("print has no newline; ");
System.out.println("println adds one.");
System.out.printf("%-10s|%5d|%8.2f|%n", "Coffee", 3, 4.5);
System.out.printf("%-10s|%05d|%8.3e|%n", "Padded", 42, 12345.678);
System.out.printf("%s %b %c %x %o %%%n", "str", true, 'J', 255, 8);
System.out.printf(Locale.GERMANY, "German: %,.2f%n", 1234567.891);
System.out.printf(Locale.US, "US:     %,.2f%n", 1234567.891);
String line = String.format("Total: %.2f", qty * price);
String line2 = "Items: %d".formatted(qty);         // Java 15
System.err.println("(this line goes to standard error)");
```

Output:

```
print has no newline; println adds one.
Coffee    |    3|    4.50|
Padded    |00042|1.235e+04|
str true J ff 10 %
German: 1.234.567,89
US:     1,234,567.89
Total: 13.50 / Items: 3
(this line goes to standard error)
```

Specifier	Meaning	Flags/width examples
`%s`	any value via `toString()` (the method every object has that returns its text form)	`%-10s` left-align in 10 chars
`%d`	integer (`int`, `long`, `BigInteger`)	`%5d` width 5, `%05d` zero-pad, `%,d` grouping
`%f`, `%e`	decimal, scientific	`%.2f` two decimals (rounds half-up)
`%b`, `%c`	boolean, char	
`%x`, `%o`	hex, octal	`%08X`
`%n`	platform line separator (`\n` on Linux/macOS, `\r\n` on Windows)	prefer over `\n` in `printf`
`%%`	a literal `%`	

Formatting is **locale-sensitive**: the default locale (a language-and-region setting such as German or US English) decides the decimal and grouping separators, which is why tests and machine-readable output should pass an explicit `Locale` (e.g. `Locale.ROOT` or `Locale.US`). A mismatched type (`%d` with a `double`) throws `IllegalFormatConversionException` at run time.

Input: four options

Option	Best for	Notes
`Scanner` (`java.util`)	Beginner programs, parsing numbers and words	Tokenises input (splits it into words, called tokens) by whitespace; `nextInt`, `nextDouble`, `next`, `nextLine`, `hasNextInt`… Slow for very large input.
`IO.readln(prompt)` (`java.lang.IO`, **Java 25**)	Simplest line input	Returns a line as `String` (or `null` at end of input); pairs with `IO.println`.
`BufferedReader` over `InputStreamReader(System.in)`	Fast line-by-line reading (competitive programming, large input)	`readLine()` returns `null` at end; you parse numbers yourself; throws `IOException`.
`System.console()` → `java.io.Console`	Passwords (`readPassword` does not echo, i.e. show, the typed characters)	Returns `null` when there is no interactive terminal (IDE consoles, redirected input — input fed from a file with `<` instead of the keyboard) — always check.

Pick one reader per program. Each of them buffers `System.in` (reads ahead into its own memory); if a `Scanner` reads ahead, a `BufferedReader` created later will not see that input.

Deterministic `Scanner` demo reading from a `String` (`code/ScannerDemo.java`) — a real program would use `new Scanner(System.in)`:

```
String fakeKeyboard = """
        Ada Lovelace
        36
        1.5 2.5 3.0
        done
        """;
Scanner in = new Scanner(fakeKeyboard);
String fullName = in.nextLine();          // whole line
int age = in.nextInt();                   // one token -> int
in.nextLine();                            // consume the rest of the "36" line!
double sum = 0;
while (in.hasNextDouble()) sum += in.nextDouble();
String word = in.next();                  // next token

Scanner trap = new Scanner("42\nAlan Turing\n");
int n = trap.nextInt();
String afterInt = trap.nextLine();        // returns the EMPTY rest of the first line

Scanner bad = new Scanner("forty-two");
if (!bad.hasNextInt()) System.out.println("not an int: " + bad.next());
```

Output:

```
name=Ada Lovelace, age=36, sum=7.0, word=done
n=42, nextLine after nextInt=[], then=[Alan Turing]
not an int: forty-two
```

The `nextInt` + `nextLine` trap: `nextInt()` reads the number but leaves the line break; the following `nextLine()` returns the empty remainder of that line. Call an extra `nextLine()` after `nextInt()` (as above), or read every line with `nextLine()` and parse with `Integer.parseInt`. Calling `nextInt()` on non-numeric input throws `InputMismatchException` — guard with `hasNextInt()`.

The interactive version, `code/Interactive.java`, takes the reading style as an argument (`java -cp out ch02.io.Interactive scanner|io|reader|console`). Key part and a real run with the `console` mode in a terminal:

```java
Console console = System.console();          // null if not an interactive terminal
if (console == null) {
    System.out.println("No interactive console (IDE or redirected input).");
    return;
}
char[] secret = console.readPassword("Password (not echoed): ");
if (secret == null) return;                  // end of input (Ctrl+D)
System.out.println("Password length: " + secret.length);
Arrays.fill(secret, ' ');                    // wipe it from memory
```

```
Password (not echoed):
Password length: 7
```

With input redirected (`… console < file.txt`) it prints `No interactive console (IDE or redirected input).` Passwords are returned as `char[]`, not `String`, so you can wipe them (a `String` stays in memory until garbage-collected, i.e. freed automatically by the JVM at some later time). (In JDK 22–24 `System.console()` could return a `Console` even when redirected; JDK 25 returns to the older default, and JDK 26 made `null` the rule whenever input or output is redirected.)

`java.lang.IO` also works in ordinary classes (`code/IOClassDemo.java`):

```java
IO.print("IO.print, ");
IO.println("IO.println");
IO.println(42);
IO.println();
IO.println("done");
```

Output:

```
IO.print, IO.println
42

done
```

Command-line arguments (`String[] args`) are the other common input channel — see Chapter 4.

Self-check

1. After `int n = sc.nextInt();` the user's next line is skipped. Why and how to fix? → `nextInt` left the newline; call `sc.nextLine()` once, or read lines and parse.
2. Why is `readPassword` a `char[]`? → So it can be overwritten after use.
3. `System.console()` returns `null` in your IDE. Bug? → No; there is no interactive terminal. Handle `null`.

Chapter TL;DR

- Eight primitives: `byte short int long` (8/16/32/64-bit signed), `float double` (IEEE 754), `char` (16-bit unsigned UTF-16 unit), `boolean`. Everything else is a reference (or `null`).
- Locals must be assigned before use; fields get defaults. `final` = assign once (the object can still change). `var` (Java 10) infers local types; the type is still fixed.
- Literals: `L` for long, `f` for float, `0x`/`0b`/leading-`0` for hex/binary/octal, `_` separators.
- Widening is automatic (but `int/long → float` can lose precision); narrowing needs a cast and truncates/wraps. `byte/short/char` promote to `int` in arithmetic; compound assignment hides a cast.
- Integer `/` truncates; `%` follows the dividend's sign (`Math.floorMod` otherwise); integer `/0` throws, floating `/0` gives Infinity/NaN.
- `&&`/`||` short-circuit; `&`/`|` do not. `==` on references compares identity — use `equals` for content.
- Precedence: unary → `* / %` → `+ -` → shifts → relational → equality → `&` `^` `|` → `&&` → `||` → `?:` → assignment. Parenthesise when in doubt.

- Overflow wraps silently: use `long`, `Math.*Exact`, or `BigInteger`. Floating point is approximate: compare with tolerance; money in `BigDecimal`/cents.
- `String` is immutable and pooled; text blocks (Java 15) for multi-line literals with incidental-indentation stripping.
- Output with `println`/`printf` (locale-aware); input with `Scanner` (mind the `nextLine` trap), `IO.readln` (Java 25), `BufferedReader`, or `Console` for passwords — one reader per program.
- Exam-trap depth: OCP Chapter 1, OCP Chapter 2. Next: Chapter 3 — Control flow.

Chapter 3 — Control Flow

By default a program runs statements top to bottom, once each. **Control flow** statements change that: they *choose* between paths (`if`, `switch`), *repeat* work (`for`, `while`, `do`), and *jump* (`break`, `continue`, `return`). Modern Java (16–21) also added **pattern matching**, which lets one construct both test the shape of a value and pull data out of it — turning long `if`/cast (an explicit type conversion written `(Type) value`) chains into short, compiler-checked `switch` expressions.

What you'll learn

#	Topic	You will be able to…
3.1	`if` / `else`	write decision chains and avoid the dangling-else trap
3.2	The classic `switch` statement	use colon labels, `break`, fall-through and `default`
3.3	Arrow labels and `switch` expressions (Java 14)	return a value from a `switch`, use `yield`, rely on exhaustiveness
3.4	Choosing between `if`, ternary and `switch`	pick the clearest construct
3.5	Pattern matching for `instanceof` (Java 16)	test and bind in one step; understand flow scoping
3.6	Pattern matching for `switch` (Java 21)	type patterns, guards, `case null`, dominance, sealed exhaustiveness
3.7	Record patterns (Java 21) and unnamed patterns (Java 22)	deconstruct records, nest patterns, use `_`
3.8	Loops: `while`, `do-while`, `for`, enhanced `for`	choose and write the right loop
3.9	`break`, `continue` and labels	exit and skip iterations, including nested loops
3.10	Loop pitfalls	spot off-by-one, concurrent modification and other silent bugs

Code: `code/`, compiled with `javac --release 25`, output pasted from real runs. Records (compact classes whose job is to carry data) and sealed interfaces (an *interface* declares methods that classes promise to provide; a *sealed* one lists every type allowed to implement it) appear here only as data for patterns; they are taught properly in Chapter 6. For exam-level traps (which types a `switch` accepts, pattern-variable scope puzzles, loop-label trickery, unreachable code), see the OCP repo's Chapter 3 — Making Decisions.

3.1 `if`, `else if`, `else`

What it is

`if (condition) statement` runs the statement only when the **boolean** (true/false) condition is `true`; an optional `else` runs otherwise. Chaining `else if` tests several conditions in order and runs the **first** branch whose condition holds.

Analogy: a receptionist checking a list top to bottom — the first rule that matches decides where you go.

How it works

1. Evaluate the condition — it must have type `boolean` (or `Boolean`, its wrapper-object form). Unlike C or JavaScript, `if (x)` with an `int x` is a compile error: *incompatible types: int cannot be converted to boolean*.
2. If `true`, run the `if` body and skip every `else`.
3. Otherwise try the next `else if`; if none match, run the final `else` (if present).

`code/IfElse.java`:

```java
static String grade(int score) {
    if (score < 0 || score > 100) {
        return "invalid";
    } else if (score >= 90) {
        return "A";
    } else if (score >= 80) {
        return "B";
    } else if (score >= 70) {
        return "C";
    } else {
        return "F";
    }
}

// Dangling else: the else belongs to the NEAREST if, whatever the indentation says.
int x = 5, y = -1;
if (x > 0)
    if (y > 0)
        System.out.println("both positive");
else
    System.out.println("printed! this else pairs with 'if (y > 0)'");
```

Output:

```
95 -> A
85 -> B
72 -> C
40 -> F
101 -> invalid
Beach day
printed! this else pairs with 'if (y > 0)'
minor
```

Order matters: because the chain stops at the first match, test the narrow or invalid cases first (`score >= 90` before `score >= 80`).

Common mistakes

- **Dangling else** (above): Java ignores indentation; an `else` attaches to the closest unmatched `if`. Always use braces for nested `if`s.
- Omitting braces, then adding a second line that you think is inside the `if`:

  ```java
  if (loggedIn)
      showDashboard();
      audit();          // ALWAYS runs — only the first statement is controlled by the if
  ```

- `if (flag = true)` — assignment, not comparison; it compiles for `boolean` and is always true. Write `if (flag)`.
- Comparing strings with `==` in a condition (Chapter 2).
- Deep nesting. Prefer **guard clauses** — return early for invalid input so the main logic stays flat.

Self-check

1. Does `if (count)` compile for `int count`? → No; conditions must be boolean.
2. In an `if`/`else if` chain, can two branches run? → No, only the first matching one.
3. How do you prevent the dangling-else problem? → Braces on every nested `if`.

3.2 The classic `switch` statement

What it is

A `switch` compares one value (the **selector**) against constant **case labels** and jumps to the matching one. It replaces a long `if (x == 1) … else if (x == 2) …` chain when you are testing one variable for equality against constants.

How it works (colon form, the original since Java 1.0)

1. Evaluate the selector once.
2. Jump to the `case` whose constant equals it, or to `default` if none does (if there is no `default`, nothing runs).
3. Execute statements from there **downwards** until a `break` (or `return`/`throw`) or the end of the `switch`. Running into the next case's statements is called **fall-through**.

`code/SwitchStatement.java`:

```
int month = 4;
int days;
switch (month) {
    case 2:
        days = 28;
        break;
    case 4: case 6: case 9: case 11:        // several labels share one body
        days = 30;
        break;
    default:
        days = 31;
}

int level = 2;
switch (level) {
    case 1:
        System.out.println("  unlock level 1 perks");
    case 2:
        System.out.println("  unlock level 2 perks");
    case 3:
        System.out.println("  unlock level 3 perks");   // runs too: no break above
        break;
    case 4:
        System.out.println("  not reached");
}
```

Output:

```
month 4 has 30 days
stopping
Fall-through without break (level 2):
  unlock level 2 perks
  unlock level 3 perks
SAT is weekend
b is a consonant
```

What can you switch on?

Selector type	Since	Case labels
`char`, `byte`, `short`, `int` and their wrappers (object versions such as `Integer`)	Java 1.0 / 5	compile-time constants (values fixed when compiling) that fit the type (`'a'`, `42`, `final` constants)
`enum` (a type with a fixed set of named constants)	Java 5	constant names (`case SAT`); qualified names (`case Day.SAT`) also allowed since Java 21
`String`	Java 7	string literals; compared with `equals` (case-sensitive)
Any reference type (any non-primitive type) with patterns	Java 21	type/record patterns, `null` (3.6)
`long`, `float`, `double`, `boolean`	not allowed	(primitive patterns are only a *preview* — an opt-in trial feature; JEP 507, a numbered JDK Enhancement Proposal, in Java 25, still preview in 27)

Labels must be unique, and a `String`/boxed (wrapper-object) selector that is `null` throws (raises as an error — see Chapter 8) `NullPointerException` unless there is a `case null` (Java 21).

When fall-through is useful — and when not

Intentional fall-through (shared code for several labels, or cumulative steps like the perks example) is rare. Forgetting a `break` is a far more common bug, which is why the arrow form below exists. If you do fall through on purpose, comment it (`// fall through`).

In the interview

Q: What is fall-through in a `switch`? In colon-style `switch`, execution continues from the matched label into the following labels' statements until a `break`. It is a frequent source of bugs; arrow labels (`case X ->`) never fall through.

- *Can you switch on a `String`? How does it compare?* Yes (Java 7); with `equals`, after a `hashCode` jump (`hashCode` is a number computed from the string's characters, used to narrow down the candidate labels quickly).
- *What happens with a `null` selector?* `NullPointerException`, unless a `case null` label is present (Java 21+).

3.3 Arrow labels and `switch` expressions (Java 14)

What it is

Java 14 (JEP 361) finalised two related improvements:

- **Arrow labels** `case A, B -> …` — several constants per label, the right side is a single expression, block `{ }` or `throw`, and there is **no fall-through**. They can be used in `switch` *statements* too.
- **`switch` expressions** — a `switch` that **produces a value**, so you can write `int x = switch (…) { … };` or `return switch (…) { … };`.

Why it matters

Classic `switch` used to compute a value needs a pre-declared variable, an assignment in every case, a `break` in every case and a `default` to keep the compiler happy about definite assignment (the compiler's check that a variable surely has a value before it is read). Each of those is a chance for a bug. The expression form removes all of them, and the compiler checks that **every possible value is handled** (*exhaustiveness*).

How it works

1. The selector is evaluated; the matching label is chosen.

2. For `case … -> expression;` the expression's value becomes the switch's value.
3. For `case … -> { … }` the block must end by `yield value;` (or throw). `yield` means "this is the result of the switch" — it is not `return`, which would leave the whole method.
4. The switch expression must be **exhaustive**: for `int`/`String` selectors that means a `default`; for an `enum` it is enough to list every constant; for sealed types, every permitted subtype (a class or record that extends or implements the type; 3.6).

`code/SwitchExpression.java`:

```
enum Size { S, M, L, XL }

static int priceCents(Size size) {
    return switch (size) {                      // no default needed: all enum constants covered
        case S -> 250;
        case M -> 300;
        case L, XL -> 350;
    };
}

static String describe(int code) {
    return switch (code) {
        case 200, 201, 204 -> "success";
        case 301, 302 -> "redirect";
        case 404 -> "not found";
        default -> {
            String family = code >= 500 ? "server error" : "other";
            yield family + " (" + code + ")";    // yield returns a value from a block
        }
    };
}

// yield also works with old-style colon labels inside a switch expression
String kind = switch (letters % 2) {
    case 0:
        yield "even";
    default:
        yield "odd";
};
```

Output:

```
S=250 M=300 L=350 XL=350
200 -> success
302 -> redirect
404 -> not found
503 -> server error (503)
418 -> other (418)
WEDNESDAY has 9 letters
9 is odd
```

If one enum constant is missing, compilation fails:

```
B.java:4: error: the switch expression does not cover all possible input values
        return switch (s) {
               ^
```

That is the real benefit: add `XXL` to `Size` later and the compiler points to every `switch` that must handle it. (If an enum gains a constant and a `switch` compiled against the old version meets it at run time, the JVM throws `MatchException`.)

Statement vs expression, colon vs arrow

	Colon labels `case X:`	**Arrow labels** `case X ->`
Statement	Classic; fall-through; `break` needed	No fall-through; one expression/block/throw per label
Expression	Allowed; must `yield` a value; fall-through possible	The usual modern form
Exhaustive?	Statement: only if it uses patterns or `null` (Java 21 rule). Expression: **always**	same

You cannot mix colon and arrow labels in one `switch`.

Common mistakes

- Forgetting the semicolon after the closing brace of a switch *expression* used in an assignment (`};`).
- Using `return` inside a switch-expression block to produce the value (compile error) — use `yield`.
- Adding `default` to an enum switch expression "just in case": it silences the exhaustiveness check that would have caught a newly added constant. Leave it out when you cover all constants.

Self-check

1. What does `yield` do? → Supplies the value of a switch expression from a block.
2. Do arrow cases fall through? → Never.
3. Why may an enum switch expression omit `default`? → Listing all constants makes it exhaustive; the compiler verifies it.

3.4 Choosing between `if`, ternary and `switch`

Situation	**Best choice**	**Why**
One condition, two actions	`if`/`else`	Clearest
One condition, pick one of two **values**	ternary `c ? a : b`	An expression; short
Ranges or unrelated conditions (`age < 13`, `isVip && total > 100`)	`if`/`else if` chain (or guarded patterns)	`switch` labels are constants/patterns, not ranges
One variable compared to many constants	`switch` with arrow labels	Readable, no fall-through
Compute a value from many cases	`switch` expression	Exhaustive, no temporary variable
Branch on the **type/shape** of an object	pattern `switch` (Java 21)	Replaces `instanceof` chains; compiler-checked for sealed types
Behaviour that varies per subtype and grows over time	**Polymorphism** (a method on each class; each class supplies its own version)	The OOP (object-oriented programming) alternative; see Chapter 6 and the LLD (low-level design) handbook

The last two rows are a genuine design choice. A pattern `switch` over a *sealed* hierarchy keeps an operation in one place and makes adding an **operation** easy; virtual methods (methods whose version is picked at run time from the object's actual class) keep each type's code together and make adding a **type** easy. This is known as the *expression problem*.

3.5 Pattern matching for `instanceof` (Java 16)

What it is

A **pattern** combines a *test* ("is this a `String`?") with *binding* ("…and if so, call it `s`"). `o instanceof String s` is a **type pattern** (JEP 394, final in Java 16): when the test succeeds, the **pattern variable** `s` is already cast and ready to use.

The problem it solves

The old idiom repeats the type three times and allows mistakes (casting to the wrong type, forgetting the check):

```
if (o instanceof String) {
    String s = (String) o;
    ...
}
```

How it works — flow scoping

A pattern variable is in scope (usable by name) **only where the compiler can prove the match succeeded**:

1. Inside the `if` body when the condition is `o instanceof T t`.
2. On the right-hand side of `&&` (`o instanceof String s && !s.isEmpty()`), because `&&` only evaluates its right side when the left is true.
3. **After** the `if` when the `if` tests the *negation* and exits (`if (!(o instanceof String s)) return 0; … s.length()`).
4. **Not** with `||` — `o instanceof String s || s.isEmpty()` fails with *cannot find symbol*, because the right side runs precisely when the match failed.

`code/InstanceofPattern.java`:

```
static String newStyle(Object o) {
    if (o instanceof String s && !s.isEmpty()) {     // s is in scope on the right of &&
        return "String of length " + s.length();
    }
    if (o instanceof Integer i) return "Integer, doubled " + (i * 2);
    return "something else: " + o;
}

static int lengthOrZero(Object o) {
    if (!(o instanceof String s)) {
        return 0;                                    // leave early when it is NOT a String
    }
    return s.length();                               // flow scoping: s is definitely a String here
}

record Money(long cents, String currency) {
    @Override public boolean equals(Object other) {  // a classic use: equals()
        return other instanceof Money m && cents == m.cents && currency.equals(m.currency);
    }
}
```

Output:

```
String of length 5
String of length 5 | Integer, doubled 42 | something else:  | something else: null
lengthOrZero: 3, 0
true
```

Notes:

- `null instanceof X` is always `false`, so a pattern never binds `null`.
- The pattern variable is an ordinary local; it is not `final` unless you declare it `final String s`.

- An *unconditional* pattern — testing a `String` variable with `instanceof String s`, which can only fail for `null` — was a compile error in Java 16–20 and is allowed since Java 21.

Real-world examples

`equals(Object)` implementations (as above), handling heterogeneous (mixed-type) JSON (a text format for structured data) nodes (`if (node instanceof ObjectNode obj)`), and exception (error-object, Chapter 8) handling code that inspects causes (`if (e.getCause() instanceof SQLException sql)`).

3.6 Pattern matching for `switch` (Java 21)

What it is

Java 21 (JEP 441) lets `case` labels contain **patterns**, not just constants. A single `switch` can test the type of any object, bind variables, apply extra conditions and handle `null`.

How it works

1. **Type pattern** — `case Integer i ->` matches when the selector is an `Integer` and binds `i`.
2. **Guard** — `case Integer i when i < 0 ->` adds a boolean condition (`when` is a contextual keyword — a keyword only in this position, still usable as a name elsewhere). The case only matches if the pattern matches **and** the guard is true.
3. **`case null`** — without it, a `null` selector throws `NullPointerException` (backward compatible — behaves exactly as old `switch` code expects). `case null, default ->` combines both.
4. **Order and dominance** — cases are tried top to bottom. A label is a compile error if an earlier label already matches everything it would (e.g. `CharSequence cs` before `String s`), so write **specific cases before general ones**, and guarded cases before the unguarded case of the same type.
5. **Exhaustiveness** — a pattern `switch` (expression *or* statement) must cover every possible value: add `default`, or cover all permitted subtypes of a **sealed** type.

`code/SwitchPatterns.java`:

```
sealed interface Shape permits Circle, Square, Rectangle {}
record Circle(double radius) implements Shape {}
record Square(double side) implements Shape {}
record Rectangle(double width, double height) implements Shape {}

static double area(Shape shape) {
    return switch (shape) {                          // exhaustive: every permitted subtype covered
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Square s -> s.side() * s.side();
        case Rectangle r -> r.width() * r.height();
    };
}

static String classify(Object o) {
    return switch (o) {
        case null -> "null!";                        // without this case, switch throws NPE
        case Integer i when i < 0 -> "negative int " + i;   // guard
        case Integer i -> "int " + i;
        case String s when s.isBlank() -> "blank string";
        case String s -> "string of " + s.length();
        case int[] arr -> "int array of " + arr.length;
        case Shape sh -> "shape with area " + String.format("%.2f", area(sh));
        default -> "other: " + o.getClass().getSimpleName();
    };
}
```

Output:

```
negative int -7
int 42
blank string
string of 5
int array of 3
shape with area 3.14
shape with area 6.00
other: Double
null!
```

A **sealed** interface (Java 17) lists all its implementations with `permits`, so the compiler knows `area`'s three cases are complete — no `default`. Add a `Triangle` to `permits` and every such `switch` fails to compile until it handles triangles.

Real compiler errors you will meet:

```
A.java:12: error: this case label is dominated by a preceding case label
            case String s -> "string";
                 ^
```

```
B.java:10: error: the switch statement does not cover all possible input values
        switch (o) {
        ^
```

Trade-offs

Approach	Pros	Cons
`if`/`instanceof` chain	Works on any Java version ≥ 16	Easy to forget a type; order-sensitive; verbose
Pattern `switch` + sealed types	Exhaustive, compact, a value-producing expression	Every `switch` must change when a subtype is added (which is the point)
Polymorphic method	Adding a type touches one place	Adding an operation touches every class

When not to use it

For a genuinely open hierarchy (plug-ins, user-defined subclasses — classes that extend another class) you cannot list the cases; a `default` branch or polymorphism fits better. And for equality against constants, the simple constant `switch` is still clearer.

In the interview

Q: What does pattern matching for `switch` give you over `instanceof` chains? Type test, cast and binding in one label; guards with `when`; explicit `null` handling; and compile-time exhaustiveness and dominance checks — especially powerful with sealed hierarchies and records, where adding a subtype makes the compiler flag every switch that must be updated.

- *What is a guard?* A `when` boolean condition attached to a pattern label.
- *Why does label order matter?* The first matching label wins, and a label dominated by an earlier one is a compile error.
- *Does a pattern switch accept `null`?* Only with a `case null`; otherwise it throws `NullPointerException`.

Self-check

1. Where must `case String s when s.isEmpty()` go relative to `case String s`? → Before it.
2. Why does `switch (shape)` above need no `default`? → `Shape` is sealed; all permitted subtypes are covered.
3. Is a pattern `switch` *statement* required to be exhaustive? → Yes (Java 21 rule for switches that use patterns or `null`).

3.7 Record patterns (Java 21) and unnamed patterns (Java 22)

What it is

A **record** is a compact class whose purpose is to carry data (Java 16; Chapter 6): `record Point(int x, int y) {}` gets a constructor (the method that builds a new object), accessors `x()` / `y()` (methods that return each stored value), `equals`, `hashCode` and `toString` (the standard methods for comparing, hashing and printing an object) automatically. A **record pattern** (JEP 440, Java 21) matches a record **and deconstructs it** into its components (the values it carries) in one step: `o instanceof Point(int x, int y)`.

The **unnamed pattern** `_` (JEP 456, Java 22) matches anything and binds nothing — use it for components you do not care about. `_` can also name unused variables (`catch (Exception _)`, `for (var _ : list)`, lambda parameters `(_, v) -> v` — a lambda is a short inline function, Chapter 11).

How it works

1. The value is tested against the record type (`null` never matches).
2. Each component is extracted through its accessor and matched against the nested pattern: a type (`int x`), `var x` (infer the type), another record pattern (nesting), or `_`.
3. Record patterns can be used in `instanceof` and in `switch`, with guards.

`code/RecordPatterns.java` — a tiny expression evaluator, a textbook use of sealed records plus patterns:

```
record Point(int x, int y) {}
sealed interface Expr permits Num, Add, Mul, Neg {}
record Num(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
record Mul(Expr left, Expr right) implements Expr {}
record Neg(Expr operand) implements Expr {}

static int eval(Expr e) {
    return switch (e) {
        case Num(int v) -> v;
        case Add(Expr l, Expr r) -> eval(l) + eval(r);
        case Mul(Num(var v), _) when v == 0 -> 0;          // nested pattern + unnamed _ + guard
        case Mul(Expr l, Expr r) -> eval(l) * eval(r);
        case Neg(var inner) -> -eval(inner);
    };
}

Object o = new Point(3, 4);
if (o instanceof Point(int x, int y)) {             // deconstruct in one step
    System.out.println("x=" + x + " y=" + y + " distance=" + Math.sqrt(x * x + y * y));
}
if (o instanceof Point(var x, _)) System.out.println("only care about x: " + x);
```

Output:

```
x=3 y=4 distance=5.0
only care about x: 3
(2 + 3 * -4) = -10
0 * (1 + 2) = 0
```

The `Mul(Num(var v), _) when v == 0` case short-circuits multiplication by zero without evaluating the right side — a nested pattern reads like the data's shape. This style (sealed interface + records + exhaustive pattern switch) is how modern Java models **algebraic data types** (a type defined as a fixed set of alternative shapes, each carrying its own data): ASTs (abstract syntax trees — the tree a compiler builds from source code) in compilers, JSON values, commands/events, and results such as `Success | Failure`.

Still in preview (don't rely on it yet)

Primitive types in patterns, `instanceof` **and** `switch` — e.g. `switch` on a `long` or `case int i` matching a `double` that fits — is a preview feature (JEP 507 in Java 25; still preview in 26 and 27). Without `--`

`enable-preview`, switching on a `long` fails: *primitive patterns are a preview feature and are disabled by default*.

Self-check

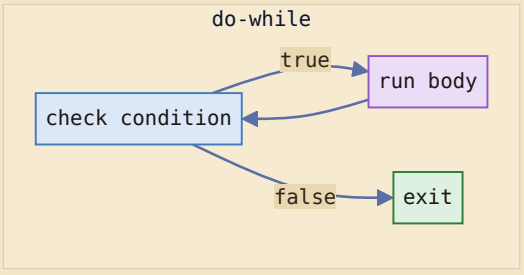
1. What does `case Add(var l, var r)` bind? → The two components of an `Add` record, with inferred types.
2. What does `_` mean in `Point(var x, _)`? → Match any `y` and ignore it (Java 22).
3. Does `null instanceof Point(int x, int y)` match? → No.

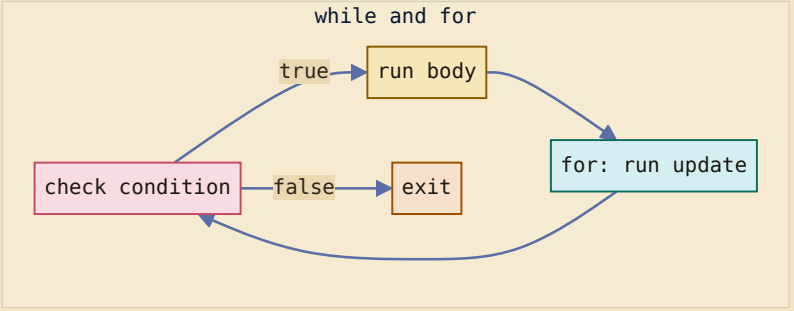
3.8 Loops

What it is

A **loop** repeats a statement (the *body*) while a condition holds, or once per element of a collection (a group of objects such as a list or set). Each run of the body is an **iteration**.

Loop	Checks condition	Runs at least once?	Typical use
`while (cond) body`	before each iteration	no	unknown number of repetitions: reading until end of input, waiting for a state
`do body while (cond);`	after each iteration	**yes**	menus, retry-until-success, input validation
`for (init; cond; update) body`	before each iteration	no	counting, index-based access
`for (T x : iterable) body` (enhanced for, Java 5)	—	no	visit every element of an array or `Iterable` (anything that can be looped over)





`code/Loops.java`:

```java
int n = 9075, digitSum = 0;
while (n > 0) {                       // condition checked BEFORE each iteration
    digitSum += n % 10;
    n /= 10;
}

int attempts = 0;
int[] fakeRolls = {2, 5, 6, 3};
int roll;
do {                                  // body runs at least once; condition checked AFTER
    roll = fakeRolls[attempts++];
    System.out.print("rolled " + roll + "; ");
} while (roll != 6);

long factorial = 1;
for (int i = 1; i <= 10; i++) {       // init; condition; update
    factorial *= i;
}

for (int i = 0, j = 10; i < j; i += 3, j -= 3) {   // several variables of one type
    System.out.print("(" + i + "," + j + ") ");
}
for (int i = 10; i > 0; i /= 2) System.out.print(i + " ");   // any update expression

int[] temps = {21, 25, 19, 30};
int max = Integer.MIN_VALUE;
for (int t : temps) max = Math.max(max, t);   // enhanced for over an array

Map<String, Integer> stock = new TreeMap<>(Map.of("apple", 3, "pear", 0, "fig", 7));
for (Map.Entry<String, Integer> e : stock.entrySet()) {
    System.out.print(e.getKey() + ":" + e.getValue() + " ");
}
```

Output:

```
digit sum of 9075 = 21
rolled 2; rolled 5; rolled 6; got a 6 after 3 attempts
10! = 3628800
(0,10) (3,7)
10 5 2 1
max temp = 30
ADA GRACE LINUS
apple:3 fig:7 pear:0
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

The `for` loop in detail

1. **Initialisation** runs once. It may declare several variables *of the same type* (`int i = 0, j = 10`); they are scoped to the loop.
2. **Condition** is checked before every iteration; if omitted, it is `true` (`for (;;)` is an infinite loop).
3. **Body** runs.
4. **Update** runs after each body (several expressions separated by commas are allowed), then back to step 2.

The enhanced `for` in detail

`for (T x : source)` works for **arrays** and anything that implements `Iterable` (all collections). The compiler turns it into an index loop for arrays and an `Iterator` (an object that walks a collection one element at a time with `hasNext()`/`next()`) loop for collections. Use it whenever you need each element and not its index. Its limits:

- No index (use a classic `for`, or `IntStream.range` — Chapter 12).
- The loop variable is a **copy** for primitives: assigning to it does not change the array.
- You must not add/remove elements of the collection being iterated (3.10).

- Iterating a `Map` (a key→value lookup table) directly is impossible — iterate `entrySet()`, `keySet()` or `values()`.

Loops vs streams

Many loops that filter, transform and aggregate (combine into one result, such as a sum or maximum) can be written as streams (pipelines of data-processing steps) (Chapter 12): `Arrays.stream(temps).max()`. Loops remain the right tool when you need early exits with complex state, index arithmetic, or maximum performance on primitive arrays; streams read better for pipelines of transformations.

In the interview

Q: `while` vs `do-while`? `while` tests before the first iteration and may run zero times; `do-while` tests after, so it runs at least once — useful for "prompt until valid" loops.

- *When do you prefer the enhanced `for`?* Whenever you only need the elements, not indexes, and are not modifying the collection's structure.
- *How is the enhanced `for` compiled for a `List`?* Into an `Iterator` loop (`hasNext`/`next`).

Self-check

1. How many times does `for (int i = 0; i < 10; i += 3)` run? → 4 (0, 3, 6, 9).
2. Can `for (int i = 0, long j = 0; …)` compile? → No; one declaration type only.
3. Which loop suits "ask for a password until it is valid"? → `do-while`.

3.9 `break`, `continue` and labels

What it is

- `break` exits the innermost loop (or `switch`) immediately.
- `continue` skips the rest of the current iteration and proceeds with the next one (in a `for`, the update still runs).
- A **label** (`name:` before a loop) lets `break name;` or `continue name;` target an **outer** loop. `return` exits the whole method, which is often the cleanest way out of nested loops.

`code/Branching.java`:

```java
for (int v : data) {
    if (v < 0) {
        System.out.println("found a negative, stopping");
        break;                                  // leave the loop entirely
    }
    System.out.println("processing " + v);
}

for (int i = 1; i <= 10; i++) {
    if (i % 3 != 0) continue;                   // skip the rest of this iteration
    System.out.print(i + " ");
}

search:                                         // a label names the outer loop
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        if (grid[r][c] == target) {
            foundRow = r;
            foundCol = c;
            break search;                       // exits BOTH loops
        }
    }
}

outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == i) continue outer;             // jump to the next i
        System.out.print(i + "" + j + " ");
    }
}

while (true) {                                   // loop-and-a-half: exit in the middle
    String line = lines[idx++];
    if (line.isEmpty()) break;
    System.out.print(line + " ");
}
```

Output:

```
processing 4
processing 8
found a negative, stopping
3 6 9 <- multiples of 3
42 found at (1,1)
21 31 32 <- pairs with j < i
alpha beta <- read until blank line
```

Trade-offs

Technique	Good for	Watch out
`break` / `continue`	Early exit, skipping invalid items	Many of them in one loop hide the control flow
Labelled `break`	Leaving nested loops (grid search)	Rarely needed; often clearer to extract a method and `return`
`while (true)` + `break`	"Loop-and-a-half": read, test, process	Make sure every path eventually breaks
`return`	Leaving everything once the answer is found	—

Java has no `goto` (the word is reserved but unused); labels only work with `break`/`continue` on enclosing statements. Code directly after an unconditional `break`/`continue`/`return` in the same block is **unreachable** and a compile error.

Self-check

1. In a `for` loop, does `continue` skip the update expression? → No, the update still runs.
2. How do you leave two nested loops at once? → A labelled `break`, or put the loops in a method and `return`.
3. What does `break` inside a `switch` inside a loop exit? → Only the `switch`.

3.10 Loop pitfalls

`code/LoopPitfalls.java` — each compiles fine:

```
int[] a = {10, 20, 30};
for (int i = 0; i <= a.length; i++) System.out.print(a[i] + " ");   // off-by-one: <= should be <

List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
for (Integer x : nums) if (x % 2 == 0) nums.remove(x);   // structural change during for-each

for (Iterator<Integer> it = nums.iterator(); it.hasNext(); ) {
    if (it.next() % 2 == 0) it.remove();                      // fix 1: Iterator.remove
}
more.removeIf(x -> x % 2 == 0);                               // fix 2: removeIf (Java 8)

for (double d = 0.0; d != 1.0 && count < 20; d += 0.1) count++; // d never equals 1.0 exactly

for (int v : values) v *= 10;                 // changes the copy, not the array

for (int i = 0; i < 5; i++); { total++; }     // stray ';' = empty loop body
```

Output:

```
10 20 30
off-by-one -> Index 3 out of bounds for length 3
remove in for-each -> ConcurrentModificationException
after Iterator.remove: [1, 3, 5]
after removeIf: [1, 3, 5]
double counter ran 20 times (guard stopped it)
values after 'v *= 10' in for-each: [1, 2, 3]
stray semicolon: block ran 1 time(s)
```

Pitfall	Why it happens	Fix
Off-by-one (`<=` with `length`)	Indexes run 0 … length−1	`i < a.length`; prefer enhanced `for`
`ConcurrentModificationException`	Collections like `ArrayList` are *fail-fast*: their iterator detects structural changes (elements added or removed) made behind its back (Chapter 10). Detection is best-effort — removing the second-to-last element can even end the loop silently without the exception, leaving a wrong result.	`Iterator.remove()`, `removeIf`, or build a new list
Floating-point loop counter	0.1 is inexact, so `d` never equals 1.0 (Chapter 2)	Loop over an `int` and compute `i * 0.1`
Assigning the for-each variable	It is a copy of the element	Index loop: `values[i] *= 10`
Stray semicolon after `for (…)` / `while (…)`	`;` is an empty statement that becomes the body	Always use braces; IDEs warn
Infinite loop	Condition never becomes false (forgot `i++`, or `while (x != 10)` when `x` jumps from 9 to 11)	Update the loop variable; prefer `<` / `>` over `!=`
Integer overflow of the counter	`for (int i = 0; i <= Integer.MAX_VALUE; i++)` never ends — `i` wraps negative	Use `<` or a `long`
Expensive condition	`for (int i = 0; i < list.size(); i++)` on a `LinkedList` (a chain of linked nodes, slow to index) with `get(i)` is O(n²) (Big-O notation: time grows with the square of the list size)	Enhanced `for`/iterator

Self-check

1. Why can removing inside a for-each fail? → The fail-fast iterator sees the list changed structurally and throws `ConcurrentModificationException`.
2. Two safe ways to remove matching elements while iterating? → `Iterator.remove()` or `Collection.removeIf`.
3. What does `for (int i = 0; i < 3; i++);` do? → Loops three times over an empty statement.

Chapter TL;DR

- `if` / `else if` / `else`: first true branch wins; conditions must be `boolean`; use braces to avoid the dangling-else and "second statement" traps; prefer guard clauses to deep nesting.
- Classic `switch` (colon labels) falls through until `break`; works on `int`-like types, wrappers, `enum`, `String` — never `long` / `float` / `double` / `boolean` (primitive patterns are still preview).
- Arrow labels (Java 14) never fall through; **switch expressions** return a value, use `yield` inside blocks, and must be exhaustive (enum constants listed or `default`).
- `instanceof` patterns (Java 16) test + cast + bind; the variable is in scope only where the match is certain (`&&`, negated early-exit), not with `||`.
- Pattern `switch` (Java 21): type patterns, `when` guards, `case null`, dominance (specific before general) and exhaustiveness — with sealed types no `default` is needed and adding a subtype breaks every incomplete switch at compile time.
- Record patterns (Java 21) deconstruct records, nest, and use `var`; `_` (Java 22) ignores parts. Sealed interface + records + pattern switch = algebraic data types in Java.
- Loops: `while` (0+ times), `do-while` (1+ times), `for` (counting), enhanced `for` (every element of an array/`Iterable`, no index, no structural changes).
- `break` exits, `continue` skips, labels target outer loops; often a method + `return` is cleaner.

- Classic bugs: off-by-one, `ConcurrentModificationException`, floating counters, assigning the for-each variable, stray `;`.
- Exam-trap depth: OCP Chapter 3. Next: Chapter 4 — Methods and arrays.

Chapter 4 — Methods and Arrays

So far every program has been one long `main`. Real programs are built from small, named pieces of behaviour — **methods** — that take inputs, do one job and hand back a result. Real programs also work with many values at once, and the most basic container (an object that holds other values) Java offers for that is the **array**. This chapter covers both, plus the two places they meet: passing arrays to methods, and the `String[] args` array that every classic `main` method receives.

What you'll learn

1. How to declare and call a method: modifiers, return type, parameters, `return`, `void`, static vs instance methods.
2. Why Java is *always* pass-by-value — and why that still lets a method change your array or list.
3. Overloading: several methods with one name, and exactly how the compiler picks one.
4. Varargs (`int... xs`): methods that accept any number of arguments.
5. Recursion: the call stack, base cases, `StackOverflowError`, memoization.
6. Arrays: creating, default values, indexing, looping, copying, covariance.
7. Two-dimensional and jagged arrays.
8. The `java.util.Arrays` toolbox: sort, search, fill, copy, compare, `asList`.
9. `main`, command-line arguments, and the Java 25 simpler `main` forms.

All runnable code is in `code/` (package `ch04.*`). Every output shown was produced by compiling with `javac --release 25` and running on JDK 26.

> **Exam depth.** For OCP-style traps on method declarations, access modifiers, varargs, overloading and pass-by-value, see the OCP repo: Chapter 5 — Methods. This chapter explains the concepts; that one drills the tricky questions.

4.1 Methods — declaring and calling

What it is

A **method** is a named block of code that belongs to a class (a blueprint that groups data and the code working on it; Chapter 5). You give it zero or more inputs (**parameters**), it runs its statements, and it may hand back one value (the **return value**). Other languages call this a *function*; in Java every function lives inside a class, so it is called a method.

Analogy: a method is a recipe card. The card's title is the method name, the ingredient list is the parameter list, the steps are the body, and the finished dish is the return value. Writing the card (declaring) does nothing by itself; cooking from it (calling) is what runs it.

Why it matters

- **Reuse** — write the logic once, call it from many places.
- **Naming** — `isEven(n)` explains itself; `n % 2 == 0` scattered around does not.
- **Testing** — a small method with clear inputs and outputs is easy to test, i.e. to check automatically with test code (Chapter 18).
- **Change** — fix a bug in one place, not twenty.

How it works — anatomy of a declaration

```
public static double area(double width, double height) throws IllegalArgumentException {
//  ^       ^     ^      ^    ^------------------------^   ^------------------------^
//  |       |     |      |    parameter list                optional throws clause
//  |       |     |      method name
//  |       |     return type (void = nothing)
//  |       optional specifier(s): static, final, abstract, synchronized ...
//  access modifier (public / protected / private / none)
    return width * height;       // method body
}
```

Part	Required?	Notes
Access modifier	No	`public`, `protected`, `private`, or none (= package-private: visible only inside the same package). Explained in Chapter 5.
Optional specifiers	No	`static` (belongs to the class), `final` (cannot be overridden), `abstract` (no body; subclasses supply one), `synchronized` (one thread at a time), `native` (written in C or similar), `strictfp` (the last has had no effect since Java 17).
Return type	**Yes**	Any type, or `void`. Constructors (special methods that set up a new object) are the only "methods" without one.
Name	**Yes**	By convention a verb in lowerCamelCase: `calculateTotal`, `isEmpty`, `toString`.
Parameter list	**Yes** (parentheses)	Can be empty `()`. Each parameter has a type and a name.
`throws` clause	No	Lists checked exceptions (error types the compiler forces callers to handle; an *exception* is an error object signalled with `throw`) the method may throw (Chapter 8).
Body	Yes, unless `abstract`/`native`	The code in `{ }`.

The **method signature** is the *name plus the parameter types in order* — for example `area(double, double)`. The return type, parameter names and modifiers are **not** part of the signature. That matters for overloading (§4.3): two methods in one class cannot have the same signature.

Steps when a method is called:

1. The caller evaluates each argument expression, left to right.
2. The JVM (Java Virtual Machine — the program that runs your compiled code) creates a new **stack frame** — a private workspace holding the method's parameters and local variables.
3. Each argument value is **copied** into the matching parameter.
4. The body runs until a `return` statement or the closing brace.
5. The frame is discarded; the return value (if any) replaces the call expression in the caller.

Parameters vs arguments. A *parameter* is the variable in the declaration (`width`); an *argument* is the actual value passed at the call site (the place in the code where the method is called; here `3`).

`return` and `void`

- A method with a non-`void` return type must `return` a value of a compatible type on **every** path; the compiler checks this ("missing return statement").
- A `void` method returns nothing. It may use a bare `return;` to leave early.
- The caller may ignore a return value — `area(1, 1);` is legal; the result is thrown away.
- A method returns exactly one value. To return several, return an object (a record — a compact data-carrying class, Chapter 6) or, for small same-typed results, an array.

Static vs instance methods

- A `static` **method** belongs to the class. Call it as `ClassName.method()` (or just `method()` from inside the same class). It has no `this` object (`this` means "the current object"), so it cannot touch instance fields (variables stored inside each object) directly.
- An **instance method** (no `static`) runs *on an object* and can read and write that object's fields: `counter.countCall()`.

Chapter 5 explains `static` fully (§5.5). For now: helper utilities (`Math.max`, `Integer.parseInt`) are static; behaviour of a particular object (`list.add`, `account.deposit`) is instance.

```
// code/ch04/methods/MethodBasics.java (key parts)
static double area(double width, double height) {
    return width * height;
}
static void greet(String name) {
    if (name == null || name.isBlank()) {
        System.out.println("Hello, stranger");
        return;                          // early exit from a void method
    }
    System.out.println("Hello, " + name);
}
static boolean isEven(int n) { return n % 2 == 0; }
static int max(int a, int b) { return a >= b ? a : b; }

private int calls;                       // instance field
int countCall() { return ++calls; }      // instance method

public static void main(String[] args) {
    double a = area(3, 4.5);             // int argument 3 is widened to double
    System.out.println("area = " + a);
    greet("Ada");
    greet("  ");
    System.out.println("isEven(7) = " + isEven(7) + ", max(3, 9) = " + max(3, 9));
    area(1, 1);                          // result ignored: legal
    MethodBasics counter = new MethodBasics();
    counter.countCall();
    System.out.println("calls = " + counter.countCall());
}
```

Output:

```
area = 13.5
Hello, Ada
Hello, stranger
isEven(7) = false, max(3, 9) = 9
calls = 2
```

Local variables and scope

- Variables declared inside a method (including parameters) are **local**: they exist only while the method runs and are visible only inside the block `{ }` where they are declared.
- Locals have **no default value**. Reading one before assigning it is a compile error ("variable might not have been initialized"). Fields and array elements, by contrast, do get defaults.
- Each call gets fresh copies of its locals — which is what makes recursion (§4.5) work.
- A parameter can be marked `final` to stop the body reassigning it: `void log(final String msg)`. It is optional documentation, not a performance trick.

Common mistakes

- **Forgetting that the result must be used.** `name.toUpperCase();` alone does nothing useful — strings are immutable (cannot be changed once created) and the new string is discarded. Write `name = name.toUpperCase();`.
- **Calling an instance method from** `static main` without an object: "non-static method cannot be referenced from a static context". Create an object first, or make the helper `static`.

- **Methods that do too much.** If you need "and" to describe it (`validateAndSaveAndEmail`), split it. See clean-code guidance in the LLD handbook: Clean code.
- **Boolean flag parameters** (`print(report, true, false)`) make call sites unreadable; prefer two well-named methods or an enum (a type with a fixed set of named constants).

Real-world examples

`Math.max(a, b)`, `Integer.parseInt(s)`, `String.valueOf(x)` are static utilities; `list.size()`, `map.get(k)`, `builder.append(x)` are instance methods. Frameworks (large libraries that call *your* code, rather than the other way round) like Spring and JUnit discover your methods by name and annotations (`@`-markers such as `@Test` attached to code) and call them for you.

Self-check

1. Which parts make up a method signature? → The name and the ordered list of parameter types. Not the return type, not parameter names, not modifiers.
2. Can a `void` method contain `return`? → Yes, a bare `return;` to exit early. It cannot return a value.
3. Why does `static void main` fail to call `countCall()` directly? → `countCall` is an instance method; `main` has no `this` object to call it on.
4. Do local variables get default values? → No. Only fields and array elements do.

TL;DR

A method = modifiers + return type + name + parameters + body. Signature = name + parameter types. Non-void methods must return on every path. Static methods belong to the class; instance methods run on an object. Locals live in the method's stack frame and must be initialised before use.

4.2 Parameters — Java is always pass-by-value

What it is

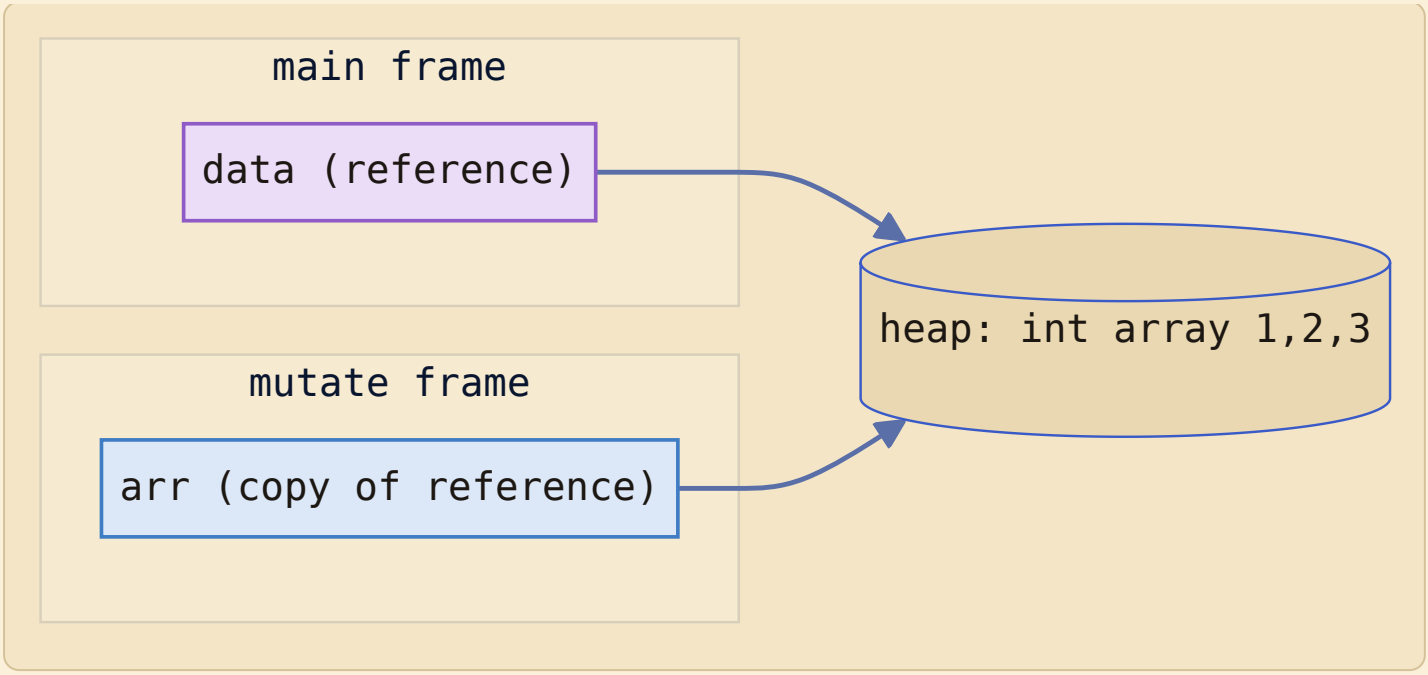
When you call a method, Java **copies the value of each argument into the parameter**. That is *pass-by-value*. There is no pass-by-reference in Java — ever. The confusion comes from what the "value" is:

Variable type	What the variable holds	What gets copied
Primitive (`int`, `double`, `boolean` …)	The number/boolean itself	The number — the method gets its own copy
Reference (`int[]`, `String`, `List`, any object)	A **reference** (think: the address of an object on the heap, the memory area where objects live)	The reference — the method gets a copy of the *address*, which still points at the *same object*

Analogy: you give a friend a photocopy of your house address. They can go to your house and repaint the door (mutate the object) and you will see it. But if they scribble a different address on their photocopy (reassign the parameter), your own copy still says your address.

How it works

1. Primitive argument → the parameter is an independent copy. Changes inside the method are invisible to the caller.
2. Reference argument → the parameter is a copy of the reference.
    - **Mutating** the object through it (`arr[0] = 99`, `list.add(x)`) is visible to the caller, because there is only one object.
    - **Reassigning** the parameter (`arr = new int[]{…}`) only changes the method's local copy. The caller's variable still points to the original object.




```
// code/ch04/methods/PassByValue.java (key parts)
static void addOne(int x) { x = x + 1; }                  // changes the local copy only
static void reassign(int[] arr) { arr = new int[] {99}; } // re-points the local copy only
static void mutate(int[] arr) { arr[0] = 99; }            // same array as the caller's
static void appendTo(List<String> list) { list.add("added"); }
static void swap(String a, String b) { String t = a; a = b; b = t; }  // cannot work in Java
```

Output:

```
n after addOne:      5
data after reassign: [1, 2, 3]
data after mutate:   [99, 2, 3]
names after append:  [a, added]
after swap:          left right
```

The classic C-style `swap(a, b)` is impossible in Java: the method only swaps its own copies. To "swap", swap elements *inside* an array or object, or return the result.

Why it matters

- It explains why a method can fill in or sort your array (`Arrays.sort(a)` sorts your array in place) but can never make your variable point to a different array.
- It is the root of **defensive copying**: if a method stores a list you passed in, later changes you make to that list leak into its state. Chapter 5 shows how to prevent that (§5.9).
- Immutable types (`String`, `Integer`, records with immutable fields) are safe to pass around because nobody *can* mutate them.

Common mistakes

- Saying "objects are passed by reference". They are not; *references are passed by value*. Interviewers probe exactly this difference with the swap example.
- Expecting `s = s.trim()` inside a method to change the caller's string. Return the new value instead.
- Returning `null` to signal "no results" from a method that returns an array or list; return an empty one (`new int[0]`, `List.of()`) so callers can loop safely (*Effective Java* — Joshua Bloch's widely read book of Java best practices — Item 54).

In the interview

30-second answer: "Java is strictly pass-by-value. For primitives the value is the number. For objects the value is a reference, so the method gets a copy of the reference: it can mutate the

shared object, but reassigning the parameter never affects the caller's variable. That's why a swap method can't work in Java."

- *Follow-up: So is `String` special?* → No. It behaves the same way; it only *looks* different because `String` is immutable, so there is no way to mutate it through the copied reference.
- *Follow-up: How do you return two values?* → Return a small record (`record MinMax(int min, int max)`) — clearest; or an array for same-typed values.

Self-check

1. After `void f(int[] a) { a = null; }` and `f(arr)`, is `arr` null? → No. Only the local copy was set to null.
2. After `void g(List<Integer> l) { l.clear(); }`, is the caller's list empty? → Yes, the shared object was mutated.
3. Can a method change which object the caller's variable refers to? → No, never.

TL;DR

Everything is passed by value. For references, the value is the reference: mutation through it is shared, reassignment is local.

4.3 Overloading

What it is

Overloading means declaring several methods with the **same name but different parameter lists** in the same class (or inherited into it — received from a parent class). The compiler picks which one to call at **compile time** from the argument types. This is sometimes called *compile-time* or *static polymorphism* (contrast with overriding — a subclass replacing an inherited method — in Chapter 6).

Examples you already use: `System.out.println` has ten overloads (`println(int)`, `println(String)`, `println(Object)`, …); `Math.max` has `int`, `long`, `float` and `double` versions.

Rules

Legal overload if the parameter lists differ in…	Not an overload (compile error "already defined")
Number of parameters: `sum(int,int)` vs `sum(int,int,int)`	Only the return type differs
Types: `sum(int,int)` vs `sum(double,double)`	Only parameter *names* differ
Order of types: `f(int,String)` vs `f(String,int)`	Only modifiers / `throws` differ
	`f(int[])` vs `f(int...)` — the same signature to the compiler
	Generic types that erase (lose their `<…>` type information at compile time) to the same type, e.g. `f(List<String>)` vs `f(List<Integer>)` (Chapter 9)

How it works — how the compiler chooses

The Java Language Specification (JLS §15.12.2 — the official rulebook for the language) resolves an overloaded call in **three phases**. It stops at the first phase that finds any applicable method:

1. **Phase 1 — exact match or primitive widening / reference subtyping (passing a more specific type where a more general one is expected).** No boxing (wrapping a primitive in its object type, `int` → `Integer`), no varargs. `byte → short → int → long → float → double`, `char → int`, `String → Object`.
2. **Phase 2 — allow boxing and unboxing** (`int ↔ Integer`), still no varargs.

3. **Phase 3 — allow varargs.**

If one phase finds several candidates, the **most specific** one wins (e.g. `int` over `long`, `String` over `Object`). If none is more specific than the others, the call is **ambiguous** — a compile error.

Consequences worth memorising: *widening beats boxing, and boxing beats varargs*.

```
// code/ch04/methods/Overloading.java (key parts)
static String describe(int x)      { return "int"; }
static String describe(long x)     { return "long"; }
static String describe(Integer x)  { return "Integer"; }
static String describe(Object x)   { return "Object"; }
static String describe(int... xs)  { return "int..."; }

static String pick(long x)    { return "long (widening)"; }
static String pick(Integer x) { return "Integer (boxing)"; }

static String box(Object o)   { return "Object"; }
static String box(double d)   { return "double"; }
```

Output:

```
describe(5)        -> int
describe(5L)       -> long
describe(b)        -> int
describe(c)        -> int
describe("hi")     -> Object
describe()         -> int...
describe(1, 2)     -> int...
pick(5)            -> long (widening)
box(5)             -> double
sum(1,2) sum(1,2,3) sum(1.5,2) -> 3 6 3.5
```

Reading the output: `describe(b)` with a `byte` finds `int` in phase 1 (widening; `int` is more specific than `long`). `pick(5)` chooses `long` over `Integer` because phase 1 already succeeds. `box(5)` chooses `double` over `Object` for the same reason — boxing `5` to `Integer` and then treating it as `Object` would need phase 2.

Trade-offs

Good use	Bad use
Same operation, different input types (`println(int)` / `println(String)`)	Same name, *different meaning* (`save(User)` saves; `save(String)` sends an email)
Convenience overloads with defaults: `connect(host)` calls `connect(host, 443)` — Java has no default parameter values, so this is the idiom	Overloads with the same number of parameters of related types (`f(Object)` vs `f(String)`), which surprise readers

Effective Java (Item 52) advises: avoid two overloads with the same number of parameters when a call could reasonably fit both; give them different names instead (`writeInt`, `writeLong`).

Common mistakes

- Believing a different return type is enough to overload.
- `list.remove(1)` on a `List<Integer>`: this calls `remove(int index)`, not `remove(Object)` — phase 1 finds the `int` version without boxing. Use `list.remove(Integer.valueOf(1))` to remove the value 1.
- Passing `null` when two reference overloads exist (`f(String)` and `f(Integer)`): ambiguous, compile error. Cast: `f((String) null)`.

Self-check

1. `void f(long x)` and `void f(Integer x)`; which runs for `f(3)`? → `f(long)` — widening in phase 1 beats boxing.
2. Can `int f()` and `void f()` coexist? → No, same signature.

3. When does a varargs overload get chosen? → Only if no fixed-arity (fixed number of parameters) method applies, even with boxing.

TL;DR

Overloads share a name and differ in parameter types, count or order. The compiler picks at compile time: exact/widening → boxing → varargs, then most specific.

4.4 Varargs

What it is

A **varargs** (variable-arity) parameter, written `Type... name`, lets callers pass zero, one or many arguments of that type. Inside the method, the parameter is simply an **array** (`int... numbers` is an `int[]`). Varargs arrived in Java 5.

How it works

1. The compiler collects the extra arguments into a new array and passes it.
2. Called with no extra arguments → an **empty array** (length 0), never `null`.
3. Called with an existing array of the right type → that array is passed as-is.
4. Rules: at most **one** varargs parameter per method, and it must be **last**: `join(String sep, String... parts)` is legal; `f(String... a, int b)` is not.

```java
// code/ch04/methods/Varargs.java (key parts)
static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}
static String join(String separator, String... parts) { return String.join(separator, parts); }
static void inspect(Object... items) { /* prints length and contents */ }

sum();  sum(4);  sum(1, 2, 3, 4);  sum(new int[] {10, 20});
inspect(1, "two");  inspect((Object) null);  inspect((Object[]) null);
```

Output:

```
0
4
10
30
a | b | c
cart has 3 items
inspect(1, "two"):
  length=2 -> [1, two]
inspect((Object) null):
  length=1 -> [null]
inspect((Object[]) null):
  length=null array -> null
```

The last two calls show the one real subtlety: with an `Object...` parameter, `(Object) null` means "one argument that happens to be null", while `(Object[]) null` means "the whole array is null". Code that accepts varargs from outside should tolerate a `null` array.

Real-world examples

`String.format(String, Object...)`, `System.out.printf`, `List.of(E...)`, `Arrays.asList(T...)`, `Path.of(String, String...)`, SLF4J's (a popular logging library) `log.info("{} {}", a, b)`.

Trade-offs and when not to use it

- Each call allocates (reserves memory for) an array. In hot paths (code that runs extremely often), JDK APIs add fixed-arity overloads for the common small cases — `List.of` has overloads for 0 to 10 elements plus a varargs one for exactly this reason (*Effective Java*, Item 53).
- If a method needs **at least one** argument, declare it: `min(int first, int... rest)` — the compiler then enforces it instead of a runtime check.
- Mixing varargs with generics (`T...`) triggers "possible heap pollution" (a generic variable ending up holding an object of the wrong type) warnings; `@SafeVarargs` suppresses them for methods that don't store or expose the array (Chapter 9).

Self-check

1. What is `numbers.length` in `sum()` called with no arguments? → 0.
2. Is `void f(int... a, String s)` legal? → No, varargs must be last.
3. Can a class have both `f(int[])` and `f(int...)`? → No, identical signatures.

TL;DR

`T... x` = an array parameter the compiler fills for you. Last parameter only, one per method, empty array when no arguments.

4.5 Recursion

What it is

A method is **recursive** when it calls itself. Each call solves a smaller version of the same problem until it reaches a case simple enough to answer directly.

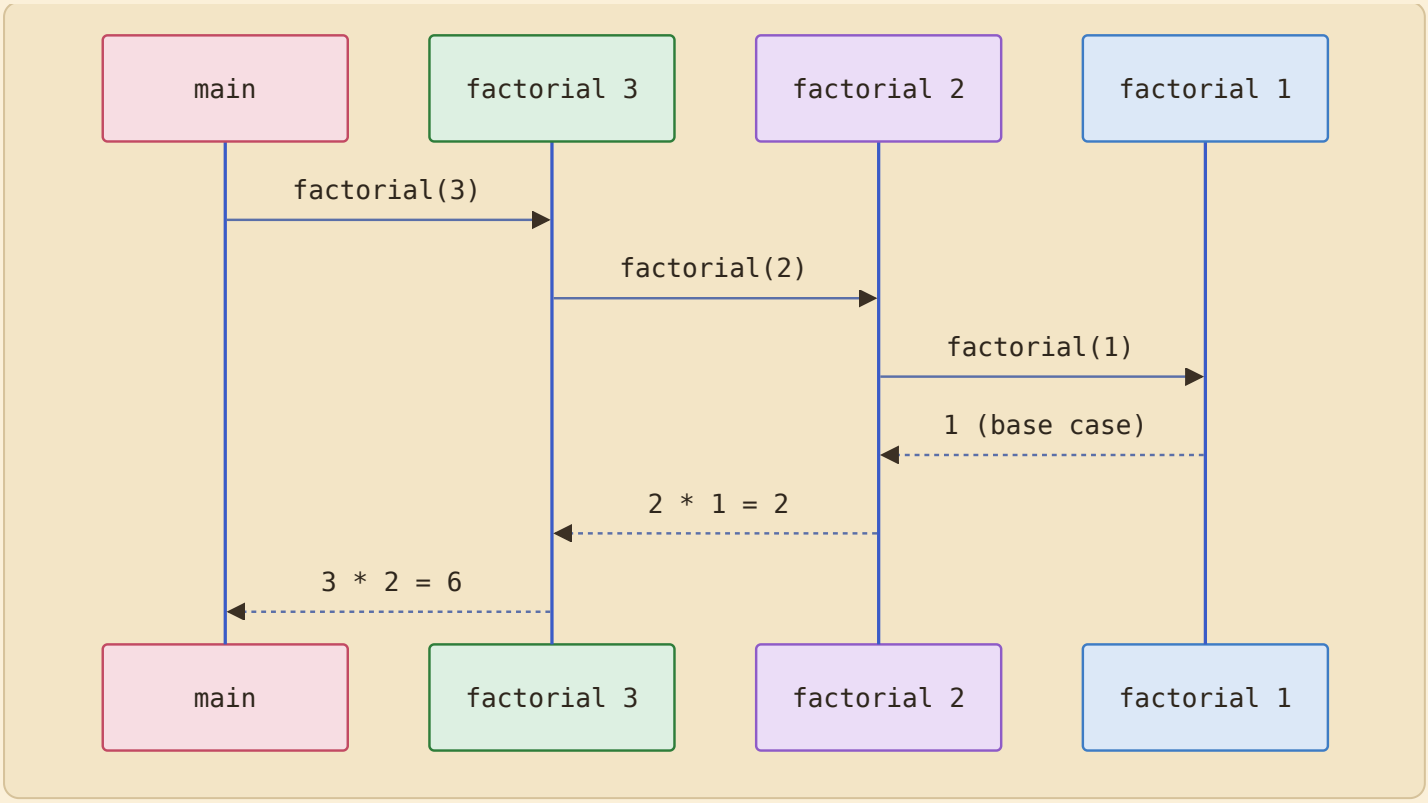
Every correct recursive method has:

1. **A base case** — an input answered without recursion (e.g. `factorial(0) = 1`).
2. **A recursive case** — the method calls itself on a *smaller* input, moving towards the base case.

Analogy: Russian nesting dolls. You open a doll to find a smaller doll, and keep going until you reach the solid one in the middle (the base case). Then you close them back up in reverse order — that's the "return" phase.

How it works — the call stack

Each call pushes a new **stack frame** with its own parameters and locals onto the thread's (a *thread* is one independent path of execution inside a program) **call stack**. When a call returns, its frame is popped and the caller continues where it left off.



The stack has a fixed size per thread (set with the JVM option `-Xss`; the default is platform-dependent, typically on the order of 1 MB on 64-bit systems). Recursion that never reaches a base case — or simply goes very deep — throws `StackOverflowError`. Java does **not** perform tail-call elimination (reusing the current frame when the very last action of a method is a call), so even a "tail-recursive" method uses one frame per call.

```
// code/ch04/methods/Recursion.java (key parts)
static long factorial(int n) {
    if (n < 0) throw new IllegalArgumentException("n must be >= 0");
    if (n <= 1) return 1;                 // base case
    return n * factorial(n - 1);          // recursive case
}
static long fibNaive(int n) { return n < 2 ? n : fibNaive(n - 1) + fibNaive(n - 2); }

static final Map<Integer, Long> memo = new HashMap<>();
static long fibMemo(int n) {              // memoization: cache each answer
    if (n < 2) return n;
    Long cached = memo.get(n);
    if (cached != null) return cached;
    long value = fibMemo(n - 1) + fibMemo(n - 2);
    memo.put(n, value);
    return value;
}
static long fibLoop(int n) {              // iterative: O(n) time, O(1) space
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}
static int binarySearch(int[] a, int key, int lo, int hi) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;         // avoids int overflow of (lo + hi)
    if (a[mid] == key) return mid;
    return a[mid] < key ? binarySearch(a, key, mid + 1, hi)
                        : binarySearch(a, key, lo, mid - 1);
}
static int depth = 0;
static void dive() { depth++; dive(); }   // no base case
```

Output (the depth in the last line differs from run to run and machine to machine):

```
5! = 120, 20! = 2432902008176640000
fibNaive(30) = 832040
fibMemo(90)  = 2880067194370816120
fibLoop(90)  = 2880067194370816120
index of 23 = 5
StackOverflowError after roughly 38000+ calls
```

Notes on the output:

- The overflow depth depends on the JVM, the platform, `-Xss` and the frame size.
- `20!` is the largest factorial that fits in a `long`; `21!` silently overflows (integer overflow, Chapter 2). Use `BigInteger` for bigger values (Chapter 7).
- `fibNaive(n)` makes roughly 1.6^n calls — `fibNaive(30)` already makes about 2.7 million calls because the same subproblems are recomputed. **Memoization** (caching results of previous calls) cuts it to O(n) calls. This idea is the heart of *dynamic programming* (solving a problem by combining stored answers to smaller, overlapping subproblems).

> **Key term — Big-O notation:** a way to describe how the work (time or memory) an algorithm needs grows as the input size *n* grows, ignoring constant factors. O(1) means constant — the same cost whatever *n* is; O(n) means it grows in proportion to *n*; $O(n^2)$ grows with the square; O(log n) grows very slowly (each doubling of *n* adds one step); $O(2^n)$ is exponential and explodes quickly.

Trade-offs — recursion vs iteration

	Recursion	Iteration (loops)
Fits naturally	Trees, nested structures, divide-and-conquer (split the problem, solve the parts, combine: merge sort, quicksort), backtracking (try a choice, undo it on a dead end), grammars	Linear scans, counters, simple accumulations
Readability	Often shorter and mirrors the definition	Can need an explicit stack for tree-like problems
Memory	One stack frame per active call; risk of `StackOverflowError` for deep inputs	Constant stack usage
Speed	Slight call overhead; exponential if subproblems repeat without memoization	Usually fastest

When not to use it

- When depth grows with input size and inputs can be large (a recursive walk of a 1-million-node linked list will overflow). Convert to a loop or use an explicit `ArrayDeque` (Java's resizable double-ended queue class) as a stack.
- When a simple loop is just as clear (factorial, sum of an array).

Common mistakes

- Missing or unreachable base case → infinite recursion → `StackOverflowError`.
- Recursive step that doesn't shrink the problem (`f(n)` calling `f(n)`), or shrinks past the base case (`n - 2` skipping `n == 1` when the base case is only `n == 1`).
- Catching `StackOverflowError` to "recover" in real code — it is an `Error` (the category of serious problems programs are not expected to catch), signalling a bug or an unsuitable algorithm. (The demo catches it only to print a message.)

Real-world examples

Directory trees are recursive structures (`Files.walkFileTree` visits them depth-first — all the way down one branch before the next); JSON parsers are typically recursive-descent (one method per grammar rule,

calling each other); compilers walk syntax trees (tree representations of source code) recursively; `Arrays.sort` for objects uses a merge-based algorithm (TimSort) that is divide-and-conquer at heart.

In the interview

> **30-second answer:** "A recursive method calls itself on a smaller input until it hits a base case. Each call gets a stack frame, so depth costs memory, and Java has no tail-call optimization — very deep recursion throws StackOverflowError. For overlapping subproblems like Fibonacci I add memoization, turning exponential time into linear."

- *Follow-up: How do you make a deep recursion safe?* → Rewrite iteratively with an explicit stack (`ArrayDeque`), or bound the depth; increasing `-Xss` is a last resort.
- *Follow-up: Time complexity of naive Fibonacci?* → $O(\varphi^n) \approx O(1.618^n)$, commonly quoted as $O(2^n)$; memoized is O(n) time and O(n) space.

Self-check

1. What two parts must every recursive method have? → A base case and a recursive case that moves towards it.
2. Why did `binarySearch` use `lo + (hi - lo) / 2`? → `(lo + hi)` can overflow `int` for huge arrays; this form cannot.
3. Does Java optimise tail calls? → No.

TL;DR

Recursion = base case + smaller self-call. Each call costs a stack frame; too deep → `StackOverflowError`. Memoize repeated subproblems; switch to loops for deep linear recursion.

4.6 Arrays

What it is

An **array** is a fixed-size, ordered container of elements that all have the same type, stored in one contiguous (side-by-side in memory) block and accessed by an integer **index** starting at 0. In Java an array is an **object**: it lives on the heap, a variable of array type holds a reference to it, and it has a `length` field.

Analogy: a row of numbered lockers. The number of lockers is fixed when the row is built; each locker holds one item of the declared kind; you open locker *i* directly without walking past the others.

Why it matters

- **Speed**: reading or writing `a[i]` is O(1) — the address is computed from the index.
- **Foundation**: `ArrayList` (resizable list), `HashMap` (key→value lookup table), `ArrayDeque`, `String` (internally a byte array) are all built on arrays.
- **Primitives without boxing**: an `int[]` stores raw ints; a `List<Integer>` stores objects. For large numeric data an array is several times smaller and faster.

How it works

1. **Declare** a variable: `int[] scores;` (preferred style; `int scores[]` also compiles but is discouraged).
2. **Create** the array with a size: `scores = new int[4];` — the size can be any `int` expression computed at runtime, but cannot change afterwards.
3. Or **create and fill** with an initializer: `double[] prices = {9.99, 4.5, 12.0};` (only in a declaration) or `new char[] {'a','e','i'}` (anywhere).
4. New arrays are filled with **default values**:

Element type	Default
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`char`	`'\u0000'` (the NUL character)
`boolean`	`false`
Any reference type (`String`, `Integer`, arrays, objects)	`null`

5. **Access** with `a[i]`, where `0 ≤ i < a.length`. Any other index throws `ArrayIndexOutOfBoundsException` at runtime (Java checks bounds on every access — no silent memory corruption as in C).
6. `length` is a field, not a method: `a.length` (compare `s.length()` for strings and `list.size()` for collections).

```
// code/ch04/arrays/ArrayBasics.java (key parts)
int[] scores = new int[4];
String[] names = new String[2];
boolean[] flags = new boolean[2];
double[] prices = {9.99, 4.5, 12.0};
char[] vowels = new char[] {'a', 'e', 'i'};

scores[0] = 90; scores[3] = 75;
for (int i = 0; i < prices.length; i++) prices[i] *= 2;   // index loop: can modify
double total = 0;
for (double p : prices) total += p;                       // enhanced for: read each element

try { scores[4] = 1; }
catch (ArrayIndexOutOfBoundsException e) { System.out.println("error: " + e.getMessage()); }

int[] alias = scores;          // copies the REFERENCE: one array, two names
int[] copy = scores.clone();   // a NEW array with the same elements
alias[1] = 50;
```

Output:

```
defaults: [0, 0, 0, 0] [null, null] [false, false]
scores=[90, 0, 0, 75] length=4
last price = 12.0, vowels = aei
doubled total = 52.980000000000004
error: Index 4 out of bounds for length 4
scores=[90, 50, 0, 75] copy=[90, 0, 0, 75]
scores == alias? true, scores == copy? false, Arrays.equals(scores, copy)? false
println(array) prints an identity string, e.g. starts with: [I@
empty array length = 0
```

Things to notice:

- `52.980000000000004` is ordinary binary floating-point rounding (Chapter 2); use `BigDecimal` for money.
- Printing an array directly prints something like `[I@1b6d3586` — the type code `[I` ("array of int") plus a hash (a number derived from the object's identity). Use `Arrays.toString(a)` (or `Arrays.deepToString` for nested arrays).
- `==` compares references; use `Arrays.equals` to compare contents.
- The enhanced `for` gives you a *copy* of each element; assigning to the loop variable does not change the array. Use an index loop to modify elements.

Copying arrays

Technique	Result	Notes
`b = a`	Same array, two names	No copy at all
`a.clone()`	New array, same length	**Shallow** (copies references, not the objects they point to): for an array of objects, both arrays share the element objects
`Arrays.copyOf(a, n)`	New array of length `n`	Truncates or pads with defaults
`Arrays.copyOfRange(a, from, to)`	New array of elements `[from, to)`	`to` is exclusive
`System.arraycopy(src, srcPos, dst, dstPos, len)`	Copies into an **existing** array	Fastest low-level copy; handles overlapping ranges correctly

Arrays are covariant (and why that can bite)

Because `String` is a subtype of `Object`, Java lets you treat a `String[]` as an `Object[]`. But the array object remembers its real element type and checks every store at runtime:

```
// code/ch04/arrays/Covariance.java (key parts)
String[] strings = {"a", "b"};
Object[] objects = strings;          // allowed: arrays are covariant
objects[0] = 42;                     // compiles, fails at runtime
```

Output:

```
ArrayStoreException: java.lang.Integer
min=-2 max=17
[] is an empty array, never return null for 'no results'
```

Generic collections are *invariant* instead (`List<String>` is not a `List<Object>`), which turns this runtime failure into a compile error — one reason *Effective Java* (Item 28) says to prefer lists to arrays in APIs (the public methods a class or library offers to callers). See Chapter 9.

Trade-offs — array vs `ArrayList`

	Array (`int[]` , `String[]`)	**`ArrayList<E>` (Chapter 10)**
Size	Fixed at creation	Grows automatically
Primitives	Stored directly	Must box (`Integer`)
Syntax	`a[i]`, `a.length`	`list.get(i)`, `list.size()`
Useful methods	Few (use `Arrays.*`)	Many (`add` , `remove` , `contains` , `sort` …)
Type safety	Covariant, runtime store checks	Invariant generics, compile-time checks
`toString` / `equals`	Identity-based (need `Arrays.*`)	Content-based
Best for	Fixed-size numeric data, performance-critical code, low-level buffers, `main(String[])`	Almost everything else in application code

Common mistakes

- **Off-by-one**: looping `i <= a.length` → `ArrayIndexOutOfBoundsException` on the last step.
- **Printing** with `println(a)` and seeing `[I@…`.

- **Comparing** with `==` or `a.equals(b)` (which is also identity for arrays) instead of `Arrays.equals`.
- **Assuming** `b = a` **copies** the array.
- **Returning** `null` for "nothing found" — return `new int[0]`.

Self-check

1. What does `new String[3]` contain? → Three `null` references (no strings yet).
2. `a.length` or `a.length()`? → `a.length` — it is a field.
3. After `int[] b = a.clone(); b[0] = 9;`, has `a[0]` changed? → No, `clone` created a new array. (For an array of mutable objects the *elements* would be shared.)
4. Why does `Object[] o = new String[1]; o[0] = 1;` compile but fail? → Array covariance; the runtime store check throws `ArrayStoreException`.

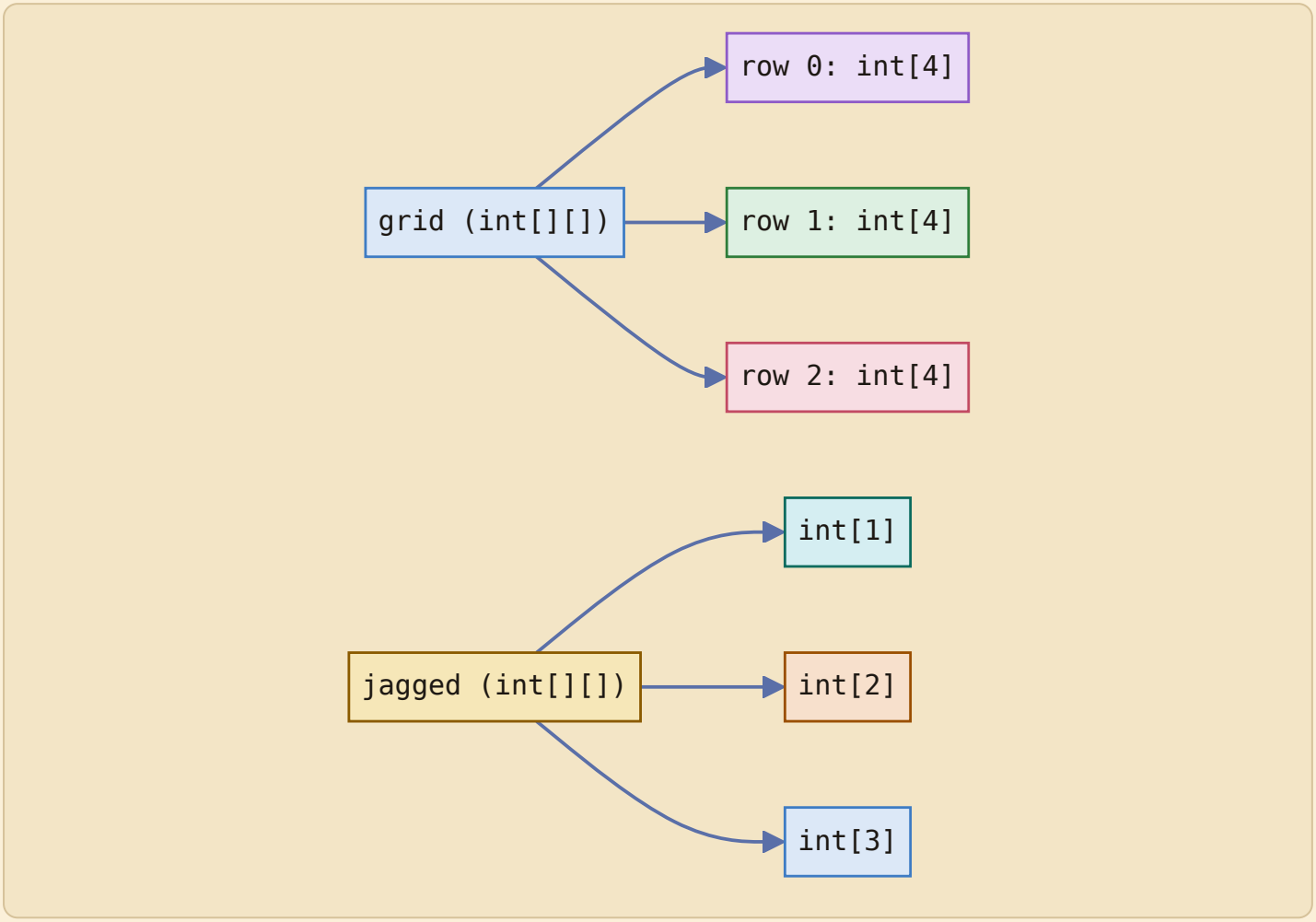
TL;DR

Arrays are fixed-size, zero-indexed, bounds-checked objects with a `length` field and default-filled elements. Assignment copies the reference; use `clone`/`copyOf`/`arraycopy` to copy, `Arrays.toString`/`Arrays.equals` to print and compare.

4.7 Multidimensional and jagged arrays

What it is

Java has no true 2-D arrays; it has **arrays of arrays**. `int[][] grid` is an array whose elements are `int[]` references — each row is its own array object. Because rows are independent, they can have different lengths: a **jagged** (or ragged) array.



How it works

1. `new int[3][4]` creates an outer array of 3 references and 3 inner `int[4]` arrays — a rectangular grid.
2. `new int[3][]` creates only the outer array; each row is `null` until you assign one (`jagged[0] = new int[1]`).
3. `grid.length` is the number of rows; `grid[r].length` is the length of row `r`.
4. Access with `grid[row][col]` — the first index picks the row array, the second the element in it.
5. Nested initializers work: `char[][] board = {{'X','O','X'}, …};`.

```
// code/ch04/arrays/MultiDim.java (key parts)
int[][] grid = new int[3][4];
for (int r = 0; r < grid.length; r++)
    for (int c = 0; c < grid[r].length; c++)
        grid[r][c] = r * 10 + c;
System.out.println(Arrays.deepToString(grid));

int[][] jagged = new int[3][];
jagged[0] = new int[] {1};
jagged[1] = new int[] {1, 1};
jagged[2] = new int[] {1, 2, 1};

int n = 6;                                          // Pascal's triangle
int[][] pascal = new int[n][];
for (int i = 0; i < n; i++) {
    pascal[i] = new int[i + 1];
    pascal[i][0] = pascal[i][i] = 1;
    for (int j = 1; j < i; j++) pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
}

char[][] board = { {'X','O','X'}, {' ','X','O'}, {'O',' ','X'} };
int[][] a = {{1, 2}, {3, 4}}, b = {{1, 2}, {3, 4}};
System.out.println("Arrays.equals: " + Arrays.equals(a, b) + ", Arrays.deepEquals: " + Arrays.deepEquals(a, b));
```

Output:

```
[[0, 1, 2, 3], [10, 11, 12, 13], [20, 21, 22, 23]]
[1] len=1
[1, 1] len=2
[1, 2, 1] len=3
row 5 of Pascal: [1, 5, 10, 10, 5, 1]
X wins on the diagonal? true
unallocated row is null
Arrays.equals: false, Arrays.deepEquals: true
```

`Arrays.equals` on a 2-D array compares the *row references* (different objects → false); `Arrays.deepEquals` compares contents recursively.

Real-world examples

Game boards (chess, tic-tac-toe, Sudoku), images as pixel grids, matrices (rectangular grids of numbers) in numeric code, dynamic-programming tables (`dp[i][j]` for edit distance or longest common subsequence), adjacency matrices for small graphs (a grid recording which nodes of a network are connected).

Common mistakes

- Using `grid.length` for the column count — use `grid[r].length` (rows may differ).
- `NullPointerException` from a jagged array whose rows were never allocated.
- Looping column-first over a big 2-D array: each row is a separate array, so iterating row by row (`for r … for c …`) is much more cache-friendly (the CPU loads memory in blocks into a fast cache, so reading neighbouring elements in order is quicker).
- `Arrays.fill(grid, 0)` on an `int[][]` compiles (it picks `fill(Object[], Object)`) but throws `ArrayStoreException` at runtime, because each slot must hold an `int[]`. Fill each row instead: `for`

`(int[] row : grid) Arrays.fill(row, 0);`.

Self-check

1. After `int[][] m = new int[2][];`, what are `m.length` and `m[0]`? → `2` and `null` (the rows are not allocated yet).
2. How do you print a 2-D array? → `Arrays.deepToString(grid)`.

TL;DR

A 2-D array is an array of row arrays; rows can differ in length (jagged). Use `deepToString`/`deepEquals`, iterate row-major.

4.8 The `java.util.Arrays` toolbox

What it is

`java.util.Arrays` is a class of static helper methods for arrays — sorting, searching, filling, copying, comparing, converting. Learn these instead of writing loops.

Method	What it does	Notes
`toString(a)` / `deepToString(a)`	Readable text `[1, 2, 3]`	Use `deep…` for nested arrays
`sort(a)` / `sort(a, from, to)`	Sorts in place, ascending	Primitives: Dual-Pivot Quicksort, O(n log n) typical. Objects: TimSort — **stable** (equal elements keep their original order), O(n log n) worst case
`sort(a, comparator)`	Sort objects by a `Comparator` (an object that decides which of two values comes first)	Comparators in Chapter 10 and Chapter 11
`parallelSort(a)`	Sorts using multiple threads	Only pays off for large arrays
`binarySearch(a, key)`	O(log n) search in a **sorted** array	Returns the index, or `-(insertionPoint) - 1` if absent. Unsorted input → undefined result
`fill(a, v)`	Sets every element to `v`	For objects, every slot gets the *same* reference
`setAll(a, i -> …)`	Computes each element from its index	Java 8
`copyOf`, `copyOfRange`	New array (truncate/pad, or a slice)	
`equals(a, b)` / `deepEquals`	Content equality	
`hashCode(a)` / `deepHashCode`	Content-based hash	Matches `equals`
`compare(a, b)`	Lexicographic (dictionary-style, element by element) comparison: negative, 0, positive	Java 9
`mismatch(a, b)`	Index of first difference, or -1	Java 9
`asList(T...)`	Fixed-size `List` view **backed by** the array	Writes go through; `add`/`remove` throw
`stream(a)`	A stream over the array (`IntStream` for `int[]`)	Streams in Chapter 12

```java
// code/ch04/arrays/ArraysUtil.java (key parts)
int[] nums = {42, 7, 19, 3, 25};
Arrays.sort(nums);
Arrays.binarySearch(nums, 19);   Arrays.binarySearch(nums, 20);
String[] words = {"pear", "Fig", "apple", "kiwi"};
Arrays.sort(words);
Arrays.sort(words, String.CASE_INSENSITIVE_ORDER);
Arrays.sort(words, Comparator.comparing(String::length).thenComparing(Comparator.reverseOrder()));
Arrays.fill(filled, -1);
Arrays.setAll(squares, i -> i * i);
System.arraycopy(nums, 0, dest, 2, nums.length);
List<String> view = Arrays.asList(backing);   view.set(0, "z");   view.add("d"); // throws
List<String> independent = List.of(backing);  backing[1] = "changed";
```

Output:

```
sort:          [3, 7, 19, 25, 42]
binarySearch(19) = 2, binarySearch(20) = -4
sort strings:  [Fig, apple, kiwi, pear]
ignore case:   [apple, Fig, kiwi, pear]
by length:     [Fig, pear, kiwi, apple]
fill:          [-1, -1, -1, -1, -1]
setAll:        [0, 1, 4, 9, 16, 25]
copyOf(3):     [3, 7, 19]
copyOf(7):     [3, 7, 19, 25, 42, 0, 0]
copyOfRange:   [7, 19, 25]
arraycopy:     [0, 0, 3, 7, 19, 25, 42]
equals=false compare=-1 mismatch=2 hashCode equal? true
stream sum:    96, max: 42
asList view:   [z, b, c] array now [z, b, c]
asList add ->  UnsupportedOperationException
List.of copy:  [z, b, c]
```

Reading it:

- `binarySearch(nums, 20)` returns `-4`: 20 would be inserted at index 3, so the result is `-(3) - 1`. A negative result always means "not found".
- String natural order (the default ordering a type defines for itself) is by UTF-16 code unit (the 16-bit number Java stores for each character), so uppercase `F` (70) sorts before lowercase `a` (97). Use `CASE_INSENSITIVE_ORDER` or a `Collator` (a language-aware text comparer) for human-friendly order.
- `Arrays.asList` is a *view*: changes to the list show in the array and vice versa, and its size is fixed. `List.of(array)` takes a *copy* and is fully unmodifiable — the later `backing[1] = "changed"` did not affect it. Wrap with `new ArrayList<>(Arrays.asList(a))` when you need a growable list.

Common mistakes

- `binarySearch` on an unsorted array.
- `Arrays.asList(intArray)` — with a primitive array you get a `List<int[]>` of **one** element, not a list of numbers. Use `Arrays.stream(intArray).boxed().toList()`.
- Expecting `Arrays.sort` on objects to handle `null` elements — it throws `NullPointerException` unless the comparator handles nulls (`Comparator.nullsFirst(...)`).

Self-check

1. What does `Arrays.binarySearch(new int[]{1,3,5}, 4)` return? → `-3` (insertion point 2).
2. Is `Arrays.sort(String[])` stable? → Yes (TimSort) — equal elements keep their relative order.
3. What happens if you call `add` on the result of `Arrays.asList`? → `UnsupportedOperationException`.

TL;DR

Don't hand-roll array loops for common tasks:
`Arrays.toString/sort/binarySearch/fill/setAll/copyOf/equals/asList/stream` cover them.

4.9 `main`, command-line arguments, and simpler `main` (Java 25)

What it is

The **entry point** of a Java program is a method named `main`. The classic form is:

```
public static void main(String[] args)
```

- `public` — the launcher, outside your class, must be able to call it.
- `static` — no object exists yet when the program starts.
- `void` — the program's exit status (a number handed back to the operating system; 0 means success) is set with `System.exit(code)`, not by returning.
- `String[] args` — the **command-line arguments**: the words typed after the class name. `String... args` is also accepted.

How it works — arguments

1. `java ch04.args.CommandLineArgs 3 7 --verbose` → `args` is `{"3", "7", "--verbose"}`.
2. With no arguments, `args` is an **empty array** (length 0), never `null`.
3. Every argument is a `String`; convert with `Integer.parseInt`, `Double.parseDouble`, etc., and handle `NumberFormatException`.
4. The shell (the command-line program that runs your commands) splits on spaces; quote to keep spaces: `java App "New York"` gives one argument.
5. The program name is **not** in `args` (unlike C's `argv[0]`).

```
// code/ch04/args/CommandLineArgs.java (key parts)
public static void main(String[] args) {
    if (args.length == 0) {
        args = new String[] {"3", "7", "x", "--verbose"};
        System.out.println("(no args given, using demo args)");
    }
    System.out.println("args = " + Arrays.toString(args) + ", count = " + args.length);
    boolean verbose = false;
    int sum = 0;
    for (String a : args) {
        if (a.equals("--verbose")) { verbose = true; continue; }
        try { sum += Integer.parseInt(a); }
        catch (NumberFormatException e) { System.out.println("skipping non-number: " + a); }
    }
    System.out.println("sum = " + sum + (verbose ? " (verbose mode)" : ""));
}
```

Output (run with no arguments, so the demo arguments are used):

```
(no args given, using demo args)
args = [3, 7, x, --verbose], count = 4
skipping non-number: x
sum = 10 (verbose mode)
```

Real command-line tools use a parser library (picocli, JCommander) instead of hand-written loops once they have more than a couple of options.

Simpler `main` — compact source files and instance `main` (Java 25, JEP 512)

Chapter 1, §1.10 introduced JEP 512 (final in Java 25). The parts that matter for `main`:

- `main` no longer has to be `public` or `static`, and the `String[]` parameter is optional. If a class has both, the launcher picks `main(String[])`; if the chosen method is an instance method, the launcher first creates an object with the class's non-private no-argument constructor.
- A **compact source file** holds just methods and fields; the compiler wraps them in an implicit `final` class that other code cannot name, and imports all of `java.base` (the JDK's core module, holding `java.lang`, `java.util` and friends) for you. `IO.println` / `IO.readln` handle simple console I/O.

```java
// code/HelloCompact.java — the entire file
void main() {
    String name = "compact world";
    IO.println("Hello, " + name + "!");
    IO.println("Lists are auto-imported: " + List.of(1, 2, 3));
}
```

Output (`java HelloCompact.java`):

```
Hello, compact world!
Lists are auto-imported: [1, 2, 3]
```

`java HelloCompact.java` compiles and runs the file in one step (source-file mode, Java 11). Compact files are for scripts, experiments and learning; anything that grows into several classes should become a normal class in a package — you can convert by simply wrapping the members in `class HelloCompact { … }` and adding imports.

Common mistakes

- Misspelling the classic signature (`public void main(String[] args)` without `static`) — on Java 25 this now *runs* as an instance main, but on Java 21 or earlier it fails with "main method is not static". Keep the classic form in code meant to run on older JDKs.
- Reading `args[0]` without checking `args.length` → `ArrayIndexOutOfBoundsException`.
- Returning an exit code from `main` — use `System.exit(1)`.

Self-check

1. What is `args.length` when no arguments are given? → 0.
2. On Java 25, which does the launcher pick if a class has both `static void main(String[] a)` and `void main()`? → The `String[]` version — it is preferred.
3. What does a compact source file import automatically? → Every exported package of the `java.base` module.

TL;DR

Classic entry point: `public static void main(String[] args)`; `args` holds the command-line words as Strings (never null). Java 25 also accepts non-public, non-static, no-arg `main` and class-less compact source files with `IO.println`.

Chapter TL;DR

Topic	Remember
Method	modifiers + return type + name + params + body; signature = name + param types
`return`	every path of a non-void method; bare `return;` in void methods
Static vs instance	static = belongs to class, no `this`; instance = runs on an object
Pass-by-value	always; references are copied, so mutation is shared but reassignment is not
Overloading	same name, different params; resolved at compile time: widening → boxing → varargs → most specific
Varargs	`T... x` is an array; last param only; empty array when no args
Recursion	base case + smaller call; stack frames; no tail-call optimisation; memoize overlaps
Arrays	fixed size, 0-based, bounds-checked, default-filled, `length` field, covariant
2-D arrays	arrays of arrays; can be jagged; `deepToString`, `deepEquals`
`Arrays`	`sort`, `binarySearch` (sorted!), `fill`, `setAll`, `copyOf`, `equals`, `asList` (fixed-size view)
`main`	classic `public static void main(String[] args)`; Java 25: instance/no-arg main, compact source files, `IO`

Where next: objects and classes — how to bundle data and methods together (Chapter 5). For exam traps on this chapter's material see OCP Chapter 5.

Chapter 5 — OOP: Classes and Objects

Java is an **object-oriented** language: programs are built from *objects* that bundle data with the code that works on that data. This chapter teaches the mechanics — how to write a class, create objects, construct them correctly, share state with `static` (members that belong to the class itself and are shared by every object), hide details with access modifiers (keywords such as `private` that control who can see what), organise code in packages, make objects immutable (impossible to change once created), and understand exactly in what order Java initialises everything.

The *design* side of OOP — when to use which relationship, SOLID (five classic object-oriented design principles), patterns (named, reusable solutions to common design problems) — is covered in depth in the LLD (low-level design) handbook; this chapter links there instead of repeating it: LLD Chapter 2 — OOP fundamentals.

What you'll learn

1. Classes vs objects, references, `new`, `null`, and where objects live (heap vs stack, garbage collection).
2. Fields, instance methods and the `this` keyword.
3. Constructors: the default constructor, overloading, `this(...)` chaining, and Java 25's flexible constructor bodies.
4. Packages and imports (including static imports and Java 25 module imports).
5. `static` fields, methods, constants and initialiser blocks.
6. The four access levels: `public`, `protected`, package-private, `private`.
7. Encapsulation: why fields are private and what getters/setters are for.
8. The `final` keyword on variables, fields, methods and classes.
9. Immutability basics: the five rules and defensive copying.
10. The exact initialisation order of a class and an object.

All runnable code is in `code/` (package `ch05.*`), compiled with `javac --release 25` and run on JDK 26.

> **Exam depth.** Constructor rules, initialisation-order puzzles, `this`/`super` traps and immutability questions are drilled in OCP Chapter 6 — Class design; access modifiers and `static` traps in OCP Chapter 5 — Methods.

5.1 Classes and objects

What it is

- A **class** is a blueprint: it declares what data each object will hold (**fields**) and what it can do (**methods**). A class is also a **type** — you can declare variables of it.
- An **object** (or **instance**) is one concrete thing built from that blueprint, with its own copy of the fields.
- **Instantiation** is creating an object: `new BankAccount()`.

Analogy: a class is the architect's plan for a house; objects are the actual houses built from it. Every house has the same layout, but each has its own furniture (field values), and painting one house doesn't paint the others.

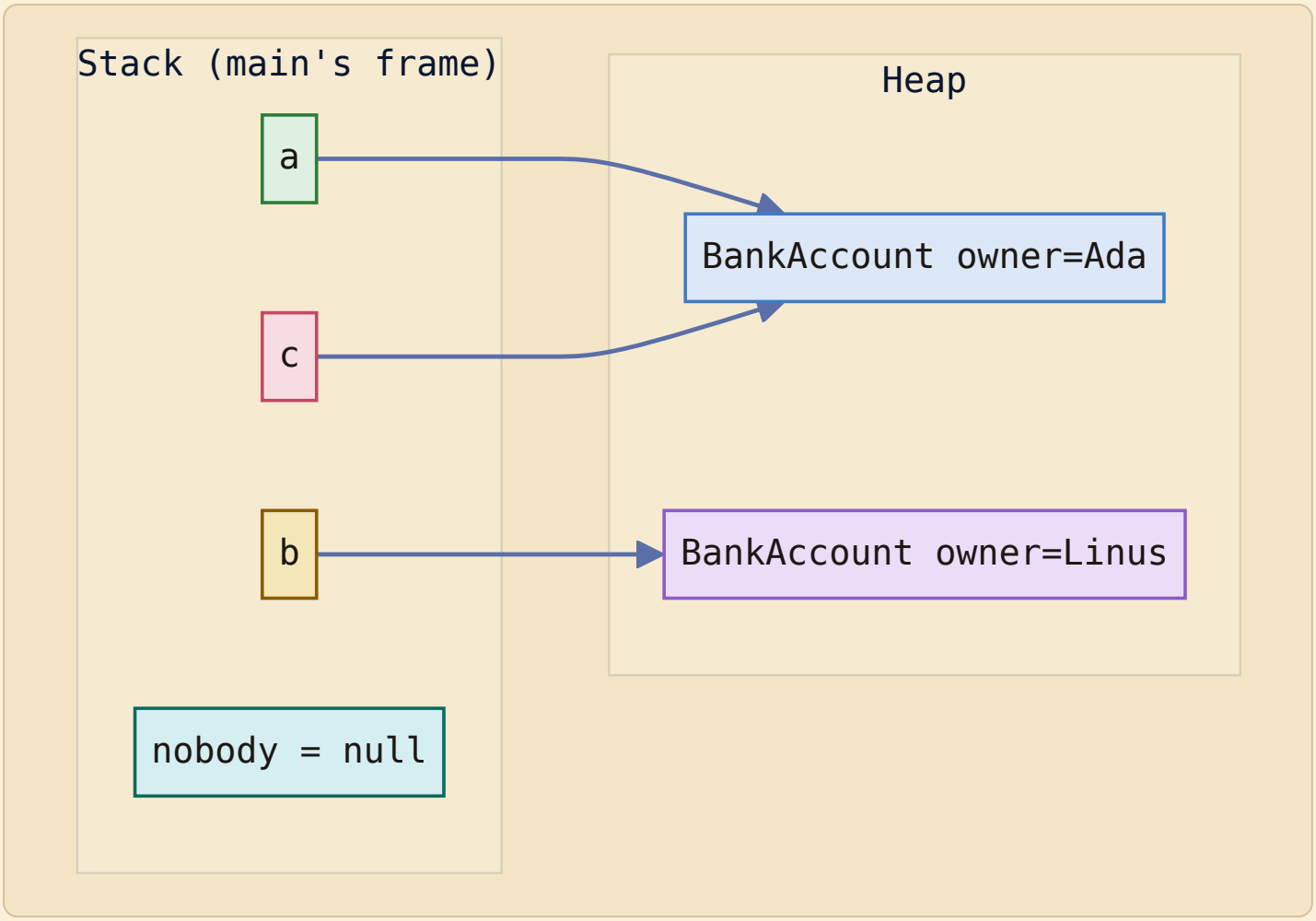
Why it matters

Grouping data with the behaviour that protects it keeps rules in one place. A `BankAccount` that knows how to `withdraw` can refuse to go negative; a bare `long balance` passed around the program cannot.

How it works

1. **Declare** the class (one public top-level class per file, file named after it: `BankAccount.java`).

2. **Create** objects with `new ClassName(args)`. `new` allocates memory on the **heap** (the JVM memory area where all objects live), runs the constructor (the object's set-up code, §5.3), and returns a **reference** to the new object.
3. **Store** the reference in a variable of the class type. The variable is not the object; it *points to* it.
4. **Use** the object with the dot operator: `a.deposit(100)`, `a.owner`.
5. When no live variable references an object any more, it becomes **unreachable** and the **garbage collector** (GC) eventually reclaims its memory. Java has no `delete`/`free` — see GC in Chapter 16.




```java
// code/ch05/classes/BankAccount.java
public class BankAccount {
    String owner;          // fields: each object gets its own copy
    long balanceCents;     // defaults: null and 0 until assigned

    void deposit(long cents) { balanceCents += cents; }

    boolean withdraw(long cents) {
        if (cents > balanceCents) return false;
        balanceCents -= cents;
        return true;
    }

    String summary() {
        return owner + ": " + balanceCents / 100 + "." + String.format("%02d", balanceCents % 100);
    }
}
```

```java
// code/ch05/classes/ClassesDemo.java (key parts)
BankAccount a = new BankAccount();
a.owner = "Ada";
a.deposit(10_050);
BankAccount b = new BankAccount();
b.owner = "Linus";
b.deposit(2_000);
System.out.println(a.summary() + " | " + b.summary());
System.out.println("withdraw 50.00 from Linus: " + b.withdraw(5_000) + " -> " + b.summary());

BankAccount c = a;                // copies the reference, not the object
c.deposit(1);
System.out.println("via alias: " + a.summary() + ", same object? " + (a == c));

BankAccount fresh = new BankAccount();
System.out.println("defaults: owner=" + fresh.owner + ", balance=" + fresh.balanceCents);

BankAccount nobody = null;
nobody.deposit(1);                // NullPointerException (caught in the file)
```

Output:

```
Ada: 100.50 | Linus: 20.00
withdraw 50.00 from Linus: false -> Linus: 20.00
via alias: Ada: 100.51, same object? true
defaults: owner=null, balance=0
NPE: Cannot invoke "ch05.classes.BankAccount.deposit(long)" because "<local5>" is null
```

What the output shows:

- `a` and `b` are independent objects; `c = a` created a second name for the *same* object.
- **Fields get default values** (`null`, `0`, `false`) — unlike local variables.
- `null` means "no object". Calling a method through it throws `NullPointerException` (an *exception* is an error object that, once thrown, stops normal execution unless caught — Chapter 8). Since Java 14 (JEP 358 — a JDK Enhancement Proposal — on by default since 15) the message says exactly what was null. It shows `<local5>` here because the class was compiled without debug info (extra data that maps compiled code back to source variable names and line numbers); compile with `javac -g` (IDEs and build tools do) and it names the variable: `because "nobody" is null`.
- The money is kept in `long` cents — never `double` for money (Chapter 2/7).

Common mistakes

- Confusing the reference with the object: `==` on objects compares *references* (same object?), not contents. Content equality is `equals`, covered in Chapter 6.
- Forgetting to initialise a field that holds an object, then calling a method on it (NPE).
- Public fields as in this first `BankAccount` — anyone can set `balanceCents = -5`. §5.7 fixes that.

Self-check

1. How many `BankAccount` objects exist after `var x = new BankAccount(); var y = x;`? → One, with two references.
2. What value does an `int` field have before assignment? A local `int`? → `0`; a local has no value and cannot be read until assigned.
3. When is an object eligible for garbage collection? → When it is no longer reachable from any live reference (thread stacks — each *thread*, an independent path of execution, has its own stack — static fields, …).

TL;DR

Class = blueprint and type; object = instance on the heap; variables hold references. `new` creates, GC reclaims unreachable objects, `null` means no object.

5.2 Fields, instance methods and `this`

What it is

- An **instance field** (instance variable) is per-object state: `owner`, `balanceCents`.
- An **instance method** is behaviour that runs on one particular object and can read/write its fields.
- `this` is a reference to *the current object* — the one the method or constructor is running on. It exists in every instance method and constructor, and not in static methods.

The four uses of `this`

Use	Example	Why
Disambiguate a field hidden by a parameter	`this.size = size;`	The parameter **shadows** the field; `size = size` would just assign the parameter to itself
Pass the current object to someone else	`registry.register(this)`	Lets other objects hold a reference to you
Return the current object for chaining	`return this;` → `pizza.addTopping("a").addTopping("b")`	Fluent APIs (method chains that read like a sentence), builders (helper objects that assemble another object step by step; LLD Builder)
Call another constructor of the same class	`this(size, false);`	Constructor chaining (§5.3)

Inside an instance method, a bare field name like `balanceCents` is shorthand for `this.balanceCents` unless a local variable or parameter with the same name shadows it.

Common mistakes

- `this.name = name` written as `name = name` — compiles, does nothing; the field stays `null`. Many IDEs warn "assignment to itself".
- Using `this` inside a `static` method — compile error; there is no current object.
- Letting `this` **escape** from a constructor (registering a listener — an object notified when something happens — or starting a thread with `this`) before construction finishes — other code can then see a half-built object. Especially dangerous with threads (Chapter 14).

TL;DR

Instance fields are per-object; `this` is the current object — used to reach shadowed fields, pass/return yourself, and chain constructors.

5.3 Constructors

What it is

A **constructor** is special code that runs when an object is created with `new`. Its job is to put the new object into a **valid** state. It looks like a method but:

- its name is exactly the class name,
- it has **no return type** (not even `void` — adding one turns it into an ordinary method with a confusing name),
- it is not inherited (subclasses do not receive it).

How it works

1. **Default constructor.** If you write *no* constructor, the compiler adds a no-argument one that just calls `super()` (the parent class's constructor). Its access matches the class's access.
2. As soon as you write **any** constructor, no default is generated. If you still want `new X()`, write it yourself.
3. **Overloading.** A class can have many constructors with different parameter lists (same rules as method overloading, Chapter 4 §4.3).
4. **Chaining with `this(...)`.** One constructor can delegate to another. Put the real work (validation, assignments) in one "full" constructor and let the others supply defaults — no duplicated logic.
5. **Every constructor starts by running a superclass constructor** (the *superclass* is the parent class this one extends; a class that extends another is its *subclass*) — either explicitly with `super(...)` or with an implicit `super()` the compiler inserts (Chapter 6). A constructor calls *either* `this(...)` *or* `super(...)`, at most once.
6. **Validate** arguments and throw (usually `IllegalArgumentException` or `NullPointerException` via `Objects.requireNonNull`) so an invalid object never exists — "fail fast" (LLD design principles).

Flexible constructor bodies (Java 25, JEP 513)

Until Java 24, `this(...)` or `super(...)` had to be the **first statement**. That forced awkward code to validate or transform arguments before delegating (usually a helper static method). **JEP 513, final in Java 25** (previewed in 22–24), allows statements *before* the explicit `this(...)`/`super(...)` call. That part is called the **prologue**, and it has limits:

Allowed in the prologue	Not allowed in the prologue
Local variables, computations, `if`/loops	Using `this` explicitly or implicitly (reading fields, calling instance methods)
Validating arguments and throwing exceptions	Using `super.x` or `super.m()`
Assigning fields of *this* class that have no initialiser (Java 25)	Reading those fields

```java
// code/ch05/constructors/Pizza.java (key parts)
public class Pizza {
    private final String size;
    private final boolean extraCheese;
    private final List<String> toppings = new ArrayList<>();

    public Pizza(String size, boolean extraCheese, String... toppings) {   // the "full" constructor
        if (!List.of("S", "M", "L").contains(size))
            throw new IllegalArgumentException("bad size: " + size);
        this.size = size;
        this.extraCheese = extraCheese;
        this.toppings.addAll(List.of(toppings));
    }
    public Pizza(String size) { this(size, false); }   // chaining
    public Pizza()            { this("M"); }

    // Java 25: statements BEFORE this(...) — parse and validate, without touching 'this'
    public Pizza(String order, char separator) {
        String[] parts = order.strip().split(String.valueOf(separator));
        if (parts.length == 0 || parts[0].isEmpty())
            throw new IllegalArgumentException("empty order");
        this(parts[0].toUpperCase(), false, Arrays.copyOfRange(parts, 1, parts.length));
    }

    public Pizza addTopping(String t) { toppings.add(t); return this; }  // fluent: returns this
    @Override public String toString() { /* Pizza[size, extra cheese?, toppings] */ }
}
```

```java
// code/ch05/constructors/ConstructorsDemo.java (key parts)
static class NoConstructorWritten { int x = 7; }            // compiler adds NoConstructorWritten() { super(); }
static class OnlyArgConstructor {
    final int x;
    OnlyArgConstructor(int x) { this.x = x; }               // now 'new OnlyArgConstructor()' will not compile
}

System.out.println(new Pizza());
System.out.println(new Pizza("L"));
System.out.println(new Pizza("S", true, "olives", "basil"));
System.out.println(new Pizza(" m,ham,pineapple ", ','));
System.out.println(new Pizza("M").addTopping("mushroom").addTopping("onion"));
new Pizza("XXL");                                            // throws, caught in the file
```

Output:

```
Pizza[M, []]
Pizza[L, []]
Pizza[S, extra cheese, [olives, basil]]
Pizza[M, [ham, pineapple]]
Pizza[M, [mushroom, onion]]
default ctor gives x=7, arg ctor gives x=42
rejected: bad size: XXL
```

Trade-offs — ways to create objects

Approach	Good for	Weak spot
Telescoping constructors (a series of constructors, each adding one more parameter, as `Pizza`)	1–3 optional parameters	Many parameters of the same type are easy to swap by mistake
Static factory method (a static method that creates and returns an instance: `Pizza.margherita()`, `List.of`, `Integer.valueOf`)	Descriptive names, caching, returning subtypes (*Effective Java* — Joshua Bloch's well-known best-practice book — Item 1)	Not obvious in docs that it replaces a constructor
Builder (`Pizza.builder().size("L").cheese(true).build()`)	Many optional parameters, readable call sites (*Effective Java*, Item 2)	More code — see LLD Builder
Records (Chapter 6)	Plain data carriers	Fixed to "all components" canonical form (one constructor taking every field)

Common mistakes

- Adding a return type: `public void Pizza() {}` is a *method*, not a constructor.
- Adding an argument constructor and then being surprised `new Pizza()` stops compiling.
- Duplicating validation in every constructor instead of chaining to one.
- Calling overridable methods (methods a subclass can replace with its own version) from a constructor — a subclass override can run before the subclass's fields are initialised (*Effective Java*, Item 19; demo in Chapter 6).
- Recursive chaining (`A() { this(1); } A(int x) { this(); }`) — compile error "recursive constructor invocation".

In the interview

> **30-second answer:** "A constructor initialises a new object into a valid state. If you declare none, Java adds a no-arg default constructor; declare any and it disappears. Constructors overload, and I chain them with `this(...)` so validation lives in one place. Every constructor first runs a superclass constructor via `super(...)`, explicit or implicit. Since Java 25, I can validate arguments before that `this`/`super` call, as long as I don't touch `this`."

- *Follow-up: Can a constructor be private?* → Yes — for singletons (classes that must have exactly one instance), static-factory-only classes, and utility classes that must not be instantiated.
- *Follow-up: Are constructors inherited?* → No. A subclass must declare its own and call a superclass one.

Self-check

1. Class `A` declares only `A(int x)`. Does `new A()` compile? → No; no default constructor is generated.
2. Where can `this(...)` appear before Java 25? In Java 25? → First statement only; in Java 25 it can follow prologue statements that don't use `this`.
3. What does `void Pizza()` inside class `Pizza` declare? → An ordinary method named `Pizza`.

TL;DR

Constructors: class name, no return type, not inherited. Default constructor only if you write none. Chain with `this(...)`; one constructor does the validation. Java 25 allows validation code before `this(...)`/`super(...)`.

5.4 Packages and imports

What it is

A **package** is a named group of related classes, like a folder — and it literally maps to a folder. Packages give you:

1. **Namespaces** (separate naming areas, so equal names don't collide) — `java.util.Date` and `java.sql.Date` can coexist.
2. **Access control** — package-private members (§5.6) are visible only inside the package.
3. **Organisation** — `com.shop.orders`, `com.shop.payments`.

How it works

1. The first statement of the file declares the package: `package ch05.access.bank;`. No declaration → the **unnamed (default) package**, fine for scratch code, not for real projects (other packages cannot import from it).
2. Directory structure must match: `ch05/access/bank/Vault.java`. Compiled classes land in the same structure (`ch05/access/bank/Vault.class`).
3. Naming convention: all lowercase, reversed internet domain: `com.example.shop.orders`.
4. The **fully qualified name** of a class is package + name: `ch05.access.bank.Vault`. You can always use it without importing.
5. **Imports** let you use the short name:

Form	Example	Notes
Single-type import	`import java.util.List;`	Preferred — explicit, no ambiguity
Type-import-on-demand (wildcard)	`import java.util.*;`	Imports types of *that* package only, **not** subpackages (`java.util.function` needs its own import)
Static import	`import static java.lang.Math.max;` / `import static java.lang.Math.*;`	Imports static members; use sparingly (constants, test assertions)
Module import (Java 25, JEP 511)	`import module java.base;`	Imports all public types of every package the module (a named group of packages, see JPMS) exports; single-type imports win on name clashes

6. `java.lang` (`String`, `Math`, `Integer`, `System`, …) and your own package are imported automatically.
7. Imports are purely a **compile-time** convenience: they don't load or include code, and have no runtime cost.
8. At runtime the JVM finds classes via the **class path** (the list of folders and JAR files to search: `java -cp out ch05.statics.StaticDemo`) or the **module path** (Chapter 16).

Common mistakes

- Name clash: importing both `java.util.*` and `java.sql.*` then writing `Date` — ambiguous, compile error. Import one explicitly or use the fully qualified name.
- Assuming `import java.util.*;` covers `java.util.concurrent.ConcurrentHashMap` — it doesn't.
- File path not matching the package declaration — `javac` may compile it, but the class won't be found at runtime.

Self-check

1. Do you need to import `String`? → No, `java.lang` is automatic.
2. Does `import java.util.*` import `java.util.function.Function`? → No.
3. Does a wildcard import make the program slower? → No, imports are compile-time only.

TL;DR

Package = namespace + folder + access boundary. Import single types, avoid ambiguous wildcards, `java.lang` is free. Java 25 adds `import module`.

5.5 Static members

What it is

`static` means **"belongs to the class, not to any object"**.

- A **static field** has exactly **one copy**, shared by all instances (and usable with no instance at all).
- A **static method** runs without an object; it has no `this`.
- A **static initialiser block** `static { … }` runs **once**, when the class is initialised.
- A **constant** is a `static final` field, named in `UPPER_SNAKE_CASE`.

Analogy: in a school (the class), each student (object) has their own backpack (instance fields), but there is one shared notice board in the hallway (static field) that every student reads and writes.

How it works

1. Access statics through the class name: `Ticket.issuedSoFar()`, `Ticket.MAX_PER_ORDER`. (Accessing via an object reference compiles but is misleading — IDEs warn.)
2. A static method can use only static members directly. An instance method can use both.

3. **Class initialisation** (running static field initialisers and static blocks, top to bottom) happens once, the first time the class is *actively used*: creating an instance, calling a static method, or reading a non-constant static field.
4. Reading a **compile-time constant** (a `static final` primitive or `String` initialised with a constant expression) does **not** initialise the class — the compiler copies the value into the caller.

```java
// code/ch05/statics/Ticket.java
public class Ticket {
    public static final int MAX_PER_ORDER = 10;      // constant
    private static int issued;                       // ONE copy, shared
    private static final String PREFIX;

    static {                                         // runs once, at class initialisation
        PREFIX = "TKT-" + (2000 + 26);
        System.out.println("[Ticket class initialised]");
    }

    private final int number;                        // one copy PER object

    public Ticket() { issued++; number = issued; }
    public String id() { return PREFIX + "-" + number; }
    public static int issuedSoFar() { return issued; }
    public static boolean canBuy(int quantity) { return quantity > 0 && quantity <= MAX_PER_ORDER; }
}
```

```java
// code/ch05/statics/StaticDemo.java (key parts)
import static java.lang.Math.max;
import static java.lang.Math.PI;

System.out.println("before first use");
System.out.println("MAX_PER_ORDER = " + Ticket.MAX_PER_ORDER);  // constant: inlined, no class init
System.out.println("issued = " + Ticket.issuedSoFar());           // first real use: class init
Ticket t1 = new Ticket(), t2 = new Ticket(), t3 = new Ticket();
System.out.println(t1.id() + ", " + t2.id() + ", " + t3.id());
System.out.println("issued = " + Ticket.issuedSoFar() + ", canBuy(12) = " + Ticket.canBuy(12));
System.out.println("max(3, 8) = " + max(3, 8) + ", PI = " + PI);
```

Output:

```
before first use
MAX_PER_ORDER = 10
[Ticket class initialised]
issued = 0
TKT-2026-1, TKT-2026-2, TKT-2026-3
issued = 3, canBuy(12) = false
max(3, 8) = 8, PI = 3.141592653589793
```

Notice that printing `MAX_PER_ORDER` did *not* trigger the static block; calling `issuedSoFar()` did.

When to use `static` — and when not

Good fit	Poor fit
Constants (`MAX_PER_ORDER`, `Math.PI`)	Mutable global state (`static List<User> users`) — hidden coupling (parts of the program silently depending on each other), hard to test, **not thread-safe** (unsafe when several threads use it at once) without synchronisation (coordinating threads, e.g. with locks)
Pure utility functions: output depends only on inputs (`Math.max`, `Integer.parseInt`)	Behaviour that should vary by implementation — static methods cannot be overridden (Chapter 6)
Static factory methods (`List.of`, `Integer.valueOf`)	Dependencies you want to swap in tests (a `static Database db`) — prefer passing objects in
Counters/ID generators — but use `AtomicInteger` (a thread-safe counter class) if threads are involved	

Utility classes (`Math`, `Collections`, `Arrays`) are `final` with a `private` constructor so they can't be instantiated (*Effective Java*, Item 4). The Singleton pattern and its pitfalls are in LLD Chapter 8.

Common mistakes

- "non-static variable cannot be referenced from a static context" — calling instance members from `main`. Create an object.
- The `issued++` counter above is a **race condition** (a bug whose outcome depends on the unpredictable timing of threads) if two threads create tickets at once (Chapter 14).
- Overusing static imports so readers can't tell where `of(...)` comes from.

In the interview

30-second answer: "Static members belong to the class: one shared copy of a static field, and static methods run without an instance, so they have no `this` and can't be overridden — only hidden (a subclass static method with the same signature just shadows it). Static initialisation happens once, lazily, when the class is first actively used. I use static for constants, pure utilities and factories, and avoid mutable static state because it's global, hard to test and needs synchronisation."

- *Follow-up: Can a static method access an instance field?* → Not directly; only through an object reference it is given or creates.
- *Follow-up: When does a static block run?* → Once, during class initialisation — on first instantiation, static method call, or non-constant static field access.

Self-check

1. Three `Ticket`s exist. How many copies of `issued`? Of `number`? → One; three.
2. Why didn't reading `Ticket.MAX_PER_ORDER` print "[Ticket class initialised]"? → It's a compile-time constant; its value was inlined into `StaticDemo`.
3. Why are `Math`'s methods static? → They are pure functions with no per-object state.

TL;DR

Static = one per class. Use for constants, pure utilities, factories. Static init runs once on first active use. Avoid mutable static state.

5.6 Access modifiers

What it is

Access modifiers control *who can see* a class, field, method or constructor. They are how Java enforces encapsulation (hiding an object's internals, §5.7) at compile time.

Modifier	Same class	Same package	Subclass in another package	Everywhere else
`private`	yes	no	no	no
(none) = **package-private**	yes	yes	no	no
`protected`	yes	yes	yes (through inheritance — a subclass receiving its parent's members)	no
`public`	yes	yes	yes	yes

Top-level classes can only be `public` or package-private. Members (fields, methods, constructors, nested types — classes declared inside another class) can use all four.

How it works

```
// code/ch05/access/bank/Vault.java
package ch05.access.bank;
public class Vault {
    public    String name = "Main vault";   // everyone
    protected int    floors = 3;            // same package + subclasses
              int    guards = 4;            // package-private: same package only
    private   String code = "0000";         // this class only
    public boolean open(String attempt) { return code.equals(attempt); }
}
```

```
// code/ch05/access/bank/Teller.java — same package: sees everything except private
return v.name + ", floors=" + v.floors + ", guards=" + v.guards;
```

```
// code/ch05/access/client/AccessDemo.java — different package
static class BranchVault extends Vault {
    int floorsPlusOne() { return floors + 1; }     // protected: visible through inheritance
}
Vault v = new Vault();
System.out.println("public name:        " + v.name);
// v.floors  -> compile error: protected, not accessed through a subclass
// v.guards  -> compile error: package-private, different package
// v.code    -> compile error: private
System.out.println("open(\"1234\"):       " + v.open("1234"));
System.out.println("Teller (same pkg):  " + Teller.inspect(v));
System.out.println("subclass protected: " + new BranchVault().floorsPlusOne());
```

Output:

```
public name:        Main vault
open("1234"):       false
Teller (same pkg):  Main vault, floors=3, guards=4
subclass protected: 4
```

The protected subtlety: a subclass in another package can use a protected member only *through inheritance* — on `this` or on references of its own subclass type — not on an arbitrary `Vault` object. That's why `v.floors` fails in `AccessDemo` even though `BranchVault.floorsPlusOne()` works. The OCP chapter has more puzzles on this.

Guidelines

1. **Make every field** `private` (constants excepted). Expose behaviour, not data.
2. Start members as **private or package-private** and widen only when needed. *Effective Java* (Item 15): "minimise the accessibility of classes and members". Every `public` member is a promise you must keep forever.
3. Use `protected` deliberately — it is part of your public API (the members other code may rely on) to subclassers.
4. Package-private is a great default for helper classes that other packages shouldn't depend on.
5. For stronger, module-level hiding (a `public` class that other *modules* still can't use), see JPMS (the Java Platform Module System) in Chapter 16.

Self-check

1. Which is wider: `protected` or package-private? → `protected` (package + subclasses).
2. Can a top-level class be `private`? → No; only `public` or package-private. Nested classes can be private.
3. What access does a member with no modifier have? → Package-private.

TL;DR

private < package-private < protected < public. Fields private, members as narrow as possible.

5.7 Encapsulation

What it is

Encapsulation is bundling an object's data with the methods that operate on it and **hiding the data** so that it can only change through those methods. The object then guards its own **invariants** — rules that must always hold (e.g. "5 ≤ target ≤ 30", "balance ≥ 0").

Analogy: a car's dashboard. You use the steering wheel and pedals (public methods); you don't reach into the engine to move the pistons (private fields). The manufacturer can redesign the engine without changing how you drive.

Why it matters

- **Validity** — nobody can put the object into a nonsense state.
- **Freedom to change** — you can change the internal representation (store Fahrenheit, add caching — keeping computed results to reuse) without breaking callers.
- **Debuggability** — every change passes through one method; put a breakpoint (a marker where the debugger pauses the program) there.

How it works

1. Make fields `private`.
2. Provide methods that express **behaviour** (`raise`, `withdraw`) rather than raw setters where possible ("tell, don't ask" — LLD principles).
3. Where you do need accessors (methods that read or write a field), a **getter** returns a value (`getTargetCelsius()`, `isActive()` for booleans) and a **setter** validates before changing (`setTargetCelsius(v)`).
4. Omit the setter for read-only properties; compute derived values instead of storing them.

```java
// code/ch05/encapsulation/Thermostat.java
public class Thermostat {
    public static final double MIN = 5.0, MAX = 30.0;
    private double targetCelsius = 20.0;
    private int changes;

    public double getTargetCelsius() { return targetCelsius; }
    public void setTargetCelsius(double value) {
        if (Double.isNaN(value) || value < MIN || value > MAX)
            throw new IllegalArgumentException("target must be in [" + MIN + ", " + MAX + "]: " + value);
        targetCelsius = value;
        changes++;
    }
    public void raise(double delta) { setTargetCelsius(Math.min(MAX, targetCelsius + delta)); }
    public double getTargetFahrenheit() { return targetCelsius * 9 / 5 + 32; }   // derived
    public int getChanges() { return changes; }                                  // read-only
}
```

```java
// code/ch05/encapsulation/EncapsulationDemo.java (key parts)
static class OpenThermostat { public double targetCelsius; }   // no encapsulation
open.targetCelsius = -273;                                      // nothing stops nonsense
Thermostat t = new Thermostat();
t.setTargetCelsius(22.5);
t.raise(10);                                                    // clamped to MAX
t.setTargetCelsius(-273);                                       // rejected
```

Output:

```
open thermostat: -273.0
target=30.0C = 86.0F, changes=2
rejected: target must be in [5.0, 30.0]: -273.0
```

Trade-offs

Getters/setters for every field ("anaemic" objects) are only slightly better than public fields: they keep the option to add validation later, but they don't express behaviour. Prefer meaningful operations; for pure data, prefer an immutable **record** (Chapter 6). The full discussion (encapsulation vs abstraction, "tell don't ask", Law of Demeter — an object should talk only to its direct collaborators) is in LLD §2.3 Encapsulation.

Common mistakes

- Private field but a getter that returns a mutable internal object (a `List`, `Date`, array) — callers can change it anyway. Return an unmodifiable view (a read-only window onto the original data) or copy (§5.9).
- Setters that don't validate.
- Generating getters/setters by reflex for every field.

In the interview

30-second answer: "Encapsulation means hiding an object's state behind methods, so the object enforces its own invariants and can change its internals without breaking callers. In Java: private fields, behaviour-oriented public methods, validated setters only where needed, and no leaking of mutable internals through getters."

Self-check

1. What is an invariant? → A condition about an object's state that must always be true (e.g. balance ≥ 0).
2. Is a class with private fields and a public setter for each, without validation, well encapsulated? → Barely; any value can still be set, and the class exposes its representation.

TL;DR

Private state + methods that protect invariants. Prefer behaviour over getters/setters; never leak mutable internals.

5.8 The `final` keyword

`final` means "cannot be changed after it is set", but *what* can't change depends on where it's used:

On	Meaning	Example
Local variable / parameter	Assigned exactly once	`final int max = 10;`
Instance field	Must be assigned exactly once — in its declaration, an instance initialiser, or **every** constructor	`private final String size;`
Static field	Assigned once, in its declaration or a static block	`static final String PREFIX;`
Method	Cannot be **overridden** by subclasses	`public final void close()`
Class	Cannot be **extended**	`final class String`, `final class Money`

Key points:

- `final` on a reference stops **reassignment**, not **mutation** of the object: a `final StringBuilder sb` can still `append` (see output in §5.9). A `final` array's elements can still change.
- A variable that is never reassigned is **effectively final** even without the keyword; lambdas (short inline functions) and inner classes may only capture (use from the surrounding method) such variables (Chapter 11).
- `final` fields also carry a **memory-model guarantee** (the Java Memory Model is the set of rules for what one thread is guaranteed to see of another thread's writes): once a constructor finishes (without leaking `this`), other threads see the correct values of `final` fields without synchronisation (Chapter 15).
- Don't confuse `final` with `finally` (exception handling, Chapter 8) or the deprecated `finalize()` method.

5.9 Immutability basics

What it is

An **immutable** object's observable state **never changes after construction**. `String`, `Integer`, `LocalDate`, `BigDecimal` and records with immutable components are immutable. To "change" one you create a new object (`s.toUpperCase()` returns a new string).

Analogy: a printed banknote. You can't edit the number on it; to get a different amount you get a different note.

Why it matters

Benefit	Why
Simple to reason about	One state, set once, validated once
Thread-safe for free	Nothing to race on; share across threads with no locks (mechanisms that let only one thread into a section at a time)
Safe to share and cache	`Integer.valueOf` caches; `String` literals are pooled (Chapter 7)
Safe as `HashMap` keys / `HashSet` elements (hash-based lookup collections)	The hash code (the number used to decide where an object is stored) can't change after insertion
No defensive copies needed when passing around	Nobody can modify it

Effective Java, Item 17: "minimise mutability". The trade-off: a new object per change, which can matter for very large objects updated in tight loops (that's why `StringBuilder` exists as the mutable companion of `String`). The design-level treatment is in LLD §2.13 Immutability and records and LLD principle 14.

How it works — the five rules

1. **Don't provide methods that modify state** (no setters).
2. **Make the class `final`** (or give it only private constructors) so a subclass can't add mutable behaviour.
3. **Make all fields `private final`.**
4. **Validate in the constructor**, so every instance is valid from birth.
5. **Ensure exclusive access to mutable components**: make **defensive copies** of mutable inputs (lists, arrays, `Date`) in the constructor, and never hand out references to mutable internals — return copies or unmodifiable views.

"Modifier" operations return a **new** object (`plus`, `with…`).

```java
// code/ch05/immutable/Money.java
public final class Money {
    private final long cents;
    private final String currency;
    public Money(long cents, String currency) {
        this.cents = cents;
        this.currency = Objects.requireNonNull(currency, "currency");
    }
    public long cents() { return cents; }
    public String currency() { return currency; }
    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(cents + other.cents, currency);   // new object
    }
    @Override public String toString() { return String.format("%d.%02d %s", cents / 100, cents % 100, currency); }
}
```

```java
// code/ch05/immutable/Playlist.java
public final class Playlist {
    private final String name;
    private final List<String> songs;
    public Playlist(String name, List<String> songs) {
        this.name = name;
        this.songs = List.copyOf(songs);          // defensive copy in; also unmodifiable
    }
    public List<String> songs() { return songs; } // safe to hand out
    public Playlist with(String song) {           // "wither": copy + change
        List<String> next = new ArrayList<>(songs);
        next.add(song);
        return new Playlist(name, next);
    }
}
```

```java
// code/ch05/immutable/ImmutabilityDemo.java (key parts)
static final class LeakyPlaylist {                 // final fields, but NOT immutable
    private final List<String> songs;
    LeakyPlaylist(List<String> songs) { this.songs = songs; }   // keeps the caller's list
    List<String> songs() { return songs; }                      // hands out the internal list
}

List<String> input = new ArrayList<>(List.of("Intro", "Verse"));
LeakyPlaylist leaky = new LeakyPlaylist(input);
input.add("SNEAKY");
leaky.songs().clear();

final StringBuilder sb = new StringBuilder("final ref");
sb.append(" but mutable object");
```

Output:

```
19.99 EUR + 5.01 EUR = 25.00 EUR (price unchanged: 19.99 EUR)
leaky after outside changes: []
safe after outside change:   Mix[Intro, Verse]
safe.songs().clear() -> UnsupportedOperationException
with("Chorus"): Mix[Intro, Verse, Chorus], original: Mix[Intro, Verse]
final ref but mutable object
```

`LeakyPlaylist` has only `final` fields yet its contents were changed twice from outside — **`final` fields are not enough**; rule 5 is what makes `Playlist` safe.

Common mistakes

- Thinking `final` = immutable (see above).
- `Collections.unmodifiableList(list)` without copying: it is a read-only *view*; the owner of the original list can still change it. `List.copyOf` copies.
- Forgetting arrays: return `array.clone()`, never the field.
- Mutable components like `java.util.Date` — prefer immutable `java.time` types (Chapter 7).

In the interview

30-second answer: "An immutable class has a fixed state after construction: final class, private final fields, no setters, validation in the constructor, and defensive copies of any mutable inputs and outputs. Benefits: thread safety without locks, safe sharing and caching, safe as map keys. String, Integer and java.time types are examples; records make simple immutable classes a one-liner if their components are immutable."

- *Follow-up: Why must the class be final?* → A subclass could add mutable state or override methods to behave mutably, breaking the guarantee for code holding a reference of the parent type.
- *Follow-up: Is a record automatically deeply immutable?* → No (deeply immutable would mean nothing reachable from it can change). Its fields are final, but a `List` component can still be mutated

unless you copy it in the compact constructor (a record's shorthand constructor for validation) (Chapter 6).

Self-check

1. `final List<String> xs = new ArrayList<>(); xs.add("a");` — legal? → Yes; `final` stops reassignment only.
2. What's the difference between `List.copyOf(x)` and `Collections.unmodifiableList(x)`? → The first is an independent unmodifiable copy; the second is a read-only view that reflects changes to `x`.
3. Name three immutable JDK classes. → `String`, `Integer`, `LocalDate` (also `BigDecimal`, `Instant`, `UUID`).

TL;DR

Immutable = final class, private final fields, no setters, validate in constructor, copy mutable stuff in and out. Changes return new objects. `final` ≠ immutable.

5.10 Initialisation order

What it is

When a class is first used and when an object is created, Java runs several kinds of initialisation code in a **fixed order**. Knowing it prevents "why is this field still null?" bugs — and it's a favourite exam and interview question.

The kinds of initialisation code:

- **Static field initialisers** (`static int x = 5;`) and **static blocks** (`static { … }`) — class level.
- **Instance field initialisers** (`private List<String> xs = new ArrayList<>();`) and **instance initialiser blocks** (`{ … }`) — object level.
- **Constructor bodies.**

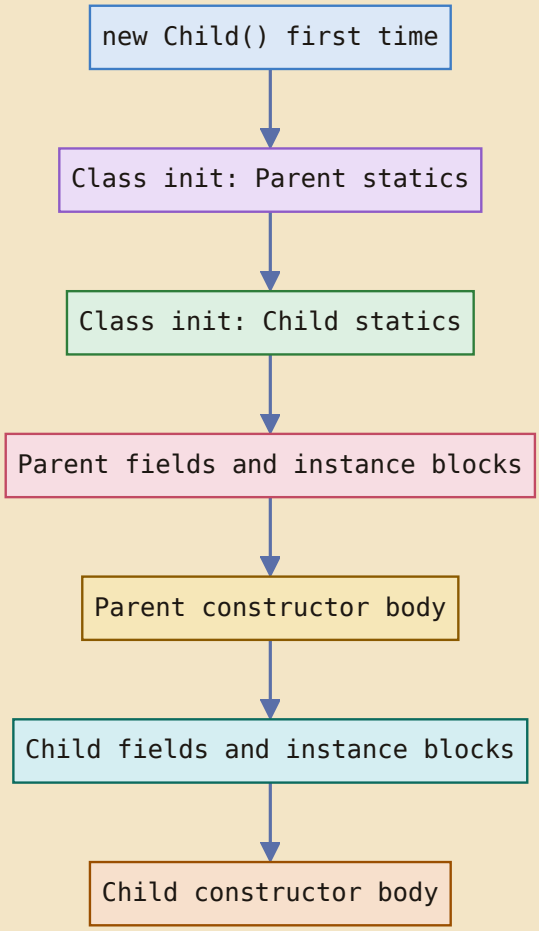
How it works

Class initialisation (once per class, on first active use):

1. Initialise the **superclass** first (recursively, up to `Object`).
2. Run this class's static field initialisers and static blocks **in the order they appear in the source**.

Object creation (`new Child()`), every time:

1. Memory is allocated; all fields get their default values (`0`, `null`, `false`).
2. The `Child` constructor is called; its first action is the superclass constructor (`super(...)`, explicit or implicit). (Java 25: any prologue statements run before that call.)
3. The **superclass** runs its instance field initialisers and instance blocks (in source order), then its constructor body — recursively up the chain.
4. Back in `Child`: its instance field initialisers and instance blocks run **in source order**.
5. The rest of the `Child` constructor body runs.




```java
// code/ch05/init/InitOrder.java (key parts)
static class Parent {
    static { log("1. Parent static block"); }
    private final String p = log("3. Parent field initializer");
    { log("4. Parent instance block"); }
    Parent() { log("5. Parent constructor body"); }
}
static class Child extends Parent {
    static final String TAG = log("2. Child static field");
    private final String c = log("6. Child field initializer");
    { log("7. Child instance block"); }
    Child() {
        super();                                    // implicit if omitted
        log("8. Child constructor body");
    }
}
public static void main(String[] args) {
    System.out.println("--- first new Child() ---");
    new Child();
    System.out.println("--- second new Child(): static parts do not run again ---");
    new Child();
}
```

Output:

```
--- first new Child() ---
1. Parent static block
2. Child static field
3. Parent field initializer
4. Parent instance block
5. Parent constructor body
6. Child field initializer
7. Child instance block
8. Child constructor body
--- second new Child(): static parts do not run again ---
3. Parent field initializer
4. Parent instance block
5. Parent constructor body
6. Child field initializer
7. Child instance block
8. Child constructor body
```

(`TAG` is initialised by a method call, so it is *not* a compile-time constant and runs as part of class initialisation.)

Common mistakes

- A static field used by a static block declared *above* it — it still has its default value (source order matters).
- Calling an overridable method from the parent constructor: it runs the child's override **before** the child's field initialisers (step 3 comes before step 4), so the child sees default values. Demo in Chapter 6 §6.3.
- Heavy work or I/O (input/output such as file or network access) in static blocks: failures surface as `ExceptionInInitializerError`, and the class becomes unusable (`NoClassDefFoundError` on later use).

Self-check

1. Which runs first: a parent's instance block or a child's static block? → The child's static block (all class init happens before any object creation).
2. Two instance blocks and a field initialiser — which order? → Source order, top to bottom.
3. How often does a static block run? → Once per class (per class loader — the JVM component that loads class files).

TL;DR

Statics once (parent first, source order). Per object: defaults → parent (fields/blocks, then constructor body) → child (fields/blocks, then constructor body).

Chapter TL;DR

Topic	Remember
Class vs object	Blueprint/type vs instance on the heap; variables hold references
`null` / NPE	No object; helpful NPE messages since Java 14
`this`	Current object: shadowed fields, pass/return self, `this(...)`
Constructors	Name = class, no return type, not inherited; default only if none written; chain with `this(...)`; Java 25 prologue before `this` / `super`
Packages	Namespace + folder + access boundary; single-type imports; `java.lang` automatic; `import module` (25)
`static`	One per class; no `this`; init once on first active use; constants are `static final`
Access	private < package-private < protected < public; fields private
Encapsulation	Hide state, expose behaviour, protect invariants
`final`	Variable: assign once. Method: no override. Class: no subclass. Not the same as immutable
Immutability	Final class, private final fields, no setters, validate, defensive copies
Init order	Parent statics → child statics → (per object) parent fields/blocks → parent ctor → child fields/blocks → child ctor

Where next: inheritance, interfaces, polymorphism, records, enums and nested classes in Chapter 6. OOP design (SOLID, relationships, patterns): LLD handbook.

Chapter 6 — OOP: Inheritance, Interfaces and More

Chapter 5 showed how to build a single class. This chapter is about **relationships between types**: how one class extends another, how different classes share a contract (a promise about which methods a type provides) through interfaces, and how Java calls the right code at runtime (polymorphism). Then it covers the modern type kinds every Java developer now uses daily — **sealed** hierarchies (with a closed list of allowed subtypes), **records** (compact data-carrying classes) and **enums** (types with a fixed set of named constants) — plus nested classes and the three `Object` methods (`equals`, `hashCode`, `toString`) that every class should get right.

This chapter is about the *language mechanics*. For *design* — when to inherit vs compose, SOLID (five classic object-oriented design principles), the Liskov Substitution Principle (a subclass must work anywhere its parent is expected), and patterns (named, reusable design solutions) built on these features — follow the links to the LLD handbook rather than re-reading it here.

What you'll learn

1. Inheritance with `extends`, `super(...)` and `super.method()`; what is and isn't inherited.
2. Overriding rules (return types, access, exceptions), `@Override`, and hiding of static methods and fields.
3. Polymorphism and dynamic dispatch; upcasting, downcasting and `instanceof` patterns.
4. Abstract classes and the template-method idea.
5. Interfaces: abstract, `default`, `static` and `private` methods; multiple inheritance of behaviour and conflict resolution.
6. Sealed classes and interfaces (Java 17) with exhaustive `switch`.
7. Records (Java 16): compact constructors, validation, defensive copies.
8. Enums: fields, constructors, per-constant behaviour, `switch`, `EnumSet`/`EnumMap`.
9. Nested, inner, local and anonymous classes (and when a lambda is better).
10. Choosing between abstract classes, interfaces, records and enums.
11. `equals`/`hashCode`/`toString` and their contracts.

All runnable code is in `code/` (package `ch06.*`), compiled with `javac --release 25` and run on JDK 26.

> **Exam depth.** Overriding vs hiding puzzles, abstract-class rules and constructor chains: OCP Chapter 6 — Class design. Interfaces, enums, sealed types, records, nested classes, polymorphism and casting traps: OCP Chapter 7 — Beyond classes.

6.1 Inheritance and `super`

What it is

Inheritance lets a new class (the **subclass**, child or derived class) be defined as an extension of an existing class (the **superclass**, parent or base class). The subclass automatically gets the parent's accessible fields and methods, can add new ones, and can **override** (replace) inherited methods. It models an **"is-a"** relationship: a `Manager` *is an* `Employee`.

Analogy: a new car model built on an existing platform. It keeps the chassis and engine (inherited members), adds a sunroof (new members), and retunes the suspension (overridden methods).

How it works

1. Declare with `extends`: `class Manager extends Employee`. Java allows **one** direct superclass (single inheritance of classes).
2. Every class without `extends` implicitly extends `java.lang.Object`, so every class has `toString`, `equals`, `hashCode`, `getClass` and friends.

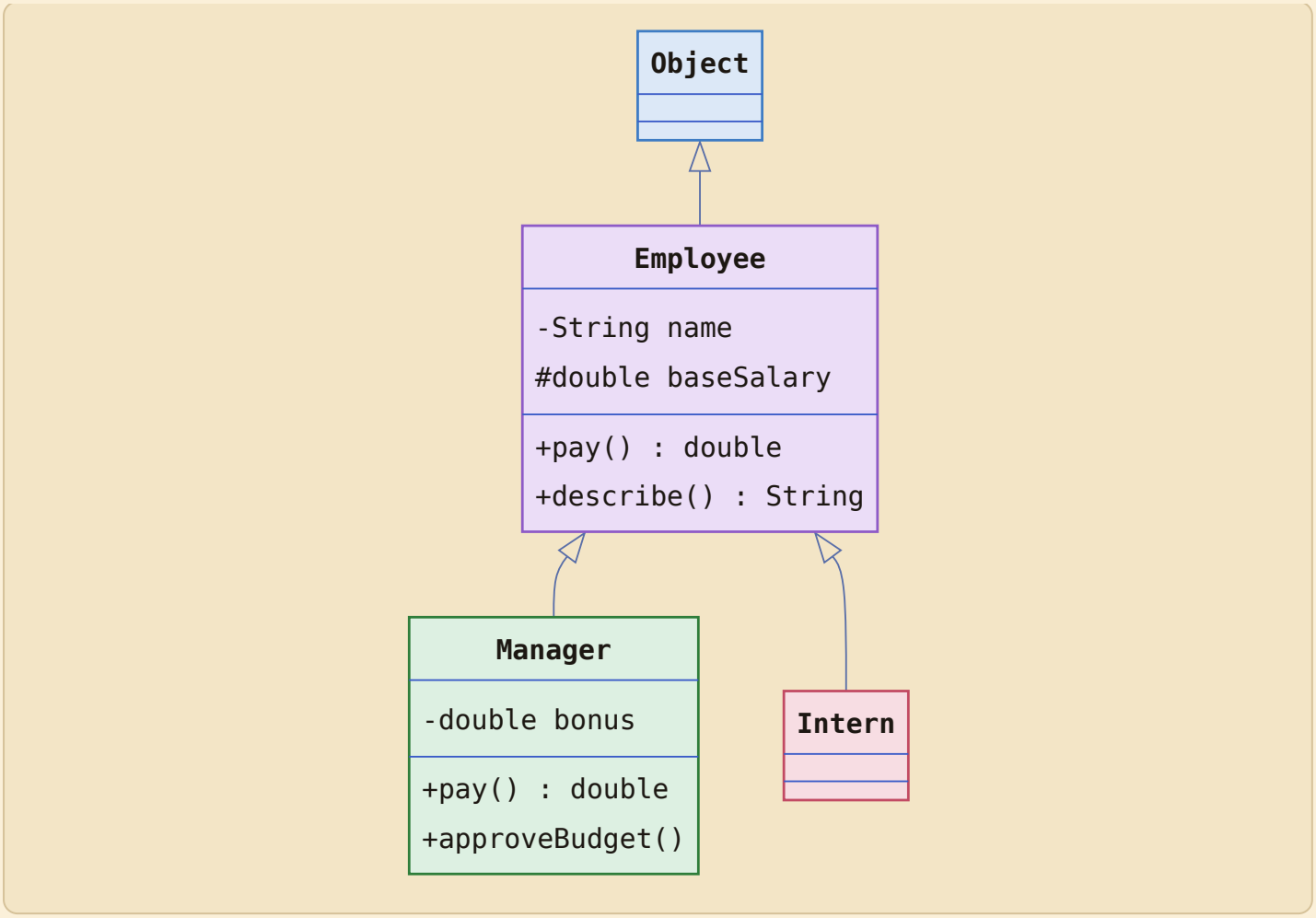
3. **Constructors are not inherited.** Each subclass constructor must initialise the parent part first by calling a superclass constructor with `super(args)`. If you don't write one, the compiler inserts `super()` — which fails to compile if the parent has no no-argument constructor.
4. `super.method()` calls the parent's version of an overridden method — handy for extending rather than replacing behaviour.
5. **What the subclass can use:** `public` and `protected` members always; package-private members only if it's in the same package. `private` members exist inside the object but are not accessible — use the parent's public/protected methods to reach them.
6. A class marked `final` cannot be extended (`String`, `Integer`, records).

```
// code/ch06/inheritance/InheritanceDemo.java (key parts)
static class Employee {
    private final String name;              // private: not accessible in subclasses
    protected double baseSalary;            // protected: accessible in subclasses
    Employee(String name, double baseSalary) { this.name = name; this.baseSalary = baseSalary; }
    String getName() { return name; }
    double pay() { return baseSalary; }
    String describe() { return getName() + " earns " + pay(); }
}

static class Manager extends Employee {     // Manager IS-AN Employee
    private final double bonus;
    Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary);            // initialise the Employee part first
        this.bonus = bonus;
    }
    @Override double pay() { return super.pay() + bonus; }   // extend the parent's version
    void approveBudget() { System.out.println(getName() + " approved the budget"); }
}

static class Intern extends Employee {
    Intern(String name) { super(name, 1_000); }             // inherits pay() unchanged
}
```

Output:

```
Ana earns 3000.0
Bo earns 6500.0
Cy earns 1000.0
Bo approved the budget
m instanceof Employee? true, superclass of Manager: Employee, superclass of Employee: Object
```

Note line 2: `describe()` is written once in `Employee`, but for a `Manager` it calls `Manager.pay()`. That's **dynamic dispatch** (§6.3–6.4) — the heart of polymorphism.



Trade-offs — inheritance vs composition

	Inheritance (`extends`)	Composition (hold a reference to another object)
Relationship	is-a	has-a / uses-a
Coupling (how strongly one piece of code depends on another)	Tight: subclass depends on parent's implementation details	Loose: depends only on the other object's public API (the methods it exposes)
Flexibility	Fixed at compile time; one parent only	Can swap the part at runtime; combine many
Risk	**Fragile base class** — a change in the parent can break subclasses	Some forwarding boilerplate (repetitive code that just passes calls on)

Effective Java Item 18 — "favour composition over inheritance" — and Item 19 — "design and document for inheritance or else prohibit it" (make classes `final` unless designed for extension). The full treatment, with worked examples, is in LLD §2.11 Composition over inheritance and LLD §3.6 Generalization.

When not to use it

- Only to reuse code, when the "is-a" sentence is false (`Stack extends Vector` — `Vector` is an old, synchronised list class — in the JDK is the classic regret: a stack exposes `insertElementAt`).
- When a subclass would need to *disable* inherited behaviour (throwing `UnsupportedOperationException` from overrides is a Liskov violation — LLD §4.8).
- Across package/team boundaries on classes not designed for it.

Common mistakes

- Parent has only `Employee(String, double)`; subclass constructor forgets `super(...)` → "constructor Employee in class Employee cannot be applied to given types".
- Expecting to access a parent's `private` field from the child.
- Deep hierarchies (5+ levels) — hard to follow; prefer shallow hierarchies plus interfaces.

Self-check

1. How many classes can a class extend directly? → One.
2. Are constructors inherited? → No; each subclass constructor calls one via `super(...)` (explicitly or implicitly).
3. What is the superclass of a class with no `extends`? → `java.lang.Object`.

TL;DR

`extends` = is-a, single parent, everything ultimately extends `Object`. `super(...)` builds the parent part first; `super.m()` reuses the parent's method. Prefer composition unless the class is designed for extension.

6.2 What a subclass gets — members, hiding and `Object`

A quick reference of what inheritance does with each kind of member:

Member in parent	In the subclass
`public` / `protected` instance method	Inherited; may be **overridden**
package-private instance method	Inherited only if subclass is in the same package
`private` method	Not inherited (a same-named method in the child is a *new* method, not an override)
`final` method	Inherited; cannot be overridden
`static` method	Inherited (callable via the subclass name); a same-signature static method in the child **hides** it
Field (static or instance)	Inherited if accessible; a same-named field in the child **hides** it (both exist)
Constructor	**Never** inherited
Static/instance initialiser blocks	Not inherited; the parent's still run during construction (Chapter 5 §5.10)

`java.lang.Object` gives every class these methods:

Method	Purpose	Override?
`toString()`	Text form; default `ClassName@hexHash`	Usually yes (§6.13)
`equals(Object)`	Logical equality; default is identity (`==`, "the very same object")	For value-like classes (§6.13)
`hashCode()`	Hash (an `int` used to place objects) for `HashMap`/`HashSet` (hash-based lookup collections); must match `equals`	Whenever you override `equals`
`getClass()`	Runtime class object	`final`, can't
`clone()`	Field-by-field copy (needs the `Cloneable` marker interface)	Rarely; prefer copy constructors (constructors that copy another instance)/factories (*Effective Java*, Item 13)
`wait()`, `notify()`, `notifyAll()`	Low-level thread (independent path of execution) coordination	`final`; see Chapter 14
`finalize()`	Cleanup before GC (garbage collection)	Deprecated for removal (JEP 421, Java 18) — use try-with-resources / `Cleaner`

6.3 Overriding

What it is

Overriding means a subclass provides its own implementation of an inherited **instance** method with the **same signature** (same name and parameter types). At runtime, the JVM calls the version that belongs to the object's **actual (runtime) class**, not the variable's declared type. This is **dynamic dispatch**, also called *runtime polymorphism* (contrast with overloading, which is resolved at compile time — Chapter 4 §4.3).

The rules

For `Dog.m` to override `Animal.m`:

#	Rule	Example that fails
1	Same name and same parameter types	`speak(String)` vs `speak()` is an overload, not an override
2	Return type the same, or a **subtype** (covariant return — a more specific return type) for reference types; identical for primitives	Returning `Object` where parent returns `Animal`
3	Access **the same or wider** (`protected` → `public` is fine)	`private Dog reproduce()` overriding a `protected` method
4	Checked exceptions (ones the compiler forces callers to handle): none new, none broader (fewer or narrower is fine); unchecked (`RuntimeException` and its subclasses) are unrestricted	Parent throws `IOException`, child declares `Exception`
5	Can't override `final` methods	`String dna()` when parent's is `final`
6	`static` methods are **hidden**, not overridden; `private` methods aren't inherited at all	—
7	An instance method cannot "override" a static one, or vice versa	compile error

Always write `@Override` (an annotation — a marker the compiler reads) — the compiler then *checks* that you really override something. Without it, a typo (`tostring()`) or a wrong parameter type (`equals(Book)`

instead of `equals(Object)`) silently creates a new method.

```
// code/ch06/overriding/OverridingRules.java (key parts)
static class Animal {
    protected Animal reproduce() { return new Animal(); }
    void speak() throws IOException { System.out.println("..."); }
    static String kind() { return "Animal (static)"; }
    String name = "animal field";
    final String dna() { return "ATGC"; }
}
static class Dog extends Animal {
    @Override public Dog reproduce() { return new Dog(); }    // covariant return + wider access
    @Override void speak() { System.out.println("Woof"); }    // throws fewer checked exceptions
    static String kind() { return "Dog (static)"; }           // HIDES Animal.kind()
    String name = "dog field";                                // HIDES Animal.name
    // String dna() {...}        -> compile error: final
    // private Dog reproduce()   -> compile error: weaker access
}

static class Base {
    Base() { init(); }                    // calls an overridable method: dangerous
    void init() { }
}
static class Derived extends Base {
    private String label = "ready";
    @Override void init() { System.out.println("Derived.init sees label = " + label); }
}

Animal a = new Dog();                     // declared type Animal, runtime type Dog
a.speak();
System.out.println("reproduce() -> " + a.reproduce());
System.out.println("a.kind()    -> " + a.kind() + "   (static method: picked by the declared type)");
System.out.println("Dog.kind()  -> " + Dog.kind());
System.out.println("a.name      -> " + a.name + " (fields use the STATIC type)");
System.out.println("((Dog) a).name -> " + ((Dog) a).name);
new Derived();
```

Output:

```
Woof
reproduce() -> Dog
a.kind()    -> Animal (static)   (static method: picked by the declared type)
Dog.kind()  -> Dog (static)
a.name      -> animal field (fields use the STATIC type)
((Dog) a).name -> dog field
Derived.init sees label = null
```

What it shows:

- **Instance methods** are chosen by the **runtime** type (`Woof`).
- **Static methods and fields** are chosen by the **declared (compile-time)** type — that is *hiding*. Calling a static method through an instance (`a.kind()`) compiles, but is misleading; always call statics via the class name.
- **Never call overridable methods from a constructor.** `Base()` called `Derived.init()` *before* `Derived`'s field initialiser ran, so `label` was still `null` (see the initialisation order in Chapter 5 §5.10). *Effective Java*, Item 19. Make such methods `private`, `final` or `static`.

Common mistakes

- Overloading by accident: `public boolean equals(Book b)` does **not** override `equals(Object)`; `HashSet` will never call it. `@Override` catches this.
- Expecting field access to be polymorphic — it isn't; that's another reason fields should be private.
- Narrowing access in an override, or adding a checked exception.

In the interview

> **30-second answer:** "Overriding is a subclass redefining an inherited instance method with the same signature; the JVM picks the implementation from the object's runtime class — dynamic dispatch. The override may return a subtype, must not reduce visibility or add broader checked exceptions, and can't override final methods. Static methods and fields are hidden, not overridden — they're resolved by the declared type. I always use @Override."

- *Follow-up: Overloading vs overriding?* → Overloading: same name, different parameters, same or related class, chosen at compile time by argument types. Overriding: same signature in a subclass, chosen at runtime by object type.
- *Follow-up: Can you override a private method?* → No. It isn't inherited; a same-named method in the subclass is unrelated.

Self-check

1. Parent: `Number get()`. Child: `Integer get()` — legal? → Yes, covariant return.
2. Parent: `public void run()`. Child: `void run()` — legal? → No, package-private is narrower than public.
3. `Animal a = new Dog(); a.name` — which field? → `Animal`'s: fields use the declared type.

TL;DR

Override = same signature, compatible return, same-or-wider access, no broader checked exceptions, `@Override`. Instance methods dispatch on runtime type; statics and fields on declared type. Don't call overridable methods from constructors.

6.4 Polymorphism and casting

What it is

Polymorphism ("many forms") means one piece of code can work with objects of many different classes through a common supertype (a parent class or interface they all share), and each object responds in its own way. You write `shape.area()` once; circles, rectangles and squares each compute their own area.

Key vocabulary:

- **Declared (static) type** — the type of the variable: `Shape s`.
- **Runtime (dynamic) type** — the class of the actual object: `new Circle(1)`.
- **Upcasting** — treating a subclass object as its supertype (`Shape s = new Circle(1)`). Always safe, always implicit.
- **Downcasting** — treating a supertype reference as a subtype (`(Rect) s`). Needs an explicit cast and is **checked at runtime**; wrong casts throw `ClassCastException`.

How it works

1. The compiler decides **which methods you may call** from the declared type: `s.side()` doesn't compile if `s` is a `Shape`, even if the object is a `Square`.
2. The JVM decides **which implementation runs** from the runtime type (dynamic dispatch).
3. To use subtype-only methods, check and cast. Since Java 16 the idiomatic way is **pattern matching for `instanceof`**: `if (s instanceof Square sq) { sq.side(); }` — the test and the cast in one step, with a new *pattern variable* `sq` in scope only where the test is true (flow-scoping rules: Chapter 3 §3.5).
4. The compiler rejects casts that can *never* succeed (e.g. `(String) someInteger` between unrelated classes); casts that *might* succeed are compiled and checked at runtime.

```java
// code/ch06/polymorphism/Shapes.java (key parts)
static class Shape  { double area() { return 0; } String name() { return getClass().getSimpleName(); } }
static class Circle extends Shape { final double r; ... @Override double area() { return Math.PI * r * r; } }
static class Rect   extends Shape { final double w, h; ... @Override double area() { return w * h; } }
static class Square extends Rect  { Square(double side) { super(side, side); } double side() { return w; } }

List<Shape> shapes = List.of(new Circle(1), new Rect(2, 3), new Square(2));   // upcasting
for (Shape s : shapes) System.out.printf("%-6s area = %.2f%n", s.name(), s.area());

Shape s = shapes.get(2);
// s.side();              -> compile error: Shape has no side()
if (s instanceof Square sq) System.out.println("pattern variable: side = " + sq.side());
Rect r = (Rect) s;                              // downcast: OK, a Square is a Rect
Circle c = (Circle) s;                          // compiles, ClassCastException at runtime
```

Output:

```
Circle area = 3.14
Rect   area = 6.00
Square area = 4.00
total = 13.14
pattern variable: side = 2.0
downcast to Rect ok: w=2.0
ClassCastException: class ch06.polymorphism.Shapes$Square cannot be cast to class ch06.polymorphism.Shapes$Circle
```

Why it matters

Polymorphism is what makes code **open for extension**: add a `Triangle` class and the loop works unchanged. This is the Open/Closed Principle (code should be open to extension but closed to modification) and the basis of most design patterns (Strategy, State, Command…) — see LLD §2.6 Polymorphism and LLD §4.7 OCP.

When not to use casting

Chains of `if (x instanceof A) … else if (x instanceof B) …` over an *open* hierarchy are usually a sign the behaviour belongs in an overridden method. The modern exception: a **sealed** hierarchy with a `switch` over patterns (§6.7), where the compiler checks that every case is handled.

Common mistakes

- Downcasting without checking → `ClassCastException`.
- Using the pattern variable outside its scope: in `if (!(s instanceof Square sq)) return; sq.side();` the variable *is* in scope after the `if` (flow scoping), but in an `else` branch of a positive test it isn't.
- Calling subtype methods on a supertype variable and being surprised it won't compile — the compiler only knows the declared type.

Self-check

1. Which type decides what methods you may call? Which decides what code runs? → Declared type; runtime type.
2. Does `Object o = "hi"; Integer i = (Integer) o;` compile? Run? → Compiles; throws `ClassCastException`.
3. What does `x instanceof Foo f` do when `x` is `null`? → Evaluates to `false`; `instanceof` is never true for `null`.

TL;DR

Code against the supertype; the runtime type picks the implementation. Upcasts are free; downcasts are checked — prefer `instanceof` patterns, and prefer overriding to type checks.

6.5 Abstract classes

What it is

An **abstract class** is a class marked `abstract` that **cannot be instantiated** and may contain **abstract methods** — methods declared without a body, which concrete subclasses must implement. It's a *partially built* class: shared state and code live in it; the missing pieces are left to subclasses.

Analogy: a form template with some fields pre-printed (shared code) and some blank boxes (abstract methods) that each department fills in.

How it works

1. `abstract class Report { … }` — `new Report()` is a compile error.
2. `protected abstract String body();` — no braces; ends with `;`.
3. A class with **any** abstract method must itself be declared `abstract`. An abstract class may also have **zero** abstract methods (just to prevent instantiation).
4. The first **concrete** (non-abstract) subclass must implement every inherited abstract method; an abstract subclass may leave them for later.
5. Abstract classes can have everything a normal class has: fields (including mutable state), constructors (called via `super(...)` from subclasses), static members, `final` methods.
6. `abstract` cannot be combined with `final` (can't be extended), `private` (can't be overridden) or `static` on a method.

A classic use is the **Template Method** pattern: a `final` method fixes the skeleton of an algorithm and calls abstract "steps" that subclasses fill in.

```
// code/ch06/abstracts/AbstractDemo.java (key parts)
static abstract class Report {
    private final String title;
    protected Report(String title) { this.title = title; }
    public final String render() { return header() + "\n" + body() + "\n" + footer(); }   // template method
    protected String header() { return "== " + title + " =="; }   // default step
    protected abstract String body();                             // required step
    protected String footer() { return "-- end --"; }
}
static class SalesReport extends Report {
    private final int units;
    SalesReport(int units) { super("Sales"); this.units = units; }
    @Override protected String body() { return "units sold: " + units; }
}
static class AuditReport extends Report {
    AuditReport() { super("Audit"); }
    @Override protected String body() { return "no issues found"; }
    @Override protected String footer() { return "-- signed: auditor --"; }
}

Report[] reports = { new SalesReport(42), new AuditReport() };
for (Report r : reports) System.out.println(r.render());
Report adHoc = new Report("Ad hoc") {                  // anonymous subclass (§6.11)
    @Override protected String body() { return "one-off content"; }
};
```

Output:

```
== Sales ==
units sold: 42
-- end --
== Audit ==
no issues found
-- signed: auditor --
== Ad hoc ==
one-off content
-- end --
```

Real-world examples

`java.util.AbstractList` (implement `get` and `size`, get a whole `List`), `java.io.InputStream` (implement `read()`), `HttpServlet` (the base class for handling web requests; override `doGet`/`doPost`). Template Method in depth: LLD §11.4 Template Method.

Self-check

1. Can an abstract class have a constructor? → Yes; subclasses call it with `super(...)`.
2. Can a class be `abstract` with no abstract methods? → Yes.
3. Can an abstract method be `private`? → No — nobody could implement it.

TL;DR

Abstract class = shared state + shared code + abstract "holes"; can't be instantiated; subclasses fill the holes. Great for template methods and skeletal implementations.

6.6 Interfaces

What it is

An **interface** is a named **contract**: a set of method signatures that implementing classes promise to provide. A class `implements` any number of interfaces; code can then depend on the interface type instead of a concrete class ("program to an interface" — LLD principle 7).

Analogy: a power socket standard. Any appliance with the right plug (implementation) works with any socket (caller) — neither needs to know the other's internals.

What an interface may contain

Member	Since	Implicit modifiers	Notes
Abstract method	Java 1.0	`public abstract`	The contract; implementing classes must provide it
Constant field	1.0	`public static final`	Must be initialised; interfaces have **no instance fields**
`default` method	Java 8	`public`	Has a body; inherited by implementers, which may override it. Lets APIs evolve without breaking implementers (e.g. `List.sort`, `Collection.stream`)
`static` method	Java 8	`public` (if not marked `private`)	Called as `InterfaceName.m()` only — **not** inherited by implementing classes
`private` method (instance or static)	Java 9	—	Helper for default/static methods; not visible outside
Nested types	1.0	`public static`	e.g. `Map.Entry`

No constructors, no instance state. An interface can `extend` several other interfaces.

How it works

1. Declare: `interface Payable { double amount(); }`.
2. Implement: `class Invoice implements Payable, Printable, Shippable { … }` — must implement all abstract methods (or be `abstract`). Implementations must be `public` (the interface methods are public).
3. Use the interface as a type: `List<Payable> bills`.
4. An interface with **exactly one abstract method** is a **functional interface** and can be implemented by a **lambda** (a short inline function, `() -> value`) — details in Chapter 11. `@FunctionalInterface` asks the compiler to check this.

Multiple inheritance and the diamond rules

The name comes from the *diamond* shape you get when one class inherits the same method along two different paths. A class can inherit the same default method from two interfaces. Java resolves conflicts with three rules:

1. **Classes win.** A method declared in the class or any superclass beats any interface default.
2. **More specific interface wins.** If `B extends A` and both define `m()`, `B`'s version wins.
3. **Otherwise it's a compile error** until the class overrides the method. Inside the override, call a specific parent with `InterfaceName.super.m()`.

```
// code/ch06/interfaces/InterfacesDemo.java (key parts)
interface Payable {
    double TAX_RATE = 0.2;                          // public static final
    double amount();                                // public abstract
    default double amountAfterTax() { return round(amount() * (1 - TAX_RATE)); }   // Java 8
    static Payable of(double value) { return () -> value; }                        // Java 8, lambda
    private static double round(double v) { return Math.round(v * 100) / 100.0; } // Java 9
}
interface Printable { default String label() { return "printable"; } }
interface Shippable { default String label() { return "shippable"; } }

static class Invoice implements Payable, Printable, Shippable {
    private final double total;
    Invoice(double total) { this.total = total; }
    @Override public double amount() { return total; }
    @Override public String label() {                // conflict: must override
        return Printable.super.label() + "+" + Shippable.super.label();
    }
}

bills.add(new Invoice(100));
bills.add(Payable.of(59.99));
bills.add(new Payable() {                           // anonymous implementation
    public double amount() { return 10; }
    public double amountAfterTax() { return 10; }     // overrides the default
});
```

Output:

```
100.0 -> after tax 80.0
59.99 -> after tax 47.99
10.0 -> after tax 10.0
label: printable+shippable, TAX_RATE = 0.2
Invoice is a Printable? true
```

Real-world examples

`Comparable<T>`, `Comparator<T>`, `Runnable`, `AutoCloseable` (try-with-resources), `Iterable<T>` (for-each), `List`/`Set`/`Map` (Chapter 10), `java.util.function.*` (Chapter 11). JDBC (Java Database Connectivity, the standard database API) is almost entirely interfaces (`Connection`, `ResultSet`) implemented by each database driver (the library that talks to one specific database product).

Common mistakes

- Implementing an interface method with default (package) access → "attempting to assign weaker access privileges; was public".
- Trying to call an interface's static method through an implementing class or instance (`invoice.of(1)`) — static interface methods aren't inherited.
- Using an interface only to hold constants ("constant interface" anti-pattern — a common but harmful practice — *Effective Java* Item 22) — use a final class or an enum.
- Putting state-like logic into default methods; defaults can't hold state and are meant for convenience/evolution.

Self-check

1. Can an interface have a constructor? Instance fields? → No and no.
2. Two interfaces both provide `default void hello()`; a class implements both. What happens? → Compile error until the class overrides `hello()`; it may call `A.super.hello()`.
3. Is an interface's `static` method inherited by implementing classes? → No.

TL;DR

Interfaces = contracts. Abstract + constants + `default` (8) + `static` (8) + `private` (9) methods, no state. Many per class. Conflicts: class wins → more specific interface wins → else override and use `X.super.m()`.

6.7 Sealed classes and interfaces (Java 17)

What it is

A **sealed** class or interface restricts **which** classes may extend or implement it, using a `permits` clause. It was finalised in **Java 17 (JEP 409)**. Sealing lets you model "a `Shape` is exactly a `Circle`, a `Rectangle` or a `Polygon` — nothing else", and lets the compiler use that knowledge.

Analogy: a club with a fixed guest list. The bouncer (compiler) knows everyone who can possibly be inside.

Why it matters

- Between "anyone may extend" (normal class) and "nobody may extend" (`final`), sealing gives a **closed, known set** of subtypes — ideal for domain (the real-world area the software models) concepts like payment methods, AST (abstract syntax tree — the tree a compiler builds from code) nodes, or API results (`Success | Failure`).
- It makes `switch` **over patterns exhaustive**: if every permitted subtype is handled, no `default` is needed — and if you later add a subtype, every non-exhaustive `switch` fails to compile, pointing you at the code to update.

How it works

1. Declare: `sealed interface Shape permits Circle, Rectangle, Polygon {}` (or `sealed class`).
2. Each permitted subclass must directly extend/implement it and must be exactly one of:
   - `final` — no further subclasses (records and enums are implicitly final),
   - `sealed` — with its own `permits` list,
   - `non-sealed` — reopens the hierarchy below it to any subclass.
3. Permitted subclasses must be in the **same module** (a named group of packages under the Java module system) — or, if the code is not in a named module, in the **same package**.
4. If all subclasses are in the same source file, the `permits` clause can be omitted; the compiler infers it.
5. Combine with **records** (§6.8) and **pattern matching for** `switch` (Java 21, JEP 441) including **record patterns** (Java 21, JEP 440) — see Chapter 3 for the `switch` syntax.

```java
// code/ch06/sealed/SealedDemo.java (key parts)
sealed interface Shape permits Circle, Rectangle, Polygon {}
record Circle(double radius) implements Shape {}                // implicitly final
record Rectangle(double w, double h) implements Shape {}
static non-sealed class Polygon implements Shape {               // open again below this point
    final int sides;
    Polygon(int sides) { this.sides = sides; }
}
static final class Triangle extends Polygon { Triangle() { super(3); } }

static String describe(Shape s) {
    return switch (s) {                                          // exhaustive: no default needed
        case Circle c                                  -> "circle r=" + c.radius();
        case Rectangle(double w, double h) when w == h -> "square " + w;          // record pattern + guard
        case Rectangle(double w, double h)             -> "rectangle " + w + "x" + h;
        case Polygon p                                 -> p.sides + "-sided polygon";
    };
}
```

Output:

```
circle r=1.5
square 2.0
rectangle 2.0x5.0
3-sided polygon
Shape is sealed? true, permitted: 3
```

Sealed interfaces + records give Java **algebraic data types** (types built by combining "and" and "or"): records are the "and" (a rectangle has a width *and* a height), sealed interfaces are the "or" (a shape is a circle *or* a rectangle *or* …). This style is sometimes called *data-oriented programming*.

Trade-offs

Use sealed when…	Avoid when…
The set of variants is part of the domain and changes rarely	You're building an extension point for other teams/plugins (they can't add subtypes)
You want exhaustive `switch` checks	New variants are added often and callers shouldn't need updates — then prefer overriding methods (classic polymorphism)

Self-check

1. What three modifiers can a permitted subclass have? → `final`, `sealed` or `non-sealed`.
2. Why doesn't the `switch` in `describe` need a `default`? → The compiler knows all permitted subtypes and each is covered.
3. When can you omit `permits`? → When all permitted subclasses are declared in the same source file.

TL;DR

`sealed … permits A, B, C` = closed set of subtypes (Java 17). Subtypes are `final`/`sealed`/`non-sealed`. Pair with records and pattern `switch` for exhaustive, compiler-checked handling.

6.8 Records

What it is

A **record** (Java 16, JEP 395) is a special kind of class for **transparent, immutable data carriers** (their data is fully visible and can't change after creation). You declare the **components** in the header; the compiler generates the rest:

`record Point(int x, int y) {}` gives you:

1. `private final` fields `x` and `y`,

2. a **canonical constructor** `Point(int x, int y)`,
3. **accessor** methods `x()` and `y()` (no `get` prefix),
4. `equals` and `hashCode` based on all components,
5. `toString()` like `Point[x=3, y=4]`.

What would be ~40 lines of a hand-written immutable class (like `Money` in Chapter 5 §5.9) becomes one line.

Rules

Records can	Records cannot
Implement interfaces	Extend another class (they implicitly extend `java.lang.Record`)
Declare static fields and methods, static factories	Declare **instance** fields beyond the components
Declare instance methods	Be extended — they're implicitly `final`
Add constructors — which must delegate to the canonical one with `this(...)`	Have setters that change components (fields are `final`)
Override any generated member (`toString`, accessor, `equals`)	Be `abstract`
Be nested (implicitly `static`) or **local** inside a method	
Be generic (take type parameters such as `<A, B>`): `record Pair<A, B>(A first, B second) {}`	

How it works — the compact constructor

To validate or normalise (convert to a standard form), write a **compact canonical constructor**: the record name with **no parameter list**. Its parameters are implicit, and the fields are assigned automatically **after** its body runs. You can reassign the *parameters* (e.g. to make a defensive copy — a private copy of a mutable input so outside changes can't affect the record), but you don't assign `this.x` yourself.

```
// code/ch06/records/RecordsDemo.java (key parts)
record Point(int x, int y) {
    static final Point ORIGIN = new Point(0, 0);
    Point translate(int dx, int dy) { return new Point(x + dx, y + dy); }
    double distanceTo(Point o) { return Math.hypot(x - o.x, y - o.y); }
}

record Range(int start, int end) {
    Range {                                              // compact canonical constructor
        if (start > end) throw new IllegalArgumentException(start + " > " + end);
    }
    Range(int single) { this(single, single); }          // extra constructors must delegate
    int length() { return end - start; }
}

record Team(String name, List<String> members) {
    Team { members = List.copyOf(members); }             // defensive copy: reassign the PARAMETER
}

record Temperature(double celsius) {
    @Override public String toString() { return celsius + "°C"; }
    static Temperature ofFahrenheit(double f) { return new Temperature((f - 32) * 5 / 9); }
}

// inside main
record Pair(String key, int count) {}                    // local record
```

Output:

```
Point[x=3, y=4] x=3 dist=5.0 moved=Point[x=4, y=5]
equals by value: true, set size: 1
Range[start=2, end=7] length 5, Range[start=5, end=5]
rejected: 9 > 1
Team[name=Core, members=[Ana, Bo]] (unchanged by the caller's add)
100.0°C
local record: [Pair[key=a, count=1]]
```

Why it matters / Real-world examples

DTOs (data transfer objects) for REST APIs (web services that expose resources at URLs and use HTTP verbs such as GET and POST — see System Design Fundamentals), query results, map keys (`record CacheKey(String user, LocalDate day)`), multiple return values (`record MinMax(int min, int max)`), events and messages, and the "and" half of sealed hierarchies. Jackson (≥ 2.12; a JSON library) serialises (converts to and from text) records; Spring Boot (a popular application framework) binds configuration to them.

Trade-offs — record vs regular class

Choose a record when…	Choose a class when…
The type *is* its data: two instances with equal components are interchangeable	Objects have identity and a lifecycle (a `BankAccount`, a `User` session)
Immutable is fine	You need mutable state or JPA entities (objects that JPA, the Java Persistence API, maps to database tables; JPA requires a no-arg constructor and non-final fields)
All components should be public via accessors	You need to hide the representation

Common mistakes

- Believing records are **deeply** immutable. Components are final references; a `List` or array component can still be mutated unless you copy it (`List.copyOf`) in the compact constructor. Arrays in records also break value-equality (arrays use identity `equals`).
- Assigning `this.start = start` inside a compact constructor — compile error; assign the parameter instead, or nothing.
- Writing `getX()` out of habit — the accessor is `x()`.

In the interview

30-second answer: "A record is a final, immutable data carrier (Java 16). From the component list the compiler generates private final fields, the canonical constructor, accessors, and equals/hashCode/toString based on all components. I use a compact constructor for validation and defensive copies. Records can implement interfaces and have static members and methods, but can't extend classes or add instance fields. They pair with sealed interfaces and record patterns for data-oriented code."

- *Follow-up: Are records deeply immutable?* → Only if their components are; copy mutable components in the compact constructor.
- *Follow-up: Can a record be a JPA entity?* → Not with standard JPA (needs no-arg constructor and mutable proxies — generated stand-in subclasses); records work well as DTOs/projections (read-only views of selected columns) instead.

Self-check

1. What's the accessor for component `name`? → `name()`.
2. Where are fields assigned when you use a compact constructor? → Automatically, after the compact constructor body.

3. Can a record extend another record? → No; records are final and implicitly extend `java.lang.Record`.

TL;DR

`record R(a, b) {}` = final immutable data class with constructor, accessors, `equals`/`hashCode`/`toString`. Validate and copy in the compact constructor. Not deeply immutable by default.

6.9 Enums

What it is

An **enum** (enumerated type, Java 5) is a class with a **fixed set of named instances** — its constants — created once when the enum class initialises: `enum Day { MON, TUE, … }`. Each constant is a full object (a singleton — the one and only instance — of its kind), so enums can have fields, constructors, methods, and even per-constant behaviour.

Analogy: a restaurant menu printed once. The dishes are fixed; you can't invent a new dish at the table, but each dish carries its own price and description.

Why it matters

Compared to `int` or `String` constants (`static final int MONDAY = 0`):

- **Type safety** — a method taking `Day` can't be passed `42` or `"Mnday"`.
- **Namespaced, printable** — `Day.MON` prints `MON`.
- **Can carry data and behaviour** — a `Planet` knows its mass; an `Operation` knows how to apply itself.
- **Safe with `==`** — each constant exists exactly once (per class loader, the JVM component that loads classes).
- **Great tooling** — `switch`, `EnumSet`, `EnumMap`.

How it works

1. Every enum implicitly extends `java.lang.Enum`, so it **can't extend another class**, but it **can implement interfaces**.
2. Constants are listed first, separated by commas; if anything follows, end the list with `;`.
3. Constructors are implicitly **`private`** — you can't `new` an enum. Each constant's arguments are passed to it: `EARTH(5.976e+24, 6.37814e6)`.
4. Built-in methods:

Method	Returns
`values()` (static)	A **new** array of all constants in declaration order
`valueOf(String)` (static)	Constant with that exact name, or `IllegalArgumentException`
`name()`	The declared name (`"FRI"`) — use this for persistence (saving to a database or file) rather than `toString`, which can be overridden
`ordinal()`	Zero-based position — don't persist or rely on it (*Effective Java* Item 35); reordering constants changes it
`compareTo(e)`	Order by ordinal (enums are `Comparable`, i.e. have a natural order)

5. **Constant-specific bodies:** a constant can have its own class body `{ … }` to override methods — commonly implementing an `abstract` method declared in the enum.
6. **`switch`** on an enum uses bare constant names (`case SAT, SUN ->`). A `switch` *expression* must be exhaustive; covering all constants satisfies that without `default`.
7. **`EnumSet`** (bit-vector set — one bit per constant) and **`EnumMap`** (array-backed map) are the fastest, most compact collections for enum keys.

8. A single-element enum is the simplest robust **singleton** (*Effective Java*, Item 3) — safe against reflection (code that inspects and creates objects at run time) and serialisation (turning objects into bytes and back) tricks. See LLD Singleton.

```
// code/ch06/enums/EnumsDemo.java (key parts)
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN;
    boolean isWeekend() { return this == SAT || this == SUN; }
}
enum Planet {
    MERCURY(3.303e+23, 2.4397e6), EARTH(5.976e+24, 6.37814e6), JUPITER(1.9e+27, 7.1492e7);
    private static final double G = 6.67300E-11;
    private final double mass, radius;
    Planet(double mass, double radius) { this.mass = mass; this.radius = radius; }   // implicitly private
    double surfaceGravity() { return G * mass / (radius * radius); }
}
enum Operation {
    PLUS("+")  { int apply(int a, int b) { return a + b; } },
    TIMES("*") { int apply(int a, int b) { return a * b; } };
    final String symbol;
    Operation(String symbol) { this.symbol = symbol; }
    abstract int apply(int a, int b);
}
static String plan(Day d) {
    return switch (d) {
        case SAT, SUN -> "rest";
        case FRI -> "wrap up";
        case MON, TUE, WED, THU -> "work";
    };
}
EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
EnumSet<Day> weekdays = EnumSet.complementOf(weekend);
Map<Day, Integer> hours = new EnumMap<>(Day.class);
```

Output:

```
MON(0) TUE(1) WED(2) THU(3) FRI(4) SAT(5)* SUN(6)*
FRI -> wrap up, compareTo(MON) = 4, == works: true
valueOf("Friday") -> IllegalArgumentException
MERCURY  g =  3.70 m/s^2
EARTH    g =  9.80 m/s^2
JUPITER  g = 24.81 m/s^2
6 + 7 = 13
6 * 7 = 42
weekdays=[MON, TUE, WED, THU, FRI] hours={MON=8, TUE=8, WED=8, THU=8, FRI=6}
```

Real-world examples

`java.time.DayOfWeek`, `Month`, `ChronoUnit`; `java.util.concurrent.TimeUnit` (each constant converts durations); `RetentionPolicy`; order/state machines (models where something moves between a fixed set of states: `OrderStatus.PLACED → SHIPPED`, see LLD state machines); HTTP methods (`GET`, `POST`… — the verbs of web requests) in web frameworks. Enums with fields and behaviour in design: LLD §2.10.

Common mistakes

- Persisting `ordinal()` to a database — inserting a new constant in the middle silently corrupts data. Store `name()` or an explicit code field.
- `valueOf` with user input without handling `IllegalArgumentException` (and case differences).
- Mutable fields in enums — they're global singletons; mutable state there is shared global state.
- Calling `values()` in a hot loop — it allocates a new array each time; cache it in a `static final` field if needed.

Self-check

1. Can an enum extend a class? Implement an interface? → No; yes.
2. What's wrong with storing `ordinal()`? → It changes when constants are reordered or inserted.
3. Why can you compare enums with `==`? → Each constant is a single instance.

TL;DR

Enums = fixed set of singleton instances with type safety. They can have fields, private constructors, methods and per-constant bodies. Use `name()` not `ordinal()` for storage; `EnumSet`/`EnumMap` for collections; exhaustive `switch` without `default`.

6.10 Nested classes: static nested and inner

What it is

A **nested class** is a class declared inside another class. There are four kinds:

Kind	Declared	Tied to an outer *instance*?	Typical use
Static nested class	As a member, with `static`	No	Helpers that belong with the outer class: `Map.Entry`, builders, `Node` in a linked list
Inner class (non-static member)	As a member, no `static`	**Yes** — each instance holds a hidden reference to an outer instance	Iterators (objects that step through a collection) and views that read the outer object's state
Local class	Inside a method/block	Yes if declared in an instance context	One-off helper used only in that method (§6.11)
Anonymous class	Inside an expression, no name	Yes if in an instance context	One-off implementation of an interface/abstract class (§6.11)

Nested records, enums and interfaces are always implicitly `static`.

How it works

1. Nested classes can access **all** members of the enclosing class, including `private` ones — and vice versa.
2. Create a static nested class like any class: `new Outer.Node(...)` (or `new Node(...)` inside `Outer`).
3. Create an inner class **from an outer instance**: `outer.new ReverseIterator()`. Inside an instance method of `Outer`, plain `new ReverseIterator()` uses `this` as the outer instance.
4. Inside an inner class, `this` is the inner object; `Outer.this` is the enclosing object — needed when names clash.
5. Since Java 16, inner classes may also declare `static` members.

```java
// code/ch06/nested/NestedDemo.java (key parts)
static class Node {                                  // static nested
    final String value;
    Node(String value) { this.value = value; }
}

private final List<String> items = new ArrayList<>(List.of("a", "b", "c"));
private final String owner;

class ReverseIterator implements Iterator<String> {   // inner: tied to an outer instance
    private int i = items.size() - 1;                  // reads the outer object's field
    public boolean hasNext() { return i >= 0; }
    public String next() { return items.get(i--) + "@" + NestedDemo.this.owner; }
}

Node n = new Node("static nested");
NestedDemo outer = new NestedDemo("list1");
NestedDemo.ReverseIterator it = outer.new ReverseIterator();
```

Output (first two lines):

```
static nested
c@list1 b@list1 a@list1
```

Common mistakes

- **Using an inner class when a static nested one would do.** Every inner instance keeps its outer object alive — a classic **memory leak** (memory that can't be freed because something still references it) when inner objects (listeners, tasks) outlive the outer one. *Effective Java* Item 24: "favour static member classes over nonstatic". Default to `static`.
- Trying `new Outer.Inner()` from a static context without an outer instance → compile error.

6.11 Local classes, anonymous classes and lambdas

What it is

- A **local class** is declared inside a method body. It can use the method's **effectively final** local variables (variables never reassigned after initialisation) — it *captures* their values.
- An **anonymous class** is declared and instantiated in a single expression, `new SomeInterface() { … }` or `new SomeClass(args) { … }`. It has no name, so it can't have a constructor and can be used only at that one spot.
- A **lambda** (Java 8) is a compact way to implement a **functional interface** (one abstract method). For that case, it replaces most anonymous classes — see Chapter 11.

```
// code/ch06/nested/NestedDemo.java (continued)
Supplier<String> counterDescription() {
    int size = items.size();                        // effectively final
    class Describer implements Supplier<String> {     // local class
        public String get() { return owner + " has " + size + " items"; }
    }
    return new Describer();
}

Comparator<String> byLengthAnon = new Comparator<>() {             // anonymous class
    @Override public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
};
Comparator<String> byLengthLambda = (a, b) -> Integer.compare(a.length(), b.length());   // lambda
```

Output (remaining lines):

```
list1 has 3 items
anonymous: [fig, kiwi, banana]
lambda reversed: [banana, kiwi, fig]
```

Anonymous class vs lambda

	Anonymous class	Lambda
Can implement	Any interface (any number of methods) or extend a class (even abstract)	Only a functional interface
Can have state (fields)	Yes	No
`this` means	The anonymous object	The enclosing instance
Verbosity	High	Low
Use when	Several methods to implement, extending a class (like `new Report("Ad hoc") {…}` in §6.5), or needing own state	Single-method callbacks (code handed over to be called later), comparators, stream operations

Common mistakes

- Modifying a captured local variable (`size++`) inside a local/anonymous class or lambda → "local variables referenced from an inner class must be final or effectively final".
- Serialising anonymous/inner classes — they carry hidden references to the outer instance and synthetic (compiler-generated) names; avoid.

Self-check (6.10–6.11)

1. Which nested kind has no hidden reference to an outer instance? → Static nested (and nested records/enums/interfaces).
2. How do you refer to the outer object from an inner class when names clash? → `Outer.this`.
3. Can a lambda implement an interface with two abstract methods? → No; use an anonymous class or a named class.

TL;DR (6.10–6.11)

Static nested = grouping, no outer link (default choice). Inner = tied to an outer instance (`outer.new Inner()`, `Outer.this`). Local and anonymous classes capture effectively final locals; lambdas replace anonymous classes for functional interfaces.

6.12 Choosing: abstract class, interface, record, enum or plain class

Need	Reach for	Why
A capability many unrelated classes share (`Comparable`, `Closeable`)	**Interface**	A class can implement many; no state forced on implementers
A contract plus a partial implementation with **shared state** or constructors	**Abstract class** (often implementing an interface)	Interfaces can't hold instance state
Evolve a published contract without breaking implementers	Interface **default** method	Added in Java 8 for exactly this
A fixed, known set of variants with different data	**Sealed interface + records**	Exhaustive `switch`, compiler-checked
A fixed set of instances known at compile time	**Enum**	Type-safe constants with behaviour
Immutable data where equal values are interchangeable	**Record**	Minimal code, correct `equals` / `hashCode`
Mutable object with identity and lifecycle	**Class** (usually `final` unless designed for extension)	Full control over encapsulation

Abstract class vs interface in one line: **an interface says what a type *can do*; an abstract class says what a family of classes *is* and shares.** Modern Java style is "interface first", with an optional skeletal abstract class (`AbstractList` alongside `List`, *Effective Java* Item 20). For the design discussion, including ISP (Interface Segregation: keep interfaces small) and DIP (Dependency Inversion: depend on abstractions), see LLD §2.9 Abstract classes vs interfaces, LLD §4.9 ISP and LLD §4.10 DIP.

In the interview

Question: "Abstract class or interface?" 30-second answer: "An interface defines a contract a class can adopt alongside others; since Java 8 it can carry default and static methods, and since 9 private helpers, but never instance state or constructors. An abstract class can have state, constructors and any access level, but a class can extend only one. I start with an interface for the abstraction clients depend on and add an abstract base class only if implementations share real state or code."

- *Follow-up: Why can Java have multiple interface inheritance but not multiple class inheritance?* → Classes carry state and constructors, so multiple parents would create ambiguous state and

initialisation (the diamond problem). Interfaces carry no instance state, and default-method conflicts have explicit resolution rules.

6.13 Object methods: equals, hashCode and toString

What it is

Every class inherits three methods from `Object` that you'll override constantly:

- **`equals(Object o)`** — "are these two objects *logically* the same?" The default is identity (`this == o`).
- **`hashCode()`** — an `int` summarising the object, used by hash-based collections to pick a bucket (one slot in the collection's internal array). The default is typically derived from identity.
- **`toString()`** — a human-readable description, used by `println`, string concatenation, loggers (libraries that write diagnostic messages) and debuggers. The default is `ClassName@hexHashCode`.

Identity vs equality: `a == b` asks "same object in memory?"; `a.equals(b)` asks "same value?". Two `new Book("123", "Dune")` objects are different objects but should be equal.

The contracts

`equals` must be (from the `Object` Javadoc, the official API documentation):

1. **Reflexive**: `x.equals(x)` is true.
2. **Symmetric**: `x.equals(y)` ⇔ `y.equals(x)`.
3. **Transitive**: if `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`.
4. **Consistent**: repeated calls return the same result while the objects don't change.
5. **Non-null**: `x.equals(null)` is false (never throws).

`hashCode` must satisfy:

1. **Equal objects ⇒ equal hash codes.** (This is the rule people break.)
2. Unequal objects *may* share a hash code (a collision) — allowed, but fewer collisions = faster maps.
3. Consistent during one execution while the object doesn't change.

Hence: override `equals` ⇒ override `hashCode`, using the *same* fields.

How it works — writing them

1. `if (this == o) return true;` — fast path.
2. `if (!(o instanceof Book other)) return false;` — type check + cast in one (also returns false for `null`).
3. Compare the significant fields: `==` for primitives (`Double.compare`/`Float.compare` for floating point), `equals` for objects, `Objects.equals(a, b)` for nullable ones, `Arrays.equals` for arrays.
4. `hashCode`: `Objects.hash(field1, field2, …)` — simple and correct (it allocates a varargs array; hand-roll `31 * h + …` only in hot paths).
5. `toString`: include the fields that help debugging; never secrets (passwords, tokens).
6. Or use a **record**, which generates all three correctly (§6.8). IDEs can also generate them.

```java
// code/ch06/objectmethods/EqualsHashCode.java (key parts)
static class EqualsOnly {                                   // BROKEN: equals without hashCode
    final String isbn;
    @Override public boolean equals(Object o) { return o instanceof EqualsOnly other && isbn.equals(other.isbn); }
}

static final class Book {                                   // correct
    private final String isbn;
    private final String title;
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Book other)) return false;
        return isbn.equals(other.isbn) && Objects.equals(title, other.title);
    }
    @Override public int hashCode() { return Objects.hash(isbn, title); }
    @Override public String toString() { return "Book[isbn=" + isbn + ", title=" + title + "]"; }
}
```

Output:

```
NoEquals:   equals=false, toString=ch06.objectmethods.EqualsHashCode$NoEquals@<hash>
EqualsOnly: equals=true, set.contains(equal copy)=false  <- hashCode not overridden
Book:       equals=true, same hash=true, set.contains(equal copy)=true, == false
toString:   Book[isbn=123, title=Dune]
equals(null)=false, Objects.equals(null, null)=true
```

Line 2 is the classic bug: the two `EqualsOnly` objects are `equals`, but their default hash codes differ, so `HashSet` looks in the wrong bucket and never even calls `equals`. How `HashMap` uses `hashCode` then `equals` is explained in Chapter 10.

`instanceof` or `getClass()` in equals?

	`instanceof` (used above)	`getClass() != o.getClass()`
Subclass instance equal to parent instance?	Can be	Never
Risk	Breaks symmetry/transitivity if a subclass adds fields to `equals`	Breaks Liskov substitution: a harmless subclass is never equal to its parent
Common advice	Use it, and make the class `final` (as `Book`) — records do the equivalent	Used by some frameworks (e.g. code aware of Hibernate proxies — Hibernate is a popular database-mapping library)

Effective Java Item 10: there is no way to extend an instantiable class, add a value component, and keep the `equals` contract — favour composition, or make value classes final.

Common mistakes

- Overriding `equals` but not `hashCode` (above).
- `public boolean equals(Book other)` — an overload, not an override. Use `@Override`.
- Using **mutable** fields in `equals`/`hashCode` and then mutating an object while it's inside a `HashSet`/`HashMap` key — it becomes unfindable. Prefer immutable keys.
- Comparing `double` fields with `==` (`NaN`, `-0.0` edge cases) — use `Double.compare`.
- Comparing strings with `==` (Chapter 7).

In the interview

30-second answer: "`==` compares references; `equals` compares logical value. equals must be reflexive, symmetric, transitive, consistent and return false for null. If two objects are equal they must have the same hashCode, otherwise hash-based collections break — they bucket by hashCode

> first. So I always override both together from the same immutable fields, typically with `Objects.equals` / `Objects.hash`, or just use a record."

- *Follow-up: Can two unequal objects have the same hashCode?* → Yes, that's a collision; it's legal but degrades performance.
- *Follow-up: What happens if a key's hashCode changes after insertion into a HashMap?* → Lookups search the wrong bucket; the entry is effectively lost until the key changes back.
- *Follow-up: Why 31 in hand-written hash codes?* → An odd prime that spreads values well, and `31 * h` can be optimised to `(h << 5) - h`; it's convention from *Effective Java* and `String.hashCode`.

Self-check

1. `a.equals(b)` is true. What must be true of their hash codes? → They must be equal.
2. Why does `EqualsOnly` fail in a `HashSet`? → Different default hash codes send lookups to a different bucket.
3. What does `Objects.equals(null, null)` return? → `true`.
4. What should `toString` never include? → Sensitive data (passwords, tokens, personal data).

TL;DR

`==` = same object; `equals` = same value. Override `equals` and `hashCode` together, from the same (ideally immutable) fields; follow the five `equals` rules; always `@Override`. Records do it for you.

Chapter TL;DR

Topic	Remember
Inheritance	`extends`, one parent, root is `Object`; constructors not inherited, `super(...)` first; prefer composition
Overriding	Same signature, covariant return, same/wider access, no broader checked exceptions, `@Override`; not final/static/private
Hiding	Static methods and fields resolve by declared type
Polymorphism	Declared type decides what you may call; runtime type decides what runs. Downcasts checked at runtime; use `instanceof` patterns
Abstract class	Can't instantiate; state + constructors + abstract methods; template method
Interface	Contract; abstract + `default` / `static` (8) + `private` (9); no state; many per class; conflicts → override + `X.super.m()`
Sealed (17)	`permits` closed set; subtypes `final` / `sealed` / `non-sealed`; exhaustive `switch`
Records (16)	Final immutable data carriers; compact constructor for validation/copies; not deeply immutable
Enums (5)	Fixed singleton instances; fields, private constructors, per-constant bodies; `EnumSet` / `EnumMap`; don't persist `ordinal()`
Nested	Prefer static nested; inner needs outer instance (`outer.new Inner()`, `Outer.this`); lambdas for functional interfaces
`equals` / `hashCode`	Override together, same fields, contract rules; `toString` for humans

Where next: the core APIs you'll use in every class — `String`, `StringBuilder`, wrappers, `BigDecimal`, `java.time` — in Chapter 7. Design with these tools: LLD SOLID, LLD relationships and UML, and the pattern chapters starting at LLD Chapter 8. Exam traps: OCP Chapter 6 and OCP Chapter 7.

Chapter 7 — Core APIs: Strings, Numbers, Dates and Randomness

Almost every Java class you write will touch a handful of library types: `String` and `StringBuilder` for text, `Math` and the wrapper classes (object versions of the primitive types: `Integer`, `Double`…) for numbers, `BigDecimal` for money, `java.time` for dates, and `Random` for simulations and tests. These live in `java.lang`, `java.math`, `java.time`, `java.text` and `java.util`, and they are part of every JDK — no dependencies (external libraries you would have to download) needed.

This chapter teaches each API the way you'll actually use it: what it is, how it behaves, the handful of methods that matter, and the mistakes that cause real bugs (comparing strings with `==`, the `Integer` cache, `new BigDecimal(0.1)`, `YYYY` in date patterns, daylight-saving gaps — the clock hour that is skipped when clocks "spring forward"). Chapter 2 §2.12 already introduced `String` basics and text blocks, and §2.11 explained why `0.1 + 0.2 != 0.3`; here we go deeper instead of repeating.

What you'll learn

§	Topic	You'll be able to…
7.1	String immutability and the pool	explain why `String` is immutable, how the pool and `intern` work, and when `==` "works" by accident
7.2	The `String` API	pick the right method for searching, splitting, comparing and transforming text; handle Unicode correctly
7.3	`StringBuilder`, `StringJoiner`	build text efficiently and know when `+` is fine
7.4	Formatting text and numbers	use `format` / `formatted` / `printf` specifiers, locales, `NumberFormat`, `DecimalFormat`, `HexFormat`
7.5	`Math`	round, divide, clamp and detect overflow correctly
7.6	Wrapper classes and autoboxing	avoid the `Integer` cache, null-unboxing and `remove(int)` traps
7.7	`BigInteger` and `BigDecimal`	do exact arithmetic and money with the right rounding
7.8	The `java.time` model	choose between `LocalDate`, `ZonedDateTime`, `Instant`, `Duration`, `Period`
7.9	Local dates and times	create, adjust, compare and measure calendar values
7.10	Zones, instants and DST	convert between zones and survive daylight-saving changes
7.11	Formatting and parsing dates	write safe patterns (and avoid `YYYY`)
7.12	Random numbers	pick between `Random`, `ThreadLocalRandom`, `SecureRandom` and `RandomGenerator`

All runnable code is in `code/` (package `ch07.*`), compiled with `javac --release 25` and run on JDK 26; every "Output:" block below is pasted from a real run.

> **Exam depth.** Tricky `String` / `StringBuilder` method-chain puzzles, `Math` return types and date-time arithmetic questions are drilled in OCP Chapter 4 — Core APIs. Number and date formatting with locales and resource bundles: OCP Chapter 11 — Exceptions and localization (sections 5–8).

7.1 Strings: immutability, the pool and memory

What it is

A `String` is an object that holds a fixed sequence of characters. **Immutable** means that once a `String` object is created, nothing can change its contents: every method that looks like it modifies a string (`toUpperCase`, `replace`, `concat`, `strip`…) actually returns a **new** `String`.

Analogy: a printed page. You can photocopy it and write on the copy, but the original page never changes — anyone else holding the original still sees the same text.

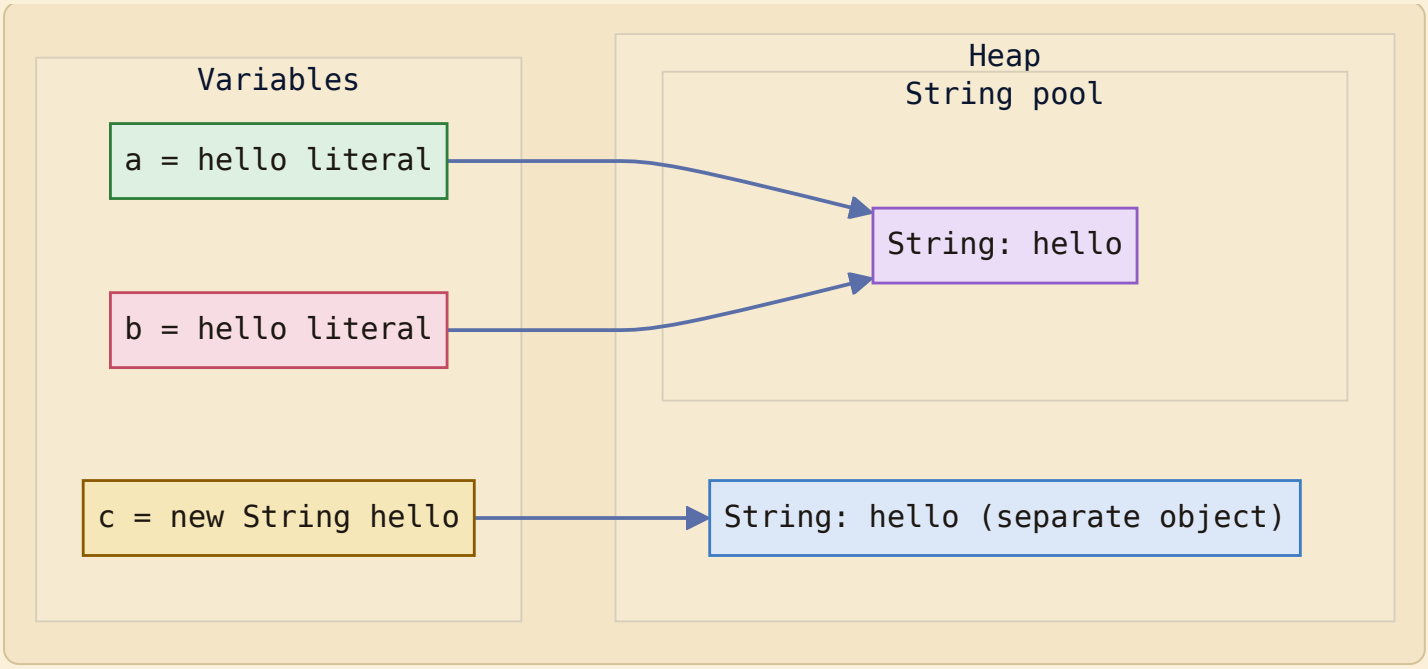
Why it matters

Immutability is a deliberate design choice with several pay-offs:

Benefit	Why immutability enables it
Safe sharing	Many variables (and threads — independent paths of execution running at the same time) can point to one `String` with no risk that one of them changes it — no locking (making other threads wait their turn) needed.
Safe `HashMap` keys	A key's hash code (an `int` computed from its contents, used by a hash table to find it) can't change after insertion, so lookups keep working. `String` even caches its `hashCode` after the first call.
Security	A file path or class name checked by security code can't be altered afterwards by the caller.
The string pool	Identical literals can share one object only because nobody can modify it.

How it works — the string pool

1. Every string **literal** in your source code (`"hello"`) is stored once in the **string pool** (also called the *intern pool*), a table the JVM keeps on the heap (the memory area where objects live; it moved there from PermGen, an older separate area for class data, in Java 7).
2. Two identical literals anywhere in the program refer to the **same** pooled object.
3. **Compile-time constant expressions** — literals joined with `+`, or `final` variables initialised with constants — are folded (computed and joined at compile time) by the compiler into one literal, so they are pooled too.
4. Strings computed **at run time** (concatenating a non-final variable, reading input, `new String(...)`) are ordinary new heap objects, not pooled.
5. `s.intern()` returns the pooled copy of `s`'s contents, adding it to the pool if necessary.



`code/ch07/strings/StringPool.java`:

```
String a = "hello";                    // literal -> string pool
String b = "hello";                    // same pooled object
String c = new String("hello");        // always a new heap object
String d = c.intern();                 // pooled copy
String e = "hel" + "lo";               // constant expression, folded at compile time
String part = "hel";
String f = part + "lo";                // computed at run time -> new object
final String constPart = "hel";
String g = constPart + "lo";           // final + literal initialiser = constant -> pooled
```

Output:

```
a == b: true
a == c: false, a.equals(c): true
a == c.intern(): true
a == "hel" + "lo": true
a == part + "lo": false
a == constPart + "lo": true
s=java, upper=JAVA, same object? false
"x".equals(null)=false, Objects.equals(null,null)=true
```

The lesson: `==` compares *references* (same object?), `equals` compares *contents*. Because of the pool, `==` sometimes *appears* to work on strings — then fails on the first string that came from user input or a database. Always use `equals` (or `Objects.equals(a, b)` when either side may be `null`, or put the literal first: `"yes".equals(answer)`).

How strings are stored (Java 9+)

Since Java 9 (**compact strings**, JEP 254 — a JEP is a numbered JDK Enhancement Proposal) a `String` stores its characters in a `byte[]`: one byte per character when every character fits in Latin-1 (a one-byte character set for Western European languages), otherwise two bytes per character (UTF-16). This roughly halves memory for mostly-ASCII (the basic 128 English letters, digits and symbols) text and is invisible to your code. Also since Java 9 (JEP 280), `a + b + c` is compiled to an optimised `invokedynamic` call (a bytecode instruction that lets the JVM choose the implementation at run time), so a *single* concatenation expression is already efficient — only concatenation *inside loops* needs `StringBuilder` (§7.3).

Common mistakes

- Calling a "modifying" method and ignoring the result: `s.trim();` does nothing useful — write `s = s.trim();`.
- Comparing with `==` (see above), including `switch`-like `if (cmd == "quit")` chains.
- Calling `intern()` everywhere "to save memory" — the pool is a shared table; modern JVMs already deduplicate where it pays (G1, the default garbage collector, has an optional string-deduplication flag). Intern only with measurements.

In the interview

> **30-second answer:** "`String` is immutable and final (no subclasses allowed). That makes it thread-safe (safe to share between threads), lets it cache its hash code so it's a good map key, keeps security checks valid, and allows the JVM to pool identical literals. Literals and compile-time constants are pooled, runtime strings aren't, so `==` is unreliable — I always compare with `equals`."

- *Follow-up: How many objects does `new String("hi")` create?* → Up to two: the pooled literal `"hi"` (if it isn't already in the pool from class loading) and the new heap `String`.
- *Follow-up: Why is `String` final?* → So no subclass can break its immutability guarantees (e.g., a mutable "String" passed to security code).
- *Follow-up: Where is the pool?* → On the heap since Java 7, so pooled strings are garbage-collected like other objects when unreachable.

Self-check

1. `String s = "a"; s.concat("b");` — what is `s`? → `"a"`; `concat` returned a new string that was discarded.
2. Is `"ab" == "a" + "b"` true? → Yes: a constant expression, folded at compile time.
3. Safest way to compare a possibly-null `input` to `"yes"`? → `"yes".equals(input)`.

TL;DR

Strings never change; methods return new strings. Literals and constants are pooled, so `==` may coincidentally be `true` — never rely on it; use `equals`.

7.2 The `String` API toolkit

What it is

`String` has around 100 methods. You need about 30. This section groups them by job, shows them in one program, and flags the ones with surprising behaviour. Indexes are **0-based** and ranges are **half-open**: `substring(begin, end)` includes `begin` and excludes `end`, so its length is `end - begin`.

The methods, by purpose

Job	Methods (Java version if newer than 8)	Notes
Size & emptiness	`length()`, `isEmpty()`, `isBlank()` (11)	`isBlank` is true for `""` and whitespace-only strings
Read characters	`charAt(i)`, `toCharArray()`, `chars()` (9), `codePointAt(i)`, `codePoints()` (9)	`charAt` returns a UTF-16 unit, see Unicode below
Search	`indexOf`, `lastIndexOf`, `contains`, `startsWith`, `endsWith`, `indexOf(ch/str, begin, end)` (21)	`indexOf` returns **-1** when not found
Extract	`substring(begin)`, `substring(begin, end)`, `subSequence`	throws `StringIndexOutOfBoundsException` for bad indexes
Whitespace	`trim()`, `strip()` (11), `stripLeading()`, `stripTrailing()` (11), `stripIndent()` (15)	`trim` removes chars ≤ U+0020; `strip` uses Unicode `Character.isWhitespace` — prefer `strip`
Case	`toUpperCase()`, `toLowerCase()`, with `Locale` (language/region setting) overloads	use `toLowerCase(Locale.ROOT)` for identifiers (see mistakes)
Replace	`replace(char, char)`, `replace(CharSequence, CharSequence)`, `replaceAll(regex, repl)`, `replaceFirst(regex, repl)`	`replace` is **literal**, `replaceAll` is **regex** (regular expression — a mini-language of text-matching patterns)
Compare	`equals`, `equalsIgnoreCase`, `compareTo`, `compareToIgnoreCase`, `contentEquals`, `regionMatches`, `matches(regex)`	`compareTo` < 0, 0, > 0 — not necessarily -1/1
Split & join	`split(regex)`, `split(regex, limit)`, `splitWithDelimiters` (21), `String.join`, `lines()` (11)	`split` takes a **regex**
Build new text	`concat`, `repeat` (11), `indent` (12), `formatted` (15), `transform` (12), `String.valueOf(x)`, `String.format`	`valueOf(null object)` → `"null"`
Other	`hashCode`, `intern`, `translateEscapes` (15)	

Code

`code/ch07/strings/StringApi.java` (key parts):

```
String s = "  Hello, Java World  ";
String t = s.strip();                                  // "Hello, Java World"
t.indexOf('o'); t.lastIndexOf('o'); t.indexOf("Kotlin");          // 4, 13, -1
t.substring(7); t.substring(7, 11);                    // "Java World", "Java"
t.replace('o', '0'); t.replaceAll("[aeiou]", "*");     // literal vs regex
"apple".compareTo("banana"); "Z".compareTo("a");       // -1, -7
"a,b,,c,,".split(",");  "a,b,,c,,".split(",", -1);     // trailing empties dropped vs kept
"1.2.3".split(".");     "1.2.3".split("\\.");          // regex dot matches everything!
"a+b-c".splitWithDelimiters("[+-]", 0);                // Java 21
"education".chars().filter(c -> "aeiou".indexOf(c) >= 0).count(); // count vowels
String emoji = "I ❤ Jäva 😀";                  // heart, a-umlaut, grinning face
emoji.length(); emoji.codePointCount(0, emoji.length());
```

Output:

```
length=21 isEmpty=false isBlank("  ")=true
strip -> [Hello, Java World] charAt(7)=J
indexOf('o')=4 lastIndexOf('o')=13 indexOf("Java")=7 indexOf("Kotlin")=-1
contains=true startsWith=true endsWith=true
substring(7)=Java World substring(7, 11)=Java
replace=Hell0, Java W0rld | replaceAll=H*ll*, J*v* W*rld
upper=HELLO, JAVA WORLD lower=hello, java world
repeat=abababab join=a-b-c joinList=usr/local/bin
equals=false equalsIgnoreCase=true compareTo("apple","banana")=-1 compareTo("b","a")=1 compareTo("Z","a")=-7
compareToIgnoreCase=25
split(",")=[a, b, , c]
split(",", -1)=[a, b, , c, , ]
split("\\s+")=[one, two, three]
split(".") on 1.2.3 -> 0 parts; split("\\.") -> [1, 2, 3]
splitWithDelimiters=[a, +, b, -, c]
palindrome(level)=true toCharArray=[a, b, c] vowels=5
lines=[a, b, c] indent=[  x\n]
formatted=Ada is 36 valueOf(3.5)=3.5 valueOf(char[])=hi
I ❤ Jäva 😀 length=11 codePoints=10 charAt(9) is high surrogate? true
```

What to notice:

- **`compareTo`** compares character by character and returns the *difference* of the first differing characters (`'Z'` is 90, `'a'` is 97, so -7), or the length difference if one string is a prefix of the other. It is **case-sensitive and uppercase sorts before lowercase** — `"Zebra"` comes before `"apple"`. For human-facing sorting use `String.CASE_INSENSITIVE_ORDER` or a locale-aware `java.text.Collator`.
- **`split` takes a regular expression.** `.` means "any character", so `"1.2.3".split(".")` splits at every character and — because trailing empty strings are removed — returns an empty array. Escape (`"\\."`) or use `Pattern.quote(".")`. Likewise `|`, `+`, `*`, `?`, `(`, `[`, `$`, `^` and `\` are special.
- **`split` drops trailing empty strings** unless you pass a negative limit (`split(",", -1)`) — important when parsing CSV-like (comma-separated values, a plain-text table format) lines where the last fields may be empty.
- **`replace` vs `replaceAll`:** both replace *all* occurrences; the difference is literal vs regex. `replaceAll("$", "")` doesn't remove dollar signs; `replace("$", "")` does.

Unicode: `length()` is not "number of characters"

Java `char` is a 16-bit **UTF-16 code unit**. Most characters fit in one `char`, but characters outside the Basic Multilingual Plane (the first 65 536 Unicode characters) — many emoji, some CJK (Chinese, Japanese, Korean) and historic scripts — need **two** `char`s, called a **surrogate pair**. A **code point** is the actual Unicode character number.

In the output above, the string has 10 visible characters but `length()` is 11, because 😀 takes two `char`s. Consequences:

- `charAt`, `substring` and `reverse` by index can cut a surrogate pair in half. (`StringBuilder.reverse` is surrogate-aware; manual loops usually aren't.)
- To iterate real characters use `s.codePoints()`; to count them, `s.codePointCount(0, s.length())`.
- Even code points aren't "what the user sees": a flag or a family emoji is several code points. For user-perceived characters (grapheme clusters) use `java.text.BreakIterator`.

Common mistakes

Mistake	Fix
`"TITLE".toLowerCase()` in Turkish locale gives `"tıtle"` (dotless ı)	Use `toLowerCase(Locale.ROOT)` for keys, protocols, file extensions
`split(".")`, `` `split(" ``	`")` ``
Forgetting `-1` limit when trailing fields may be empty	`line.split(",", -1)`
`s.substring(0, 10)` on a shorter string	Check `length()` first, or `s.substring(0, Math.min(10, s.length()))`
Using `trim` on text with Unicode spaces (e.g. non-breaking or ideographic spaces)	`strip()`
Building a string in a loop with `+=`	`StringBuilder` (§7.3)

Real-world examples

- Log parsing: `line.split("\\s+", 4)` — limit the split so the free-text message stays in one piece.
- Normalising user input: `email.strip().toLowerCase(Locale.ROOT)` before looking up an account.
- `startsWith("https://")`, `endsWith(".json")` for quick routing checks.

Self-check

1. `"banana".indexOf("an", 2)`? → 3.
2. `"a:b:".split(":").length`? → 2 (trailing empty string removed).
3. Why can `length()` exceed the number of emoji-containing characters you see? → It counts UTF-16 units; supplementary characters use two.
4. Literal replacement of `.` with `-` in `"1.2.3"`? → `replace(".", "-")` (not `replaceAll`).

TL;DR

Know the half-open ranges, that `indexOf` returns -1, that `split`/`replaceAll`/`matches` take regexes, that `compareTo` is case-sensitive, and that `length()` counts UTF-16 units. Prefer `strip`, `isBlank`, `repeat`, `lines` (Java 11+).

7.3 `StringBuilder`, `StringBuffer` and `StringJoiner`

What it is

A **`StringBuilder`** is a *mutable* (changeable) sequence of characters: you can append, insert, delete and replace in place, then call `toString()` once at the end. It's the tool for building text step by step.

Analogy: a `String` is a printed page; a `StringBuilder` is a whiteboard you keep editing, then photograph (`toString()`) when finished.

Why it matters

Because strings are immutable, `result += piece` inside a loop creates a brand-new `String` each iteration and copies all the previous characters into it. For n pieces that's $O(n^2)$ character copying (Big-O notation: the work grows with the square of n). A `StringBuilder` grows an internal buffer (a block of memory it writes into; default capacity 16, roughly doubling when full), so appending is amortised $O(1)$ (constant time on average, even though an occasional append must copy everything) and the whole build is $O(n)$.

When is `+` fine? For a single expression like `"Hello, " + name + "!"` — the compiler already produces efficient code (§7.1). Use `StringBuilder` when you concatenate **in a loop** or across many statements.

How it works

1. Create: `new StringBuilder()`, `new StringBuilder("start")`, or `new StringBuilder(expectedCapacity)`.
2. Mutate with methods that **return the same builder** (so they chain): `append`, `insert`, `delete`, `deleteCharAt`, `replace`, `reverse`, `repeat` (21). `setCharAt` and `setLength` return `void`.
3. Read with `length()`, `charAt`, `indexOf`, `substring` (returns a `String`, doesn't change the builder).
4. Finish with `toString()`.

`code/ch07/strings/BuilderDemo.java` (key parts):

```
StringBuilder sb = new StringBuilder("Java");
sb.append(' ').append(25).append(" LTS");          // methods return this -> chaining
sb.insert(0, ">> ");
sb.replace(3, 7, "JAVA");                          // replace [3, 7) with "JAVA"
sb.delete(0, 3);
sb.deleteCharAt(sb.length() - 1).setCharAt(0, 'j');
new StringBuilder("stressed").reverse();
new StringBuilder().repeat("=-", 5).repeat('!', 2);    // Java 21

StringBuilder x = new StringBuilder("abc"), y = new StringBuilder("abc");
x.equals(y);                                       // false! identity comparison

StringBuilder csv = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    if (!csv.isEmpty()) csv.append(',');
    csv.append(i * i);
}
StringJoiner joiner = new StringJoiner(", ", "[", "]");
```

Output:

```
append:  Java 25 LTS (length 11)
insert:  >> Java 25 LTS
replace: >> JAVA 25 LTS
delete:  JAVA 25 LTS
deleteCharAt+setCharAt: jAVA 25 LT
reverse: desserts
indexOf("25")=5 charAt(1)=A
repeat (21): =-=-=-=-=-!!
x.equals(y)=false x.toString().equals(y.toString())=true x.compareTo(y)=0 "abc".contentEquals(x)=true
loop: 1,4,9,16,25
StringJoiner: [Java, Kotlin, Scala]
capacity=1000 length=0
```

Trade-offs

	`String` + `+`	`StringBuilder`	`StringBuffer`	`StringJoiner` / `String.join` / `Collectors.joining`
Mutable	No	Yes	Yes	builds internally
Thread-safe	Yes (immutable)	**No**	Yes (every method `synchronized` — only one thread at a time may run it)	No
Use for	one-off expressions	loops, incremental building	legacy code only	joining a sequence with delimiter/prefix/suffix

`StringBuffer` is the Java 1.0 version of `StringBuilder` (added in Java 5). Its locking is almost never what you want — sharing one half-built string between threads is rarely a sensible design — so use `StringBuilder`.

Common mistakes

- `sb1.equals(sb2)` — `StringBuilder` does **not** override `equals`/`hashCode`, so this is identity. Compare `sb1.toString().equals(sb2.toString())`, `sb1.compareTo(sb2) == 0` (Java 11), or `"abc".contentEquals(sb)`.

- Using a `StringBuilder` as a `HashMap` key or in a `HashSet` (same reason, and it's mutable).
- `sb.append(a + b + c)` in a loop — the inner `+` builds a temporary string; write `.append(a).append(b).append(c)`.
- Sharing one `StringBuilder` between threads.
- `new StringBuilder('x')` — a `char` is widened to `int` and treated as the **capacity**, giving an empty builder. Use `new StringBuilder("x")`.

In the interview

> **30-second answer:** "Strings are immutable, so repeated `+=` in a loop copies everything each time — O(n²). `StringBuilder` is a growable mutable buffer with amortised O(1) appends. `StringBuffer` is the same but synchronized, which is legacy. A single `+` expression is fine; the compiler optimises it."

- *Follow-up: Does `StringBuilder` override `equals`?* → No; compare via `toString()` or `compareTo`.
- *Follow-up: Default capacity?* → 16 characters (or 16 + initial string length); it grows automatically.

Self-check

1. `new StringBuilder("abc").reverse().insert(1, "-")`? → `"c-ba"`.
2. Does `sb.substring(1)` change `sb`? → No, it returns a `String`.
3. Why not `StringBuffer`? → Unneeded synchronisation; `StringBuilder` is the modern default.

TL;DR

Loop-building text → `StringBuilder`; joining a list → `String.join`/`StringJoiner`/`Collectors.joining`; one-line concatenation → `+`. `StringBuilder` has no value-based `equals`.

7.4 Formatting text and numbers

What it is

Formatting turns values into text with a controlled layout: width, alignment, decimal places, thousands separators, currency symbols. Java's main tool is the `java.util.Formatter` mini-language used by `String.format(...)`, `"...".formatted(...)` (Java 15) and `System.out.printf(...)`. For locale-aware business formatting there's `java.text.NumberFormat`/`DecimalFormat`, and for bytes `java.util.HexFormat` (Java 17).

How it works — format specifiers

A specifier has the shape `%[argument_index$][flags][width][.precision]conversion`:

Part	Examples	Meaning
conversion	`s` string, `d` integer, `f` fixed decimal, `e` scientific, `x`/`X` hex, `o` octal, `c` char, `b` boolean, `n` line separator, `%%` literal %	what kind of value
width	`%10s`, `%8d`	minimum characters; right-aligned by default
flags	`-` left-align, `0` zero-pad, `,` grouping separators, `+` always show sign, `(` negatives in parentheses	layout tweaks
precision	`%.2f` (2 decimals), `%.2s` (max 2 chars)	decimals for numbers, max length for strings
argument index	`%2$s`	use the 2nd argument (lets you reuse or reorder)

`code/ch07/format/Formatting.java` (key parts):

```
String.format("|%s|%10s|%-10s|%.2s|", name, name, name, name);
String.format("|%d|%8d|%-8d|%08d|%,d|%+d|", 42, 42, 42, 42, 1234567, 42);
String.format("|%f|%.2f|%10.3f|%-10.1f|%e|", price, price, price, price, 12345.678);
String.format("|%x|%X|%o|%c|%b|%%|", 255, 255, 8, 'J', null);
String.format("|%2$s %1$s|%s|", "world", "hello");
"%s bought %d items for $%.2f".formatted(name, 3, 9.5);             // Java 15
String.format(Locale.US, "US: %,.2f", 1234567.891);
String.format(Locale.GERMANY, "DE: %,.2f", 1234567.891);
NumberFormat.getCurrencyInstance(Locale.US).format(new BigDecimal("1234.5"));
NumberFormat.getPercentInstance(Locale.US).format(0.256);
NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.SHORT).format(1_250_000); // Java 12
new DecimalFormat("#,##0.00", DecimalFormatSymbols.getInstance(Locale.US)).format(1234.5);
HexFormat.ofDelimiter(":").withUpperCase().formatHex(new byte[]{10, -1, 127});        // Java 17
```

Output:

```
|Ada|       Ada|Ada       |Ad|
|42|      42|42      |00000042|1,234,567|+42|
|3.141590|3.14|     3.142|3.1       |1.234568e+04|
|ff|FF|10|J|false|%|
|hello world|world|
Ada bought 3 items for $9.50
printf with %n line ending
US: 1,234,567.89
DE: 1.234.567,89
currency=$1,234.50 percent=26% compact=1M
DecimalFormat: 1,234.50 0.50 -7.00
HexFormat: 0A:FF:7F
```

What to notice:

- `%f` defaults to **6** decimals; `%.2f` **rounds** (HALF_UP — a trailing 5 rounds away from zero) rather than truncates.
- `%b` prints `false` for `null`, and `true` for *any* other non-Boolean object — it is not a "truthiness" check.
- In `|%2$s %1$s|%s|` the plain `%s` after indexed ones takes the *first* ordinary argument, `"world"` — mixing styles is confusing; pick one.
- `%n` produces the platform line separator (`\n` on macOS/Linux, `\r\n` on Windows); prefer it in `printf`.
- Wrong specifier for the type throws at run time: `String.format("%d", 3.5)` → `IllegalFormatConversionException`. Missing arguments → `MissingFormatArgumentException`.
- **Locale matters.** `String.format` uses the JVM's default locale, so a server running in Germany prints `1.234.567,89`. For machine-readable output (CSV, JSON, logs parsed by tools) pass `Locale.ROOT` or `Locale.US` explicitly. `Locale.of("de", "DE")` (Java 19) replaces the deprecated (officially discouraged and liable to be removed) `new Locale(...)` constructors.
- `DecimalFormat` patterns: `0` = digit, always shown; `#` = digit, hidden if zero; `,` grouping; `.` decimal point; `%` multiply by 100. `DecimalFormat` is **not thread-safe** — create one per use or per thread. (`DateTimeFormatter` from §7.11, in contrast, is immutable and thread-safe.)
- Compact number formatting rounds to a whole unit by default (`1.25M` → `1M`); `setMinimumFractionDigits(1)` keeps one decimal. Its rounding mode is `HALF_EVEN`, so 1,250,000 then prints `1.2M` while 1,260,000 prints `1.3M`.

When not to use it

- Money *arithmetic* — format only at the edge; compute with `BigDecimal` or `long` cents (§7.7).
- Building SQL or HTML by formatting user input — use prepared statements (SQL with placeholders that the database driver fills in safely) / templating that escapes special characters.
- Hot logging paths — use your logger's placeholders (`log.info("user {} logged in", id)`), which skip formatting when the level is disabled.

Self-check

1. `String.format("%05.1f", 3.14159)`? → `"003.1"` (width 5 includes the dot).
2. `String.format("%-5s|", "ab")`? → `"ab |"`.
3. Why pass a `Locale` to `format` in a CSV exporter? → So decimal separators don't change with the server's locale.

TL;DR

`%[index$][flags][width][.precision]conv`. `%s %d %.2f %,d %08d %-10s %x %n`. Be explicit about `Locale` for machine output; use `NumberFormat` for currency/percent/compact, `HexFormat` for bytes.

7.5 `Math`

What it is

`java.lang.Math` is a utility class of `static` methods for arithmetic and trigonometry on primitives. You never create it (`Math` has a private constructor). Everything below is `double`-based unless noted, and the `*Exact` methods throw on overflow instead of wrapping silently.

The methods you'll use

Group	Methods	Return type & behaviour
Absolute, min, max	`abs`, `min`, `max`	same type as the arguments (after promotion)
Powers & roots	`pow(a, b)`, `sqrt`, `cbrt`, `hypot(x, y)`, `exp`, `log`, `log10`	`double`
Rounding	`round` → `long` (or `int` for `float`), `floor`/`ceil`/`rint` → `double`	`round` = nearest integer, ties toward +∞ (`-2.5` → `-2`); `rint` = ties to **even**
Integer division	`floorDiv`, `floorMod` (8), `ceilDiv`, `ceilMod` (18)	round toward −∞ / +∞ instead of toward 0
Limiting	`clamp(value, min, max)` (21)	keeps a value inside a range
Overflow-safe	`addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact`, `toIntExact` (8), `absExact` (15), `divideExact` (18), `powExact` (25)	throw `ArithmeticException` on overflow
Trig & angles	`sin`, `cos`, `tan`, `atan2`, `toRadians`, `toDegrees`, constants `PI`, `E`	radians
Misc	`signum`, `random()`	`random()` returns a `double` in [0.0, 1.0)

`code/ch07/math/MathDemo.java` output:

```
abs(-7)=7 max(3,9)=9 min(2.5,-1)=-1.0
pow(2,10)=1024.0 sqrt(144)=12.0 cbrt(27)=3.0 hypot(3,4)=5.0
round(2.5)=3 round(-2.5)=-2 rint(2.5)=2.0 floor(-1.1)=-2.0 ceil(-1.1)=-1.0
7/2=3 -7/2=-3 floorDiv(-7,2)=-4 ceilDiv(7,2)=4 floorMod(-7,3)=2
clamp(150,0,100)=100 clamp(-3.5,0.0,1.0)=0.0
signum(-4.2)=-1.0 log10(1000)=3.0 exp(1)=2.7182818284590455
PI=3.141592653589793 sin(PI/2)=1.0 toDegrees(PI)=180.0
multiplyExact overflow -> ArithmeticException: integer overflow
pages=6 (same as (items + perPage - 1) / perPage = 6)
Math.random() in [0,1)? true
```

What to notice:

- `min(2.5, -1)` printed `-1.0`: the `int` was promoted to `double` first (Chapter 2 §2.7).

- `pow` always returns `double` — `(int) Math.pow(10, 2)` is fine for small values, but for big integer powers use `BigInteger.pow` or `Math.powExact` (Java 25) to avoid rounding and overflow.
- `/` on integers truncates toward zero (`-7/2 = -3`); `floorDiv` rounds toward negative infinity (`-4`). Use `floorMod` for "wrap-around" indexes (clock arithmetic, circular buffers), because `%` keeps the dividend's sign.
- **Paging:** `Math.ceilDiv(items, perPage)` (Java 18) replaces the classic `(items + perPage - 1) / perPage` trick, which can overflow for huge values.
- `Math.abs(Integer.MIN_VALUE)` is still negative — see Chapter 2 §2.10; `absExact` throws instead.
- `StrictMath` gives bit-for-bit identical results on every platform; since Java 17 (JEP 306) all floating-point arithmetic is strict anyway, and `Math` may still use faster intrinsics (CPU-specific implementations the JIT compiler swaps in) for functions like `sin`.

Self-check

1. Type of `Math.round(2.7)`? → `long` (3). `Math.round(2.7f)`? → `int`.
2. `Math.floorMod(-1, 7)`? → 6.
3. How do you detect that `a * b` overflowed an `int`? → `Math.multiplyExact(a, b)` throws `ArithmeticException`.

TL;DR

`round` → `long`, half up; `floor`/`ceil`/`rint` → `double`. `floorDiv`/`floorMod`/`ceilDiv` for correct negative and paging arithmetic, `clamp` (21) for ranges, `*Exact` for overflow detection.

7.6 Wrapper classes and autoboxing

What it is

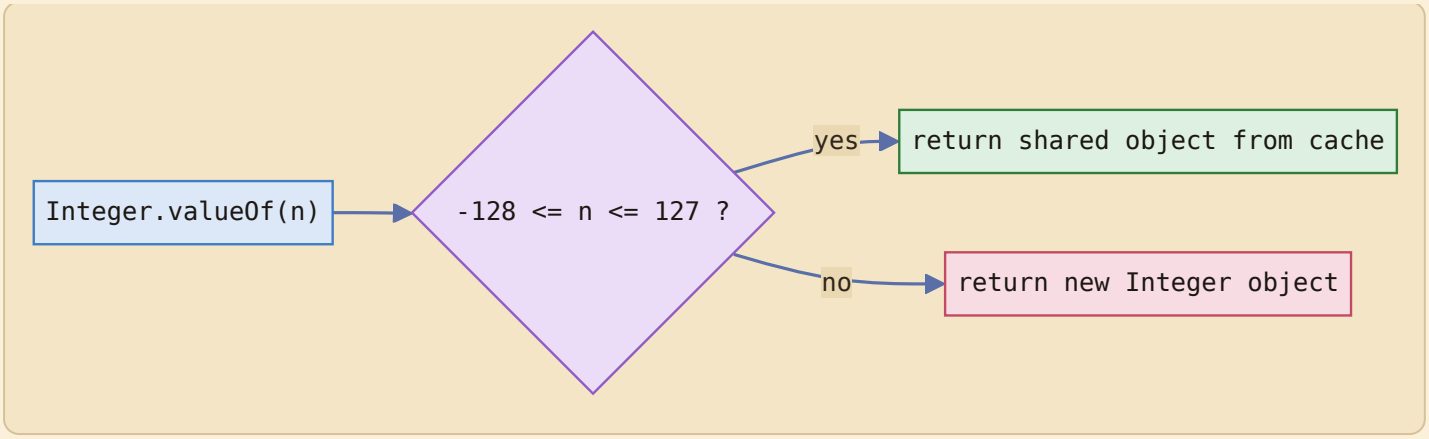
Each primitive has a **wrapper class** — an immutable object holding one value: `byte→Byte`, `short→Short`, `int→Integer`, `long→Long`, `float→Float`, `double→Double`, `char→Character`, `boolean→Boolean`. You need them wherever Java requires an *object*: collections (`List<Integer>`, not `List<int>`), generics (Chapter 9), and fields that may be "absent" (`null`).

Autoboxing (Java 5) is the compiler automatically converting a primitive to its wrapper (`Integer x = 5;` becomes `Integer.valueOf(5)`); **unboxing** is the reverse (`int y = x;` becomes `x.intValue()`).

Analogy: a primitive is a coin; a wrapper is the coin in a small labelled envelope. Collections only accept envelopes; Java puts coins in and takes them out for you — but an empty envelope (`null`) has no coin to take out.

How it works

1. **Boxing** calls `valueOf`. For `Integer`, `Short`, `Byte`, `Long` (−128 to 127), `Character` (0 to 127) and `Boolean`, `valueOf` returns **cached, shared objects**. The Java Language Specification (§5.1.7; the official rulebook of the language) guarantees this for boxed `int`, `short` and `byte` values in −128…127, `char` values up to 127, and `true`/`false`; the `Long` cache is a JDK implementation choice. `Integer`'s upper bound can be raised with `-XX:AutoBoxCacheMax=<n>`. `Float` and `Double` have no cache.
2. **Unboxing** calls `xxxValue()` — which throws `NullPointerException` (NPE; throwing an exception signals an error, Chapter 8) if the reference is `null`.
3. **Arithmetic and `<`/`>`** on wrappers unbox first. **`==` between two wrappers compares references**, not values. `==` between a wrapper and a primitive unboxes the wrapper.
4. The `new Integer(5)` style constructors have been deprecated since Java 9 — always use `valueOf` (or just autobox).



`code/ch07/wrappers/WrappersDemo.java` (key parts):

```
Integer a = 127, b = 127, c = 128, d = 128;
(a == b); (c == d); c.equals(d);                    // true, false, true

Long big = 42L;
big.equals(42);                                     // false: 42 boxes to Integer, a different class
big == 42;                                          // true: big is unboxed, numeric comparison

Map<String, Integer> stock = new HashMap<>();
int apples = stock.get("apples");                   // get returns null -> NPE while unboxing

List<Integer> list = new ArrayList<>(List.of(10, 20, 30));
list.remove(1);                                     // remove(int index)  -> [10, 30]
list.remove(Integer.valueOf(10));                   // remove(Object o)   -> [30]

Integer maybe = null;
Integer result = true ? maybe : 0;                  // mixed boxed/primitive ternary unboxes -> NPE

Long total = 0L;
for (long i = 0; i < 1_000; i++) total += i;        // unbox, add, re-box: 1,000 allocations
```

Output:

```
boxed then unboxed: 5
127 == 127: true, 128 == 128: false, 128 equals 128: true
parseInt=42 valueOf=42 parseInt("ff",16)=255 toBinaryString(10)=1010 Double.parseDouble("1e3")=1000.0
Boolean.parseBoolean("YES")=false
NumberFormatException: For input string: "4 2"
Integer.MAX_VALUE=2147483647 Integer.compare(3,7)=-1 Integer.sum(2,3)=5 Integer.bitCount(255)=8
Character.isDigit('7')=true Character.toUpperCase('q')=Q Character.getNumericValue('9')=9
Long(42).equals(Integer 42): false, big == 42: true
NPE: Cannot invoke "java.lang.Integer.intValue()" because the return value of "java.util.Map.get(Object)" is null
safe: 0
after remove(1) and remove(Integer.valueOf(10)): [30]
ternary unboxing NPE
total=499500
```

Parsing and converting

You have	You want	Use
`String`	`int`	`Integer.parseInt(s)` (radix — number base — overload: `parseInt("ff", 16)`)
`String`	`Integer`	`Integer.valueOf(s)`
`int`	`String`	`String.valueOf(i)`, `Integer.toString(i)`, `Integer.toBinaryString/HexString`
`String`	`boolean`	`Boolean.parseBoolean(s)` — `true` **only** for `"true"` (any case); everything else, including `"YES"`, is `false`
`double`	`int`	cast `(int) d` (truncates) or `Math.round`

Invalid numeric text throws the unchecked (the compiler doesn't force you to catch it) `NumberFormatException`. The parse methods don't trim: `" 42"` fails. Since the parser is strict, validate or catch at the boundary where text enters your program.

Handy statics: `Integer.MAX_VALUE`/`MIN_VALUE`, `Integer.compare(a, b)` (use this instead of `a - b` in comparators — objects that decide the order of two values — which can overflow), `Integer.sum/max/min` (useful as method references, the `Integer::sum` shorthand for a lambda), `Character.isDigit/isLetter/isWhitespace/toUpperCase`, `Character.getNumericValue`.

Trade-offs — primitive vs wrapper

	`int`	`Integer`
Memory	4 bytes	object header (bookkeeping data the JVM stores in every object) + value (typically 16 bytes) + reference
Can be `null`	No (default 0)	Yes — useful for "unknown", dangerous when unboxed
Collections/generics	Not allowed	Required
`==`	compares values	compares references
Speed in hot loops	fast	boxing allocates (except cached values)

When not to use wrappers

- Local variables, counters, loop indexes, arithmetic — use primitives. The `Long total` loop above allocates on every iteration (*Effective Java*, Item 61: prefer primitives to boxed primitives).
- Large numeric arrays/collections in performance-sensitive code — prefer `int[]` or primitive streams (`IntStream`) over `List<Integer>`.

Common mistakes

1. `==` on wrappers — works for small cached values in tests, fails in production at 128. Use `equals` or unbox.
2. `Long.equals(Integer)` is always false — the classes differ. Watch `map.get(42)` on a `Map<Long, …>` (it returns `null`; use `42L`).
3. Unboxing `null`: from `Map.get`, from a nullable database column, or in a ternary (`cond ? a : b`) mixing `Integer` and `int`.
4. `List<Integer>.remove(1)` removes by **index**; use `remove(Integer.valueOf(1))` for the value.
5. Synchronising on a wrapper (`synchronized (count)`) — cached instances are shared JVM-wide; `javac` warns about synchronising on (using as a lock) these *value-based* classes (classes whose instances are meant to be treated as plain values, not distinct identities).

In the interview

> **30-second answer:** "Wrappers let primitives live in collections and generics. Autoboxing calls `valueOf`, which caches −128 to 127, so `==` on two `Integer`s is true for 127 and false for 128 — always use `equals`. Unboxing `null` throws an NPE, which bites with `Map.get` and mixed ternaries. I use primitives for arithmetic and wrappers only where an object or null is needed."

- *Follow-up: Is `Integer i1 = 1000, i2 = 1000; i1 == i2` guaranteed false?* → Not guaranteed — the cache upper bound is configurable, but by default it's false. The point is it's *reference* comparison, so never rely on it.
- *Follow-up: `new Integer(5) == new Integer(5)`?* → Always false (two objects); the constructor is deprecated anyway.

Self-check

1. `Integer x = null; int y = x;` → ? → `NullPointerException`.
2. `Short s = 100; s.equals(100)`? → `false` — `100` boxes to `Integer`.
3. `Boolean.parseBoolean("True")`? → `true` (case-insensitive).

TL;DR

Autoboxing = `valueOf`, unboxing = `xxxValue()`. Cache −128…127 makes `==` lie; use `equals`. Null unboxing → NPE. `remove(int)` vs `remove(Object)`. Primitives for math, wrappers for collections.

7.7 `BigInteger` and `BigDecimal`

What it is

Two immutable classes in `java.math` for numbers that primitives can't represent exactly:

- **`BigInteger`** — whole numbers of *any* size (limited only by memory). Use when values exceed `long` (about 9.2×10^{18}): factorials, cryptography, exact combinatorics.
- **`BigDecimal`** — exact *decimal* numbers: an arbitrary-precision (as many digits as needed) integer **unscaled value** plus a **scale** (the number of digits after the decimal point). `12.30` is unscaled `1230`, scale `2`. Use for money and anything that must match human decimal arithmetic.

Analogy: `double` is a ruler with fine but binary tick marks — 0.1 falls between two ticks. `BigDecimal` is counting coins: exact, but slower.

How it works

1. Both are **immutable**: `a.add(b)` returns a new object — assign the result (`total = total.add(x);`). There are no operators; you call `add`, `subtract`, `multiply`, `divide`, `pow`, `negate`, `abs`, `compareTo`.
2. **Create `BigDecimal` from a `String`** (`new BigDecimal("0.10")`) or `BigDecimal.valueOf(double)` (which goes through `Double.toString`, giving the short decimal you expect). `new BigDecimal(0.1)` copies the double's exact binary value — 55 digits of noise.
3. **Division** must say how to round when the result doesn't terminate: `divide(divisor, scale, RoundingMode)` or `divide(divisor, MathContext)` (an object bundling a precision and a rounding mode). Plain `divide(divisor)` throws `ArithmeticException` for 10/3.
4. **Rounding** is explicit: `setScale(2, RoundingMode.HALF_EVEN)`. Modes: `UP`/`DOWN` (away from / toward zero), `CEILING`/`FLOOR` (toward +∞ / −∞), `HALF_UP` (school rounding), `HALF_DOWN`, `HALF_EVEN` (**banker's rounding**: ties go to the even neighbour, which avoids systematic upward bias over many operations), `UNNECESSARY` (throws if rounding would be needed).
5. **`equals` compares value *and scale*** (`2.0` ≠ `2.00`); **`compareTo` compares value only**. Use `compareTo(...) == 0` for numeric equality.

`code/ch07/big/BigNumbers.java` (key parts):

```java
BigInteger f = BigInteger.ONE;
for (int i = 2; i <= 30; i++) f = f.multiply(BigInteger.valueOf(i));      // 30!
BigInteger.TWO.pow(100);
BigInteger.valueOf(3).modPow(BigInteger.valueOf(200), BigInteger.valueOf(1000));
f.longValueExact();                                      // throws: doesn't fit

new BigDecimal("0.1").add(new BigDecimal("0.2"));        // 0.3 exactly
new BigDecimal(0.1);                                     // the double's binary value
new BigDecimal("2.0").equals(new BigDecimal("2.00"));    // false (scale differs)
ten.divide(three);                                       // ArithmeticException
ten.divide(three, 4, RoundingMode.HALF_UP);              // 3.3333

BigDecimal price = new BigDecimal("19.99");
BigDecimal subtotal = price.multiply(BigDecimal.valueOf(3));
BigDecimal tax = subtotal.multiply(new BigDecimal("0.0825")).setScale(2, RoundingMode.HALF_EVEN);
```

Output:

```
30! = 265252859812191058636308480000000 (108 bits)
2^100 = 1267650600228229401496703205376
gcd(84,36)=12 3^200 mod 1000=1 isProbablePrime(97)=true sqrt(10^20)=10000000000
30!.longValueExact(): BigInteger out of long range
double 0.1+0.2=0.30000000000000004 BigDecimal=0.3
new BigDecimal(0.1)=0.1000000000000000055511151231257827021181583404541015625
BigDecimal.valueOf(0.1)=0.1
2.0 equals 2.00? false, compareTo == 0? true, scales 1 vs 2, stripTrailingZeros equal? true
10/3: Non-terminating decimal expansion; no exact representable decimal result.
10/3 scale 4 HALF_UP=3.3333 MathContext.DECIMAL64=3.333333333333333
2.345 -> HALF_UP 2.35, HALF_EVEN 2.34, DOWN 2.34, FLOOR 2.34
2.355 -> HALF_UP 2.36, HALF_EVEN 2.36, DOWN 2.35, FLOOR 2.35
-2.345 -> HALF_UP -2.35, HALF_EVEN -2.34, DOWN -2.34, FLOOR -2.35
subtotal=59.97 tax=4.95 total=64.92
share=33.33 leftover cent(s)=0.01 (divideAndRemainder: 33.00 r 1.00)
```

Read the rounding rows: for `2.345`, the tie goes *up* with `HALF_UP` (2.35) but to the *even* digit with `HALF_EVEN` (2.34); for `2.355` both give 2.36 because 6 is even. `FLOOR` differs from `DOWN` only for negatives.

The last line shows a classic money problem: $100.00 split three ways is 33.33 each with **one cent left over**. Real systems allocate the remainder explicitly (e.g., the first account gets 33.34) rather than letting it vanish.

Trade-offs

	`double`	`long` cents	`BigDecimal`
Exact decimals	No	Yes (fixed 2 places)	Yes (any scale)
Speed	Fastest (hardware)	Fast	Much slower, allocates
Range	Huge, approximate	$\pm 9.2 \times 10^{16}$ dollars	Unlimited
Good for	Science, graphics, statistics	Payments in one currency, high throughput	Finance, tax, currency conversion, accounting

Common mistakes

- `new BigDecimal(0.1)` → use the `String` constructor or `valueOf`.
- `total.add(x);` without assignment (immutable!).
- `equals` for numeric comparison; also, `HashSet` / `HashMap` use `equals`, so `2.0` and `2.00` are different keys, while a `TreeSet` (a sorted set; uses `compareTo`) treats them as one. Normalise with `stripTrailingZeros()` or a fixed `setScale` before using as keys.
- Unbounded `divide` → `ArithmeticException`; always pass a scale/rounding or a `MathContext`.

- Rounding too early (per line item) vs. at the end — follow your domain's (often legal) rules; tax is frequently rounded per invoice line.
- Using `BigInteger`/`BigDecimal` in tight numeric loops where `long` suffices.

Real-world examples

- Payment platforms store amounts as integer minor units (cents) *or* `DECIMAL(19,4)` columns (an exact SQL number type: 19 digits, 4 after the decimal point) mapped to `BigDecimal` — see the SQL handbook's data types chapter for `NUMERIC` in PostgreSQL.
- RSA (a public-key encryption algorithm) keys and `KeyPairGenerator` internals use `BigInteger.modPow` and `isProbablePrime`.

In the interview

> **30-second answer:** "For money I use `BigDecimal` built from strings, or long minor units. Doubles are binary fractions, so 0.1 isn't representable. `BigDecimal` is immutable, needs an explicit scale and `RoundingMode` for division — usually `HALF_EVEN` for finance — and I compare with `compareTo` because `equals` also checks scale."

- *Follow-up: Why is `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` false?* → `equals` includes scale; `compareTo` returns 0.
- *Follow-up: What's banker's rounding?* → `HALF_EVEN`: ties round to the even neighbour, eliminating the upward bias of always rounding .5 up.

Self-check

1. `BigDecimal.valueOf(1.10).scale()`? → 1 (`Double.toString(1.10)` is `"1.1"`).
2. What does `divide` without rounding do for 1/3? → Throws `ArithmeticException`.
3. `new BigDecimal("-1.5").setScale(0, RoundingMode.HALF_EVEN)`? → `-2`.

TL;DR

`BigInteger` for huge integers, `BigDecimal` for exact decimals. Build from `String`, assign every result, give `divide` a scale + `RoundingMode`, compare with `compareTo`, and prefer `HALF_EVEN` for money.

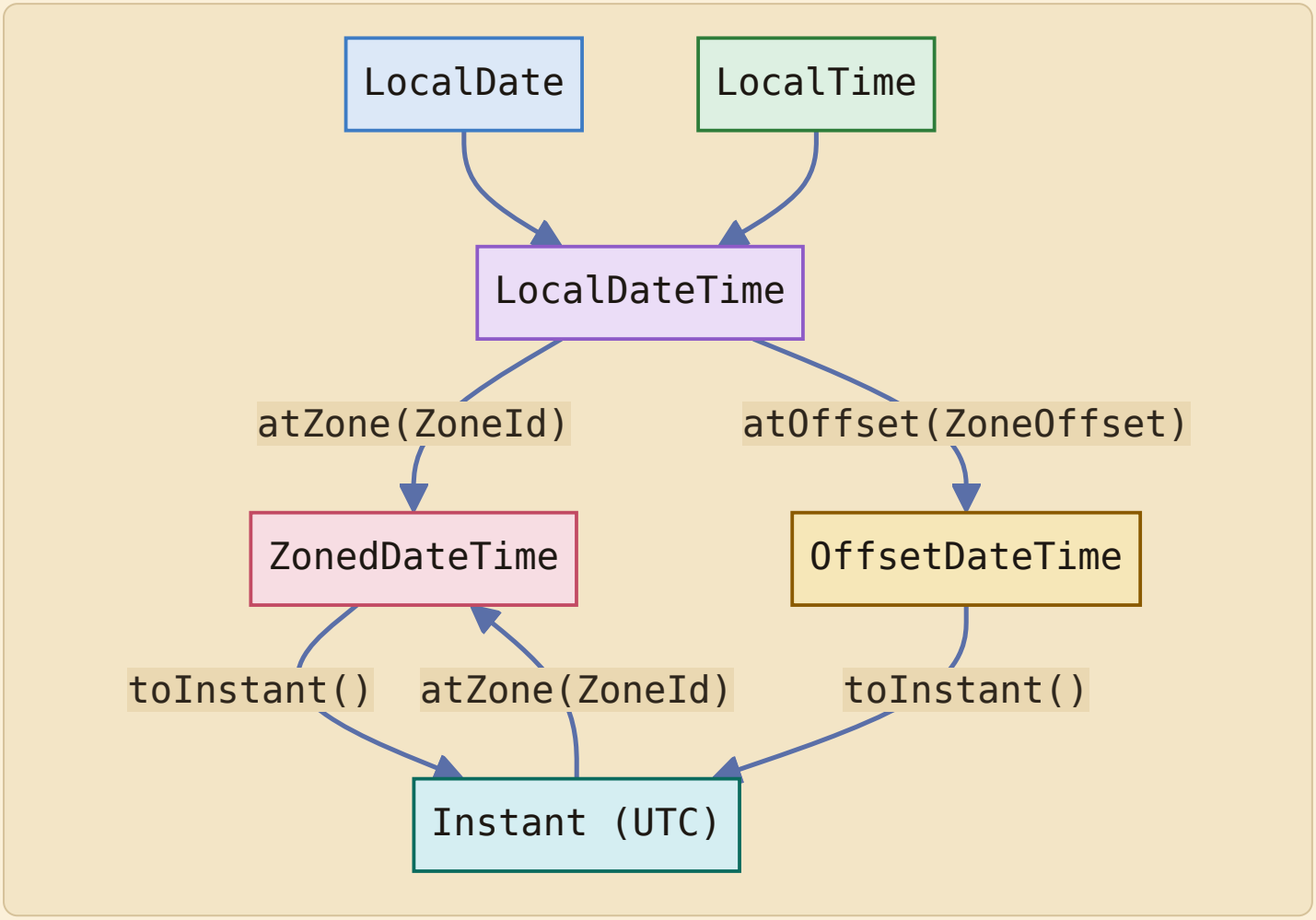
7.8 The `java.time` model

What it is

`java.time` (JSR 310 — a Java Specification Request, the formal proposal for a new API — **Java 8**, designed by the author of the Joda-Time library, the popular earlier date library) is Java's date-and-time API. It replaced `java.util.Date`, `Calendar` and `SimpleDateFormat`, which were mutable, not thread-safe, used 0-based months, and mixed up "a point in time" with "a date on a calendar". All `java.time` types are **immutable** and **thread-safe**.

The key idea: **different questions need different types.** "What's the user's birthday?" has no time zone (a region's rules for its offset from UTC — Coordinated Universal Time, the world reference clock — including daylight saving time, DST, when clocks move forward or back an hour). "When did this payment happen?" is an exact moment. "What time does the 9:00 meeting start in Tokyo?" needs a zone.

Type	Holds	Example	Use for
`LocalDate`	year-month-day	`2025-09-16`	birthdays, holidays, due dates
`LocalTime`	hour-minute-second-nanos	`09:30`	opening hours, alarm times
`LocalDateTime`	date + time, **no zone**	`2025-09-16T09:30`	a wall-clock time (what a clock on the wall shows) whose zone is implied elsewhere
`ZonedDateTime`	date + time + zone (`ZoneId`) with DST rules	`2026-03-20T09:00-04:00[America/New_York]`	scheduling in a region, displaying to users
`OffsetDateTime`	date + time + fixed offset	`2026-03-20T09:00-04:00`	APIs, databases (`timestamp with time zone` values), logs
`Instant`	a point on the UTC timeline (seconds + nanos since 1970-01-01T00:00Z)	`2025-09-16T05:20:00Z`	event timestamps, storing "when", measuring machine time
`Duration`	an amount of **time** (seconds + nanos)	`PT2H15M30S`	timeouts, elapsed time
`Period`	an amount of **calendar** time (years, months, days)	`P1Y2M3D`	ages, subscriptions, "add one month"
`ZoneId` / `ZoneOffset`	region like `Europe/London` / fixed `+05:30`		converting between zones
Also	`Year`, `YearMonth`, `MonthDay`, `DayOfWeek`, `Month` (enums), `Clock`		



How to choose

1. Storing *when something happened* (logs, orders, audit — records kept for later checking) → `Instant` (store in UTC; convert for display).
2. A date with no time (birthday, invoice date) → `LocalDate`. Don't store it as midnight UTC — that shifts the day for users west of Greenwich.
3. A future appointment in a place → store `LocalDateTime` + `ZoneId` (not an `Instant`): if the government changes DST rules before the meeting, the meeting should still be at 9:00 local.
4. Machine elapsed time → `System.nanoTime()` or `Duration.between(instantA, instantB)`; calendar arithmetic → `Period`.

Common API conventions

Prefix	Meaning	Example
`of`	create from parts	`LocalDate.of(2025, 9, 16)`
`now`	current value (optionally from a `Clock` or zone)	`LocalDate.now(clock)`
`parse`	from text	`LocalDate.parse("2025-09-16")`
`get`	read a field	`getDayOfWeek()`
`with`	copy with one field changed	`withDayOfMonth(1)`
`plus` / `minus`	copy shifted by an amount	`plusMonths(1)`
`to` / `at`	convert to another type	`toLocalDate()`, `atZone(zone)`
`is`	query	`isBefore`, `isLeapYear`

Months are **1-based** (`Month.JANUARY` = 1), unlike `Calendar`.

Legacy interop

You'll meet `Date` in older libraries. Convert at the boundary: `Date.from(instant)`, `date.toInstant()`, `GregorianCalendar.from(zdt)`, `java.sql.Timestamp.from(instant)`. With JDBC (Java Database Connectivity, the standard database API) 4.2+ drivers you can read and write `LocalDate`, `LocalDateTime`, `OffsetDateTime` directly with `getObject(col, LocalDate.class)`.

7.9 Local dates, times, adjusters and `Period`

How it works

1. **Create**: `LocalDate.of(2025, Month.SEPTEMBER, 16)`, `LocalTime.of(9, 30)`, `LocalDateTime.of(date, time)`, `LocalDate.parse("2023-09-19")`. Invalid values throw `DateTimeException` immediately.
2. **Read**: `getYear`, `getMonth`/`getMonthValue`, `getDayOfMonth`, `getDayOfWeek`, `getDayOfYear`, `isLeapYear`, `lengthOfMonth`.
3. **Adjust** (always returns a new object): `plusDays/Weeks/Months/Years`, `minus…`, `withDayOfMonth`, and `TemporalAdjusters` for rules like "last day of month" or "next Friday".
4. **Compare**: `isBefore`, `isAfter`, `isEqual`, `equals`, `compareTo`.
5. **Measure**: `Period.between(a, b)` gives years + months + days; `ChronoUnit.DAYS.between(a, b)` gives a single total.

`code/ch07/time/LocalDates.java` (key parts):

```java
LocalDate release = LocalDate.of(2025, Month.SEPTEMBER, 16);     // Java 25 GA date
LocalDate d = LocalDate.of(2024, 1, 31);
d.plusDays(1);                                                   // result ignored -> d unchanged
d.plusMonths(1);                                                 // 2024-02-29
release.with(TemporalAdjusters.lastDayOfMonth());
release.with(TemporalAdjusters.next(DayOfWeek.FRIDAY));
Period gap = Period.between(LocalDate.of(2023, 9, 19), release);
ChronoUnit.DAYS.between(LocalDate.of(2023, 9, 19), release);
Period.between(LocalDate.of(1990, 2, 28), LocalDate.of(2026, 2, 27)).getYears();   // age
LocalDate.of(2026, 3, 2).datesUntil(LocalDate.of(2026, 3, 9))                     // Java 9
         .filter(x -> x.getDayOfWeek().getValue() <= 5) ...
LocalDate.of(2025, 2, 29);                                       // DateTimeException
```

Output:

```
date=2025-09-16 time=09:30 dateTime=2025-09-16T09:30
dayOfWeek=TUESDAY dayOfYear=259 month=SEPTEMBER monthValue=9 leapYear=false lengthOfMonth=30
d after ignored plusDays: 2024-01-31
2024-01-31 plusMonths(1)=2024-02-29 (clamped to month end, leap year)
2025-01-31 plusMonths(1)=2025-02-28
withDayOfMonth(1)=2025-09-01 lastDayOfMonth=2025-09-30 nextFriday=2025-09-19
isBefore=true isAfter=false equals=true
Period between Java 21 and 25 GA: P1Y11M28D = 1y 11m 28d; total days=728 total months=23
age on 2026-02-27 = 35
weekdays in week of 2026-03-02: 2 3 4 5 6
YearMonth 2028-02 has 29 days
DateTimeException: Invalid date 'February 29' as '2025' is not a leap year
```

What to notice:

- **Month arithmetic clamps** to the last valid day: Jan 31 + 1 month = Feb 28 (or 29). So `d.plusMonths(1).minusMonths(1)` may not return `d`. Billing systems must decide what "monthly on the 31st" means.
- **`Period` vs total units:** `P1Y11M28D` is not "728 days expressed differently" — months have different lengths. Use `Period` for human display ("1 year, 11 months"), `ChronoUnit.X.between` for a single count.
- The age calculation gives 35 on the day *before* the 36th birthday — `Period.between` handles this correctly; subtracting years does not.
- `datesUntil` (Java 9) streams dates with the end **exclusive**, like most Java ranges.

Common mistakes

- Ignoring return values (`date.plusDays(1);`) — the object is immutable.
- Using `LocalDateTime` for timestamps of events: it has no zone, so it can't identify an instant. Use `Instant` or `OffsetDateTime`.
- Calling `LocalDate.now()` directly inside business logic (the code that implements the application's real rules) — tests become date-dependent. Inject a `Clock` (pass it in from outside instead of creating it inside; §7.10).

Self-check

1. `LocalDate.of(2024, 3, 31).minusMonths(1)`? → `2024-02-29`.
2. Days in the half-open range from 2026-03-02 to 2026-03-09? → 7.
3. Which gives "3 months" between Jan 15 and Apr 15: `Period` or `ChronoUnit.DAYS`? → `Period` (`P3M`); `DAYS` gives 90.

7.10 Zones, instants, `Duration` and daylight saving

What it is

A **time zone** (`ZoneId`, e.g. `America/New_York`) is a region's set of rules: its offset from UTC and when that offset changes (**daylight saving time**, DST). An **offset** (`ZoneOffset`, e.g. `-04:00`) is just a fixed number of hours/minutes. Zone rules come from the IANA (Internet Assigned Numbers Authority) time-zone database, shipped with the JDK and updated in JDK updates when governments change rules.

How it works

1. `instant.atZone(zone)` shows one moment as local time in a zone. The same `Instant` is 01:20 in New York, 10:50 in Kolkata, 14:20 in Tokyo.
2. `zdt.withZoneSameInstant(otherZone)` converts a meeting to another zone (same moment, different clock reading). `withZoneSameLocal` keeps the clock reading and changes the moment — rarely what you want.
3. **DST gap** (spring forward): local times that don't exist are shifted forward by the gap length. **DST overlap** (fall back): local times that occur twice default to the earlier offset (`withLaterOffsetAtOverlap()` picks the other).
4. **`Duration` is exact time; `Period` is calendar time.** Across a DST change, `plus(Period.ofDays(1))` keeps the same local clock time (a "day" of 23 hours), while `plus(Duration.ofDays(1))` adds exactly 24 hours.
5. A **`Clock`** is the source of "now". Production uses `Clock.systemUTC()` or `systemDefaultZone()`; tests pass `Clock.fixed(...)`.

`code/ch07/time/ZonesAndInstants.java` (key parts):

```
Instant t = Instant.ofEpochSecond(1_758_000_000L);
t.atZone(ZoneId.of("America/New_York")); // and Asia/Kolkata, Asia/Tokyo
ZonedDateTime call = ZonedDateTime.of(LocalDateTime.of(2026, 3, 20, 9, 0), ny);
call.withZoneSameInstant(kolkata).toLocalTime();
ZonedDateTime gap = ZonedDateTime.of(LocalDateTime.of(2026, 3, 8, 2, 30), ny);     // doesn't exist
ZonedDateTime before = ZonedDateTime.of(LocalDateTime.of(2026, 3, 7, 12, 0), ny);
before.plus(Period.ofDays(1));  before.plus(Duration.ofDays(1));
Duration.between(before, before.plusDays(1)).toHours();
Clock fixed = Clock.fixed(Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC);
LocalDate.now(fixed);
```

Output:

```
epoch=1970-01-01T00:00:00Z t=2025-09-16T05:20:00Z t.toEpochMilli()=1758000000000
NY:      2025-09-16T01:20-04:00[America/New_York]
Kolkata: 2025-09-16T10:50+05:30[Asia/Kolkata]
Tokyo:   2025-09-16T14:20+09:00[Asia/Tokyo]
9:00 NY call in Kolkata: 18:30; withZoneSameLocal keeps 09:00 but changes the instant: 2026-03-
20T09:00+05:30[Asia/Kolkata]
02:30 in the gap becomes: 2026-03-08T03:30-04:00[America/New_York]
plus(Period.ofDays(1)) = 2026-03-08T12:00-04:00[America/New_York]
plus(Duration.ofDays(1)) = 2026-03-08T13:00-04:00[America/New_York]
hours between noon and noon next day: 23
Duration PT2H15M30S = 2h 15m 30s; Period.of(1,2,3)=P1Y2M3D Period.ofWeeks(2)=P14D
OffsetDateTime=2026-03-20T09:00-04:00 toInstant=2026-03-20T13:00:00Z
LocalDate.now(fixedClock)=2026-01-01 Instant.now(fixedClock)=2026-01-01T00:00:00Z
Date.from(t).getTime()==t.toEpochMilli(): true, back to Instant: 2025-09-16T05:20:00Z
```

In 2026, US clocks spring forward on Sunday 8 March at 02:00, so 02:30 doesn't exist and becomes 03:30 EDT; the "day" from Saturday noon to Sunday noon is 23 hours long. Note also the Kolkata offset is `+05:30` — offsets aren't always whole hours. `Period.ofWeeks(2)` is stored as 14 days (`P14D`).

Trade-offs — which "now"?

Need	Use
Timestamp to store/send	`Instant.now(clock)`
Measuring how long code took	`System.nanoTime()` (monotonic — only ever moves forward; `Instant.now()` can jump if the system clock is adjusted)
Today's date for a user	`LocalDate.now(clock.withZone(userZone))` — "today" depends on the zone
Testable business logic	Inject `Clock`; in tests use `Clock.fixed` or `Clock.offset`

Common mistakes

- Using the server's default zone (`ZoneId.systemDefault()`) implicitly — servers are often UTC while users aren't. Pass zones explicitly.
- Abbreviations like `"EST"`/`"IST"` — ambiguous (IST is India, Ireland and Israel). Use region IDs (`Asia/Kolkata`).
- Adding `Duration.ofDays(1)` when you mean "same time tomorrow".
- Storing future local events as `Instant` (breaks when DST rules change) or past events as `LocalDateTime` (loses the moment).

In the interview

> **30-second answer:** "I store moments as `Instant` in UTC and convert to `ZonedDateTime` with the user's `ZoneId` only for display. Dates without time are `LocalDate`. `Duration` is exact seconds, `Period` is calendar units, and they differ across DST. All java.time types are immutable and thread-safe, and I inject a `Clock` so time-dependent code is testable."

- *Follow-up: Difference between `ZonedDateTime` and `OffsetDateTime`?* → Zoned knows the region's DST rules; Offset has only a fixed offset — better for serialising (turning into text or bytes to send or store) a moment unambiguously.
- *Follow-up: Why was `SimpleDateFormat` a problem?* → Mutable and not thread-safe; sharing one in a static field corrupted results under load. `DateTimeFormatter` is immutable.

Self-check

1. Same `Instant`, two zones — equal instants? → Yes; only the local representation differs.
2. How many hours between 2026-11-01T00:00 and 2026-11-02T00:00 in New York? → 25 (fall-back day).
3. Why inject a `Clock`? → To control "now" in tests and avoid hidden dependence on the system clock/zone.

TL;DR

`Instant` for moments, `LocalDate` for dates, `ZonedDateTime` for local times in a region. Convert with `atZone`/`withZoneSameInstant`. DST: gaps shift forward, overlaps pick the earlier offset. `Duration` ≠ `Period`. Inject `Clock`.

7.11 Formatting and parsing dates

How it works

1. Every `java.time` type's `toString()` prints **ISO-8601** (the international standard text format for dates and times) (`2025-12-29T14:05:09`), and `parse(text)` reads it back — ideal for JSON, logs and databases.

2. `DateTimeFormatter` has predefined formatters (`ISO_LOCAL_DATE`, `ISO_OFFSET_DATE_TIME`, `BASIC_ISO_DATE`, `RFC_1123_DATE_TIME`…), localized styles (`ofLocalizedDate(FormatStyle.MEDIUM)`), and custom patterns (`ofPattern("dd MMM uuuu")`).
3. Format with `value.format(formatter)`; parse with `LocalDate.parse(text, formatter)`. Bad text throws `DateTimeParseException`.
4. `DateTimeFormatter` is **immutable and thread-safe**, so store it in a `static final` field.

Pattern letters you need

Letter	Meaning	Example (2025-12-29 14:05:09)
`uuuu` / `yyyy`	year / year-of-era	2025
`YYYY`	**week-based year** — almost never what you want	2026 (!)
`MM` / `MMM` / `MMMM`	month number / short / full name	12 / Dec / December
`dd`	day of month	29
`D`	day of year	363
`EEE` / `EEEE`	day name short / full	Mon / Monday
`HH` / `hh` + `a`	hour 0–23 / hour 1–12 + AM/PM	14 / 02 PM
`mm`	**minute** (not month)	05
`ss`, `SSS`	second, fraction	09, 000
`VV`, `z`, `XXX`	zone ID, zone name, offset	Europe/London, GMT, Z
`'text'`	literal text	`'T'`

`code/ch07/time/DateFormatting.java` output:

```
toString (ISO-8601): 2025-12-29T14:05:09
ISO_LOCAL_DATE: 2025-12-29
BASIC_ISO_DATE: 20251229
custom: 29 Dec 2025, 02:05 PM
EEEE d MMMM (fr): lundi 29 décembre
MEDIUM (US): Dec 29, 2025
yyyy-MM-dd: 2025-12-29   YYYY-MM-dd: 2026-12-29
MM=12 mm=05
parsed=2026-03-08 SUNDAY
DateTimeParseException: Text '2026-13-01' could not be parsed: Invalid value for MonthOfYear (valid values 1 - 12): 13
ISO_OFFSET_DATE_TIME: 2025-12-29T14:05:09Z | zone pattern: 14:05 Europe/London (GMT)
```

The `YYYY` bug: 29 December 2025 is a Monday in the week that contains 1 January 2026, so in the US week definition it belongs to *week-based year* 2026. `YYYY-MM-dd` therefore prints `2026-12-29` — a date a year in the future. This has caused real production outages (expired certificates, broken booking systems) during the last days of December. Use `uuuu` (or `yyyy`) — never `YYYY` unless you're printing ISO week numbers with `ww`.

`uuuu` vs `yyyy`: `u` is the proleptic year (a year count extended backwards through year 0 into negatives), `y` is year-of-era (needs an era `G` — AD/BC — with `ResolverStyle.STRICT`). With the default (smart) resolver both work for modern dates; use `uuuu` if you switch to strict parsing.

Common mistakes

- `MM` vs `mm`, `HH` vs `hh`, `YYYY` vs `yyyy`, `DD` (day of year) vs `dd`.
- Parsing month/day names without a `Locale` — `ofPattern("MMM", Locale.US)`, otherwise the JVM default locale decides whether "Dec" or "déc." is expected.
- Formatting a `LocalDate` with a pattern containing time fields → `UnsupportedTemporalTypeException`.
- Rolling your own ISO format instead of `toString()`/`parse()` or `ISO_*` constants.

Self-check

1. Pattern for `08/03/2026` meaning 3 August? → `MM/dd/uuuu`.
2. Why store formatters in `static final` fields? → They're immutable and thread-safe, and building one parses the pattern.
3. What's wrong with `ofPattern("YYYY-MM-dd")`? → Week-based year; wrong in late December/early January.

TL;DR

ISO `toString`/`parse` for machines; `DateTimeFormatter.ofPattern(..., Locale)` for humans. `uuuu`/`yyyy` for year, `MM` month, `mm` minute, `HH` 24-hour. Never `YYYY`.

7.12 Random numbers

What it is

A **pseudo-random number generator** (PRNG) produces a sequence of numbers that looks random but is fully determined by a starting value, the **seed**. Same seed → same sequence, which is great for reproducible tests and simulations and terrible for security.

The options

Class	Since	Thread-safe?	Use for
`Math.random()`	1.0	yes (one shared generator)	quick scripts; a `double` in [0, 1)
`java.util.Random`	1.0	yes, but contended (threads must wait for each other) across threads	seeded, reproducible sequences
`ThreadLocalRandom.current()`	7	per-thread instance, no contention	general randomness in concurrent code (code where several threads run at once)
`SplittableRandom`	8	no — split per task	parallel computations (e.g. Fork/Join, Java's framework for splitting work across CPU cores)
`SecureRandom`	1.1	yes	**security**: tokens, passwords, keys, session IDs (secret random strings that identify a logged-in user)
`RandomGenerator` interface + `RandomGeneratorFactory`	17 (JEP 356)	depends on algorithm	choosing modern algorithms (LXM family — a modern design of fast generators) by name

Java 17 made all of these implement a common `java.util.random.RandomGenerator` interface, which also added `nextInt(origin, bound)` and `nextLong(origin, bound)` to every generator. `Random.from(RandomGenerator)` (Java 19) adapts a new-style generator for APIs that still want a `Random`.

`code/ch07/random/RandomDemo.java` (key parts):

```
Random r1 = new Random(42), r2 = new Random(42);          // same seed -> same sequence
r1.nextInt(100);                                          // [0, 100)
new Random(7).nextInt(1, 7);                              // dice: [1, 7), Java 17
Collections.shuffle(deck, new Random(1));                 // reproducible shuffle
new Random(3).ints(5, 0, 10).boxed().toList();            // stream of 5 ints in [0, 10)
RandomGeneratorFactory.of("L64X128MixRandom").create(42L);
ThreadLocalRandom.current().nextInt(1, 7);
new SecureRandom().nextBytes(token);
```

Output:

```
seed 42 twice: 30/30 63/63 48/48 84/84 70/70
nextInt(1,7) dice: 5 3 4 | nextBoolean=false | nextDouble in [0,1)? true
nextGaussian (mean 0, sd 1): 0.5840
shuffle with seed 1: [C, D, B, E, A]
ints stream: [4, 0, 0, 1, 8]
L64X128MixRandom seeded: 98 1
default algorithm: L32X64MixRandom
ThreadLocalRandom roll in 1..6? true
SecureRandom token bytes: 16
```

The seeded lines are identical on every run and every JVM (the `Random` algorithm is specified). The default `RandomGenerator` algorithm is an implementation detail — don't rely on it being `L32X64MixRandom` forever.

Common mistakes

- `Math.abs(random.nextInt()) % n` — biased, and `Math.abs(Integer.MIN_VALUE)` is negative. Use `nextInt(n)` or `nextInt(origin, bound)`.
- `(int) (Math.random() * 6) + 1` works, but `ThreadLocalRandom.current().nextInt(1, 7)` states the range clearly.
- Creating `new Random()` inside a loop — wasteful; keep one.
- Using `Random` or `Math.random` for passwords, reset tokens or session IDs — predictable. Use `SecureRandom`.
- Forgetting that bounds are **exclusive** at the top: `nextInt(1, 6)` never returns 6.

Self-check

1. Two `Random(99)` objects — same first `nextInt()`? → Yes, same seed, same sequence.
2. Which class for generating an API key? → `SecureRandom`.
3. Range of `nextInt(10, 20)`? → 10 to 19.

TL;DR

Seeded `Random` for reproducible tests; `ThreadLocalRandom` in concurrent code; `SecureRandom` for anything security-related; upper bounds are exclusive.

Chapter TL;DR

Topic	Remember
`String`	Immutable, pooled literals, compare with `equals`; methods return new strings
`String` API	0-based, half-open ranges; `indexOf` → -1; `split` / `replaceAll` use regex; `length()` counts UTF-16 units; prefer `strip` / `isBlank`
`StringBuilder`	Mutable, chainable, for loops; no value `equals`; `StringBuffer` is legacy
Formatting	`%[i$][flags][width][.prec]conv`; explicit `Locale` for machine output; `NumberFormat`, `DecimalFormat` (not thread-safe), `HexFormat` (17)
`Math`	`round` → long, half up; `floorDiv` / `floorMod` / `ceilDiv` (18); `clamp` (21); `*Exact` throws on overflow
Wrappers	Autobox = `valueOf`; cache −128…127 makes `==` unreliable; null unboxing → NPE; `remove(int)` vs `remove(Object)`
`BigDecimal`	Build from `String`; assign results; `divide` needs scale + `RoundingMode`; `compareTo` not `equals`; `HALF_EVEN` for money
`java.time`	Immutable, thread-safe; `Instant` = moment, `LocalDate` = date, `ZonedDateTime` = local time in a region; `Duration` ≠ `Period` across DST; inject `Clock`
Date patterns	`uuuu-MM-dd HH:mm`; never `YYYY`; pass a `Locale` for names
Random	Seeded `Random` for tests, `ThreadLocalRandom` for concurrency, `SecureRandom` for security

Where next: what happens when these APIs fail — `NumberFormatException`, `DateTimeParseException`, `ArithmeticException` — and how to design your own error handling: Chapter 8 — Exceptions. Collections of these values: Chapter 10. Exam traps: OCP Chapter 4 — Core APIs and OCP Chapter 11 for localization.

Chapter 8 — Exceptions

Programs fail: files go missing, users type "forty-two" where a number belongs, networks time out (give up after waiting too long for an answer), and code has bugs. An **exception** is Java's way of reporting that something went wrong *without* forcing every method to return an error code. This chapter explains the mechanism (how an exception travels up the call stack — the chain of method calls currently in progress), the type hierarchy (checked vs unchecked vs errors), every piece of syntax (`try`/`catch`/`finally`, multi-catch, try-with-resources, `throw`, `throws`), how to design your own exceptions, and the practices that separate robust code from code that hides its failures.

You've already met exceptions in passing: `NumberFormatException` from `Integer.parseInt`, `ArithmeticException` from `Math.addExact` (Chapter 2), `ArrayIndexOutOfBoundsException` (Chapter 4), and the rule that an overriding method (a subclass's replacement for an inherited method) can't throw broader checked exceptions (Chapter 6 §6.3).

What you'll learn

§	Topic	You'll be able to…
8.1	What an exception is	throw and catch one, and trace how it propagates up the call stack
8.2	The `Throwable` hierarchy	tell checked, unchecked and errors apart and know the common ones
8.3	Handle or declare	satisfy the compiler for checked exceptions with `try`/`catch` or `throws`
8.4	`finally` and control flow	predict exactly what runs, and what gets returned, in every case
8.5	Multi-catch, catch order, rethrow	write compact handlers that compile
8.6	try-with-resources	close files, sockets (network connection endpoints) and locks (objects that make threads take turns) correctly, including suppressed exceptions
8.7	Throwing and custom exceptions	choose a standard exception or design your own
8.8	Chaining and translation	wrap low-level causes without losing them
8.9	Reading stack traces	find the real cause in a trace in seconds
8.10	Best practices and anti-patterns	apply the *Effective Java* (Joshua Bloch's best-practice book) rules and avoid swallowed (caught and silently ignored) or misused exceptions

All runnable code is in `code/` (package `ch08.*`), compiled with `javac --release 25` and run on JDK 26. Outputs are pasted from real runs.

> **Exam depth.** "Does this compile?" puzzles about unreachable catch blocks, `finally` return values, try-with-resources scope and suppressed exceptions, plus overriding and `throws`: OCP Chapter 11 — Exceptions and localization (sections 1–4). For error-handling *strategy* in services (retries — automatically trying a failed operation again, see System Design Fundamentals — error codes at API boundaries, logging), see LLD Clean Code §6 Error handling and §16 Error-handling strategy.

8.1 What an exception is

What it is

An **exception** is an object that describes an abnormal event, plus the act of **throwing** it: the JVM stops executing the current code and jumps to the nearest matching **handler** (a `catch` block). If no method on

the call stack handles it, the thread (an independent path of execution; every program has at least the `main` thread) dies and the JVM prints a **stack trace** (a list of the method calls that were active when the error happened, §8.9).

Analogy: a factory production line. When a worker finds a defect they can't fix, they don't carry on building; they pull the alarm cord (throw). Supervisors up the chain (callers) each get a chance to deal with it (catch). If nobody does, the line stops (the thread terminates).

The problem it solves

Without exceptions (as in C), every function returns an error code that every caller must check — and forgetting one check silently corrupts data. Exceptions:

1. **Separate** normal logic from error handling.
2. **Can't be silently ignored** by forgetting a check — an unhandled exception is loud.
3. **Propagate automatically** to the level that knows what to do (a web request handler, a UI — user interface — a retry loop).
4. **Carry information**: type, message, cause, and the exact call path (stack trace).

How it works

1. Something goes wrong: the JVM detects it (division by zero, `null` dereference — trying to use the object behind a `null` reference — bad array index) or your code executes `throw new SomeException(...)`.
2. Execution of the current block stops immediately — remaining statements are skipped.
3. The JVM looks for an enclosing `try` whose `catch` type matches (the thrown object is an instance of it). If found, that catch block runs, then execution continues **after** the whole `try` statement.
4. If not found in this method, the method ends abruptly and the exception is re-thrown in the **caller**, at the point of the call. This repeats up the stack (**stack unwinding**), running `finally` blocks on the way.
5. If it reaches the top of a thread's stack, the thread's *uncaught exception handler* prints the stack trace and the thread ends. If that was the `main` thread and no other non-daemon threads (daemon threads are background helpers that don't keep the JVM alive) are running, the program exits with status 1 (the exit code reported to the operating system; non-zero means failure).

`code/ch08/basics/FirstException.java` (key parts):

```
static int average(int[] values) {
    int sum = 0;
    for (int v : values) sum += v;
    return sum / values.length;           // length 0 -> ArithmeticException
}

try {
    System.out.println("before the call");
    int avg = average(new int[0]);        // throws; the rest of the try block is skipped
    System.out.println("never printed " + avg);
} catch (ArithmeticException e) {
    System.out.println("caught " + e.getClass().getName() + ": " + e.getMessage());
}
System.out.println("program continues normally");

static void setAge(int age) {
    if (age < 0) throw new IllegalArgumentException("age must be >= 0, was " + age);
    ...
}
```

Output:

```
average {4, 8} = 6
before the call
caught java.lang.ArithmeticException: / by zero
program continues normally
caught IllegalArgumentException: age must be >= 0, was -5
```

Propagation through the call stack

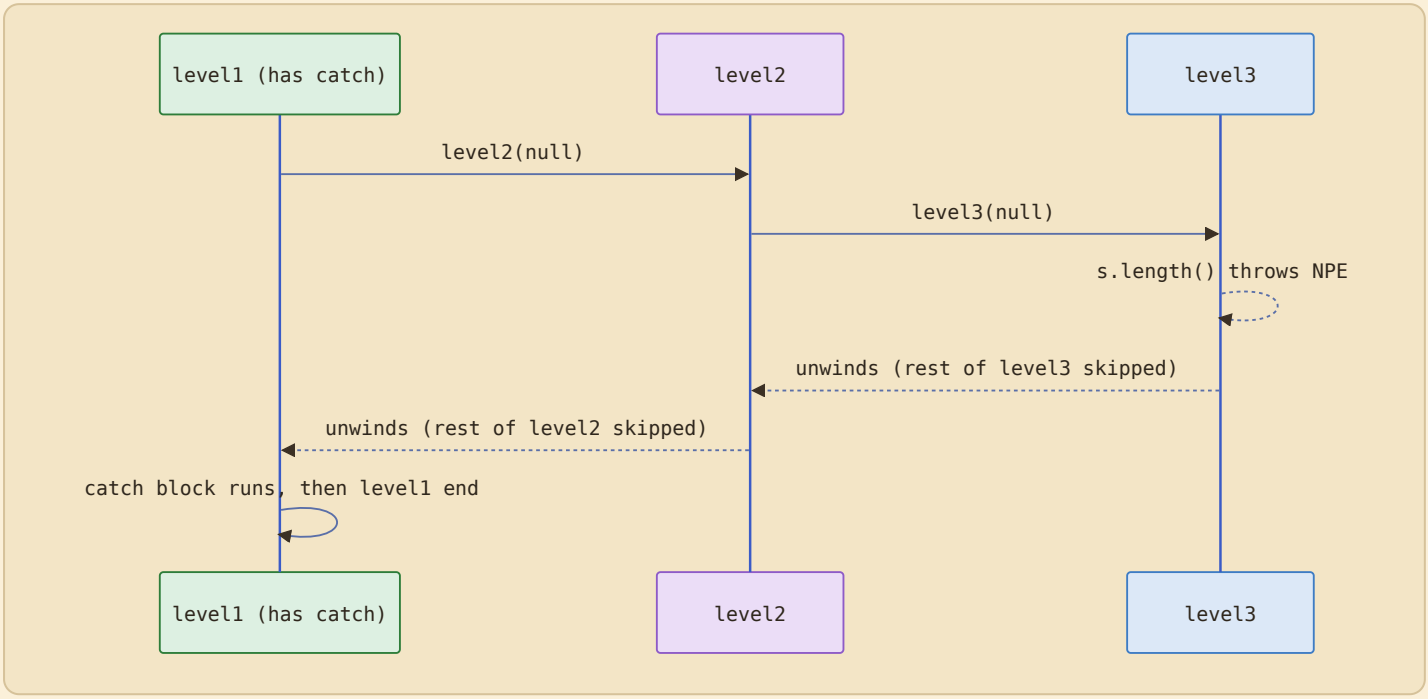
`code/ch08/basics/Propagation.java` calls `level1 → level2 → level3`; only `level1` has a `try`/`catch`:

```
static void level3(String s) {
    System.out.println("  level3 start");
    System.out.println("  length = " + s.length());   // NPE when s is null
    System.out.println("  level3 end");
}
static void level2(String s) { System.out.println(" level2 start"); level3(s); System.out.println(" level2 end");
}
static void level1(String s) {
    System.out.println("level1 start");
    try { level2(s); }
    catch (NullPointerException e) { System.out.println("level1 caught: " + e.getMessage()); }
    System.out.println("level1 end");
}
```

Output:

```
level1 start
 level2 start
  level3 start
  length = 2
  level3 end
 level2 end
level1 end
---
level1 start
 level2 start
  level3 start
level1 caught: Cannot invoke "String.length()" because "<parameter1>" is null
level1 end
```

With `null`, "level3 end" and "level2 end" never print: both methods were abandoned mid-way as the exception unwound to `level1`.



The message says `"<parameter1>"` because the runner compiles without debug info (extra data mapping compiled code back to variable names and line numbers). Compile with `javac -g` (IDEs and Maven/Gradle — the common build tools — do this by default) and the same line reads `because "s" is null`. These **helpful NullPointerException messages** arrived in Java 14 (JEP 358 — a numbered JDK Enhancement Proposal) and are on by default since Java 15.

Self-check

1. After a `catch` block finishes, where does execution continue? → After the entire `try` statement, not back at the throw point.
2. What happens to statements after the throwing line in the same `try`? → Skipped.
3. What happens if nobody catches an exception in `main`? → The stack trace is printed and the thread ends (exit status 1 if nothing else is running).

8.2 The `Throwable` hierarchy: checked, unchecked and errors

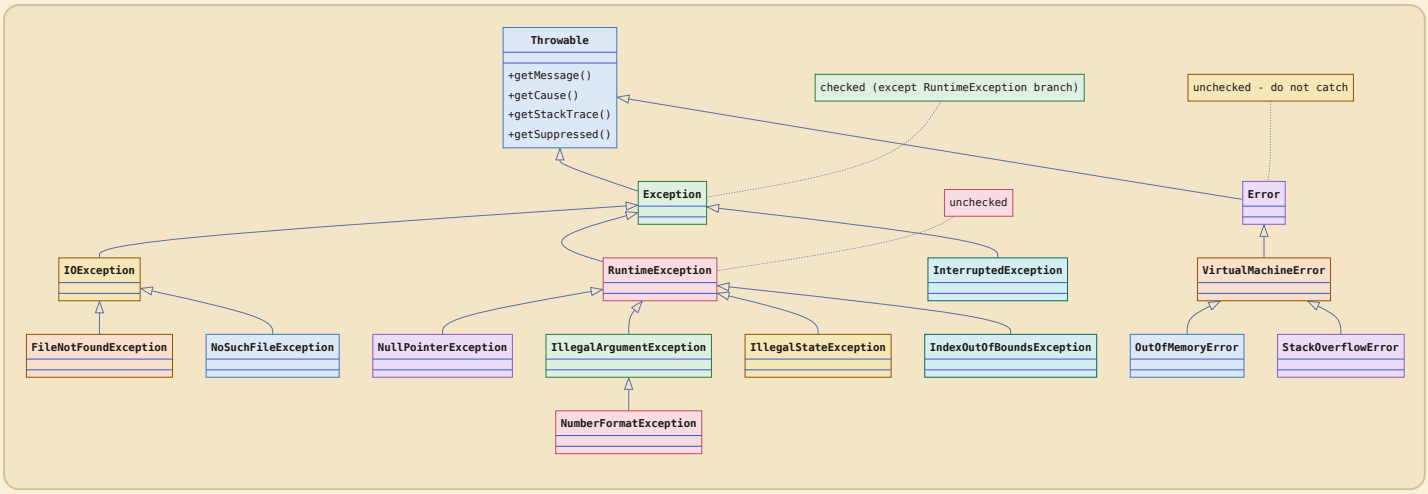
What it is

Only objects of class `java.lang.Throwable` (or its subclasses) can be thrown. It has two branches:

- `Error` — serious problems in the JVM or environment that an application normally shouldn't try to handle: `OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`, `ExceptionInInitializerError`.
- `Exception` — conditions a program might want to handle. Its subclass `RuntimeException` is special.

The compiler divides throwables into two kinds:

- **Checked exceptions**: `Exception` and its subclasses, *except* `RuntimeException` and its subclasses. The compiler **forces** you to handle or declare them (§8.3). They model problems outside the program's control that a caller can reasonably recover from: I/O (input/output) failures, network errors, "file not found", `InterruptedException` (thrown when a waiting thread is asked to stop).
- **Unchecked exceptions**: `RuntimeException` and its subclasses, **plus** `Error` and its subclasses. No compiler enforcement. `RuntimeException`s usually signal **programming errors** — a violated precondition (a condition that must be true before a method is called) such as a `null` argument, a bad index, an illegal state.



(`NoSuchFileException` is in `java.nio.file` and actually extends `FileSystemException`, which extends `IOException`.)

`code/ch08/hierarchy/Hierarchy.java` walks `getSuperclass()` for common types:

```
NullPointerException [unchecked]
    extends RuntimeException > Exception > Throwable
IllegalArgumentException [unchecked]
    extends RuntimeException > Exception > Throwable
NumberFormatException [unchecked]
    extends IllegalArgumentException > RuntimeException > Exception > Throwable
ArrayIndexOutOfBoundsException [unchecked]
    extends IndexOutOfBoundsException > RuntimeException > Exception > Throwable
IllegalStateException [unchecked]
    extends RuntimeException > Exception > Throwable
UnsupportedOperationException [unchecked]
    extends RuntimeException > Exception > Throwable
ClassCastException [unchecked]
    extends RuntimeException > Exception > Throwable
ArithmeticException [unchecked]
    extends RuntimeException > Exception > Throwable
UncheckedIOException [unchecked]
    extends RuntimeException > Exception > Throwable
IOException [CHECKED]
    extends Exception > Throwable
FileNotFoundException [CHECKED]
    extends IOException > Exception > Throwable
SQLException [CHECKED]
    extends Exception > Throwable
InterruptedException [CHECKED]
    extends Exception > Throwable
CloneNotSupportedException [CHECKED]
    extends Exception > Throwable
StackOverflowError [error, unchecked]
    extends VirtualMachineError > Error > Throwable
OutOfMemoryError [error, unchecked]
    extends VirtualMachineError > Error > Throwable
```

The exceptions you'll meet most

Exception	Kind	Typical cause
`NullPointerException`	unchecked	calling a method/field on `null`, unboxing (converting a wrapper like `Integer` to a primitive) `null`
`IllegalArgumentException`	unchecked	a method rejects an argument's value
`NumberFormatException`	unchecked (IAE subclass)	`Integer.parseInt("abc")`
`IllegalStateException`	unchecked	a call at the wrong time (e.g. `iterator.remove()` twice, using a closed object)
`IndexOutOfBoundsException` (+ `Array…`, `String…`)	unchecked	bad index
`ClassCastException`	unchecked	invalid downcast (cast to a more specific type)
`ArithmeticException`	unchecked	integer `/ 0`, `*Exact` overflow, non-terminating `BigDecimal` division
`UnsupportedOperationException`	unchecked	modifying an immutable collection (`List.of(...).add(x)`)
`ConcurrentModificationException`	unchecked	changing a collection while iterating it (Chapter 10)
`NoSuchElementException`	unchecked	`iterator.next()` past the end, `Optional.get()`/`orElseThrow()` on empty (`Optional` is a box that may or may not hold a value)
`DateTimeException`, `DateTimeParseException`	unchecked	invalid date values/text (Chapter 7)
`IOException` (+ `FileNotFoundException`, `NoSuchFileException`…)	checked	file/network I/O failures
`InterruptedException`	checked	a blocked thread was interrupted (Chapter 14)
`SQLException`	checked	database errors (JDBC, Java's standard database API)
`ExecutionException`, `TimeoutException`	checked	a `Future` (a handle to the result of a task running in the background) task failed / waited too long
`ExceptionInInitializerError`	error	exception in a `static` initialiser
`StackOverflowError`, `OutOfMemoryError`	error	runaway recursion; heap exhausted

Trade-offs — checked vs unchecked

	Checked	Unchecked
Compiler enforcement	Must catch or declare	None
Signals	Recoverable external condition	Programming error (bug) or unrecoverable state
API cost	Every caller must deal with it — even when they can't do anything useful	Callers can ignore it; documentation (`@throws`) matters more
Works with lambdas/streams (short inline functions / data-processing pipelines)	Poorly — `java.util.function` interfaces declare no checked exceptions (Chapter 11)	Fine
Examples	`IOException`, `SQLException`	`IllegalArgumentException`, `NullPointerException`

Java is unusual in having checked exceptions (C#, Kotlin, Python, JavaScript don't). *Effective Java* Items 70–71: use checked exceptions for conditions the caller can **recover** from, runtime exceptions for programming errors, and avoid checked exceptions when the caller can't do anything useful anyway. Most modern frameworks (Spring, an application framework; JPA/Hibernate, database-mapping libraries) wrap checked exceptions in unchecked ones for this reason.

In the interview

> **30-second answer:** "Everything thrown is a `Throwable`. `Error`s are JVM-level problems you don't catch. `Exception`s are either checked — the compiler forces catch-or-declare, used for recoverable external failures like `IOException` — or unchecked `RuntimeException`s, used for programming errors like `IllegalArgumentException` or `NullPointerException`. I use unchecked for precondition violations and checked only when the caller can genuinely recover."

- *Follow-up: Is `Error` checked?* → No; `Error` and `RuntimeException` (and subclasses) are the unchecked ones.
- *Follow-up: Should you catch `Throwable`?* → Only at the very top of a thread/request to log and fail cleanly — it also catches `Error`s like `OutOfMemoryError` where continuing may be unsafe.

Self-check

1. Is `FileNotFoundException` checked? → Yes (it's an `IOException`).
2. Is `NumberFormatException` a subclass of `IllegalArgumentException`? → Yes.
3. Which branch does `StackOverflowError` belong to? → `Error` (via `VirtualMachineError`).

TL;DR

`Throwable` → `Error` (don't catch) and `Exception`; `RuntimeException` + `Error` = unchecked; the rest of `Exception` = checked (must handle or declare).

8.3 Handle or declare: checked exceptions and `throws`

What it is

The **handle-or-declare rule** (sometimes called "catch or specify"): if code inside a method can throw a **checked** exception, the method must either

- **handle** it — wrap the code in `try` with a matching `catch`; or
- **declare** it — list it in the method's `throws` **clause**, passing the responsibility to its callers.

Otherwise the program doesn't compile: *"unreported exception IOException; must be caught or declared to be thrown"*.

`throw` vs `throws`: **`throw`** is a statement that throws one exception object now (`throw new IOException("x")`). **`throws`** is part of a method signature (the method's declaration line) listing exception *types* the method may throw (`void read() throws IOException, SQLException`).

How it works

`code/ch08/checked/CheckedDemo.java`:

```java
// "Declare": this method does not handle IOException, it passes it to the caller
static String readConfig(Path path) throws IOException {
    return Files.readString(path);
}

// "Handle": this method catches and decides what to do
static String readConfigOrDefault(Path path) {
    try {
        return readConfig(path);
    } catch (NoSuchFileException e) {           // more specific first
        System.out.println("no such file: " + e.getFile() + " -> using defaults");
        return "mode=default";
    } catch (IOException e) {                   // any other I/O problem
        System.out.println("I/O failure: " + e);
        return "mode=default";
    }
}

public static void main(String[] args) throws IOException {   // main may declare too
    Path tmp = Files.createTempFile("ch08-", ".properties");
    Files.writeString(tmp, "mode=fast");
    System.out.println("existing file: " + readConfigOrDefault(tmp));
    Files.delete(tmp);
    System.out.println("deleted file:  " + readConfigOrDefault(Path.of("missing-config.properties")));
    for (String in : new String[]{"42", "forty-two"}) {
        try { System.out.println("parsed " + Integer.parseInt(in)); }
        catch (NumberFormatException e) { System.out.println("bad number '" + in + "': " + e.getMessage()); }
    }
}
```

Output:

```
existing file: mode=fast
no such file: missing-config.properties -> using defaults
deleted file:  mode=default
parsed 42
bad number 'forty-two': For input string: "forty-two"
```

The rules in one table

Rule	Detail
Checked exceptions must be caught or declared	including those thrown by methods you call
Unchecked exceptions *may* be declared	useful as documentation, but the compiler ignores it; document with Javadoc `@throws`
You can't catch a checked exception that the `try` body can't throw	*"exception IOException is never thrown in body of corresponding try statement"* — except `Exception` and `Throwable`, which could always hide an unchecked one
Overriding methods may not throw **new or broader** checked exceptions	they may throw fewer, narrower, or any unchecked ones (Chapter 6 §6.3)
Lambdas	a lambda body can throw a checked exception only if the functional interface's (an interface with one abstract method, which the lambda implements) method declares it; `Runnable`, `Function` etc. don't (Chapter 11)
Initialiser blocks	a `static` initialiser may not throw a checked exception (unchecked ones are wrapped in `ExceptionInInitializerError`); an instance initialiser may, only if every constructor declares it

When to handle vs declare

- **Handle** where you can do something meaningful: retry, fall back to a default, ask the user again, translate to a better exception (§8.8).
- **Declare** when this layer can't fix it and the caller has more context. Low-level helpers usually declare; boundaries (a request handler, `main`, a UI event handler) usually handle.
- Never "handle" by catching and doing nothing (§8.10).

Self-check

1. `void m() { Thread.sleep(10); }` compiles? → No: `InterruptedException` is checked.
2. Must you declare `throws IllegalArgumentException`? → No, it's unchecked (you may, for documentation).
3. Can `catch (Exception e)` appear after a `try` that throws nothing? → Yes — `Exception` includes unchecked types.

8.4 `try` / `catch` / `finally` and control flow

What it is

A `finally` block runs after the `try` (and any `catch` that ran) **no matter how** the `try` ended: normally, by an exception (caught or not), or by `return`/`break`/`continue`. It exists for cleanup that must always happen — releasing a lock, resetting a flag, closing a resource (though for resources §8.6 is better).

How it works — the syntax rules

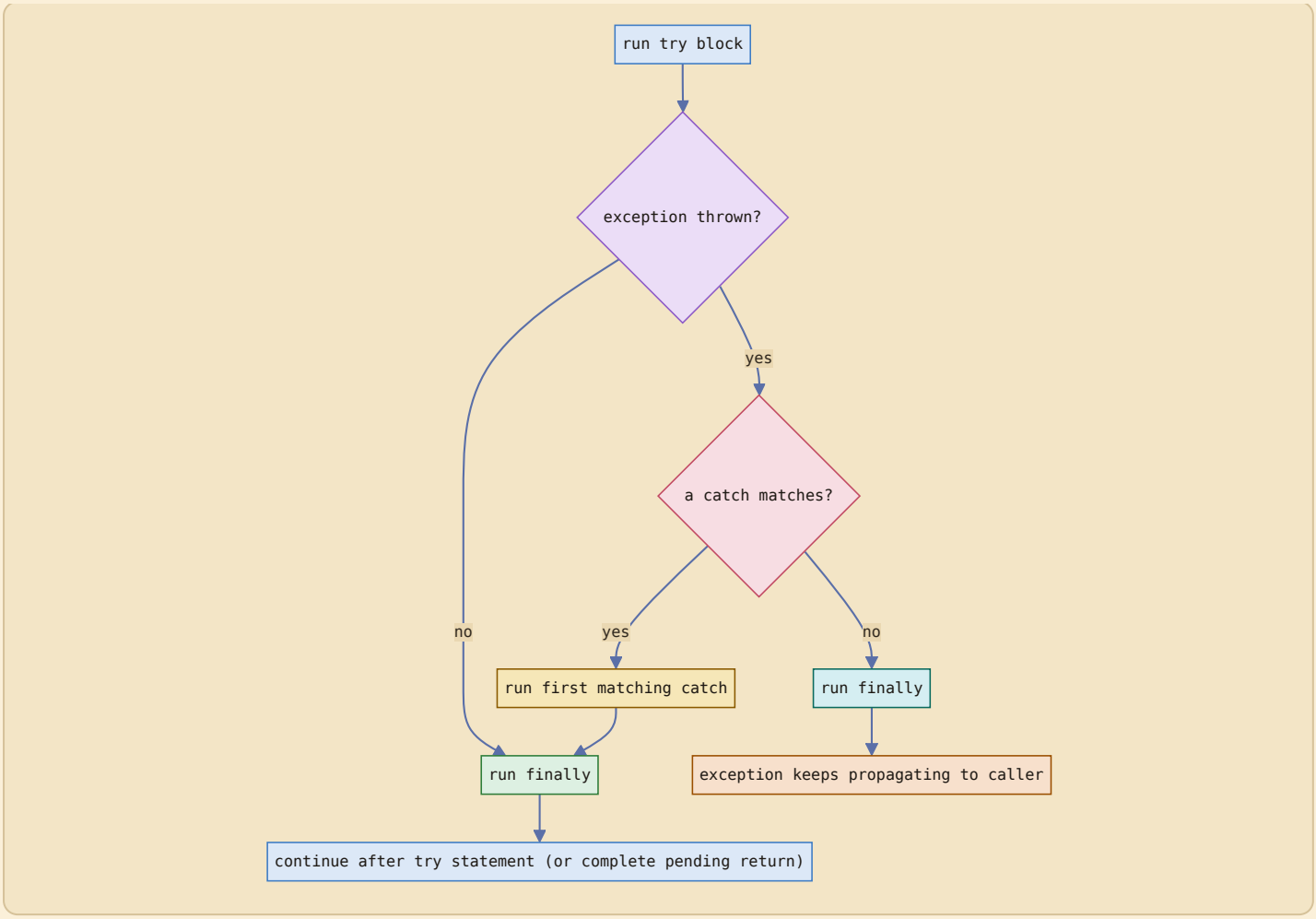
1. A `try` must be followed by at least one `catch` **or** a `finally` (or be a try-with-resources).
2. Order is fixed: `try`, then zero or more `catch`, then at most one `finally`.
3. At most one `catch` runs — the **first** one whose type matches.
4. `finally` runs after that `catch`, or directly after `try` if nothing was thrown or nothing matched (in which case the exception continues propagating **after** `finally`).
5. Braces are required even for one statement.

`code/ch08/finallyflow/FinallyDemo.java` (key parts):

```
static int returnInTry() {
    int x = 1;
    try {
        return x;             // the value 1 is saved here...
    } finally {
        x = 99;               // ...so changing x in finally does not change the returned value
        System.out.println("finally ran before the method returned");
    }
}

static int finallyOverrides() {
    try {
        throw new RuntimeException("lost!");
    } finally {
        return -1;            // BAD: discards the exception and any return value from try
    }
}
```

Output:

```
no failure: try try-end finally
failure:    try catch finally
finally ran before the method returned
returnInTry() = 1
finallyOverrides() = -1 (the RuntimeException vanished)
inner finally runs even though nobody here catches
outer caught: inner
```



The tricky cases

Situation	What happens
`return` in `try`, `finally` modifies the variable	The return **value** was already evaluated; a primitive result is unchanged (a returned object's *contents* can still be mutated)
`return` in `finally`	Overrides the `try`'s return **and silently discards any exception** — never do this (javac `-Xlint:finally` warns)
`finally` throws	The new exception replaces the original one, which is lost
`catch` throws	`finally` still runs, then the new exception propagates
`System.exit()` in `try`, JVM crash, power loss, infinite loop in `try`	`finally` does **not** run

Common mistakes

- Returning or throwing from `finally` (hides the real failure).
- Closing resources manually in `finally` with nested `try`s — verbose and easy to get wrong; use try-with-resources.
- Confusing `final` (can't reassign), `finally` (always-run block) and `finalize()` (deprecated for removal since Java 18, JEP 421).

Self-check

1. `try { return "a"; } finally { System.out.print("b"); }` — output and return value? → Prints `b`, returns `"a"`.
2. Is `try { } finally { }` legal? → Yes.
3. When doesn't `finally` run? → `System.exit`, JVM halt/crash, or the `try` never finishes.

8.5 Multi-catch, catch order and rethrowing

What it is

- **Multi-catch** (Java 7): one `catch` block for several unrelated exception types, separated by `|`: `catch (NumberFormatException | IndexOutOfBoundsException e)`.
- **Catch order**: because the first matching `catch` wins, a subclass must be caught **before** its superclass; otherwise the subclass block is unreachable and the code doesn't compile.
- **Precise rethrow** (Java 7): if you catch a broad type and rethrow the same (effectively final) variable, the compiler knows it can only be one of the checked types the `try` actually throws.
- **Unnamed variable** `_` (Java 22, JEP 456): write `catch (NumberFormatException _)` when the handler doesn't use the exception object.

`code/ch08/multicatch/MultiCatch.java` (key parts):

```
try {
    risky(i);        // 0: parseInt("x")  1: List.of(1,2).get(5)  2: IOException  3: fine
} catch (NumberFormatException | IndexOutOfBoundsException e) {     // multi-catch
    System.out.println(i + ": bad input -> " + e.getClass().getSimpleName());
} catch (java.io.IOException e) {
    System.out.println(i + ": I/O -> " + e.getMessage());
} finally {
    System.out.println(i + ": finally");
}

try { Integer n = (Integer) (Object) "text"; }
catch (ClassCastException e) { System.out.println("ClassCastException first"); }   // specific first
catch (RuntimeException e)   { System.out.println("RuntimeException"); }           // then general

try { value = Integer.parseInt(input); }
catch (NumberFormatException _) { value = -1; }                                    // Java 22

static void rethrow(boolean io) throws java.io.IOException, java.util.concurrent.TimeoutException {
    try {
        if (io) throw new java.io.IOException("io failed");
        throw new java.util.concurrent.TimeoutException("too slow");
    } catch (Exception e) {
        System.out.println("log and rethrow: " + e);
        throw e;                    // compiler knows e is IOException | TimeoutException
    }
}
```

Output:

```
0: bad input -> NumberFormatException
0: finally
1: bad input -> IndexOutOfBoundsException
1: finally
2: I/O -> disk full
2: finally
  no problem
3: finally
ClassCastException first
value = -1
log and rethrow: java.util.concurrent.TimeoutException: too slow
caller got TimeoutException
```

Compiler rules (real `javac` messages)

You write	Result
`catch (RuntimeException e) {} catch (ClassCastException e) {}`	error: *exception ClassCastException has already been caught*
`catch (IllegalArgumentException | NumberFormatException e)`	error: *Alternatives in a multi-catch statement cannot be related by subclassing* — just catch the parent
`catch (IllegalStateException | ArithmeticException e) { e = null; }`	error: *multi-catch parameter e may not be assigned* (it's implicitly `final`)
`catch (java.io.IOException e)` around code that can't throw it	error: *exception IOException is never thrown in body of corresponding try statement*

In a multi-catch, the variable's static type is the closest common supertype of the alternatives, so you can only call methods they share (usually those from `Throwable`).

Self-check

1. Why can't you write `catch (IOException | FileNotFoundException e)`? → `FileNotFoundException` is a subclass of `IOException`; catch `IOException` alone.
2. What's the benefit of precise rethrow? → You can catch broadly (to log) yet keep a narrow `throws` clause.
3. When is `catch (X _)` appropriate? → When the type alone tells you what to do and the object isn't needed (Java 22+).

8.6 try-with-resources

What it is

A **resource** is an object that must be closed when you're done — files, sockets, database connections, streams, locks wrapped as closeables. **try-with-resources** (Java 7) declares resources in parentheses after `try`; Java closes them automatically when the block ends, however it ends.

Any class implementing `java.lang.AutoCloseable` (single method `void close() throws Exception`) can be used. `java.io.Closeable` is the older sub-interface for I/O, whose `close()` throws `IOException` and must be idempotent (safe to call twice — calling it again has no further effect; the same idea matters a lot in distributed systems, see System Design Fundamentals).

Why it matters

Before Java 7, correct cleanup needed a `finally` with a null check and a nested `try` around `close()`, and a failure in `close()` could *replace* the real error. try-with-resources fixes both. *Effective Java* Item 9: "Prefer try-with-resources to try-finally."

How it works

1. Resources are initialised left to right in the header; each is implicitly `final`.
2. The body runs.
3. When the body finishes (normally or by exception), resources are **closed in reverse order** of declaration — **before** any `catch` or `finally` block of the same statement runs.
4. If a resource's initialisation failed, only the ones already opened are closed; `null` resources are skipped.
5. If the body throws *and* `close()` throws, the body's exception is the **primary** one; the close exceptions are attached to it as **suppressed** exceptions (`getSuppressed()`), never lost.
6. Java 9: you can use an existing **effectively final** (never reassigned after it is first set) variable directly: `try (reader) { ... }`.

`code/ch08/resources/TryWithResources.java` (key parts):

```java
static class Res implements AutoCloseable {
    Res(String name, boolean failOnClose) { ...; System.out.println("open " + name); }
    void use(boolean fail) { System.out.println("use " + name); if (fail) throw new IllegalStateException("use of
" + name + " failed"); }
    @Override public void close() {
        System.out.println("close " + name);
        if (failOnClose) throw new IllegalStateException("close of " + name + " failed");
    }
}

try (Res a = new Res("A", false); Res b = new Res("B", false)) { a.use(false); b.use(false); }

try (Res a = new Res("A", false)) { a.use(true); }
catch (IllegalStateException e) { System.out.println("catch: " + e.getMessage()); }
finally { System.out.println("finally"); }

try (Res a = new Res("A", true); Res b = new Res("B", true)) { b.use(true); }
catch (IllegalStateException e) {
    System.out.println("primary: " + e.getMessage());
    for (Throwable s : e.getSuppressed()) System.out.println("suppressed: " + s.getMessage());
}

BufferedReader reader = new BufferedReader(new StringReader("line one\nline two"));
try (reader) { System.out.println("first line: " + reader.readLine()); }     // Java 9
```

Output:

```
-- 1. normal: closed in reverse order
open A
open B
use A
use B
close B
close A
-- 2. body fails: resources closed BEFORE catch runs
open A
use A
close A
catch: use of A failed
finally
-- 3. body AND close fail: close exception is suppressed
open A
open B
use B
close B
close A
primary: use of B failed
suppressed: close of B failed
suppressed: close of A failed
-- 4. Java 9: effectively final variable as a resource
first line: line one
after close: Stream closed
```

Real-world use:

```java
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement("SELECT name FROM users WHERE id = ?")) {
    ps.setLong(1, id);
    try (ResultSet rs = ps.executeQuery()) { ... }
}   // rs, ps, con closed in that order — even if the query throws
```

(Not run — needs a database; JDBC is covered in Chapter 18.)

Common mistakes

- Opening a resource *outside* the `try (...)` without passing it in — it won't be closed if something fails in between.

- Returning a resource (e.g. a `Stream` — a lazily read sequence — from `Files.lines`) out of the `try` — it's closed by the time the caller uses it. Stream-returning I/O methods like `Files.lines`/`Files.walk` must themselves be used in try-with-resources (Chapter 13).
- Reassigning a resource variable: *"auto-closeable resource r may not be assigned"*.
- Wrapping streams: `try (var in = new BufferedReader(new FileReader(f)))` closes the outer reader, which closes the inner one; but if the *outer* constructor throws, the inner one leaks (is never closed). For strictness, declare each as its own resource.

In the interview

30-second answer: "try-with-resources closes any `AutoCloseable` automatically, in reverse order, before catch and finally run. If both the body and `close` throw, the body's exception wins and the close failures are attached as suppressed exceptions, so nothing is lost. It replaced error-prone `finally { if (x != null) x.close(); }` code."

- *Follow-up: `AutoCloseable` vs `Closeable`?* → `Closeable` (java.io) extends `AutoCloseable`, throws `IOException`, and its `close` should be idempotent.
- *Follow-up: How do you see a suppressed exception?* → `e.getSuppressed()`; stack traces print them under "Suppressed:".

Self-check

1. Three resources `a, b, c` — close order? → `c, b, a`.
2. Does `catch` run before or after `close`? → After.
3. Can a try-with-resources have no `catch`/`finally`? → Yes.

TL;DR

Anything `AutoCloseable` goes in `try (...)`. Reverse-order close, before catch/finally; secondary failures become suppressed, not lost.

8.7 Throwing exceptions and writing custom ones

Reuse standard exceptions first

Effective Java Item 72: favour standard exceptions — readers already know what they mean.

Situation	Throw
An argument value is invalid	`IllegalArgumentException`
A required argument is `null`	`NullPointerException` — `Objects.requireNonNull(x, "x")`
An index is out of range	`IndexOutOfBoundsException` — `Objects.checkIndex(i, size)` (Java 9)
The object is in the wrong state for this call	`IllegalStateException`
The operation isn't supported by this implementation	`UnsupportedOperationException`
A lookup found nothing and the method must return a value	`NoSuchElementException` (or return `Optional`, §8.10)
Concurrent modification detected	`ConcurrentModificationException`

Don't throw `Exception`, `RuntimeException`, `Throwable` or `Error` directly — callers can't tell your failure apart from anything else.

When to write your own

Create a custom exception when callers need to **distinguish** it (catch it specifically) or when it should **carry data** (an account ID, a shortfall amount, an error code). Guidelines:

1. Name it `…Exception`.
2. Extend `RuntimeException` for programming/state errors and most domain (business) rule violations in modern code; extend `Exception` only if callers are expected to recover and you want the compiler to remind them.
3. Provide the standard constructors you need: `(String message)` and `(String message, Throwable cause)`; the cause constructor is essential for chaining (§8.8).
4. Make added fields `final` and expose getters. Exceptions are `Serializable` (convertible to bytes for storage or transfer); strict linting (extra compiler warnings, `-Xlint:serial`) asks for a `serialVersionUID` (a version number for the serialised form).
5. Build a message with the failure-capture details (*Effective Java* Item 75): the values that caused it, not "Error occurred". Never include passwords or personal data.

`code/ch08/custom/CustomExceptions.java` (key parts):

```
// Checked: the caller is expected to recover (e.g. show "insufficient funds" to the user)
static class InsufficientFundsException extends Exception {
    private final BigDecimal shortfall;
    InsufficientFundsException(BigDecimal shortfall) {
        super("insufficient funds: short by " + shortfall);
        this.shortfall = shortfall;
    }
    BigDecimal getShortfall() { return shortfall; }
}

// Unchecked: a programming/state error in our domain
static class AccountFrozenException extends RuntimeException {
    AccountFrozenException(String accountId) { super("account " + accountId + " is frozen"); }
}

void withdraw(BigDecimal amount) throws InsufficientFundsException {
    if (amount.signum() <= 0) throw new IllegalArgumentException("amount must be positive: " + amount);
    if (frozen) throw new AccountFrozenException(id);
    if (amount.compareTo(balance) > 0) throw new InsufficientFundsException(amount.subtract(balance));
    balance = balance.subtract(amount);
}
```

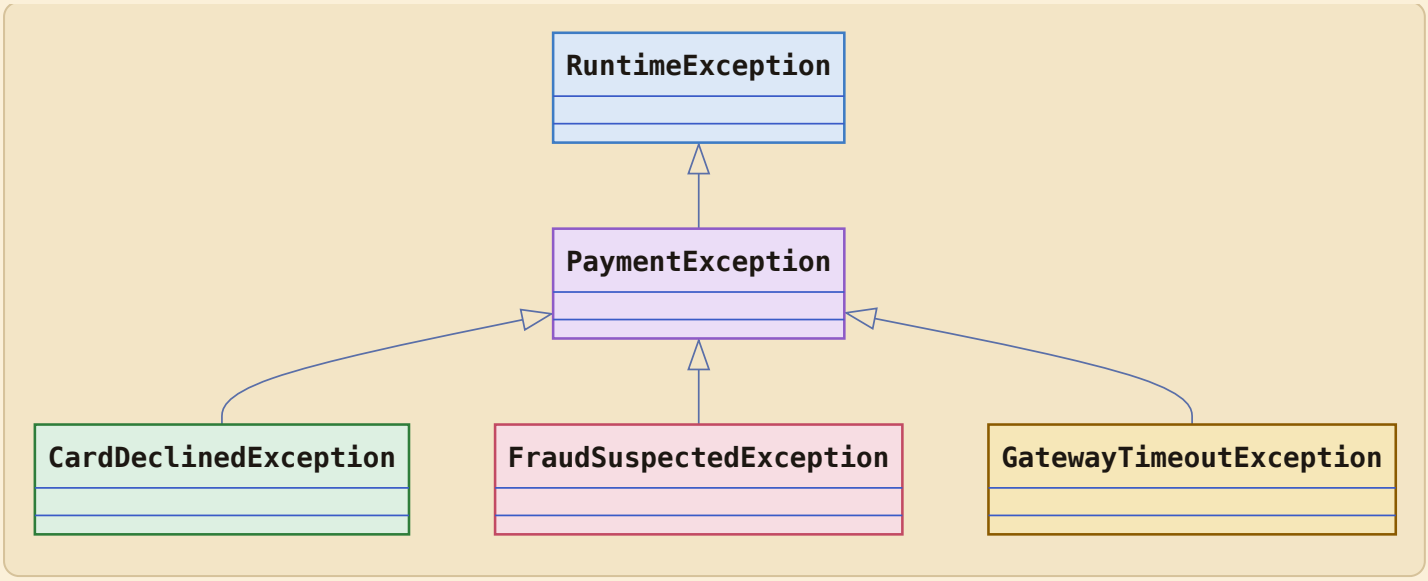
Output (first four lines):

```
withdrew 30.00, balance 70.00
insufficient funds: short by 430.00 (shortfall field = 430.00)
rejected: amount must be positive: -1
rejected: account ACC-1 is frozen
```

Note `withdraw` checks every condition **before** changing `balance`. That's **failure atomicity** (*Effective Java* Item 76): a failed call should leave the object as it was.

Custom exception hierarchies

For a library or service, a small hierarchy lets callers catch broadly or narrowly:



Keep it shallow; three or four types that callers actually handle differently beat twenty that nobody catches individually. A sealed interface (one that lists all its allowed implementations) or an error-code enum field can also work.

Self-check

1. Which exception for `setAge(-5)`? → `IllegalArgumentException`.
2. Which for calling `start()` twice on an engine? → `IllegalStateException`.
3. Why add fields to a custom exception? → So handlers can act on data (e.g. shortfall) without parsing the message.

8.8 Chaining and exception translation

What it is

Exception chaining stores the original exception as the **cause** of a new one: `new ReportException("could not build report", e)`. **Exception translation** (*Effective Java* Item 73) means a higher layer (a tier of the program, such as web → service → data access) catches low-level exceptions and throws ones that make sense at *its* level of abstraction — a `ReportService` shouldn't leak `SQLException` or `IOException` to its callers.

Chaining keeps the full story: the stack trace prints the high-level exception, then **"Caused by:"** with the original.

How it works

1. Catch the low-level exception.
2. Throw a new exception, passing the original as `cause` (constructor argument, or `initCause` for old exception types without a cause constructor).
3. Callers use `getCause()` to inspect the root, and logs show the whole chain.
4. To pass a checked exception through code that can't declare it (lambdas, `Runnable`), wrap it in an unchecked one — the JDK provides **`UncheckedIOException`** for `IOException`.

```
static String buildReport(String name) throws ReportException {
    try {
        return loadTemplate(name);                        // throws IOException
    } catch (IOException e) {
        throw new ReportException("could not build report " + name, e);   // translate + chain
    }
}

java.util.List.of("a").forEach(n -> {
    try { loadTemplate(n); }
    catch (IOException e) { throw new UncheckedIOException(e); }         // lambda can't throw IOException
});
```

Output (last three lines of `CustomExceptions`):

```
error: could not build report monthly
cause: java.io.IOException: template 'monthly' not found on disk
unwrapped cause: template 'a' not found on disk
```

Common mistakes

- `throw new ReportException("failed: " + e.getMessage());` — loses the original type and stack trace. **Always pass `e` as the cause.**
- Log-and-rethrow at every layer — the same failure appears five times in the logs. Log once, where it's handled.
- Wrapping everything in `RuntimeException` without a message — at least say what the operation was.

Self-check

1. How do you get the original exception from a wrapper? → `getCause()` (repeatedly, for the root).
2. What does "Caused by:" in a trace mean? → The next exception down the cause chain.

8.9 Reading stack traces

What it is

A **stack trace** lists the chain of method calls (**frames**) that were active when the exception was *created*, most recent first. Each frame shows `class.method(File.java:line)`. It's the single most useful debugging artefact in Java.

`code/ch08/stacktrace/StackTraces.java`:

```
static int parsePort(String text) { return Integer.parseInt(text); }

static int loadPort(String text) {
    try {
        return parsePort(text);
    } catch (NumberFormatException e) {
        throw new ServiceException("invalid port setting", e);
    }
}
// in main:
try { loadPort("80a"); }
catch (ServiceException e) { e.printStackTrace(System.out); ... }
```

Output (the line numbers inside `java.base` frames depend on the exact JDK build — these are from JDK 26):

```
ch08.stacktrace.StackTraces$ServiceException: invalid port setting
    at ch08.stacktrace.StackTraces.loadPort(StackTraces.java:17)
    at ch08.stacktrace.StackTraces.main(StackTraces.java:23)
Caused by: java.lang.NumberFormatException: For input string: "80a"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:527)
    at java.base/java.lang.Integer.parseInt(Integer.java:624)
    at ch08.stacktrace.StackTraces.parsePort(StackTraces.java:10)
    at ch08.stacktrace.StackTraces.loadPort(StackTraces.java:15)
    ... 1 more
top frame: ch08.stacktrace.StackTraces.loadPort line 17
caught StackOverflowError (an Error: normally you do NOT catch these)
```

How to read one

1. **First line**: exception class + message — *what* happened (`ServiceException: invalid port setting`).
2. **Frames below it**: *where*, most recent call first. The first frame in **your** code is usually where to look (`loadPort`, line 17).
3. **"Caused by:"**: the underlying exception. **Scroll to the last "Caused by:"** — the root cause is usually the real problem (`"80a"` isn't a number, from `parsePort` line 10).
4. `... 1 more`: the remaining frames are identical to the enclosing trace's frames, so they're elided.
5. `java.base/` prefix: the module (a named group of packages) the class belongs to (Chapter 16). `$` in a class name means a nested class.
6. **"Suppressed:"** sections list exceptions attached by try-with-resources (§8.6).

Tools

API	Use
`e.getMessage()`, `e.toString()`	message; class name + message
`e.printStackTrace()`	to `System.err` — fine for learning; in applications use a logger (a library that writes timestamped diagnostic messages: `log.error("msg", e)`) which records the trace
`e.getStackTrace()`	`StackTraceElement[]` for programmatic inspection
`e.getCause()`, `e.getSuppressed()`	navigate the chain
`StackWalker` (Java 9)	lazily inspect the *current* thread's stack without creating an exception
`Thread.setDefaultUncaughtExceptionHandler`	last-chance logging for threads that die from uncaught exceptions

The stack trace is captured when the exception object is **constructed**, which costs time proportional to stack depth. That's one reason not to use exceptions for normal control flow (§8.10). Frameworks sometimes pre-allocate exceptions or disable trace capture via the protected `Throwable(String, Throwable, boolean enableSuppression, boolean writableStackTrace)` constructor — a niche optimisation.

Self-check

1. Which "Caused by:" is usually the real problem? → The last one (the root cause).
2. Where does `printStackTrace()` write by default? → `System.err`.
3. What does `... 3 more` mean? → Three frames identical to the enclosing exception's frames were omitted.

8.10 Best practices and anti-patterns

The rules (with *Effective Java* item numbers)

#	Practice	Why
1	**Use exceptions only for exceptional conditions** (Item 69) — never for normal control flow such as ending a loop	Slower (stack capture), obscures logic, can hide real bugs
2	**Checked for recoverable conditions, runtime for programming errors** (Item 70); avoid needless checked exceptions (Item 71)	Don't force callers to catch what they can't handle
3	**Prefer standard exceptions** (Item 72)	Familiar semantics (meaning)
4	**Throw exceptions appropriate to the abstraction** (Item 73) — translate and chain	Layers stay decoupled; the cause is preserved
5	**Document every exception** with `@throws` (Item 74), including unchecked ones that callers should know about	The compiler can't tell callers about unchecked ones
6	**Put failure details in the message** (Item 75): the offending values	Traces become self-explanatory
7	**Strive for failure atomicity** (Item 76): validate before mutating	A failed call doesn't leave half-updated state
8	**Don't ignore exceptions** (Item 77)	Swallowed errors become silent data corruption
9	**Fail fast**: validate arguments at the start of public methods/constructors	The error surfaces near its cause
10	**Catch the most specific type you can handle**; catch broadly only at boundaries	Avoid accidentally catching bugs
11	**Log or rethrow, not both**, and log once with the exception object	No duplicate log noise; keep the trace
12	**Use try-with-resources** for anything closeable (Item 9)	Correct cleanup, suppressed exceptions

`code/ch08/practices/BestPractices.java` (key parts):

```
record User(String name, String email) {
    User {
        Objects.requireNonNull(name, "name");                                   // fail fast
        if (email == null || !email.contains("@")) throw new IllegalArgumentException("bad email: " + email);
    }
}

static Optional<String> findUser(int id) { return Optional.ofNullable(USERS.get(id)); }   // absence is normal
static String getUser(int id) {                                                           // absence is a bug
    String u = USERS.get(id);
    if (u == null) throw new NoSuchElementException("no user with id " + id);
    return u;
}

System.out.println("bounds check instead of catching: " + (i < data.length ? data[i] : "out of range"));

try {
    Thread.sleep(1);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();        // restore the interrupt flag
    ...
}
```

(The *interrupt flag* is a per-thread marker meaning "someone asked this thread to stop"; catching `InterruptedException` clears it, so you set it again.) Output:

```
findUser(3) -> guest
getUser(3) -> no user with id 3
requireNonNull -> NPE: name
validation -> bad email: not-an-email
helpful NPE: Cannot invoke "String.toUpperCase()" because "<local1>[1]" is null
bounds check instead of catching: out of range
done
```

(With `javac -g`, the helpful NPE line names the variable: `because "names[1]" is null`.)

Exception or `Optional` or special value?

The situation	Return / throw
"Not found" is a normal outcome (search, cache lookup)	`Optional<T>` (Chapter 11) or an empty collection — never `null` for collections
The caller broke the contract (null argument, bad index)	Unchecked exception
An external failure the caller can act on (file missing, remote service down)	Checked exception, or unchecked with clear docs
Validation of user input with many possible errors	A result object listing all errors (throwing on the first one gives a poor user experience)

Anti-patterns

```
// 1. Swallowing: the failure disappears
try { save(order); } catch (Exception e) { }

// 2. Pokemon catching ("gotta catch 'em all") deep inside business logic
catch (Throwable t) { ... }                // also catches OutOfMemoryError, StackOverflowError

// 3. Losing the cause
catch (IOException e) { throw new RuntimeException("failed"); }       // pass e!

// 4. Exceptions as control flow
try { int i = 0; while (true) arr[i++].process(); } catch (ArrayIndexOutOfBoundsException e) { }

// 5. Returning from finally (hides exceptions) — see 8.4

// 6. Swallowing InterruptedException without restoring the flag
catch (InterruptedException e) { }          // callers can no longer see the interrupt
```

If ignoring really is correct (e.g., closing something during shutdown), name the variable `ignored` or `_` and add a comment explaining why.

Assertions — a note

`assert condition : "message";` throws `AssertionError` if the condition is false, **but only when the JVM runs with `-ea`** (enable assertions); they're off by default. Use them for internal invariants (conditions that must always be true) during development and testing, never for validating arguments of public methods or for anything with side effects (changes beyond computing a result, such as writing a file or modifying a field).

Real-world examples

- Web frameworks (e.g. Spring's `@ControllerAdvice`) catch exceptions at the request boundary and map them to HTTP statuses (the numeric result codes of web responses; see System Design Fundamentals): `IllegalArgumentException` → 400, a custom `NotFoundException` → 404, anything else → 500 with a logged trace.
- JDBC and file APIs throw checked exceptions; frameworks translate them into unchecked hierarchies (Spring's `DataAccessException`) so business code (the code implementing the application's actual rules) isn't cluttered.

- `Objects.requireNonNull` at the top of constructors is used throughout the JDK itself.

In the interview

> **30-second answer:** "I validate early and throw standard unchecked exceptions for contract violations, use checked exceptions only for recoverable external failures, translate low-level exceptions at layer boundaries while chaining the cause, never swallow exceptions or return from `finally`, use try-with-resources for anything closeable, and handle — log once, map to a response — at the boundary. For expected absence I return `Optional` instead of throwing."

- *Follow-up: What's wrong with `catch (Exception e) {}`?* → It hides every failure, including bugs, leaving the system in an unknown state.
- *Follow-up: `final` vs `finally` vs `finalize`?* → Modifier; always-run block; deprecated GC hook (JEP 421, Java 18) — use try-with-resources or `Cleaner`.
- *Follow-up: Why restore the interrupt flag?* → Catching `InterruptedException` clears it; re-setting lets code further up see that cancellation was requested (Chapter 14).

Self-check

1. Should a `findById` that often finds nothing throw? → Usually no — return `Optional`.
2. What's failure atomicity? → A failed operation leaves the object in its prior state.
3. Are assertions enabled by default? → No; they need `-ea`.

TL;DR

Validate early, throw standard unchecked exceptions for bugs, checked only when recovery is realistic, translate with a cause at boundaries, never swallow, log once, and use `Optional` for normal absence.

Chapter TL;DR

Topic	Remember
Mechanism	`throw` stops the block; the stack unwinds to the first matching `catch`; uncaught → thread dies with a stack trace
Hierarchy	`Throwable` → `Error` (don't catch) / `Exception`; `RuntimeException` + `Error` unchecked; rest checked
Handle or declare	Checked exceptions must be caught or listed in `throws`; overriding can't widen checked exceptions
`finally`	Always runs (except `System.exit`/crash); return value already evaluated; never `return`/`throw` from it
Catch rules	First match wins; subclass before superclass; multi-catch types unrelated, variable final; precise rethrow; `catch (X _)` (22)
try-with-resources	`AutoCloseable`; reverse-order close before catch/finally; close failures suppressed; `try (existingVar)` (9)
Custom exceptions	Reuse standard ones first; custom when callers must distinguish or need data; message with details; cause constructor
Chaining	Always pass the cause; translate to your layer's abstraction; `UncheckedIOException` for lambdas
Stack traces	Read top line, first frame in your code, then the last "Caused by:"; compile with `-g` for variable names in NPE messages
Practices	Exceptions for exceptional cases, fail fast, failure atomicity, don't swallow, log once, `Optional` for normal absence

Where next: type-safe, reusable code with Chapter 9 — Generics. Checked exceptions inside lambdas and `Optional`: Chapter 11. Exceptions in threads and `ExecutionException`: Chapter 15. Exam traps: OCP Chapter 11. Service-level error handling: LLD Clean Code §16.

Chapter 9 — Generics

Every time you write `List<String>` or `Map<Long, Order>`, you're using **generics**: types and methods that take *other types* as parameters. Generics (Java 5) let one piece of code — a list, a cache (a store that keeps recently used values for quick reuse; see System Design Fundamentals), a repository (an object that saves and loads data), a `max` function — work with many types while the compiler still checks that you never put an `Integer` where a `String` belongs. They're the reason modern Java code has almost no casts (explicit `(Type) value` conversions).

This chapter covers writing your own generic classes, interfaces and methods; restricting type parameters with bounds (limits on which types are allowed); making APIs flexible with wildcards (`?`, standing for an unknown type) (and the **PECS** rule); and **type erasure** — the compiler removing generic type information after checking it, which is how generics are implemented, which explains every strange restriction (no `new T()`, no `new List<String>[10]`, no `instanceof List<String>` on an `Object`). It ends with the design rules from *Effective Java* (Joshua Bloch's best-practice book; Items 26–33) and how to read intimidating JDK signatures like `<T extends Comparable<? super T>>`.

Collections are the main *users* of generics; they get their own chapter (Chapter 10). Arrays' covariance (a `String[]` counts as an `Object[]`) and why generics are invariant (a `List<String>` does not count as a `List<Object>`) was previewed in Chapter 4 §4.6.

What you'll learn

§	Topic	You'll be able to…
9.1	Why generics	explain the problem raw types had and the vocabulary (type parameter, type argument, diamond)
9.2	Generic classes and records	write `Box<T>`, `Pair<K, V>` and a generic record
9.3	Generic interfaces	implement a generic interface with a fixed or generic type argument
9.4	Generic methods and inference	write `static <T> …` methods and understand how the compiler infers `T`
9.5	Bounded type parameters	use `<T extends Number>`, multiple bounds and recursive bounds
9.6	Invariance, wildcards and PECS	choose between `List<T>`, `List<? extends T>`, `List<? super T>` and `List<?>`
9.7	Type erasure	describe what the compiler does and what exists at run time (bridge methods)
9.8	Restrictions and workarounds	handle "no `new T()`", generic arrays, heap pollution and `@SafeVarargs`
9.9	Designing generic APIs	apply *Effective Java* Items 26–33 and read JDK signatures

All runnable code is in `code/` (package `ch09.*`), compiled with `javac --release 25` and run on JDK 26. Outputs are pasted from real runs; compile errors quoted in comments are real `javac` messages.

> **Exam depth.** Diamond/`var` combinations, wildcard "does this compile?" puzzles and bounded-type questions are drilled in OCP Chapter 9 — Collections and generics (sections 2 and 10). Generics in object design: LLD §2.15 Generics basics for LLD.

9.1 Why generics

What it is

Generics let you write a class, interface or method with a placeholder for a type — a **type parameter**, conventionally a single capital letter like `T` — which each user fills in with a concrete **type**

argument. `List<E>` is declared once; `List<String>` and `List<Order>` are two **parameterized types** made from it.

Analogy: a labelled storage box. A plain box accepts anything, and you must check what you pulled out. A box labelled "Screws only" is checked at the door — the wrong item never gets in, and you know exactly what comes out.

The problem it solves

Before Java 5, collections held `Object`. Nothing stopped a wrong type going in, and every read needed a cast that could fail far from the bug that caused it.

`code/ch09/why/WhyGenerics.java`:

```
List raw = new ArrayList();              // raw type: no type argument
raw.add("Ada");
raw.add(42);                             // compiles: nothing stops a wrong type going in
for (Object o : raw) {
    String s = (String) o;               // fails at RUN time on the Integer
    System.out.println("raw: " + s.toUpperCase());
}

List<String> names = new ArrayList<>();  // diamond <> (Java 7) infers <String>
names.add("Ada");
// names.add(42);                        // compile error: incompatible types: int cannot be converted to String
String first = names.get(0);             // no cast
```

Output:

```
raw: ADA
raw list -> ClassCastException: class java.lang.Integer cannot be cast to class java.lang.String
(java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
generic: ADA
```

Generics move the error from **run time** (a `ClassCastException` — the error thrown when a cast fails — in production, the live system real users depend on) to **compile time** (a red underline in your IDE), and remove the casts.

Vocabulary

Term	Example	Meaning
Generic type	`class Box<T>`	a class/interface declared with type parameters
Type parameter	`T` in `Box<T>`	the placeholder
Type argument	`String` in `Box<String>`	the concrete type supplied by the user
Parameterized type	`Box<String>`	generic type + arguments
Raw type	`Box`	the generic type used **without** arguments — legacy only
Diamond	`new Box<>("x")`	let the compiler infer the type arguments (Java 7; with anonymous classes — unnamed one-off classes — since Java 9)
Generic method	`static <T> T first(List<T> xs)`	a method with its own type parameter
Bound	`<T extends Number>`	a restriction on the type parameter
Wildcard	`List<?>`, `List<? extends Number>`	an unknown type argument at the use site

Raw types

A **raw type** exists only for compatibility with pre-2004 code. Using one turns off generic checking for that expression and produces "unchecked" warnings. *Effective Java* Item 26: **don't use raw types** in new

code. If you truly don't care about the element type, use the unbounded wildcard `List<?>` (§9.6), which stays type-safe (you can't add arbitrary things to it).

Two exceptions where the raw name is required: class literals (`List.class`, not `List<String>.class` — the object that represents a class at run time) and `instanceof` checks against a non-reifiable type (one whose full generic type isn't known at run time, §9.7) (write `instanceof List<?>` or `instanceof List`).

Self-check

1. What does `<>` do in `new HashMap<>()`? → Asks the compiler to infer the type arguments from the context.
2. When does a raw-type mistake fail? → At run time, typically a `ClassCastException` at a read far from the bad write.
3. What should you write instead of raw `List` when the element type doesn't matter? → `List<?>`.

9.2 Generic classes and records

How it works

1. Declare type parameters in angle brackets after the class name: `class Box<T>`, `class Pair<K, V>`.
2. Inside the class, use `T` like any type: field types, parameter types, return types, local variables.
3. Users supply the arguments: `Box<String>`, `Pair<String, Integer>`. With `var` or the diamond you rarely write them twice.
4. Type arguments must be **reference types** (class, interface or array types — not primitives) — `Box<int>` doesn't compile (*"unexpected type"*); use `Box<Integer>` (autoboxing, Chapter 7 §7.6).
5. Records (Java 16) and enums' surrounding classes can be generic too; enums themselves can't declare type parameters.

Naming conventions

Letter	Usually means	JDK example
`T` (then `U`, `V`, `S`)	Type	`Optional<T>`, `BiFunction<T, U, R>`
`E`	Element (collections)	`List<E>`, `Set<E>`
`K`, `V`	Key, Value	`Map<K, V>`
`R`	Result/return type	`Function<T, R>`
`N`	Number	
`ID` or longer names	when single letters confuse	`Repository<T, ID>` (Spring Data style — a library that generates database repositories)

`code/ch09/classes/GenericClasses.java` (key parts):

```java
static final class Box<T> {
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
    void set(T value) { this.value = value; }
    <R> Box<R> map(Function<? super T, ? extends R> f) {     // generic method inside a generic class
        return new Box<>(f.apply(value));
    }
}

record Pair<K, V>(K key, V value) {                          // generic record (Java 16)
    static <K, V> Pair<K, V> of(K k, V v) { return new Pair<>(k, v); }
    Pair<V, K> swap() { return new Pair<>(value, key); }
}

Box<String> s = new Box<>("hello");
Box<Integer> len = s.map(String::length);
Pair<String, Integer> p = Pair.of("age", 36);
var nested = new Box<List<Pair<String, Integer>>>(List.of(p));   // type arguments can nest
```

Output (first three lines):

```
Box[hello] -> Box[5], len.get() + 1 = 6
Pair[key=age, value=36] swapped Pair[key=36, value=age]
nested: age
```

What to notice:

- `len.get() + 1` needs no cast: the compiler knows `get()` returns `Integer`.
- `map` declares its **own** type parameter `R` in addition to the class's `T` — a generic method in a generic class. Its parameter uses wildcards (`? super T`, `? extends R`); §9.6 explains why that makes it more flexible.
- The static factory `Pair.of` declares its own `<K, V>`: static members can't use the class's type parameters (§9.8), so static generic factories always redeclare them.

When not to use it

- Don't make a class generic "just in case". If every user passes the same type, a plain class is simpler.
- If the type parameter appears only once in a method signature and nowhere else, a wildcard is usually clearer (§9.9).

Self-check

1. Can you write `new Box<int>(5)`? → No — type arguments must be reference types; use `Box<Integer>`.
2. Why does `Pair.of` declare `<K, V>` again? → Static methods can't see the class's type parameters.

9.3 Generic interfaces

What it is

Interfaces take type parameters exactly like classes: `Comparable<T>`, `Iterable<T>`, `Function<T, R>`, `Repository<T, ID>`. A class implementing one can:

1. **Fix the type argument** — `class StringStack implements Container<String>` (the class is not generic);
2. **Pass its own type parameter through** — `class Queue<T> implements Container<T>` (the class stays generic);
3. Be an **anonymous class or lambda** (a short inline function) with the type inferred.

`code/ch09/interfaces/GenericInterfaces.java` (key parts):

```java
@FunctionalInterface
interface Transformer<A, B> {
    B apply(A input);
    default <C> Transformer<A, C> andThen(Transformer<? super B, ? extends C> next) {
        return a -> next.apply(apply(a));
    }
}

record Version(int major, int minor) implements Comparable<Version> {    // concrete type argument
    public int compareTo(Version o) {
        int c = Integer.compare(major, o.major);
        return c != 0 ? c : Integer.compare(minor, o.minor);
    }
}

interface Container<T> { void put(T item); T take(); boolean isEmpty(); }
static class StringStack implements Container<String> { ... }            // 1. fixed
static class Queue<T> implements Container<T> { ... }                    // 2. stays generic
Container<Runnable> tasks = new Container<>() { ... };                   // 3. anonymous + diamond (Java 9)

static <T> List<T> drain(Container<T> c) { ... }                         // works for all three
```

Output:

```
8 chars
natural order: [8.0, 17.2, 21.0, 21.1]
by minor, then descending: [21.0, 8.0, 21.1, 17.2]
stack drains LIFO: [c, b, a], queue drains FIFO: [1, 2, 3]
task ran
```

And from `GenericClasses.java`, the repository pattern — a generic interface, a generic implementation and a subclass that fixes the types:

```java
interface Repository<T, ID> {
    void save(ID id, T item);
    Optional<T> findById(ID id);
    List<T> findAll();
}
static class InMemoryRepository<T, ID> implements Repository<T, ID> {
    private final Map<ID, T> store = new HashMap<>();
    ...
}
static class BookRepository extends InMemoryRepository<Book, String> {
    List<String> titles() { return findAll().stream().map(Book::title).sorted().toList(); }
}
```

Output:

```
findById: Effective Java | missing: false | titles: [Effective Java, Java Concurrency in Practice]
numbers: [100]
```

This is the shape of Spring Data's `CrudRepository<T, ID>`: write the persistence (saving to and loading from a database) logic once, reuse it for every entity type (a class whose objects are stored as database rows). On the design side, a repository is the class that owns persistence so domain classes (classes modelling the business concepts) don't have to — see the `EmployeeRepository` example in the LLD handbook's Single Responsibility Principle section.

Why `Comparable<Version>` and not raw `Comparable`

Implementing `Comparable<Version>` gives `compareTo(Version o)` — no cast, and comparing a `Version` to a `String` is a compile error. Raw `Comparable` would force `compareTo(Object o)` with a cast inside. `Comparable` and `Comparator` are covered fully in Chapter 10; functional interfaces like `Function<T, R>` in Chapter 11.

Self-check

1. `class Celsius implements Comparable<Celsius>` — what's `compareTo`'s parameter type? → `Celsius`.
2. Can a non-generic class implement a generic interface? → Yes, by fixing the type argument.

9.4 Generic methods and type inference

What it is

A **generic method** declares its own type parameters, written in angle brackets **before the return type**: `static <T> void swap(T[] arr, int i, int j)`. It can live in any class, generic or not. The JDK is full of them: `List.of`, `Collections.sort`, `Arrays.asList`, `Optional.of`, `Stream.map`.

Type inference is the compiler working out `T` from the arguments and from the **target type** (the type the result is assigned to or passed as), so you rarely write it.

How it works

1. Declare: `static <T> List<T> repeat(T item, int times)`.
2. Call normally: `repeat("hi", 3)` — `T` is inferred as `String` from the argument.
3. With no arguments to infer from, the target type decides: `List<String> empty = Collections.emptyList();`.
4. If needed, give an explicit **type witness**: `GenericMethods.<Number>repeat(1, 2)` (the class or `this.` prefix is required before `<…>`).
5. When arguments have different types, the compiler infers their most specific common supertype, which may be an *intersection type* (a type that is several types at once) like `Number & Comparable<…>`.

`code/ch09/methods/GenericMethods.java` (key parts):

```
static <T> void swap(T[] arr, int i, int j) { T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp; }
static <T> List<T> repeat(T item, int times) { ... }
static <T> T firstOrDefault(List<T> list, T fallback) { return list.isEmpty() ? fallback : list.get(0); }

swap(words, 0, 2);                                  // T inferred as String
swap(nums, 0, 1);                                   // T inferred as Integer
List<String> empty = Collections.emptyList();       // T = String from the assignment
List<Number> mixed = GenericMethods.<Number>repeat(1, 2);   // explicit witness
mixed.add(2.5);
var list = Arrays.asList(1, 2.0, 3L);               // List<Number & Comparable<...>>
```

Output:

```
[c, b, a] [2, 1, 3]
repeat: [hi, hi, hi]
firstOrDefault: none
witness <Number>: [1, 1, 2.5]
Arrays.asList(1, 2.0, 3L) element types: [Integer, Double, Long]
```

Without the witness, `repeat(1, 2)` would infer `List<Integer>` — then `add(2.5)` wouldn't compile. (Writing `List<Number> mixed = repeat(1, 2);` also works, because the target type drives inference.)

Common mistakes

- Putting `<T>` in the wrong place: it goes **after modifiers and before the return type** — `public static <T> T pick(...)`, not `public static T <T> pick(...)`.
- Declaring a method `<T>` in a generic class `Box<T>` — the method's `T` **shadows** (hides) the class's `T`; they're unrelated. Use a different letter.
- Writing `<T> void print(List<T> list)` when `T` is used once — `void print(List<?> list)` says the same thing more simply.

Self-check

1. How does the compiler infer `T` for `Collections.emptyList()` with no arguments? → From the target type of the assignment/parameter.
2. Syntax for an explicit type argument on a static generic method? → `ClassName.<Type>method(args)`.

9.5 Bounded type parameters

What it is

An unbounded `T` could be anything, so inside the code you can only call `Object` methods on it. A **bound** restricts which types are allowed — and in exchange lets you use the bound's methods:

- **Upper bound** `<T extends Number>`: `T` must be `Number` or a subtype (a class that extends it or implements it), so `t.doubleValue()` compiles. (`extends` is used for both classes and interfaces here.)
- **Multiple bounds** `<T extends Number & Comparable<T>>`: `T` must satisfy all. At most one class, and it must come **first**; the rest are interfaces.
- **Recursive bound** `<T extends Comparable<T>>`: "T can be compared with itself" — the pattern behind sorting and `max`.

(Type *parameters* have only upper bounds; `super` appears only in *wildcards*, §9.6.)

`code/ch09/bounds/Bounded.java` (key parts):

```
static <T extends Number> double sum(List<T> values) {
    double total = 0;
    for (T v : values) total += v.doubleValue();       // allowed because T extends Number
    return total;
}

static <T extends Comparable<? super T>> T max(List<? extends T> list) {
    T best = list.get(0);
    for (T x : list) if (x.compareTo(best) > 0) best = x;
    return best;
}

static <T extends Number & Comparable<T>> T clamp(T value, T lo, T hi) { ... }

static class Stats<T extends Number> {                  // bounded generic class
    double average() { return data.isEmpty() ? 0 : sum(data) / data.size(); }
}
// sum(List.of("a"));            // compile error: String is not within bound Number
```

Output:

```
sum ints=6.0 sum doubles=4.0
max words=zucchini max ints=9
clamp(15, 0, 10)=10 clamp(2.5, 0.0, 1.0)=1.0
Stats average=5.0
```

Reading `<T extends Comparable<? super T>>`

This is the signature of `Collections.max` and `Collections.sort` (simplified), and it looks scary. Read it in pieces:

1. `T extends Comparable<…>` — elements must be comparable.
2. `Comparable<? super T>` — comparable to `T` **or to some supertype of** `T`.

Why the `? super`? Suppose `class Fruit implements Comparable<Fruit>` and `class Apple extends Fruit`. `Apple` is `Comparable<Fruit>`, not `Comparable<Apple>`. With the simpler bound `<T extends Comparable<T>>`, you couldn't find the max of a `List<Apple>`; with `? super T` you can. Same story for `java.time`: `LocalDate` implements `Comparable<ChronoLocalDate>`.

Self-check

1. `<T extends Runnable & Serializable & Number>` — legal? → No; the class (`Number`) must come first.
2. Why can `sum` call `doubleValue()` on `T`? → The bound guarantees every `T` is a `Number`.
3. Why `Comparable<? super T>` instead of `Comparable<T>`? → To accept types that inherit `compareTo` from a supertype.

9.6 Invariance, wildcards and PECS

What it is

Invariance: even though `Integer` is a subtype of `Number`, **`List<Integer>` is *not* a subtype of `List<Number>`**. If it were, you could do this:

```
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;        // imagine this compiled...
nums.add(3.14);                  // ...then a Double would sit in a List<Integer>
```

So the compiler rejects the assignment: *"incompatible types: ArrayList cannot be converted to List"*.

Arrays made the opposite choice (they're **covariant**: `String[]` *is* an `Object[]`), and pay for it with a run-time check — `ArrayStoreException` (Chapter 4 §4.6). Generics catch the same mistake at compile time.

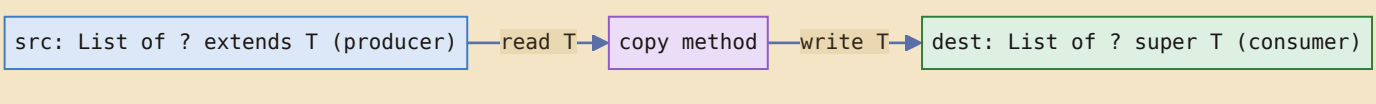
Wildcards restore flexibility safely. A wildcard `?` means "some specific type I don't know":

Wildcard	Read as	You can read elements as	You can add	Accepts
`List<? extends Number>`	a list of *some* subtype of Number	`Number`	nothing (except `null`)	`List<Number>`, `List<Integer>`, `List<Double>`
`List<? super Integer>`	a list of *some* supertype of Integer	`Object` only	`Integer` (and subtypes)	`List<Integer>`, `List<Number>`, `List<Object>`
`List<?>`	a list of *some* type	`Object`	nothing (except `null`)	any `List`
`List<Number>`	exactly a list of Number	`Number`	any `Number`	only `List<Number>`

PECS: Producer `extends`, Consumer `super`

The rule (coined by Joshua Bloch, *Effective Java* Item 31) for choosing a wildcard on a **method parameter**:

- If the parameter **produces** values your method reads → `? extends T`.
- If the parameter **consumes** values your method writes into it → `? super T`.
- If it does both → no wildcard, use exact `T`.
- If you only use `Object` methods / `size()` → `?`.



`code/ch09/wildcards/Wildcards.java` (key parts):

```java
static double total(Collection<? extends Number> source) {       // producer
    double t = 0;
    for (Number n : source) t += n.doubleValue();
    // source.add(1);                  // compile error: can't add to ? extends (except null)
    return t;
}

static void fillWithSquares(List<? super Integer> sink, int n) {  // consumer
    for (int i = 1; i <= n; i++) sink.add(i * i);
    Object o = sink.get(0);                // reading gives only Object
}

static <T> void copy(List<? super T> dest, List<? extends T> src) {   // both: shape of Collections.copy
    for (T item : src) dest.add(item);
}

static int countNulls(List<?> list) { ... }                       // unbounded

static void swapFirstLast(List<?> list) { swapHelper(list); }     // wildcard capture
private static <T> void swapHelper(List<T> list) {
    T first = list.get(0);
    list.set(0, list.get(list.size() - 1));
    list.set(list.size() - 1, first);
}
```

Output:

```
total(ints)=6.0 total(doubles)=0.75
view.get(0).intValue()=1
numbers=[1, 4, 9] objects=[1, 4]
copy -> [start, 1, 2, 3]
countNulls=2
swapFirstLast=[z, y, x]
arrays: ArrayStoreException java.lang.Integer
```

`total` accepts both `List<Integer>` and `List<Double>`; `fillWithSquares` accepts `List<Number>` and `List<Object>`. Without wildcards, each method would accept exactly one list type.

The commented-out `source.add(1)` fails with *"incompatible types: int cannot be converted to CAP#1"*. **CAP#1** is the compiler's name for the captured unknown type — it can't prove that 1 fits "some unknown subtype of Number" (the list might be a `List<Double>`).

Wildcard capture

You can't `set` into a `List<?>` because the element type is unknown. The **capture helper** trick passes it to a private generic method, where the unknown type gets a name (`T`), and reading-then-writing the same `T` is provably safe. This is how you'd implement something like `Collections.swap(List<?>, int, int)` — whose public signature uses `List<?>` precisely so callers don't need a type parameter. (The JDK's own implementation takes a shortcut with a raw type internally; a comment in its source notes that capturing the wildcard with a private helper would be the type-safe alternative.)

`List<?>` vs `List<Object>` vs raw `List`

	Accepts a `List<String>`?	Can add a `String`?	Type-safe?
`List<Object>`	No (invariance)	Yes	Yes
`List<?>`	Yes	No (only `null`)	Yes
raw `List`	Yes	Yes (with a warning)	**No**

When not to use wildcards

- **Not in return types.** `List<? extends Number> getValues()` forces every caller to deal with wildcards. Return `List<Number>` or `List<T>`.
- Not when the method both reads and writes the same parameter with type `T` — use `T`.

In the interview

> **30-second answer:** "Generics are invariant: `List<Integer>` isn't a `List<Number>`, otherwise you could add a `Double` to a list of integers. Wildcards make APIs flexible: PECS — if a parameter produces values I read, use `? extends T`; if it consumes values I write, use `? super T`. `Collections.copy(List<? super T> dest, List<? extends T> src)` is the textbook example. `List<?>` is a read-only-ish list of unknown type, safer than a raw `List`."

- *Follow-up: Why can't you add to `List<? extends Number>`?* → It might be a `List<Double>`, so adding an `Integer` could break it; the compiler only allows `null`.
- *Follow-up: What can you read from `List<? super Integer>`?* → Only `Object`, because the list could be a `List<Object>`.
- *Follow-up: Arrays vs generics?* → Arrays are covariant and reified (checked at run time, `ArrayStoreException`); generics are invariant and erased (checked at compile time).

Self-check

1. Parameter type for a method that sums a collection of any numeric type? → `Collection<? extends Number>`.
2. Parameter type for a method that adds `Integer`s to a caller's list? → `List<? super Integer>`.
3. Is `List<Object> x = new ArrayList<String>();` legal? → No.
4. Is `List<?> x = new ArrayList<String>();` legal? → Yes.

TL;DR

Generics are invariant. Producer → `extends`, consumer → `super`, both → exact `T`, don't care → `?`. Wildcards on parameters, never on return types.

9.7 Type erasure: how generics really work

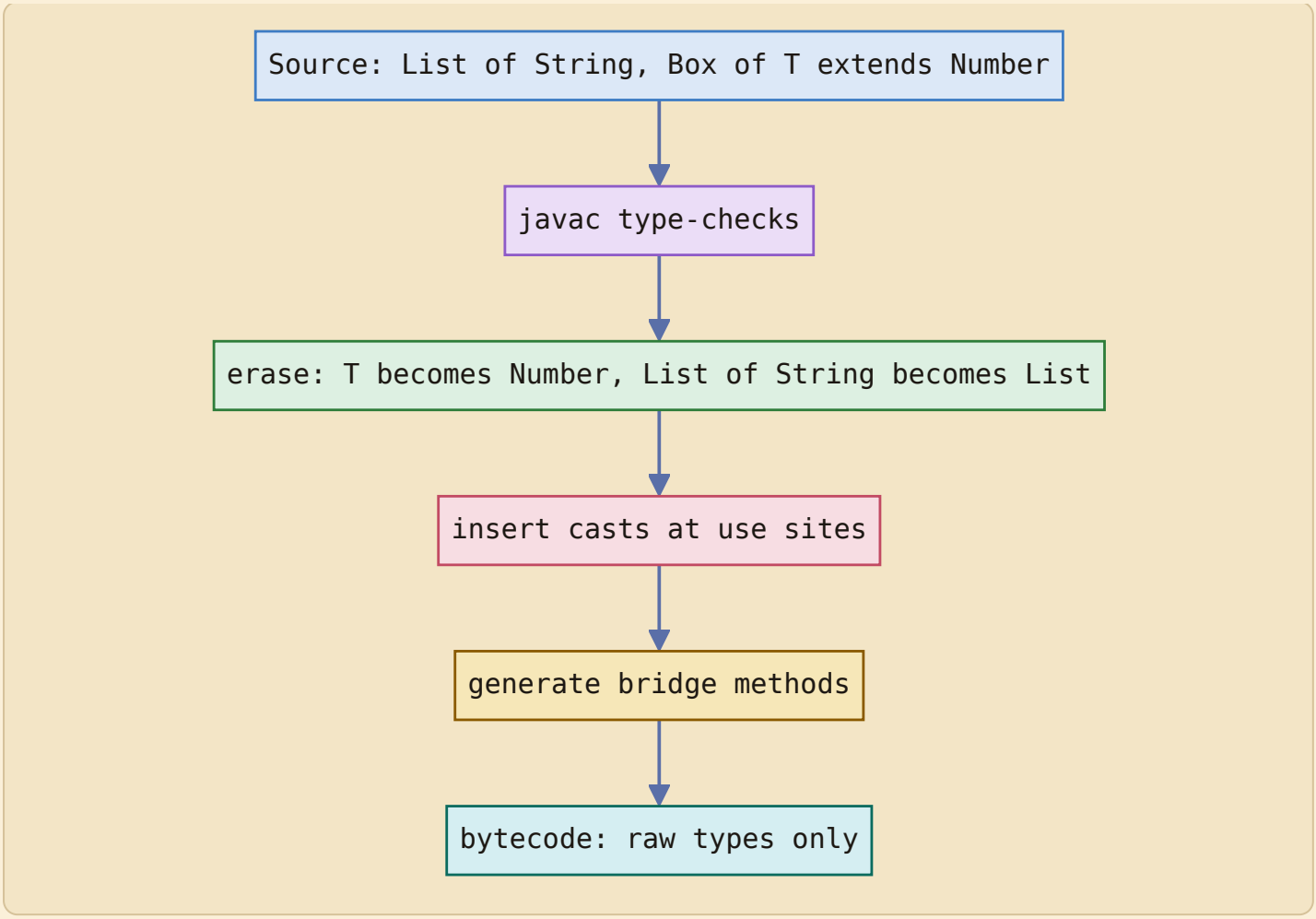
What it is

Java implements generics by **type erasure**: the compiler checks the generic types, then **removes** (erases) them from the bytecode (the compiled instructions the JVM runs). At run time, `List<String>` and `List<Integer>` are both just `ArrayList` — the JVM has no idea what the type argument was. Generic types are therefore **non-reifiable** (their full type isn't available at run time), unlike arrays, which *are* reified (`String[]` knows it holds `String`s).

Why? **Migration compatibility.** When generics arrived in Java 5 (2004), billions of lines of code and existing class files used raw collections. Erasure let new generic code and old non-generic code call each other, and let the JVM stay unchanged.

How it works

1. The compiler type-checks all uses of generic types.
2. Each type parameter is replaced by its **erasure**: its first bound (`<T extends Comparable<T>>` → `Comparable`), or `Object` if unbounded.
3. Parameterized types become raw types (`List<String>` → `List`).
4. The compiler **inserts casts** where a generic value is used as a specific type (`String s = list.get(0)` becomes `String s = (String) list.get(0)` in bytecode). These casts can never fail if you had no unchecked warnings.
5. Where erasure changes a method's signature so that overriding would break, the compiler generates a **bridge method**.



`code/ch09/erasure/Erasure.java` (key parts):

```
List<String> strings = new ArrayList<>();
List<Integer> ints = new ArrayList<>();
strings.getClass() == ints.getClass();             // true: one class at run time

static class Node<T extends Comparable<T>> implements Comparable<Node<T>> {
    public int compareTo(Node<T> o) { return value.compareTo(o.value); }
}
for (Method m : Node.class.getDeclaredMethods())
    if (m.getName().equals("compareTo")) System.out.println(... m.isBridge());
```

Output (relevant lines):

```
same runtime class? true -> java.util.ArrayList
o instanceof List<?>: true
compareTo(Node) bridge=false
compareTo(Object) bridge=true
```

`Comparable<T>` erases to `Comparable` with method `compareTo(Object)`. Our `compareTo(Node)` doesn't override that erased method, so the compiler adds a **bridge** `compareTo(Object o) { return compareTo((Node) o); }` to keep polymorphism (calling the right override for the object's actual class) working. You'll occasionally see bridge methods in stack traces (the list of active method calls printed with an error).

What survives erasure

Erasure removes type arguments from *objects and local variables*, but generic **declarations** keep their signatures in the class file (the `Signature` attribute). That's why reflection (code that inspects classes, fields and methods at run time) can tell you a field is declared `List<String>`, and why frameworks like Jackson and Gson (JSON libraries) use the **super type token** trick (`new`

`TypeReference<List<User>>() {}`) — an anonymous subclass records its generic superclass, which *is* preserved.

Trade-offs — erasure vs reification (e.g. C#)

	Java (erasure)	C# (reified generics)
Runtime knows `List<String>`?	No	Yes
`new T()`, `T[]`, `is List<string>`	Not allowed	Allowed
Primitives as type arguments	No (boxing)	Yes (`List<int>` without boxing)
Old/new code interoperability	Seamless	Separate generic collection types were added

Project Valhalla (OpenJDK) is working toward value types (objects without identity, stored as compactly as primitives) and better generics over primitives, but in Java 25 type arguments must still be reference types.

Self-check

1. What is `List<String>` at run time? → A plain `List`/`ArrayList` — type arguments are erased.
2. What does `<T extends Number>` erase to? → `Number`.
3. Why do bridge methods exist? → To preserve overriding after erasure changes method signatures.

9.8 Restrictions and workarounds (consequences of erasure)

Every restriction below follows from "the type argument doesn't exist at run time". Messages are real `javac` output.

You can't…	`javac` says	Workaround
Use primitives: `List<int>`	*unexpected type*	Wrapper types (`List<Integer>`) or primitive specialisations (`IntStream`, `int[]`)
Create an instance: `new T()`	*unexpected type*	Pass a `Supplier<T>` (a function that produces a value: `StringBuilder::new`) or a `Class<T>` token
Create a generic array: `new T[10]`, `new List<String>[10]`	*generic array creation*	Use `List<T>`/`ArrayList<T>`; or `Array.newInstance(type, n)` with a `Class<T>`; or `(T[]) new Object[n]` inside a class that never exposes it (as `ArrayList` does)
Use `T` in a static context: `static T shared;`	*non-static type variable T cannot be referenced from a static context*	Static generic methods declare their own `<T>`
`instanceof` a parameterized type from an unrelated static type: `o instanceof List<String>`	*Object cannot be safely cast to List*	`o instanceof List<?>`; (since Java 16, `instanceof ArrayList<String>` **is** allowed when the static type is already `List<String>`, because it's provably safe)
Overload on type arguments: `f(List<String>)` and `f(List<Integer>)`	*name clash: … have the same erasure*	Use different method names
Make a generic exception: `class Bad<X> extends Exception`	*a generic class may not extend java.lang.Throwable*	Store the extra data as a field of a concrete type; `catch` needs a runtime type
Use `.class` on a parameterized type: `List<String>.class`	syntax error	`List.class`

Workarounds in code

```
// Factory instead of new T()
static <T> List<T> fill(int n, Supplier<T> factory) { ... }           // fill(3, StringBuilder::new)

// Class token
static <T> T create(Class<T> type) throws ReflectiveOperationException {
    return type.getDeclaredConstructor().newInstance();
}

// Generic array via reflection
@SuppressWarnings("unchecked")
static <T> T[] newArray(Class<T> type, int n) {
    return (T[]) java.lang.reflect.Array.newInstance(type, n);
}
```

Output:

```
fill: 3 builders, create: ""
newArray component type: String[2]
```

Heap pollution and unchecked warnings

Heap pollution is when a variable of a parameterized type refers to an object that isn't of that type — e.g. a `List<Integer>` that actually contains a `String`. It can only happen through **unchecked** operations (ones the compiler can't verify: raw types, unchecked casts, generic varargs — `T...` parameters that accept any number of arguments), which is exactly why the compiler warns about them. The failure appears later, at an innocent-looking line where the compiler's inserted cast runs.

```
List<Object> objs = new ArrayList<>(List.of("text"));
@SuppressWarnings("unchecked")
List<Integer> lie = (List<Integer>) (List<?>) objs;   // compiles with a warning, no check at run time
Integer n = lie.get(0);                               // ClassCastException HERE, far from the lie
```

Output:

```
heap pollution -> ClassCastException on read
```

Effective Java Item 27: **eliminate every unchecked warning** you can. If you've proved a cast is safe, suppress the warning on the **smallest possible scope** (a single local variable declaration, as above — not the whole class) and add a comment explaining why it's safe.

Generic varargs and `@SafeVarargs`

A varargs parameter `T... items` is really an array `T[]`, and generic arrays can't be created safely — so the compiler creates an `Object[]` (the erasure) and warns: *"Possible heap pollution from parameterized vararg type T"*. The array is dangerous if the method **stores** it or **returns** it:

```
static <T> T[] toArray(T... items) { return items; }          // returns the varargs array: UNSAFE
static <T> T[] pickTwo(T a, T b) { return toArray(a, b); }    // T erased -> creates Object[]
String[] two = pickTwo("x", "y");                             // ClassCastException: Object[] is not String[]
```

Output:

```
pickTwo -> ClassCastException: generic varargs array escaped
```

A varargs method is **safe** if it only *reads* the array's elements and never lets the array escape. Then annotate it `@SafeVarargs` to silence the warning at every call site:

```
@SafeVarargs
static <T> List<T> listOf(T... items) { return new ArrayList<>(Arrays.asList(items)); }
```

`@SafeVarargs` is allowed only on methods that can't be overridden: `static`, `final`, constructors, and (since Java 9) `private` methods. `List.of(E...)`, `Arrays.asList` and `Stream.of` are all `@SafeVarargs`. *Effective Java* Item 32: combine generics and varargs judiciously; Item 28: prefer lists to arrays.

Self-check

1. How do you create a `T` inside a generic method? → Take a `Supplier<T>` or `Class<T>` parameter.
2. Why can't you have both `void f(List<String>)` and `void f(List<Integer>)`? → Same erasure, `f(List)`.
3. When is a generic varargs method safe? → When it doesn't store or expose the varargs array, only reads it.
4. What is heap pollution? → A parameterized-type variable referring to an object of the wrong type, caused by unchecked operations.

9.9 Designing generic APIs

Best practices (*Effective Java* Items 26–33)

Item	Rule	In one line
26	Don't use raw types	`List<?>` if you don't care about the type
27	Eliminate unchecked warnings	suppress only proven-safe cases, narrowly, with a comment
28	Prefer lists to arrays	lists are invariant and compile-time checked
29	Favour generic types	make containers generic instead of `Object`-based
30	Favour generic methods	especially static utilities and factories
31	Use bounded wildcards to increase API flexibility	PECS on parameters; never wildcards in return types
32	Combine generics and varargs judiciously	`@SafeVarargs` only when truly safe
33	Consider typesafe heterogeneous containers (holding values of many different types safely)	`Map<Class<?>, Object>` keyed by `Class<T>` tokens, e.g. `<T> T get(Class<T> key)`

More guidelines:

- **Type parameter vs wildcard:** if a type parameter appears only once in the signature, replace it with a wildcard (`void printAll(List<?> xs)` instead of `<T> void printAll(List<T> xs)`). Use a type parameter when it links two things — a parameter and the return type, or two parameters.
- **Return concrete generic types**, not wildcards.
- **Prefer `List<T>` over `T[]`** in public APIs.
- Keep type parameter lists short; three or more parameters is a smell (a hint the design could be simpler) unless it's a genuine tuple (fixed group of values)/function type (`BiFunction<T, U, R>`).

Reading JDK signatures

Signature	How to read it
`static <T> void sort(List<T> list, Comparator<? super T> c)`	Sort a list of `T` using a comparator that can compare `T` or any supertype (a `Comparator<Object>` or `Comparator<Fruit>` works for a `List<Apple>`)
`static <T extends Object & Comparable<? super T>> T max(Collection<? extends T> coll)`	Elements must be comparable to themselves or a supertype; `& Object` is there so the *erasure* of `T` stays `Object` for binary compatibility (already compiled code keeps working without recompiling) with pre-Java-5 code
`<R> Stream<R> map(Function<? super T, ? extends R> mapper)`	The function may accept any supertype of `T` (consumer of `T`) and return any subtype of `R` (producer of `R`) — PECS on a function
`static <T, U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? extends U> keyExtractor)`	Build a `Comparator<T>` from a function extracting a comparable key `U` from each `T`
`boolean addAll(Collection<? extends E> c)`	Accept any collection whose elements are `E` or subtypes

Once PECS clicks, these stop being noise: every `? super` is something that **takes** your values, every `? extends` is something that **gives** you values.

Common mistakes

- Using raw types (often by forgetting `<>`: `new ArrayList()` → a raw-type warning).
- `List<Object>` parameters where `List<?>` or a type parameter was meant — callers with `List<String>` can't pass it.
- Wildcards in return types.
- Suppressing unchecked warnings at class level.
- Forgetting that generics give **no runtime checks** — data from reflection, serialization or raw-typed legacy code can still pollute the heap. `Collections.checkedList(list, String.class)` adds runtime checks when you need to find where bad data enters.

Real-world examples

- `Optional<T>`, `CompletableFuture<T>` (the future result of background work), `Stream<T>`, `Map<K, V>` — the whole modern JDK API is generic.
- Spring Data `JpaRepository<T, ID>`, Jackson `TypeReference<T>`, JUnit's (the standard Java testing library) `assertThrows(Class<T> expectedType, Executable)` returning `T` — note the `Class<T>` token letting the method return the right exception type without a cast.
- Event buses (objects that deliver events to subscribers) and dependency-injection containers (frameworks that create objects and hand each one the objects it needs) use typesafe heterogeneous containers keyed by `Class<T>` (Item 33).

In the interview

30-second answer: "Generics give compile-time type safety and remove casts. They're implemented by erasure: type arguments are checked and then removed, so at run time a `List<String>` is just a `List`. That's why you can't do `new T()`, create generic arrays, use primitives as type arguments, or overload on `List<String>` vs `List<Integer>`. They're invariant, so I use bounded wildcards with PECS to make APIs flexible."

- *Follow-up: What is a bridge method?* → A synthetic (not written by you) method the compiler generates so that overriding still works after erasure changes a signature.
- *Follow-up: How do libraries get the full generic type at run time?* → Super type tokens — an anonymous subclass of `TypeReference<List<User>>` keeps its generic superclass in the class file, readable via reflection.

- *Follow-up:* `<T extends Comparable<T>>` *vs* `<T extends Comparable<? super T>>`*?* → The second also accepts types that inherit `compareTo` from a supertype (e.g. `Apple extends Fruit implements Comparable<Fruit>`).

Self-check

1. `<T> void log(List<T> xs)` — better signature? → `void log(List<?> xs)`.
2. Why does `Collections.max` use `T extends Object & Comparable<? super T>`? → So the erased signature returns `Object`, preserving binary compatibility with pre-generics code.
3. What does `Collections.checkedList` do? → Wraps a list with runtime type checks on insertion to catch heap pollution at its source.

TL;DR

Generic types/methods for reuse with type safety; bounds to use a type's methods; PECS wildcards on parameters; erasure explains every restriction; no raw types, no unchecked warnings left unexplained, lists over arrays.

Chapter TL;DR

Topic	Remember
Purpose	Compile-time type safety, no casts; raw types are legacy — use `List<?>` if unsure
Syntax	`class Box<T>`, `record Pair<K, V>`, `interface Repo<T, ID>`, `static <T> T m(...)`; diamond `<>` (7, anonymous classes 9)
Inference	From arguments and target type; explicit witness `Cls.<T>m()`
Bounds	`<T extends Number>`, `<T extends A & I1 & I2>` (class first), recursive `<T extends Comparable<? super T>>`
Invariance	`List<Integer>` is not a `List<Number>`; arrays are covariant (runtime `ArrayStoreException`)
Wildcards	PECS: producer `? extends`, consumer `? super`, both → `T`, don't care → `?`; not in return types
Erasure	Type args removed after checking; `T` → bound or `Object`; casts inserted; bridge methods
Restrictions	No primitives, `new T()`, `new T[]`, static `T`, `instanceof List<String>` (from `Object`), overload by type arg, generic exceptions
Heap pollution	From unchecked operations; fix warnings; `@SafeVarargs` only for non-escaping varargs arrays
Design	*Effective Java* 26–33; lists over arrays; wildcard if a type param appears once

Where next: the biggest consumer of generics — Chapter 10 — Collections — then generic functional interfaces in Chapter 11 — Lambdas and functional programming. Exam traps: OCP Chapter 9 — Collections and generics.

Chapter 10 — Collections

Almost every real program holds *groups* of things: the items in a cart, the users online, the word counts of a document, the jobs waiting to run. Arrays (Chapter 4) have a fixed length and almost no behaviour. The **Java Collections Framework** (in `java.util`) gives you resizable lists, duplicate-free sets, key→value maps, queues (first-in, first-out waiting lines), stacks (last-in, first-out piles) and priority queues — all sharing a small set of interfaces (contracts that list the methods a type must offer) so you can swap one implementation (a concrete class that fulfils the contract) for another without rewriting code.

This chapter teaches what each collection is, **how it works inside** (so the big-O costs — how an operation's running time grows as the collection gets bigger, explained in §10.12 — make sense rather than being memorised), which one to pick, and the traps that cause real bugs. It leans on generics (`List<String>`, `? extends T`) from Chapter 9 and on `equals`/`hashCode` from §6.13. Lambdas such as `n -> n % 2 == 0` appear in a few examples; they are explained fully in Chapter 11 — for now read them as "a small function".

What you'll learn

1. The collection hierarchy: `Iterable`, `Collection`, `List`, `Set`, `Queue`, `Deque`, and why `Map` stands apart.
2. `ArrayList` vs `LinkedList` — internals, costs and the classic traps.
3. `HashSet`, `LinkedHashSet`, `TreeSet` and navigation queries.
4. The `Map` API (`merge`, `computeIfAbsent`, …), `LinkedHashMap` (LRU cache — a cache that throws out the Least Recently Used entry when full; see System Design Fundamentals), `TreeMap`.
5. `HashMap` **internals**: hashing, buckets, load factor, resizing, treeification — with a measured collision demo.
6. `Queue`, `Deque` (`ArrayDeque` as stack and queue) and `PriorityQueue` (a heap — a tree-shaped structure that always keeps the smallest item on top; unrelated to the JVM's memory heap).
7. Iterators, fail-fast behaviour and safe removal.
8. `Comparable` vs `Comparator` and comparator composition.
9. The `Collections` utility class, unmodifiable views vs immutable copies (`List.of`, `copyOf`).
10. Sequenced collections (Java 21).
11. A big-O table, a measured performance demo, and a decision guide for choosing a collection.

All runnable code is in `code/` (package `ch10.*`), compiled with `javac --release 25` and run on JDK 26.

> **Exam depth.** `List.of` vs `Arrays.asList` puzzles, `remove(int)` vs `remove(Object)`, `TreeSet`/`TreeMap` ordering questions, `Comparator` chaining and generics wildcards are drilled in OCP Chapter 9 — Collections and generics (see its *Gotchas & exam traps* section). This chapter focuses on understanding and real use.

10.1 The Collections Framework and its hierarchy

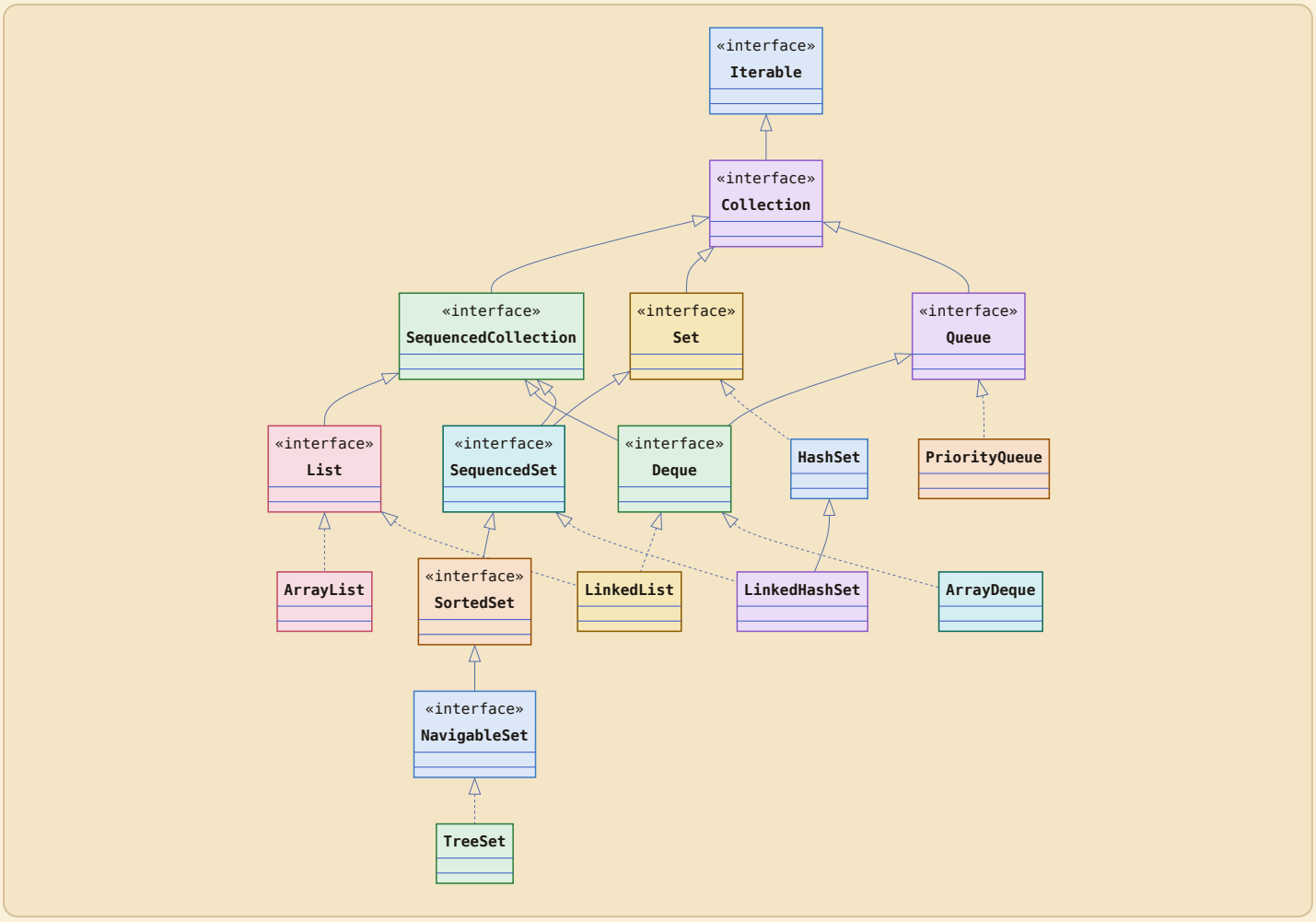
What it is

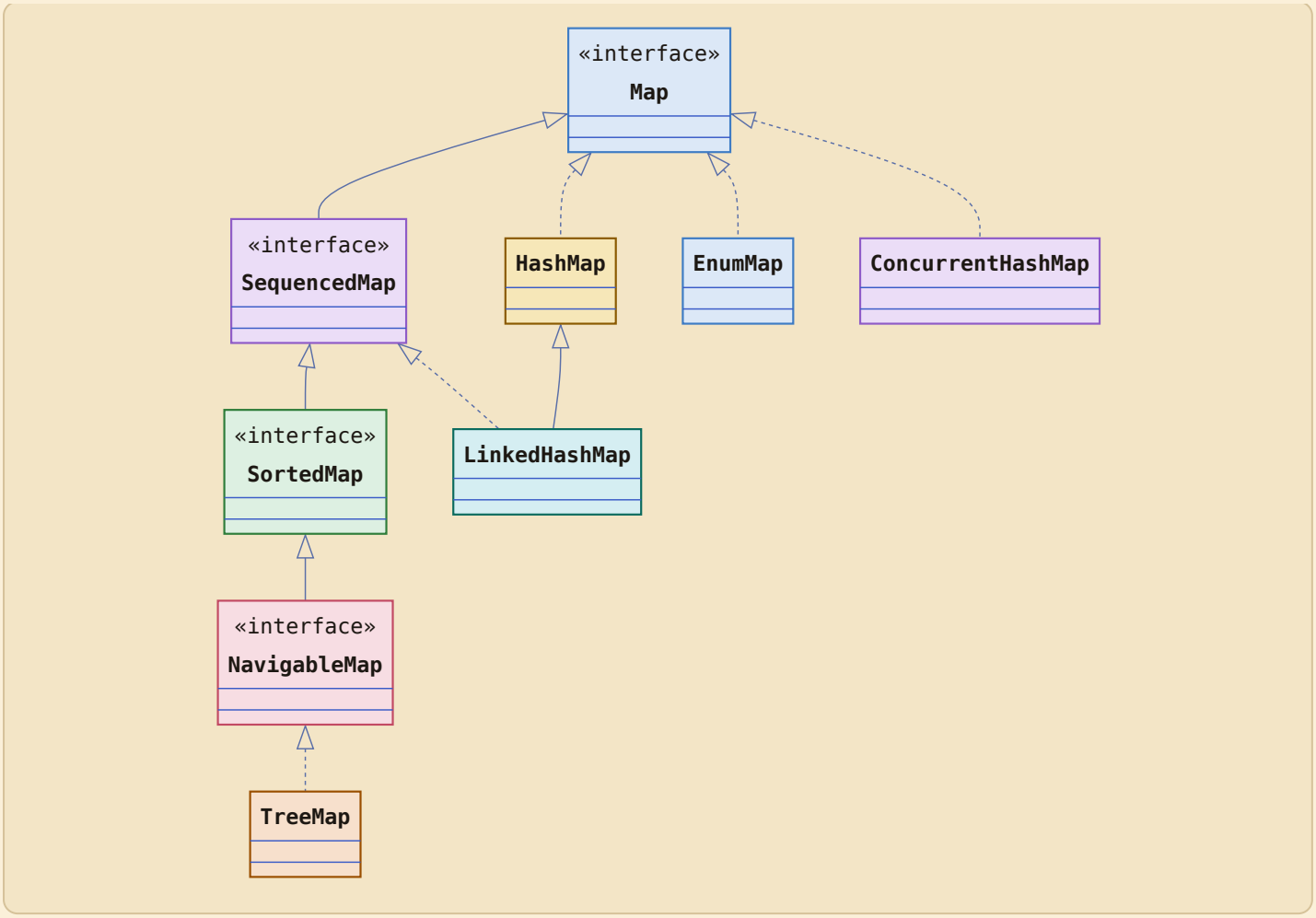
A **collection** is an object that groups other objects (its **elements**). The **Collections Framework** is three things:

1. **Interfaces** — abstract contracts such as `List` ("ordered, indexable, duplicates allowed") and `Set` ("no duplicates").
2. **Implementations** — concrete classes such as `ArrayList` and `HashSet` that fulfil those contracts with a particular data structure (a way of organising data in memory).
3. **Algorithms** — reusable static methods (sorting, searching, shuffling) in `Collections` and `Arrays`.

Analogy: the interface is the job description, the implementation the person hired — you can hire someone else without rewriting the description.

How it is organised





Interface	Promise	Main implementations
`Iterable<T>`	Can be looped over with for-each	every collection
`Collection<E>`	A group of elements: `add`, `remove`, `contains`, `size`, `iterator`, `stream`	(root of the element-based types)
`List<E>`	Ordered by index, duplicates allowed	`ArrayList`, `LinkedList`
`Set<E>`	No duplicates (by `equals`)	`HashSet`, `LinkedHashSet`, `TreeSet`, `EnumSet`
`Queue<E>`	Elements wait to be processed, usually FIFO (first in, first out)	`ArrayDeque`, `LinkedList`, `PriorityQueue`
`Deque<E>`	"Double-ended queue": add/remove at both ends; also a stack	`ArrayDeque`, `LinkedList`
`Map<K,V>`	Unique keys, each mapped to one value	`HashMap`, `LinkedHashMap`, `TreeMap`, `EnumMap`
`SequencedCollection` / `SequencedMap` (21)	Has a defined first and last element	`List`, `Deque`, `LinkedHashSet`, `TreeSet`, `LinkedHashMap`, `TreeMap` (§10.11)

Key terms used throughout:

- **Ordered / encounter order** — iteration visits elements in a defined sequence (index order, insertion order or sorted order).
- **Sorted** — ordered by comparing elements (natural order — the type's own built-in ordering — or a `Comparator`, §10.8).

- **Structural modification** — a change in size (add/remove), as opposed to replacing an element with `set`.
- **Optional operation** — an interface method an implementation may reject with `UnsupportedOperationException` (e.g. `add` on an unmodifiable list).

`Map` is not a `Collection`. A map stores pairs, so `add(E)` makes no sense for it. Instead it offers three **views** (live windows onto the map's data — changes show through both ways) that *are* collections: `keySet()`, `values()` and `entrySet()`.

Program to the interface

```
// code/ch10/hierarchy/HierarchyDemo.java (key parts)
Collection<String> langs = new ArrayList<>();          // declared type = interface
langs.add("Java"); langs.add("Go"); langs.add("Java"); // lists allow duplicates
langs.addAll(List.of("Rust", "Kotlin"));
langs.remove("Java");                                  // removes the FIRST match only
langs.removeIf(s -> s.length() <= 2);                  // bulk remove by condition (Java 8)

Collection<String> unique = new HashSet<>(List.of("Java", "Go", "Rust"));
System.out.println("add duplicate to set -> " + unique.add("Java"));   // add returns false

String[] arr = langs.toArray(String[]::new);           // typed array (Java 11 form)
Map<String, Integer> ages = new TreeMap<>(Map.of("ana", 31, "bo", 25));
System.out.println("keys=" + ages.keySet() + " values=" + ages.values() + " entries=" + ages.entrySet());
```

Output:

```
list-backed: [Java, Go, Java, Rust, Kotlin] size=5
contains Go? true
after removes: [Java, Rust, Kotlin]
add duplicate to set -> false
array: [Java, Rust, Kotlin] isEmpty=false
keys=[ana, bo] values=[31, 25] entries=[ana=31, bo=25]
Map is a Collection? false
```

Declaring variables and parameters as `List`, `Set`, `Map` (not `ArrayList`…) means the method works with any implementation and you can change your mind in one place. Use the concrete type only when you need its extra methods (e.g. `TreeMap.floorKey`, `LinkedList.addFirst` before Java 21).

Legacy classes you will still meet

`Vector` (synchronized `ArrayList`), `Stack` (extends `Vector`), `Hashtable` (synchronized map, no nulls) and `Enumeration` date from Java 1.0. They still work but every method takes a lock (only one thread at a time may run it, which costs time), and `Stack` inherits index methods a stack should not have. Use `ArrayList`, `ArrayDeque`, `HashMap`, and — when threads (independent paths of execution running at the same time) share the collection — the `java.util.concurrent` classes (Chapter 14).

Self-check

1. Which interface lets a class be used in a for-each loop? → `Iterable`.
2. Is `Map` a `Collection`? → No; it provides collection *views* (`keySet`, `values`, `entrySet`).
3. Why declare `List<String> xs = new ArrayList<>()` rather than `ArrayList<String>`? → Code depends only on the contract, so the implementation can change without touching callers.

TL;DR

Interfaces (`List`, `Set`, `Queue`, `Deque`, `Map`) describe behaviour; classes (`ArrayList`, `HashSet`, `ArrayDeque`, `HashMap`, `TreeMap`) pick the data structure. Program to the interface. Avoid `Vector`, `Stack`, `Hashtable`.

10.2 Lists: `ArrayList` and `LinkedList`

What it is

A `List` is an ordered sequence you can index into (`get(i)`), like a numbered shopping list. It allows duplicates and (for most implementations) `null`.

- `ArrayList` — a resizable array. Analogy: a row of numbered lockers; jumping to locker 500 is instant, but inserting a new locker at position 0 means shifting every locker one place.
- `LinkedList` — a **doubly linked list**: each element sits in a small *node* object holding pointers (references) to the previous and next node. Analogy: a treasure hunt — each clue tells you where the next one is, so reaching clue 500 means following 500 clues.

How `ArrayList` works

1. It holds an internal `Object[] elementData` and an `int size`.
2. `new ArrayList<>()` starts with a shared empty array; the first `add` allocates capacity **10**.
3. `add(e)` puts `e` at `elementData[size++]`. When the array is full it allocates a new one about **1.5×** larger (`old + old >> 1`) and copies everything across. Because growth is geometric, the total copying is proportional to n, so `add` at the end is **amortised O(1)** (occasionally O(n), on average constant).
4. `get(i)` / `set(i, e)` are one array access: **O(1)**.
5. `add(i, e)` and `remove(i)` shift the tail with `System.arraycopy`: **O(n − i)**.
6. `contains`/`indexOf` scan linearly with `equals`: **O(n)**.
7. It never shrinks on its own; `trimToSize()` releases unused capacity. `new ArrayList<>(expectedSize)` or `ensureCapacity(n)` avoid repeated growth.

How `LinkedList` works

1. Each element is wrapped in a `Node { item, prev, next }`; the list keeps `first`, `last` and `size`.
2. Adding or removing at either end rewires two pointers: **O(1)**.
3. `get(i)` walks from whichever end is closer: **O(n)**.
4. Inserting in the middle is O(1) *only once you are already there* (via a `ListIterator`, an iterator that can move both ways and insert — §10.7); getting there is O(n).
5. Each node is a separate heap object (about 24 bytes of overhead each on a typical 64-bit JVM with compressed pointers — references squeezed into 32 bits to save memory), and nodes are scattered in memory, which defeats the CPU cache (a small, very fast memory beside the processor that works best when data sits side by side).

`LinkedList` also implements `Deque`, so it can act as a queue or stack — but `ArrayDeque` does that job faster (§10.6). In practice `ArrayList` **is the right default for almost every list**; the measured demo in §10.12 shows why.

Code

```
// code/ch10/lists/ListDemo.java (key parts)
List<String> todo = new ArrayList<>(List.of("write", "test", "ship"));
todo.add(1, "review");                 // insert at index 1, shifts the rest right
todo.set(0, "design");                 // replace; returns the old value

List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 1));
nums.remove(1);                        // int argument -> remove(int index): removes 20
nums.remove(Integer.valueOf(1));       // object argument -> remove(Object): removes the value 1

List<String> firstTwo = todo.subList(0, 2);
firstTwo.clear();                      // subList is a VIEW: clears those elements from todo

String[] arr = {"a", "b", "c"};
List<String> view = Arrays.asList(arr); // fixed-size view backed by the array
view.set(0, "z");                       // writes through to arr
view.add("d");                          // UnsupportedOperationException

LinkedList<String> ll = new LinkedList<>(List.of("b", "c"));
ll.addFirst("a"); ll.addLast("d");

List<String> names = new ArrayList<>(List.of("mia", "Al", "zoe", "Bob", "ann"));
names.sort(null);                       // null comparator = natural order
names.sort(String.CASE_INSENSITIVE_ORDER);
```

Output:

```
[design, review, test, ship] get(2)=test indexOf(ship)=3
nums=[10, 30]
after subList.clear(): [test, ship]
array now [z, b, c]
asList.add -> UnsupportedOperationException
linked=[a, b, c, d] removeFirst=a peekLast=d
natural (uppercase first): [Al, Bob, ann, mia, zoe]
case-insensitive: [Al, ann, Bob, mia, zoe]
ArrayList equals LinkedList? true
```

Notes on the output:

- `remove(1)` on a `List<Integer>` picks the **`remove(int index)`** overload because `1` is an `int` and no boxing (wrapping a primitive in its object type, `int` → `Integer`) is needed (overload resolution, §4.3). Box explicitly to remove by value.
- Natural `String` order compares UTF-16 code units (the 16-bit numbers Java stores for each character), so every uppercase letter sorts before every lowercase one.
- `List.equals` is defined by the interface: same elements in the same order, regardless of implementation.

Trade-offs

Operation	ArrayList	LinkedList
`get(i)` / `set(i, e)`	**O(1)**	O(n)
`add(e)` at end	amortised **O(1)**	O(1)
`add(0, e)` / `remove(0)`	O(n) (shift)	**O(1)**
insert/remove in middle	O(n) shift	O(n) to find + O(1) to link
`contains` / `indexOf`	O(n)	O(n)
Memory per element	one reference (+ spare capacity)	node object ≈ 24 B + reference
Cache friendliness	high (contiguous — side by side in memory)	low (pointer chasing — hopping from node to node)

When not to use them

- Frequent "is x in here?" checks → use a `Set` (O(1) or O(log n)) instead of `list.contains` (O(n)).
- Queue/stack work (add/remove at the ends) → `ArrayDeque`, not `ArrayList.remove(0)`.
- Fixed-size numeric data in hot loops → a primitive array (`int[]`) avoids boxing each element into an `Integer`.

Common mistakes

- `remove(int)` vs `remove(Object)` on a `List<Integer>` (above).
- Holding a `subList` and then structurally modifying the parent: the sublist becomes invalid and throws `ConcurrentModificationException` on next use.
- Expecting `Arrays.asList(...)` to be growable. Wrap it: `new ArrayList<>(Arrays.asList(...))`.
- `Arrays.asList(intArray)` gives a `List<int[]>` with **one** element — generics can't hold primitives.
- Calling `list.get(i)` in a loop over a `LinkedList` — O(n²) overall; use for-each or an iterator.

In the interview

> **30-second answer:** "ArrayList is a growable array: O(1) random access, amortised O(1) append, O(n) inserts or removes in the middle because it shifts elements. LinkedList is a doubly linked list: O(1) at the ends but O(n) to reach an index, more memory per element and poor cache locality. I default to ArrayList, and use ArrayDeque when I need a queue or stack."

- *Follow-up: How does ArrayList grow?* → Starts at 10 on first add, then about 1.5× each time with an array copy; geometric growth makes append amortised O(1).
- *Follow-up: When would LinkedList actually win?* → Many inserts/removals at positions you are already at via a `ListIterator`, in a very long list — rare in practice; measure first.
- *Follow-up: Difference between `List.of` and `Arrays.asList`?* → `List.of` is fully unmodifiable and rejects nulls; `Arrays.asList` is a fixed-size, writable view of an array that allows nulls (§10.10).

Self-check

1. What is the cost of `list.add(0, x)` on an `ArrayList` of n elements? → O(n) — every element shifts.
2. `List<Integer> l = new ArrayList<>(List.of(5, 6, 7)); l.remove(2);` — what's left? → `[5, 6]` (index 2 removed).

TL;DR

`ArrayList` = array + 1.5× growth: fast index and append, slow front/middle edits. `LinkedList` = linked nodes: fast ends, slow indexing, heavy. Default to `ArrayList`.

10.3 Sets: `HashSet`, `LinkedHashSet`, `TreeSet`

What it is

A `Set` holds each element at most once — "at most once" meaning no two elements are `equals`. Analogy: a guest list; writing a name twice doesn't add a second guest.

Class	Built on	Order	`add` / `contains` / `remove`	Nulls
`HashSet`	a `HashMap` (elements are the keys)	none guaranteed	**O(1)** average	one `null` allowed
`LinkedHashSet`	a `LinkedHashMap`	insertion order	O(1) average	one `null`
`TreeSet`	a `TreeMap` (red-black tree, see below)	sorted	**O(log n)**	no `null` with natural order
`EnumSet`	a bit vector (one bit per enum constant)	enum declaration order	O(1), extremely fast	no `null` (§6.9)

How it works

- `HashSet` literally stores each element as a key in an internal `HashMap`, with a shared dummy object as the value. So everything in §10.5 applies: it uses `hashCode()` to find a bucket and `equals()` to confirm a match.
- `LinkedHashSet` adds a doubly linked list through the entries, remembering insertion order (re-adding an existing element does not move it).
- `TreeSet` keeps elements in a **red-black tree**, a self-balancing binary search tree (each node's left subtree holds smaller values and its right subtree larger ones) whose height stays O(log n). It never calls `equals` or `hashCode`: two elements are "the same" when `compareTo`/`compare` returns 0.

Code

```
// code/ch10/sets/SetDemo.java (key parts)
List<String> visits = List.of("home", "cart", "home", "search", "cart", "about");
new HashSet<>(visits);  new LinkedHashSet<>(visits);  new TreeSet<>(visits);

Set<String> union = new TreeSet<>(a); union.addAll(b);      // A ∪ B
Set<String> inter = new TreeSet<>(a); inter.retainAll(b);   // A ∩ B
Set<String> diff  = new TreeSet<>(a); diff.removeAll(b);    // A − B

TreeSet<Integer> prices = new TreeSet<>(List.of(5, 12, 20, 35, 50));
prices.floor(19); prices.ceiling(19); prices.lower(20); prices.higher(20);
prices.headSet(20); prices.tailSet(20); prices.subSet(10, 40);

List<String> key = new ArrayList<>(List.of("x"));
Set<List<String>> set = new HashSet<>(); set.add(key);
key.add("y");                                               // mutates a key already in the set
```

Output:

```
HashSet       [search, about, cart, home]  (order not guaranteed)
LinkedHashSet [home, cart, search, about]  (insertion order)
TreeSet       [about, cart, home, search]  (sorted)
union=[docker, git, java, sql] intersection=[git] difference=[java, sql]
floor(19)=12 ceiling(19)=20 lower(20)=12 higher(20)=35
headSet(20)=[5, 12] tailSet(20)=[20, 35, 50] subSet(10,40)=[12, 20, 35]
first=5 pollLast=50 descending=[35, 20, 12, 5]
contains mutated key? false, size=1
HashSet null ok=true, TreeSet.add(null) -> NPE
```

Navigation methods (`NavigableSet`): `floor(x)` = greatest element ≤ x, `ceiling(x)` = least ≥ x, `lower`/`higher` = strictly < / >. `headSet(x)` is exclusive of x, `tailSet(x)` inclusive, `subSet(from, to)` is `[from, to)`. These range views are *live*: changes show through both ways.

The mutated-key line is the most important bug in this section: after `key.add("y")` the list's `hashCode` changed, so `contains` looks in a different bucket and reports `false` even though the object is in the set. **Never mutate an object while it is a set element or map key** — use immutable keys (strings, records of immutable fields).

Common mistakes

- Putting objects whose class overrides `equals` but not `hashCode` into a `HashSet` → duplicates appear (§6.13).
- A `TreeSet` comparator that returns 0 for different elements silently drops them (§10.8).
- Relying on `HashSet` print order in tests — it can change between JDK versions or after a resize.

Real-world examples

Unique visitors per day (`HashSet<UserId>`), de-duplicating while keeping first-seen order (`LinkedHashSet`), a leaderboard or an index of time-stamped events that needs "next event after t" (`TreeSet.ceiling`), feature flags (on/off switches for features; `EnumSet`).

Self-check

1. Which set gives insertion order? → `LinkedHashSet`.
2. What does `TreeSet` use to detect duplicates? → `compareTo`/`compare` returning 0, not `equals`.
3. `new TreeSet<>(List.of(1, 5, 9)).floor(4)`? → `1`.

TL;DR

`HashSet` for fast membership, `LinkedHashSet` to also keep insertion order, `TreeSet` for sorted order and range/nearest queries (O(log n)). Keys must not change while inside.

10.4 Maps: `HashMap`, `LinkedHashMap`, `TreeMap`

What it is

A `Map<K,V>` associates each unique **key** with one **value** — a dictionary: look up a word (key), get its definition (value). Putting a second value under an existing key *replaces* the old value.

The everyday API

Method	Does	Returns
`put(k, v)`	insert or replace	previous value or `null`
`get(k)` / `getOrDefault(k, d)`	look up	value, or `null` / `d` if absent
`containsKey(k)` / `containsValue(v)`	test	`boolean` (`containsValue` is O(n))
`remove(k)`	delete	removed value or `null`
`putIfAbsent(k, v)`	insert only if missing (or mapped to `null`)	existing value or `null`
`merge(k, v, fn)`	if absent put `v`, else put `fn(old, v)`; `null` result removes	new value
`compute(k, fn)`	`fn(k, oldOrNull)`; `null` result removes	new value
`computeIfAbsent(k, fn)`	if missing, compute and store `fn(k)`	current value
`computeIfPresent(k, fn)`	if present, recompute; `null` removes	new value
`forEach((k, v) -> …)`, `replaceAll((k, v) -> …)`	iterate / transform all values	—
`keySet()`, `values()`, `entrySet()`	live views	collections

The `merge`/`compute*` methods arrived in Java 8 and replace most "get, check for null, put" boilerplate (repetitive code).

Code

```
// code/ch10/maps/MapDemo.java (key parts)
Map<String, Integer> stock = new HashMap<>();
stock.put("apple", 5);
Integer old = stock.put("apple", 7);                    // replaces, returns 5
stock.putIfAbsent("pear", 3);
stock.merge("apple", 10, Integer::sum);                 // 7 + 10
stock.merge("plum", 1, Integer::sum);                   // absent -> puts 1
stock.compute("pear", (k, v) -> v == null ? 1 : v * 2);
stock.computeIfPresent("plum", (k, v) -> null);         // null result REMOVES plum

Map<String, Integer> counts = new TreeMap<>();          // word count idiom
for (String w : "to be or not to be".split(" ")) counts.merge(w, 1, Integer::sum);

Map<Character, List<String>> byInitial = new TreeMap<>(); // multimap idiom
for (String n : List.of("ana", "bo", "al", "cy", "ben"))
    byInitial.computeIfAbsent(n.charAt(0), c -> new ArrayList<>()).add(n);

for (Map.Entry<String, Integer> e : counts.entrySet())  // iterate entries, not keys + get
    if (e.getValue() > 1) System.out.print(e.getKey() + "x" + e.getValue() + " ");

TreeMap<Integer, String> bracket = new TreeMap<>(Map.of(0, "10%", 11_000, "12%", 44_725, "22%", 95_375, "24%"));
bracket.floorEntry(50_000).getValue();                  // bracket whose start <= 50,000

Map<String, String> lru = new LinkedHashMap<>(16, 0.75f, true) {   // true = access order
    @Override protected boolean removeEldestEntry(Map.Entry<String, String> eldest) { return size() > 3; }
};
lru.put("a", "A"); lru.put("b", "B"); lru.put("c", "C");
lru.get("a");       // a becomes most recently used
lru.put("d", "D");  // evicts b
```

Output:

```
old=5 get=7 missing=null default=0
stock={apple=17, pear=6}
counts={be=2, not=1, or=1, to=2}
byInitial={a=[ana, al], b=[bo, ben], c=[cy]}
bex2 tox2
income 50000 -> 22%, first bracket above 50000 starts at 95375, headMap(44725)={0=10%, 11000=12%}
LRU keys=[c, a, d]
HashMap null key -> 0
```

(The brackets are illustrative numbers in the style of a progressive income-tax table, not current law.)

The three main implementations

	HashMap	LinkedHashMap	TreeMap
Structure	array of buckets (§10.5)	`HashMap` + doubly linked list of entries	red-black tree
Iteration order	unspecified	insertion order (or access order)	sorted by key
`get` / `put` / `remove`	**O(1)** average	O(1) average	**O(log n)**
`null` key	one allowed	one allowed	not with natural ordering
Extras	—	`removeEldestEntry` hook → LRU cache	`floorKey`, `ceilingEntry`, `headMap`, `tailMap`, `subMap`, `firstKey`, `descendingMap`

Other maps worth knowing: **`EnumMap`** (enum keys, backed by an array indexed by `ordinal()` — very fast), **`IdentityHashMap`** (compares keys with `==`, used for object-graph algorithms like serialization — turning objects into bytes), **`WeakHashMap`** (keys held weakly — through references that don't stop garbage collection — so an entry disappears once nothing else references its key — for caches of metadata), **`ConcurrentHashMap`** (thread-safe — safe for many threads at once — no null keys/values — Chapter 14; a simplified build is in LLD §13.7).

Common mistakes

- `if (map.get(k) != null)` to test presence — wrong if `null` values are allowed; use `containsKey`.
- `for (K k : map.keySet()) use(map.get(k))` — two lookups per entry; iterate `entrySet()` or use `map.forEach`.
- `map.get(k) + 1` when `k` is absent → `NullPointerException` from unboxing `null`. Use `merge` or `getOrDefault`.
- In `computeIfAbsent`, the mapping function must not modify the same map (for `HashMap` this throws `ConcurrentModificationException` since Java 9). This bites recursive memoisation (caching the results of earlier calls) — see §11.8.

Real-world examples

HTTP headers (key–value metadata sent with each web request) and configuration (`Map<String,String>`), caches keyed by ID, word or event counts (`merge`), grouping orders by customer (`computeIfAbsent`), an LRU cache for thumbnails (`LinkedHashMap`), time-series (data points indexed by time) lookups and rate tables (`TreeMap.floorEntry`).

Self-check

1. What does `map.put(k, v)` return when `k` was already present? → The previous value.
2. How do you count occurrences in one line per element? → `counts.merge(word, 1, Integer::sum)`.
3. Which map gives "the entry with the largest key ≤ x"? → `TreeMap.floorEntry(x)`.
4. What does returning `null` from a `compute` function do? → Removes the key.

TL;DR

Map = unique keys → values. `HashMap` default, `LinkedHashMap` for predictable order or LRU, `TreeMap` for sorted keys and range queries. Use `merge`, `computeIfAbsent`, `getOrDefault` instead of null-check boilerplate.

10.5 `HashMap` internals

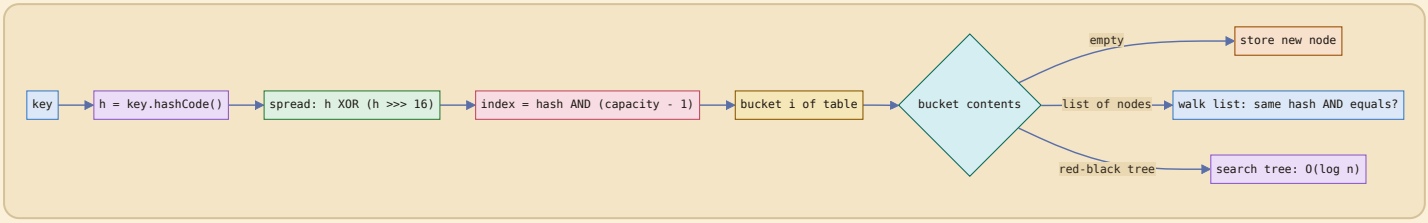
What it is

`HashMap` is a **hash table**: an array of **buckets** (also called *bins*), where a key's **hash code** (an `int` computed from the key's contents by `hashCode()`) decides which bucket it goes into. Analogy: a coat check with 16 numbered hooks. The attendant computes a number from your ticket, hangs your coat on that hook, and later goes straight to that hook — no searching through every coat. If two coats land on the same hook, they hang one behind the other, and the attendant checks each ticket on that hook.

Why it matters

Average **O(1)** `get`/`put` makes `HashMap`/`HashSet` the most-used collections — and a favourite interview topic. The internals also explain real bugs: slow maps from bad `hashCode`, lost keys from mutation, surprising iteration order, wasted resizes.

How it works (JDK 8 and later)



1. **Hash.** `put(key, value)` calls `key.hashCode()` (a 32-bit `int`). A `null` key is given hash 0 (so it lives in bucket 0).
2. **Spread.** `HashMap.hash()` XORs (bitwise exclusive-or: a result bit is 1 where the two input bits differ) the high 16 bits into the low 16: `h ^ (h >>> 16)`. The next step only looks at the low bits, and without spreading, keys whose hash codes differ only in high bits would all collide.
3. **Index.** The table length `n` is always a **power of two** (default **16**), so `hash & (n − 1)` equals `hash mod n` (the remainder after dividing by n) but is a single cheap AND (bitwise and: keeps only bits set in both) instruction.
4. **Store.** Each bucket holds a chain of `Node{hash, key, value, next}`. If the bucket is empty, the node goes straight in. Otherwise the map walks the chain: a node matches when **the hashes are equal and** (`==` or `equals`). A match gets its value replaced (and `put` returns the old value); no match → a new node is appended at the tail.
5. **Load factor and resize.** The **load factor** (default **0.75**) is the fullness at which the table grows. The **threshold** = capacity × load factor (16 × 0.75 = **12**). When `size` exceeds the threshold, the table **doubles** (16 → 32 → 64 …) and every node is redistributed. Because capacity is a power of two, each node either stays at index `i` or moves to `i + oldCapacity`, decided by one bit of its hash (`hash & oldCapacity`), so the split is cheap and keeps chain order.
6. **Treeification (Java 8).** If a single bucket's chain reaches **8** nodes (`TREEIFY_THRESHOLD`) and another is added, the chain is converted into a **red-black tree** of `TreeNode`s, turning that bucket's worst case from O(n) into **O(log n)**. But if the whole table is smaller than **64** buckets (`MIN_TREEIFY_CAPACITY`), the map **resizes instead**, because a small table crowded in one bucket is better fixed by spreading keys out. When a resize split leaves a tree bucket with **≤ 6** nodes (`UNTREEIFY_THRESHOLD`), it turns back into a list. With a decent hash function and load factor 0.75, a bucket of 8 is astronomically unlikely (the JDK source comment estimates about 6 in 100 million), so trees are a defence against bad or malicious hash codes, not a normal case.

7. **Inside a tree bucket**, nodes are ordered by hash; for equal hashes, by `compareTo` **if the keys are `Comparable` and of the same class**. Otherwise the tree cannot tell them apart and a lookup may have to search both subtrees — which is why the non-`Comparable` case in the demo below stays slow.
8. **Get** repeats steps 1–3 and searches only that one bucket.

The constants, straight from `java/util/HashMap.java` in JDK 26:

Constant	Value	Meaning
`DEFAULT_INITIAL_CAPACITY`	16	buckets allocated on first `put` (the table is created lazily)
`DEFAULT_LOAD_FACTOR`	0.75	resize when size > capacity × 0.75
`TREEIFY_THRESHOLD`	8	chain length that triggers list → tree
`UNTREEIFY_THRESHOLD`	6	tree → list when a resize split leaves this few
`MIN_TREEIFY_CAPACITY`	64	below this table size, resize instead of treeifying
`MAXIMUM_CAPACITY`	2^30	table never grows beyond this

Why 0.75? It's a time/space trade-off: a lower load factor means fewer collisions but more empty buckets (memory); a higher one packs buckets tighter but lengthens chains. 0.75 keeps chains very short on average while wasting at most about a quarter of the table.

Watching it happen: a teaching-size hash map

`TinyHashMap` below implements steps 1–5 in ~60 lines (no trees, no null keys) and prints what it does. It starts with 4 buckets so we can watch two resizes.

```
// code/ch10/maps/TinyHashMap.java (key parts)
static int spread(Object key) { int h = key.hashCode(); return h ^ (h >>> 16); }

void put(K key, V value) {
    int h = spread(key), i = h & (table.length - 1);          // bucket index
    for (Node<K, V> n = table[i]; n != null; n = n.next())
        if (n.hash() == h && n.key().equals(key)) { table[i] = replace(table[i], key, value); return; }
    table[i] = new Node<>(h, key, value, table[i]);           // new node in bucket i
    if (++size > table.length * loadFactor) resize();         // threshold = capacity * 0.75
}

private void resize() {                                       // double and re-bucket every node
    Node<K, V>[] old = table;
    table = (Node<K, V>[]) new Node[old.length * 2];
    for (Node<K, V> head : old)
        for (Node<K, V> n = head; n != null; n = n.next()) {
            int i = n.hash() & (table.length - 1);            // stays at i or moves to i + oldCapacity
            table[i] = new Node<>(n.hash(), n.key(), n.value(), table[i]);
        }
}
```

Output:

```
put apple   hashCode=   93029210 spread=   93030097 index(n=4)=1
put banana  hashCode=-1396355227 spread=-1396317280 index(n=4)=0
put cherry  hashCode=-1361513063 spread=-1361552575 index(n=4)=1
put date    hashCode=    3076014 spread=    3075968 index(n=4)=0
  resize: size=4 exceeded threshold 3 -> capacity 4 -> 8
put elder   hashCode=   96592394 spread=   96593867 index(n=8)=3
put fig     hashCode=     101380 spread=     101381 index(n=8)=5
put grape   hashCode=   98615627 spread=   98616491 index(n=8)=3
  resize: size=7 exceeded threshold 6 -> capacity 8 -> 16
final table (capacity 16):
  bucket  0: date -> banana -> null
  bucket  1: cherry -> apple -> null
  bucket  5: fig -> null
  bucket 11: elder -> grape -> null
get(fig)=99 get(kiwi)=null size=7
```

Read it like this: with 4 buckets, `apple` and `cherry` collide in bucket 1. After the 4th key, size 4 > 4 × 0.75 = 3, so the table doubles. After the final resize to 16, `elder` and `grape` still share bucket 11 because their spread hashes agree in the lowest 4 bits. `put("fig", 99)` finds the existing key and replaces the value, so size stays 7. (`String.hashCode` is a fixed, documented formula — `s[0]·31^(n−1) + … + s[n−1]` — so these numbers are the same on every JVM.)

What bad hash codes cost (measured)

`HashCollisionDemo` puts 20,000 keys into a `HashMap` whose `hashCode()` returns the constant 42 — every key collides into one bucket — and times 60,000 lookups. **Timings are machine-dependent** (this run: Apple M4 Pro, JDK 26); compare the ratios.

```
// code/ch10/maps/HashCollisionDemo.java (key parts)
record CmpKey(int id) implements Comparable<CmpKey> {          // constant hash, but Comparable
    @Override public int hashCode() { return 42; }
    @Override public int compareTo(CmpKey o) { return Integer.compare(id, o.id); }
}
record PlainKey(int id) { @Override public int hashCode() { return 42; } }  // constant hash, not Comparable
record GoodKey(int id) {}                                        // record's generated hashCode
```

Output:

```
good hashCode (GoodKey)            n=20,000  3n lookups:      0.7 ms
constant hash, Comparable (tree)   n=20,000  3n lookups:      6.5 ms
constant hash, not Comparable      n=20,000  3n lookups:   1648.2 ms
```

- **Good hash:** keys spread across buckets → O(1) each.
- **Constant hash + `Comparable`:** one giant bucket, but it was treeified and ordered by `compareTo` → O(log n) each; ~10× slower than good hashing, still fast.
- **Constant hash, not `Comparable`:** the tree can't order equal-hash keys, so lookups degrade toward a linear scan → O(n) each, **thousands of times slower**. This is the scenario hash-flooding denial-of-service attacks (sending a server many keys with colliding hashes to make it grind to a halt) exploit, and the reason Java 8 added treeification (and why `String` — being `Comparable` — is well protected as a key).

Sizing a map correctly

`new HashMap<>(100)` does **not** mean "room for 100 entries". The argument is an initial *capacity*, rounded up to a power of two (128); the threshold is then 128 × 0.75 = 96, so the 97th `put` still triggers a resize. To pre-size for an expected number of **entries**, use the Java 19 factories, which do the load-factor maths for you:

```
Map<String, User> users = HashMap.newHashMap(100);       // Java 19: capacity for 100 mappings without resizing
Set<String> seen        = HashSet.newHashSet(100);        // also LinkedHashMap.newLinkedHashMap,
LinkedHashSet.newLinkedHashSet
```

Thread safety

`HashMap` is **not** thread-safe. Concurrent `put`s can lose entries or corrupt a bucket (in Java 7 and earlier, a concurrent resize could even link a chain into a cycle so `get` spun forever). Use `ConcurrentHashMap` (a thread-safe map) for shared maps (Chapter 14). `Collections.synchronizedMap` works but locks the whole map on every call.

Trade-offs

	HashMap	TreeMap
Needs	good `hashCode` + consistent `equals`	`Comparable` keys or a `Comparator`
Average `get` / `put`	O(1)	O(log n)
Worst case	O(log n) per bucket if keys are `Comparable`, else O(n)	O(log n) guaranteed
Order	none	sorted
Memory	table + a node per entry	a tree node per entry (more pointers)

Common mistakes

- Overriding `equals` without `hashCode` (or using different fields in each) → "equal" keys land in different buckets.
- `hashCode` that returns a constant or uses only a low-variety field (e.g. a boolean) → every lookup scans.
- Relying on iteration order: it follows bucket order and can change after a resize or between JDK versions. Use `LinkedHashMap` or `TreeMap` when order matters.

In the interview

30-second answer: "HashMap is an array of buckets. It takes the key's hashCode, spreads the high bits into the low bits, and ANDs with capacity minus one to pick a bucket. Collisions chain in a linked list; since Java 8 a bucket with more than 8 entries becomes a red-black tree if the table has at least 64 buckets, so the worst case is O(log n) instead of O(n). When size passes capacity × 0.75 the table doubles and entries are re-bucketed — each moves to index i or i + oldCapacity. Lookups compare hash first, then equals, which is why equals and hashCode must agree and keys shouldn't be mutable."

- *Follow-up: Why is the capacity a power of two?* → So `hash & (n−1)` replaces a slower modulo, and resizing splits each bucket into exactly two predictable positions.
- *Follow-up: Can HashMap have a null key?* → Yes, one, with hash 0; `Hashtable` and `ConcurrentHashMap` reject nulls.
- *Follow-up: What happens if two keys have the same hashCode but aren't equal?* → Both are stored in the same bucket; lookup distinguishes them with `equals`. Correct but slower.

Self-check

1. Default capacity, load factor and first resize threshold? → 16, 0.75, 12.
2. When does a bucket become a tree? → When adding to a bucket that already holds 8 nodes, provided the table has at least 64 buckets (else it resizes).
3. During a resize from 16 to 32, where can the entry in bucket 5 go? → Bucket 5 or bucket 21.

TL;DR

hashCode → spread → `& (n−1)` → bucket; match by hash then `equals`. Load factor 0.75 triggers doubling; buckets over 8 become red-black trees (table ≥ 64). Good, stable hash codes = O(1); bad ones = slow maps. Pre-size with `HashMap.newHashMap(n)`. Not thread-safe.

10.6 Queues, deques and priority queues

What it is

- A **queue** holds elements waiting to be processed — normally **FIFO** (first in, first out), like a line at a till.
- A **deque** ("deck", double-ended queue) allows adding and removing at both ends, so it can be a FIFO queue *or* a **LIFO stack** (last in, first out, like a stack of plates).
- A **priority queue** always hands out the *smallest* element next (by natural order or a comparator), like a hospital triage desk.

Two method families

`Queue` offers each operation twice: one throws when it can't proceed, the other returns a special value. Prefer the special-value versions for ordinary control flow.

Operation	Throws exception	Returns special value
Insert at tail	`add(e)`	`offer(e)` → `false` if full (bounded queues)
Remove head	`remove()`	`poll()` → `null` if empty
Examine head	`element()`	`peek()` → `null` if empty

`Deque` adds `First`/`Last` versions of each (`offerFirst`, `pollLast`, `peekFirst`…) plus the stack methods `push` (= `addFirst`), `pop` (= `removeFirst`) and `peek`.

How the implementations work

- `ArrayDeque` — a resizable **circular array** with `head` and `tail` indexes that wrap around. Adding/removing at either end is amortised **O(1)** with no node allocations. It rejects `null` (because `null` is how `poll` signals "empty"). The Javadoc notes it is likely faster than `Stack` as a stack and faster than `LinkedList` as a queue. **Use it for both.**
- `PriorityQueue` — a **binary min-heap** stored in an array: a complete binary tree (every level full except possibly the last, which fills left to right) where each parent ≤ its children, and element *i*'s children sit at *2i+1* and *2i+2*. `offer` appends at the end and "sifts up"; `poll` moves the last element to the root and "sifts down". Both are **O(log n)**; `peek` is O(1); `contains`/`remove(Object)` are O(n). Building one from a collection (heapify) is O(n). The array is *not* sorted — only the root is guaranteed smallest — so `toString()` and the iterator show heap order, not sorted order.

Code

```
// code/ch10/queues/QueueDemo.java (key parts)
Queue<String> printer = new ArrayDeque<>();
printer.offer("doc1"); printer.offer("doc2"); printer.offer("doc3");

Deque<Character> stack = new ArrayDeque<>();                // balanced-brackets check
for (char c : "{[()]}(".toCharArray()) {
    if ("([{".indexOf(c) >= 0) stack.push(c);
    else if (!stack.isEmpty() && "([{".indexOf(stack.peek()) == ")]}".indexOf(c)) stack.pop();
    else { stack.push('!'); break; }
}

PriorityQueue<Integer> heap = new PriorityQueue<>(List.of(7, 3, 9, 1, 5));
while (!heap.isEmpty()) out.append(heap.poll()).append(' ');

PriorityQueue<Task> jobs = new PriorityQueue<>(
        Comparator.comparingInt(Task::priority).reversed().thenComparing(Task::name));

int k = 3; PriorityQueue<Integer> topK = new PriorityQueue<>();  // top-k largest with a min-heap
for (int x : new int[]{15, 2, 99, 42, 8, 77, 23}) { topK.offer(x); if (topK.size() > k) topK.poll(); }
```

Output:

```
peek=doc1 poll=doc1 left=[doc2, doc3]
empty.poll()=null empty.peek()=null
empty.remove() -> NoSuchElementException
{[()]}( balanced? false (left on stack: [(])
ArrayDeque.push(null) -> NPE
heap toString (NOT sorted): [1, 3, 9, 7, 5]
poll order: 1 3 5 7 9
backup deploy lint email
top-3 = [99, 77, 42]
```

The top-k trick keeps a heap of only *k* elements: each new element costs O(log k), and the smallest of the current top k is evicted. Total O(n log k) instead of sorting everything in O(n log n) — a staple of coding interviews (Chapter 19).

When not to use them

- Threads producing and consuming (the producer–consumer pattern: some threads add work, others take it) → a `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`) that waits instead of returning `null` (Chapter 14).
- You need all elements in sorted order once → just sort a list; a heap shines when elements keep arriving.
- Changing an element's priority after insertion is not supported — remove and re-add it (O(n)), or insert a new entry and skip stale ones when polled.

Common mistakes

- Using `Stack` or `LinkedList` where `ArrayDeque` fits.
- Printing a `PriorityQueue` and expecting sorted output; iterate with `poll()` instead.
- Mixing `add`/`remove` (exceptions) with `offer`/`poll` expectations.
- Using `push` then `pollLast` — `push` adds at the **front** of a deque.

Self-check

1. Which call returns `null` rather than throwing on an empty queue? → `poll()` / `peek()`.
2. What does `PriorityQueue.poll()` return? → The smallest element per its ordering.
3. Why can't `ArrayDeque` hold `null`? → `null` is the "empty" return value of `poll`/`peek`.

TL;DR

`ArrayDeque` for FIFO queues and LIFO stacks; `PriorityQueue` (binary heap) when the next item is "the smallest": O(log n) add/poll, O(1) peek, unsorted iteration.

10.7 Iterators and fail-fast behaviour

What it is

An `Iterator<E>` is a cursor over a collection with three methods: `hasNext()`, `next()` and the optional `remove()`. The enhanced for loop is compiler shorthand for using one. `ListIterator` adds `hasPrevious`/`previous`, index queries, `set` and `add`.

How fail-fast works

Most `java.util` collections keep a **modification counter** (`modCount`) that increments on every structural change. An iterator records the count when created and re-checks it on each `next()`. If the collection was changed by anything *other than the iterator itself*, `next()` throws `ConcurrentModificationException` (CME). Despite the name, this happens in single-threaded code too. It is "best effort": it catches bugs early but is not a thread-safety guarantee.

Fail-safe iterators (in `java.util.concurrent`) iterate over a snapshot (a frozen copy) or a weakly consistent view (one that may or may not show concurrent changes, but never fails) and never throw CME.

`CopyOnWriteArrayList` copies the whole array on every write, so it suits read-mostly lists like event listeners (objects notified when something happens).

Code

```
// code/ch10/iteration/IteratorDemo.java (key parts)
for (Integer n : nums) if (n % 2 == 0) nums.remove(n);          // WRONG: CME

for (Iterator<Integer> it = nums.iterator(); it.hasNext(); )    // RIGHT: iterator's own remove
    if (it.next() % 2 == 0) it.remove();

nums.removeIf(n -> n % 2 == 0);                                 // SIMPLEST (Java 8)

for (ListIterator<String> li = words.listIterator(); li.hasNext(); ) {
    String w = li.next();
    li.set(w.toUpperCase());
    if (w.equals("b")) li.add("b2");                            // inserted after the cursor
}
scores.entrySet().removeIf(e -> e.getValue() < 50);             // remove map entries by condition

List<String> listeners = new CopyOnWriteArrayList<>(List.of("L1", "L2"));
for (String l : listeners) listeners.add(l + "'");              // iterates the old snapshot
```

Output:

```
for-each + list.remove -> ConcurrentModificationException, list now [1, 3, 4, 5, 6]
iterator.remove -> [1, 3, 5]
removeIf        -> [1, 3, 5]
listIterator set/add -> [A, B, b2, C]
backwards -> Cb2BA
passed -> {bo=90, cy=55}
CopyOnWriteArrayList -> [L1, L2, L1', L2']
```

The first line shows the list was already changed (`2` removed) before the exception — fail-fast stops you *after* the first bad change, so don't rely on catching CME.

Common mistakes

- Removing inside for-each (above). Also: calling `it.remove()` twice per `next()`, or before the first `next()` → `IllegalStateException`.
- Believing CME means "a threading problem" — usually it is a loop modifying its own collection.
- Removing via index in a forward `for (int i…)` loop and skipping the element that shifted into position `i`; loop backwards or use `removeIf`.

Self-check

1. Name two safe ways to remove while iterating. → `Iterator.remove()` and `Collection.removeIf`.
2. Does CME need two threads? → No.
3. Why doesn't `CopyOnWriteArrayList` throw CME? → Its iterator walks an immutable snapshot of the array.

TL;DR

Don't structurally modify a collection while for-each-ing over it. Use `removeIf`, `Iterator.remove`, or collect changes and apply them after. Fail-fast = bug detector, not a lock.

10.8 Ordering: `Comparable` and `Comparator`

What it is

Sorting needs a rule for "which comes first".

- `Comparable<T>` — the class defines its **natural order** by implementing `int compareTo(T other)`. `String`, `Integer`, `LocalDate` and enums (declaration order) are `Comparable`.

- `Comparator<T>` — a separate object with `int compare(T a, T b)` that defines *any* order you want, without changing the class. Analogy: `Comparable` is a book's own shelf number; a `Comparator` is you choosing to arrange books by colour today.

Both return a **negative** number if the first comes before the second, **zero** if they tie, and **positive** if it comes after. Only the sign matters.

The contract

`compareTo`/`compare` must be **antisymmetric** (sign of `a.compareTo(b)` is the opposite of `b.compareTo(a)`), **transitive** (if a comes before b and b before c, then a comes before c), and ideally **consistent with equals** (`compareTo == 0` exactly when `equals` is true). Sorted collections (`TreeSet`, `TreeMap`) use the comparison *instead of* `equals`, so an inconsistent comparator makes them drop or merge elements. (`BigDecimal` is a famous JDK exception: `2.0` and `2.00` compare equal but aren't `equals`.)

Building comparators (Java 8+)

Factory / method	Meaning
`Comparator.comparing(Employee::name)`	by a `Comparable` key
`comparingInt/Long/Double(keyFn)`	by a primitive key (no boxing)
`comparing(keyFn, keyComparator)`	by a key using a given comparator
`.thenComparing(...)`	tie-breaker
`.reversed()`	flip the whole comparator so far
`Comparator.naturalOrder()` / `reverseOrder()`	natural / reverse natural
`nullsFirst(c)` / `nullsLast(c)`	tolerate nulls
`String.CASE_INSENSITIVE_ORDER`	ready-made

Code

```
// code/ch10/sorting/SortingDemo.java (key parts)
record Employee(int id, String name, String dept, double salary) implements Comparable<Employee> {
    @Override public int compareTo(Employee o) { return Integer.compare(id, o.id); }   // natural order: id
}
Collections.sort(staff);                                          // uses compareTo
staff.sort(Comparator.comparing(Employee::name).reversed());
staff.sort(Comparator.comparing(Employee::dept)
        .thenComparing(Employee::salary, Comparator.reverseOrder())   // reverse just this key
        .thenComparing(Employee::name));
staff.sort(Comparator.comparingDouble(Employee::salary));
cities.sort(Comparator.nullsLast(String.CASE_INSENSITIVE_ORDER));

Comparator<Integer> bad = (a, b) -> a - b;                        // overflow bug
Set<String> byLength = new TreeSet<>(Comparator.comparingInt(String::length));
byLength.addAll(List.of("kiwi", "pear", "fig", "plum", "banana"));
```

Output:

```
by id:      [Ana, Bo, Cy, Dee]
name desc:  [Dee, Cy, Bo, Ana]
dept,salary desc,name: [Bo, Cy, Dee, Ana]
salary asc: [Ana, Cy, Dee, Bo]
nullsLast, case-insensitive: [Accra, lima, Oslo, null]
bad.compare(MIN_VALUE, 1) = 2147483647 (positive, so MIN_VALUE looks BIGGER), Integer.compare = -1
TreeSet by length keeps only [fig, kiwi, banana]  (pear and plum were dropped!)
stable (dept, then previous name order): [Bo, Cy, Dee, Ana]
```

- `reversed()` applies to **everything chained before it**; to reverse only one key, pass `Comparator.reverseOrder()` as that key's comparator (as done for salary).

- `a - b` overflows for large magnitudes (`MIN_VALUE − 1` wraps to `MAX_VALUE`). Always use `Integer.compare`, `Long.compare`, `Double.compare` or `comparingInt`.
- The `TreeSet` ordered by length considers `"kiwi"`, `"pear"` and `"plum"` duplicates (length 4 ties), keeping only the first. Add a tie-breaker: `comparingInt(String::length).thenComparing(Comparator.naturalOrder())`.
- **Stability.** `List.sort` / `Collections.sort` (TimSort — a merge-sort-based algorithm — O(n log n)) are **stable**: equal elements keep their relative order. That's why sorting by name, then by dept, leaves each dept sorted by name.

Common mistakes

- Subtraction comparators; comparing `double` fields with `<` / `>` and forgetting `NaN` (use `Double.compare`).
- `Comparable` whose natural order disagrees with `equals` used as `TreeMap` keys.
- Putting non-`Comparable` objects into a `TreeSet` without a comparator → `ClassCastException` on the first add.

In the interview

> **30-second answer:** "Comparable is the class's single natural order via compareTo; Comparator is an external strategy (a plug-in object holding one way of doing something) for any number of orders. Since Java 8 I build comparators with comparing, thenComparing and reversed, use primitive variants to avoid boxing, and never subtract to compare. Sorted collections treat compare == 0 as equality, so the order should be consistent with equals."

- *Follow-up: Is `Collections.sort` stable?* → Yes (TimSort for objects).
- *Follow-up: How do you sort descending by one key but ascending by another?* → `comparing(a).thenComparing(b, reverseOrder())`.

Self-check

1. What does a negative return from `compare(a, b)` mean? → `a` sorts before `b`.
2. How to sort `List<String>` by length, then alphabetically? → `list.sort(comparingInt(String::length).thenComparing(naturalOrder()))`.
3. Why is `(a, b) -> a - b` dangerous? → Integer overflow can flip the sign.

TL;DR

`Comparable` = one built-in order; `Comparator` = any order, composable with `comparing` / `thenComparing` / `reversed` / `nullsLast`. Keep it consistent with `equals` for sorted collections. Sorting objects is stable.

10.9 The `Collections` utility class

`java.util.Collections` (plural, with an *s*) holds static algorithms and wrappers — the collection counterpart of `java.util.Arrays` (§4.8).

```java
// code/ch10/utilities/CollectionsUtilDemo.java (key parts)
Collections.max(xs); Collections.min(xs); Collections.frequency(xs, 3);
Collections.sort(xs); Collections.binarySearch(xs, 8);    // binarySearch needs a sorted list
Collections.reverse(xs); Collections.swap(xs, 0, xs.size() - 1);
Collections.rotate(xs, 2); Collections.shuffle(xs, new Random(42));   // seeded -> repeatable
Collections.nCopies(3, "ab"); Collections.disjoint(List.of(1, 2), List.of(3));
Collections.addAll(letters, "d", "b");

List<String> view = Collections.unmodifiableList(source);  // read-only VIEW
List<String> copy = List.copyOf(source);                   // immutable COPY
source.add("y");

List<Integer> sync = Collections.synchronizedList(new ArrayList<>(List.of(1, 2)));
synchronized (sync) { for (int v : sync) System.out.print("sync " + v + " "); }   // lock while iterating
```

Output:

```
max=9 min=1 frequency(3)=2
sorted=[1, 3, 3, 5, 8, 9] binarySearch(8)=4 binarySearch(4)=-4 (negative: -(insertionPoint)-1)
reverse=[9, 8, 5, 3, 3, 1]
swap ends=[1, 8, 5, 3, 3, 9]
rotate 2=[3, 9, 1, 8, 5, 3]
shuffle(seed 42)=[9, 3, 5, 3, 8, 1]
nCopies=[ab, ab, ab] disjoint=true
addAll varargs=[c, a, d, b]
view sees change: [x, y]  copy does not: [x]
view.add -> UnsupportedOperationException
emptyList=[] singleton=[only]
sync 1 sync 2
```

Group	Methods
Algorithms	`sort`, `binarySearch`, `reverse`, `shuffle`, `swap`, `rotate`, `fill`, `copy`, `min`, `max`, `frequency`, `disjoint`, `addAll`, `nCopies`
Read-only wrappers (views)	`unmodifiableList/Set/Map/Collection`, `unmodifiableSequencedCollection/Set/Map` (21), sorted/navigable variants
Synchronized wrappers	`synchronizedList/Set/Map/...` — each call locks; you must lock manually around iteration
Constants and singletons	`emptyList/Set/Map`, `singletonList/singleton/singletonMap` (immutable)
Type-checked wrappers	`checkedList(list, String.class)` — catches heap pollution (a generic collection ending up with the wrong type inside) from raw types (generics used without `<…>`) at insertion time (Chapter 9)
Other	`reverseOrder()`, `newSetFromMap(map)`, `list(enumeration)`, `enumeration(coll)`

`binarySearch` returns the index if found, otherwise `-(insertionPoint) - 1` — always negative, so "not found" can't be confused with index 0. Here 4 would be inserted at index 3, so the result is −4.

Self-check

1. `Collections.binarySearch` on an unsorted list? → Undefined result — sort first.
2. What's the difference between `unmodifiableList(l)` and `List.copyOf(l)`? → The first is a read-only view that reflects later changes to `l`; the second is an independent immutable snapshot.

10.10 Unmodifiable and immutable collections

What it is

- **Unmodifiable** — the collection object rejects changes (`add`, `set`, `remove` throw `UnsupportedOperationException`).
- **Immutable** — it is unmodifiable *and* nothing else can change it either (no backing collection behind it).
- A **view** is a wrapper over another collection; it can be unmodifiable yet still change when the underlying collection changes.

Why it matters

Returning an immutable collection from a method (or storing one in a record) means callers can't corrupt your object's state, the collection is automatically safe to share across threads, and intent is explicit. *Effective Java* (Joshua Bloch's best-practice book) Item 17 ("minimise mutability") and Item 50 ("make defensive copies when needed" — private copies so outsiders can't change your data) both point here.

The factory methods

Factory	Since	Nulls	Duplicates	Order
`List.of(...)`	9	NPE	allowed	as given
`Set.of(...)`	9	NPE	**`IllegalArgumentException`**	unspecified (randomised per JVM run)
`Map.of(k1, v1, … up to 10 pairs)` / `Map.ofEntries(Map.entry(k, v), …)`	9	NPE	duplicate key → IAE	unspecified
`List.copyOf` / `Set.copyOf` / `Map.copyOf`	10	NPE	`Set.copyOf` drops duplicates	—
`Stream.toList()`	16	allowed	allowed	encounter order
`Collectors.toUnmodifiableList/Set/Map`	10	NPE	—	—
`Arrays.asList(...)`	1.2	allowed	allowed	fixed-size, **`set` works**, write-through to the array
`Collections.unmodifiableX(c)`	1.2	as `c`	as `c`	read-only **view** of `c`

Code

```
// code/ch10/immutable/ImmutableDemo.java (key parts)
List<String> days = List.of("Mon", "Tue", "Wed");
attempt("List.of(...).add", () -> days.add("Thu"));
attempt("List.of with null", () -> List.of("a", null));
attempt("List.of(...).contains(null)", () -> days.contains(null));
attempt("Set.of duplicate", () -> Set.of("a", "a"));
attempt("Map.of duplicate key", () -> Map.of("k", 1, "k", 2));
List<String> snap = List.copyOf(src); src.add("b");
List<Integer> t = Stream.of(3, 1, 2).toList();
List<StringBuilder> sbs = List.of(new StringBuilder("hi")); sbs.get(0).append("!");
```

Output:

```
List.of(...).add -> UnsupportedOperationException
List.of(...).set -> UnsupportedOperationException
List.of with null -> NullPointerException
List.of(...).contains(null) -> NullPointerException
Set.of duplicate -> IllegalArgumentException
Map.of duplicate key -> IllegalArgumentException
Map.ofEntries get = 404
Arrays.asList set -> ok
Arrays.asList add -> UnsupportedOperationException
copyOf snapshot=[a] src=[a, b] copyOf(List.of) same object? true
Stream.toList().add -> UnsupportedOperationException
element mutated inside an 'immutable' list: [hi!]
Set.of size=3 (print order may differ between runs)
```

Points to remember:

- These collections are **shallowly** immutable: the list can't change, but mutable elements inside can. Store immutable elements (strings, records of immutable fields) for true immutability.
- `List.of(...).contains(null)` throws `NullPointerException` — a surprise when code passes a nullable value to `contains`.
- `copyOf` is free when its argument is already an unmodifiable `List.of`-style list (it returns the same object).
- `Collectors.toList()` makes no promise about mutability (today it returns an `ArrayList`); if you want an unmodifiable result, say so with `Stream.toList()` (Chapter 12).

Defensive copy in a record — records are shallowly immutable (§6.8), so copy collection components in the compact constructor:

```
record Team(String name, List<String> members) {
    Team { members = List.copyOf(members); }   // callers can't change our list later, and we reject nulls
}
```

Self-check

1. Which factory throws on a duplicate element? → `Set.of` (and `Map.of` on a duplicate key).
2. Can you `set(0, x)` on `Arrays.asList(...)`? → Yes; you can't `add`/`remove`.
3. Is `List.of(new StringBuilder())` deeply immutable? → No — the builder inside is still mutable.

TL;DR

Prefer `List.of`/`Set.of`/`Map.of` for constants and `List.copyOf` for defensive copies; they reject nulls and are shallowly immutable. `unmodifiableList` is only a view. `Arrays.asList` is fixed-size but writable.

10.11 Sequenced collections (Java 21)

The problem it solves

Before Java 21 "get the first/last element" or "iterate in reverse" had a different spelling for every type: `list.get(0)` and `list.get(list.size() - 1)`, `deque.getFirst()`, `sortedSet.first()`/`last()`, and for a `LinkedHashSet` no direct way to get the last element at all. There was also no common type meaning "a collection with a defined order".

What it is

JEP 431 (a JEP is a numbered JDK Enhancement Proposal; final in Java 21) added three interfaces that capture "has a well-defined encounter order":

Interface	New methods	Retrofitted onto
`SequencedCollection<E>`	`addFirst`, `addLast`, `getFirst`, `getLast`, `removeFirst`, `removeLast`, `reversed()`	`List`, `Deque`
`SequencedSet<E>`	`reversed()` returns a `SequencedSet`	`LinkedHashSet`, `SortedSet` (so `TreeSet`)
`SequencedMap<K,V>`	`firstEntry`, `lastEntry`, `pollFirstEntry`, `pollLastEntry`, `putFirst`, `putLast`, `reversed()`, `sequencedKeySet()`, `sequencedValues()`, `sequencedEntrySet()`	`LinkedHashMap`, `SortedMap` (so `TreeMap`)

Plus `Collections.unmodifiableSequencedCollection/Set/Map`. `HashSet` and `HashMap` are **not** sequenced (they have no defined order).

How it behaves

1. `reversed()` returns a **live reverse-ordered view**, not a copy — O(1) to create.
2. `getFirst`/`removeFirst` etc. on an empty collection throw `NoSuchElementException`.
3. For sorted collections (`TreeSet`, `TreeMap`), `addFirst`/`addLast`/`putFirst`/`putLast` throw `UnsupportedOperationException`: position is decided by the comparator, not by you.
4. For `LinkedHashSet`/`LinkedHashMap`, `addFirst`/`putFirst` on an element that's already present **moves** it to the front.
5. Unmodifiable collections throw `UnsupportedOperationException` from the mutating methods, as usual.

Code

```
// code/ch10/sequenced/SequencedDemo.java (key parts)
List<String> list = new ArrayList<>(List.of("b", "c"));
list.addFirst("a"); list.addLast("d");
list.getFirst(); list.getLast(); list.reversed();

LinkedHashSet<String> tags = new LinkedHashSet<>(List.of("java", "sql", "git"));
tags.addFirst("git");                  // already present -> moved to the front

TreeSet<Integer> sorted = new TreeSet<>(List.of(30, 10, 20));
sorted.addFirst(5);                    // UnsupportedOperationException

List<String> rev = list.reversed(); list.add("e");   // the view sees the change

LinkedHashMap<String, Integer> steps = new LinkedHashMap<>();
steps.put("build", 1); steps.put("test", 2); steps.put("deploy", 3);
steps.putFirst("checkout", 0);
steps.firstEntry(); steps.lastEntry(); steps.reversed().keySet(); steps.pollFirstEntry();

static <E> E lastOf(SequencedCollection<E> c) { return c.getLast(); }   // one method for any ordered collection
```

Output:

```
List        first=a last=d reversed=[d, c, b, a]
LinkedHashSet [git, java, sql] removeLast=sql -> [git, java]
Deque       reversed=[3, 2, 1] getLast=3
TreeSet     first=10 last=30 reversed=[30, 20, 10]
TreeSet.addFirst -> UnsupportedOperationException (order comes from the comparator)
view after list.add(e): [e, d, c, b, a]
SequencedMap first=checkout=0 last=deploy=3 reversed keys=[deploy, test, build, checkout]
pollFirstEntry=checkout=0 -> [build, test, deploy]
lastOf(list)=e lastOf(dq)=3 lastOf(tags)=java
empty getFirst -> NoSuchElementException
HashSet is sequenced? false
```

Common mistakes

- Assuming `reversed()` copies — later changes to the original show through (and vice versa for mutable views).
- Calling `addFirst` on a `TreeSet` or on `List.of(...)`.
- Compatibility note: a home-grown class that implements `List` or `Deque` and already declared a method such as `reversed()` or `getFirst()` with a different return type stops compiling on Java 21. JEP 431 lists this as a known source-compatibility risk.

Self-check

1. Which of `HashSet`, `LinkedHashSet`, `TreeSet` are `SequencedSet`s? → `LinkedHashSet` and `TreeSet`.
2. What does `list.reversed()` cost? → O(1); it's a view.
3. What happens on `new LinkedHashMap<>().firstEntry()`? → Returns `null` for an empty map (the `SequencedMap` entry methods return `null`; only `getFirst`/`removeFirst`-style collection methods throw).

TL;DR

Java 21 gives every ordered collection the same `getFirst/getLast/addFirst/addLast/removeFirst/removeLast/reversed` API (and maps `firstEntry/lastEntry/pollFirstEntry/putFirst/...`). `reversed()` is a view; sorted collections refuse positional adds.

10.12 Big-O cheat sheet and a measured demo

Big-O describes how an operation's cost grows with the number of elements *n*: O(1) = constant, O(log n) = grows very slowly (tree height), O(n) = proportional (scan). "Amortised" means averaged over many calls; "average" for hash structures assumes a reasonable `hashCode`.

Collection	add (end)	add/remove front	get(i)	contains / get(key)	remove(element/key)	Iteration order	Nulls
`ArrayList`	O(1)*	O(n)	O(1)	O(n)	O(n)	index	yes
`LinkedList`	O(1)	O(1)	O(n)	O(n)	O(n)	index	yes
`ArrayDeque`	O(1)*	O(1)*	—	O(n)	O(n)	head→tail	**no**
`PriorityQueue`	O(log n)	poll O(log n)	—	O(n)	O(n)	heap (unsorted)	no
`HashSet` / `HashMap`	O(1)†	—	—	O(1)†	O(1)†	none	one null key
`LinkedHashSet` / `LinkedHashMap`	O(1)†	—	—	O(1)†	O(1)†	insertion/access	one null key
`TreeSet` / `TreeMap`	O(log n)	—	—	O(log n)	O(log n)	sorted	no (natural order)
`EnumSet` / `EnumMap`	O(1)	—	—	O(1)	O(1)	enum order	no
`CopyOnWriteArrayList`	O(n)	O(n)	O(1)	O(n)	O(n)	index	yes

* amortised. † average; worst case O(log n) for a treeified bucket (or O(n) for non-`Comparable` keys with colliding hashes).

Measured demo

`CollectionBenchmark` times a few operations once after a warm-up run (running the code first so the JIT compiler has optimised it). It is a *demonstration*, not a rigorous benchmark — use JMH (Java

Microbenchmark Harness) for real measurements (Chapter 16). **Timings are machine-dependent** (this run: Apple M4 Pro, JDK 26); what matters is the ratio within each row.

```
// code/ch10/perf/CollectionBenchmark.java (key parts)
Runnable getA = () -> { for (int i = 0; i < 20_000; i++) sink += array.get((i * 7919) % n); };
Runnable getL = () -> { for (int i = 0; i < 20_000; i++) sink += linked.get((i * 7919) % n); };
Runnable frontA = () -> { List<Integer> l = new ArrayList<>(); for (int i = 0; i < 50_000; i++) l.add(0, i); };
Runnable frontD = () -> { Deque<Integer> d = new ArrayDeque<>(); for (int i = 0; i < 50_000; i++) d.addFirst(i);
};
Runnable cL = () -> { for (int i = 0; i < 2_000; i++) sink += array.contains(n - 1 - i) ? 1 : 0; };
Runnable cH = () -> { for (int i = 0; i < 2_000; i++) sink += hash.contains(n - 1 - i) ? 1 : 0; };
// ... TreeSet contains, full iteration, presized vs default ArrayList
```

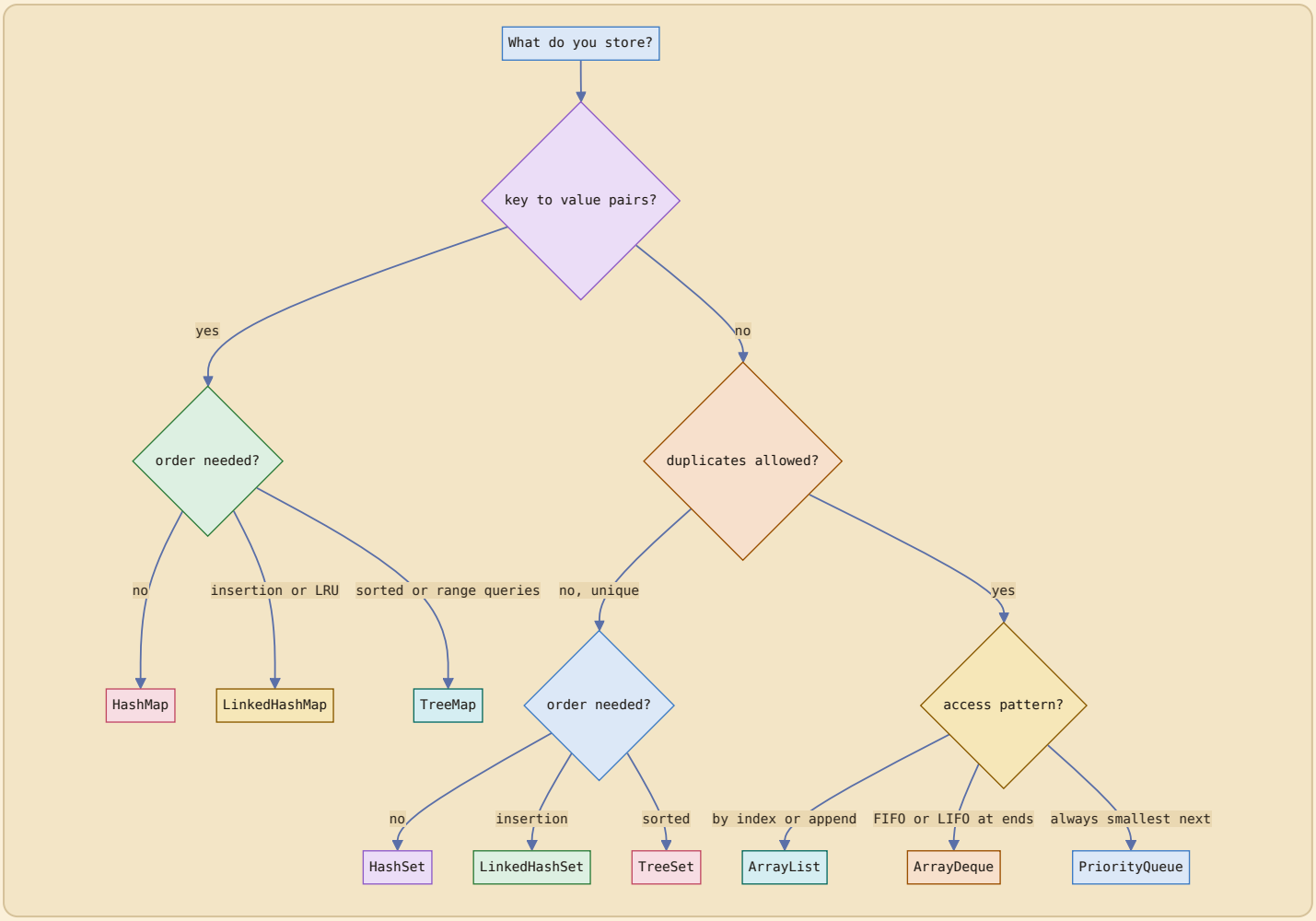
Output:

```
20k get(i) n=100k        ArrayList     0.6 ms | LinkedList   870.1 ms
50k add-at-front          ArrayList    83.6 ms | ArrayDeque     1.0 ms
2k contains n=100k         ArrayList    65.7 ms | HashSet 0.1 ms | TreeSet 0.3 ms
20 full iterations       ArrayList     4.2 ms | LinkedList     4.1 ms
2M adds                   default    42.7 ms | presized     7.7 ms
```

What each row shows:

1. **Indexing** — `LinkedList.get(i)` walks up to 50,000 nodes per call: >1000× slower.
2. **Front inserts** — `ArrayList.add(0, x)` shifts everything (O(n²) total); `ArrayDeque.addFirst` is O(1).
3. **Membership** — list scan vs hashing vs a ~17-level tree walk.
4. **Iteration** — both O(n); these freshly allocated nodes happen to sit close together, so the gap is small. In long-running apps with fragmented heaps (free memory scattered in many small gaps), linked structures usually iterate noticeably slower.
5. **Pre-sizing** — skipping ~30 grow-and-copy steps (and their garbage — discarded arrays the garbage collector must clean up) is several times faster.

10.13 Choosing a collection



Extra rules of thumb:

- Enum keys or elements → `EnumMap` / `EnumSet`.
- Fixed, known-at-compile-time data → `List.of` / `Set.of` / `Map.of`.
- Shared between threads → `ConcurrentHashMap`, `CopyOnWriteArrayList` (read-mostly), `BlockingQueue` (Chapter 14).
- Millions of primitives in a hot path → primitive arrays (or a primitive-collections library such as Eclipse Collections or fastutil) to avoid boxing.
- In doubt: `ArrayList` and `HashMap`, then measure.

Python readers: the same ideas as `list`, `set`, `dict`, `deque` and `heapq` in Python Chapter 6.

In the interview — collections quick round

- *`HashSet` vs `TreeSet`?* → O(1) unordered vs O(log n) sorted with range queries; `TreeSet` needs `Comparable` / `Comparator`, uses compare-to-zero for equality.
- *Why is `ArrayDeque` preferred over `Stack`?* → `Stack` extends synchronized `Vector` (locking overhead, exposes index methods); `ArrayDeque` is a lean circular array.
- *How would you build an LRU cache?* → `LinkedHashMap` with `accessOrder = true` and `removeEldestEntry` (or `HashMap` + doubly linked list by hand — LLD case studies).
- *Is `List.of` thread-safe?* → Yes for reading — it's immutable; the elements themselves may not be.

Chapter TL;DR

Topic	Remember
Hierarchy	`Iterable` → `Collection` → `List` / `Set` / `Queue` / `Deque`; `Map` separate with `keySet` / `values` / `entrySet` views; program to interfaces
`ArrayList`	growable array (10, then ×1.5): O(1) get/append, O(n) middle edits; the default list
`LinkedList`	doubly linked nodes: O(1) ends, O(n) get; rarely the best choice
Sets	`HashSet` O(1), `LinkedHashSet` insertion order, `TreeSet` sorted O(log n) + `floor` / `ceiling`; never mutate elements
Map API	`getOrDefault`, `putIfAbsent`, `merge`, `compute*`; iterate `entrySet`; `TreeMap` for ranges; `LinkedHashMap` for LRU
`HashMap` internals	hash spread → `& (n−1)`; chain then tree (>8 in bucket, table ≥ 64); load factor 0.75 → double; equal hash then `equals`; `newHashMap(n)` (19); not thread-safe
Queues	`ArrayDeque` for queue and stack; `PriorityQueue` = binary heap (O(log n), unsorted iteration); `offer/poll/peek` return special values
Iteration	fail-fast CME; remove with `removeIf` or `Iterator.remove`
Ordering	`Comparable` natural vs `Comparator` composed; never subtract; consistent with equals for `TreeSet` / `TreeMap`; sorts are stable
Immutability	`List/Set/Map.of` (9), `copyOf` (10): no nulls, shallow; `Stream.toList` (16) unmodifiable but allows nulls; `unmodifiableX` = view; `Arrays.asList` fixed-size
Sequenced (21)	`getFirst/getLast/addFirst/addLast/removeFirst/removeLast/reversed` on lists, deques, `LinkedHashSet`, sorted sets; maps get `firstEntry/pollFirstEntry/putFirst/...`

Where next: pass behaviour into collections methods cleanly with lambdas and method references in Chapter 11, then transform collections declaratively with Chapter 12 — Streams. Exam traps: OCP Chapter 9. Thread-safe collections: Chapter 14.

Chapter 11 — Lambdas and Functional Programming

Until Java 8, if you wanted to hand a piece of *behaviour* to a method — "sort by length", "run this later", "keep only the big orders" — you had to wrap it in an object, usually an anonymous class with five lines of ceremony around one line of logic. **Lambdas** (Java 8) let you write that behaviour directly as a value: `s -> s.length()`. Together with **functional interfaces**, **method references** and `Optional`, they brought a **functional style** (programming mostly by passing and combining functions, rather than by changing variables step by step) into Java that the collections API (Chapter 10) and streams (Chapter 12) are built on.

This chapter explains how lambdas work, which built-in function types exist and when to use each, how to combine small functions into bigger ones, how to use `Optional` well (and badly), and which functional-programming ideas pay off in everyday Java.

What you'll learn

1. Why lambdas exist: passing behaviour as a parameter.
2. Functional interfaces, lambda syntax and target typing; how lambdas differ from anonymous classes.
3. The `java.util.function` catalogue, including primitive specialisations.
4. The four kinds of method reference.
5. Variable capture and "effectively final"; what `this` means inside a lambda.
6. Composing functions: `andThen`, `compose`, `Predicate.and/or/not`, function factories.
7. `Optional`: the right uses, the wrong uses.
8. Functional style in practice: pure functions, immutability, higher-order functions, currying, laziness, memoisation, strategies, checked exceptions.

All runnable code is in `code/` (package `ch11.*`), compiled with `javac --release 25` and run on JDK 26 (the **JDK**, Java Development Kit, is the toolset that compiles and runs Java; `javac` is its compiler, and `--release 25` tells it to target the Java 25 language and library level).

> **Exam depth.** Which lambda forms compile, `var` in parameters, method-reference puzzles, the full built-in interface table with method names (`test`, `apply`, `accept`, `get`, `applyAsInt`…) and "effectively final" traps are drilled in OCP Chapter 8 — Lambdas and functional interfaces (see *Gotchas & exam traps*). `Optional` exam questions are in OCP Chapter 10.

11.1 Why lambdas: passing behaviour

The problem it solves

Many methods need a small piece of logic from the caller: `sort` needs "how to compare", a button needs "what to do when clicked", a thread needs "what to run", a filter needs "what to keep". This is called **behaviour parameterisation** — making behaviour an argument, just like data.

Before Java 8 the only way was an object implementing an interface, typically an **anonymous class** (a class defined and instantiated in one expression, §6.11):

```
names.sort(new Comparator<String>() {
    @Override public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
});
```

Five lines, of which only `Integer.compare(a.length(), b.length())` is the idea. A **lambda expression** is an anonymous function: parameters, an arrow, a body.

```
names.sort((a, b) -> Integer.compare(a.length(), b.length()));
```

Analogy: instead of hiring a whole employee (a class) to do one task, you hand over a sticky note with the instructions (the lambda). The receiving method decides when — and whether — to follow them.

Why it matters

- **Less noise**, more intent — *Effective Java* (Joshua Bloch's widely used book of numbered Java best-practice "Items") Item 42: "prefer lambdas to anonymous classes".
- APIs could become far more flexible: `Map.merge`, `List.removeIf`, `Comparator.comparing`, `CompletableFuture`, the Streams API — all take functions.
- Deferred and conditional execution (code that runs later, or only if some condition holds): pass a `Supplier` and the work happens only if needed (§11.8).

11.2 Functional interfaces and lambda syntax

What it is

A **functional interface** is an interface with **exactly one abstract method** (a method declared with no body, which an implementing class or lambda must supply) (a "single abstract method", or **SAM**, type). A lambda is simply a compact way to create an instance of one. `Runnable` (`void run()`), `Comparator<T>` (`int compare(T, T)`) and `Callable<V>` are classic examples.

Rules:

1. Only *abstract* methods count. `default` and `static` methods are fine, and so are abstract re-declarations of `Object`'s public methods (that's why `Comparator` can declare `equals` and still be functional).
2. The optional `@FunctionalInterface` **annotation** (an `@Name` tag attached to code that the compiler or tools can read) asks the compiler to verify the "exactly one" rule — a guard against someone later adding a second abstract method.
3. A lambda has **no type of its own**. Its type comes from the context it appears in (the compiler works it out, which is called **type inference**) — the **target type** (an assignment, a method parameter, a return, a cast). So the same text `s -> s.isEmpty()` can become a `Predicate<String>` or a `Function<String, Boolean>`. That's also why `var f = x -> x;` doesn't compile: there is no target type.

Syntax

Form	Example	Notes
No parameters	`() -> System.out.println("hi")`	parentheses required
One parameter, inferred type	`s -> s.length()`	parentheses optional
Several parameters	`(a, b) -> a + b`	parentheses required
Explicit types	`(Integer a, Integer b) -> a + b`	all or none typed
`var` parameters (Java 11)	`(var a, var b) -> a * b`	lets you add annotations: `(@NonNull var s) -> …`; all or none
Unnamed parameter (Java 22)	`(key, _) -> key`	`_` for a parameter you don't use
Expression body	`x -> x * 2`	value of the expression is returned
Block body	`n -> { if (n < 0) return "neg"; return "ok"; }`	braces, semicolons and `return` required

Code

```java
// code/ch11/lambdas/LambdaBasics.java (key parts)
@FunctionalInterface
interface Discount {
    double apply(double price);                                    // the single abstract method
    default Discount then(Discount next) { return p -> next.apply(apply(p)); }   // defaults allowed
}

Runnable noArgs                 = () -> System.out.println("no parameters");
Function<String, Integer> oneArg = s -> s.length();
BinaryOperator<Integer> typed    = (Integer a, Integer b) -> a + b;
BiFunction<Integer, Integer, Integer> withVar = (var a, var b) -> a * b;
Function<Integer, String> block  = n -> { if (n < 0) return "negative"; return "non-negative"; };

Predicate<String> isEmpty1          = s -> s.isEmpty();   // same text,
Function<String, Boolean> isEmpty2  = s -> s.isEmpty();   // different target types

Discount tenOff = p -> p - 10, twentyPercent = p -> p * 0.8;
tenOff.then(twentyPercent).apply(100);                    // (100 - 10) * 0.8

Map<String, BinaryOperator<Integer>> ops = Map.of("+", Integer::sum, "*", (a, b) -> a * b, "max", Math::max);
```

Output:

```
anonymous class: [al, Bea, Charlie]
lambda:          [Charlie, Bea, al]
no parameters
6 5 20 negative
Predicate.test=true Function.apply=false
100 -> tenOff then 20% = 72.0
+=13 *=42 max=7
```

The last line shows lambdas are *values*: stored in a map, looked up, called — a calculator's operator table in one line.

How it works under the hood

1. The compiler turns the lambda body into a private method of the enclosing class (named like `lambda$main$0`).
2. At the lambda's location it emits an `invokedynamic` instruction (a **bytecode** instruction — bytecode being the compact, portable instruction format that `javac` produces and the JVM runs — whose target is decided at run time on first use instead of being fixed at compile time). The first time it runs, the JVM's (Java Virtual Machine: the program that loads and executes Java bytecode) `LambdaMetafactory` generates a small class implementing the functional interface that calls that private method.
3. No separate `.class` file per lambda appears on disk (anonymous classes produce `Outer$1.class`).
4. A lambda that captures nothing is typically created once and reused; one that captures variables gets a new instance holding those values. (Don't rely on identity either way — it's unspecified.)

Lambda vs anonymous class

	Lambda	Anonymous class
Implements	only functional interfaces	any interface or class (can extend an abstract class)
Number of methods	one	any; can have fields and state
`this` refers to	the **enclosing** instance	the anonymous object itself
Shadowing (a new variable hiding an outer one with the same name)	can't redeclare a local variable of the enclosing scope	can shadow
Compiled as	`invokedynamic` + private method	a separate class file

Use an anonymous class only when you need state, several methods, or to extend a class.

Common mistakes

- A block body that forgets `return`, or an expression body followed by a semicolon inside: `x -> { x * 2 }` ✗ vs `x -> x * 2` ✓.
- Mixing typed and untyped parameters: `(String a, b) -> …` ✗.
- Naming a lambda parameter the same as an existing local variable → "variable is already defined".
- Long multi-statement lambdas — extract a named method and use a method reference; stack traces (the list of method calls printed when an exception is thrown) and readers both benefit.

Self-check

1. What makes an interface "functional"? → Exactly one abstract method (Object's public methods and defaults/statics don't count).
2. Why can't you write `var f = () -> 42;`? → A lambda needs a target type; `var` provides none.
3. What does `this` mean inside a lambda? → The enclosing object, same as outside the lambda.

TL;DR

A lambda is an anonymous function that implements a functional interface (one abstract method). Its type comes from context. Syntax: `params -> expression` or `params -> { statements; }`. Prefer it over anonymous classes unless you need state or `this` to mean the object itself.

11.3 The built-in functional interfaces (`java.util.function`)

What it is

You rarely need to write your own functional interface: `java.util.function` has **43** of them covering the common shapes. *Effective Java* Item 44: favour the standard ones — callers already know them and they come with useful **default methods** (interface methods that have a body, so every implementation gets them for free).

The core shapes

Interface	Shape	Method	Typical use
`Supplier<T>`	`() → T`	`get()`	factories, lazy values, defaults
`Consumer<T>`	`T → void`	`accept(t)`	`forEach`, logging, sending
`BiConsumer<T,U>`	`(T, U) → void`	`accept(t, u)`	`Map.forEach`
`Predicate<T>`	`T → boolean`	`test(t)`	`filter`, `removeIf`
`BiPredicate<T,U>`	`(T, U) → boolean`	`test(t, u)`	two-argument checks
`Function<T,R>`	`T → R`	`apply(t)`	`map`, key extractors
`BiFunction<T,U,R>`	`(T, U) → R`	`apply(t, u)`	`Map.merge`, `compute`
`UnaryOperator<T>`	`T → T`	`apply(t)`	`List.replaceAll`, transformations
`BinaryOperator<T>`	`(T, T) → T`	`apply(a, b)`	`reduce`, `merge` functions

Older JDK types are functional interfaces too: `Runnable` (`() → void`), `Callable<V>` (`() → V`, may throw), `Comparator<T>`.

Primitive specialisations

Generics (type parameters like `<T>`, Chapter 9) only work with objects, so a `Function<Integer, Integer>` **boxes** every `int` into an `Integer` (wraps the plain number in a heap object; *unboxing* is the reverse)

(allocating objects outside the small `Integer` cache — the JDK keeps ready-made `Integer` objects for -128..127 — Chapter 7). The primitive versions avoid that. Their names follow a pattern:

Pattern	Meaning	Examples
`IntX`, `LongX`, `DoubleX`	the **input** is that primitive	`IntPredicate`, `IntFunction<R>`, `IntConsumer`, `IntUnaryOperator`, `IntBinaryOperator`
`ToIntX`, `ToLongX`, `ToDoubleX`	the **output** is that primitive	`ToIntFunction<T>`, `ToDoubleBiFunction<T,U>`
`IntToLongFunction`, `DoubleToIntFunction`, …	primitive to primitive	
`XSupplier`	supplies a primitive	`IntSupplier` (`getAsInt`), `BooleanSupplier` (`getAsBoolean`)
`ObjIntConsumer<T>` etc.	object + primitive, no result	`(s, n) -> …`

The method names change accordingly: `applyAsInt`, `applyAsDouble`, `getAsInt`, `test`, `accept`. Only `int`, `long` and `double` get full sets (plus `BooleanSupplier`) — there is no `CharFunction` or `FloatPredicate`.

Code

```
// code/ch11/functional/BuiltInInterfaces.java (key parts)
Supplier<List<String>> newList = ArrayList::new;
Consumer<String> log = s -> System.out.println("log: " + s);
BiConsumer<String, Integer> put = (k, v) -> System.out.println(k + "=" + v);
Predicate<Order> big = o -> o.amount() > 100;
Function<Order, String> toId = Order::id;
BiFunction<Double, Double, Double> pow = Math::pow;
UnaryOperator<String> shout = s -> s.toUpperCase() + "!";
BinaryOperator<Integer> max = BinaryOperator.maxBy(Comparator.naturalOrder());

IntPredicate even = n -> n % 2 == 0;           IntUnaryOperator square = n -> n * n;
ToIntFunction<String> len = String::length;    IntFunction<String> stars = "*"::repeat;
ObjIntConsumer<String> repeatPrint = (s, n) -> System.out.println(s.repeat(n));

Function<Integer, Integer> boxedInc = x -> x + 1;   // boxes on every call
IntUnaryOperator primInc = x -> x + 1;              // no boxing
```

Output:

```
log: supplier made [x]
retries=3
big? true, longerThan("java",3)? true, id=A-1, 2^10=1024.0, HEY!, max=9
even(6)=true square(7)=49 add=5 len=5 stars=*** dice=4 ratio=0.25
ababab
20M calls: Function<Integer,Integer> 57 ms vs IntUnaryOperator 8 ms (machine-dependent) results equal=true
```

The last line is a rough single-run timing (Apple M4 Pro, JDK 26 — **machine-dependent**; the JIT — the JVM's *just-in-time* compiler, which turns frequently run bytecode into optimised machine code while the program runs — can sometimes remove boxing on its own). The point stands: in hot loops over numbers, pick the primitive interface.

When to write your own

- The shape doesn't exist (three parameters, or it throws a checked exception — §11.8).
- A domain name adds clarity and you'll attach default methods (`Discount.then` in §11.2).
- Annotate it with `@FunctionalInterface`.

Self-check

1. Which interface fits `list.removeIf(...)`? → `Predicate<? super E>`.

2. Which fits `map.merge(k, 1, ...)`? → `BiFunction<? super V, ? super V, ? extends V>` (commonly written `Integer::sum`).
3. What's the method of `ToIntFunction`? → `applyAsInt`.
4. `UnaryOperator<T>` extends which interface? → `Function<T, T>`.

TL;DR

Supplier (get), Consumer (accept), Predicate (test), Function (apply) plus `Bi`/`Unary`/`Binary` variants cover almost everything. Use `Int`/`Long`/`Double` versions for primitives to avoid boxing.

11.4 Method references

What it is

A **method reference** is shorthand for a lambda that does nothing but call one existing method: `String::length` means `s -> s.length()`. Like lambdas they need a target functional interface. *Effective Java* Item 43: prefer them when they are clearer than the lambda — and keep the lambda when it isn't.

The four kinds

Kind	Syntax	Equivalent lambda	Example
Static method	`Class::staticMethod`	`(args) -> Class.staticMethod(args)`	`Integer::parseInt`
Bound instance (specific object)	`obj::method`	`(args) -> obj.method(args)`	`System.out::println`, `prefix::concat`
Unbound instance (any object of a type)	`Class::instanceMethod`	`(obj, args) -> obj.method(args)`	`String::toUpperCase`, `String::startsWith`
Constructor	`Class::new`	`(args) -> new Class(args)`	`ArrayList::new`, `User::new`, `int[]::new`

The **unbound** form is the one that confuses people: the *first* parameter of the functional interface becomes the object the method is called on. So `BiFunction<String, String, Boolean> f = String::startsWith` means `(s, p) -> s.startsWith(p)`.

Code

```
// code/ch11/methodrefs/MethodRefDemo.java (key parts)
record User(String name, int age) {
    boolean isAdult() { return age >= 18; }
    static int byAge(User a, User b) { return Integer.compare(a.age, b.age); }
}
Function<String, Integer> parse = Integer::parseInt;          // static
Comparator<User> ageOrder = User::byAge;                      // static, shape (User, User) -> int
Function<String, String> addPrefix = prefix::concat;          // bound
Consumer<Object> printer = System.out::println;               // bound
Function<String, String> upper = String::toUpperCase;        // unbound
Predicate<User> adult = User::isAdult;                        // unbound
BiFunction<String, String, Boolean> startsWith = String::startsWith;   // unbound, 2 params
Supplier<List<User>> listMaker = ArrayList::new;              // constructor
BiFunction<String, Integer, User> makeUser = User::new;       // constructor with 2 args
IntFunction<int[]> arrayMaker = int[]::new;                   // array constructor

StringBuilder sb = new StringBuilder("v1");
Supplier<String> snap = sb::toString;                         // receiver sb fixed now
sb.append("+v2");
```

Output:

```
[User[name=Zed, age=17], User[name=Kim, age=25], User[name=Amy, age=34]]
43 ID-7 OK adult(Zed)=false startsWith=true array length=5
4 5
bound receiver is the same object: v1+v2
Amy Kim Zed
```

Notes:

- `parse.apply("42") + 1` printed `43`: the `Integer` result was unboxed and added before string concatenation started.
- `String::length` fits both `ToIntFunction<String>` and `Function<String, Integer>` — the target type decides.
- A **bound** reference evaluates its receiver expression **once**, when the reference is created (and throws `NullPointerException` — the error you get when using a reference that points to no object — right then if it's `null`). It holds that *object*, so later mutations of the object are visible (`v1+v2`), but reassigning the variable afterwards would not change the reference.
- Overloaded methods (several methods sharing one name but taking different parameters) can make a reference ambiguous (e.g. `Integer::toString` fits both the static `toString(int)` and the instance `toString()` for a `Function<Integer, String>`) — use a lambda then.

Self-check

1. Rewrite `s -> s.trim()` as a method reference. → `String::trim`.
2. Rewrite `() -> new HashMap<String, Integer>()`. → `HashMap::new` (target `Supplier<Map<String,Integer>>`).
3. What does `Comparator<String> c = String::compareToIgnoreCase;` mean? → `(a, b) -> a.compareToIgnoreCase(b)` — unbound, first parameter is the receiver.

TL;DR

Four kinds: `Class::static`, `obj::method` (bound), `Class::method` (unbound: first argument is the receiver), `Class::new`. Use when it reads better than the lambda.

11.5 Variable capture and "effectively final"

What it is

A lambda can use variables from the surrounding code — this is called **capturing**, and a function that carries captured variables is a **closure**. Java's rule: a captured **local variable or parameter** must be **final or effectively final** — never reassigned after initialisation (whether or not it says `final`).

Why the rule exists

1. Local variables live on the method's **stack frame** (the private scratch area a method call gets for its locals and parameters), which disappears when the method returns, but a lambda can outlive it (stored, passed to another **thread** — an independent path of execution running alongside others in the same program, Chapter 14). So Java **copies the value** into the lambda object.
2. If the variable could change later, the copy and the original would silently disagree. Forbidding reassignment makes the copy always correct.
3. It also prevents a whole class of **data races** (two threads touching the same variable at the same time, at least one writing, with no coordination — giving unpredictable results): a local variable mutated by lambdas running on several threads.

Fields are different. Instance fields are reached through the captured `this` reference (and static fields directly), so lambdas may read and modify them freely. That is legal — and exactly why it's easy to create thread-safety bugs (code that breaks only when several threads use it at once) with shared **mutable** (changeable) fields.

Code

```java
// code/ch11/capture/CaptureDemo.java (key parts)
int threshold = 10;                          // effectively final
Predicate<Integer> above = x -> x > threshold;
// threshold = 11;                           // would make the lambda above a compile error

for (String s : List.of("a", "b", "c")) suppliers.add(() -> s.toUpperCase());   // new 's' each iteration: fine
for (int i = 0; i < 3; i++) { int copy = i; byIndex.add(() -> copy * 10); }    // i is reassigned: copy it

AtomicInteger count = new AtomicInteger();                  // mutable holder: legal but a smell
List.of("x", "yy", "zzz").forEach(w -> count.addAndGet(w.length()));
int better = List.of("x", "yy", "zzz").stream().mapToInt(String::length).sum();   // return a result instead

Runnable lambda = () -> System.out.println("lambda this -> " + this.name);        // enclosing instance
Runnable anon = new Runnable() { public void run() { System.out.println(this.getClass().getName()); } };
Runnable inc = () -> counterField++;                        // fields may be modified
```

Output:

```
above(12)=true
ABC
0 10 20
holder count=6 vs stream sum=6
lambda this      -> CaptureDemo instance
anonymous this   -> ch11.capture.CaptureDemo$1
field modified from lambda: 2
```

Common mistakes

- Capturing the classic `for (int i…)` index — it's reassigned by `i++`. Copy to a local or use `IntStream.range(0, n).forEach(i -> …)`.
- Working around the rule with one-element arrays (`int[] c = {0}; list.forEach(x -> c[0]++)`) or `AtomicInteger` (a thread-safe integer holder whose updates happen as one indivisible step) just to accumulate. It compiles, but you've re-created shared mutable state; if the code later runs in parallel it breaks. Compute a result instead (`sum`, `count`, `collect`).
- Assuming a captured *object* is frozen — only the variable is effectively final; the object it points to can still be mutated.

Self-check

1. Can a lambda modify a captured local variable? → No; it can't even read one that is reassigned anywhere.
2. Can it modify an instance field? → Yes (via `this`), which is also why that can be unsafe across threads.
3. Why does capturing the variable of an enhanced for loop work? → Each iteration declares a fresh variable that is never reassigned.

TL;DR

Captured locals must be effectively final (values are copied into the lambda). Fields aren't restricted. Don't smuggle mutation through arrays or atomics — return results instead.

11.6 Composing functions

What it is

Composition builds a new function from existing ones, like joining pipes. The functional interfaces ship default and static methods for this, so you can assemble behaviour from small, tested, named pieces.

Interface	Composition methods
`Function`	`f.andThen(g)` = g(f(x)); `f.compose(g)` = f(g(x)); `Function.identity()`
`Predicate`	`and`, `or`, `negate`, `Predicate.not(p)` (11), `Predicate.isEqual(x)`
`Consumer` / `BiConsumer`	`andThen` — run one after the other
`BiFunction`	`andThen(Function)`
`BinaryOperator`	`minBy(comparator)`, `maxBy(comparator)`
`UnaryOperator`	`identity()`
`Comparator`	`comparing`, `thenComparing`, `reversed`, `nullsFirst` (§10.8)

Code

```
// code/ch11/composition/CompositionDemo.java (key parts)
Function<Integer, Integer> plus2 = x -> x + 2, times3 = x -> x * 3;
plus2.andThen(times3).apply(5);     // (5 + 2) * 3
plus2.compose(times3).apply(5);     // 5 * 3 + 2

UnaryOperator<String> trim = String::strip, lower = String::toLowerCase;
UnaryOperator<String> collapse = s -> s.replaceAll("\\s+", " ");
Function<String, String> clean = trim.andThen(lower).andThen(collapse);

Predicate<String> nonBlank = Predicate.not(String::isBlank);
Predicate<String> shortWord = s -> s.length() <= 4, startsJ = s -> s.startsWith("J");
filter(words, nonBlank.and(shortWord));   filter(words, startsJ.or(shortWord));
filter(words, startsJ.negate().and(nonBlank));   filter(words, Predicate.isEqual("Go"));

Consumer<String> audit = s -> System.out.print("[audit " + s + "] ");
audit.andThen(send).accept("msg-1");

Function<Double, UnaryOperator<Double>> taxAdder = rate -> price -> price * (1 + rate);   // function factory
UnaryOperator<Double> vat20 = taxAdder.apply(0.20);
```

Output:

```
plus2.andThen(times3)(5) = 21, plus2.compose(times3)(5) = 17
identity = same
clean -> [hello world]
nonBlank&short: [Java, Go, JVM]  startsJ|short: [Java, , Jakarta, Go,   , JVM]  !startsJ: [Go]  isEqual(Go): [Go]
[audit msg-1] [send msg-1]
area.andThen(format) = 12 m2, minBy length = fig
vat20(50.0) = 60.0
length, then reverse alpha: [a, cc, bb, aa, ddd]
```

Notes:

- `and` / `or` **short-circuit** (stop as soon as the answer is known) like `&&` / `||`: the second predicate isn't evaluated when the first decides.
- `startsJ.or(shortWord)` kept `""` and `" "` because they are short — combine with `nonBlank` when that matters.
- `Predicate.not(String::isBlank)` exists because you can't write `String::isBlank.negate()` — a method reference has no type until it's assigned.
- `trim.andThen(lower)` returns a `Function`, not a `UnaryOperator` (the default method is inherited from `Function`).

Real-world examples

Validation rules (`Predicate<Order>` combined per market), request-processing **middleware** (code that sits between an incoming request and the handler, e.g. adding logging or auth; here a chain of `UnaryOperator<Request>`), text-normalisation pipelines in search indexing (preparing text so it can be searched quickly), configurable pricing rules (function factories).

Self-check

1. `f.andThen(g)` vs `f.compose(g)`? → `andThen` runs f first; `compose` runs g first.
2. How to keep strings that are *not* empty using a method reference? → `Predicate.not(String::isEmpty)`.
3. What does `Function.identity()` return? → A function that returns its input unchanged (handy as a key/value mapper, e.g. `toMap(identity(), …)`).

TL;DR

Build big behaviour from small functions: `andThen`/`compose` for functions, `and`/`or`/`negate`/`not` for predicates, `andThen` for consumers, and functions that return functions for configurable logic.

11.7 `Optional`

What it is

`Optional<T>` (Java 8) is a container that holds either one non-null value or nothing. It is meant for **method return types** where "no result" is a normal outcome — `findById`, `max`, `findFirst` — so the signature itself tells the caller to handle absence. Analogy: a parcel delivery that may arrive empty; the box forces you to look inside before using the contents, whereas a `null` is a missing box you trip over.

Why it matters

`null` returned from a method says nothing in the signature, and forgetting a check causes a `NullPointerException` far from the cause. `Optional` makes absence explicit and gives fluent operations to handle it without nested `if (x != null)` chains.

The API

Group	Methods
Create	`Optional.of(v)` (NPE if `v` is null), `Optional.ofNullable(v)`, `Optional.empty()`
Test	`isPresent()`, `isEmpty()` (11)
Transform	`map(f)` (wraps the result; a `null` result becomes empty), `flatMap(f)` (when `f` already returns an `Optional`), `filter(p)`
Extract	`orElse(default)`, `orElseGet(supplier)` (a supplier is a no-argument function that produces a value), `orElseThrow()` (10), `orElseThrow(exceptionSupplier)`, `get()` (throws if empty)
Act	`ifPresent(consumer)`, `ifPresentOrElse(consumer, runnable)` (9)
Combine	`or(() -> otherOptional)` (9), `stream()` (9) → 0 or 1 elements
Primitive versions	`OptionalInt`, `OptionalLong`, `OptionalDouble` (`getAsInt()` …)

Code

```
// code/ch11/optional/OptionalDemo.java (key parts)
record Address(String city, String zip) {}
record Customer(String name, Address address, String email) {
    Optional<Address> addressOpt() { return Optional.ofNullable(address); }
    Optional<String> emailOpt() { return Optional.ofNullable(email); }
}
static Optional<Customer> findById(int id) { return Optional.ofNullable(DB.get(id)); }

String city = findById(1).flatMap(Customer::addressOpt).map(Address::city).orElse("no city");

findById(1).map(Customer::name).orElse(expensiveDefault());           // default computed ANYWAY
findById(1).map(Customer::name).orElseGet(OptionalDemo::expensiveDefault); // computed only if empty

findById(9).orElseThrow(() -> new IllegalArgumentException("customer 9 not found"));
findById(2).flatMap(Customer::emailOpt).ifPresentOrElse(e -> ..., () -> System.out.println("Bo has no email ->
send SMS"));
findById(2).flatMap(Customer::emailOpt).or(() -> Optional.of("sms:+000"));
List.of(1, 2, 9).stream().map(OptionalDemo::findById).flatMap(Optional::stream)...   // drop empties
```

Output:

```
Optional[hi] Optional.empty Optional.empty isPresent=true isEmpty=true
Optional.of(null) -> NPE
cities: Lisbon, no city, no city
gmail? false
orElse on present:    (computing default) Ana
orElseGet on present: Ana
orElseThrow() -> No value present
orElseThrow(supplier) -> customer 9 not found
Bo has no email -> send SMS
fallback Optional via or(): sms:+000
all known emails: [ana@example.com]
OptionalInt max=3, empty average=OptionalDouble.empty
anti-pattern isPresent+get: Ana
better: Ana
```

Read the chain `findById(id).flatMap(Customer::addressOpt).map(Address::city).orElse("no city")` as: find the customer → if found, get their address (which might itself be missing, so `flatMap` to avoid `Optional<Optional<Address>>`) → if present, take the city → otherwise "no city". Three possible "nothing" points, zero `if` statements.

The `orElse` line proves that **`orElse`'s argument is always evaluated** — it's an ordinary method argument. Use `orElseGet` when the default is expensive or has **side effects** (anything beyond returning a value — creating objects, DB calls, logging; defined fully in §11.8).

Right uses and wrong uses

✅ Do	❌ Don't
Return `Optional<T>` from a method when "no result" is normal	Return `null` from a method declared to return `Optional`
Chain `map` / `flatMap` / `filter` and finish with `orElse*` / `orElseThrow` / `ifPresent*`	`if (opt.isPresent()) opt.get()` — a verbose null check
Use `orElseThrow()` (10) when absence is a bug — its name says it can throw	Call `get()` without a check (`orElseThrow()` is the same thing with an honest name)
Use `OptionalInt` / `OptionalDouble` for primitives	`Optional<Integer>` in numeric code
Return an **empty collection** for "no items"	`Optional<List<T>>` — an empty list already means "none"
Use nullable fields internally and expose `Optional` from getters if helpful	`Optional` **fields** (not `Serializable` — can't be turned into bytes by Java's built-in serialisation — extra object per field)
Overload or use a default for optional inputs	`Optional` **parameters** (callers must wrap; they can still pass `null`)
Use it as a return value	Use it as a map key/value or in collections

These guidelines follow *Effective Java* Item 55 ("return optionals judiciously") and the intent stated by the JDK designers: `Optional` is primarily for return types. It is also a *value-based* class (one whose instances should be treated purely as values, with no meaningful identity) — don't compare instances with `==` or synchronise on them (use them as a lock, Chapter 14).

In the interview

> **30-second answer:** "Optional is a return-type container for 'maybe a value'. It makes absence explicit in the signature and lets me chain map, flatMap and filter and end with orElse, orElseGet or orElseThrow instead of null checks. I don't use it for fields, parameters or collections, I never return null from an Optional method, and I use orElseGet when the default is expensive because orElse evaluates its argument eagerly."

- *Follow-up:* `map` *vs* `flatMap`*?* → Use `flatMap` when the function itself returns an `Optional`, to avoid nesting.
- *Follow-up:* `Optional.of` *vs* `ofNullable`*?* → `of` throws NPE on null (use when null is a bug); `ofNullable` returns empty.
- *Follow-up: Is* `Optional` *serializable?* → No — one reason not to use it for fields.

Self-check

1. What does `Optional.ofNullable(null).map(String::length)` return? → `Optional.empty`.
2. Which is lazy: `orElse` or `orElseGet`? → `orElseGet`.
3. How do you turn a `Stream<Optional<T>>` into a `Stream<T>` of present values? → `.flatMap(Optional::stream)` (Java 9).

TL;DR

`Optional` = explicit "maybe" for return values. Transform with `map` / `flatMap` / `filter`, end with `orElse` / `orElseGet` / `orElseThrow` / `ifPresentOrElse`. Not for fields, parameters or collections; never `null` itself.

11.8 Functional style in everyday Java

What it is

Functional programming (FP) builds programs from **functions** that compute results from inputs, avoiding shared mutable state. Java is not a pure functional language, but several FP ideas make Java code simpler and safer. Key terms:

Term	Meaning
Pure function	Result depends only on its arguments; no side effects (no I/O, no mutation of outside state). Same input → same output, every time.
Side effect	Anything a function does besides returning a value: printing, writing a field, changing an argument, I/O.
Immutability	Data that can't change after creation (records, `List.of`, `String`).
Higher-order function	Takes a function as a parameter and/or returns one (`retryUntil`, `memoize`, `Comparator.comparing`).
Currying	Turning `f(a, b)` into `a -> b -> …`, so you can fix `a` first (**partial application**).
Laziness	Delaying a computation until its result is needed (`Supplier`, streams in Chapter 12).
Memoisation	Caching a pure function's results by input.
Declarative	Describe *what* result you want (`filter`, `map`) rather than *how* to loop — the style Chapter 12 is built on.

Code

```
// code/ch11/fp/FunctionalStyle.java (key parts)
static int total = 0;
static int addToTotal(int x) { total += x; return total; }    // impure: result depends on history
static int add(int a, int b) { return a + b; }                // pure

static <T> List<T> retryUntil(Supplier<T> attempt, Predicate<T> ok, int max) { ... }   // higher-order

Function<Integer, Function<Integer, Integer>> curriedAdd = a -> b -> a + b;
Function<Integer, Integer> add10 = curriedAdd.apply(10);      // partial application

static void logIf(boolean enabled, Supplier<String> message) { if (enabled) System.out.println(message.get()); }
logIf(debug, () -> "state dump " + "x".repeat(1_000_000).length());   // string never built when debug=false

static <T, R> Function<T, R> memoize(Function<T, R> f) {
    Map<T, R> cache = new HashMap<>();
    return x -> { R r = cache.get(x); if (r == null) { r = f.apply(x); cache.put(x, r); } return r; };
}

Map<String, DoubleUnaryOperator> shipping = Map.of(           // strategy pattern as data
        "standard", w -> 4.99, "express", w -> 9.99 + 1.5 * w, "free", w -> 0.0);

@FunctionalInterface interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }
static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (RuntimeException e) { throw e; }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}
```

Output:

```
impure twice: 5, 10   pure twice: 5, 5
retryUntil six: [2, 5, 6]
add10(5)=15
lazy log: supplier not called because debug=false
memoized: 5 calls, underlying function ran 2 times
standard(2kg)=4.99  express(2kg)=12.99  free(2kg)=0.00
app.properties=ok
wrapped: java.io.IOException: no such file: missing.properties
original [10, 20, 30] untouched; new [9, 18, 27]
```

What each idea buys you:

- **Pure functions** are trivially testable (no setup, no mocks), safe to cache and safe to run in parallel.
- **Higher-order functions** remove duplicated control flow: the retry loop is written once; callers pass *what* to try and *when* it succeeded.
- **Laziness via** `Supplier` is how logging frameworks avoid building expensive messages (SLF4J — the Simple Logging Facade for Java, a common logging front end — and its fluent API and `java.util.logging.Logger.fine(Supplier<String>)` both accept suppliers).
- **Strategies as lambdas** replace a hierarchy of one-method classes; see the Strategy pattern in LLD Chapter 10.
- **Immutability**: transform into a new list instead of editing in place; the original stays valid for anyone else holding it.

Memoisation note. The obvious `cache.computeIfAbsent(x, f)` is wrong when `f` is *recursive* (e.g. memoised Fibonacci calls back into the same map): `HashMap.computeIfAbsent` must not be modified by its own mapping function and throws `ConcurrentModificationException` (since Java 9) — `ConcurrentHashMap` (the thread-safe map) can **deadlock** (get stuck forever waiting on itself) or throw `IllegalStateException`. The get-then-put version above is safe for single-threaded use.

Checked exceptions (exceptions the compiler forces you to either catch or declare with `throws`, such as `IOException`). `java.util.function` methods don't declare `throws`, so a lambda calling `Files.readString` (which throws `IOException`) won't compile as a `Function`. Options: handle it inside the lambda, wrap it as above (rethrow as unchecked — `UncheckedIOException` is the JDK's own choice for I/O), or define your own throwing interface. Don't silently swallow the exception. See Chapter 8 for checked vs unchecked.

When not to use functional style

- The loop is simpler: a `for` with early `return` or several interacting variables is often clearer than a clever chain.
- Deeply nested lambdas (a function returning a function returning a function) — name intermediate pieces.
- Performance-critical inner loops where boxing or **megamorphic** calls (one call site that sees many different implementing classes, which stops the JIT from inlining it) matter — measure (Chapter 16).
- Stack traces through lambdas include **synthetic** frames (methods the compiler generated, not ones you wrote) like `lambda$main$0`; debugging a long chain is harder than a named method.

Real-world examples

Spring's (the most popular Java application framework) `RestClient`/`WebClient` builders and Security configuration **DSLs** (domain-specific languages: mini-APIs designed to read like a small language for one job) take lambdas; `CompletableFuture` chains functions over **async** (asynchronous: finishing later, without blocking the caller) results (Chapter 15); event listeners and callbacks in UI frameworks; retry/backoff helpers (backoff = waiting a bit longer between each retry; Resilience4j, a fault-tolerance library, decorates `Supplier`s and `Function`s); JUnit's (the standard Java unit-testing library, Chapter 18) `assertThrows(Class, Executable)`.

In the interview

> **30-second answer:** "In Java I use functional ideas where they reduce bugs: pure functions and immutable data for anything shared or tested, higher-order functions to remove repeated control flow, Suppliers for laziness, and lambdas as strategies. I stop when a plain loop is clearer, and I'm careful with checked exceptions and mutable captured state."

- *Follow-up: Is Java a functional language?* → No — it's object-oriented with functional features (first-class-ish functions — "first-class" meaning they can be stored, passed and returned like any other value — via functional interfaces, lambdas, streams, records, pattern matching). Functions are objects implementing interfaces, not a separate function type.
- *Follow-up: What's a pure function and why care?* → Output depends only on input, no side effects; easy to test, cache, and parallelise.

Self-check

1. Is `x -> { System.out.println(x); return x; }` pure? → No; printing is a side effect.
2. What's partial application of `a -> b -> a * b` with `a = 3`? → A function `b -> 3 * b`.
3. Why can't a lambda that calls `Files.readString` be a `Function<Path, String>` directly? → `IOException` is checked and `Function.apply` doesn't declare it.

TL;DR

Pure functions + immutable data + higher-order functions = code that's easier to test and parallelise. Use `Supplier` for laziness, lambdas for strategies, wrap checked exceptions deliberately, and fall back to loops when they're clearer.

Chapter TL;DR

Topic	Remember
Why	Pass behaviour as a value; lambdas replace anonymous classes (EJ Item 42)
Functional interface	Exactly one abstract method; `@FunctionalInterface`; lambda's type comes from the target
Syntax	`x -> expr`, `(a, b) -> { …; return r; }`, `(var a, var b)` (11), `_` for unused (22)
Built-ins	`Supplier.get`, `Consumer.accept`, `Predicate.test`, `Function.apply`, `Bi*`, `UnaryOperator`, `BinaryOperator`; `IntX`/`ToIntX` to avoid boxing
Method refs	`Class::static`, `obj::method`, `Class::method` (first arg is receiver), `Class::new`
Capture	Locals must be effectively final (value copied); fields free; `this` = enclosing instance
Composition	`andThen`/`compose`/`identity`, `and`/`or`/`negate`/`Predicate.not`, functions returning functions
Optional	Return type only; `map`/`flatMap`/`filter` → `orElse`/`orElseGet` (lazy)/`orElseThrow`; no fields/params/collections; never null
FP style	Pure, immutable, higher-order, lazy; wrap checked exceptions; loops when clearer

Where next: put all of this to work on collections with Chapter 12 — Streams. Exam traps: OCP Chapter 8. Python equivalents (lambdas, closures, `functools`): Python Chapter 7.

Chapter 12 — Streams

The **Streams API** (`java.util.stream`, Java 8) lets you describe *what* you want to compute from a sequence of values — "the names of customers who spent over 450, upper-cased and sorted" — as a **pipeline** (a chain of steps where each step's output feeds the next) of small operations, instead of writing *how* to loop, test, accumulate and sort by hand. It is built on the lambdas, method references and functional interfaces from Chapter 11 and usually starts from a collection (Chapter 10).

This chapter explains how a stream pipeline actually executes (**lazily** — work is postponed until a result is actually asked for — one element at a time), every everyday operation, the collectors that turn streams into maps and reports, primitive streams, parallel streams and when *not* to use them, the new **stream gatherers** (final in Java 24), and the mistakes that make stream code wrong or slow.

What you'll learn

1. What a stream is and isn't; building a pipeline; ways to create streams.
2. Lazy evaluation, vertical processing, short-circuiting, stateless vs stateful operations.
3. Intermediate operations: `filter`, `map`, `flatMap`, `mapMulti`, `distinct`, `sorted`, `limit`, `skip`, `takeWhile`, `dropWhile`, `peek`.
4. Terminal operations and the three forms of `reduce`.
5. Collectors: `toList`, `toMap`, `joining`, `groupingBy`, `partitioningBy`, downstream collectors, `teeing`, custom collectors.
6. Primitive streams: `IntStream`, `LongStream`, `DoubleStream`.
7. Parallel streams — how they work, measured wins and losses, and correctness rules.
8. Gatherers (Java 24): built-in windows, `scan`, `fold`, `mapConcurrent`, and writing your own.
9. Common mistakes, streams vs loops, and a realistic reporting pipeline.

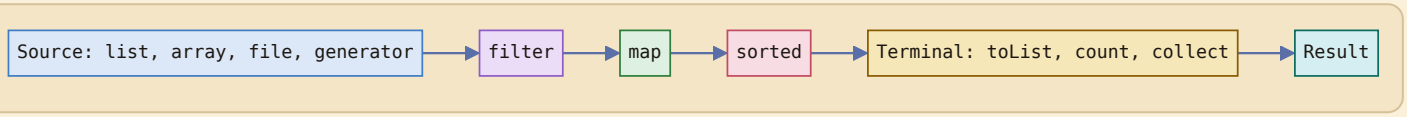
All runnable code is in `code/` (package `ch12.*`), compiled with `javac --release 25` (the Java compiler, targeting Java 25) and run on JDK 26 (the Java Development Kit: compiler plus runtime).

> **Exam depth.** Output-prediction puzzles (laziness, `peek`, infinite streams that hang), the exact return types of every terminal operation, `Optional` vs primitive optionals, `Spliterator`, and collector signatures are drilled in OCP Chapter 10 — Streams (see *Gotchas & exam traps*).

12.1 What a stream is

What it is

A **stream** is a sequence of elements that supports a chain of operations computed on demand. Think of a factory conveyor belt: items come off a **source** (a collection, an array, a file, a generator), pass through stations that inspect, reshape or discard them (**intermediate operations**), and end at a station that produces a final product (**terminal operation**). Nothing moves until that final station is switched on.



A stream is **not** a collection:

	Collection	Stream
Purpose	stores elements	computes over elements
Elements	all exist in memory	produced on demand; may be infinite
Traversal	as often as you like	**once** (then it's consumed)
Iteration (stepping through elements)	external (you write the loop)	internal (the library runs the loop)
Changes source?	`add` / `remove` modify it	never modifies its source

Why it matters

- **Readability**: each step names its intent (`filter`, `map`, `groupingBy`), and multi-step data processing reads top to bottom.
- **Fewer bugs**: no index variables, no forgetting to reset an accumulator.
- **Optimisation room**: because the library runs the loop, it can fuse steps (merge them into a single pass), stop early, and (optionally) split work across **cores** (the independent processing units inside a CPU, each able to run one thread at a time).

Code: loop vs pipeline

```
// code/ch12/basics/PipelineDemo.java (key parts)
record Order(String customer, String product, int qty, double price) { double total() { return qty * price; } }

List<String> loop = new ArrayList<>();                          // imperative: HOW
for (Order o : ORDERS) if (o.total() >= 450) loop.add(o.customer().toUpperCase());
Collections.sort(loop);

List<String> stream = ORDERS.stream()                           // declarative: WHAT
        .filter(o -> o.total() >= 450)
        .map(o -> o.customer().toUpperCase())
        .sorted()
        .toList();                                              // Java 16
```

The loop is **imperative** (step-by-step instructions saying *how*); the pipeline is **declarative** (a description of *what* result you want).

Output (first two lines):

```
loop:   [ANA, ANA, BO, CY]
stream: [ANA, ANA, BO, CY]
```

Creating streams

Source	Code	Notes
Any collection	`list.stream()`, `set.parallelStream()`	the usual start
Values	`Stream.of("a", "b")`	varargs (a method that accepts any number of arguments)
Array	`Arrays.stream(arr)`, `Arrays.stream(arr, from, to)`	`int[]` → `IntStream`
Number ranges	`IntStream.range(0, n)` (exclusive), `rangeClosed(1, n)`	
Infinite, by rule	`Stream.iterate(seed, f)`; `Stream.iterate(seed, hasNext, next)` (9)	use `limit` or the 3-arg form
Infinite, by supplier	`Stream.generate(supplier)`	unordered, needs `limit`
Maybe one value	`Stream.ofNullable(x)` (9), `optional.stream()` (9)	0 or 1 elements
Joining two	`Stream.concat(a, b)`	
Characters / lines	`str.chars()`, `str.lines()` (11)	
Files	`Files.lines(path)`, `bufferedReader.lines()`	lazy; close it — Chapter 13
Map	`map.entrySet().stream()`	maps aren't streams themselves
Random numbers	`new Random(seed).ints(n, lo, hi)`	seed = starting value; same seed, same numbers
Builder	`Stream.<T>builder().add(x).build()`	

```
Stream.iterate(1, x -> x * 2).limit(8).toList();
Stream.iterate(1, x -> x < 100, x -> x * 3).toList();          // like a for loop
Stream.generate(() -> "ab").limit(3).collect(Collectors.joining("-"));
Stream<String> s = Stream.of("once"); s.forEach(x -> {}); s.count();   // IllegalStateException
```

Output (remaining lines):

```
Stream.of        [a, b, c]
Arrays.stream    [3, 1, 2]
IntStream.range  [0, 1, 2, 3, 4] rangeClosed [1, 2, 3]
iterate(seed,f)  [1, 2, 4, 8, 16, 32, 64, 128]
iterate(s,has,f) [1, 3, 9, 27, 81]
generate         ab-ab-ab
String.chars     [h, e, y]
Map.entrySet     [x1, y2]
ofNullable(null) 0
concat           [1, 2, 3]
lines()          3 lines
builder          [built, stream]
reuse -> stream has already been operated upon or closed
source [3, 1, 2] unchanged, result [1, 2, 3]
```

Self-check

1. What are the three parts of a pipeline? → Source, zero or more intermediate operations, one terminal operation.
2. Can you run two terminal operations on the same stream? → No — `IllegalStateException` (Java's exception for "this object isn't in a state where that call is allowed"); create a new stream from the source.
3. Does `list.stream().sorted()` sort `list`? → No; streams never modify their source.

TL;DR

Stream = a one-shot, lazy pipeline over a source: source → intermediate ops → terminal op → result. It describes a computation; it doesn't store data.

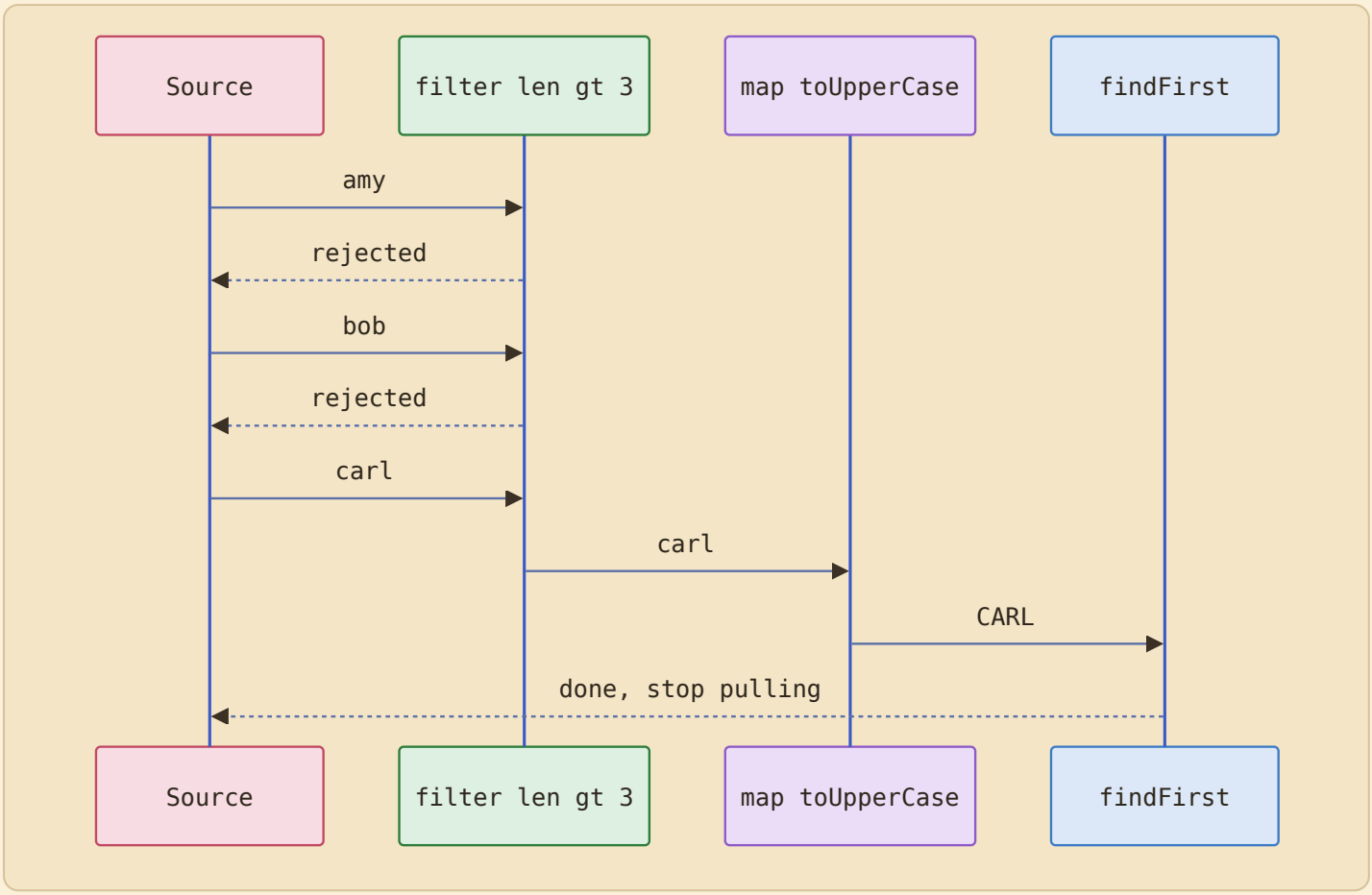
12.2 Laziness: how a pipeline really executes

What it is

Intermediate operations are lazy: calling `filter` or `map` only *records* a step and returns a new stream. The work happens when a **terminal operation** pulls elements through. And it happens **element by element through all stages** ("vertical" processing), not stage by stage over the whole list ("horizontal").

How it works

1. Each intermediate call adds a *stage* to a linked pipeline description.
2. The terminal operation asks the source for elements — via a `Spliterator` (a "splittable iterator" that can also split its range in two for parallel work).
3. Each element travels through every stage in order; stages are fused into one pass, so there are no intermediate lists.
4. **Short-circuiting** operations (`findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, `limit`, `takeWhile`) stop pulling as soon as the answer is known — which is what makes infinite streams usable.
5. **Stateless** operations (`filter`, `map`, `flatMap`, `peek`) handle each element independently. **Stateful** ones need to remember other elements: `distinct` (a set of seen items), `sorted` (**buffers everything** before emitting anything — a *barrier*), `limit`/`skip` (counters).



Code

```
// code/ch12/laziness/LazinessDemo.java (key parts)
Stream<String> pipeline = names.stream().filter(n -> { System.out.println("filter " + n); return true; });
System.out.println("pipeline built, nothing printed yet");   // no terminal op -> nothing ran

Optional<String> first = names.stream()
        .filter(n -> { System.out.println("filter " + n); return n.length() > 3; })
        .map(n -> { System.out.println("map    " + n); return n.toUpperCase(); })
        .findFirst();

List<Integer> squares = Stream.iterate(1, i -> i + 1)          // infinite
        .peek(i -> System.out.print("gen " + i + " | "))
        .map(i -> i * i).filter(sq -> sq % 2 == 1).limit(3).toList();

Stream.of("c", "a", "b").peek(x -> System.out.println("before sort " + x))
      .sorted().forEach(x -> System.out.println("after sort  " + x));
```

Output:

```
pipeline built, nothing printed yet
--- vertical processing + short-circuit (findFirst)
filter amy
filter bob
filter carl
map    carl
result CARL  (dave and eve were never touched)
--- limit on an infinite stream
gen 1 | gen 2 | gen 3 | gen 4 | gen 5 |
odd squares [1, 9, 25]
--- sorted() is a barrier
before sort c
before sort a
before sort b
after sort  a
after sort  b
after sort  c
map-then-filter: 1000 map calls; filter-then-map: 5 map calls (same count 5=5)
```

What to notice:

- The first pipeline never printed anything — it was never run.
- `map` ran **once**, not five times: `findFirst` stopped the pipeline at `carl`.
- The infinite `iterate` generated exactly 5 numbers, because `limit(3)` stopped pulling after the third odd square.
- `sorted()` consumed all three elements before passing any on.
- **Order of operations affects cost**: filtering first meant 5 `map` calls instead of 1,000. Put cheap, selective `filter`s early and expensive or stateful steps (`sorted`, `distinct`) late.

Common mistakes

- Expecting a side effect in `map`/`peek` to happen without a terminal operation.
- Putting `sorted()` or `distinct()` *before* `limit` on an infinite stream → it never finishes (the barrier waits forever).
- Relying on `peek` for real work — the library may skip it entirely (§12.10).

Self-check

1. `Stream.of(1, 2, 3).map(x -> { System.out.println(x); return x; });` prints what? → Nothing — no terminal operation.
2. Which operations short-circuit? → `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, `limit`, `takeWhile`.
3. Why is `sorted` called a barrier? → It must receive every element before it can emit the first one.

TL;DR

Nothing runs until a terminal operation. Elements flow one at a time through all fused stages; short-circuiting ops stop early; stateful ops (`sorted`, `distinct`) buffer. Filter early, sort late.

12.3 Intermediate operations

Operation	Does	Stateful?	Notes
`filter(Predicate)`	keep matching elements	no	
`map(Function)`	transform each element (1 → 1)	no	may change type
`mapToInt/Long/Double`, `mapToObj`, `boxed`	switch between object and primitive streams	no	§12.6
`flatMap(Function<T, Stream<R>>)`	each element → 0..n elements, flattened	no	
`mapMulti(BiConsumer<T, Consumer<R>>)` (16)	push 0..n results imperatively	no	avoids creating tiny streams
`distinct()`	drop duplicates (by `equals` / `hashCode`, the methods objects use to compare and bucket themselves)	**yes**	keeps first occurrence
`sorted()` / `sorted(Comparator)`	sort	**yes (barrier)**	stable (equal items keep their original order) for ordered streams
`limit(n)` / `skip(n)`	first n / all but first n	yes (counter)	`limit` short-circuits
`takeWhile(p)` / `dropWhile(p)` (9)	take / drop the **prefix** while `p` holds	yes	stop at the first failure
`peek(Consumer)`	look at elements passing by	no	debugging only
`parallel()`, `sequential()`, `unordered()`, `onClose(Runnable)`	change execution mode / characteristics	—	§12.8

Code

```java
// code/ch12/ops/IntermediateOps.java (key parts)
List<Integer> nums = List.of(5, 3, 8, 3, 1, 9, 8, 2);
nums.stream().distinct().toList();
nums.stream().sorted(Comparator.reverseOrder()).toList();
nums.stream().skip(2).limit(3).toList();
nums.stream().takeWhile(n -> n < 8).toList();
nums.stream().dropWhile(n -> n < 8).toList();

books.stream().map(Book::tags).toList();                                  // List<List<String>>
books.stream().flatMap(b -> b.tags().stream()).distinct().sorted().toList(); // List<String>
Stream.of("to be", "or not").flatMap(line -> Arrays.stream(line.split(" "))).toList();

books.stream().<String>mapMulti((b, sink) -> {
    for (String t : b.tags()) if (t.startsWith("d")) sink.accept(b.title() + ":" + t);
}).toList();
```

Output:

```
filter even      [8, 8, 2]
map x10          [50, 30, 80, 30, 10, 90, 80, 20]
distinct         [5, 3, 8, 1, 9, 2]
sorted           [1, 2, 3, 3, 5, 8, 8, 9]
sorted desc      [9, 8, 8, 5, 3, 3, 2, 1]
skip 2 limit 3   [8, 3, 1]
takeWhile <8     [5, 3]   (Java 9: stops at first failure)
dropWhile <8     [8, 3, 1, 9, 8, 2]
filter <8        [5, 3, 3, 1, 2]   (compare: looks at every element)
map (nested)     [[java, best-practices], [java, streams, fp], [databases, distributed]]
flatMap          [best-practices, databases, distributed, fp, java, streams]
flatMap words    [to, be, or, not]
mapMulti         [DDIA:databases, DDIA:distributed]
mapToInt sum     6053
peek saw 3 titles, kept [Effective Java, DDIA]
sorted by length [a, bb, ccc]
boxed / asLong   [1, 2]
unordered/parallel/sequential/onClose also exist (see 12.8)
```

`map` vs `flatMap` — the one to really understand

- `map` is **one in, one out**. If your function returns a list (or stream, or `Optional`), you get a stream *of lists*: `Stream<List<String>>`.
- `flatMap` is **one in, many out**: the function returns a *stream* for each element, and `flatMap` splices all of those streams into one: `Stream<String>`. Analogy: `map` puts each book's tag list into a box; `flatMap` empties all the boxes onto one belt.
- Uses: all words from all lines, all order lines from all orders, all present values from `Optional`s (`flatMap(Optional::stream)`).
- **`mapMulti`** (Java 16) does the same job imperatively — you call `sink.accept(x)` for each output. It's handy when each element yields very few results (avoids allocating a stream per element) or when the logic is easier as a loop. Note the explicit **type witness** `<String>mapMulti(...)` (spelling out the generic type yourself because the compiler can't work it out); inference often needs it.

`takeWhile`/`dropWhile` vs `filter`

`takeWhile(n -> n < 8)` stopped at the first 8 and returned `[5, 3]`, even though `3, 1, 2` later are also < 8. It is meant for **sorted or naturally ordered** data ("log lines until the first error", "prices while under budget"). `filter` examines every element.

Self-check

1. What type does `Stream.of("a b", "c").map(s -> s.split(" "))` produce? → `Stream<String[]>`; use `flatMap(s -> Arrays.stream(s.split(" ")))` for `Stream<String>`.
2. `Stream.of(1, 5, 2, 7).dropWhile(x -> x < 3).toList()`? → `[5, 2, 7]`.
3. Which intermediate ops are stateful? → `distinct`, `sorted`, `limit`, `skip`, `takeWhile`, `dropWhile`.

TL;DR

`filter` keeps, `map` transforms 1→1, `flatMap`/`mapMulti` 1→many, `distinct`/`sorted` need state, `limit`/`skip`/`takeWhile`/`dropWhile` cut the sequence, `peek` is for debugging.

12.4 Terminal operations and `reduce`

Operation	Returns	Short-circuits?	Notes
`forEach(c)` / `forEachOrdered(c)`	`void`	no	`forEachOrdered` keeps **encounter order** (the order the source delivers elements in) in parallel
`toList()` (16)	unmodifiable (read-only: `add` / `remove` throw) `List<T>`	no	allows `null` elements
`collect(collector)`	anything	no	§12.5
`toArray()` / `toArray(String[]::new)`	`Object[]` / `T[]`	no	
`count()`	`long`	no	may skip the pipeline if size is known
`min(c)` / `max(c)`	`Optional<T>`	no	
`findFirst()` / `findAny()`	`Optional<T>`	**yes**	`findAny` is cheaper in parallel
`anyMatch` / `allMatch` / `noneMatch`	`boolean`	**yes**	on an empty stream: `false` / `true` / `true`
`reduce(...)`	`T` or `Optional<T>` or `U`	no	three forms below
`iterator()` / `spliterator()`	pull elements manually	—	escape hatch (a way out of the stream API when you need it)

`reduce` — folding a stream into one value

Reduction repeatedly combines elements until one value remains — summing, multiplying, finding the longest.

Form	Signature	Use
Identity + accumulator	`T reduce(T identity, BinaryOperator<T> acc)`	`reduce(0, Integer::sum)`
Accumulator only	`Optional<T> reduce(BinaryOperator<T> acc)`	empty stream → `Optional.empty()`
Identity + accumulator + combiner	`U reduce(U identity, BiFunction<U,T,U> acc, BinaryOperator<U> combiner)`	result type differs from element type, e.g. total length of strings; the combiner merges partial results in parallel

Rules that keep `reduce` correct (they matter the moment you go parallel):

1. The **identity** must be neutral: `combine(identity, x) == x` (0 for +, 1 for ×, `""` for concatenation).
2. The function must be **associative** (grouping doesn't change the answer): `(a ⊕ b) ⊕ c == a ⊕ (b ⊕ c)`. Subtraction and averaging are not.
3. It must be **stateless** and side-effect free.

For summing numbers, prefer `mapToInt(...).sum()`; for building collections use `collect`, not `reduce` (reducing into a new list at every step is O(n²) — in **Big-O** notation, which describes how work grows with input size *n*, that means doubling the input roughly quadruples the time).

Code

```
// code/ch12/ops/TerminalOps.java (key parts)
words.stream().max(Comparator.comparingInt(String::length)).orElseThrow();
words.stream().filter(w -> w.startsWith("z")).findAny();              // Optional.empty
Stream.<String>empty().allMatch(w -> false);                          // true (vacuous truth)
Stream.of(1, 2, 3, 4).reduce(0, Integer::sum);
Stream.of(1, 2, 3, 4).reduce((a, b) -> a * b);                        // Optional
words.stream().reduce(0, (acc, w) -> acc + w.length(), Integer::sum);  // String -> int
words.stream().collect(Collectors.toCollection(ArrayList::new));       // when you need a mutable list
words.parallelStream().forEachOrdered(w -> System.out.print(w + " "));
```

Output:

```
count           3
min / max       java / stream
findFirst       Optional[record]
findAny         Optional.empty
anyMatch        true
allMatch        true
noneMatch       true
allMatch on EMPTY true (vacuous truth)
reduce          sum=10 product=24 totalLength=25 emptyReduce=Optional.empty
reduce longest  stream
toList [STREAM, LAMBDA, JAVA, RECORD, VAR] | toCollection+add 6 | toArray 5
forEachOrdered (parallel): stream lambda java record var
iterator first two: java, lambda
```

`allMatch` on an empty stream is `true` by **vacuous truth** ("every element matches" can't be false when there are no elements). `max` by length returned `stream`, the first of three 6-letter words. The Javadoc (the official API documentation generated from comments in the JDK source) doesn't promise which of several tied elements you get, so add a tie-breaker (`thenComparing`) when it matters.

`toList()` vs `collect(Collectors.toList())`

	`stream.toList()` (16)	`collect(Collectors.toList())`	`collect(Collectors.toUnmodifiableList())` (10)
Mutability	unmodifiable	unspecified (currently `ArrayList`)	unmodifiable
Nulls	allowed	allowed	**NPE** (`NullPointerException`)

Use `toList()` by default; `toCollection(ArrayList::new)` when you genuinely need to modify the result.

Self-check

1. What does `reduce` without an identity return? → `Optional<T>`.
2. `IntStream.empty().anyMatch(x -> true)`? → `false`.
3. Why is `reduce(5, Integer::sum)` wrong for a sum? → 5 isn't an identity for addition; in parallel it's added once per chunk (§12.8).

TL;DR

Terminal ops run the pipeline: `toList`, `collect`, `count`, `min`/`max`, `find*`, `*Match`, `reduce`, `forEach`. `reduce` needs a true identity and an associative, stateless function. `toList()` is unmodifiable.

12.5 Collectors

What it is

A `Collector` is a recipe for **mutable reduction**: accumulate stream elements into a container (list, map, string builder, statistics) and optionally finish it. `Collectors` (with an *s*) provides ready-made ones;

you pass them to `collect`. Add `import static java.util.stream.Collectors.*;` to keep pipelines readable.

The toolbox

Collector	Result	Notes
`toList()`, `toSet()`, `toUnmodifiableList/Set()` (10)	`List`, `Set`	
`toCollection(TreeSet::new)`	any collection	choose the implementation
`toMap(keyFn, valueFn)`	`Map`	**duplicate key → `IllegalStateException`**; `null` value → NPE
`toMap(keyFn, valueFn, mergeFn)`	`Map`	merge function resolves duplicates
`toMap(keyFn, valueFn, mergeFn, TreeMap::new)`	chosen map type	
`joining()`, `joining(", ")`, `joining(", ", "[", "]")`	`String`	elements must be `CharSequence` (the interface behind `String`, `StringBuilder` and friends)
`groupingBy(classifier)`	`Map<K, List<T>>`	the SQL `GROUP BY` (database clause that buckets rows by a column) of streams
`groupingBy(classifier, downstream)` / `(classifier, mapFactory, downstream)`	`Map<K, D>`	downstream collector per group
`partitioningBy(predicate[, downstream])`	`Map<Boolean, …>`	always both keys `false` and `true`
`counting()`, `summingInt/Long/Double`, `averagingX`, `minBy`, `maxBy`	numbers / `Optional`	usually as downstream
`summarizingInt/Long/Double`	`IntSummaryStatistics` etc.	count, sum, min, max, average in one pass
`mapping(fn, downstream)`, `filtering(p, downstream)` (9), `flatMapping(fn, downstream)` (9)	adapt elements inside a group	
`collectingAndThen(downstream, finisher)`	transformed result	e.g. unwrap an `Optional`, freeze a list
`teeing(c1, c2, merger)` (12)	anything	two collectors in one pass, then combine
`reducing(identity, op)`	`T`	reduction as a downstream collector
`groupingByConcurrent`, `toConcurrentMap`	`ConcurrentMap`	for parallel streams, unordered

Code

```
// code/ch12/collectors/CollectorsDemo.java (key parts)
record Employee(String name, String dept, String city, double salary, List<String> skills) {}

STAFF.stream().map(Employee::name).collect(joining(", ", "[", "]"));
STAFF.stream().collect(toMap(Employee::dept, Employee::name));                    // throws: duplicate ENG
STAFF.stream().collect(toMap(Employee::dept, Employee::name, (a, b) -> a + "+" + b, TreeMap::new));

STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new, mapping(Employee::name, toList())));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new, counting()));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        collectingAndThen(maxBy(Comparator.comparingDouble(Employee::salary)), o -> o.orElseThrow().name())));
STAFF.stream().collect(groupingBy(Employee::city, TreeMap::new, groupingBy(Employee::dept, TreeMap::new,
counting())));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        filtering(e -> e.salary() > 100_000, mapping(Employee::name, toList()))));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        flatMapping(e -> e.skills().stream(), toCollection(TreeSet::new))));

STAFF.stream().collect(partitioningBy(e -> e.salary() >= 110_000, mapping(Employee::name, toList())));
STAFF.stream().collect(summarizingDouble(Employee::salary));
STAFF.stream().collect(teeing(minBy(bySalary), maxBy(bySalary),
        (min, max) -> min.orElseThrow().name() + ".." + max.orElseThrow().name()));

Collector<String, StringJoiner, String> pipes = Collector.of(
        () -> new StringJoiner("|"),   // supplier: new container
        StringJoiner::add,             // accumulator: add one element
        StringJoiner::merge,           // combiner: merge two containers (parallel)
        StringJoiner::toString);       // finisher: container -> result
```

Output:

```
joining          [Ana, Bo, Cy, Dee, Eli]
toSet(TreeSet)   [Berlin, Lisbon]
toMap            {Ana=120000.0, Bo=150000.0, Cy=90000.0, Dee=80000.0, Eli=110000.0}
toMap dup key -> IllegalStateException: Duplicate key ENG (attempted merging values Ana and Bo)
toMap merge      {ENG=Ana+Bo+Eli, OPS=Cy, SALES=Dee}
toMap identity   Bo -> ENG
groupingBy       {ENG=[Ana, Bo, Eli], OPS=[Cy], SALES=[Dee]}
  counting       {ENG=3, OPS=1, SALES=1}
  averaging      {ENG=126666.66666666667, OPS=90000.0, SALES=80000.0}
  summing        {Berlin=230000.0, Lisbon=320000.0}
  maxBy          {ENG=Bo, OPS=Cy, SALES=Dee}
  multi-level    {Berlin={ENG=1, SALES=1}, Lisbon={ENG=2, OPS=1}}
  filtering(9)   {ENG=[Ana, Bo, Eli], OPS=[], SALES=[]}
  flatMapping(9) {ENG=[go, java, k8s, sql], OPS=[bash, k8s], SALES=[excel]}
partitioningBy   {false=[Cy, Dee], true=[Ana, Bo, Eli]}
partition empty  {false=[], true=[]}
summarizing      count=5 min=80000 max=150000 avg=110000 sum=550000
teeing           lowest..highest paid = Dee..Bo
collectingAndThen UnmodifiableRandomAccessList
reducing         550000.0
Collector.of     Ana|Bo|Cy|Dee|Eli
```

How to read the grouping results:

- `groupingBy` = SQL `GROUP BY dept`; the **downstream** collector is the aggregate (`COUNT(*)`, `AVG(salary)`, `MAX(...)`). Compare with the SQL book's grouping chapter if you know SQL.
- `groupingBy` with no map factory (a function that creates the map) returns a `HashMap` (no order); pass `TreeMap::new` (a map that keeps keys sorted) for sorted keys — used here so the output is predictable.
- `filtering` inside the group kept the `OPS` and `SALES` keys with empty lists. A `filter` *before* `groupingBy` would have removed those groups entirely — choose based on whether you want empty groups reported.
- `partitioningBy` always contains both `false` and `true` keys, even for an empty stream; `groupingBy` only creates keys that occur.

- The `maxBy` downstream yields `Optional<Employee>` per group; `collectingAndThen` unwraps it.

How a collector works (for writing your own)

A `Collector<T, A, R>` bundles four functions — the same shape as the `Collector.of` call above: a **supplier** creates an empty container `A`, an **accumulator** adds one `T`, a **combiner** merges two containers (used by parallel streams), and a **finisher** turns `A` into the result `R`. Optional **characteristics** (`IDENTITY_FINISH`, `UNORDERED`, `CONCURRENT`) let the library optimise. Write one when the same aggregation appears in many places.

Common mistakes

- `toMap` without a merge function on keys that can repeat → `IllegalStateException` in production data.
- `toMap` when a value can be `null` → NPE (use `groupingBy` / `collect(HashMap::new, (m, e) -> m.put(...), Map::putAll)` if nulls are legitimate).
- Expecting `groupingBy` to preserve insertion or sorted order without a map factory.
- Nesting four collectors deep — at that point, a named helper method or a small loop is clearer.

In the interview

> **30-second answer:** "collect performs a mutable reduction via a Collector: supplier, accumulator, combiner, finisher. The workhorses are toList, toMap with a merge function, joining, and groupingBy with a downstream collector — counting, summing, mapping, maxBy — which is basically SQL GROUP BY. partitioningBy is a boolean groupingBy that always has both keys, and teeing runs two collectors in one pass."

- *Follow-up: How do you count word frequencies with streams?* → `words.stream().collect(groupingBy(Function.identity(), counting()))`.
- *Follow-up: What happens on duplicate keys in `toMap`?* → `IllegalStateException` unless you pass a merge function.

Self-check

1. Collector for "names per department, sorted by department"? → `groupingBy(Employee::dept, TreeMap::new, mapping(Employee::name, toList()))`.
2. `partitioningBy` on an empty stream returns? → `{false=[], true=[]}`.
3. What's `teeing` for? → Computing two aggregates in one pass and merging them (e.g. min and max, or count and sum).

TL;DR

`collect` + `Collectors`: `toList` / `toSet` / `toCollection`, `toMap` (+ merge fn!), `joining`, `groupingBy` + downstream (`counting`, `summingX`, `averagingX`, `mapping`, `filtering`, `flatMapping`, `maxBy`), `partitioningBy`, `summarizingX`, `teeing`, `collectingAndThen`. Custom: `Collector.of(supplier, accumulator, combiner, finisher)`.

12.6 Primitive streams

What it is

`IntStream`, `LongStream` and `DoubleStream` hold **primitive** values (plain `int` / `long` / `double`, not objects) directly, avoiding the **boxing** (wrapping each number in an `Integer` / `Long` object on the heap) a `Stream<Integer>` performs on every element. They add numeric operations: `sum()`, `average()`, `summaryStatistics()`, `range` / `rangeClosed` (int and long only), and they return `OptionalInt` / `OptionalLong` / `OptionalDouble` for possibly-empty results.

From → To	Method
`Stream<T>` → `IntStream`	`mapToInt(ToIntFunction)` (also `mapToLong`, `mapToDouble`)
`IntStream` → `Stream<Integer>`	`boxed()`
`IntStream` → `Stream<R>`	`mapToObj(IntFunction)`
`IntStream` → `LongStream` / `DoubleStream`	`asLongStream()`, `asDoubleStream()`, `mapToLong`, `mapToDouble`

Operation on empty stream	Result
`sum()`	`0`
`average()`	`OptionalDouble.empty` (average is always `OptionalDouble`, even for `IntStream`)
`min()` / `max()`	`OptionalInt.empty` etc.
`count()`	`0`

Code

```
// code/ch12/primitive/PrimitiveStreams.java (key parts)
int[] scores = {72, 95, 88, 61, 79};
IntStream.of(scores).sum(); IntStream.of(scores).max().getAsInt(); Arrays.stream(scores).average();
IntStream.of(scores).summaryStatistics();
words.stream().mapToInt(String::length).sum();
IntStream.rangeClosed(1, 5).boxed().toList();
IntStream.range(0, names.size()).mapToObj(i -> i + ":" + names.get(i));      // index-based loop
text.toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();
LongStream.rangeClosed(1, 20).reduce(1, (a, b) -> a * b);                    // 20!
new Random(7).ints(5, 1, 7).boxed().toList();                                // 5 dice rolls, seeded

long a = IntStream.range(0, n).asLongStream().sum();                         // primitive
long b = Stream.iterate(0L, i -> i + 1).limit(n).reduce(0L, Long::sum);      // boxed
```

Output:

```
sum=395 max=95 average=79.0
empty average -> OptionalDouble.empty, empty sum -> 0
stats IntSummaryStatistics{count=5, sum=395, min=61, average=79.000000, max=95}
totalChars=12 boxed=[1, 2, 3, 4, 5] labels=[row0, row1, row2] halves=[0.5, 1.0, 1.5]
indexed: 0:x 1:y 2:z
vowels in 'Hello World' = 3
20! = 2432902008176640000 (21! overflows long)
dice: [5, 3, 4, 5, 5]
sum 0..10M: LongStream 6 ms vs Stream<Long> 76 ms (same result: true)
```

The last line is a single rough timing (Apple M4 Pro, JDK 26 — **machine-dependent**): the boxed version allocates up to ten million `Long` objects and is roughly an order of magnitude slower here. For numeric work, stay primitive until you need objects.

`sum()` wraps silently on **overflow** (a result too big for the type wraps around to a wrong, often negative, number) like `+` does (§2 integer overflow); use `LongStream` or `mapToObj(BigInteger::valueOf)` for large totals.

Self-check

1. Return type of `IntStream.average()`? → `OptionalDouble`.
2. How do you turn `IntStream` into `List<Integer>`? → `.boxed().toList()`.
3. Why prefer `mapToInt(...).sum()` over `map(...).reduce(0, Integer::sum)`? → No boxing, and it reads better.

TL;DR

Use `IntStream` / `LongStream` / `DoubleStream` for numbers: `sum`, `average` (→ `OptionalDouble`), `summaryStatistics`, `range`. Convert with `mapToInt` / `boxed` / `mapToObj`. Avoid boxing in **hot paths** (code that runs extremely often, so small costs add up).

12.7 Streams, `Optional` and nulls

Streams and `Optional` (§11.7) are designed to work together:

- `findFirst`, `findAny`, `min`, `max` and single-argument `reduce` return `Optional` because the stream might be empty.
- `optional.stream()` (Java 9) turns an `Optional` into a 0- or 1-element stream, so `flatMap(Optional::stream)` drops the empties from a `Stream<Optional<T>>` — used in the sales report below to skip unparseable lines.
- `Stream.ofNullable(x)` (Java 9) turns a possibly-`null` value into a 0- or 1-element stream.
- Streams tolerate `null` elements, but many operations don't (`toMap` values, `Collectors.toUnmodifiableList`, `max` on a `null`, method references like `String::length`). Filter them out early: `.filter(Objects::nonNull)`.

12.8 Parallel streams

What it is

Calling `.parallel()` on a stream (or `collection.parallelStream()`) lets the library split the source into chunks, process chunks on several **threads** (independent flows of execution that can run at the same time on different cores), and merge the partial results. You change one word; correctness and performance are your responsibility.

How it works

1. The source's `Spliterator` splits its range recursively (like halving an array) into chunks.
2. Chunks run as tasks on the **common** `ForkJoinPool` (a built-in **thread pool** — a fixed set of reusable worker threads — designed for splitting work and joining results; shared by the whole JVM, the Java Virtual Machine that runs your program; by default its parallelism is the number of cores minus one, and the calling thread also helps).
3. Partial results are merged with the reduction's **combiner** (or the collector's combiner).
4. For ordered streams, operations like `findFirst`, `limit`, `forEachOrdered` and `sorted` must respect **encounter order**, which costs coordination; `unordered()`, `findAny` and `forEach` relax it.

When it helps — measured

`ParallelDemo` runs each case twice and times the second run. **Timings are machine-dependent** (this run: 14-core Apple M4 Pro, JDK 26); some lines, such as the `forEach` order and the wrong `reduce` result, vary between runs by design.

```
// code/ch12/parallel/ParallelDemo.java (key parts)
LongStream.rangeClosed(1, 3_000_000).filter(ParallelDemo::isPrime).count();              // heavy per element
LongStream.rangeClosed(1, 3_000_000).parallel().filter(ParallelDemo::isPrime).count();
IntStream.range(0, 1_000).parallel().map(i -> i * 2).sum();                              // tiny work
Stream.iterate(1L, i -> i + 1).limit(2_000_000).parallel().reduce(0L, Long::sum);       // unsplittable source

IntStream.range(0, 10_000).parallel().boxed().toList();                                  // safe: collector merges
IntStream.range(0, 10_000).parallel().boxed()
        .collect(Collectors.groupingByConcurrent(i -> i % 3, ConcurrentHashMap::new, Collectors.counting()));
IntStream.rangeClosed(1, 100).parallel().reduce(5, Integer::sum);                        // WRONG identity
```

Output (one run):

```
cores available: 14 (common pool parallelism 13)
count primes <= 3M      sequential  227.2 ms | parallel   33.5 ms
sum 1k doubled          sequential   0.07 ms | parallel   0.29 ms
iterate+limit 2M sum    sequential   13.7 ms | parallel   29.2 ms
safe toList size=10000, in encounter order? true, groupingByConcurrent={0=3334, 1=3333, 2=3333}
reduce identity 0 -> 5050, identity 5 -> 5550 (5 added once per chunk; the exact value depends on how the work was split)
forEach (parallel, order not guaranteed):   4 3 5 7 2 1 6 0
forEachOrdered (parallel, encounter order): 0 1 2 3 4 5 6 7
```

- **Prime counting** (CPU-heavy — limited by processor speed rather than by waiting on disk or network — independent elements, perfectly splittable range): ~7× faster on 14 cores.
- **1,000 tiny operations**: splitting and thread hand-offs cost more than the work — parallel is slower. A common rule of thumb (Brian Goetz's "NQ model"): parallelism pays when N (elements) × Q (work per element) is large; for trivial per-element work you need well over ten thousand elements.
- **`Stream.iterate` + `limit`**: the source can only produce elements one after another and `limit` must respect order, so parallel is *slower*. Sources split well when they know their size and can be cut anywhere: arrays, `ArrayList`, `IntStream.range`. They split badly: `LinkedList`, `Stream.iterate`, `BufferedReader.lines()`, most `Iterator`-based sources.
- **Wrong identity**: `reduce(5, Integer::sum)` added 5 once per chunk (in this run the extra 500 means the 100 numbers were split into 100 chunks). Sequentially it would be 5055. Both are wrong — 5 was never an identity.

Correctness rules

1. **No shared mutable state** (data that several threads can change) in lambdas. Adding to an `ArrayList` from `forEach` in parallel loses elements or throws `ArrayIndexOutOfBoundsException`; let `toList()`/`collect` do the merging.
2. **Reduction functions must be associative, stateless, with a true identity** (§12.4).
3. **Don't block** (make a thread sit idle waiting — on I/O, i.e. disk/network input-output, on **locks** that guard shared data, or on `Thread.sleep`) inside parallel streams — you tie up the JVM-wide common pool, starving every other parallel stream and `CompletableFuture` using it. For concurrent I/O use **virtual threads** (very cheap threads managed by the JVM rather than the operating system, Chapter 15) or the `mapConcurrent` gatherer (§12.9).
4. **Order costs**: if you don't need order, `unordered()`, `findAny()` and `forEach` are cheaper; if you do, `forEachOrdered`.

When not to use it

Most of the time. *Effective Java* Item 48: "use caution when making streams parallel" — measure first with a proper **benchmark** (a controlled performance measurement; JMH, the Java Microbenchmark Harness, is the standard tool, Chapter 16). Avoid it in web servers (requests already run in parallel), for small inputs, poorly splittable sources, **I/O-bound** work (time spent mostly waiting on disk or network), or pipelines with `limit`/`sorted`/`findFirst` on ordered data.

In the interview

30-second answer: "Parallel streams split the source with a Spliterator and run chunks on the common ForkJoinPool, combining partial results. They help only for large, CPU-bound, independent work on splittable sources like arrays or ranges. They hurt for small inputs, LinkedList or iterate sources, ordered operations like limit, and anything blocking — which also starves the shared pool. Lambdas must be stateless and reductions associative with a real identity. I measure before switching."

- *Follow-up: Which thread pool do parallel streams use?* → The common `ForkJoinPool` (plus the caller thread).
- *Follow-up: Why might `parallel().forEach(list::add)` fail?* → `ArrayList` isn't thread-safe; concurrent adds lose data or throw. Use `collect`/`toList`.

Self-check

1. Name two sources that split well and two that don't. → Well: arrays, `ArrayList`, `IntStream.range`. Badly: `LinkedList`, `Stream.iterate`, `Files.lines`.
2. Why is `findAny` preferred over `findFirst` in parallel? → It doesn't have to respect encounter order.
3. Is `reduce(0, (a, b) -> a - b)` safe in parallel? → No — subtraction isn't associative.

TL;DR

Parallel = split + common ForkJoinPool + combine. Worth it only for big, CPU-heavy, splittable, stateless work — measure. Never share mutable state, never block, and respect reduction rules.

12.9 Stream gatherers (Java 24)

The problem it solves

The built-in intermediate operations are a fixed set. Operations that need **state across elements** — "sliding windows of 3", "running totals", "distinct by a key", "emit groups of consecutive equal items" — were awkward: you either broke out of the stream into a loop or wrote a custom `Spliterator`. Collectors could keep state but only at the *end* of a pipeline.

What it is

JEP 485 (a JEP, *JDK Enhancement Proposal*, is the numbered design document for a Java feature; this one was **previewed** — shipped switched-off-by-default for feedback — in Java 22 and 23, **final in Java 24**, so available in Java 25 **LTS**, a *long-term support* release that gets years of updates) adds `Stream.gather(Gatherer)`: a way to plug **custom intermediate operations** into a pipeline. A `Gatherer<T, A, R>` consumes elements of type `T`, keeps private state `A`, and pushes zero or more `R` elements **downstream** (on to the next stage of the pipeline). It can be one-to-one, one-to-many, many-to-one or many-to-many, and it can stop the stream early. Think of it as "a collector, but in the middle of the pipeline".

A gatherer has up to four functions (the same idea as a collector's):

Part	Role
initializer	creates the private state (e.g. a `HashSet` of seen keys)
integrator	called per element with (state, element, downstream); may `push` results; returns `false` to **short-circuit** the stream
combiner	merges states when running in parallel (optional)
finisher	called after the last element; may push final results (e.g. a last partial window)

Factory methods: `Gatherer.ofSequential(...)` (no combiner, sequential only) and `Gatherer.of(...)` (parallel-capable). `Gatherer.Integrator.ofGreedy(...)` marks an integrator that never short-circuits, allowing optimisations. Gatherers compose with `a.andThen(b)`.

Built-in gatherers (`java.util.stream.Gatherers`)

Gatherer	Does	Example result
`windowFixed(n)`	non-overlapping lists of n (last may be shorter)	batches of 3
`windowSliding(n)`	overlapping windows, each shifted by one	moving averages (average of the last n values, recomputed at each step)
`scan(initial, f)`	running accumulation, emits every intermediate value	running totals
`fold(initial, f)`	like reduce, but as an intermediate op emitting one final value	single aggregate
`mapConcurrent(maxConcurrency, f)`	applies `f` using **virtual threads**, at most n at once, preserving encounter order	parallel I/O calls

Code

```
// code/ch12/gatherers/GatherersDemo.java (key parts)
temps.stream().gather(Gatherers.windowFixed(3)).toList();
temps.stream().gather(Gatherers.windowSliding(3))
     .map(w -> w.stream().mapToInt(Integer::intValue).average().orElseThrow()).toList();   // moving average
Stream.of(5, 10, 20).gather(Gatherers.scan(() -> 0, Integer::sum)).toList();
Stream.of("a", "b", "c").gather(Gatherers.fold(() -> "", (acc, s) -> acc + s)).toList();
Stream.of("u1", "u2", "u3", "u4")
      .gather(Gatherers.mapConcurrent(2, url -> { sleep(20); return url.toUpperCase(); })).toList();

static <T, K> Gatherer<T, ?, T> distinctBy(Function<? super T, ? extends K> key) {
    return Gatherer.ofSequential(
            HashSet<K>::new,                                          // initializer
            Gatherer.Integrator.ofGreedy((seen, element, downstream) -> {  // integrator
                if (seen.add(key.apply(element))) return downstream.push(element);
                return true;
            }));
}

static Gatherer<Integer, ?, Integer> takeWithinBudget(int budget) {
    return Gatherer.ofSequential(() -> new int[1], (spent, cost, downstream) -> {
        if (spent[0] + cost > budget) return false;                   // stop the whole stream
        spent[0] += cost;
        return downstream.push(cost);
    });
}

static <T> Gatherer<T, ?, List<T>> runs() {                           // with a finisher
    class State { List<T> current = new ArrayList<>(); }
    return Gatherer.ofSequential(State::new,
            Gatherer.Integrator.ofGreedy((st, e, down) -> {
                if (!st.current.isEmpty() && !st.current.getLast().equals(e)) { down.push(st.current); st.current
= new ArrayList<>(); }
                st.current.add(e);
                return true;
            }),
            (st, down) -> { if (!st.current.isEmpty()) down.push(st.current); });   // flush the last run
}
```

Output:

```
windowFixed(3)    [[21, 23, 22], [25, 27, 26], [24]]
windowSliding(3)  [[21, 23, 22], [23, 22, 25], [22, 25, 27], [25, 27, 26], [27, 26, 24]]
3-day moving avg  [22.0, 23.3, 24.7, 26.0, 25.7]
scan (running sum)[5, 15, 35]
fold (one result) [abc]
mapConcurrent(2)  [U1, U2, U3, U4]
distinctBy user   [/home, /cart, /home]
within budget 50  [10, 15, 20, 5]
runs              [[a, a], [b], [c, c, c], [a]]
andThen           [[1, 2], [3, 4], [5]]
```

Notes:

- `downstream.push(x)` returns `false` when downstream no longer wants elements (e.g. a later `limit` is satisfied); returning that value propagates the short-circuit.
- `takeWithinBudget` stopped at 30 even though a later small element might fit — that's the difference from `filter`, the same as `takeWhile`.
- `runs()` needs the finisher: without it the final `[a]` group would be lost.
- `mapConcurrent` is the **idiomatic** (natural, conventional) way to call a slow service for each element with bounded **concurrency** (a cap on how many calls are in flight at once) — a job parallel streams do badly (§12.8).

When not to use it

If a built-in operation or a collector already does the job (`distinct`, `groupingBy`), use that. Gatherers are for stateful *mid-pipeline* logic. Keep state private to the gatherer — the initializer creates a fresh state object for each evaluation of the stream.

Self-check

1. Which gatherer computes a running total? → `Gatherers.scan`.
2. How does an integrator stop the stream? → It returns `false`.
3. What's the finisher for? → Emitting results that depend on having seen all elements, like the final partial window or run.

TL;DR

`stream.gather(g)` (Java 24) = custom intermediate operations with private state. Built-ins: `windowFixed`, `windowSliding`, `scan`, `fold`, `mapConcurrent`. Write your own with `Gatherer.ofSequential(initializer, integrator[, finisher])`; return `false` to short-circuit.

12.10 Common mistakes

```java
// code/ch12/mistakes/StreamMistakes.java (key parts)
Stream.of(arr).count();                                  // int[] -> Stream<int[]> with ONE element
Stream.of("a", "b").map(String::toUpperCase).forEach(out::add);   // side effect: use toList()
src.stream().forEach(x -> { if (x == 2) src.add(99); });  // modifying the source
Stream.of("apple", "avocado").collect(Collectors.toMap(s -> s.charAt(0), s -> s));   // duplicate key
Stream.of("x").collect(Collectors.toMap(s -> s, s -> (String) null));                // null value
Stream.of(1).toList().add(2);                            // unmodifiable
List.of("a", "b", "c").stream().peek(log::add).count();   // peek skipped!
```

Output:

1. Stream.of(int[]).count() = 1 vs Arrays.stream(arr).count() = 3
2. forEach(out::add) [A, B] -> prefer toList() [A, B]
3. modifying source -> ConcurrentModificationException
4. toMap duplicate -> IllegalStateException
5. toMap null value -> NullPointerException
6. Stream.toList().add -> UnsupportedOperationException
7. min() = 2 (same answer as sorted().findFirst(), less work)
8. [3, 2, 1] (limit first)
10. count()=3 but peek ran 0 times -- never put logic in peek
12. loop=2 stream=2 (both fine -- choose the clearer one)

#	Mistake	Fix
1	`Stream.of(intArray)`	`Arrays.stream(intArray)` or `IntStream.of(...)`
2	Side effects in `forEach` / `map` to build results (*Effective Java* Item 46: use side-effect-free functions)	`toList()`, `collect(...)`
3	Modifying the source during the pipeline (→ `ConcurrentModificationException`, thrown when a collection changes while being iterated)	Collect first, then modify; or `removeIf`
4	`toMap` on non-unique keys	pass a merge function or use `groupingBy`
5	`toMap` with `null` values	filter nulls or collect manually
6	Mutating `toList()` results	`toCollection(ArrayList::new)`
7	`sorted().findFirst()` for a minimum	`min(comparator)` — O(n), no buffering
8	Stateful op (`sorted`, `distinct`) before `limit` on an infinite stream	`limit` first, or a bounded source
9	`Stream.generate(() -> 1).distinct().limit(2)`	hangs: only one distinct value exists
10	Logic in `peek`	Since Java 9, `count()` may compute the size without running the pipeline — `peek` never ran. Use `peek` only for debugging
11	Checked exceptions in lambdas	wrap (§11.8) or use a loop
12	Streams everywhere	a loop with `break`, several accumulators or complex control flow is often clearer (*Effective Java* Item 45: use streams judiciously)
13	Returning `Stream` from APIs that callers will iterate twice	return a `Collection` (EJ Item 47) — a stream can be traversed only once
14	Not closing I/O-backed streams	`try (Stream<String> lines = Files.lines(p)) { … }` — `BaseStream` is `AutoCloseable` (usable in try-with-resources, which closes it automatically)
15	Reusing a stream variable	create a new stream each time, or keep a `Supplier<Stream<T>>`

12.11 Streams vs loops, and a real-world pipeline

Choosing

Prefer a stream when…	Prefer a loop when…
The work is a clear sequence of transform / filter / aggregate steps	You need `break` / `continue` / `return` from the middle in complex ways
You'd otherwise write grouping or counting boilerplate	You update several variables per iteration
Readers benefit from named steps (`groupingBy`, `partitioningBy`)	You need checked exceptions to propagate naturally
You may later want `parallel()` for heavy CPU work	You're in a very hot loop over primitives and need every nanosecond (measure)
Data is already in a collection	You need the index and neighbours constantly (or use a gatherer)

Performance: for typical business code, a sequential stream costs about the same as a loop — the JIT (the JVM's just-in-time compiler, which turns hot bytecode into fast machine code at run time) optimises both. Overhead shows up with tiny collections in hot paths or boxing; readability usually matters more.

A realistic pipeline: a sales report

```
// code/ch12/realworld/SalesReport.java (key parts)
record Sale(LocalDate date, String region, String product, int units, double unitPrice) {
    double revenue() { return units * unitPrice; }
    static Optional<Sale> parse(String line) { ... }        // Optional.empty() for malformed lines
}
List<Sale> sales = csv.lines().map(Sale::parse).flatMap(Optional::stream).toList();

sales.stream().collect(groupingBy(Sale::region, TreeMap::new, summingDouble(Sale::revenue)));
sales.stream().collect(groupingBy(s -> s.date().getMonthValue(), TreeMap::new, summingLong(Sale::units)));
sales.stream().collect(groupingBy(Sale::product, summingDouble(Sale::revenue)))
     .entrySet().stream()
     .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
     .limit(2)
     .map(e -> e.getKey() + "=" + String.format("%.2f", e.getValue()))
     .toList();
sales.stream().collect(partitioningBy(s -> s.revenue() >= 500, counting()));
```

Output:

```
parsed 6 of 8 lines
  APAC    1050.00
  EU      1445.40
  US       925.00
units by month {1=19, 2=30}
top-2 products by revenue [monitor=2225.00, keyboard=707.90]
orders >= 500: 2, smaller: 4
```

The input is **CSV** (comma-separated values: one record per line, fields split by commas). The pipeline reads like the requirements: parse, drop bad lines, total by region, total by month, top-2 products, count big orders. Two malformed lines (`bad line` and a non-numeric unit count) were skipped by `parse` returning `Optional.empty()` and `flatMap(Optional::stream)`. In production you'd also log or count rejects.

Real-world uses: report generation and **ETL**-style transforms (Extract, Transform, Load: pulling data from one place, reshaping it, and storing it elsewhere) in back-end services, request validation (`allMatch`), building lookup maps from query results (`toMap`), batch processing of log lines (`Files.lines` + `filter` + `groupingBy`), and — with Spring Data or JPA (the Java Persistence API, the

standard way of mapping Java objects to database tables) — processing result streams inside a **transaction** (a group of database operations that succeed or fail as one unit).

In the interview

> **30-second answer:** "A stream is a lazy, single-use pipeline: source, intermediate operations, terminal operation. Nothing runs until the terminal op, elements flow one at a time, and short-circuiting ops can stop early, so infinite streams work with limit. I use filter/map/flatMap, then collect with toList, toMap with a merge function, or groupingBy with a downstream collector. I keep lambdas side-effect free, use primitive streams for numbers, and only go parallel for large CPU-bound work after measuring. Since Java 24, gatherers let me write custom stateful intermediate ops like sliding windows."

- *Follow-up: Difference between `map` and `flatMap`?* → 1→1 vs 1→many (flattening nested streams).
- *Follow-up: Intermediate vs terminal?* → Intermediate returns a new lazy `Stream`; terminal produces a result or side effect and consumes the stream.
- *Follow-up: Are streams faster than loops?* → Not inherently; similar for typical code, sometimes slower for tiny inputs. Choose for clarity; parallel can win on big CPU-bound work.
- *Follow-up: What's a stateful intermediate operation?* → One that needs to see other elements — `sorted`, `distinct`, `limit`, `skip` — and may buffer.

Chapter TL;DR

Topic	Remember
Pipeline	source → lazy intermediate ops → one terminal op; single use; never modifies the source
Laziness	nothing runs without a terminal op; vertical, fused processing; short-circuit ops stop early; `sorted` / `distinct` buffer
Intermediate	`filter`, `map`, `flatMap` (1→many), `mapMulti` (16), `distinct`, `sorted`, `limit`, `skip`, `takeWhile` / `dropWhile` (9), `peek` (debug only)
Terminal	`toList` (16, unmodifiable), `collect`, `count`, `min` / `max`, `findFirst` / `findAny`, `any/all/noneMatch` (empty → false/true/true), `reduce`
`reduce`	real identity + associative + stateless; 3-arg form with combiner for type change
Collectors	`toMap` (merge fn!), `joining`, `groupingBy` + downstream, `partitioningBy` (both keys), `teeing` (12), `filtering` / `flatMapping` (9), `Collector.of`
Primitive	`IntStream` / `LongStream` / `DoubleStream`; `sum`, `average` → `OptionalDouble`, `summaryStatistics`, `boxed`, `mapToObj`
Parallel	common ForkJoinPool; big CPU-bound splittable work only; no shared state, no blocking; measure
Gatherers (24)	`gather(...)`: `windowFixed`, `windowSliding`, `scan`, `fold`, `mapConcurrent`; custom via `Gatherer.ofSequential`
Style	side-effect-free lambdas; filter early; loops when clearer; return collections, not streams

Where next: stream file contents with `Files.lines` and `Files.walk` in Chapter 13 — I/O and NIO; `CompletableFuture` and virtual threads for concurrent work in Chapter 15; gatherers and other recent features in context in Chapter 17 — Modern Java. Exam traps: OCP Chapter 10. Python comparison (generators, comprehensions, `itertools`): Python Chapter 11.

Chapter 13 — I/O and NIO

Almost every real program reads or writes something outside itself: configuration files, CSV exports (CSV = comma-separated values, a plain-text table with one row per line), logs, uploaded images, the keyboard, the terminal. **I/O** (input/output) is how a program moves data across that boundary. Java has two generations of file APIs that you will meet side by side in real code: the original `java.io` package (Java 1.0, built around *streams* of bytes and characters) and **NIO.2** (`java.nio.file`, Java 7, built around `Path` and the `Files` utility class). This chapter teaches both, shows which one to reach for, and covers the topics that trip people up in production: buffering, character encodings (the rules for turning text into bytes), serialization (turning objects into bytes so they can be saved or sent), and configuration files.

You already know the tools this chapter leans on: try-with-resources (the `try (Resource r = …) { }` form that closes `r` automatically, even after an exception) and `IOException` from Chapter 8, and lazy streams from Chapter 12 (`Files.lines` and `Files.walk` return `Stream`s).

What you'll learn

§	Topic	You'll be able to…
13.1	The big picture	tell `java.io` and NIO.2 apart and pick the right API
13.2	`Path`	build, combine, normalise and compare file paths without touching the disk
13.3	`Files`: managing files and directories	create, copy, move, delete, inspect, list and walk directory trees safely
13.4	Reading and writing whole files	use the `Files` one-liners and stream large files line by line
13.5	Byte streams and character streams	understand `InputStream`/`OutputStream`/`Reader`/`Writer` and the decorator chain
13.6	Buffering	know why unbuffered I/O is slow and why forgetting `flush` loses data
13.7	Charsets and encodings	avoid mojibake (garbled text, defined in §13.7), and know what changed with UTF-8 by default (Java 18)
13.8	Console and standard streams	read user input with `Scanner`, `BufferedReader`, `Console` and `IO`
13.9	Binary data, channels and buffers	read/write primitives, use `FileChannel` and `ByteBuffer`, know what `WatchService` is for
13.10	Serialization (and why to avoid it)	explain how it works, `transient`, `serialVersionUID`, filters, and its security risk
13.11	Properties files	load, read and store `key=value` configuration
13.12	Working with JSON	know that JSON needs a library, and what the code looks like
13.13	Best practices and common mistakes	a checklist for correct, portable I/O

All runnable code is in `code/` (package `ch13.*`), compiled with `javac --release 25` (the Java compiler, targeting Java 25) and run on JDK 26 (the Java Development Kit). Every program works inside a directory made by `Files.createTempDirectory` and deletes it at the end, so nothing is left on your disk. Outputs are pasted from real runs.

> **Exam depth.** `File` vs `Path` method-name differences, `resolve`/`relativize` puzzles, `Files` exception rules, stream class tables and `Console` details: OCP Chapter 14 — I/O. This chapter stays at "use it correctly at work" depth and links there for traps.

13.1 The big picture: `java.io` vs NIO.2

What it is

- `java.io` (Java 1.0) models I/O as **streams**: a one-way flow of data you read from (an *input stream*) or write to (an *output stream*), one piece after another, like water through a pipe. Files are represented by the old `java.io.File` class. (These I/O streams are unrelated to the `java.util.stream.Stream` of Chapter 12 — same word, different idea.)
- **NIO** ("New I/O", Java 1.4, `java.nio`) added **buffers** (`ByteBuffer`) and **channels** (`FileChannel`, `SocketChannel`) — lower-level, block-oriented (moving data in chunks rather than one byte at a time), and capable of **non-blocking** network I/O (a read or write returns immediately even if no data is ready, instead of making the thread wait).
- **NIO.2** (Java 7, `java.nio.file`) replaced `File` with the `Path` interface and the `Files` class full of static helpers, plus file attributes, **symbolic links** (symlinks: small files that just point to another file or directory, like a shortcut), directory walking and file-change watching.

Analogy: `java.io` is the plumbing — pipes you connect together. NIO.2 is the building manager's office — it knows where every room (file) is, can create, move or inspect rooms, and hands you a pipe when you need one.

Why it matters

Both APIs are current and interoperate. Modern code uses `Path` + `Files` for everything about *files*, and still uses `java.io` stream classes whenever data flows from something that isn't a file (network **sockets** — the OS endpoint for one network connection — HTTP bodies, the data part of a web request or response, zip entries, in-memory arrays) — `Files.newInputStream(path)` even *returns* a `java.io.InputStream`.

Need	Use	Since
Represent a file/directory location	`Path` (`Path.of(...)`)	7 (`Path.of` 11)
Create, copy, move, delete, check, list	`Files.*` static methods	7
Read/write a small text file in one call	`Files.readString` / `Files.writeString`	11
Read a huge text file line by line	`Files.lines` (lazy `Stream`) or `Files.newBufferedReader`	8 / 7
Process bytes from any source (socket, zip, HTTP)	`InputStream` / `OutputStream`	1.0
Process text from any source	`Reader` / `Writer` (+ a charset)	1.1
Random access (jumping to any position), big files, memory mapping	`FileChannel` + `ByteBuffer`	1.4
Legacy (older, still-in-use) APIs that want a `File`	`path.toFile()` / `file.toPath()`	7

Why `File` was replaced

The old `File` class has real flaws that NIO.2 fixes:

1. **Silent failures.** `file.delete()`, `mkdir()` and `renameTo()` return `false` without saying *why* (missing? permission? not empty?). `Files.delete` throws a specific exception (`NoSuchFileException`, `DirectoryNotEmptyException`, `AccessDeniedException`).
2. **No symbolic-link support, poor metadata** (data *about* a file — size, dates, owner — rather than its contents). NIO.2 reads attributes (size, times, POSIX permissions — the Unix read/write/execute flags for owner, group and others — owner) in one call.
3. **No efficient directory walking.** `File.listFiles()` loads everything into an array; `Files.walk`/`Files.list` stream lazily.

TL;DR

Use `Path` + `Files` for files and directories; use `InputStream`/`Reader` families for data flowing from anywhere; convert with `toFile()`/`toPath()` only at legacy boundaries.

13.2 `Path`: naming files and directories

What it is

A `Path` is an **immutable** (can't be changed after creation) object that represents a location in a file system as a sequence of **name elements** (`reports`, `2025`, `q3`, `sales.csv`) plus an optional **root** (`/` on Unix — the family of operating systems that includes Linux and macOS — `C:\` on Windows). Crucially, a `Path` is *just a name*: creating one does not check that the file exists or touch the disk at all. Think of it as a street address written on paper — you can write down an address for a house that hasn't been built.

Key terms:

- **Absolute path** — starts at a root (`/srv/app/logs`). **Relative path** — interpreted from the **current working directory** (the folder the program was started from) of the JVM **process** (a running program with its own memory, managed by the OS) (`reports/q3.csv`).
- **Separator** — `/` on Unix/macOS, `\` on Windows. `Path.of("a", "b")` inserts the right one, so don't concatenate strings with `"/"` yourself.
- `.` means "this directory", `..` means "the parent directory".

How it works

1. Create a path: `Path.of("reports", "2025", "sales.csv")` (Java 11) or the older, identical `Paths.get(...)` (Java 7).
2. Take it apart: `getFileName()`, `getParent()`, `getRoot()`, `getNameCount()`, `getName(i)`, `subpath(from, to)`.
3. Combine paths: `resolve` (append), `resolveSibling` (replace the last element), `relativize` (the route from one path to another).
4. Clean it: `normalize()` removes `.` and resolves `name/..` pairs *textually*; `toAbsolutePath()` prefixes the working directory; `toRealPath()` asks the file system (resolves symlinks and **requires the file to exist**).

`code/ch13/paths/PathBasics.java` (key parts):

```
Path p = Path.of("reports", "2025", "q3", "sales.csv");
p.getFileName(); p.getParent(); p.getRoot(); p.getNameCount(); p.getName(1); p.subpath(1, 3);

Path base = Path.of("/srv/app");
base.resolve("logs/app.log");                // append a relative path
base.resolve("/etc/hosts");                  // an absolute argument simply wins
base.resolve("config.yml").resolveSibling("secrets.yml");

Path.of("/srv/app/./logs/../data/./users.db").normalize();
Path from = Path.of("/srv/app/logs"), to = Path.of("/srv/app/data/users.db");
from.relativize(to);                         // how do I get from 'from' to 'to'?
to.startsWith("/srv/ap");                    // compares whole NAME elements, not characters
```

Output:

```
path          = reports/2025/q3/sales.csv
fileName      = sales.csv
parent        = reports/2025/q3
root          = null
nameCount     = 4
name(1)       = 2025
subpath(1,3)  = 2025/q3
isAbsolute    = false
resolve       = /srv/app/logs/app.log
resolve abs   = /etc/hosts
resolveSibling= /srv/app/secrets.yml
normalize     = /srv/app/data/users.db
relativize    = ../data/users.db
round trip    = /srv/app/data/users.db
startsWith    = true / string-wise 'app' = false
endsWith      = true
File <-> Path = reports/2025/q3/sales.csv -> true
Paths.get     = true
```

What to notice:

- `getRoot()` is `null` for a relative path, and `getName(0)` is the first element *after* the root.
- `subpath(1, 3)` is **half-open** (includes the start index, excludes the end) like `String.substring`: elements 1 and 2.
- `resolve` with an absolute argument returns the argument — a classic source of bugs when a user-supplied "file name" starts with `/`.
- `relativize` requires both paths to be of the same kind (both absolute or both relative), otherwise it throws `IllegalArgumentException`.
- `startsWith("/srv/ap")` is `false`: comparison is by name elements, not by characters.

Common mistakes

- Building paths by string concatenation (`dir + "/" + name`) — breaks on Windows and doubles separators. Use `resolve`.
- Assuming `normalize()` checks the disk. It is purely textual; `a/link/..` normalises to `a` even if `link` is a symlink pointing elsewhere. Use `toRealPath()` when it matters.
- **Path traversal**: `uploads.resolve(userInput)` where `userInput` is `../../etc/passwd`. Guard with `Path target = uploads.resolve(name).normalize(); if (!target.startsWith(uploads)) throw ...` — this is exactly the check web servers do. (Path traversal is an attack where `..` segments in input escape the folder you meant to allow.)

Self-check

1. Does `Path.of("missing.txt")` throw if the file doesn't exist? → No; a `Path` is only a name.
2. `Path.of("/a/b").resolve("/c")`? → `/c`.
3. `Path.of("/a/b").relativize(Path.of("/a/c/d"))`? → `../c/d`.

13.3 `Files`: managing files and directories

What it is

`java.nio.file.Files` is a class of static methods that *do* touch the disk: test, create, copy, move, delete, read attributes, list and walk. Every method that can fail throws a specific subclass of `IOException` (a *checked* exception, so the compiler makes you handle it — Chapter 8 §8.3).

How it works — the core operations

Task	Method	Fails with (if…)
Does it exist?	`exists`, `notExists`, `isRegularFile`, `isDirectory`, `isReadable`, `isSymbolicLink`	never throws; returns `false` if unsure
Make a directory	`createDirectory(p)` / `createDirectories(p)`	`FileAlreadyExistsException` / parents made too, no error if present
Make an empty file	`createFile(p)`	`FileAlreadyExistsException`
Temporary file/dir	`createTempFile(prefix, suffix)`, `createTempDirectory(prefix)`	in `java.io.tmpdir` (the system's temp folder)
Copy	`copy(src, dst, options...)`	`FileAlreadyExistsException` unless `REPLACE_EXISTING`
Move / rename	`move(src, dst, options...)`	same; add `ATOMIC_MOVE` for all-or-nothing (**atomic**: other programs see either the old state or the new one, never a half-done step) within one file system
Delete	`delete(p)` / `deleteIfExists(p)`	`NoSuchFileException`, `DirectoryNotEmptyException` / returns `boolean`
Size, times, type	`size`, `getLastModifiedTime`, `readAttributes(p, BasicFileAttributes.class)`	`NoSuchFileException`
Same content?	`mismatch(a, b)` (Java 12) → `-1` if identical, else first differing byte index	
Same file?	`isSameFile(a, b)` — follows links	
One level listing	`list(dir)` → `Stream<Path>` (lazy, **close it**)	`NotDirectoryException`
Recursive walk	`walk(dir[, maxDepth])`, `find(dir, maxDepth, (path, attrs) -> …)` → `Stream<Path>`	
Fine-grained walk with callbacks	`walkFileTree(dir, FileVisitor)`	

Details worth knowing:

- `copy` of a directory copies only the directory itself (empty), not its contents. Recursive copy = `walk` + `copy` each entry.
- `walk` visits **depth-first** (goes all the way down one branch of the folder tree before moving to the next), starts with the directory itself, and by default does **not** follow symbolic links (pass `FileVisitOption.FOLLOW_LINKS` to change that).
- `readAttributes(..., BasicFileAttributes.class)` fetches size, times and type in **one** file-system call — much cheaper than calling `size`, `isDirectory` and `getLastModifiedTime` separately in a loop over thousands of files.

`code/ch13/files/FileOps.java` (key parts):

```java
Path root = Files.createTempDirectory("java1315_fileops");
try {
    Path docs = Files.createDirectories(root.resolve("docs/2025"));   // makes parents too
    Path a = Files.writeString(docs.resolve("a.txt"), "alpha\n");
    Files.createDirectory(docs);                                       // -> FileAlreadyExistsException
    Path b = Files.copy(a, docs.resolve("b.txt"));
    Files.copy(a, b);                                                  // -> FileAlreadyExistsException
    Files.copy(a, b, StandardCopyOption.REPLACE_EXISTING);             // allowed
    Path moved = Files.move(b, archive.resolve("b-old.txt"));          // move = rename
    Files.mismatch(a, moved) == -1;                                    // identical content
    BasicFileAttributes attrs = Files.readAttributes(a, BasicFileAttributes.class);

    try (Stream<Path> s = Files.walk(root)) {                          // lazy + holds OS handles
        s.map(p -> root.relativize(p).toString()).sorted().forEach(...);
    }
    try (Stream<Path> s = Files.find(root, 10,
            (p, at) -> at.isRegularFile() && p.toString().endsWith(".md"))) { ... }
    Files.delete(docs);                                                // -> DirectoryNotEmptyException
} finally {
    deleteRecursively(root);
}

static void deleteRecursively(Path root) throws IOException {
    try (Stream<Path> s = Files.walk(root)) {
        List<Path> all = s.sorted(Comparator.reverseOrder()).toList();  // children before parents
        for (Path p : all) Files.delete(p);
    }
}
```

Output:

```
exists a.txt?        true
isDirectory docs?    true
size a.txt           6 bytes
createDirectory again -> FileAlreadyExistsException
copy onto existing   -> FileAlreadyExistsException
after move b exists? false, moved exists? true
same content a/moved true
attrs: regular=true dir=false size=6 created<=modified true
-- Files.list(root) (one level, sorted):
   archive
   docs
   readme.md
-- Files.walk(root) (recursive, sorted):
   ''
   'archive'
   'archive/b-old.txt'
   'docs'
   'docs/2025'
   'docs/2025/a.txt'
   'docs/2025/notes.md'
   'readme.md'
-- Files.find *.md:
   docs/2025/notes.md
   readme.md
delete non-empty dir -> DirectoryNotEmptyException
deleteIfExists(missing) = false
read missing file    -> NoSuchFileException
temp dir removed?    true
```

- The empty string `''` in the walk output is the root itself (`root.relativize(root)`), proving that `walk` includes its starting point.
- We **sort** listing output because the order returned by the operating system is not specified; never rely on it.
- `deleteRecursively` sorts paths in reverse so `docs/2025/a.txt` comes before `docs/2025`, which comes before `docs` — a directory can only be deleted once it's empty. (`walkFileTree` with a `postVisitDirectory` callback is the other standard way.)

Watching a directory for changes

`WatchService` (Java 7) lets a program be notified when files in a directory are created, modified or deleted — how IDEs (Integrated Development Environments such as IntelliJ IDEA or VS Code) and dev servers notice edits:

```
// (not run: event delivery timing is OS-dependent; macOS uses polling with a delay of seconds)
try (WatchService ws = FileSystems.getDefault().newWatchService()) {
    dir.register(ws, StandardWatchEventKinds.ENTRY_CREATE, StandardWatchEventKinds.ENTRY_MODIFY);
    WatchKey key = ws.take();                          // blocks until something happens
    for (WatchEvent<?> e : key.pollEvents()) System.out.println(e.kind() + " " + e.context());
    key.reset();                                       // re-arm, or no more events arrive
}
```

It watches a single directory (not subdirectories), and events can be coalesced (merged into one) or overflow (`OVERFLOW` kind), so treat events as hints and re-scan when in doubt.

Common mistakes

- **Not closing** `Files.list` / `walk` / `find` / `lines`. They hold an open directory or **file handle** (an OS-issued token for an open file; each process may hold only a limited number); in a long-running server, leaking them eventually hits "Too many open files". Always use try-with-resources.
- **Check-then-act**: `if (!Files.exists(p)) Files.createFile(p);` — another process can create it in between (a **race condition**: the result depends on the timing of two independent actors). Call `createFile` and catch `FileAlreadyExistsException` instead (or use `CREATE_NEW`, §13.4).
- Using `Files.exists` as proof: it returns `false` both for "doesn't exist" and "can't tell" (no permission). `Files.notExists` is not simply its negation — both can be `false`.
- Hard-coding `/tmp` — use `Files.createTempDirectory` (portable, unique name).

Real-world examples

Log rotation (periodically renaming the current log file and starting a fresh one; `move` with `ATOMIC_MOVE` so readers never see half-written files), build tools walking source trees (`walk` + `find`), and "write to a temp file, then atomically move over the real file" — the standard trick for crash-safe config and **cache** (a saved copy of data kept for fast reuse) updates.

In the interview

30-second answer: "I use `Path.of` and the `Files` class: `createDirectories`, `copy` / `move` with explicit `StandardCopyOption`s, `delete` which throws a descriptive exception, and `walk` / `list` / `find` inside try-with-resources because they hold OS handles. For crash-safe updates I write to a temp file in the same directory and `move` it with `ATOMIC_MOVE`."

- *Why prefer `Files.delete` over `File.delete`?* → It throws a specific exception explaining the failure; `File.delete` just returns `false`.
- *How do you delete a directory tree?* → `walk`, sort in reverse order, delete each; or `walkFileTree` deleting in `postVisitDirectory`.
- *Is `walk` depth-first or breadth-first?* → Depth-first; it includes the start directory.

Self-check

1. What does `Files.copy(dir, target)` do for a non-empty directory? → Creates an empty directory; contents are not copied.
2. What does `Files.mismatch` return for identical files? → `-1`.
3. Why sort the results of `Files.list` in a test? → The OS order is unspecified.

13.4 Reading and writing whole files

What it is

For everyday text files, `Files` has one-line methods. They default to **UTF-8** (the dominant text encoding, explained in §13.7; this was true for `Files` methods even before Java 18) and open/close the file for you.

Method	Reads/writes	Memory	Since
`readString(p[, cs])` / `writeString(p, text[, cs], opts...)`	whole file as one `String`	whole file	11
`readAllLines(p)` / `write(p, lines, opts...)`	`List<String>` of lines	whole file	7
`readAllBytes(p)` / `write(p, bytes, opts...)`	`byte[]`	whole file	7
`lines(p)`	lazy `Stream<String>` — **must be closed**	one line at a time	8
`newBufferedReader(p)` / `newBufferedWriter(p, opts...)`	`BufferedReader` / `BufferedWriter`	small buffer	7
`newInputStream(p)` / `newOutputStream(p, opts...)`	raw bytes	small buffer	7

Open options (`StandardOpenOption`) control what "write" means. The default for the write methods is `CREATE`, `TRUNCATE_EXISTING`, `WRITE` — create if missing, **otherwise wipe it first** (*truncate* = cut the file to zero length). Others: `APPEND` (add at the end), `CREATE_NEW` (fail if it exists — an atomic "create only if absent"), `READ`, `SYNC` / `DSYNC` (force every write to the storage device — `DSYNC` syncs only the contents, `SYNC` also the metadata), `DELETE_ON_CLOSE`.

`code/ch13/files/ReadWrite.java` (key parts):

```
Files.writeString(f, "buy milk\n");                              // create or truncate
Files.writeString(f, "call bob\n", StandardOpenOption.APPEND);
Files.write(f, List.of("fix bug #42", "ship v2"), StandardOpenOption.APPEND);
Files.readString(f);
List<String> lines = Files.readAllLines(f);
try (Stream<String> s = Files.lines(f)) {                         // lazy; MUST be closed
    long withB = s.filter(l -> l.contains("b")).count();
}
Files.writeString(f, "x", StandardOpenOption.CREATE_NEW);         // FileAlreadyExistsException

try (BufferedWriter w = Files.newBufferedWriter(csv, StandardCharsets.UTF_8)) {
    w.write("name,score"); w.newLine();
    w.write("ana,91");     w.newLine();
    w.write("raj,78");     w.newLine();
}                                                                 // close() flushes the buffer
try (BufferedReader r = Files.newBufferedReader(csv)) {
    r.readLine();                                                 // skip header
    String line;
    while ((line = r.readLine()) != null) total += Integer.parseInt(line.split(",")[1]);
}
```

Output:

```
readString:
buy milk
call bob
fix bug #42
ship v2
readAllLines size = 4 -> [buy milk, call bob, fix bug #42, ship v2]
lines containing 'b' = 3
bytes = 38
CREATE_NEW on existing -> FileAlreadyExistsException
total score = 169
```

- `Files.write(path, lines)` writes each line followed by the platform line separator (`\n` here). `readAllLines` strips line terminators.
- `readLine()` returns `null` at end of file — the classic `while ((line = r.readLine()) != null)` idiom.

Trade-offs

Style	Pros	Cons
`readString` / `readAllLines`	shortest code	loads the whole file; a 4 GB log → `OutOfMemoryError` (the JVM ran out of heap memory)
`lines()`	constant memory, stream operations	must remember try-with-resources; `UncheckedIOException` if a read fails mid-stream
`BufferedReader` loop	constant memory, full control, easy early exit	more code

Rule of thumb: whole-file methods for files you know are small (configs, test **fixtures** — fixed sample data used by tests — < a few MB); streaming for anything user-supplied or unbounded.

Common mistakes

- Forgetting that the default write **truncates** — appending needs `APPEND`.
- Writing a CSV "by hand" with `split(",")` for real data: quoted fields containing commas break it. Use a CSV **library** (reusable code someone else wrote that you add as a dependency — e.g. Apache Commons CSV, OpenCSV) for anything external.
- Writing important files in place. If the program crashes mid-write the file is corrupt. Write a temp file in the same directory, then `Files.move(tmp, target, ATOMIC_MOVE, REPLACE_EXISTING)`.

Self-check

1. Which option makes `writeString` fail if the file exists? → `CREATE_NEW`.
2. Why wrap `Files.lines` in try-with-resources? → It keeps the file open until the stream is closed.
3. What does `readLine()` return at end of file? → `null`.

TL;DR

`readString` / `writeString` for small text, `lines` / `newBufferedReader` for big text, and remember: writes truncate unless you say `APPEND`.

13.5 Byte streams and character streams

What it is

In `java.io` all I/O is built from four **abstract classes** (classes that can't be instantiated directly and exist to be extended):

	Input (read)	Output (write)	Unit
Byte streams	`InputStream`	`OutputStream`	`byte` (8-bit raw data: images, zip, protobuf — Google's compact binary data format — anything)
Character streams	`Reader`	`Writer`	`char` (16-bit UTF-16 text unit; see §13.7)

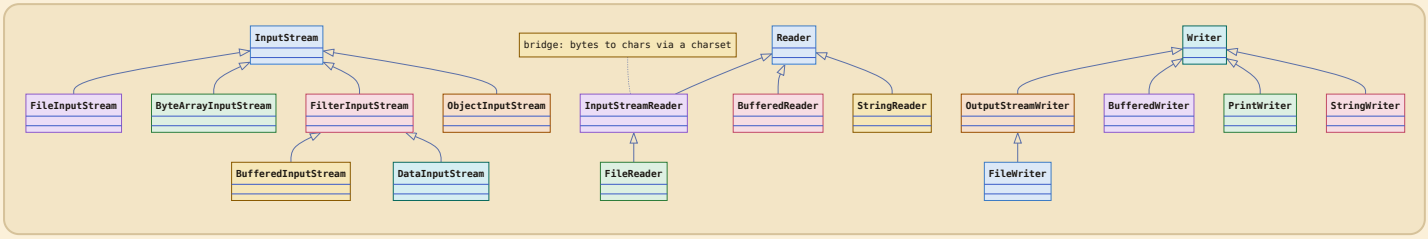
Byte streams move raw bytes. Character streams move text, and at the point where text meets bytes (a file, a socket) something must **encode** chars to bytes or **decode** bytes to chars using a **charset** (§13.7). The **bridge** classes are `InputStreamReader` (bytes → chars) and `OutputStreamWriter` (chars → bytes).

How it works: the decorator pattern

Concrete classes come in two kinds:

1. **Source/sink streams** that actually connect to something: `FileInputStream`, `ByteArrayInputStream`, a socket's stream, `FileReader`, `StringReader`, `StringWriter`.
2. **Filter (wrapper) streams** that take another stream in their constructor and add a feature: buffering (`BufferedInputStream`, `BufferedReader`), primitives (`DataInputStream`), objects (`ObjectInputStream`), compression (`GZIPInputStream`), formatting (`PrintStream`, `PrintWriter`).

You stack wrappers like nested boxes. Each `read`/`write` passes through the layers, and **closing the outermost wrapper closes the whole chain**. This is the **Decorator** pattern (wrapping an object in another object of the same type that adds behaviour, LLD Chapter 9).



(The output side mirrors this: `OutputStream` → `FileOutputStream`, `ByteArrayOutputStream`, `BufferedOutputStream`, `DataOutputStream`, `ObjectOutputStream`, `PrintStream`.)

The read contracts, which every subclass follows:

- `int read()` returns the next byte (0–255) or char (0–65535) **as an** `int`, or `-1` **at end of stream**. It returns `int` rather than `byte` precisely so `-1` can't be confused with a real value.
- `int read(byte[] buf)` fills *up to* `buf.length` bytes and returns **how many it actually read** (possibly fewer!) or `-1`. Always write `buf, 0, n`, never the whole array.
- Reads **block**: the **thread** (the flow of execution running your code) waits until data arrives, the stream ends, or an error occurs.

`code/ch13/streams/ByteAndCharStreams.java` (key parts):

```java
// 1) the classic copy loop
try (InputStream in = new FileInputStream(src.toFile());
     OutputStream out = new FileOutputStream(dst.toFile())) {
    byte[] buf = new byte[4];                  // tiny, to make the loop visible
    int n;
    while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
}
// 2) Java 9: let the library do the loop
in.transferTo(out);
// 3) characters: specify the charset at the byte boundary
try (Writer w = new OutputStreamWriter(new FileOutputStream(text.toFile()), StandardCharsets.UTF_8)) {
    w.write("naïve café ☕");
}
// 4) decorators: text -> buffered -> encoded -> gzipped -> file
try (PrintWriter pw = new PrintWriter(new BufferedWriter(new OutputStreamWriter(
        new GZIPOutputStream(new FileOutputStream(gz.toFile())), StandardCharsets.UTF_8)))) {
    for (int i = 1; i <= 1000; i++) pw.printf("line %d: all work and no play%n", i);
}
// 5) in memory
ByteArrayOutputStream bos = new ByteArrayOutputStream();
new StringReader("abc").transferTo(sw);        // Reader.transferTo: Java 10
```

Output:

```
copied with 4 read() calls -> Hello, bytes!
transferTo copied 13 bytes
chars written = 12, bytes on disk = 16
chars read back = 12
gzip file 2453 bytes holds 1000 lines
ByteArrayOutputStream -> captured
StringReader -> StringWriter: abc
```

- 13 bytes through a 4-byte buffer = 4 reads (4+4+4+1); the last `write(buf, 0, 1)` writes only one byte.
- 12 characters became 16 bytes: `ï` and `é` take 2 bytes in UTF-8, `☕` takes 3.
- ~30 KB of repetitive text compressed to about 2.4 KB — and the code reading it back just adds `GZIPInputStream` to the chain.

Useful extras (Java 9–25)

Method	What it does	Since
`InputStream.readAllBytes()`, `readNBytes(n)`	read everything / exactly up to n bytes	9 / 11
`InputStream.transferTo(out)`, `Reader.transferTo(w)`	copy to the end	9 / 10
`InputStream.nullInputStream()`, `OutputStream.nullOutputStream()`	empty source / discard sink (tests)	11
`Reader.of(CharSequence)`	a `Reader` over a string	24
`Reader.readAllAsString()`, `readAllLines()`	read the rest of a reader	25

`mark`/`reset` (remember a position and go back) and `skip` also exist; only some streams support `mark` (check `markSupported()`) — details in OCP Chapter 14 §8a.

Common mistakes

- `FileReader`/`FileWriter` **without a charset** on Java 17 and earlier used the platform default (often Windows-1252 on Windows) — files written on one machine read wrongly on another. On Java 11+ pass the charset: `new FileReader(file, StandardCharsets.UTF_8)`.
- Ignoring the return value of `read(buf)` and writing the whole buffer → garbage at the end of the copy.

- Closing only the inner stream, or not closing at all — buffered data is lost and the file handle leaks. Declare the outermost wrapper in try-with-resources.
- Using `PrintWriter`/`PrintStream` for important output: they **swallow** `IOException`s (check `checkError()`), which is fine for the console but not for writing a file you need.

In the interview

> **30-second answer:** "`java.io` has byte streams, `InputStream`/`OutputStream`, and character streams, `Reader`/`Writer`, bridged by `InputStreamReader`/`OutputStreamWriter` with an explicit charset. Features are added by wrapping — buffering, data types, compression — which is the Decorator pattern. `read` returns -1 at end of stream, and I always close the outermost wrapper with try-with-resources."

- *Why does `read()` return `int`?* → To fit all 256 byte values plus `-1` as the end marker.
- *Byte or character stream for a PNG?* → Byte stream; it isn't text.

Self-check

1. Which class turns bytes into chars? → `InputStreamReader` (with a charset).
2. What does closing a `BufferedReader` wrapping an `InputStreamReader` do? → Closes the whole chain.
3. What's wrong with `out.write(buf)` in a copy loop? → Writes stale bytes on the last partial read; use `write(buf, 0, n)`.

13.6 Buffering

What it is

A **buffer** is a block of memory (8 KiB — 8 × 1024 bytes — by default for the `Buffered*` classes) that collects many small reads or writes into one large operation. Without it, each `write('x')` on a `FileOutputStream` becomes a **system call** — a request to the operating-system **kernel** (the core of the OS that controls hardware such as disks), which costs far more than copying a byte in memory. Analogy: carrying groceries from the car one item per trip versus filling a bag.

How it works

1. `BufferedOutputStream.write(b)` puts the byte in its internal array.
2. When the array is full, it writes the whole block to the wrapped stream in one call.
3. `flush()` forces out whatever is buffered now; `close()` flushes then closes.
4. On input, `BufferedInputStream`/`BufferedReader` read a large block at once and hand out pieces from memory; `BufferedReader` also adds `readLine()` and `lines()`.

`code/ch13/streams/BufferingDemo.java` (key parts):

```
try (OutputStream out = new FileOutputStream(f1.toFile())) {
    for (int i = 0; i < 200_000; i++) out.write('x');          // one system call per byte
}
try (OutputStream out = new BufferedOutputStream(new FileOutputStream(f2.toFile()))) {
    for (int i = 0; i < 200_000; i++) out.write('x');          // one system call per 8 KiB
}
BufferedWriter w = new BufferedWriter(new FileWriter(lost.toFile()));
w.write("this sits in the buffer");
Files.size(lost);   // 0 !
w.flush();
Files.size(lost);   // 23
```

Output (timings are machine-dependent; the ratio is the lesson):

```
same size: true (200000 bytes)
unbuffered: 205 ms   [machine-dependent]
buffered:   2 ms   [machine-dependent]
buffered faster? true
before flush: 0 bytes on disk
after flush:  23 bytes on disk
```

Same 200,000 bytes, roughly 100× faster with a buffer on this machine. And until `flush()` / `close()`, written data lives only in the buffer: if the JVM crashes or you forget to close, it never reaches the file.

When not to use it

- When you already read/write big arrays (`read(byte[8192])`, `Files.readAllBytes`, `transferTo`) — a buffer adds only a copy.
- The `Files` convenience methods and `Files.newBufferedReader/Writer` already buffer.
- Interactive **protocols** (agreed message formats between two programs, e.g. over a network) where each message must go out immediately — you can still buffer, but call `flush()` after each message.

Common mistakes

- Forgetting `flush()` on a socket writer → the other side waits forever for a message sitting in your buffer.
- Wrapping twice (`new BufferedReader(Files.newBufferedReader(p))`) — harmless but pointless.
- Note: `flush()` hands data to the OS, not necessarily to the physical disk. For **durability** (data surviving a crash or power cut) after power loss use `FileChannel.force(true)` or the `SYNC` / `DSYNC` open options (databases do this).

TL;DR

Always buffer small reads/writes; `close()` flushes; `flush()` ≠ on disk.

13.7 Charsets and encodings

What it is

Files and networks carry **bytes**; Java `String`s hold **characters** (as UTF-16 `char` units). A **charset** (character encoding) is the rulebook that maps between them.

- **UTF-8** — variable length: 1 byte for ASCII (the basic 128 English letters, digits and symbols), 2 for most European accents, 3 for most other symbols (`€`, CJK — Chinese, Japanese, Korean characters), 4 for emoji. The web's and Java's default.
- **UTF-16** — 2 or 4 bytes per character; what `String` uses internally (conceptually).
- **ISO-8859-1** (Latin-1) — exactly 1 byte per character, only 256 characters.
- **US-ASCII** — 7-bit, 128 characters.

Mojibake is the garbled text you get when bytes are decoded with a different charset from the one used to encode them (`cafÃ©`).

Why it matters: JEP 400 (Java 18)

A **JEP** (*JDK Enhancement Proposal*) is the numbered design document behind each new Java feature. Before Java 18, APIs without an explicit charset (`FileReader`, `new String(bytes)`, `String.getBytes()`, `Scanner(File)`) used the **platform default**, which depended on the OS and **locale** (the user's language/region settings) (UTF-8 on most Linux/macOS, Windows-1252 on many Windows machines). The same program could read a file correctly on a developer's Mac and wrongly on a Windows server. Since **Java 18 (JEP 400) the default charset is UTF-8 everywhere**. (The console's own encoding is separate and still follows the terminal: see `System.getProperty("stdout.encoding")`.) Being explicit — `StandardCharsets.UTF_8` — is still best practice, because it documents intent and keeps code correct if someone runs it with `-Dfile.encoding=COMPAT` (`-Dname=value` on the `java` command line sets a *system property*, §13.11).

`code/ch13/charsets/CharsetDemo.java` (key parts):

```
String s = "café €";
for (Charset cs : new Charset[]{UTF_8, UTF_16BE, ISO_8859_1, US_ASCII}) {
    byte[] b = s.getBytes(cs);                                  // encode
    System.out.printf("%-10s %2d bytes  %s%n", cs, b.length, hex.formatHex(b));
}
byte[] utf8 = "café".getBytes(UTF_8);
new String(utf8, ISO_8859_1);                                   // decode with the wrong charset
Files.writeString(latin, "café", ISO_8859_1);
Files.readString(latin);                                        // assumes UTF-8 -> throws
String e = "hi👋";  e.length(); e.codePoints().count();
```

Output:

```
default charset = UTF-8 (UTF-8 by default since Java 18, JEP 400)
UTF-8       9 bytes  63 61 66 c3 a9 20 e2 82 ac
UTF-16BE   12 bytes  00 63 00 61 00 66 00 e9 00 20 20 ac
ISO-8859-1  6 bytes  63 61 66 e9 20 3f
US-ASCII    6 bytes  63 61 66 3f 20 3f
decoded wrong  -> cafÃ©
decoded right  -> café
readString(latin1 file) -> MalformedInputException
with right charset -> café
length()=4 codePoints=3 utf8 bytes=6
```

What to notice:

- The same 6-character string is 9, 12 or 6 bytes depending on the charset.
- `3f` is `?`: `String.getBytes` **silently replaces** characters the charset can't represent (`€` in Latin-1; `é` and `€` in ASCII). Data is lost without an error. (To get an error instead, use a `CharsetEncoder` with `CodingErrorAction.REPORT`.)
- `Files.readString`/`readAllLines` are **strict**: invalid UTF-8 throws `MalformedInputException` rather than guessing — better than silent corruption. `BufferedReader` built with `InputStreamReader` is lenient and substitutes `�` (U+FFFD).
- `"hi👋".length()` is 4: the emoji is one **code point** but two `char`s (a *surrogate pair*). Use `codePoints()` to count what humans see as characters (more in Chapter 7).

Common mistakes

- Decoding a byte array that was cut in the middle of a multi-byte character (e.g. splitting network data at a fixed size) — decode after reassembling, or use a `Reader` which handles the boundary.
- A **BOM** (byte-order mark, `EF BB BF`) at the start of files from some Windows tools: Java's UTF-8 decoder keeps it as the character `\uFEFF`, so the first header of a CSV might not equal `"name"`. Strip it explicitly.
- Using `Properties.load(InputStream)`, which assumes ISO-8859-1 (§13.11).

In the interview

30-second answer: "Strings are characters; files are bytes; a charset converts. Since Java 18 the default is UTF-8 on every platform, but I still pass `StandardCharsets.UTF_8` explicitly. Wrong charset gives mojibake; `getBytes` silently substitutes `?` for unmappable characters while `Files.readString` throws on malformed input."

- *How many bytes is `é` in UTF-8?* → 2.
- *Why can `String.length()` differ from the number of visible characters?* → It counts UTF-16 `char`s; characters outside the Basic Multilingual Plane (emoji) take two.

Self-check

1. What was JEP 400? → UTF-8 as the default charset, Java 18.
2. What does ISO-8859-1 do with `€`? → Can't represent it; `getBytes` writes `?`.

3. Which is strict on bad input: `Files.readString` or `InputStreamReader`? → `Files.readString`.

13.8 Console and standard streams

What it is

Every process starts with three **standard streams** connected by the OS: **standard input** (`System.in`, an `InputStream` — usually the keyboard), **standard output** (`System.out`, a `PrintStream`) and **standard error** (`System.err`, a `PrintStream` for diagnostics). **Shells** (the command-line programs, like bash or zsh, that you type commands into) can **redirect** them (point them at files instead of the screen/keyboard) independently: `java App < input.txt > out.txt 2> errors.txt`.

Ways to read input:

Tool	Best for	Notes
`Scanner(System.in)`	small interactive programs, exercises	parses tokens (`nextInt`, `nextDouble`, `nextLine`); slow for big input
`BufferedReader(new InputStreamReader(System.in))`	large inputs (competitive programming, piped data — output of one command fed straight into another)	line-based; parse with `split` / `Integer.parseInt`
`System.console()`	real terminals; passwords	`readPassword()` doesn't echo and returns `char[]` (can be wiped)
`IO.readln(prompt)` / `IO.println(x)`	beginner programs	`java.lang.IO`, **Java 25** (JEP 512), usable without imports

`code/ch13/console/ConsoleInput.java` simulates typed input with an in-memory stream so the output is repeatable:

```
String typed = "Ada\n36\n3 4 5\n";
Scanner sc = new Scanner(new ByteArrayInputStream(typed.getBytes(UTF_8)));  // real code: new Scanner(System.in)
String name = sc.nextLine();
int age = sc.nextInt();
while (sc.hasNextInt()) sum += sc.nextInt();

BufferedReader br = new BufferedReader(new StringReader(typed));  // real code: new InputStreamReader(System.in)
String first = br.readLine();
int parsedAge = Integer.parseInt(br.readLine().trim());
for (String tok : br.readLine().split("\\s+")) total += Integer.parseInt(tok);

Scanner trap = new Scanner("42\nhello\n");
int n = trap.nextInt();
String rest = trap.nextLine();           // "" - the rest of the line after 42!

Console c = System.console();
c != null && c.isTerminal();             // isTerminal(): Java 22
IO.println("IO.println works without System.out");              // Java 25
```

Output (the stderr line may appear in a different position when both streams go to one terminal — they are separate streams):

```
Scanner: name=Ada age=36 sum=12
BufferedReader: name=Ada age=36 sum=12
trap: n=42 nextLine=[] then [hello]
stdout and stderr are separate streams (redirect with > and 2>)
this line went to stderr
System.console() available? false
IO.println works without System.out
```

- **The `nextInt`/`nextLine` trap**: `nextInt()` consumes `42` but leaves the newline; the following `nextLine()` returns the empty remainder of that line. Fix: call an extra `nextLine()` after `nextInt()`, or read whole lines and parse them.
- `System.console()` returns `null` when input or output is redirected (IDEs, pipes, CI — *continuous integration*, servers that automatically build and test every code change) — always null-check it. Java 22–24 used a JLine-based (JLine is a library for rich terminal input) console by default that could return a `Console` even then; Java 25 reverted to the old behaviour. `isTerminal()` (22) tells you whether a returned console is really interactive.

Common mistakes

- Closing a `Scanner` wrapping `System.in` in a helper method — that closes `System.in` for the rest of the program.
- Using `System.out.println` for logging in applications — use a logging framework (a library that writes timestamped, levelled, configurable log messages) (Chapter 18).
- Reading passwords with `Scanner` into a `String` — use `Console.readPassword()`.

Self-check

1. Which stream should error messages go to? → `System.err`.
2. Why is `BufferedReader` preferred for 10^6 numbers? → `Scanner` uses **regex** (regular expression: a pattern language for matching text) parsing and is much slower.
3. What does `IO.println` need to be imported? → Nothing; `java.lang.IO` is imported automatically (Java 25).

13.9 Binary data, channels and buffers

Primitive values: `DataOutputStream` / `DataInputStream`

To store numbers compactly (not as text), `DataOutputStream` writes Java **primitives** (`int`, `double`, `boolean` …) in a fixed **big-endian** (most significant byte first) binary format; `DataInputStream` reads them back. **You must read in exactly the same order and types you wrote** — the file has no labels.

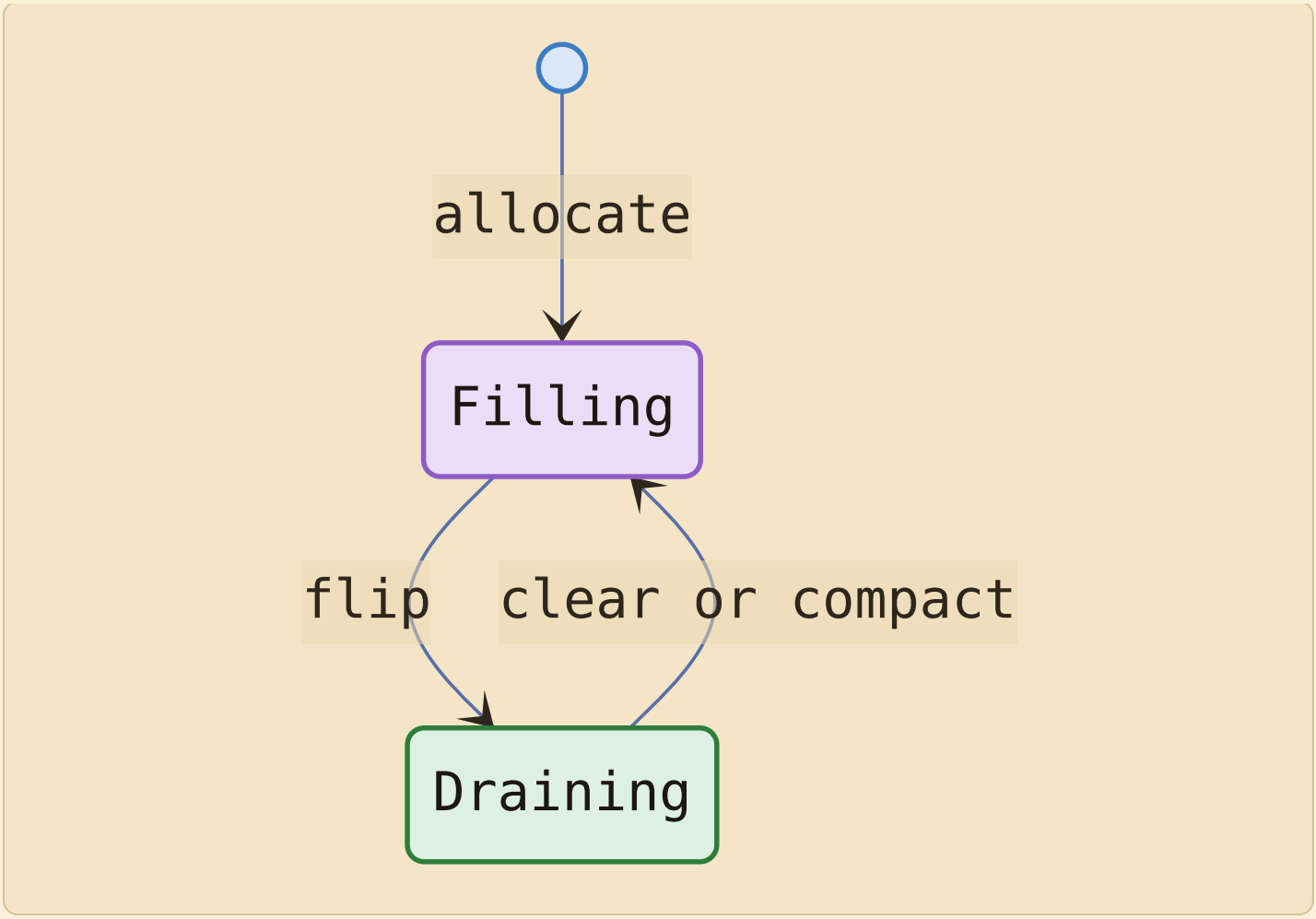
`code/ch13/binary/DataAndChannels.java` (key parts):

```
try (DataOutputStream out = new DataOutputStream(new BufferedOutputStream(Files.newOutputStream(f)))) {
    out.writeInt(2025);  out.writeDouble(3.5);  out.writeBoolean(true);  out.writeUTF("ok");
}
try (DataInputStream in = new DataInputStream(new BufferedInputStream(Files.newInputStream(f)))) {
    in.readInt(); in.readDouble(); in.readBoolean(); in.readUTF();
}
```

Channels and `ByteBuffer`

NIO models I/O differently from streams:

- A **channel** (`FileChannel`, `SocketChannel`) is a connection that can read *into* and write *from* a buffer, often in both directions, and supports **positioned** (random-access) reads.
- A `ByteBuffer` is a fixed-capacity block of bytes with three cursors: **position** (next index to use), **limit** (end of valid data), **capacity**. You *fill* it (channel read), call `flip()` to switch to *draining* (limit = position, position = 0), consume it, then `clear()` to fill again (position = 0, limit = capacity) — or `compact()`, which keeps any unread bytes at the front.




```
try (FileChannel ch = FileChannel.open(g, READ)) {
    ByteBuffer buf = ByteBuffer.allocate(8);
    while (ch.read(buf) != -1) {         // fill
        buf.flip();                      // switch to draining
        sb.append(UTF_8.decode(buf));
        buf.clear();                     // ready to fill again
    }
}
ch.read(word, 13);                       // positioned read at byte offset 13
```

Output:

```
file size = 17 bytes (4 + 8 + 1 + 2 + 2)
read back: 2025, 3.5, true, ok
channel wrote 20 bytes, position now 20
channel read: channels and buffers
bytes 13..15 = buf
```

(`writeUTF("ok")` = a 2-byte length prefix + 2 bytes. Decoding an 8-byte UTF-8 chunk independently is only safe here because the text is ASCII — with multi-byte characters use a `CharsetDecoder` or a `Reader`.)

What else NIO offers (overview)

Feature	What it's for
`FileChannel.map(...)` → `MappedByteBuffer`	**memory-mapped files**: the OS maps the file into memory; fast random access to huge files (databases, search indexes)
`FileChannel.transferTo/transferFrom`	**zero-copy** (the OS moves the data without copying it through your program's memory) file-to-socket transfer (web servers sending static files)
`FileChannel.lock()`	OS-level file locks between processes
`Selector` + non-blocking `SocketChannel`	one thread watching thousands of connections — the basis of Netty (a popular high-performance networking library); far less needed now that **virtual threads** (lightweight JVM-managed threads) make blocking I/O cheap (Chapter 15)
`AsynchronousFileChannel`	**callback** (a function you hand over to be called when the work finishes) / `Future`-based (a handle to a result that will be ready later) file I/O
`ByteBuffer.allocateDirect`	memory outside the **heap** (the JVM-managed memory area where objects live and the garbage collector cleans up) for **native** I/O (I/O done directly by OS / C code) (the newer Foreign Function & Memory API, final in Java 22, is the modern route to off-heap memory)
`RandomAccessFile` (java.io)	the older way to seek and read/write at positions

Self-check

1. What does `flip()` do? → Sets limit to the current position and position to 0, so you can read what you just wrote into the buffer.
2. Can a `DataInputStream` tell you what type is next? → No; the reader must know the layout.

13.10 Serialization (and why to avoid it)

What it is

Java serialization converts an **object graph** (an object and everything it references) into a byte stream with `ObjectOutputStream.writeObject`, and rebuilds it with `ObjectInputStream.readObject`. A class opts in by implementing the **marker interface** `java.io.Serializable` (an interface with no methods that only "tags" a class).

Rules:

1. Every non-`transient`, non-`static` field is written; referenced objects must also be `Serializable`, or `NotSerializableException` is thrown.
2. `transient` fields are skipped and come back as default values (`null`, `0`, `false`).
3. On deserialization of a normal class, **its constructor does not run**. Java allocates the object and fills fields directly; only the no-arg constructor of the first *non-serializable* superclass runs. Validation in constructors is therefore bypassed.
4. **Records** are the exception: they are rebuilt **through the canonical constructor** (the record's main constructor taking every component in order), so validation runs (Java 16).
5. `serialVersionUID` is a version stamp. If you don't declare one, the JVM computes it from the class structure; almost any change (adding a method) changes it, and old data then fails with `InvalidClassException`. Declare it explicitly (`@Serial private static final long serialVersionUID = 1L;` — `@Serial` is Java 14) if you serialize at all. (*Deserialization* is the reverse step: bytes back into objects.)

`code/ch13/serial/SerializationDemo.java` (key parts):

```
static class Account implements Serializable {
    @Serial private static final long serialVersionUID = 1L;
    private final String owner;
    private final long balanceCents;
    private transient String sessionToken;                      // skipped
    Account(...) { ...; System.out.println("  constructor ran for " + owner); }
}
record Point(int x, int y) implements Serializable {
    Point { if (x < 0 || y < 0) throw new IllegalArgumentException("negative"); ... }
}

ObjectInputFilter allow = ObjectInputFilter.Config.createFilter(
        "ch13.serial.SerializationDemo$Account;ch13.serial.SerializationDemo$Point;!*");
in.setObjectInputFilter(allow);                                 // before readObject()
```

Output:

```
create:
  constructor ran for ana
serialized to 116 bytes
deserialize (note: NO constructor call for a normal class):
  Account[ana, 1250, token=null]  same object? false
record - create:
  canonical constructor ran for (1,2)
record - deserialize (the canonical constructor runs again):
  canonical constructor ran for (1,2)
  Point[x=1, y=2]
  canonical constructor ran for (1,2)
filtered, allowed class: Point[x=1, y=2]
filtered, ArrayList -> InvalidClassException: filter status: REJECTED
```

- The transient token came back `null`; the account is a new object with equal state.
- The record's canonical constructor ran on every deserialization (including the filtered one) — so an invalid byte stream can't create an invalid `Point`.
- The allow-list filter (`!*` = reject everything else) blocked an `ArrayList` before any of its code ran.

Why to avoid it

Effective Java (Item 85) is blunt: there is no reason to use Java serialization in new systems.

1. **Security.** `readObject` can **instantiate** (create an object of) any serializable class on the **classpath** (the list of folders and JAR files the JVM loads classes from) and run its deserialization methods. Attackers chain such "gadget" classes (existing harmless-looking classes whose side effects can be combined into an attack) to execute code — many real **remote-code-execution** (RCE: an attacker running their own code on your server) vulnerabilities in application servers and libraries came from deserializing untrusted bytes. Filters (`ObjectInputFilter`, Java 9, JEP 290; JVM-wide filter factories, Java 17, JEP 415; or the `jdk.serialFilter` system property) reduce the risk but must be configured correctly.
2. **Hidden constructor.** Deserialization bypasses constructors and invariants (except for records).
3. **Fragile versioning** and a format only Java can read.
4. **Freezes your implementation**: private fields become part of the public **wire format** (the exact byte layout sent over a network or stored on disk).

Use instead: JSON (§13.12) for human-readable data and APIs, Protocol Buffers/Avro (compact binary formats with a schema, readable from many languages) for compact cross-language data, a database for **persistence** (keeping data after the program stops). If you're stuck with serialization (legacy RMI — Remote Method Invocation, Java's old way to call methods on another machine — some caches), serialize records, set an allow-list filter, and never deserialize data from untrusted sources.

In the interview

> **30-second answer:** "Serialization turns an object graph into bytes via `Serializable`; `transient` fields are skipped, `serialVersionUID` guards compatibility, and normal classes are rebuilt without calling their constructor. I avoid it for new code because deserializing untrusted data can lead to remote code execution and it's brittle; I use JSON or protobuf. If I must, I use records and an `ObjectInputFilter` allow-list."

- *What happens to a `static` field?* → Not serialized; it belongs to the class.
- *Superclass not `Serializable`?* → Its no-arg constructor runs on deserialization and its fields are not restored (and it must have an accessible no-arg constructor).

Self-check

1. Does a normal `Serializable` class's constructor run on `readObject`? → No.
2. And a record's? → Yes, the canonical constructor.
3. What exception when a referenced object isn't serializable? → `NotSerializableException`.

13.11 Properties files

What it is

A **properties file** is plain text of `key=value` lines — the traditional Java configuration format (Spring Boot's — Spring Boot being the most popular Java framework for building services — `application.properties`, `log4j2.properties`, `gradle.properties`). `java.util.Properties` loads and stores them.

Format rules: `#` or `!` start a comment; the separator can be `=`, `:` or whitespace, with spaces around it ignored; a trailing `\` continues the value on the next line (leading whitespace of the next line is dropped); values are **always strings**.

`code/ch13/props/PropertiesDemo.java` (key parts):

```
Files.writeString(file, """
        # database settings
        db.url = jdbc:postgresql://localhost:5432/shop
        db.pool.size=10
        ! bang also starts a comment
        greeting=Hello, \\
                 world
        city=Zürich
        """);
Properties p = new Properties();
try (Reader r = Files.newBufferedReader(file, StandardCharsets.UTF_8)) { p.load(r); }
p.getProperty("db.url");
Integer.parseInt(p.getProperty("db.pool.size"));
p.getProperty("timeout", "30");                  // default when absent
p.setProperty("timeout", "45");
try (Writer w = Files.newBufferedWriter(out, UTF_8)) { p.store(w, "saved by demo"); }
```

Output:

```
keys        = [city, db.pool.size, db.url, greeting]
db.url      = jdbc:postgresql://localhost:5432/shop
pool+1      = 11   (values are always Strings)
greeting    = Hello, world
city        = Zürich
timeout     = 30   (default used)
saved 5 key lines
system property java.version starts with 2? true
```

(`\\` in the Java text block produces a single `\` in the file.)

Common mistakes

- `load(InputStream)` decodes as **ISO-8859-1**, so `Zürich` saved as UTF-8 becomes `ZÃ¼rich`. Use `load(Reader)` with UTF-8. (Resource bundles, `PropertyResourceBundle`, read UTF-8 since Java 9.)
- `Properties` extends `Hashtable<Object,Object>` (an old, synchronised map class; a historical design mistake); use only `getProperty`/`setProperty`/`stringPropertyNames`, never `put`/`get` with non-strings.
- **Committing** (saving into version control such as Git) secrets (passwords, **API keys** — secret tokens that identify your program to a service) in properties files — use **environment variables** (named values the OS passes to each process) or a **secret manager** (a service that stores and hands out credentials securely).
- **System properties** (`System.getProperty("user.home")`, set with `-Dkey=value`) are a separate, JVM-wide `Properties` object; **environment variables** come from `System.getenv("HOME")`.

For locale-specific messages with `ResourceBundle`, see OCP Chapter 11.

13.12 Working with JSON

What it is

JSON (JavaScript Object Notation: nested `{"key": value}` objects and `[ … ]` lists in plain text) is the text format of web APIs and most modern config/data files. As of Java 25/26 the JDK has **no public JSON API** — you use a library. The two common ones are **Jackson** (the default in Spring Boot) and **Gson**; JSON-B/Yasson is the Jakarta EE (the enterprise Java specifications, formerly Java EE) standard. Jackson 3 (October 2025, default in Spring Boot 4) moved to `tools.jackson.*` packages and prefers building an immutable `JsonMapper` via `JsonMapper.builder()`; the calls below are otherwise the same.

How it works (Jackson 2.x — not run: needs the `jackson-databind` dependency)

A **dependency** is an external library your project declares and the build tool downloads for you.

```
// (Jackson - not run)
record Order(String id, List<String> items, double total) {}

ObjectMapper mapper = new ObjectMapper();                  // thread-safe once configured; reuse one
String json = mapper.writeValueAsString(new Order("A1", List.of("pen", "ink"), 9.5));
// {"id":"A1","items":["pen","ink"],"total":9.5}
Order back = mapper.readValue(json, Order.class);          // records supported since Jackson 2.12
JsonNode tree = mapper.readTree(json);                     // untyped tree when there's no class
tree.get("items").get(0).asText();                         // "pen"
List<Order> orders = mapper.readValue(arrayJson, new TypeReference<List<Order>>() {});  // generics survive
erasure
```

- **Data binding** (JSON ↔ your classes; records fit perfectly) for known shapes; the **tree model** (`JsonNode`) for dynamic or partial data; **streaming** (`JsonParser`) for gigantic files.
- `TypeReference` works around **type erasure** (generic type arguments like `<Order>` are removed after compilation, so they're unknown at run time; Chapter 9).
- Money: use `BigDecimal`, not `double` (Chapter 7).
- Adding the dependency with Maven/Gradle: Chapter 18.

Common mistakes

Hand-building JSON with string concatenation (quotes and escapes break it; it's an **injection** risk — attacker-controlled text changing the structure of what you build), creating a new `ObjectMapper` per request (it's expensive — share one), and silently ignoring unknown fields vs failing on them without deciding which you want (`FAIL_ON_UNKNOWN_PROPERTIES`).

13.13 Best practices and common mistakes (checklist)

1. **`Path` + `Files`** for files; `java.io.File` only for legacy APIs.
2. **try-with-resources** for every stream, reader, writer, channel, and for `Files.lines/list/walk/find`.
3. **Always name the charset** (`StandardCharsets.UTF_8`), even though it's the default since Java 18.
4. **Buffer** small reads/writes; remember `close()` flushes and `flush()` isn't a disk sync.
5. **Stream large inputs**; load whole files only when they're known to be small.
6. **Don't check-then-act** (`exists` then `create`); do the operation and handle `FileAlreadyExistsException`/`NoSuchFileException`.
7. **Write atomically**: temp file in the same directory + `move(..., ATOMIC_MOVE)`.
8. **Validate user-supplied paths**: `normalize()` then `startsWith(baseDir)`.
9. **Don't swallow `IOException`**: it carries the reason (disk full, permission denied). Wrap in `UncheckedIOException` inside lambdas (Chapter 8 §8.8).
10. **No Java serialization of untrusted data**; prefer JSON/protobuf; if unavoidable, records + filters.
11. **Tests**: use `@TempDir` (JUnit Jupiter, the current version of Java's standard test framework) or `Files.createTempDirectory`, and in-memory streams (`ByteArrayInputStream`, `StringReader`) to test parsing without files.
12. **Blocking I/O is fine on virtual threads** — thousands of **concurrent** (in progress at the same time) file/socket operations no longer need **async** APIs (ones that return immediately and deliver the result later) (Chapter 15 §15.4).

In the interview (chapter-level)

- *How would you count lines in a 50 GB log file?* → `Files.lines(p)` in try-with-resources (or a `BufferedReader` loop) — constant memory; never `readAllLines`.
- *How do you make a config update crash-safe?* → Write to a temp file, `force` if durability matters, then `ATOMIC_MOVE` over the old file.
- *NIO vs IO?* → `java.io` = blocking streams; NIO = channels/buffers/selectors (non-blocking sockets, memory mapping); NIO.2 = `Path`/`Files` file-system API. Most file code today is NIO.2 + `java.io` readers.

Chapter TL;DR

Topic	Remember
APIs	`java.io` streams (1.0), NIO channels/buffers (1.4), NIO.2 `Path`/`Files` (7) — use NIO.2 for files
`Path`	Just a name; `resolve` appends (absolute arg wins), `relativize` gives the route, `normalize` is textual
`Files` ops	Specific exceptions; `copy`/`move` need `REPLACE_EXISTING`; delete trees children-first; close `list`/`walk`/`find`/`lines`
Whole files	`readString`/`writeString` (11) for small files; writes truncate unless `APPEND`; `CREATE_NEW` = atomic create
Streams	Byte (`InputStream`/`OutputStream`) vs char (`Reader`/`Writer`); bridge with a charset; decorators stack; `read` → `-1` at end
Buffering	~100× fewer system calls; `close()` flushes; unflushed data is lost
Charsets	UTF-8 default since Java 18 (JEP 400); still pass it explicitly; wrong charset = mojibake; `getBytes` substitutes `?`
Console	`Scanner` (easy), `BufferedReader` (fast), `Console` (passwords, `isTerminal` 22), `IO` (25); beware `nextInt`+`nextLine`
Binary/NIO	`Data*Stream` for primitives in a fixed order; `ByteBuffer` fill → `flip` → drain → `clear`; memory mapping for huge files
Serialization	`Serializable`, `transient`, `serialVersionUID`; constructors skipped (records excepted); security risk — avoid, or filter
Properties	`load(Reader)` with UTF-8; values are strings; `getProperty(key, default)`
JSON	Needs a library (Jackson/Gson); bind to records

Where next: doing many I/O operations at once — threads and thread pools in Chapter 14 — Concurrency I, then virtual threads, which make blocking I/O scale, in Chapter 15. Exam traps: OCP Chapter 14 — I/O. Python comparison (`open`, `pathlib`): Python Chapter 9.

Chapter 14 — Concurrency I: Threads, Locks and the `java.util.concurrent` Toolkit

A web server handling thousands of requests, an app downloading images while the user scrolls, a **batch job** (a program that processes a large pile of data in one go, usually with no user waiting) using all 16 **cores** (the independent processing units inside one CPU): all of these do several things **at the same time**. Java has supported this since version 1.0 through **threads** (defined in §14.1), and since Java 5 through the high-level `java.util.concurrent` package (designed by Doug Lea and the authors of *Java Concurrency in Practice*, "JCIP"). Concurrency is also where Java programs get their nastiest bugs — results that are wrong only sometimes, on some machines, under load.

This chapter builds the foundation: creating and controlling threads, *why* shared data goes wrong (race conditions and visibility), the tools that make it right (`synchronized`, `volatile`, locks, atomics), and the everyday toolkit you should actually use at work (thread pools, concurrent collections, synchronizers). Chapter 15 continues with `CompletableFuture`, Fork/Join, **virtual threads**, structured concurrency, scoped values, the Java Memory Model (the rules saying when one thread's writes become visible to another) and **liveness** problems — situations where threads stop making progress: **deadlock** (threads each waiting forever for a lock another holds), **livelock** (threads busily reacting to each other without getting anywhere) and **starvation** (a thread never getting its turn).

What you'll learn

§	Topic	You'll be able to…
14.1	Concurrency basics	explain processes, threads, concurrency vs parallelism, and why threads share memory
14.2	Creating and starting threads	use `Thread`, `Runnable`, lambdas and `Thread.Builder`; `start` vs `run`; `join`; daemon threads
14.3	Thread lifecycle	name every `Thread.State` and what causes each
14.4	Interruption	cancel threads cooperatively and handle `InterruptedException` correctly
14.5	Race conditions	recognise read-modify-write and check-then-act bugs
14.6	`synchronized` and intrinsic locks	protect shared state; know what object is locked
14.7	`volatile` and visibility	stop a thread with a flag; know what `volatile` does *not* do
14.8	`wait` / `notify`	write a guarded block correctly (and know why you rarely should)
14.9	Explicit locks	use `ReentrantLock`, `tryLock`, `Condition`, `ReadWriteLock`, `StampedLock`
14.10	Atomic variables	use `AtomicInteger`, CAS, `AtomicReference`, `LongAdder`
14.11	Executors and thread pools	submit tasks, use `Future`, size and configure pools, shut them down
14.12	Concurrent collections	pick `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue` … and avoid compound-action bugs
14.13	Synchronizers	coordinate threads with `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `Phaser`
14.14	Designing thread-safe classes	apply JCIP's strategies: immutability, confinement, encapsulated locking

All runnable code is in `code/` (package `ch14.*`), compiled with `javac --release 25` (the Java compiler, targeting Java 25) and run on JDK 26 (the Java Development Kit) on a 14-core laptop. **Concurrency output is not always repeatable.** Lines marked `[nondeterministic]` (not predictable from the code alone) change from run to run (that is the point being demonstrated); lines marked `[machine-dependent]` depend on your hardware. Everything else is forced into a fixed order with `join`, latches or barriers (waiting tools explained in §14.2 and §14.13).

Related material — linked, not repeated.

- Exam-level API tables and trick questions (`Future` methods, `Executors` factories, `TimeUnit`, lock methods, parallel streams): OCP Chapter 13 — Concurrency.
- Design-level concepts with diagrams and interview framing (mutex — a lock that lets one thread in at a time — semaphores, producer-consumer, reader-writer, thread-pool patterns): LLD Chapter 12 — Concurrency concepts.
- Classic problems solved step by step (FooBar alternation, H2O, dining philosophers, blocking queue, TTL cache — entries expire after a *time to live* — rate limiter, which caps how many requests are allowed per second): LLD Chapter 13 — Concurrency problems.

14.1 Concurrency basics

What it is

- A **process** is a running program with its own memory, isolated from other processes by the operating system. Starting `java App` creates one process: the JVM (Java Virtual Machine, the program that runs your compiled Java code).
- A **thread** is an independent path of execution *inside* a process. Each thread has its own **call stack** (local variables, method frames) and **program counter** (the marker of which instruction runs next), but **all threads in a process share the same heap** (the memory area where all objects live) — the same objects and fields.
- **Concurrency** means several tasks are *in progress* during the same period (possibly taking turns on one CPU core). **Parallelism** means tasks literally run *at the same instant* on different cores. Concurrency is about structure; parallelism is about hardware. (Detailed comparison with diagrams: LLD §12.1–12.2.)

Analogy: a restaurant kitchen. The kitchen (process) has shared fridges and counters (heap). Each cook (thread) has their own recipe card and hands (stack). Cooks working at the same time get a lot done — until two of them grab the last egg at once.

Why it matters

1. **Responsiveness** — a UI (user interface) or server keeps answering while slow work (disk, network) happens elsewhere.
2. **Throughput** (how much work gets done per second; see System Design Fundamentals) — while one request waits for a database, another can use the CPU.
3. **Using all cores** — CPU-heavy work (image processing, sorting big data) can be split across cores.

The core problem

Because threads share the heap, two threads touching the same object can interfere. Three things can go wrong:

1. **Atomicity** — an operation that looks like one step (`count++`) is really several, and threads interleave in the middle (§14.5).
2. **Visibility** — a value written by one thread may not be seen by another, because of **CPU caches** (small, fast per-core copies of memory) and **compiler reordering** (the compiler or CPU shuffling independent instructions to run faster) (§14.7, and the Java Memory Model in Chapter 15).
3. **Ordering** — operations may appear to happen in a different order to another thread.

Everything in this chapter is a tool against one or more of these.

Key terms

Term	Meaning
Thread-safe	behaves correctly when used by several threads at once, with no extra coordination by callers
Shared mutable state	data that more than one thread can access *and* that can change — the source of all concurrency bugs
Critical section	code that touches shared mutable state and must not be executed by two threads at once
Lock / monitor	a mechanism that lets only one thread (or a controlled number) into a critical section
Blocking	a thread waiting (for a lock, I/O, a sleep) without doing work
Context switch	the OS pausing one thread and running another on the same core

---

14.2 Creating and starting threads

What it is

A `java.lang.Thread` object represents one thread. The *work* it does is a `Runnable` — an interface with one method, `void run()`. Since Java 21 there are two kinds of threads: **platform threads** (a thin wrapper over an **OS thread**, i.e. a thread created and scheduled by the operating system — this chapter) and **virtual threads** (lightweight threads managed by the JVM — Chapter 15 §15.4).

How it works

1. Describe the work: subclass `Thread` and override `run()`, or (better) write a `Runnable` — often a lambda.
2. Create the thread: `new Thread(runnable, "name")`, or the **builder** (an object you configure step by step and then ask to create the thing) `Thread.ofPlatform().name("x").unstarted(runnable)` (Java 21).
3. Call `start()`: the JVM creates a new OS thread, which calls `run()`. `start()` returns immediately; the new thread runs concurrently.
4. Optionally `join()`: the calling thread waits until the other thread has finished.
5. The thread ends when `run()` returns (or throws). A thread can be started **only once**.

`code/ch14/threads/CreatingThreads.java` (key parts):

```
static class Greeter extends Thread {                       // 1) subclass Thread
    Greeter() { super("greeter"); }
    @Override public void run() { System.out.println("1 subclass  running in " +
Thread.currentThread().getName()); }
}
static class Task implements Runnable { ... }               // 2) implement Runnable

Thread t1 = new Greeter();
Thread t2 = new Thread(new Task(), "worker-2");
Thread t3 = new Thread(() -> System.out.println(...), "worker-3");          // 3) lambda
Thread t4 = Thread.ofPlatform().name("worker-4").unstarted(() -> ...);      // 4) builder (Java 21)
for (Thread t : new Thread[]{t1, t2, t3, t4}) { t.start(); t.join(); }

Thread notStarted = new Thread(() -> ...);
notStarted.run();                                           // just a method call - runs in main!
t1.start();                                                 // second start -> IllegalThreadStateException

Thread a = new Thread(() -> System.out.println("   [order varies] A says hi"));
Thread b = new Thread(() -> System.out.println("   [order varies] B says hi"));
a.start(); b.start(); a.join(); b.join();

Thread daemon = new Thread(() -> { while (true) Thread.onSpinWait(); }, "background-spinner");
daemon.setDaemon(true);                                     // before start()
daemon.start();
```

Output:

```
main thread is called 'main'
1 subclass  running in greeter
2 Runnable  running in worker-2
3 lambda    running in worker-3
4 builder   running in worker-4
run() called directly executes in main
state after run(): NEW
start() twice -> IllegalThreadStateException
   [order varies] A says hi
   [order varies] B says hi
both finished (join guarantees this line comes last)
daemon? true, alive? true, id>0? true
main ends; the JVM exits even though the daemon still spins
```

What to notice:

- Calling `run()` **directly** does *not* create a thread: the code runs in `main`, and the `Thread` object stays `NEW`.
- A and B were started together, so either may print first — that's what `[order varies]` means. `join()` guarantees the line after it runs only when both are done.
- **Daemon threads** are background helpers (e.g. the **garbage collector**'s threads — the GC is the JVM component that automatically frees memory of objects no longer in use). The JVM exits when only daemon threads remain — so the infinite spinner didn't keep the program alive. Normal (non-daemon) threads *do* keep the JVM running.

Useful `Thread` methods

Method	Purpose
`Thread.currentThread()`	the thread running this code
`getName()` / `setName()`	names appear in logs and **thread dumps** (a snapshot listing every thread and what it is doing) — always name your threads
`threadId()` (19)	unique id (`getId()` is deprecated — still works but officially discouraged and may be removed)
`Thread.sleep(ms)` / `sleep(Duration)` (19)	pause the current thread; throws `InterruptedException`
`join()` / `join(ms)` / `join(Duration)` (19)	wait for another thread to finish
`isAlive()`, `getState()`	inspect
`setDaemon(true)`	background thread (set before `start`)
`setPriority(n)`	a *hint* (1–10) the OS may ignore — never rely on it for correctness
`setUncaughtExceptionHandler(h)`	what to do if `run()` throws (default: print **stack trace** — the chain of method calls that led to the error; the thread dies, others continue)
`Thread.onSpinWait()` (9)	hint inside **busy-wait** loops (loops that keep checking a condition instead of sleeping)
`Thread.yield()`	hint to let others run; rarely useful

Trade-offs: `extends Thread` vs `Runnable`

`extends Thread`	`implements Runnable` / lambda
Uses up your single **superclass** (Java classes can extend only one parent class)	Class can extend something else
Couples *what* to do with *how* it runs	The same task can run in a `Thread`, a pool, a virtual thread
Rarely the right choice	Preferred

When not to use raw threads

In application code you should almost never call `new Thread(...)` directly: creating a platform thread is expensive (an OS thread with a 1–2 MB stack reservation by default, depending on the platform), and unmanaged threads are hard to shut down and monitor. Use an `ExecutorService` (§14.11) or, for I/O-heavy tasks on Java 21+, virtual threads (Chapter 15).

Self-check

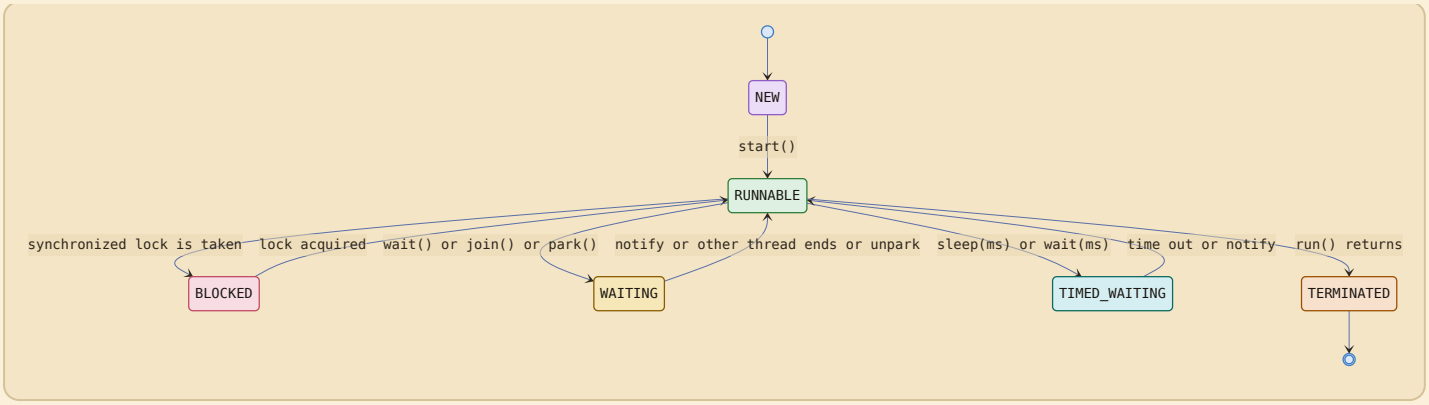
1. What's the difference between `t.start()` and `t.run()`? → `start` creates a new thread that calls `run`; calling `run` directly executes on the current thread.
2. Can you restart a finished thread? → No: `IllegalThreadStateException`. Create a new `Thread` (or reuse the `Runnable`).
3. Why doesn't a running daemon thread stop the JVM from exiting? → The JVM exits when all non-daemon threads have finished.

14.3 Thread lifecycle

What it is

`Thread.getState()` returns one of six `Thread.State` values:

State	Meaning	Typical cause
`NEW`	created, not yet started	`new Thread(...)`
`RUNNABLE`	running **or ready to run** (waiting for a CPU, or even blocked in OS-level I/O)	after `start()`
`BLOCKED`	waiting to **acquire a monitor lock** (`synchronized`) held by another thread	entering a `synchronized` block that's taken
`WAITING`	waiting indefinitely for another thread's action	`Object.wait()`, `join()`, `LockSupport.park()` (low-level "pause this thread until woken"; used by `Lock`, latches, queues)
`TIMED_WAITING`	waiting with a timeout	`sleep(ms)`, `wait(ms)`, `join(ms)`, `tryLock(timeout)`
`TERMINATED`	`run()` has finished (normally or by exception)	end of `run()`



`code/ch14/threads/Lifecycle.java` drives one thread through every state, polling `getState()` until each is reached (so the output is deterministic):

```
Thread t = new Thread(() -> {
    while (keepSpinning) Thread.onSpinWait();   // RUNNABLE
    Thread.sleep(300);                          // TIMED_WAITING
    go.await();                                 // WAITING (a CountDownLatch parks the thread)
    synchronized (LOCK) { }                     // BLOCKED while main holds LOCK
});
```

Output:

```
after new:              NEW
while spinning:         RUNNABLE
during sleep(300):      TIMED_WAITING
during latch.await():   WAITING
waiting for a monitor:  BLOCKED
after run() returns:    TERMINATED
```

Note the subtle split: waiting for a `synchronized` monitor is `BLOCKED`, but waiting for a `ReentrantLock` is `WAITING` (locks park the thread). A thread blocked reading a socket shows as `RUNNABLE` — the JVM doesn't distinguish OS-level I/O waits. Thread dumps (`jstack <pid>`, `jcmd <pid> Thread.print` — `pid` is the OS *process id*) show these states and are the first tool for diagnosing a hung program. More: LLD §12.3.

14.4 Interruption: cooperative cancellation

What it is

Java has no safe way to forcibly kill a thread (`Thread.stop()` was deprecated because it could leave objects half-updated; since Java 20 it throws `UnsupportedOperationException`, and JDK 26 removed it entirely). Instead, one thread **requests** another to stop by calling `t.interrupt()`, which sets the target's **interrupt flag** (a per-thread true/false marker meaning "someone asked you to stop"). The target decides how and when to stop. Analogy: tapping a colleague on the shoulder, not pulling their chair away.

How it works

1. `t.interrupt()` sets `t`'s interrupt flag.
2. If `t` is blocked in an **interruptible** method (`sleep`, `wait`, `join`, `BlockingQueue.take`, `Lock.lockInterruptibly`, `Future.get`…), that method wakes up and throws **`InterruptedException`** — and **clears the flag**.
3. If `t` is busy computing, nothing happens automatically; the code must check `Thread.currentThread().isInterrupted()` periodically.
4. `Thread.interrupted()` (static) returns the current thread's flag **and clears it**; `isInterrupted()` only reads it.

`code/ch14/threads/Interruption.java` (key parts):

```
try {
    Thread.sleep(60_000);
} catch (InterruptedException e) {
    // flag was cleared when the exception was thrown
    Thread.currentThread().interrupt();          // restore it so callers can see the request
}

while (!Thread.currentThread().isInterrupted()) loops++;     // busy code polls the flag

Thread.currentThread().interrupt();
Thread.interrupted();   // true  (and clears)
Thread.interrupted();   // false
```

Output:

```
sleeper: woke up early, flag now = false
sleeper: flag restored = true
worker: saw the interrupt and stopped cleanly (loops > 0: true)
interrupted() first call  = true
interrupted() second call = false
```

Common mistakes

- **Swallowing** `InterruptedException`: `catch (InterruptedException e) {}`. The cancellation request is lost; `shutdownNow()` on a pool then can't stop the task. Either **rethrow** it (declare `throws InterruptedException`) or **restore the flag** with `Thread.currentThread().interrupt()` — JCIP's rule, and why every `catch` in this chapter's code does so.
- Using a plain `boolean` field as a stop flag without `volatile` (§14.7). Interruption is preferable because it also wakes blocked threads.
- Assuming interruption stops *non-interruptible* blocking such as classic `java.io` socket reads — closing the socket is the way to unblock those.

In the interview

30-second answer: "Interruption is cooperative cancellation: `interrupt()` sets a flag; blocking methods respond by throwing `InterruptedException` and clearing the flag; long computations should poll `isInterrupted()`. When I catch `InterruptedException` and can't rethrow it, I restore the flag with `Thread.currentThread().interrupt()`."

- *Why was `Thread.stop` removed?* → It released all locks mid-operation, exposing corrupted state.

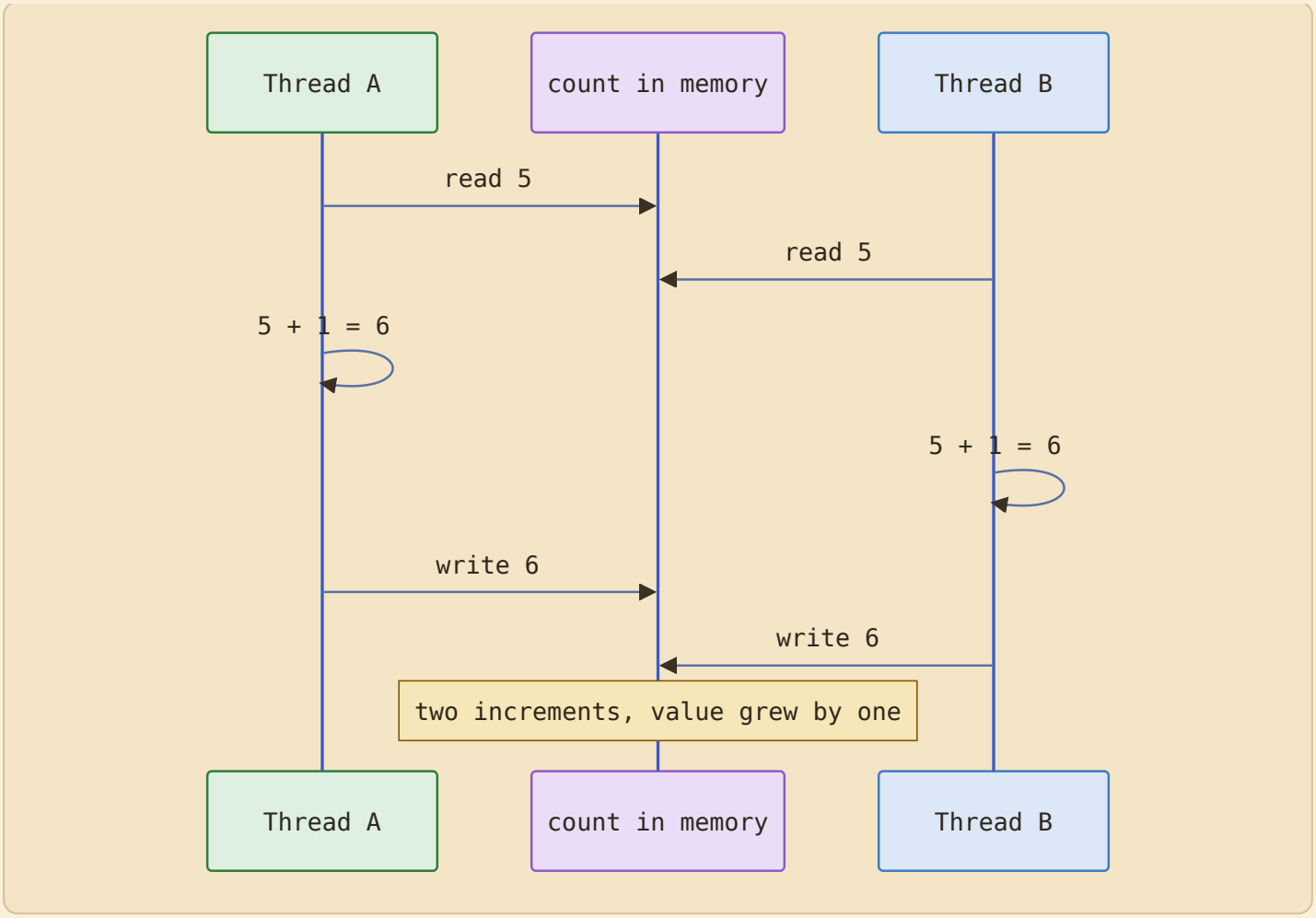
14.5 Race conditions

What it is

A **race condition** is a bug where the result depends on the unlucky timing (**interleaving** — how the steps of different threads happen to get mixed together) of threads. The two classic shapes:

1. **Read-modify-write**: `count++` is really *read* `count` → *add* 1 → *write* back. If two threads both read 5, both write 6: one increment is **lost**.
2. **Check-then-act**: `if (seat.isFree()) seat.book(me);` — between the check and the act another thread can book the same seat. **Lazy initialisation** (creating an object only the first time it's needed: `if (instance == null) instance = new X();`) has the same shape.

A **data race**, more narrowly, is when two threads access the same variable without synchronisation and at least one writes. Data races also cause *visibility* problems (§14.7).



`code/ch14/race/RaceCondition.java` (key parts) — 4 threads each add 1,000,000:

```
run(() -> { for (int i = 0; i < PER_THREAD; i++) unsafeCount++; });
run(() -> { for (int i = 0; i < PER_THREAD; i++) synchronized (LOCK) { syncCount++; } });
run(() -> { for (int i = 0; i < PER_THREAD; i++) atomicCount.incrementAndGet(); });
run(() -> { for (int i = 0; i < PER_THREAD; i++) adder.increment(); });

void bookUnsafe(String who) {
    if (owner == null) {           // check ...
        sleepQuietly();            // (real work would widen the window like this)
        owner = who;               // ... then act
        successes++;
    }
}
synchronized void bookSafe(String who) { ...same body... }
```

Output (two runs; the first line differs every time):

```
unsynchronized count++ : 1,127,175   [nondeterministic - lost updates, changes every run]
synchronized           : 4,000,000
AtomicInteger          : 4,000,000
LongAdder              : 4,000,000
unsafe booking attempt 1: successful bookings = 2   [nondeterministic - 2 means double-booked]
unsafe booking attempt 2: successful bookings = 2   [nondeterministic - 2 means double-booked]
unsafe booking attempt 3: successful bookings = 2   [nondeterministic - 2 means double-booked]
synchronized booking: successful bookings = 1
```

```
unsynchronized count++ : 1,927,211   [nondeterministic - lost updates, changes every run]
...
```

Almost **three quarters of the increments vanished** in the first run, and a different number in the second. The seat was double-booked every time. Both fixed versions are exact.

Why these bugs are nasty

- They may **never appear** in tests on a laptop and then appear under **production** load (production = the real system that real users rely on).
- Adding a `println` to debug them changes the timing and often hides them ("Heisenbugs").
- The JIT compiler (the JVM's *just-in-time* compiler, which turns hot code into machine code while the program runs) may keep a non-volatile field in a **register** (a tiny storage slot inside the CPU core) for a whole loop, making results even stranger than simple interleaving suggests.

How to fix (preview of the rest of the chapter)

1. **Don't share** — confine data to one thread (local variables, `ThreadLocal` — a variable that holds a separate value per thread) — §14.14.
2. **Don't mutate** — share only immutable objects (records with immutable fields) — §14.14.
3. **Synchronise** every access to shared mutable state — `synchronized` (§14.6), locks (§14.9).
4. **Use atomic or concurrent building blocks** — `AtomicInteger`, `ConcurrentHashMap.merge` (§14.10, §14.12).

More examples and the "critical section" framing: LLD §12.5.

Self-check

1. Why is `count++` not atomic? → It's three steps: read, add, write.
2. Name the other classic race shape. → Check-then-act (e.g. lazy initialisation, "if free then book").
3. Does making a field `volatile` fix `count++`? → No (§14.7).

14.6 `synchronized` and intrinsic locks

What it is

Every Java object has a built-in lock called its **intrinsic lock** or **monitor**. A `synchronized` block or method acquires it on entry and releases it on exit — even if an exception is thrown. While one thread holds a monitor, other threads trying to enter a block synchronised on the *same object* wait (state `BLOCKED`).

`synchronized` gives two guarantees:

1. **Mutual exclusion (atomicity)**: only one thread at a time runs code guarded by that lock. (*Atomic* = happens as one indivisible step that other threads can't see half-done.)
2. **Visibility**: everything a thread wrote before releasing a lock is visible to the next thread that acquires the *same* lock (a *happens-before* edge — a guaranteed "this write is visible to that read" ordering — Chapter 15 §15.8).

How it works

Form	Lock used
`synchronized void m()` (instance method)	`this`
`static synchronized void m()`	the `Class` object (`Counter.class`)
`synchronized (obj) { ... }`	`obj` — any object you choose

Intrinsic locks are **reentrant** (re-enterable by the thread that already holds them): a thread that holds a lock can enter another block guarded by the same lock (e.g. a synchronized method calling another synchronized method on `this`) without deadlocking itself. The JVM counts the entries.

`code/ch14/sync/SynchronizedBasics.java` (key parts):

```
static class Counter {
    private int value;
    public synchronized void increment() { value++; }                 // locks this
    public void incrementBlock() { synchronized (this) { value++; } }  // same lock
    public synchronized int get() { return value; }                   // reads need it too
    public synchronized void addTwice() { increment(); increment(); }  // reentrant
    public static synchronized String describe() { ... }              // locks Counter.class
}
synchronized (c) { Thread.holdsLock(c); Thread.holdsLock(Counter.class); }

Object lockA = new Object(), lockB = new Object();       // different locks don't exclude each other
```

Output:

```
4 threads x 500,000 synchronized increments = 2000000
reentrant call worked, value = 2000002
static synchronized locks Counter.class
inside synchronized(c): holdsLock(c)=true holdsLock(Counter.class)=false
different locks -> both threads were inside at once (latch reached 0: true)
```

The last line proves the most common mistake: protection only works if **all** code touching the data uses **the same lock**. Here the two threads each held a different lock, so both were "inside" at once (the latch could only reach 0 because both got in).

Guidelines

- **Guard every access** — reads too. An unsynchronised `get()` might return a **stale** (out-of-date) value (visibility).
- **Keep critical sections short**; never do slow I/O (input/output: disk, network) or call unknown code (**callbacks** and **listeners** — functions other code registered to be called when something happens) while holding a lock — that's how deadlocks and stalls happen.
- **Prefer a private final lock object** (`private final Object lock = new Object();`) over `synchronized(this)` in library code, so outside code can't grab your lock and interfere.
- **Never lock on** `String` literals, boxed integers (`Integer` cache!) or other shared/**interned** objects (objects the JVM reuses so one copy is shared program-wide), and not on a field that can be reassigned.
- Document which lock guards which fields (JCIP's `@GuardedBy("lock")` convention — an **annotation**, i.e. a `@` label on code for tools and readers).
- Since Java 24 (JEP 491 — a JEP, *JDK Enhancement Proposal*, is the numbered design document for a Java feature), `synchronized` no longer "pins" (glues to one OS thread) virtual threads — details in Chapter 15 §15.5.

Common mistakes

- Synchronising the writer but not the reader.
- Synchronising two methods on *different* objects and assuming they're mutually exclusive (static vs instance synchronized methods use different locks!).
- Thinking `Collections.synchronizedList(list)` makes *iteration* safe — you must still `synchronized (list) { for (...) }` (§14.12).

In the interview

30-second answer: "`synchronized` acquires the monitor of an object — `this` for instance methods, the `Class` for static ones, or any object for a block. It gives mutual exclusion and visibility, it's reentrant, and the lock is released automatically, even on exceptions. All accesses to a piece of shared state must use the same lock."

- *Is `synchronized` reentrant?* → Yes; the same thread can re-acquire a lock it holds.
- *Static and instance synchronized methods — do they block each other?* → No; different locks (`Class` object vs instance).

Deeper treatment with diagrams: LLD §12.7.

14.7 `volatile` and visibility

What it is

Modern CPUs keep copies of memory in per-core caches, and both the compiler and the CPU reorder instructions for speed. So without synchronisation, a value written by one thread **may never become visible** to another thread, or become visible out of order. A field declared `volatile` fixes visibility:

- Every read of a volatile field sees the **most recent write** by any thread.
- Writes made *before* a volatile write become visible to a thread that later *reads* that volatile field (it creates a happens-before edge — Chapter 15 §15.8).
- `volatile long` / `double` reads and writes are atomic (plain 64-bit writes are allowed to be split in two by the JLS — the *Java Language Specification*, the official rulebook of the language).

`volatile` does **not** provide mutual exclusion: `volatileCounter++` is still read-modify-write.

`code/ch14/sync/VolatileFlag.java` (key parts):

```
private static volatile boolean running = true;          // without volatile the worker MAY spin forever

Thread worker = new Thread(() -> { long spins = 0; while (running) spins++; ... });
worker.start();
Thread.sleep(200);
running = false;                                         // guaranteed to be seen

// 4 threads x 1,000,000 of volatileCounter++
```

Output:

```
worker saw running=false and stopped (spins > 0: true)
volatile count++ from 4 threads: 1,113,395 of 4,000,000   [nondeterministic - still loses updates]
```

(Why "may spin forever" without `volatile`: the JIT is allowed to read `running` once and turn the loop into `while (true)`. Whether it does depends on the JVM and timing, which is exactly why you can't test your way to safety — follow the rules instead.)

When to use `volatile`

- A **status flag** written by one thread and read by others (`shutdownRequested`).
- **Publishing** an immutable object reference (`volatile Config current;` replaced wholesale).
- The double-checked-locking **idiom** (a standard code pattern; here, lazy initialisation that only locks the first time) (Chapter 15 §15.8).

When not to use it

When the new value depends on the old (`count++`, `balance -= x`), or several fields must change together — use locks or atomics.

	`volatile`	`synchronized`	`AtomicInteger`
Visibility	✓	✓	✓
Atomic compound update (`++`)	✗	✓	✓ (single variable)
Multi-variable invariants	✗	✓	✗
Blocking	never	may block	never (retries)

More: LLD §12.6.

14.8 `wait` / `notify`: guarded blocks

What it is

Sometimes a thread must wait until a *condition* becomes true ("the queue is not empty"). Every object's monitor has a **wait set**:

- `obj.wait()` — releases `obj`'s lock and sleeps until notified (or interrupted, or a *spurious wakeup* — waking up for no reason, which the JVM is allowed to do); then re-acquires the lock before returning.
- `obj.notify()` — wakes **one** waiting thread (which one is unspecified). `obj.notifyAll()` wakes **all** of them.
- All three must be called **while holding `obj`'s lock**, or you get `IllegalMonitorStateException`.

How it works — the guarded-block idiom

1. Acquire the lock (`synchronized`).
2. `while` the condition is false, `wait()`. (A `while`, not `if`: after waking, the condition may already be false again — another thread got there first — or the wakeup may be spurious.)
3. Do the action.
4. After changing state that others wait on, `notifyAll()`.

`code/ch14/sync/WaitNotify.java` — a two-slot buffer between a **producer** (a thread that creates work items) and a **consumer** (a thread that processes them):

```
public synchronized void put(T item) throws InterruptedException {
    while (items.size() == capacity) wait();     // wait for "not full"
    items.add(item);
    notifyAll();                                 // wake consumers
}
public synchronized T take() throws InterruptedException {
    while (items.isEmpty()) wait();              // wait for "not empty"
    T item = items.remove();
    notifyAll();                                 // wake producers
    return item;
}
```

Output:

```
consumer received in order: 1 2 3 4 5 6 7 8 9 10
sum = 55 (1..10 = 55)
wait() without the lock -> IllegalMonitorStateException
```

The producer ends with a **poison pill** (`-1`), a special value meaning "no more items" — a common way to stop consumers.

When not to use it

In real code, use `BlockingQueue` (§14.12), `CountDownLatch`/`Semaphore` (§14.13) or `Condition` (§14.9), which are easier to get right. Know `wait`/`notify` for interviews and legacy code. `notify()` (single) is only safe when all waiters wait for the same condition and any one can proceed; otherwise a "lost wakeup" can leave threads stuck — `notifyAll()` is the safe default. Step-by-step coverage: LLD §12.13 and the blocking queue built two ways in LLD §13.8.

Self-check

1. Why `while` and not `if` around `wait()`? → Spurious wakeups, and the condition may have changed before the thread re-acquired the lock.
2. Does `wait()` release the lock? → Yes (only that object's lock); `sleep()` does not release any lock.

14.9 Explicit locks: `ReentrantLock`, `Condition`, `ReadWriteLock`, `StampedLock`

What it is

`java.util.concurrent.locks` (Java 5) provides lock *objects* with the same mutual-exclusion and visibility guarantees as `synchronized`, plus features `synchronized` lacks:

Feature	`synchronized`	`ReentrantLock`
Released automatically	✓ (end of block)	✗ — must `unlock()` in `finally`
Try without waiting / with timeout	✗	`tryLock()`, `tryLock(time, unit)`
Interruptible wait for the lock	✗	`lockInterruptibly()`
Fairness (FIFO-ish order — first in, first out: longest waiter goes next)	✗	`new ReentrantLock(true)`
Multiple wait conditions	one wait set per object	many `Condition`s per lock
Lock/unlock in different methods	✗	✓ (use with care)
Inspect (`isLocked`, `getHoldCount`, queued threads)	✗	✓
Syntax	concise, can't forget to unlock	verbose

How it works

```
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();          // ALWAYS in finally, or an exception leaves it locked forever
}
```

`code/ch14/locks/LocksDemo.java` (key parts):

```
lock.lock(); try { lock.lock(); try { ... } finally { lock.unlock(); } } finally { lock.unlock(); }  // reentrant
boolean got = lock.tryLock(150, TimeUnit.MILLISECONDS);      // another thread holds it
lock.lockInterruptibly();                                     // in a waiter thread, then waiter.interrupt()

Condition ready = l.newCondition();
while (!flag[0]) ready.await();                               // like wait(); signalAll() like notifyAll()

ReadWriteLock rw = new ReentrantReadWriteLock();
rw.readLock().lock();   // 3 readers wait at a CyclicBarrier(3) while holding the read lock
rw.writeLock().lock();  // exclusive: a reader's tryLock fails
```

Output:

```
holding lock: isHeldByCurrentThread=true holdCount=1
re-entered: holdCount=2
tryLock(150ms) while another thread holds it -> false after >=150ms: true
waiter: interrupted while waiting for the lock
Condition: woke up, flag = true
ReadWriteLock: readers holding the read lock simultaneously = 3
writer holds write lock; can a reader get in now? false
```

The barrier trick proves read-sharing: a `CyclicBarrier(3)` only opens when three threads are waiting at it, and they were all holding the read lock at the time.

`ReadWriteLock` and `StampedLock`

- `ReentrantReadWriteLock`: many readers **or** one writer. Worth it when reads vastly outnumber writes and critical sections are not tiny (e.g. an in-memory catalogue refreshed hourly). For small, fast sections a plain lock or `ConcurrentHashMap` is usually faster.
- `StampedLock` (Java 8): adds **optimistic reads** — read without locking, then `validate(stamp)` to check that no write happened; fall back to a real read lock if one did. Very fast for read-mostly data, but **not reentrant** and doesn't support `Condition`s — for experts. Examples: LLD §12.10.

When not to use explicit locks

If you don't need `tryLock`, timeouts, interruptibility, fairness or several conditions, **prefer `synchronized`**: shorter, impossible to forget to unlock, and (since Java 24) equally friendly to virtual threads. Fair locks have much lower throughput; use them only when starvation is a real problem.

In the interview

30-second answer: "`ReentrantLock` offers what `synchronized` doesn't: `tryLock` with timeouts to avoid deadlock, interruptible acquisition, optional fairness and multiple `Condition`s. The price is that I must unlock in `finally`. I default to `synchronized` and switch to `ReentrantLock` when I need those features; `ReadWriteLock` for read-heavy data, `StampedLock` for optimistic reads."

- *How does `tryLock` help against deadlock?* → A thread that can't get the second lock backs off, releases the first and retries, breaking the circular wait (Chapter 15 §15.9).

14.10 Atomic variables

What it is

`java.util.concurrent.atomic` classes (`AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference<V>`, `AtomicIntegerArray`, …) offer single-variable operations that are **atomic without locks**. They're built on the CPU's **compare-and-set (CAS)** instruction (a single hardware step that checks and updates a memory location together): "if the value is still `expected`, set it to `new` and return true; otherwise change nothing and return false". A thread that loses the race simply **retries**; no thread ever blocks.

Analogy: editing a shared document with version numbers — "save my change only if the document is still version 17; otherwise reload and try again."

`code/ch14/atomic/AtomicDemo.java` (key parts):

```
AtomicInteger n = new AtomicInteger(10);
n.incrementAndGet();  n.getAndIncrement();  n.addAndGet(5);
n.compareAndSet(17, 100);                  // true: value was 17
n.compareAndSet(17, 200);                  // false: value is 100 now
n.updateAndGet(x -> x * 2);
n.accumulateAndGet(50, Math::max);

do { prev = hits.get(); next = prev + 1; } while (!hits.compareAndSet(prev, next));   // what updateAndGet does

AtomicReference<Range> range = new AtomicReference<>(new Range(0, 10));
range.updateAndGet(r -> new Range(r.low() + 1, r.high() + 1));     // 4 threads x 10,000

LongAdder requests = new LongAdder();       // 8 threads x 100,000 increments
```

Output:

```
incrementAndGet -> 11
getAndIncrement -> 11 (now 12)
addAndGet(5)    -> 17
CAS 17->100     -> true, value 100
CAS 17->200     -> false, value 100 (expected value was stale)
updateAndGet x2 -> 200
accumulateAndGet(max with 50) -> 200
hand-written CAS loop -> 1
AtomicReference after 40,000 shifts -> Range[low=40000, high=40010]
LongAdder sum -> 800000
```

- `AtomicReference` + an **immutable** (unchangeable once built) record lets you update several related values (low and high) atomically: build a new object, CAS it in. The **invariant** (a rule about the data that must always be true) `high - low == 10` always holds.
- The function passed to `updateAndGet` may run **more than once** (on retries), so it must have no side effects.
- **`LongAdder`** (Java 8) spreads increments over several internal cells to avoid CAS **contention** (many threads fighting over the same variable or lock); `sum()` adds them up. Much faster than `AtomicLong` for hot counters (**metrics** — numbers a service records about itself, like request counts), but `sum()` is not an atomic snapshot and there is no `compareAndSet`.

Trade-offs

	Locks	Atomics
Scope	any number of variables	one variable (or one reference to an immutable object)
Under low contention	slower	very fast
Under very high contention	threads sleep	many CAS retries burn CPU (`LongAdder` helps)
Deadlock possible?	yes	no

CAS internals, the ABA problem (a value changes A→B→A, so CAS wrongly thinks nothing happened) and `AtomicStampedReference`: LLD §12.15.

Self-check

1. What does `compareAndSet` return if the current value isn't the expected one? → `false`, and the value is unchanged.
2. When is `LongAdder` better than `AtomicLong`? → Many threads updating, rare reads.
3. Why must the `updateAndGet` lambda be side-effect free? → It may be retried.

14.11 Executors and thread pools

What it is

An **`ExecutorService`** separates **submitting tasks** from **running them**. You hand it `Runnable`s or `Callable`s; it runs them on a **thread pool** — a set of reusable worker threads fed from a **work queue**. Analogy: a taxi rank. Passengers (tasks) queue; a fixed number of taxis (threads) take the next passenger when free, instead of buying a new car for every trip.

- **`Runnable`** — `void run()`, can't return a value or throw checked exceptions.
- **`Callable<V>`** — `V call() throws Exception`, returns a result. (**Checked exceptions** are the ones the compiler forces you to catch or declare.)
- **`Future<V>`** — a handle to a pending result: `get()` (blocks), `get(timeout, unit)`, `isDone()`, `cancel(mayInterrupt)`, `isCancelled()`, and since Java 19 `state()`, `resultNow()`, `exceptionNow()`.

Why it matters

1. **Reuse**: creating an OS thread per task costs time and memory; pools **amortise** it (spread the one-off cost over many tasks).
2. **Bounded resources** (with a fixed upper limit): a pool of 20 threads can't create 10,000 threads during a traffic spike.
3. **Lifecycle management**: one place to shut everything down, wait for completion, collect results and exceptions.

How it works

1. Create a pool (factory methods below, or a `ThreadPoolExecutor` directly).
2. `execute(runnable)` (fire and forget) or `submit(callable)` → `Future`.
3. `invokeAll(tasks)` → all `Future`s when every task is done; `invokeAny(tasks)` → the first successful result, cancelling the rest.
4. Shut down: `shutdown()` (stop accepting, finish queued), `awaitTermination(timeout)` (wait), `shutdownNow()` (interrupt running tasks, return queued ones). Since **Java 19** `ExecutorService` is `AutoCloseable`: **`close()`** = shutdown + wait, so try-with-resources (the `try (X x = …) { }` form that closes `x` automatically) works.

`code/ch14/executors/ExecutorBasics.java` (key parts):

```
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {
    pool.execute(() -> ...);                                        // 6 tasks
    Future<Integer> f = pool.submit(() -> { Thread.sleep(100); return 6 * 7; });
    f.isDone(); f.get(); f.state(); f.resultNow();

    Future<Integer> bad = pool.submit(() -> Integer.parseInt("forty-two"));
    bad.get();                                                      // ExecutionException wrapping the cause

    Future<String> slow = pool.submit(() -> { Thread.sleep(10_000); return "late"; });
    slow.get(100, TimeUnit.MILLISECONDS);                           // TimeoutException
    slow.cancel(true);                                              // interrupts the sleeping task

    pool.invokeAll(List.of(() -> "a", () -> { Thread.sleep(50); return "b"; }, () -> "c"));
    pool.invokeAny(List.of(() -> { Thread.sleep(300); return "slow mirror"; },
                           () -> { Thread.sleep(10);  return "fast mirror"; }));
}                                                                   // close(): waits for all tasks

ex.shutdown(); ex.awaitTermination(5, TimeUnit.SECONDS); ex.submit(...);   // rejected

try (ScheduledExecutorService sched = Executors.newScheduledThreadPool(1)) {
    sched.scheduleAtFixedRate(() -> ..., 50, 100, TimeUnit.MILLISECONDS);
    sched.schedule(() -> "delayed hello", 50, TimeUnit.MILLISECONDS).get();
}
```

Output:

```
6 tasks ran on at most 3 threads: true
isDone right after submit? false
get() blocks, then returns 42
state = SUCCESS, resultNow = 42
get() -> ExecutionException caused by NumberFormatException
get(100ms) -> TimeoutException; cancel(true) = true, isCancelled = true
invokeAll results in order: abc
invokeAny -> fast mirror
pool closed (all tasks finished)
single-thread executor task
awaitTermination -> true, isTerminated = true
submit after shutdown -> RejectedExecutionException
tick 1 at ~50 ms   [timing approximate]
tick 2 at ~150 ms   [timing approximate]
tick 3 at ~250 ms   [timing approximate]
delayed hello
```

- An exception inside a task **doesn't crash anything**: it's stored in the `Future` and rethrown by `get()` as `ExecutionException` with the original as its *cause*. If you never call `get()`, a `submit`ted task's exception is silently lost — a frequent source of "the job just stopped working" mysteries. (`execute()`d tasks, by contrast, let the exception reach the thread's uncaught-exception handler, which prints it.)
- `invokeAll` returns results in **submission order**, not completion order.
- `scheduleAtFixedRate` runs at a fixed *rate* (start times 100 ms apart); `scheduleWithFixedDelay` waits a fixed *delay* after each run ends. If a periodic task throws, its future runs are silently cancelled.

The `Executors` factory methods

Factory	Pool	Queue	Watch out
`newFixedThreadPool(n)`	exactly n threads	**unbounded** `LinkedBlockingQueue`	a flood of tasks piles up in memory
`newCachedThreadPool()`	0…∞ threads, idle ones die after 60 s	hand-off (`SynchronousQueue`)	a flood creates unbounded threads
`newSingleThreadExecutor()`	1 thread	unbounded	tasks run sequentially in order
`newScheduledThreadPool(n)`	n threads	delay queue	periodic tasks die silently on exception
`newWorkStealingPool()` (8)	a `ForkJoinPool`	per-thread deques	for many small CPU tasks
`newVirtualThreadPerTaskExecutor()` (21)	a new virtual thread per task	none	not a pool at all — Chapter 15

Configuring `ThreadPoolExecutor`

For production services, build the pool explicitly so every limit is a decision. The rules for a new task are:

1. Fewer than **core** threads running → start a new thread.
2. Otherwise → put it in the **queue**.
3. Queue full and fewer than **max** threads → start an extra thread.
4. Otherwise → call the **rejection policy**.

`code/ch14/executors/PoolConfig.java` (key parts) — core 2, max 3, queue capacity 2, 7 tasks that all block:

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
        2, 3, 30, TimeUnit.SECONDS,
        new ArrayBlockingQueue<>(2),
        new ThreadPoolExecutor.AbortPolicy());
```

Output:

```
available processors: 14   [machine-dependent]
task 1 accepted  (threads=1, queued=0)
task 2 accepted  (threads=2, queued=0)
task 3 accepted  (threads=2, queued=1)
task 4 accepted  (threads=2, queued=2)
task 5 accepted  (threads=3, queued=2)
task 6 REJECTED  (threads=3, queued=2)
task 7 REJECTED  (threads=3, queued=2)
completed tasks = 5
third task ran in: main (the caller)
```

Note the surprise in step 3: the pool grows beyond the core size only when the queue is **full** — so with an unbounded queue, `max` is never used.

Rejection policies: `AbortPolicy` (default, throws `RejectedExecutionException`), `CallerRunsPolicy` (the submitting thread runs the task itself — natural **back-pressure** (a signal that pushes back on whoever sends work too fast, slowing them down) that slows the producer; shown in the last line), `DiscardPolicy` (silently drop), `DiscardOldestPolicy` (drop the oldest queued task).

Sizing (JCIP): for **CPU-bound** work (limited by processor speed, not by waiting), about N_{cpu} **(+1)** threads — more just adds context switches. For tasks that wait, $N_{threads} \approx N_{cpu} \times (1 + \text{wait time} / \text{compute time})$; e.g. 8 cores, tasks that wait 90 ms per 10 ms of CPU → ~80 threads. Then measure. For **I/O-bound** tasks (ones that spend most of their time waiting on disk or network) on Java 21+, virtual threads usually remove the need for this calculation.

Common mistakes

- Forgetting to shut down a pool → non-daemon worker threads keep the JVM alive forever.
- Unbounded queues/pools in servers → `OutOfMemoryError` (the JVM ran out of heap memory) under load instead of a clean rejection.
- Tasks that wait on other tasks **in the same bounded pool** → **thread-starvation deadlock** when all threads wait for queued tasks that can't run.
- Ignoring `Future`s from `submit` → exceptions disappear.
- Sharing one pool for fast and slow work → slow tasks hog all threads (use separate pools, the *bulkhead* pattern — named after a ship's watertight walls: a failure in one compartment can't sink the rest).

In the interview

> **30-second answer:** "I use an `ExecutorService` rather than raw threads: tasks go in as `Runnable`/`Callable`, results and exceptions come back through `Future`, and I shut it down with `shutdown`/`awaitTermination` or try-with-resources on Java 19+. In production I configure `ThreadPoolExecutor` explicitly — bounded queue, a sized max, a rejection policy like `CallerRunsPolicy` for back-pressure, and named threads."

- *`shutdown` vs `shutdownNow`?* → Graceful (queued tasks still run) vs interrupt running tasks and return the queued ones.
- *Why is `newFixedThreadPool` risky?* → Its queue is unbounded.
- *`submit` vs `execute`?* → `submit` returns a `Future` and captures exceptions; `execute` returns nothing and exceptions go to the uncaught handler.

Design discussion: LLD §12.21 Thread pool pattern. Exam tables (`TimeUnit`, `ScheduledExecutorService` methods): OCP Chapter 13.

14.12 Concurrent collections

What it is

The ordinary collections of Chapter 10 (`ArrayList`, `HashMap`) are not thread-safe. There are two ways to share collections:

1. **Synchronised wrappers** — `Collections.synchronizedList/Map/Set(...)`, and the legacy `Vector`/`Hashtable`: every method takes one lock. Simple, but only one thread at a time, and **compound actions and iteration still need external locking**.
2. **Concurrent collections** (`java.util.concurrent`) — designed for concurrent access with **fine-grained locking** (many small locks, each guarding a small part) or **lock-free** algorithms (built on CAS, so no thread ever waits for a lock), plus **atomic compound operations** built in.

Class	Use for	Notes
`ConcurrentHashMap`	shared maps, caches, counters	atomic `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`; reads don't block; **no null keys/values**; weakly consistent iterators (they never throw but may or may not see changes made during iteration)
`ConcurrentSkipListMap` / `Set`	sorted concurrent map/set	concurrent `TreeMap`; O(log n) (time grows with the logarithm of size — very slowly)
`CopyOnWriteArrayList` / `Set`	read-mostly lists (listeners, config)	every write copies the array; iterators see a snapshot, never throw CME (`ConcurrentModificationException`)
`ArrayBlockingQueue`	bounded producer-consumer	`put` waits when full, `take` waits when empty
`LinkedBlockingQueue`	producer-consumer (optionally bounded)	unbounded by default
`PriorityBlockingQueue`, `DelayQueue`	priority / delayed tasks	unbounded
`SynchronousQueue`	direct hand-off (capacity 0)	used by cached thread pools
`LinkedTransferQueue`	producer can wait until consumed	
`ConcurrentLinkedQueue` / `Deque` (double-ended queue)	non-blocking queue	lock-free; `poll` returns `null` when empty

`BlockingQueue` methods come in four flavours: throw (`add`/`remove`), special value (`offer`/`poll` → `false`/`null`), **block** (`put`/`take`), and **time out** (`offer(e, t, unit)`/`poll(t, unit)`).

`code/ch14/collections/ConcurrentCollections.java` (key parts):

```
counts.merge(w, 1, Integer::sum);                                  // atomic per key

Integer old = syncMap.get(w);                                      // each call is thread-safe,
syncMap.put(w, old == null ? 1 : old + 1);                         // the combination is NOT

groups.computeIfAbsent(w.substring(0, 1), k -> new CopyOnWriteArrayList<>()).add(w);
counts.put("x", null);                                             // NullPointerException

for (String s : plain) if (s.equals("a")) plain.add("d");          // ArrayList: CME
for (String s : cow)   { seen++; if (s.equals("a")) cow.add("d"); } // snapshot iteration

BlockingQueue<String> queue = new ArrayBlockingQueue<>(3);         // producer put()s, consumer take()s
```

Output:

```
ConcurrentHashMap merge : {and=80000, bat=40000, cat=40000, hat=40000, the=120000}
synchronizedMap get+put : {and=55878, bat=32934, cat=33358, hat=33388, the=76273}   [nondeterministic - lost
updates; expected the=120000]
computeIfAbsent sizes   : {a=8, b=4, c=4, h=4, t=12}
ConcurrentHashMap rejects null values -> NullPointerException
ArrayList modify during for-each -> ConcurrentModificationException
CopyOnWriteArrayList: iterator saw 3 elements, list now [a, b, c, d]
BlockingQueue consumed  : [job-1, job-2, job-3, job-4, job-5]
offer on full queue     : offer(2) = false (returns false instead of blocking)
poll with timeout, empty: null
ConcurrentSkipListMap sorted descending? true
ConcurrentLinkedQueue size after 4 x 1000 offers = 4000
```

The second line is the key lesson: a thread-safe collection does **not** make *your sequence of calls* atomic. `get` then `put` is check-then-act; a third of the counts were lost. Use the atomic methods (`merge`, `compute`, `computeIfAbsent`, `putIfAbsent`) — or hold one lock around the whole sequence.

Common mistakes

- Using `containsKey` + `put` (or `get` + `put`) on a `ConcurrentHashMap` — use `putIfAbsent` / `computeIfAbsent` / `merge`.
- Long or blocking work inside `computeIfAbsent`'s function — it runs while that key's **hash bin** (the internal slot that keys with the same hash land in) is locked, so other updates to keys in the same bin wait. Never modify the same map from inside the function.
- `size()` on a busy `ConcurrentHashMap` is an estimate at that moment; don't use it for logic.
- `CopyOnWriteArrayList` for write-heavy lists — each add copies the whole array: O(n).
- Iterating a `synchronizedList` without `synchronized (list) { ... }` → `ConcurrentModificationException` or worse.

In the interview

30-second answer: "For shared maps I use `ConcurrentHashMap` and its atomic methods — `merge`, `computeIfAbsent` — because separate `get` and `put` calls are a check-then-act race even on a thread-safe map. `CopyOnWriteArrayList` for read-mostly lists like listeners, `BlockingQueue` for producer-consumer, and the skip-list classes (a **skip list** is a sorted linked list with extra "express lane" links for fast search) when I need ordering."

- *How does `ConcurrentHashMap` scale?* → Since Java 8: CAS for inserting into empty bins and a lock on only the first node of a bin for updates; reads are lock-free. (Java 7 used segments.) A simplified build: LLD §13.7.
- *Why no nulls?* → `get` returning `null` must unambiguously mean "absent"; with concurrent updates you can't follow up with `containsKey` safely.

More in LLD §12.16 and §12.22 Producer-consumer.

14.13 Synchronizers: `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `Phaser`

What it is

Synchronizers are objects whose job is to coordinate the *flow* of threads based on their own state.

Synchronizer	Idea	Reusable?	Typical use
`CountDownLatch(n)`	threads `await()` until `countDown()` has been called n times	no (one-shot)	"start all workers together"; "wait until N services are initialised"
`CyclicBarrier(n, action)`	n threads `await()` each other; when the last arrives, the optional action runs and all proceed	yes	iterative algorithms in phases (simulations)
`Semaphore(permits)`	`acquire()` takes a permit (waits if none), `release()` returns one	yes	limit concurrency: at most 10 DB (database) connections / API calls
`Phaser` (7)	like a barrier, but parties can register/deregister dynamically; numbered phases	yes	variable number of participants
`Exchanger<V>`	two threads swap objects at a meeting point	yes	double-buffering (one buffer is filled while the other is used, then they swap; rare)

`code/ch14/synchronizers/Synchronizers.java` (key parts):

```
CountDownLatch startGate = new CountDownLatch(1), finished = new CountDownLatch(n);
// workers: startGate.await(); work...; finished.countDown();
startGate.countDown();  finished.await();

CyclicBarrier barrier = new CyclicBarrier(3, () -> System.out.println("barrier: all 3 arrived, ..."));
// 3 threads: for (round 0..1) barrier.await();

Semaphore permits = new Semaphore(3);
permits.acquire();
try { /* at most 3 threads here */ } finally { permits.release(); }

Phaser phaser = new Phaser(1);  phaser.register();  phaser.arriveAndAwaitAdvance();
```

Output:

```
latch: work before opening the gate = 0
latch: work after all finished = 40, count = 0
barrier: all 3 arrived, phase 1 complete
barrier: all 3 arrived, phase 2 complete
semaphore: 20 tasks on 10 threads, max ever inside = 3 (limit 3)
phaser: main + 2 workers passed phase 0, current phase = 1
```

- `countDown()` goes in `finally` so a failing worker can't leave the waiter stuck forever. Prefer `await(timeout, unit)` in production.
- A `Semaphore` has no owner: any thread can `release()`, and releasing without acquiring *increases* the permits — always pair `acquire` with `release` in `finally`.
- If one thread waiting at a `CyclicBarrier` is interrupted or times out, the barrier becomes **broken** and all others get `BrokenBarrierException`.

Worked examples: LLD §12.12 Semaphores, §12.14 Latch and barrier, and the rate limiter in LLD §13.11.

Self-check

1. Latch or barrier for "main waits until 5 workers finish"? → `CountDownLatch(5)`.
2. How do you cap concurrent calls to an API at 10? → `Semaphore(10)`, acquire/release around each call.
3. Can a `CountDownLatch` be reset? → No; use `CyclicBarrier` or `Phaser` for reuse.

14.14 Designing thread-safe classes

Java Concurrency in Practice boils thread safety down to one sentence: **manage access to shared, mutable state.** You can remove "shared", remove "mutable", or manage the access:

Strategy	How	Example
Thread confinement	only one thread ever touches the data	local variables; objects created and used inside one task; one thread owning a UI; `ThreadLocal` (Chapter 15)
Immutability	no state changes after construction; `final` fields; defensive copies (copying incoming/outgoing mutable data so outsiders can't change yours)	records with immutable components, `String`, `List.of(...)`, `java.time` — always thread-safe
Encapsulated synchronisation	private state, every access under the same private lock	the `Counter` in §14.6
Delegation	keep state in thread-safe components	a `ConcurrentHashMap` field, an `AtomicLong` counter

Guidelines:

1. **Prefer immutable objects**; make fields `final` unless they must change (Chapter 5).
2. **Don't let `this` escape** from a constructor (starting a thread or registering a listener in the constructor) — other threads could see a half-built object.
3. **Invariants involving several variables** need one lock around all of them (atomics per field aren't enough).
4. **Document** thread-safety guarantees in Javadoc (the `/** … */` API documentation comments) ("This class is thread-safe" / "not thread-safe").
5. **Prefer high-level utilities** (executors, concurrent collections, synchronizers) over `wait`/`notify` and raw threads — *Effective Java* Items 80–81.
6. **Test under load** and use tools: thread dumps, JFR (Java Flight Recorder, the JVM's built-in low-overhead profiler/recorder), and stress-testing harnesses such as OpenJDK's jcstress for low-level code — but remember tests can't prove absence of races; reasoning about happens-before can (Chapter 15).

Choosing the tool

Need	Reach for
Run tasks in the background	`ExecutorService` (virtual threads for I/O-bound — Chapter 15)
Counter / accumulator	`AtomicInteger`/`AtomicLong`; `LongAdder` if very hot
Stop flag, publish an immutable snapshot	`volatile`
Multi-field invariant	`synchronized` (or `ReentrantLock` if you need `tryLock`/timeouts)
Shared map / cache	`ConcurrentHashMap` + atomic methods
Producer-consumer	`BlockingQueue`
Wait for N things to finish	`CountDownLatch` (or `invokeAll`, or `CompletableFuture.allOf`)
Limit concurrent access	`Semaphore`
Read-mostly data with rare writes	`CopyOnWriteArrayList`, `ReadWriteLock`, or an immutable snapshot in a `volatile`/`AtomicReference`

The LLD book's version of this table with interview framing: LLD §12.26; how concurrency appears in design interviews (seat booking, parking lots, inventory): LLD §12.25.

In the interview (chapter-level)

- *How would you make a seat-booking method safe?* → Make check-and-book atomic: `synchronized` on the show (or a per-seat lock), or `ConcurrentHashMap.putIfAbsent(seatId, userId)` — and in a real system a database **constraint** (a rule like "seat id must be unique" that the database enforces) or `SELECT … FOR UPDATE` (an SQL query that locks the rows it reads until the transaction ends), because several servers run the code.
- *`synchronized` vs `volatile` vs `AtomicInteger`?* → Mutual exclusion + visibility / visibility only / lock-free atomic single-variable updates.
- *What is a thread-safe class?* → One that stays correct under any interleaving of calls from multiple threads without callers adding synchronisation.

Chapter TL;DR

Topic	Remember
Threads	Share the heap, own their stack; `start()` (not `run()`) creates a thread; `join()` waits; daemon threads don't keep the JVM alive; `Thread.Builder` (21)
States	NEW, RUNNABLE, BLOCKED (monitor), WAITING, TIMED_WAITING, TERMINATED
Interruption	Cooperative; blocking calls throw `InterruptedException` and clear the flag; restore it or rethrow
Races	Read-modify-write and check-then-act; results vary run to run; tests can't prove absence
`synchronized`	Mutual exclusion + visibility; `this` / `Class` / chosen object; reentrant; same lock for every access
`volatile`	Visibility (and ordering) only; perfect for flags; `++` still races
`wait` / `notify`	Hold the lock, `while` loop, prefer `notifyAll`; prefer higher-level tools
Locks	`ReentrantLock`: `tryLock`, timeouts, interruptible, fair, `Condition`s — unlock in `finally`; `ReadWriteLock`; `StampedLock`
Atomics	CAS-based, lock-free, single variable; `AtomicReference` + immutable object; `LongAdder` for hot counters
Executors	Prefer to raw threads; `Future` + `ExecutionException`; shut down (or `close()`, 19); bounded queues + rejection policy in servers; size by CPU vs wait
Collections	`ConcurrentHashMap` atomic methods (`merge`, `computeIfAbsent`); COW lists for read-mostly; `BlockingQueue` for producer-consumer
Synchronizers	Latch (one-shot), barrier (cyclic), semaphore (permits), phaser (dynamic)
Design	Confinement, immutability, encapsulated locking, delegation

Where next: composing asynchronous work with `CompletableFuture`, Fork/Join, **virtual threads**, structured concurrency, scoped values, the **Java Memory Model** and deadlock/livelock/starvation in Chapter 15 — Concurrency II. Practise with classic problems in LLD Chapter 13; exam traps in OCP Chapter 13. Python comparison (GIL, `threading`, `asyncio`): Python Chapter 16.

Chapter 15 — Concurrency II: Modern Concurrency

Chapter 14 gave you threads, locks, atomics, thread pools and concurrent collections. This chapter covers how concurrent Java is written **today**: composing **asynchronous** results (results of work that finishes later, while the caller carries on) with `CompletableFuture`, splitting CPU work with Fork/Join, handling huge numbers of **blocking** tasks (tasks that spend time waiting, e.g. for a network reply) with **virtual threads** (Java 21), passing context safely with **scoped values** (final in Java 25), and grouping related tasks with **structured concurrency** (still a **preview** feature — shipped in the JDK for feedback but switched off unless you pass `--enable-preview`, and still allowed to change). It then goes underneath all of them — the **Java Memory Model** (the rules for when one thread's writes become visible to another) and *happens-before* (the ordering guarantee those rules are built on), which explain *why* the Chapter 14 tools work — and ends with the ways concurrent programs stop making progress: deadlock, livelock and starvation.

What you'll learn

§	Topic	You'll be able to…
15.1	`Future` and its limits	say why `Future.get()` isn't enough for real pipelines
15.2	`CompletableFuture`	create, transform, compose, combine and time out async results; handle errors properly
15.3	Fork/Join	split CPU-bound work recursively and understand work stealing (and parallel streams)
15.4	Virtual threads	explain how they work, measure the win, and know when they help and when they don't
15.5	Pinning and JEP 491	know what pinning was, what Java 24 fixed, and what still pins
15.6	`ThreadLocal` and scoped values	pass request context implicitly, and choose between the two
15.7	Structured concurrency (preview)	read and write `StructuredTaskScope` code and know its status
15.8	The Java Memory Model and happens-before	reason about visibility; publish objects safely; write correct lazy initialisation
15.9	Liveness: deadlock, livelock, starvation	produce, detect and prevent each one
15.10	Best practices and choosing a model	pick the right concurrency tool for a job

All runnable code is in `code/` (package `ch15.*`), compiled with `javac --release 25` (the Java compiler, targeting Java 25) and run on JDK 26 (the Java Development Kit) on a 14-core Apple-silicon (Apple's own ARM-based processors) laptop. As in Chapter 14, lines tagged `[nondeterministic]` vary between runs and `[machine-dependent]` lines depend on your hardware; timing numbers are rounded. The structured-concurrency example (§15.7) uses a **preview** API, so it lives only in this README and was run separately with `--enable-preview` on JDK 25 and JDK 26.

> **Related material — linked, not repeated.** Virtual threads, `Future`, parallel-stream and thread-problem exam traps: OCP Chapter 13 — Concurrency. Deadlock, livelock and starvation with more diagrams and the dining philosophers: LLD §12.17–12.19 and LLD §13.5. Fork/Join merge sort: LLD §13.10. Parallel streams: Chapter 12.

15.1 `Future` and its limits

What it is

A `Future<V>` (Java 5, Chapter 14 §14.11) is a handle to a result that will exist later. You can `get()` it (blocking), poll `isDone()`, `cancel()` it, and since Java 19 read `state()` / `resultNow()` / `exceptionNow()` without blocking.

The problem it doesn't solve

Real work is a *pipeline* (a chain of steps, each feeding the next): "fetch the user, **then** fetch their orders, **combined with** today's exchange rate, **falling back** to a **cached** value (a saved copy from earlier) on error, **giving up** after 2 seconds". With a plain `Future`:

1. The only way to use the result is **`get()`, which blocks** the calling thread — so "then do X" means parking a thread just to wait.
2. There's **no way to chain** a next step, combine two futures, or react to "whichever finishes first".
3. **Error handling** is a `try/catch (ExecutionException)` around each `get()`.
4. You can't **complete it yourself** (e.g. from a callback-based library — one that reports results by calling a function you gave it).

`CompletableFuture` (Java 8) fixes all four.

15.2 `CompletableFuture`

What it is

`CompletableFuture<T>` implements both `Future<T>` and **`CompletionStage<T>`**, an interface of ~40 methods for saying *what should happen when this completes* — without blocking. Analogy: instead of standing at the counter waiting for your coffee (`get()`), you leave your phone number and a list of instructions: "when it's ready, add milk, then bring it to table 5; if they're out of beans, bring tea".

How it works

1. **Start** async work: `supplyAsync(supplier[, executor])` (returns a value) or `runAsync(runnable[, executor])`. Or create an already-finished one with `completedFuture(v)` / `failedFuture(ex)` (9), or an empty one with `new CompletableFuture<>()` that you later `complete(v)` / `completeExceptionally(ex)` — a **promise** (a placeholder for a result that some code promises to fill in later).
2. **Attach stages**; each returns a *new* `CompletableFuture`, so calls chain:

Method	Input → output	Stream analogy
`thenApply(fn)`	`T` → `U`	`map`
`thenAccept(consumer)`	`T` → nothing	`forEach`
`thenRun(runnable)`	nothing → nothing	
`thenCompose(fn)`	`T` → `CompletableFuture<U>`, flattened	`flatMap`
`thenCombine(other, bifn)`	`T` and `U` → `V` (both needed)	`zip`
`applyToEither(other, fn)` / `acceptEither`	whichever of two finishes first	
`allOf(cfs...)`	wait for all → `CompletableFuture<Void>`	
`anyOf(cfs...)`	first to finish → `CompletableFuture<Object>`	

3. **Handle errors**: `exceptionally`, `handle`, `whenComplete`, `exceptionallyCompose` (12).
4. **Time out** (Java 9): `orTimeout(t, unit)` fails with `TimeoutException`; `completeOnTimeout(value, t, unit)` substitutes a default.

5. **Get the result at the edge** of your program: `join()` (unchecked `CompletionException` — *unchecked* means the compiler doesn't force you to catch it) or `get()` (checked `ExecutionException` — one you must catch or declare), or hand the `CompletableFuture` to a framework that accepts it (Spring MVC **controllers** — the classes in the Spring web framework that handle incoming HTTP requests — can return one).

Which thread runs a stage? A non-async stage (`thenApply`) runs on whichever thread completes the previous stage — or immediately on the *calling* thread if it's already complete. Every method has an `…Async` twin (`thenApplyAsync(fn[, executor])`) that submits the stage to an **executor** (an object that runs tasks on its threads, usually a thread pool — Chapter 14 §14.11). **Without an executor argument, async methods use `ForkJoinPool.commonPool()`** — shared by the whole JVM and sized for CPU work, so **always pass your own executor for blocking I/O** (or a virtual-thread executor, §15.4).

`code/ch15/cf/CompletableFutureBasics.java` (key parts; each `fetch…` sleeps to simulate a remote call):

```
try (ExecutorService io = Executors.newFixedThreadPool(8)) {
    CompletableFuture<Integer> idF = CompletableFuture.supplyAsync(() -> fetchUserId("grace"), io);
    idF.thenApply(id -> "user-" + id).thenApply(String::toUpperCase).join();

    // thenApply with an async function nests; thenCompose flattens
    CompletableFuture<CompletableFuture<String>> nested = idF.thenApply(id -> supplyAsync(() -> fetchProfile(id),
io));
    CompletableFuture<String> flat = idF.thenCompose(id -> supplyAsync(() -> fetchProfile(id), io));

    price.thenCombine(taxRate, (p, t) -> p * (1 + t));

    List<CompletableFuture<Double>> quotes = List.of(
            supplyAsync(() -> fetchPrice("A", 300, 19.99), io),
            supplyAsync(() -> fetchPrice("B", 200, 17.49), io),
            supplyAsync(() -> fetchPrice("C", 250, 18.25), io));
    CompletableFuture<Void> all = CompletableFuture.allOf(quotes.toArray(CompletableFuture[]::new));
    double best = all.thenApply(v ->
quotes.stream().mapToDouble(CompletableFuture::join).min().orElseThrow()).join();

    CompletableFuture.anyOf(supplyAsync(() -> fetchPrice("slow", 300, 1.0), io),
                            supplyAsync(() -> fetchPrice("fast", 50, 2.0), io)).join();

    CompletableFuture<String> promise = new CompletableFuture<>();
    io.submit(() -> { sleep(50); promise.complete("completed by another thread"); });
}
```

Output:

```
1 submitted, done yet? false
2 thenApply chain -> USER-35
3 thenApply gives a nested future: CompletableFuture; thenCompose gives the value: profile#35
4 thenCombine -> total = 25.0
5 allOf best price = 17.49, took ~300 ms (sequential would be ~750)   [timing machine-dependent]
6 anyOf -> 2.0
7 thenAccept got cached
7 thenRun: nothing to receive, just runs after
8 manual complete -> completed by another thread
9 thenApplyAsync(fn, io) ran on our pool
```

- `allOf` returns `CompletableFuture<Void>` — it only signals "all done". Collect the values by calling `join()` on each original future inside the next stage; by then they're complete, so `join()` doesn't block.
- The three price lookups ran **in parallel**: total time ≈ the slowest (300 ms), not the sum (750 ms). This **fan-out/fan-in** (send several requests at once, then gather all the answers) is the most common real use (price comparison, aggregating **microservice** calls — a microservice being a small, separately deployed service that owns one business area).

Error handling

When a stage throws, all later *transforming* stages are skipped and the exception travels down the chain until a handler deals with it — like an exception unwinding a **call stack** (the chain of method calls

currently in progress).

Method	Runs when	Can change the result?	Use for
`exceptionally(ex -> fallback)`	only on failure	yes — recovers	default value
`handle((val, ex) -> …)`	always	yes	map success and failure into one result
`whenComplete((val, ex) -> …)`	always	no — passes the original through	logging, metrics (counters and timings a service records about itself), cleanup
`exceptionallyCompose(ex -> otherCf)` (12)	only on failure	yes, asynchronously	retry / call a backup service

`code/ch15/cf/CompletableFutureErrors.java` (key parts):

```
CompletableFuture<Integer> failed = supplyAsync(() -> parse("x")).thenApply(n -> n * 2);  // thenApply skipped
failed.join();   // CompletionException(cause NumberFormatException)
failed.get();    // ExecutionException(cause NumberFormatException)

supplyAsync(() -> parse("x")).exceptionally(ex -> -1);
supplyAsync(() -> parse(in)).handle((val, ex) -> ex == null ? "ok " + val * 2 : "failed: " +
ex.getCause().getMessage());
supplyAsync(() -> parse("oops")).whenComplete((val, ex) -> log(...));

slow.orTimeout(100, MILLISECONDS).join();                          // TimeoutException
slow.completeOnTimeout("default", 100, MILLISECONDS).join();       // "default"
CompletableFuture.<String>failedFuture(new IllegalStateException("primary down"))
        .exceptionallyCompose(ex -> supplyAsync(() -> "from backup"));

CompletableFuture.runAsync(() -> { throw new IllegalStateException("nobody will ever see this"); });
```

Output:

```
1 join -> CompletionException, cause NumberFormatException
1 get  -> ExecutionException,  cause NumberFormatException
1 isCompletedExceptionally = true, state = FAILED
2 exceptionally sees CompletionException wrapping NumberFormatException
2 recovered value = -1
3 handle(21) -> ok 42
3 handle(oops) -> failed: For input string: "oops"
4 whenComplete logs: error
4 still failed afterwards? true
5 orTimeout -> TimeoutException
5 completeOnTimeout -> default
6 exceptionallyCompose -> from backup
7 fire-and-forget failure printed nothing (that's the danger)
```

- The exception a handler receives is often **wrapped in** `CompletionException` (line 2). Unwrap before checking its type: `Throwable cause = ex instanceof CompletionException ce ? ce.getCause() : ex;`.
- Line 7 is the biggest practical danger: a failed `CompletableFuture` that nobody joins or handles **fails silently**. Every chain should end in a `join`, a handler, or at least a `whenComplete` that logs.

Trade-offs

	`Future` + pool	`CompletableFuture`	Virtual threads + plain blocking code
Composition	none	rich	ordinary sequential code
Readability	simple	chains can get long; **stack traces** (the printed list of calls at an error) are fragmented	simplest
Threads blocked while waiting	yes	no (callbacks)	yes, but they're cheap
Best for	simple fire-and-get	fan-out/fan-in, async APIs, callbacks	most request/response I/O (Java 21+)

Common mistakes

- Blocking I/O in `supplyAsync` **without an executor** → the common pool (≈ one thread per core) is exhausted and every parallel stream in the JVM stalls.
- Calling `join()` right after `supplyAsync` everywhere → you've written slow synchronous code with extra steps.
- Using `thenApply` with a function returning a `CompletableFuture` → `CompletableFuture<CompletableFuture<T>>`; use `thenCompose`.
- Expecting `cancel(true)` to interrupt the running task: for `CompletableFuture` it just completes the future with `CancellationException`; the computation keeps running.
- Swallowing errors with `exceptionally(ex -> null)`.

In the interview

> **30-second answer:** "`CompletableFuture` is a composable, completable `Future`. I start work with `supplyAsync` on a dedicated executor, chain with `thenApply` / `thenCompose`, combine with `thenCombine` / `allOf`, recover with `exceptionally` / `handle`, and bound **latency** (how long a call takes, start to finish) with `orTimeout`. Non-async stages run on the completing thread; async variants default to the common pool, so for I/O I always pass an executor."

- *`thenApply` vs `thenCompose`?* → `map` vs `flatMap`: use `thenCompose` when the function itself returns a `CompletableFuture`.
- *`join` vs `get`?* → Same result; `join` throws unchecked `CompletionException`, `get` throws checked `ExecutionException` / `InterruptedException` and has a timeout overload.
- *How do you wait for a list of futures?* → `allOf(...).thenApply(v -> list.stream().map(CompletableFuture::join).toList())`.

Self-check

1. What does `allOf` return? → `CompletableFuture<Void>`; read the individual futures for values.
2. Which handler cannot change the result? → `whenComplete`.
3. Which pool do `…Async` methods use without an executor? → `ForkJoinPool.commonPool()`.

15.3 Fork/Join

What it is

The **Fork/Join framework** (Java 7) runs **divide-and-conquer** algorithms (split a problem into smaller copies of itself, solve those, combine the answers) in parallel: split a big problem into halves recursively until pieces are small, solve the pieces, and combine the answers. Its `ForkJoinPool` uses **work stealing**: each worker thread has its own double-ended queue (deque) of tasks; when a worker runs out, it **steals** tasks from the other end of a busy worker's deque, keeping all cores busy with little

contention (threads competing for the same lock or data). Analogy: a team splitting a stack of exam papers — whoever finishes their pile grabs half of a colleague's remaining pile.

How it works

1. Extend `RecursiveTask<V>` (returns a value) or `RecursiveAction` (no value) and implement `compute()`.
2. In `compute()`: if the problem is below a **threshold**, solve it directly (plain loop).
3. Otherwise split: `left.fork()` (push onto this worker's deque), compute the other half directly (`right.compute()`), then `left.join()` (wait for — or run — the forked half), and combine.
4. Start with `pool.invoke(task)`; `ForkJoinPool.commonPool()` has parallelism = cores − 1 by default.

`code/ch15/forkjoin/ForkJoinSum.java` (key parts):

```
static class SumTask extends RecursiveTask<Long> {
    static final int THRESHOLD = 10_000;
    @Override protected Long compute() {
        if (to - from <= THRESHOLD) { long s = 0; for (int i = from; i < to; i++) s += data[i]; return s; }
        int mid = (from + to) >>> 1;
        SumTask left = new SumTask(data, from, mid), right = new SumTask(data, mid, to);
        left.fork();                  // others may steal it
        long r = right.compute();     // keep this thread busy
        long l = left.join();
        return l + r;
    }
}
long fj = ForkJoinPool.commonPool().invoke(new SumTask(data, 0, data.length));
long ps = Arrays.stream(data).parallel().sum();        // same framework underneath
```

Output (20 million longs):

```
common pool parallelism = 13   [machine-dependent: cores - 1]
round 1: correct=true  sequential 7 ms, fork/join 15 ms, parallel stream 28 ms   [timings machine-dependent]
round 2: correct=true  sequential 11 ms, fork/join 1 ms, parallel stream 0 ms   [timings machine-dependent]
round 3: correct=true  sequential 5 ms, fork/join 1 ms, parallel stream 0 ms   [timings machine-dependent]
tasks stolen between workers so far > 0: true
```

- Round 1 is *slower* in parallel: thread start-up and JIT compilation (the JVM's *just-in-time* compiler turning frequently used bytecode into machine code at run time) dominate. Parallelism pays off only once **warmed up** (after the JIT has optimised the hot code) and for enough data — measure with a proper **benchmark harness** (a tool that runs timing experiments correctly — warm-up, repetition, statistics; JMH, the Java Microbenchmark Harness, Chapter 16) before concluding anything.
- The order `fork` one half → `compute` the other → `join` matters: calling `left.fork(); right.fork(); left.join(); right.join();` wastes the current thread.

When to use it / when not

- **Use** for **CPU-bound** (limited by processing, not by waiting), recursively splittable work on large in-memory data: sorting, searching, image processing, tree traversal. Most of the time, a **parallel stream** (Chapter 12 §12.8) gives you the same engine with one method call.
- **Don't** do blocking I/O in Fork/Join tasks (or parallel streams): the common pool is shared by the whole JVM and has few threads. (`ForkJoinPool.ManagedBlocker` exists for special cases.) For I/O (input/output — disk, network) concurrency use virtual threads.
- **Don't** make the threshold too small (task overhead dominates) or too large (not enough tasks to balance). Thousands of elements per leaf is a typical starting point.

Self-check

1. What is work stealing? → Idle workers take queued tasks from busy workers' deques.
2. `RecursiveTask` vs `RecursiveAction`? → Returns a value vs doesn't.
3. What else uses the common pool? → Parallel streams and `CompletableFuture` async methods without an executor.

15.4 Virtual threads

What it is

A **virtual thread** (final in **Java 21, JEP 444** — a JEP, *JDK Enhancement Proposal*, is the numbered design document for a Java feature; previews in 19 and 20) is a `java.lang.Thread` that is **not tied to an operating-system thread**. The JVM schedules many virtual threads onto a small pool of OS threads called **carrier threads**. When a virtual thread performs a blocking operation (sleep, socket read, `BlockingQueue.take`, lock wait…), the JVM **unmounts** it — saves its stack to the **heap** (the JVM's memory area for objects) — and the carrier is free to run another virtual thread. When the operation completes, the virtual thread is **mounted** again on some carrier and continues.

Analogy: platform threads are like giving every restaurant customer a dedicated waiter who stands by the table even while the customer reads the menu. Virtual threads are like a few waiters serving hundreds of tables, going to whichever table actually needs something.

	Platform thread	**Virtual thread**
Backed by	one OS thread, 1:1	carrier threads, M:N (many virtual threads share a few OS threads)
Creation cost	expensive (OS call, 1–2 MB stack reserved)	cheap (a small heap object; stack grows/shrinks on the heap)
How many	thousands	millions
Blocking	holds the OS thread	unmounts; carrier is released
Pool them?	yes	**no** — create one per task
Daemon / priority	configurable	always daemon (doesn't keep the JVM alive), normal priority
API	same `Thread` API	same `Thread` API (`isVirtual()`)

Why it matters

A server that handles each request on its own thread ("**thread-per-request**") is the easiest style to write and debug: plain sequential code, readable stack traces, `try/catch` works. But with platform threads, if each request spends 100 ms waiting on a database, a server with 200 threads can handle only ~2,000 requests per second, no matter how idle the CPU is. **Little's law** says concurrency = **throughput** (requests completed per second) × latency (both explained in System Design Fundamentals): 10,000 requests/s × 0.1 s = 1,000 requests in flight — more than a platform-thread pool comfortably holds. The previous fix was asynchronous/**reactive** code (a style where code is written as reactions to streams of events, never blocking — `CompletableFuture` chains, reactive libraries), which scales but is harder to write, read and debug. Virtual threads let simple blocking code scale like async code.

How it works

1. Create one: `Thread.ofVirtual().name("vt-1").start(runnable)`, `Thread.startVirtualThread(runnable)`, or — most commonly — `Executors.newVirtualThreadPerTaskExecutor()`, which starts a new virtual thread for every submitted task.
2. The default **scheduler** (the component that decides which virtual thread runs on which carrier next) is a `ForkJoinPool` (FIFO — first in, first out — mode) whose parallelism equals the number of cores.
3. When the virtual thread blocks in a JDK operation that supports it (almost all of `java.io`, `java.net`, `java.util.concurrent`, `Thread.sleep`), its stack frames are copied to the heap and the carrier moves on.
4. When unblocked, it's rescheduled on any free carrier.

`code/ch15/virtual/VirtualThreadsDemo.java` (key parts):

```
Thread v1 = Thread.ofVirtual().name("vt-1").start(() -> ...);
Thread v2 = Thread.startVirtualThread(() -> {});
Thread.currentThread().toString();       // inside a virtual thread: shows its carrier

static long time(ExecutorService executor) {
    try (executor) {                                  // close() waits for all tasks
        for (int i = 0; i < 10_000; i++) executor.submit(() -> { Thread.sleep(Duration.ofMillis(100)); return
null; });
    }
}
time(Executors.newFixedThreadPool(200));
time(Executors.newVirtualThreadPerTaskExecutor());

Semaphore dbPermits = new Semaphore(10);            // limit a resource, not the number of threads
```

Output:

```
1 hello from vt-1
1 isVirtual=true, daemon=true, main isVirtual=false
2 VirtualThread[#N]/runnable@ForkJoinPool-1-worker-K   [ids vary]
3 fixed pool of 200 platform threads: 5,202 ms  (theory: 10,000 / 200 x 100 ms = 5,000)   [machine-dependent]
3 one virtual thread per task:        126 ms  (theory: ~100 ms + overhead)            [machine-dependent]
4 carrier threads used: 14 <= cores (14)? true   [machine-dependent]
5 200 virtual threads, Semaphore(10): max concurrent 'DB calls' = 10
```

- **10,000 blocking tasks: ~5.2 s on a 200-thread pool vs ~0.13 s on virtual threads** — 40× faster here, because all 10,000 could wait at the same time. With a 10,000-thread platform pool you'd get similar speed at a much higher memory cost, and at 100,000 or 1,000,000 tasks platform threads run out while virtual threads keep going.
- Only **14 carrier threads** (the number of cores) ran all 10,000 virtual threads.
- Note that virtual threads don't make anything *faster* per task: each still took 100 ms. They raise **throughput** for tasks that **wait**.

When they help

- Servers and clients with **many concurrent, mostly-waiting tasks**: HTTP handlers (code that answers web requests), calls to databases and other services, message consumers (code that processes items from a message queue). Spring Boot (3.2+; the most popular Java framework for building services) enables them for request handling with `spring.threads.virtual.enabled=true`; Tomcat, Jetty and Helidon (Java web servers / frameworks) support them.
- Replacing complex async code with straightforward blocking code.
- Concurrent mapping inside a stream: the `Gatherers.mapConcurrent(n, fn)` gatherer (Java 24) runs `fn` on virtual threads with at most `n` in flight (Chapter 12 §12.9).

When they don't help (and pitfalls)

1. **CPU-bound work** — there are still only as many cores as there are. Use a platform pool sized to the cores, Fork/Join or parallel streams.
2. **Don't pool virtual threads.** A pool exists to limit an expensive resource; virtual threads aren't expensive. To limit access to a scarce resource (a database allowing 10 connections, a **rate-limited** API that accepts only so many calls per second), use a **`Semaphore`** (a counter of permits; Chapter 14 §14.13) or a bounded **connection pool** (a fixed set of reusable, already-open database connections), as in line 5.
3. **Heavy `ThreadLocal` caching**: code that caches expensive objects per thread (one parser per thread) was fine with 200 pooled threads, but with a million short-lived virtual threads you get a million copies, each used once (§15.6).
4. **Pinning** on Java 21–23 when blocking inside `synchronized` (§15.5).
5. Your **downstream** systems (the services and databases your code calls) still have limits: 10,000 concurrent requests may overwhelm a database that could handle 200 at a time.

Observing virtual threads

`jcmd <pid> Thread.dump_to_file -format=json dump.json` (`pid` = the OS process id of the JVM) writes a **thread dump** (a snapshot of every thread and what it's doing) that includes virtual threads (the older `jstack` output shows only platform threads). JDK Flight Recorder (JFR, the JVM's built-in low-overhead event recorder) has `jdk.VirtualThreadStart`/`End` (off by default), `jdk.VirtualThreadPinned` and `jdk.VirtualThreadSubmitFailed` events.

In the interview

> **30-second answer:** "Virtual threads, final in Java 21, are JVM-managed threads **multiplexed** (many sharing a few) onto a few carrier threads. When one blocks on I/O it unmounts and frees the carrier, so millions of blocking tasks can be in flight and simple thread-per-request code scales like async code. They help I/O-bound, high-concurrency workloads, not CPU-bound ones; you don't pool them — you create one per task and limit scarce resources with semaphores."

- *Are virtual threads faster?* → Not per task; they give higher throughput for waiting tasks.
- *How many carrier threads?* → By default one per core.
- *What replaces a pool size limit?* → A `Semaphore` or a bounded resource pool (e.g. HikariCP, the most common Java database connection pool).
- *Does existing code need to change?* → Usually not: same `Thread` API; switch the executor.

Self-check

1. What happens to a virtual thread's stack when it blocks? → It's copied to the heap; the carrier is released.
2. Why is `newFixedThreadPool(100)` of virtual threads a bad idea? → It defeats their purpose and still limits concurrency the wrong way; use one per task plus a semaphore.
3. Will a CPU-heavy image resize run faster on virtual threads? → No.

15.5 Pinning and JEP 491

What it is

A virtual thread is **pinned** when it blocks but *cannot* unmount, so it holds its carrier thread hostage for the whole wait. If all carriers are pinned, no other virtual thread can run — throughput collapses back to "one task per carrier", and in the worst case the application deadlocks.

What changed in Java 24

In Java 21–23, a virtual thread was pinned when it blocked **inside a `synchronized` block or method** (including `Object.wait()`), because the JVM's **monitor** (the built-in lock behind `synchronized`) implementation tracked lock ownership by carrier thread. The advice at the time was "replace `synchronized` with `ReentrantLock` around blocking operations". **JEP 491 (Java 24) re-implemented monitors so virtual threads can acquire, hold and release them independently of their carrier.** Blocking inside `synchronized` — and `Object.wait()` — now unmounts normally, so that advice no longer applies on Java 24+ (including Java 25 LTS — a *long-term support* release that receives updates for years).

`code/ch15/virtual/PinningDemo.java` uses only Java 21 APIs, so it was also compiled and run on JDK 21 for comparison:

```
try (ExecutorService ex = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000; i++) {
        Object lock = new Object();                // one lock per task: no contention
        ex.submit(() -> { synchronized (lock) { Thread.sleep(50); } return null; });
    }
}
```

Output on JDK 26 (the runner):

```
Java 26: 1000 virtual threads sleeping 50 ms inside synchronized took 63 ms   [machine-dependent]
if every carrier were pinned: ~3600 ms with 14 carriers
```

Output of the same program on JDK 21 (Amazon Corretto 21.0.10 — Amazon's free build of the JDK — run by hand):

```
Java 21: 1000 virtual threads sleeping 50 ms inside synchronized took 3869 ms   [machine-dependent]
if every carrier were pinned: ~3600 ms with 14 carriers
```

On Java 21 every sleeping virtual thread pinned its carrier, so only 14 could sleep at a time: 1,000 / 14 × 50 ms ≈ 3.6 s. On Java 24+ all 1,000 slept at once.

What still pins (Java 24+)

According to JEP 491, a virtual thread still pins its carrier when:

1. It calls **native code** (code compiled for the machine, typically C, reached via JNI — the Java Native Interface — or the Foreign Function & Memory API) and that native code calls back into Java code that blocks.
2. It blocks while **loading a class** (resolving a symbolic reference — turning a class/method *name* in bytecode into the real thing in memory) or **inside a class initializer** (`static { … }`, code that runs once when a class is first used), or waits for another thread to finish initialising a class.

These are rare in application code. The JFR event `jdk.VirtualThreadPinned` (on by default, reported when the pin lasts over 20 ms) now also records the *reason* and the carrier thread. The old diagnostic **system property** (a `-Dname=value` setting passed to the JVM) `jdk.tracePinnedThreads` was removed by JEP 491 and has no effect.

Takeaway

On Java 21–23: avoid long blocking operations inside `synchronized` in virtual-thread code (use `ReentrantLock` there). On Java 24+: choose `synchronized` or `ReentrantLock` on their merits (Chapter 14 §14.9); watch JFR for the remaining native/class-initialisation cases.

15.6 `ThreadLocal` and scoped values

The problem

Code deep in a call stack often needs *context* that was established near the top: the current user, a request id for logging, a **transaction** (a group of database operations that succeed or fail together). Passing it as a parameter through every method is noisy, especially through framework code you don't own. Java has two ways to pass it **implicitly**.

`ThreadLocal` (Java 1.2)

A `ThreadLocal<T>` is a variable with a separate value **per thread**: `set(v)`, `get()`, `remove()`, `ThreadLocal.withInitial(supplier)`. Uses: per-request context in older frameworks (Spring Security's `SecurityContextHolder`, logging MDC — *Mapped Diagnostic Context*, key/value pairs automatically added to every log line), and per-thread caches of non-thread-safe objects (the classic being `SimpleDateFormat` — which `java.time.DateTimeFormatter`, being immutable, made unnecessary).

Problems:

1. **Unconstrained mutability** — any code can `set` it at any time; data flow is hard to follow.
2. **Unbounded lifetime** — the value lives until `remove()` or the thread dies. With **thread pools, a forgotten** `remove()` **leaks** the value into the next, unrelated task (a security bug if it's a user identity) and can **leak memory** (keep objects alive that nothing needs any more, so the garbage collector can't free them).
3. **Expensive inheritance** — `InheritableThreadLocal` copies values into every child thread; with millions of virtual threads that's costly.

Scoped values (final in Java 25, JEP 506)

A `ScopedValue<T>` is an **immutable** value **bound for the duration of a method call**. You bind it with `ScopedValue.where(KEY, value).run(runnable)` (or `.call(callable)` for a result); any code called — directly or indirectly — from inside that call can `KEY.get()` it; when the call returns, the binding is gone automatically. There is no `set`. Code can **rebind** it for a nested call, which shadows the outer value only inside that call. History: **incubator** in Java 20 (JEP 429; an incubator module is an even earlier experimental stage than preview), previews in 21–24 (JEPs 446, 464, 481, 487), **final in Java 25 (JEP 506)** — so it's usable without flags in Java 25 LTS.

Analogy: a `ThreadLocal` is a whiteboard in each worker's office — anyone can scribble on it and it stays until someone wipes it. A scoped value is a visitor badge handed out at the door for one meeting: everyone in the meeting can read it, nobody can change it, and it's collected when the meeting ends.

`code/ch15/scoped/ScopedValuesDemo.java` (key parts):

```
static final ThreadLocal<String> REQUEST_ID_TL = new ThreadLocal<>();
static final ScopedValue<String> USER = ScopedValue.newInstance();

// ThreadLocal leak on a pooled thread
single.submit(() -> REQUEST_ID_TL.set("req-OLD")).get();                 // forgot remove()
single.submit(() -> System.out.println(... REQUEST_ID_TL.get())).get();  // sees req-OLD!

ScopedValue.where(USER, "alice").run(() -> handleRequest());            // handler -> service -> repository
static void repository() { System.out.println("3 repository sees USER = " + USER.get()); }

String result = ScopedValue.where(USER, "bob").call(() -> {
    String outer = USER.get();
    String inner = ScopedValue.where(USER, "admin").call(USER::get);    // nested rebinding
    return outer + " -> " + inner + " -> " + USER.get();
});
USER.get();                 // unbound -> NoSuchElementException
USER.orElse("anonymous");
```

Output:

```
1 ThreadLocal in A: requestId = req-A
1 ThreadLocal in B: requestId = req-B
1 main thread sees null (never set here)
2 next task on the same pooled thread sees: req-OLD
2 withInitial -> per-thread scratch buffer
3 bound before? false
3 repository sees USER = alice
3 bound after?  false
4 nested rebinding: bob -> admin -> bob
5 get() when unbound -> NoSuchElementException; orElse -> anonymous
6 plain new thread sees bound? false
```

- Line 2 is the pooled-thread leak in action: the second task inherited the first task's request id.
- Line 3: `repository()` read `alice` three calls deep without any parameter; outside the `run`, nothing is bound.
- Line 6: bindings are **not** inherited by threads you start yourself. They *are* inherited by subtasks forked in a `StructuredTaskScope` (§15.7), which is how context flows into concurrent subtasks safely.

Trade-offs

	`ThreadLocal`	`ScopedValue` (25)
Mutability	`set` anywhere	immutable; rebind only for a nested call
Lifetime	until `remove()` / thread death	exactly the `run` / `call`
Leak risk with pools	yes	none
Inheritance	`InheritableThreadLocal` copies	shared (no copy) with structured-concurrency subtasks
Good for	per-thread caches of mutable objects; legacy frameworks	request context: user, tenant (the customer account in a multi-customer system), trace id (an id following one request across services), transaction

Common mistakes

- Not calling `remove()` in a `finally` when using `ThreadLocal` with pools.
- Treating a scoped value like a mutable holder — to "change" it, rebind for a nested call.
- Declaring `ScopedValue` keys as instance fields; they're meant to be `static final` constants (often private, exposed through accessors to control who can read or bind them).

In the interview

30-second answer: "`ThreadLocal` gives each thread its own mutable copy; it's easy to leak across pooled tasks and costly with millions of virtual threads. Scoped values, final in Java 25, bind an immutable value for the duration of a call; callees read it, it disappears when the call returns, and structured-concurrency subtasks share it. I use scoped values for request context and keep `ThreadLocal` for legacy integration and per-thread caches."

15.7 Structured concurrency (preview)

Status (verified against the JEPs)

Structured concurrency is **still a preview API** — not final — in Java 25 and 26:

Release	JEP	Status
19, 20	428, 437	incubator
21, 22, 23, 24	453, 462, 480, 499	preview 1–4
25 (LTS)	**505**	fifth preview — API redesigned around `StructuredTaskScope.open()` and `Joiner`
26	**525**	sixth preview — small changes (e.g. `Joiner.anySuccessfulResultOrThrow()` renamed `anySuccessfulOrThrow()`, `allSuccessfulOrThrow()` now returns a list of results, new `Joiner.onTimeout()`)
27 (GA — general availability, the official release — 15 Sept 2026)	533	seventh preview — built-in joiners make `join()` throw `ExecutionException`, a third type parameter for the exception type, `awaitAll()` removed, timeouts reworked

Because it's a preview, you must compile and run with `--enable-preview` (and the exact API can still change between releases), so the runnable code in this book doesn't depend on it. Details below use the part of the API that is the same in Java 25 and 26.

What it is

Structured concurrency treats a group of related concurrent tasks as **one unit of work** with a clear lifetime, the way a code block structures sequential code: subtasks are started inside a scope, and the scope **cannot be exited until they've all finished**. Consequences:

1. **No leaked threads** — when the method returns, no subtask is still running in the background.
2. **Error propagation and cancellation** — if one subtask fails, the others are **cancelled** (interrupted) automatically and the failure surfaces from `join()`.
3. **Observability** — thread dumps show subtasks as children of the task that forked them.
4. **Scoped values** bound in the parent are visible in the forked subtasks.

Compare with `ExecutorService` + `Future`s: if you submit two lookups and the first fails, nothing stops the second; if the caller is interrupted, the subtasks keep running; the relationship between them exists only in your head.

How it works (Java 25/26 preview API)

1. Open a scope in try-with-resources: `StructuredTaskScope.open()` (default policy: all subtasks must succeed) or `open(Joiner)` for another policy.
2. `fork(callable)` each subtask → a `Subtask<T>`; each runs in its own **virtual thread**.
3. `join()` waits for all subtasks (or until the policy decides — e.g. first failure).
4. Read results with `subtask.get()`; `subtask.state()` is `SUCCESS`, `FAILED` or `UNAVAILABLE`.
5. Closing the scope guarantees every subtask has finished.

```
// (preview API - run with: java --enable-preview --source 26 StructuredDemo.java ; same output on JDK 25 with --source 25)
static final ScopedValue<String> USER = ScopedValue.newInstance();
record Page(String profile, int orders) {}

static String fetchProfile() throws InterruptedException { Thread.sleep(100); return "profile of " + USER.get(); }
static int fetchOrders() throws InterruptedException { Thread.sleep(150); return 3; }
static int fetchOrdersBroken() throws InterruptedException { Thread.sleep(50); throw new IllegalStateException("orders service down"); }
static String slowProfile() throws InterruptedException {
    try { Thread.sleep(5_000); return "too late"; }
    catch (InterruptedException e) { System.out.println("  slowProfile: cancelled (interrupted)"); throw e; }
}

static Page load() throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {                  // all must succeed
        Subtask<String> profile = scope.fork(StructuredDemo::fetchProfile);
        Subtask<Integer> orders = scope.fork(StructuredDemo::fetchOrders);
        scope.join();                                               // wait for both, or the first failure
        return new Page(profile.get(), orders.get());
    }
}
static Page loadWithFailure() throws InterruptedException {
    try (var scope = StructuredTaskScope.open()) {
        Subtask<String> profile = scope.fork(StructuredDemo::slowProfile);
        Subtask<Integer> orders = scope.fork(StructuredDemo::fetchOrdersBroken);
        scope.join();                                               // throws FailedException
        return new Page(profile.get(), orders.get());
    }
}
// main:
Page p = ScopedValue.where(USER, "alice").call(StructuredDemo::load);
try { loadWithFailure(); }
catch (StructuredTaskScope.FailedException e) { ... e.getCause().getMessage() ... }
```

Output (JDK 26 and JDK 25, both with `--enable-preview`):

```
page = Page[profile=profile of alice, orders=3] in ~150 ms (max of 100 and 150, not the sum)
  slowProfile: cancelled (interrupted)
join -> FailedException, cause: orders service down; returned after ~50 ms, not 5000
```

- The forked `fetchProfile` read `USER` — scoped values flow into subtasks.
- When `fetchOrdersBroken` failed after 50 ms, the scope **interrupted** `slowProfile` immediately instead of letting it run for 5 s, and `join()` threw `FailedException` carrying the original cause.

Other built-in policies (`Joiner` factory methods): all must succeed (the default policy of `open()`; `allSuccessfulOrThrow` also collects the results), first success wins and cancels the rest (`anySuccessfulOrThrow` in 26 — "hedged" requests: sending the same request to several **replicas**, i.e. identical copies of a service, and taking the fastest answer), wait for all regardless of outcome (`awaitAll`), and a predicate-based `allUntil`. Timeouts are set through the configuration function passed to `open`. Check the Javadoc of the JDK you run, since names changed between previews (Java 27's seventh preview removes `awaitAll` and has the built-in joiners make `join()` throw `ExecutionException`).

When to use it

Once final, for any "call several things concurrently, then combine" code in a request handler — the fan-out that §15.2 did with `CompletableFuture.allOf`, but with automatic cancellation and ordinary blocking code. Until then: use it for experiments, or use `CompletableFuture`/executors in production and cancel manually.

In the interview

> **30-second answer:** "Structured concurrency groups subtasks into a scope whose lifetime is a code block: fork subtasks in virtual threads, `join`, and on failure the siblings are cancelled and the error propagates; no thread outlives the scope and scoped values flow into subtasks. It's still a preview API — fifth preview in Java 25, sixth in 26, seventh in 27 — while scoped values became final in 25."

15.8 The Java Memory Model and happens-before

What it is

The **Java Memory Model (JMM)** — JLS chapter 17 (the JLS is the *Java Language Specification*, the official rulebook), redesigned in Java 5 (JSR-133, a *Java Specification Request*, the older name for a formal spec change) — is the contract that says **which values a read of a shared variable is allowed to return** when several threads run. It exists because compilers, JIT and CPUs all **reorder** (run independent instructions in a different order) and **cache** (keep private, faster copies of) memory operations to go faster. Within a single thread these optimisations are invisible (the thread always sees its own actions in program order), but *another* thread can observe writes late or in a surprising order.

The JMM's central idea is the **happens-before** relation. If action A *happens-before* action B, then A's effects (and everything before A in its thread) are **guaranteed visible** to B. If two conflicting accesses (same variable, at least one write, different threads) are **not** ordered by happens-before, the program has a **data race**, and the read can see **stale** (out-of-date) or reordered values.

The happens-before rules

Rule	A happens-before B when…
Program order	A comes before B in the same thread
Monitor lock	A is an unlock of a monitor and B is a *later* lock of the **same** monitor (`synchronized`)
Volatile	A is a write to a volatile field and B a *later* read of **that** field
Thread start	A is `t.start()` and B is any action in `t`
Thread termination	A is any action in `t` and B is another thread returning from `t.join()` (or seeing `isAlive()` false)
Interruption	A is `t.interrupt()` and B is `t` detecting the interrupt
Transitivity	A hb B and B hb C ⇒ A hb C
`java.util.concurrent`	e.g. putting an object into a concurrent collection hb its retrieval; submitting a task hb its execution; task actions hb `Future.get()` returning; `release` / `countDown` / `unlock` hb a successful `acquire` / `await` / `lock`

Plus **final-field semantics** (JLS 17.5): if a constructor doesn't leak `this`, every thread that obtains a reference to the object sees its `final` fields correctly initialised, even without synchronisation.

Seeing reordering for real

`code/ch15/jmm/ReorderingDemo.java` runs the classic "store buffering" test millions of times. For each slot `i`, two threads run concurrently:

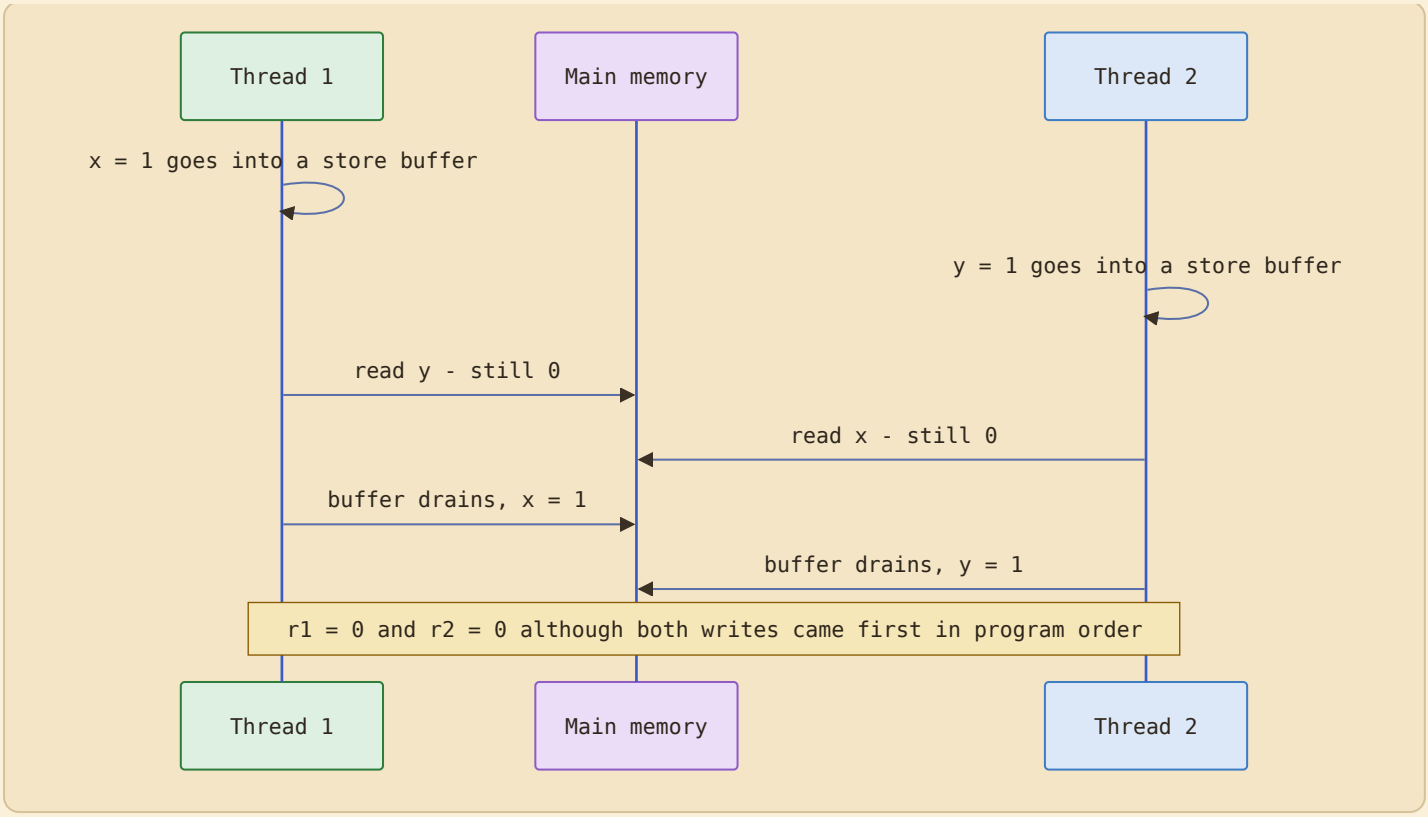
```
// thread 1                    // thread 2
x[i] = 1;   r1[i] = y[i];      y[i] = 1;   r2[i] = x[i];
```

In **any** interleaving of these four statements in program order, at least one thread reads a 1 — so `(r1, r2) = (0, 0)` is impossible without reordering. Then the same with `AtomicIntegerArray` `set`/`get` (volatile semantics):

Output:

```
plain arrays:          (0,0) seen 1,041 times in 50,000,000 trials   [nondeterministic - varies run to run; any
value > 0 proves reordering]
AtomicIntegerArray:    (0,0) seen 0 times in 50,000,000 trials   [always 0 - forbidden by the JMM]
```

Each CPU core put its write into a *store buffer* (a small per-core queue of pending writes) and read the other variable before its own write became visible to the other core — both reads saw 0. With volatile semantics the JMM forbids that outcome, and it never occurred.



Safe publication

Publishing an object means making it reachable by other threads. Publishing *unsafely* (assigning to a plain shared field with no happens-before) can let another thread see the reference **before** the object's fields are initialised. Safe ways (JCIP):

1. Initialise it in a **static initializer** (class initialisation is thread-safe).
2. Store the reference in a `volatile` field or an `AtomicReference`.
3. Store it in a `final` field of a properly constructed object.
4. Store it in a field **guarded by a lock**, or put it in a **concurrent collection**.
5. Make the object **immutable** (all fields `final`, no `this` escape) — then even racy publication shows it correctly.

`code/ch15/jmm/SafePublication.java` (key parts):

```
// Double-checked locking: correct ONLY because 'instance' is volatile (Java 5+)
private static volatile Registry instance;
static Registry get() {
    Registry r = instance;
    if (r == null) {
        synchronized (Registry.class) {
            r = instance;
            if (r == null) instance = r = new Registry();
        }
    }
    return r;
}
// Initialization-on-demand holder: lazy and thread-safe with no locking code
private static class Holder { static final Heavy INSTANCE = new Heavy(); }
static Heavy get() { return Holder.INSTANCE; }

static volatile Config current;                              // publish through a volatile write
```

Output:

```
64 threads raced: Registry constructed 1 time(s), Heavy constructed 1 time(s)
reader sees fully built config: Config[url=jdbc:postgresql://db/shop, poolSize=10]
```

Without `volatile`, double-checked locking is **broken**: the write of the reference can become visible before the writes inside the constructor, so a second thread skips the lock and uses a half-built object. The holder idiom is usually simpler — the JVM initialises `Holder` exactly once, on first use, under its internal class-initialisation lock. For **singletons** (classes meant to have exactly one instance), an `enum` with one constant is simplest of all (*Effective Java* Item 3; see LLD Chapter 8).

Common mistakes

- Believing "it worked in testing" proves correctness — reorderings depend on CPU architecture (ARM, as on this laptop, reorders more than x86 — the Intel/AMD chip family), JIT decisions and load.
- Synchronising writes but reading without the lock (no happens-before for the reader).
- Using `volatile` for a reference to a *mutable* object and assuming later changes to its fields are visible — only the reference write is covered.

In the interview

> **30-second answer:** "The JMM defines when a write by one thread is guaranteed visible to another, through happens-before: program order, unlock→lock on the same monitor, volatile write→read, `start`, `join`, and the `java.util.concurrent` hand-offs, all transitive. Without a happens-before edge you have a data race and can see stale or reordered values. That's why double-checked locking needs `volatile`, and why immutable objects with final fields are always safe to share."

- *Does `volatile` make `i++` atomic?* → No; it gives visibility and ordering only.
- *What does `final` guarantee?* → Correct visibility of final fields after construction, if `this` doesn't escape.

More: LLD §12.6; JCIP chapter 16; JLS §17.4.

15.9 Liveness: deadlock, livelock, starvation

Safety means "nothing bad happens" (no races). **Liveness** means "something good eventually happens" — the program makes progress. Liveness failures:

Failure	What happens	Threads are…	Analogy
Deadlock	two or more threads each wait for a lock another holds, forever	`BLOCKED` / `WAITING`, 0% CPU	two cars at a one-lane bridge, each waiting for the other to reverse
Livelock	threads keep reacting to each other and retrying, but none progresses	running, busy CPU	two people in a corridor stepping aside in the same direction again and again
Starvation	one thread never gets the resource because others keep taking it	runnable but never lucky	the polite customer never served at a crowded bar

Deadlock

Four conditions (the **Coffman conditions**) must all hold: **mutual exclusion**, **hold and wait** (holding one lock while waiting for another), **no preemption** (locks can't be taken away), and **circular wait** (A waits for B, B waits for A, or a longer loop). Break any one and deadlock is impossible — in practice, you break *circular wait* or *hold-and-wait*.

`code/ch15/liveness/DeadlockDemo.java` (key parts):

```
static void transferBad(Account from, Account to, int amt, CountDownLatch bothHoldFirstLock) {
    synchronized (from) {
        bothHoldFirstLock.countDown(); bothHoldFirstLock.await();   // make the deadlock certain
        synchronized (to) { ... }
    }
}
// thread 1: transferBad(a, b, ...)     thread 2: transferBad(b, a, ...)
long[] ids = ManagementFactory.getThreadMXBean().findDeadlockedThreads();

// Fix 1 - global lock order: always lock the account with the lower id first
Account first = from.id < to.id ? from : to, second = first == from ? to : from;
synchronized (first) { synchronized (second) { ... } }

// Fix 2 - tryLock with timeout, release everything, random back-off, retry
if (from.lock.tryLock(10, MILLISECONDS)) { try { if (to.lock.tryLock(10, MILLISECONDS)) { ... } } finally {
from.lock.unlock(); } }
Thread.sleep(ThreadLocalRandom.current().nextInt(1, 5));
```

Output:

```
deadlock detected between 2 threads:
  transfer-a-to-b is BLOCKED, waiting for a lock held by transfer-b-to-a
  transfer-b-to-a is BLOCKED, waiting for a lock held by transfer-a-to-b
t1 alive=true, t2 alive=true (stuck forever; only a restart helps)
lock ordering: finished, total money = 2000
tryLock + back-off: finished, total money = 2000
```

- `ThreadMXBean.findDeadlockedThreads()` (`ThreadMXBean` is the JVM's built-in monitoring interface for threads; this is also what `jstack` and `jcmd Thread.print` report as "Found one Java-level deadlock") detects cycles among `synchronized` monitors and `java.util.concurrent` locks. Java can't *recover* from a deadlock — the threads can't be forced to release their locks — so prevention is the only real strategy. (The demo marks the stuck threads as daemons so the program can still exit.)
- Both fixes completed 2,000 opposite-direction transfers with money conserved.

Prevention checklist:

1. **Lock ordering** — acquire multiple locks in one global order (by id, by `System.identityHashCode` with a tie-breaker lock).
2. **Timeouts** — `tryLock(timeout)`, then release and retry with random back-off.
3. **Fewer locks, shorter holds** — don't call unknown code (callbacks, listeners, overridable methods) while holding a lock ("open calls" — JCIP 10.1.4).
4. **Higher-level tools** — a single-writer design, `ConcurrentHashMap` atomic methods, message passing via queues.
5. **Thread-starvation deadlock** in pools: tasks waiting for other tasks in the same bounded pool (Chapter 14 §14.11) — use separate pools, `CompletableFuture` composition, or virtual threads.

Livelock

Threads aren't blocked; they're busy responding to each other. Typical cause: every thread detects a conflict and backs off **in exactly the same way at the same time** — e.g. the `tryLock` version above *without* the random sleep can end up with both threads repeatedly grabbing their first lock, failing on the second, releasing, and trying again in lock-step. The fix is **randomised back-off** (wait a random, growing delay before retrying — as in Ethernet collision handling, where network cards that collide each wait a random time) or an ordering rule that decides who goes first. Also found in message processing: a message that always fails is put back at the head of the queue and retried forever (use a **dead-letter queue** — a side queue where messages that keep failing are parked for a human to inspect).

Starvation

A thread is ready to run but never gets the lock or CPU time:

- **Unfair locks**: `synchronized` and default `ReentrantLock` allow **barging** — a newly arriving thread can grab a just-released lock ahead of long waiters. That improves throughput but can starve a thread

under heavy contention. `new ReentrantLock(true)` / `new Semaphore(n, true)` grant in (roughly) arrival order, at a throughput cost.

- **Readers starving a writer** with read-write locks under constant read traffic (the fair mode of `ReentrantReadWriteLock` helps).
- **Thread priorities** — don't use them to fix starvation; they're only hints (Chapter 14 §14.2).
- **Long-running tasks** hogging a small pool while short tasks queue behind them — separate pools (bulkheads).

Detailed walk-throughs and the dining philosophers: LLD §12.17 Deadlock, §12.18 Livelock, §12.19 Starvation, LLD §13.5.

In the interview

> **30-second answer:** "Deadlock needs mutual exclusion, hold-and-wait, no preemption and a circular wait; I prevent it with a global lock order or `tryLock` with timeouts and back-off, and by not calling foreign code while holding locks. I diagnose it with a thread dump, which reports Java-level deadlocks. Livelock is threads busily retrying in lock-step — fixed with randomised back-off. Starvation is a thread never getting a resource — fairness settings, separate pools, or less contention fix it."

- *How do you find a deadlock in production?* → `jstack <pid>` / `jcmd <pid> Thread.print` (or `ThreadMXBean` in a **health check** — an endpoint or job that regularly reports whether the service is working).
- *Deadlock vs livelock CPU usage?* → ~0% vs high.

Self-check

1. Which Coffman condition does lock ordering break? → Circular wait.
2. Why random back-off rather than fixed? → Fixed delays keep contenders in lock-step → livelock.
3. What does a fair lock trade away? → Throughput.

15.10 Best practices and choosing a concurrency model

Best practices (JCIP, *Effective Java* Items 78–84, JDK guidance)

1. **Minimise shared mutable state.** Prefer immutable objects (records), confinement and message passing.
2. **Synchronise all access to shared mutable data** — reads and writes — with the same lock (Item 78).
3. **Keep synchronised regions small; never call alien code** (code you don't control, e.g. callbacks or overridable methods) **inside them** (Item 79).
4. **Prefer executors, tasks and streams to raw threads** (Item 80); prefer **concurrency utilities to** `wait` / `notify` (Item 81).
5. **Document thread safety** of every class you publish (Item 82).
6. **Lazy initialisation**: only when needed; use the holder idiom or volatile double-checked locking (Item 83).
7. **Don't depend on the thread scheduler** — no `Thread.yield`, priorities or busy-waits for correctness (Item 84).
8. **Handle interruption** properly — rethrow or restore the flag.
9. **Always bound things**: queue sizes, pool sizes, timeouts on `get` / `join` / `tryLock` /remote calls.
10. **Never lose exceptions** in async code: join, handle or log every `Future` / `CompletableFuture`.
11. **Use virtual threads for I/O-bound concurrency on Java 21+**; don't pool them; limit scarce resources with semaphores; avoid `ThreadLocal` caches.
12. **Prefer scoped values over** `ThreadLocal` for request context on Java 25+.
13. **Test and observe**: stress tests, thread dumps, JFR; but rely on happens-before reasoning, not on tests passing.

Choosing a model

Workload	Recommended tool (Java 25)
Many concurrent blocking calls (HTTP, DB, files)	virtual threads — `newVirtualThreadPerTaskExecutor()`, thread-per-request
Fan out to N services, combine, with cancellation	`CompletableFuture.allOf` today; `StructuredTaskScope` once final
CPU-heavy work on large in-memory data	parallel streams / Fork/Join / fixed pool of ≈ #cores platform threads
Periodic or delayed jobs	`ScheduledExecutorService`
Producer-consumer pipeline	`BlockingQueue` + executor (virtual threads are fine as consumers)
Shared counters, caches, registries	atomics, `LongAdder`, `ConcurrentHashMap`
Callback-based library integration	`CompletableFuture` as a promise (`complete`)
Very high-throughput event streaming with back-pressure (consumers signalling producers to slow down)	reactive libraries (Project Reactor, RxJava) or messaging systems — beyond this book

Real-world examples

- **Web servers**: Tomcat/Jetty with virtual threads let each request run plain blocking JDBC (Java Database Connectivity, the standard low-level database API) code at high concurrency.
- **API gateways / BFFs** (a gateway is the single entry point in front of many services; a BFF, *Backend For Frontend*, is a gateway tailored to one app — see System Design Fundamentals): fan out to several microservices per request (`CompletableFuture.allOf` with per-call `orTimeout` and fallbacks).
- **Batch analytics**: parallel streams over large in-memory collections; Fork/Join merge sorts.
- **Caches**: Caffeine (a popular in-memory caching library) builds on `ConcurrentHashMap` and `CompletableFuture` (async loading).
- **Observability** (being able to see what a running system is doing through logs, metrics and traces): request ids carried in logging context (MDC, `ThreadLocal`-based today, scoped values in newer designs).

Chapter TL;DR

Topic	Remember
`Future`	Blocking `get()`, no composition; `state()` / `resultNow()` since 19
`CompletableFuture`	`supplyAsync(…, executor)` → `thenApply` / `thenCompose` / `thenCombine` / `allOf` / `anyOf`; `exceptionally` / `handle` / `whenComplete`; `orTimeout` / `completeOnTimeout` (9); pass your own executor for I/O; never leave failures unobserved
Fork/Join	Recursive split + work stealing; `RecursiveTask`; fork one half, compute the other, join; powers parallel streams; CPU-bound only
Virtual threads (21)	Cheap JVM-managed threads on ~#cores carriers; unmount on blocking; huge throughput for waiting tasks; don't pool; limit with `Semaphore`; no help for CPU work
Pinning	Java 21–23: blocking in `synchronized` pinned carriers; Java 24 (JEP 491) fixed it; native callbacks and class initialisation still pin; `jdk.VirtualThreadPinned` JFR event
Context	`ThreadLocal`: per-thread, mutable, leaks with pools; `ScopedValue` (final 25): immutable, bounded to a call, inherited by structured subtasks
Structured concurrency	Preview in 25 (JEP 505), 26 (JEP 525), 27 (JEP 533): scope = unit of work, auto-cancel on failure, no leaked threads
JMM	Happens-before via program order, monitor unlock→lock, volatile write→read, start/join, j.u.c hand-offs; data race = stale/reordered reads; safe publication; DCL needs `volatile`; holder idiom
Liveness	Deadlock (Coffman; lock ordering, `tryLock`+timeouts; detect with thread dumps), livelock (random back-off), starvation (fairness, bulkheads)
Choosing	I/O-bound → virtual threads; CPU-bound → parallel streams/Fork/Join; fan-out → `CompletableFuture` (structured concurrency later)

Where next: what the JVM does underneath — memory areas, garbage collectors, JIT, profiling with JFR and benchmarking with JMH — in Chapter 16 — JVM internals and performance; the version-by-version feature tour in Chapter 17 — Modern Java 17 to 25. Practise with LLD Chapter 13 — Concurrency problems; exam traps in OCP Chapter 13; compare with Python's `asyncio` and GIL in Python Chapter 16.

Chapter 16 — JVM Internals and Performance

Chapter 1 gave the one-page picture: `javac` makes **bytecode** (the compact, CPU-independent instruction format stored in `.class` files); the JVM loads it lazily (only when first needed), **interprets** it (executes it instruction by instruction), **JIT-compiles** hot code (turns frequently run methods into native machine code *just in time*, while the program runs) and collects garbage (frees memory of objects nothing uses any more). This chapter opens the hood — with measurements: class loading, object sizes, five garbage collectors on one workload, the JIT deleting allocations, lying benchmarks, the JDK's diagnostic tools, a leak hunt (a **memory leak** is memory that is never released though no longer needed), a two-module JPMS (Java Platform Module System, §16.12) application, and (§16.13) the operating-system mechanics underneath every process.

Everything here is about **HotSpot**, the JVM inside OpenJDK (the open-source reference implementation of Java) and almost every vendor build (a company's packaged, supported distribution of it: Oracle, Temurin, Corretto, Zulu, Microsoft). Other JVMs (OpenJ9, GraalVM native images — programs compiled ahead of time into a standalone executable) differ in details.

What you'll learn

#	Topic	You will be able to…
16.1	JVM architecture	name the subsystems
16.2	Class loading	explain loading, linking, initialization, delegation, CDS (Class Data Sharing)
16.3	Runtime memory areas	map each area to its `OutOfMemoryError` (the error thrown when the JVM can't get more memory)
16.4	Object layout	estimate object sizes; headers, compressed oops (shrunken object references), compact headers
16.5	GC (garbage collection) algorithms	pick between Serial, Parallel, G1, ZGC, Shenandoah, Epsilon
16.6	Collectors compared	read `-Xlog:gc` and compare pauses on one workload
16.7	GC tuning basics	size the heap, set a goal, read `-XX:+PrintFlagsFinal`
16.8	JIT and escape analysis	explain C2's (the JVM's optimising compiler) optimisations and prove an allocation vanished
16.9	Benchmarking and JMH (Java Microbenchmark Harness)	avoid warm-up and dead-code traps
16.10	Profiling and diagnostics	use `jcmd`, `jstat`, JFR (Java Flight Recorder) and friends
16.11	Memory leaks	find a leak from growth, histograms and JFR
16.12	Modules (JPMS) basics	write `module-info.java`; use `jdeps` and `jlink`
16.13	OS fundamentals for engineers	explain processes vs threads, scheduling, virtual memory and page faults, syscalls, file descriptors, blocking vs epoll I/O and the page cache

How the code was run. Programs are in `code/` (package `ch16.*`), compiled with `javac --release 25`, run on **OpenJDK 26** (`26+35`), Apple M4 Pro (14 cores, 48 GiB — **GiB/MiB/KiB** are binary units: 1 GiB = 1024 MiB = 1024 × 1024 KiB — macOS). The JPMS example has its own `jpms/run.sh`. **All timings, byte counts, heap sizes and pauses are machine-dependent** — compare the *shape* of the results. Shenandoah isn't in the Oracle-built OpenJDK 26, so its numbers come from **Amazon Corretto 21.0.10**.

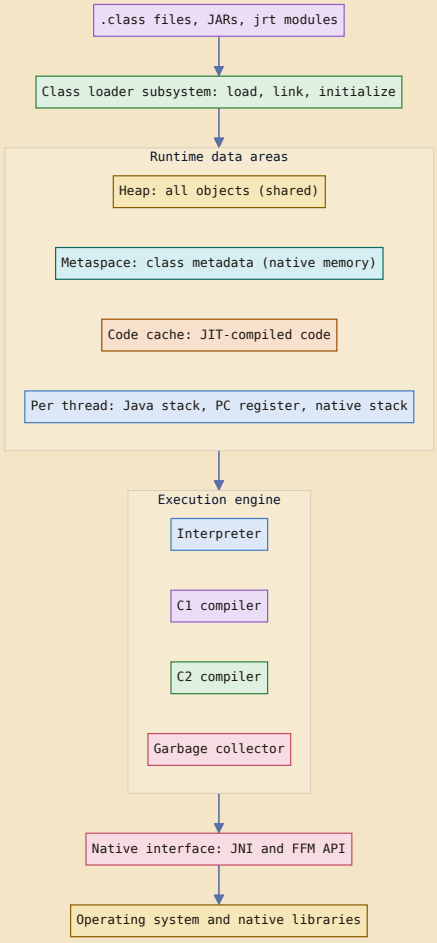
> **Exam depth.** Module directives, command-line options and service-loader traps are drilled in OCP Chapter 12 — Modules. Concurrency-related JVM behaviour (memory model, thread states) is in

> Chapter 15 and the LLD handbook's concurrency chapters.

16.1 The JVM's architecture

What it is

A **Java Virtual Machine** runs bytecode — a small operating system for Java programs: it loads code, gives each **thread** (an independent path of execution) a **stack** (per-call scratch memory for local variables), owns and cleans a **heap** (the shared memory area where objects live) of objects, and compiles bytecode to machine code on the fly. The *JVM Specification* says **what** a JVM must do; HotSpot decides **how** (which collectors, which compilers).



Subsystem	Job	Covered in
Class loader subsystem	Find class bytes, verify them, prepare statics, run static initialisers	16.2
Runtime data areas	Memory the JVM manages for you	16.3, 16.4
Interpreter + JIT compilers	Execute bytecode; compile hot code to machine code	16.8, Chapter 1 §1.7
Garbage collector	Reclaim unreachable objects, compact memory	16.5–16.7
Native interface	Call C code: JNI (Java Native Interface, old) or the Foreign Function & Memory API (Java 22, Chapter 17)	—
Serviceability	Tools and events that observe a live JVM: JFR, `jcmd`, JMX (Java Management Extensions, the standard API for monitoring a running JVM)	16.10

16.2 Class loading in depth

What it is

Class loading is how a class goes from bytes on disk to something the JVM can execute. It happens **lazily**: a class is loaded the first time the program needs it, not when the program starts. Analogy: a library that fetches a book from the warehouse only when a reader first asks for it, checks it isn't damaged (verification), shelves it, and the first reader cuts the ribbon (initialization).

How it works

1. **Loading** — a *class loader* finds the bytes (`.class` file, JAR entry — a **JAR** is a zip file of compiled classes — or the JDK's `jrt:` image) and creates a `Class` object plus **metadata** (the JVM's internal description of the class: methods, fields, bytecode) in **metaspace** (§16.3).
2. **Linking** — **verification** (the bytecode is **type-safe** — never treats a value as a type it isn't — so hostile class files can't crash the JVM); **preparation** (static fields get default values); **resolution** (symbolic references like "`println` in `java/io/PrintStream`" become direct pointers, lazily on first use).
3. **Initialization** — static initialisers and `static { }` blocks run top to bottom, exactly once, under a lock.

What triggers initialization? Creating an instance (`new`), calling a static method, reading or writing a static field that is **not** a compile-time constant, initialising a subclass (the superclass goes first), **reflection** (code inspecting or using classes by name at run time) like `Class.forName("X")`, or being the main class. Merely *loading* a class (for example because it is named in a signature) does not initialise it.

The loader hierarchy and parent delegation

Loader	Loads	`getClassLoader()` returns
Bootstrap (inside the JVM, written in C++)	`java.base` and other core modules	`null`
Platform	Other Java SE / JDK modules such as `java.sql`	the platform loader
Application (also called *system*)	Your class path (the list of folders/JARs searched for classes) and module path (the same for modules, §16.12)	the app loader
Custom (`new URLClassLoader(…)`, frameworks)	Plugins, web apps in **app servers** (servers like Tomcat that host several deployed web applications), hot-reload tools (reload changed code without a restart)	your loader

A loader first **delegates to its parent**; only if the parent can't find the class does it look itself. So nobody can sneak in a fake `java.lang.String`: the bootstrap loader always wins. A class's identity is **its name plus its defining loader** — the same `.class` file loaded by two loaders gives two different, incompatible classes (the source of the infamous "`ClassCastException`: `Foo` cannot be cast to `Foo`" in app servers).

Seeing it: `code/ch16/classload/LoadingDemo.java`

```
static class Config {
    static final String NAME = "compile-time constant"; // inlined by javac: does NOT trigger init
    static final long STARTED = System.nanoTime();       // not a constant: reading it DOES trigger init
    static { System.out.println("  [Config static initializer runs]"); }
}
static class Parent { static { System.out.println("  [Parent initialized]"); } static int shared = 1; }
static class Child extends Parent { static { System.out.println("  [Child initialized]"); } }

// in main:
System.out.println("2. reading Config.NAME   -> " + Config.NAME);
long t = Config.STARTED;          // 3.
int s = Child.shared;             // 4. field declared in Parent
new Child();                      // 5.
show("java.lang.String", String.class);
show("java.sql.Date", Class.forName("java.sql.Date"));
```

Output:

```
1. main started
2. reading Config.NAME   -> compile-time constant
3. reading Config.STARTED (triggers init):
  [Config static initializer runs]
4. Child.shared via subclass name (only Parent initializes):
  [Parent initialized]
5. new Child():
  [Child initialized]
6. Who loaded what?
   java.lang.String  loaded by bootstrap (null)
   java.sql.Date     loaded by platform
   LoadingDemo       loaded by app
   app loader's parent  : platform
   platform's parent    : null (bootstrap is native)
   module of String     : java.base
   module of LoadingDemo: unnamed module (class path)
```

What to notice:

- Step 2 printed nothing from `Config`: `NAME` is a `static final` String initialised with a literal — a **constant variable** — so `javac` copied its value into `main`'s bytecode (**inlined** it).
- Step 4 initialised only `Parent`: a static field is "owned" by the class that declares it.

The class-loading log

`java -Xlog:class+load=info -cp out ch16.classload.LoadingDemo` prints one line per loaded class (real output; paths shortened, timestamps machine-dependent):

```
[0.006s][info][class,load] java.lang.Object source: shared objects file
...
[0.016s][info][class,load] ch16.classload.LoadingDemo source: file:.../out/
[0.017s][info][class,load] ch16.classload.LoadingDemo$Config source: file:.../out/
...
[0.017s][info][class,load] ch16.classload.LoadingDemo$Child source: file:.../out/
...
[0.023s][info][class,load] java.sql.Date source: jrt:/java.sql
```

The run loaded **786 classes**: 770 from the *shared objects file*, 9 from `jrt:/` (the JDK's module image), the rest from `out/`. `Child` is **loaded** at step 4 but **not initialised** until step 5 — separate steps. The **shared objects file** is **Class Data Sharing (CDS)**: a pre-parsed, pre-verified archive of core classes mapped straight into memory, which is why Hello World starts in tens of milliseconds. The JDK ships a default archive (JEP 341, Java 12 — a JEP, *JDK Enhancement Proposal*, is the numbered design document behind a Java feature); you can archive your own classes (AppCDS, JEP 310, Java 10) and, since Java 24–25, a full **AOT cache** (*ahead-of-time*: work done before the program runs and saved for reuse) that also records linked classes and method profiles (JEPs 483, 514, 515 — see Chapter 1 §1.7).

Common mistakes

- Expecting a `static {}` block to run at program start. It runs at first *active use*.
- Heavy work in static initialisers. An exception there becomes `ExceptionInInitializerError`, and every later use of the class throws `NoClassDefFoundError` — the class is permanently broken for that loader.
- Confusing `ClassNotFoundException` (checked; thrown by `Class.forName`/`loadClass` when a name can't be found) with `NoClassDefFoundError` (an error; the class was present at compile time but is missing or failed to initialise at run time).
- Holding references to a custom loader's classes from shared code: that loader and all its classes can never be unloaded — the *classloader leak* (16.11) behind `OutOfMemoryError: Metaspace` after **redeploys** (replacing a running application with a new version without restarting the server).

In the interview

Q: Walk me through what happens when the JVM first uses a class. It is loaded by a class loader using parent delegation (bootstrap → platform → application), linked (verified, statics prepared with defaults, references resolved lazily), then initialised: static initialisers run once, in textual order, under a lock.

- *Why parent delegation?* Core classes can't be spoofed, and each class is loaded once per hierarchy.
- *What is CDS?* A memory-mapped archive of pre-processed classes: faster start-up, metadata shared between JVMs.

Self-check

1. Does reading a `static final int MAX = 10` initialise its class? → No, it is a compile-time constant inlined by `javac`.
2. What does `String.class.getClassLoader()` return? → `null` (the bootstrap loader).
3. Name the three linking steps. → Verification, preparation, resolution.

TL;DR

Load (find bytes) → link (verify, prepare, resolve) → initialise (static code, once). Lazy. Loaders delegate to parents first. CDS/AOT caches pre-do this work for faster start-up.

16.3 Runtime memory areas

What it is

The JVM splits its memory into areas with different owners and lifetimes. Knowing them tells you *which* `OutOfMemoryError` you are looking at and which flag controls it.

Area	Per thread or shared	Holds	Size flag	When it runs out
Heap	Shared	All objects and arrays	`-Xms` (initial), `-Xmx` (max), `-XX:MaxRAMPercentage`	`OutOfMemoryError: Java heap space` (also `GC overhead limit exceeded` when GC time dominates)
Java stack	Per thread	One **frame** per active method call: local variables, operand stack (the scratch stack bytecode instructions push and pop values on), return address	`-Xss` / `Thread` constructor `stackSize`	`StackOverflowError`
PC register (program counter)	Per thread	Address of the current bytecode instruction	—	—
Native method stack	Per thread	Frames of native (C) code	OS	crash or `StackOverflowError`
Metaspace	Shared, native memory (memory the JVM gets straight from the OS, outside the Java heap)	Class metadata: methods, bytecode, constant pools (each class's table of literals and symbolic references), field layouts	`-XX:MaxMetaspaceSize` (unlimited by default)	`OutOfMemoryError: Metaspace`
Compressed class space	Part of metaspace	Class structures addressed by compressed class pointers	`-XX:CompressedClassSpaceSize` (1 GiB default)	`OutOfMemoryError: Compressed class space`
Code cache	Shared	JIT-compiled machine code	`-XX:ReservedCodeCacheSize`	Warning "CodeCache is full. Compiler has been disabled" — the app keeps running, slowly
Direct / native memory	Shared	`ByteBuffer.allocateDirect`, FFM (Foreign Function & Memory API) `Arena` memory, thread stacks, GC data structures	`-XX:MaxDirectMemorySize`	`OutOfMemoryError: Direct buffer memory` / `unable to create native thread`

Since **Java 8** (JEP 122) class metadata lives in **metaspace** (native memory) instead of the old fixed-size *PermGen*, so the classic `PermGen space` error is gone — but an unbounded metaspace can still grow with a classloader leak.

Seeing it: `code/ch16/memory/MemoryAreas.java`

The `java.lang.management` API reports the pools the running JVM actually has; a **recursion** (a method calling itself) in threads of different stack sizes measures stack depth.

```
for (MemoryPoolMXBean p : ManagementFactory.getMemoryPoolMXBeans()) {
    MemoryUsage u = p.getUsage();
    ... p.getType(), p.getName(), u.getUsed() / 1024, u.getMax() / MiB ...
}
for (long stackSize : new long[] {256 * 1024, 1024 * 1024, 8 * 1024 * 1024}) {
    depth = 0;
    Thread t = new Thread(null, () -> {
        try { recurse(); } catch (StackOverflowError e) { /* expected */ }
    }, "deep", stackSize);
    t.start(); t.join();
}
```

Output (machine-dependent: default G1, 48 GiB machine so max heap = 12 288 MiB):

```
Garbage collector(s):
  G1 Young Generation
  G1 Concurrent GC
  G1 Old Generation
Memory pools (type, used KiB / max MiB):
  NON-HEAP CodeHeap 'non-nmethods'               1174 / 5
  NON-HEAP Metaspace                              738 / no limit
  NON-HEAP CodeHeap 'profiled nmethods'           138 / 117
  NON-HEAP Compressed Class Space                  62 / 1024
  HEAP     G1 Eden Space                            0 / no limit
  HEAP     G1 Old Gen                               0 / 12288
  HEAP     G1 Survivor Space                        0 / no limit
  NON-HEAP CodeHeap 'non-profiled nmethods'        38 / 117
Stack   256 KiB -> StackOverflowError after ~1,473 frames
Stack  1024 KiB -> StackOverflowError after ~23,200 frames
Stack  8192 KiB -> StackOverflowError after ~206,700 frames
```

- The **code cache** has three segments: JVM stubs (small helper routines), C1 code still **profiling** (recording how the code is actually used) ("profiled nmethods" — *nmethod* is HotSpot's name for a compiled method), and final C2 code ("non-profiled").
- Depth isn't proportional to stack size: at 256 KiB the method overflowed while still **interpreted** (big frames); with bigger stacks it got **JIT-compiled** (small frames). Never rely on an exact depth.
- Under `-XX:+UseZGC` the pools become `ZGC Young/Old Generation`.

Native memory tracking

The heap is only part of a Java process's memory. **Native Memory Tracking (NMT)** accounts for the rest. Start the JVM with `-XX:NativeMemoryTracking=summary`, then ask it: `jcmd <pid> VM.native_memory summary scale=MB` (real output for the leak demo of 16.11 with `-Xmx256m`, trimmed):

```
Total: reserved=1722MB, committed=278MB
-                 Java Heap (reserved=256MB, committed=186MB)
-                     Class (reserved=1024MB, committed=0MB)
-                      Code (reserved=244MB, committed=8MB)
-                        GC (reserved=60MB, committed=59MB)
...
```

Reserved = **address space** (a range of memory addresses) set aside; *committed* = backed by RAM. The **GC's own bookkeeping** (59 MB: remembered sets — records of which old objects point to young ones — and mark bitmaps) and other areas push committed memory (278 MB) past the heap — why a 512 MB **container** (an isolated, resource-limited process environment, e.g. Docker) must not get `-Xmx512m`.

Common mistakes

- Sizing a container by `-Xmx` alone. Budget heap + metaspace + code cache + thread stacks (roughly 1 MB each by default, 2 MB on macOS/AArch64 — AArch64 being 64-bit ARM, as in Apple Silicon) + direct buffers + GC overhead. A common start is `-XX:MaxRAMPercentage=75` and measure.
- "Fixing" `StackOverflowError` with a giant `-Xss`. Usually the recursion has no base case, or should be a loop (Chapter 4 §4.5).

Self-check

1. Which area holds a method's local `int` variables? → That thread's Java stack frame.
2. What replaced PermGen, and in which release? → Metaspace, Java 8.
3. The heap is at 40% but the process was OOM-killed (terminated by the OS for exceeding its memory limit, §16.13) by the container. Where do you look? → Native memory: NMT summary, thread count, direct buffers, metaspace.

TL;DR

Heap = objects (`-Xmx`); stack = frames per thread (`-Xss`); metaspace = class metadata; code cache = JIT output; plus native memory. Each has its own error message — read it.

16.4 Object layout basics

What it is

Every heap object carries a hidden **header** before its fields, and sizes are rounded up to a multiple of 8 bytes (**alignment**). Knowing this explains why a million tiny objects cost more than you'd guess, and why `int[]` beats `List<Integer>` for big numeric data.

On 64-bit HotSpot with default settings:

Part	Size	Contains
Mark word	8 bytes	Identity hash code (the default `hashCode()` value), lock state, GC age bits (how many collections the object survived)
Class pointer	4 bytes (*compressed class pointers*)	Which class this object is
Array length (arrays only)	4 bytes	`length`
Fields	`long` / `double` 8, `int` / `float` 4, `short` / `char` 2, `byte` / `boolean` 1, **reference 4** (*compressed oops*)	Your data, reordered by the JVM to reduce gaps
Padding	0–7 bytes	Round total up to a multiple of 8

Compressed oops ("ordinary object pointers"): with heaps below about 32 GB the JVM stores references as 32-bit offsets and shifts them by 3 (because objects are 8-byte aligned), so each reference costs 4 bytes instead of 8. That is why going from `-Xmx31g` to `-Xmx33g` can *reduce* usable capacity.

Compact object headers (JEP 519, a product feature in **Java 25**, opt-in with `-XX:+UseCompactObjectHeaders`; made the default in JDK 27 by JEP 534) squeeze mark word and class pointer into a single **8-byte** header.

Seeing it: `code/ch16/layout/ObjectSizes.java`

No library needed: HotSpot counts the bytes each thread allocates. Allocate a million objects into an array (so the JIT can't remove them) and divide.

```
static double bytesPerObject(Maker m) {
    for (int i = 0; i < N; i++) sink[i] = m.make(i);          // warm-up round
    long before = THREADS.getCurrentThreadAllocatedBytes();
    for (int i = 0; i < N; i++) sink[i] = m.make(i);
    long after = THREADS.getCurrentThreadAllocatedBytes();
    return (after - before) / (double) N;
}
... bytesPerObject(i -> new Point(i, i)) ...    // record Point(int x, int y)
```

Output of `java -cp out ch16.layout.ObjectSizes`, then with `-XX:+UseCompactObjectHeaders` (JDK 26):

Object	Default	Compact headers	Why (default)
`new Object()`	16.0	8.0	12-byte header → padded to 16
`record Point(int x, int y)`	24.0	16.0	12 + 4 + 4 = 20 → 24
`Node { Object value; Node next; }`	24.0	16.0	12 + 4 + 4 (compressed refs) = 20 → 24
`Long` (not cached)	24.0	16.0	12 + 8 = 20 → 24
`new int[0]`	16.0	16.0	12 + 4 length = 16
`new int[10]`	56.0	56.0	16 + 40 = 56
`new byte[10]`	32.0	24.0	16 + 10 = 26 → 32
a 6-character `String`	48.0	48.0	`String` object 24 + its `byte[6]` 24

What this means in practice:

- A `List<Integer>` of a million values costs ~4 MB of references **plus** ~16 MB of `Integer`s (outside the -128..127 cache — §7.6); an `int[]` costs 4 MB — 5× less.
- A two-`int` record is 24 bytes for 8 bytes of data: headers dominate small objects. Compact headers cut them by a third (JEP 519 reports 22% less heap and 8% less CPU on SPECjbb2015, an industry-standard Java server benchmark).
- 6 ASCII characters cost 48 bytes as a `String` (Latin-1 — a one-byte-per-character encoding covering Western European text — strings use 1 byte per char — Java 9 *compact strings*).

When not to care

Most business code allocates moderately and the GC handles it. Worry about layout when profiling (16.10) shows a type with millions of instances dominating the heap — then consider primitive arrays, fewer wrapper objects, or compact headers.

Self-check

1. Why does `new Object()` take 16 bytes when the header is 12? → Alignment to 8 bytes.
2. What does "compressed oops" save, and when is it disabled? → 4 bytes per reference; disabled for heaps of about 32 GB and above.
3. How do you turn on compact object headers in Java 25? → `-XX:+UseCompactObjectHeaders` (default from JDK 27).

TL;DR

Header 12 bytes (8 with compact headers), references 4 bytes (compressed oops, heap < ~32 GB), everything padded to 8. Small objects are mostly header; primitive arrays are dense.

16.5 Garbage-collection algorithms

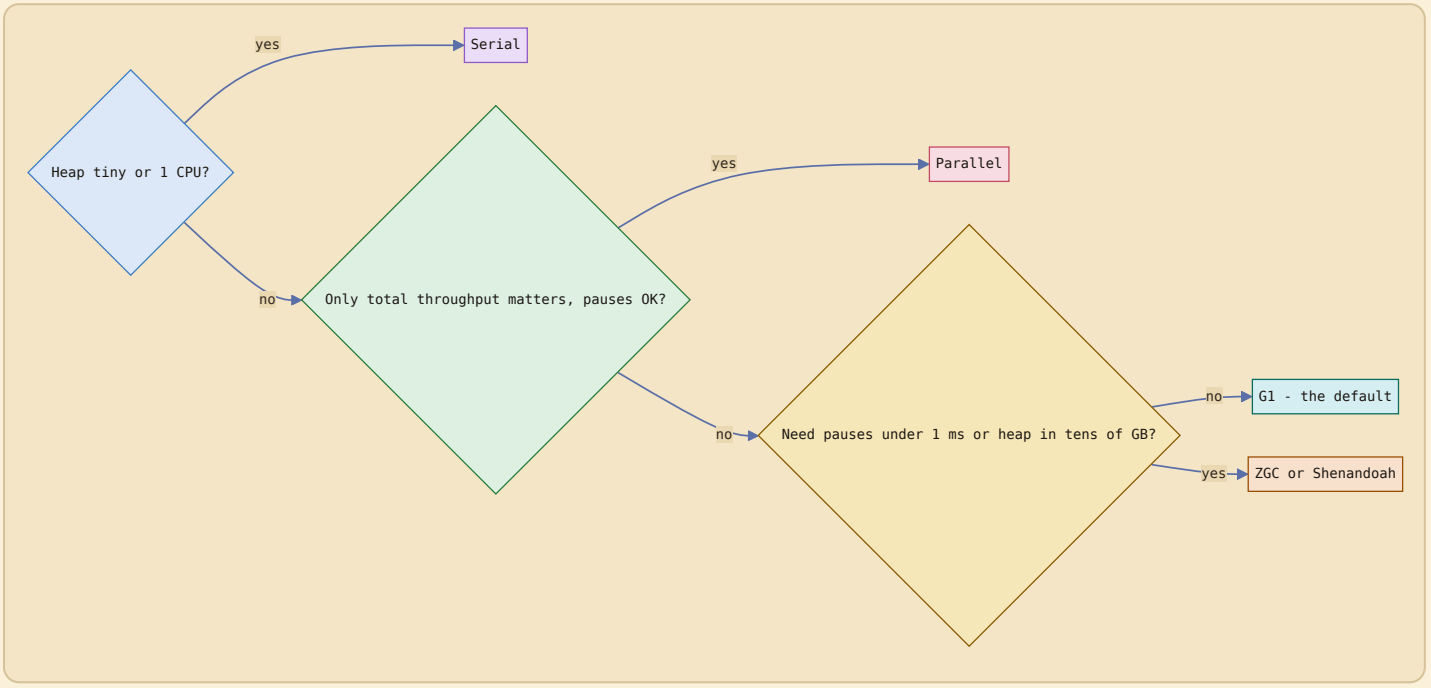
Chapter 1 §1.8 explained **reachability** (an object is alive if a chain of references leads to it), **GC roots** (the starting points of those chains: local variables, static fields, active threads) and the **generational** idea (young and old objects are kept apart and collected differently). Here are the mechanics behind each HotSpot collector.

The building blocks

Technique	How it works	Cost	Used by
Mark	Trace from GC roots, marking every reachable object	Proportional to **live** objects	All
Sweep	Walk memory, add unmarked space to free lists (lists of reusable gaps)	Proportional to heap size; leaves **fragmentation** (free memory split into many small gaps)	Old gen of Serial (with compaction)
Compact	Slide live objects together, fix up references	Proportional to live data; no fragmentation	Serial/Parallel old gen, G1 full GC
Copy (evacuate)	Copy live objects into an empty area; the old area is freed wholesale	Proportional to live data; needs spare space	Young generations everywhere; G1, ZGC, Shenandoah for all regions
Generations	Collect the young area often (most objects are already dead), the old area rarely	Needs to track old→young references (**card table / remembered sets** — side tables noting which old areas point into young ones)	Serial, Parallel, G1, generational ZGC and Shenandoah
Concurrent phases	GC threads work *while* application threads run	Needs **barriers** — small code the JIT inserts around reference reads/writes — and extra CPU	G1 marking, ZGC, Shenandoah

Terms: a **stop-the-world (STW) pause** halts all application threads at a **safepoint** (where every thread's state is known). **Throughput** = share of time running your code, **latency** = pause lengths, **footprint** = memory needed (System Design Fundamentals explains throughput and latency more generally); a collector optimises two, rarely all three.

The collectors



Collector	Since	How it works	Pauses	Choose it for
Serial `-XX:+UseSerialGC`	The original	One GC thread; copying young gen, mark-sweep-compact old gen; all STW	Grow with heap size	Small heaps, 1 CPU, CLI tools (command-line programs), small containers
Parallel `-XX:+UseParallelGC`	Default before Java 9	Serial's design with many GC threads	Shorter, still grow with live data	Batch jobs: max throughput, pauses irrelevant
G1 (Garbage-First) `-XX:+UseG1GC`	Default since Java 9 (JEP 248)	Equal **regions** (1–32 MB, sized from the heap). Young regions evacuated in parallel STW pauses; old regions **marked concurrently**, then **mixed** collections evacuate the old regions with the most garbage first (hence the name). Objects ≥ half a region are **humongous** (own regions). *Evacuate* = copy live objects out so the region can be freed. Full GC only as a fallback	Aims at `-XX:MaxGCPauseMillis` (default 200 ms); typically a few to tens of ms	General purpose — the default for most services
ZGC `-XX:+UseZGC`	Experimental 11, production 15, generational 21 (JEP 439), generational-only since 24 (JEP 490)	Nearly everything concurrent, including **relocation** (moving objects). **Coloured pointers** keep GC state in spare bits of each reference; **load barriers** fix references as the app reads them	Well under 1 ms, independent of heap size (up to 16 TB)	Latency-sensitive services, large heaps
Shenandoah `-XX:+UseShenandoahGC`	Experimental 12, production 15 (JEP 379); generational mode product in 25 (JEP 521)	Concurrent marking **and** concurrent compaction using load-reference barriers	Sub-millisecond to a few ms	Low latency; shipped in most OpenJDK builds (Temurin, Corretto, Red Hat) but not in Oracle's builds
Epsilon `-XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC`	Java 11 (JEP 318)	Allocates, never frees	None — until the heap is full and the JVM exits	Performance testing, ultra-short jobs

The removed **CMS** collector (removed in Java 14, JEP 363) is still mentioned in old articles and flags; it no longer exists.

Default choice: G1, unless the JVM sees fewer than 2 CPUs or less than 1792 MB of memory, where it picks Serial. JDK 27 (JEP 523) makes G1 the default in all environments.

How G1 collects, step by step

1. New objects go into **eden** regions (fast: each thread **bump-allocates** — just moves a pointer forward — in its own *TLAB*, thread-local allocation buffer).
2. When eden fills, a **young pause** copies live objects to **survivor** regions; objects that survived enough collections (up to `MaxTenuringThreshold` = 15) are **promoted** (moved) to old regions.
3. When the old generation passes a threshold (*IHOP*, Initiating Heap Occupancy Percent, adaptive), a **concurrent mark cycle** starts: short STW pauses begin and end it (*Concurrent Start*, *Remark*, *Cleanup*); the tracing runs alongside the app.
4. Then several **mixed** pauses evacuate young regions plus a slice of old regions ordered by garbage.
5. If old regions can't be reclaimed fast enough — allocation outruns marking — an **evacuation failure** and eventually a **Full GC** (STW, multi-threaded compaction) occur. Seeing `Pause Full` in a G1 log is a tuning signal (16.6).

In the interview

Q: Compare G1 and ZGC. Both are region-based and generational. G1 copies live objects in stop-the-world pauses aimed at a goal (200 ms default, usually far less) with lower CPU and memory overhead. ZGC also moves objects concurrently (coloured pointers, load barriers), so pauses stay under a millisecond whatever the heap size, at some cost in throughput and **headroom** (spare free memory).

- *Why generational?* Most objects die young, so collecting only the young area is cheap and productive.
- *Why do concurrent collectors need barriers?* The app changes references while the GC traces or moves objects; barriers record or repair those changes.

Self-check

1. Which collector would you use for a nightly batch job on 32 cores? → Parallel (throughput).
2. What does "Garbage-First" refer to? → Collecting the old regions with the most garbage first in mixed collections.
3. Since which release is ZGC generational-only? → Java 24 (JEP 490); generational mode arrived in 21 and became the default in 23.
4. Name one collector not in Oracle's JDK builds. → Shenandoah.

TL;DR

Serial (tiny), Parallel (throughput), **G1 (default, balanced)**, ZGC and Shenandoah (concurrent compaction, sub-ms pauses), Epsilon (no GC). All modern ones are generational and region-based.

16.6 Collectors compared on a real workload

The workload: `code/ch16/gc/AllocWorkload.java`

A server-like pattern: a "**cache**" (data kept in memory for reuse) of 100 000 byte arrays (~100 MB) with random replacement, so objects live a while, get promoted, then die in the old generation — the hard case for generational GC. A **heartbeat** thread sleeps 1 ms in a loop and records its worst extra delay: the latency users would feel.

```
Thread heartbeat = Thread.ofPlatform().daemon().start(() -> {
    long last = System.nanoTime();
    while (running) {
        try { Thread.sleep(1); } catch (InterruptedException e) { return; }
        long now = System.nanoTime();
        long stall = (now - last) / 1_000 - 1_000;           // time beyond the 1 ms we asked for
        if (stall > worstStallMicros) worstStallMicros = stall;
        last = now;
    }
});
byte[][] cache = new byte[100_000][];                         // the live set
SplittableRandom rnd = new SplittableRandom(42);              // fixed seed: same work every run
for (long i = 0; i < allocations; i++) {
    byte[] b = new byte[64 + rnd.nextInt(1_984)];            // 64..2047 bytes, avg ~1 KiB
    b[0] = (byte) i;
    bytes += b.length;
    int slot = rnd.nextInt(cache.length);
    if (cache[slot] != null) checksum += cache[slot][0];     // read the entry we evict
    cache[slot] = b;                                          // old entry becomes garbage
}
```

G1 vs ZGC with `-Xlog:gc`

`java -Xmx1g -XX:+UseG1GC -Xlog:gc -cp out ch16.gc.AllocWorkload` (real output, start and end):

```
[0.004s][info][gc] Using G1
[0.019s][info][gc] GC(1) Pause Young (Concurrent Start) (G1 Evacuation Pause) 6M->6M(10M) 0.322ms
[0.019s][info][gc] GC(2) Concurrent Mark Cycle
[0.021s][info][gc] GC(2) Pause Remark 12M->12M(33M) 0.112ms
[0.021s][info][gc] GC(2) Pause Cleanup 14M->14M(33M) 0.012ms
[0.021s][info][gc] GC(2) Concurrent Mark Cycle 1.846ms
...
[0.601s][info][gc] GC(49) Pause Young (Mixed) (G1 Evacuation Pause) 746M->282M(1013M) 4.061ms
allocated      : 6,331 MB of payload in 6,000,000 arrays (checksum -2929001)
elapsed        : 641 ms wall, 1,673 ms CPU (whole process)
  G1 Young Generation    count   44, time   132 ms
  G1 Concurrent GC       count   14, time     2 ms
  G1 Old Generation      count    0, time     0 ms
worst stall    : 9.1 ms (seen by the heartbeat thread)
```

Read a G1 line as: GC number, pause kind (Young Normal / Concurrent Start / Prepare Mixed / Mixed / Remark / Cleanup / Full), cause, `before->after(heap size)`, pause length. The concurrent mark cycle itself is not a pause — only Remark and Cleanup are.

`java -Xmx1g -XX:+UseZGC -Xlog:gc -cp out ch16.gc.AllocWorkload`:

```
[0.004s][info][gc] Using The Z Garbage Collector
[0.053s][info][gc] GC(1) Major Collection (Warmup)
[0.059s][info][gc] GC(1) Major Collection (Warmup) 206M(20%)->148M(14%) 0.006s
...
[0.581s][info][gc] GC(13) Minor Collection (Allocation Rate) 640M(62%)->228M(22%) 0.011s
allocated      : 6,331 MB of payload in 6,000,000 arrays (checksum -2929001)
elapsed        : 580 ms wall, 947 ms CPU (whole process)
  ZGC Minor Cycles       count   10, time   110 ms
  ZGC Minor Pauses       count   30, time     0 ms
  ZGC Major Cycles       count    4, time    31 ms
  ZGC Major Pauses       count   20, time     0 ms
worst stall    : 0.8 ms (seen by the heartbeat thread)
```

The time at the end of a ZGC line (`0.011s`) is the **whole concurrent cycle**, not a pause. The actual pauses are logged under `gc+phases` (`-Xlog:gc+phases`):

```
[0.034s][info][gc,phases] GC(0) Y: Pause Mark Start (Major) 0.005ms
[0.039s][info][gc,phases] GC(0) Y: Pause Mark End 0.007ms
[0.039s][info][gc,phases] GC(0) Y: Pause Relocate Start 0.002ms
```

Five **micro**seconds. `Minor` = young-only collection, `Major` = young + old; `Y:`/`O:` = young/old generation.

Five collectors, same program

Three runs each with `-Xmx1g`, pauses parsed from `-Xlog:gc*` (Shenandoah on Corretto 21.0.10; the rest on OpenJDK 26). **Machine-dependent numbers** — M4 Pro, 14 cores:

Collector	Wall time	Process CPU	Pauses	Longest pause	Worst stall seen by app
Serial	1 570–1 590 ms	~1 670 ms	42	**128–133 ms**	128–133 ms
Parallel	690–700 ms	~1 940 ms	62	5–10 ms	5–10 ms
G1 (default)	630–660 ms	~1 670 ms	58	9–17 ms	8–17 ms
ZGC	585–605 ms	~950–1 010 ms	~50	**0.010–0.015 ms**	0.3–1.8 ms
Shenandoah (JDK 21)	560–570 ms	~870 ms	48	0.18–0.25 ms	0.4–0.8 ms

How to read it:

- **Serial** works on one thread while the app waits: slowest, with 130 ms freezes. **Parallel** spreads the pauses over 11 GC threads: short pauses, most CPU. **G1** matches Parallel here, with tunable pause goals.
- **ZGC/Shenandoah** pauses are tiny; the heartbeat's leftover sub-millisecond "stall" is OS scheduling noise. On this many-core machine they also finished fastest; in a CPU-starved container their concurrent GC threads compete with yours and the picture can flip — measure on production-like hardware.
- JFR agrees (16.10): `jfr view gc-pauses` reported **P99 8.40 ms** (99% of pauses were this short or shorter — the 99th **percentile**) for G1 and **P99 0.012 ms** for ZGC on a 20-million-allocation run.

When the heap is too small

The same G1 run with `-Xmx512m` (real log excerpt):

```
[0.207s][info][gc] GC(42) Pause Young (Normal) (G1 Evacuation Pause) (Evacuation Failure: Allocation) 500M->511M(512M) 1.864ms
[0.208s][info][gc] GC(43) Pause Young (Normal) (G1 Evacuation Pause) (Evacuation Failure: Allocation) 512M->512M(512M) 1.192ms
[0.215s][info][gc] GC(44) Pause Full (G1 Compaction Pause) 512M->104M(367M) 6.408ms
```

At ~6.5 GB/s of allocation and ~400 MB of headroom, G1's marking couldn't keep up, evacuation failed (no free region for survivors) and G1 fell back to **six Full GCs** in under a second. The heap after the full GC (104M) is the real **live set** (the memory taken by objects that are still reachable). The fix is headroom, not a magic flag: at `-Xmx1g` there were no full GCs. ZGC at 512 MB still had no allocation stalls (longest pause 0.019 ms).

TL;DR

Read `before->after(size) pause` in G1 logs; for ZGC, pauses are in `gc+phases`. Serial pauses scale with the heap; G1 gives single-digit milliseconds here; ZGC/Shenandoah give microseconds. `Pause Full` or `Evacuation Failure` in G1 = not enough headroom.

16.7 GC tuning basics

The problem it solves

Default settings are good for most programs. Tuning is for when you have a **measured** problem: pauses over an SLA (service-level agreement), throughput too low, or memory too high.

How to tune (in this order)

1. **Define the goal** — "p99 latency < 50 ms", "batch done in 20 min", "fits a 2 GB container".
2. **Log GC permanently** (cheap): `-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m`.
3. **Size the heap**: `-Xmx`, or `-XX:MaxRAMPercentage` in containers (**cgroup** limits — the Linux kernel feature containers use to cap CPU and memory — are detected on Linux). Rule of thumb: 3–4× the **live set** (heap after a full or mixed GC). `-Xms` = `-Xmx` avoids resizing in long-running services.
4. **Pick the collector** (16.5), then **set a goal, not mechanisms**: G1 `-XX:MaxGCPauseMillis=100`; ZGC mostly needs only `-Xmx`.
5. **Change one thing at a time** and re-measure under realistic load.
6. **Reduce allocation** in code — often the biggest win (JFR allocation views, 16.10, show where).

The flags you'll actually use

Flag	Meaning
`-Xms<size>` / `-Xmx<size>`	Initial / maximum heap
`-XX:MaxRAMPercentage=75`	Max heap as % of machine or container memory (default 25)
`-XX:+UseG1GC`, `+UseZGC`, `+UseParallelGC`, `+UseSerialGC`, `+UseShenandoahGC`	Select a collector
`-XX:MaxGCPauseMillis=<ms>`	G1/Parallel pause goal (a hint, not a guarantee)
`-Xss<size>`	Thread stack size
`-XX:MaxMetaspaceSize=<size>`	Cap metaspace (turns a classloader leak into a clear error)
`-Xlog:gc*`	Unified GC logging (Java 9+; replaces `-XX:+PrintGCDetails`)
`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=<dir>`	Write a **heap dump** (a file containing every object in the heap, for offline analysis) when the heap is exhausted
`-XX:+AlwaysPreTouch`	Touch all heap pages at start-up (slower start, no page-fault hiccups later — page faults are explained in §16.13)
`-XX:+UseCompactObjectHeaders`	Java 25+: 8-byte headers (16.4)

What the JVM chose for you: `-XX:+PrintFlagsFinal`

`java -XX:+PrintFlagsFinal -version` prints all ~800 flags with their final values. Excerpt (real, this machine):

```
   size_t G1HeapRegionSize                         = 8388608                                   {product}
{ergonomic}
    uintx MaxGCPauseMillis                         = 200                                       {product} {default}
   size_t MaxHeapSize                              = 12884901888                               {product}
{ergonomic}
     uint ParallelGCThreads                        = 11                                        {product} {default}
     intx ThreadStackSize                          = 2048                                    {pd product} {default}
     bool UseCompactObjectHeaders                  = false                          {product lp64_product}
{default}
     bool UseG1GC                                  = true                                      {product}
{ergonomic}
...
```

`{ergonomic}` means the JVM computed the value from the machine (**ergonomics**): 12 GiB max heap = 25% of 48 GiB, 8 MiB G1 regions for that heap size, G1 selected because the machine is "server class" (at least 2 CPUs and ~2 GB of memory). `{default}` is the built-in default; `{command line}` would mark your own settings. `pd` = platform-dependent (e.g. 2 MiB stacks on macOS/AArch64). `jcmd <pid> VM.flags` shows the same for a running JVM.

When not to tune

- Without logs or metrics showing a problem.

- By copying flag lists from old blog posts (many flags are obsolete — CMS, PermGen — or harmful, like tiny young generations).
- To hide a leak: a bigger heap only delays the `OutOfMemoryError` (16.11).

Common mistakes

- Setting `-Xmn`/`NewRatio` with G1: fixing the young size disables G1's pause-time adaptation.
- Calling `System.gc()` in application code (it triggers a full collection; disable with `-XX:+DisableExplicitGC` if a library does it).
- Measuring GC on a laptop and deploying to a 2-CPU container.

Self-check

1. First two things to do before tuning? → Define a measurable goal and enable GC logging.
2. What does `{ergonomic}` mean in `PrintFlagsFinal`? → The JVM computed that value from the hardware.

TL;DR

Measure → size heap from the live set → pick collector → set a goal → one change at a time.

16.8 JIT compilation, inlining and escape analysis

Chapter 1 §1.7 showed **tiered compilation** (interpreter → C1, a fast simple compiler → C2, a slower optimising one) with `-XX:+PrintCompilation`. This section covers the optimisations that make C2 fast.

Key optimisations

Optimisation	What it does	Why it matters
Inlining	Copies a **callee**'s (the called method's) body into the caller (hot methods up to `FreqInlineSize` = 325 bytecode bytes, others up to `MaxInlineSize` = 35)	Removes call overhead and lets other optimisations see across methods; small methods and getters are free
Devirtualisation	If profiling shows a **virtual call** (a call whose target depends on the object's runtime class) only ever sees one or two **receiver** classes (the class of the object the method is called on), call them directly (with a guard — a quick check that the assumption still holds)	Interfaces and **polymorphism** (one call working on many types) cost almost nothing in hot code
Escape analysis	Proves an object never leaves the method/thread	**Scalar replacement** (fields become locals — no allocation) and **lock elision**
Loop optimisations	Unrolling (repeating the loop body to do fewer jumps), range-check elimination, vectorisation (SIMD — *single instruction, multiple data*: one CPU instruction processes several values)	Array loops approach C speed
Dead-code elimination	Removes computations whose results are unused	Great for apps, a trap for benchmarks (16.9)
Speculation + deoptimisation	Optimise for what was observed; fall back to the interpreter if it changes (*deoptimise*)	Aggressive yet correct

Seeing escape analysis: `code/ch16/jit/EscapeAnalysis.java`

```
record Point(int x, int y) {
    Point plus(Point o) { return new Point(x + o.x, y + o.y); }
}
static Point escaped;

static long sumLocal(int n) {                       // Points never leave the method
    long total = 0;
    for (int i = 0; i < n; i++) {
        Point p = new Point(i, i).plus(new Point(1, 2)); // 3 Point allocations per iteration... on paper
        total += p.x() + p.y();
    }
    return total;
}
static long sumEscaping(int n) {                    // one Point per iteration is published
    ... escaped = p; ...
}
```

After warming both methods up, it counts the heap bytes allocated during 10 million iterations.

`java -cp out ch16.jit.EscapeAnalysis`, then `java -XX:-DoEscapeAnalysis -cp out ch16.jit.EscapeAnalysis`:

```
DoEscapeAnalysis = true
local      result=100000020000000  heap allocated for 10M iterations: 0 bytes
escaping   result=100000020000000  heap allocated for 10M iterations: 240,000,000 bytes
DoEscapeAnalysis = false
local      result=100000020000000  heap allocated for 10M iterations: 720,000,000 bytes
escaping   result=100000020000000  heap allocated for 10M iterations: 720,000,000 bytes
```

- With escape analysis, 30 million `Point`s in `sumLocal` cost **zero** bytes: after inlining, C2 kept `x` and `y` in **registers** (tiny storage slots inside the CPU). In `sumEscaping` only the published `Point` is allocated (10M × 24 B = 240 MB).
- Switched off, all 3 × 10M × 24 B = 720 MB are allocated.

Lesson: short-lived small objects in hot code (records, iterators, `Optional`, lambdas) are often free — write clear code first, profile before "optimising" allocations away by hand.

Interpreter vs C1 vs C2 on real work

The first part of `code/ch16/bench/NaiveBenchmark.java` computes Collatz sequence lengths (repeatedly halve even numbers and map odd n to $3n+1$ until reaching 1 — a cheap CPU workout) for 20 000 numbers per round. Round times (ms; machine-dependent):

Mode	Round 1	Rounds 6–8
Default (tiered, C2)	3.37	2.0–2.2
`-XX:TieredStopAtLevel=1` (C1 only)	4.54	4.3–4.6
`-Xint` (interpreter only)	23.21	23.1–23.6

The interpreter is ~10× slower than C2 here; C1 alone is about 2× slower. `TieredStopAtLevel=1` is sometimes used for short-lived tools where start-up matters more than peak speed.

Code cache

Compiled code lives in the code cache (16.3). If it fills, the JIT stops compiling and new hot code stays interpreted. `jcmd <pid> Compiler.codecache` shows its use; increase `-XX:ReservedCodeCacheSize` if logs say *CodeCache is full*.

In the interview

Q: What is escape analysis? C2 proves an object is never seen outside the method or thread that made it, then replaces it with its fields in registers (no allocation, no GC) and drops locks on it. It depends on inlining, so it works best on small, hot code.

- *Does Java allocate on the stack?* Not literally; scalar replacement has the same effect for non-escaping objects.
- *What is deoptimisation?* Discarding compiled code whose speculation failed (e.g. a new subclass appeared) and continuing in the interpreter.

Self-check

1. Why was the escaping case 240 MB and not 720 MB? → Only the published `Point` escaped; the two temporaries were scalar-replaced.
2. Which flag keeps only the C1 compiler? → `-XX:TieredStopAtLevel=1`.
3. What happens when the code cache is full? → Compilation stops; the app runs on with slower code.

TL;DR

C2 inlines, devirtualises, removes non-escaping allocations and dead code, and speculates with a safety net. Prove optimisations with measurement (allocated bytes, JFR), not guesses.

16.9 Benchmarking: why naive timing lies, and JMH

The problem it solves

"Is `A` faster than `B`?" sounds simple, but on the JVM the same code runs at three different speeds (interpreter, C1, C2), the JIT may delete work whose result you never use, and GC pauses can land in any measurement.

Two traps, measured

`code/ch16/bench/NaiveBenchmark.java`:

```
for (int i = 0; i < 50_000_000; i++) Math.sqrt(i);          // result discarded
...
double acc = 0;
for (int i = 0; i < 50_000_000; i++) acc += Math.sqrt(i);   // result used
sink = acc;
```

Output (machine-dependent):

```
1) Warm-up: identical rounds of 20,000 Collatz sequences
   round 1:   3.37 ms (result 1834634)
   round 2:   2.42 ms (result 1834634)
   ...
   round 8:   1.99 ms (result 1834634)
2) Dead code: 50M square roots, result ignored vs used
   round 1: ignored    1.55 ms   used   27.41 ms
   round 2: ignored    3.48 ms   used   26.12 ms
   round 3: ignored    0.00 ms   used   26.35 ms
```

1. **Warm-up**: round 1 is ~70% slower than round 8 — measure the first run and you compare the interpreter, not your algorithm.
2. **Dead-code elimination**: 50 million square roots "in 0.00 ms" is impossible — the JIT removed the loop because nobody used `Math.sqrt`'s result.

Other traps: **constant folding** (inputs known at compile time get precomputed), **loop hoisting** (moving work that doesn't change out of the loop), **GC noise** (allocation from an earlier test collected during a later one), **profile pollution** (benchmarking two implementations in one JVM makes a call site polymorphic), **CPU frequency scaling** (the CPU speeding up or slowing down for heat and power), and `System.currentTimeMillis()` (low resolution — use `System.nanoTime()` for intervals).

What JMH is

JMH (Java Microbenchmark Harness, an OpenJDK project) handles these traps: separate warm-up and measurement iterations, fresh **forked** JVMs (new JVM processes started per benchmark), a `Blackhole` that

"consumes" results so they can't be eliminated, state fields that prevent constant folding, and error bars.

```
// (JMH — not run in this book: needs the jmh-core library and its annotation processor)
@State(Scope.Thread)
@BenchmarkMode(Mode.AverageTime) @OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1) @Measurement(iterations = 5, time = 1) @Fork(2)
public class SqrtBench {
    double x = 12345.678;                          // field, not a constant: no constant folding

    @Benchmark public double sqrt() { return Math.sqrt(x); }          // returned value is consumed
    @Benchmark public void sqrtBlackhole(Blackhole bh) { bh.consume(Math.sqrt(x)); }
}
```

To run: generate a project from the **Maven archetype** (a project template for Maven, the build tool covered in Chapter 18) `org.openjdk.jmh:jmh-java-benchmark-archetype`, `mvn package`, `java -jar target/benchmarks.jar`. Results read `Score ± Error`; overlapping error ranges mean no demonstrated difference.

When not to use microbenchmarks

Microbenchmarks answer tiny questions. For "why is my service slow?", **profile the real application** under load (16.10) — the **bottleneck** (the slowest part, which limits everything else) is usually I/O, locks or an algorithm.

Self-check

1. Two reasons naive timing loops mislead? → Warm-up (interpreted vs compiled phases) and dead-code elimination; also GC noise.
2. What does `Blackhole.consume` prevent? → The JIT deleting the computation as unused.
3. Why does JMH fork a new JVM? → To isolate profiles and JIT decisions between benchmarks.

TL;DR

Never trust a single `nanoTime` loop. Use JMH for micro questions and a profiler for real ones.

16.10 Profiling and diagnostic tools

The JDK's own toolbox (all in `$JAVA_HOME/bin`)

Tool	What it does	Typical use
`jps -l`	Lists running JVMs with PIDs (process ids: the OS's number for each running program) and main classes	Find the process
`jcmd <pid> <command>`	Swiss-army knife: sends diagnostic commands to a live JVM	Flags, heap info, class histogram, thread dump, JFR start/stop, heap dump, NMT
`jstat -gcutil <pid> <ms>`	Samples GC statistics periodically	Watch heap occupancy and GC counts live
`jstack <pid>`	Thread dump (same as `jcmd <pid> Thread.print`)	Deadlocks, stuck threads
`jmap`, `jinfo`	Older heap/flag tools, mostly superseded by `jcmd`	Legacy scripts
`jfr`	Reads Flight Recorder files	Offline analysis
`jconsole` / `jhsdb`	Basic JMX GUI (graphical user interface) / inspect a hung process or **core dump** (a snapshot of a crashed process's memory)	Quick look / last resort

Free external tools: **JDK Mission Control (JMC)**, the JFR GUI (separate download); **VisualVM** (visualvm.github.io) — live monitoring, sampling profiler, heap-dump browser; **async-profiler** — low-overhead CPU/allocation **flame graphs** (charts where each bar is a method and its width shows how much time or memory it accounts for) (Linux/macOS).

`jcmd` on a live process

The leak demo of 16.11 can be kept alive with `java -Xmx256m -cp out ch16.leak.ListenerLeak leak 12`. Real session (PID machine-specific):

```
$ jps -l | grep ListenerLeak
1796 ch16.leak.ListenerLeak
$ jcmd 1796 VM.version
1796:
OpenJDK 64-Bit Server VM version 26+35-2893
JDK 26.0.0
$ jcmd 1796 GC.heap_info
1796:
garbage-first heap   total reserved 262144K, committed 191488K, used 105337K [0x00000007f0000000,
0x0000000800000000)
 region size 1M, 2 eden (2M), 0 survivor (0M), 104 old (104M), 0 humongous (0M), 81 free (81M)
$ jcmd 1796 Thread.print | head
Full thread dump OpenJDK 64-Bit Server VM (26+35-2893 mixed mode, sharing)
...
"main" #3 [6403] prio=5 os_prio=31 cpu=57.63ms elapsed=5.99s tid=0x0000000148019400 nid=6403 waiting on condition
[0x000000016d7b6000]
   java.lang.Thread.State: TIMED_WAITING (sleeping)
    ...
    at ch16.leak.ListenerLeak.main(ListenerLeak.java:68)
```

`jcmd <pid> help` lists every command (on JDK 26: `GC.class_histogram`, `GC.heap_dump`, `JFR.start`, `JFR.dump`, `JFR.view`, `Thread.dump_to_file` — which also lists virtual threads —, `VM.flags`, `VM.native_memory`, `VM.metaspace`, `Compiler.codecache` and more).

`jstat` — GC counters every 500 ms

While `AllocWorkload` ran with 60 million allocations: `jstat -gcutil <pid> 500 6`

```
  S0     S1     E      O      M     CCS    YGC     YGCT     FGC    FGCT     CGC    CGCT       GCT
     - 100.00  68.66  30.61  52.38  13.36     56     0.171     0     0.000    20     0.004     0.174
     - 100.00   7.26  36.10  52.38  13.36     69     0.220     0     0.000    26     0.005     0.225
     ...
     - 100.00  47.49  30.37  52.38  13.36    119     0.406     0     0.000    50     0.010     0.415
```

Columns are percentages of capacity for **S**urvivor 0/1, **E**den, **O**ld, **M**etaspace, **C**ompressed **C**lass **S**pace, then counts and total seconds for **Y**oung GC, **F**ull GC, **C**oncurrent GC. Here: ~25 young GCs per second, no full GCs, old gen oscillating 30–44% as mixed collections clean it — healthy. A steadily rising `O` that never drops after GCs is the signature of a leak.

Java Flight Recorder (JFR)

What it is: an event recorder built into the JVM (open-sourced in JDK 11, JEP 328) — think of an aircraft's black box. It records GC, JIT, locks, I/O, allocation samples and CPU samples with typically ~1% overhead (default settings), so it is safe to leave on in production.

How to use it

1. Start at launch: `java -XX:StartFlightRecording=filename=rec.jfr,settings=profile …` (or attach later: `jcmd <pid> JFR.start duration=60s filename=rec.jfr`).
2. Dump or stop: the recording is written when the program exits or with `jcmd <pid> JFR.dump name=1 filename=rec.jfr`.
3. Analyse: `jfr summary`, `jfr print --events <type>`, `jfr view <view>` (JDK 21+), or open in JMC.

Real session on `AllocWorkload 20000000` (G1, `-Xmx1g`):

```
$ jfr summary rec.jfr
 ...
 Event Type                              Count  Size (bytes)
============================================================
 jdk.GCPhaseParallel                     37056        978649
 ...
 jdk.ObjectAllocationSample                570          8201
 jdk.ExecutionSample                       117          1166
 jdk.GarbageCollection                      80          1778
 ...
$ jfr print --events jdk.GarbageCollection rec.jfr | head -11
jdk.GarbageCollection {
  ...
  name = "G1New"
  cause = "G1 Evacuation Pause"
  sumOfPauses = 1.72 ms
  ...
}
```

```
$ jfr view gc-pauses rec.jfr          (G1)          $ jfr view gc-pauses recz.jfr    (same run, ZGC)
Total Pause Time: 270 ms                            Total Pause Time: 0.955 ms
Number of Pauses: 94                                Number of Pauses: 173
Median Pause Time: 3.68 ms                          Median Pause Time: 0.00521 ms
P99 Pause Time: 8.40 ms                             P99 Pause Time: 0.0120 ms
Maximum Pause Time: 8.40 ms                         Maximum Pause Time: 0.0121 ms
```

(The two outputs are shown side by side and trimmed to the key lines.)

```
$ jfr view hot-methods rec.jfr
Method                                                  Samples Percent
ch16.gc.AllocWorkload.main(String[])                        116  99.15%
...
$ jfr view allocation-by-class rec.jfr
Object Type                                          Allocation Pressure
byte[]                                                          100.00%
...
```

`jfr view` with no name lists all views (`allocation-by-site`, `contention-by-site`, `gc`, `pinned-threads`, …). Recordings can contain environment variables and system properties — review before sharing (`jfr scrub` removes events).

Which tool for which symptom?

Symptom	First tool	Then
High CPU	JFR `hot-methods` / async-profiler flame graph	Check GC CPU in JFR `gc` view
Latency spikes	GC log pauses; JFR `gc-pauses`, `safepoints`	Lock contention views, thread dumps
App hangs	`jcmd <pid> Thread.print` two or three times	Look for `BLOCKED` threads and "Found one Java-level deadlock"
Memory keeps growing	`jstat -gcutil` (old gen after GC); `GC.class_histogram` twice	JFR `OldObjectSample` with `path-to-gc-roots=true`; heap dump in VisualVM/Eclipse MAT (Memory Analyzer Tool)
Process RSS (resident set size: RAM the process really uses, §16.13) > heap	NMT `VM.native_memory`	Thread count, direct buffers, metaspace
Slow start-up	`-Xlog:class+load`, `-Xlog:startuptime`	CDS / AOT cache

In the interview

Q: Latency spikes every few minutes. How do you investigate? Correlate spikes with the GC log (long pauses, full GCs, evacuation failures), then record JFR across a spike and check GC pauses, safepoints, lock **contention** (threads queuing for the same lock) and allocation hot spots. If GC is the cause, fix allocation rate or headroom before touching flags.

- *Thread dump vs heap dump?* Thread dump: every thread's stack and locks — small and fast. Heap dump: every object — large, pauses the JVM, may contain secrets.

Self-check

1. Which `jcmd` command shows instances and bytes per class without a heap dump? → `GC.class_histogram`.
2. What does a rising `O` column in `jstat -gcutil` that never falls indicate? → Old-generation growth — a possible leak.
3. How do you start JFR on an already-running JVM? → `jcmd <pid> JFR.start …`.

TL;DR

`jps` → `jcmd` (everything) → `jstat` (live GC counters) → JFR for low-overhead recordings (`jfr summary/print/view`, JMC). Heap dumps are the heavy tool of last resort.

16.11 Memory leaks

What it is

A **memory leak** in Java is memory that stays **reachable** although the program will never use it again. The GC can't free it — something still points to it. Analogy: a coat-check where the attendant never hands coats back because the tickets are kept in a drawer; the room fills up even though the guests left long ago.

Why it matters

Leaks grow slowly: fine for days, then ever more frequent GCs, rising latency, and finally `OutOfMemoryError: Java heap space` — often at peak traffic.

Common leak shapes

Shape	Example	Fix
Unbounded static collection	`static Map<String, Result> cache` that only grows	Bounded cache with **eviction** (automatically removing old or least-used entries; Caffeine is a popular caching library, or `LinkedHashMap.removeEldestEntry` — Chapter 10)
Listeners never removed	Per-request subscription to an app-wide **event bus** (a central object that forwards published events to every subscribed listener)	Unsubscribe (try-with-resources handle)
`ThreadLocal` on pooled threads	Setting per-request data on a thread pool thread and never calling `remove()`	`try { … } finally { tl.remove(); }`; scoped values (Java 25)
Wrong `equals`/`hashCode` for map keys	Mutable key changed after insertion; entry can never be found or removed	Immutable keys, e.g. records (Chapter 6 §6.13)
Classloader leaks	A shared class keeps a reference into a redeployed web app	Clean up drivers (e.g. database drivers), threads, thread-locals on undeploy
Unclosed resources	Streams, connections holding native buffers	try-with-resources (Chapter 8)
Inner-class capture	A stored inner-class instance or lambda keeps its outer object alive	Static nested classes; capture less

A measured leak: `code/ch16/leak/ListenerLeak.java`

```
static final class EventBus {
    static final List<Consumer<String>> LISTENERS = new CopyOnWriteArrayList<>();
    static AutoCloseable subscribe(Consumer<String> l) {
        LISTENERS.add(l);
        return () -> LISTENERS.remove(l);                // the handle that lets callers clean up
    }
}
static final class RequestSession {
    final byte[] buffer = new byte[10 * 1024];           // per-request working memory
    void onEvent(String e) { buffer[0]++; }
}
static void handleRequestLeaky() {
    RequestSession s = new RequestSession();
    EventBus.subscribe(s::onEvent);                      // BUG: never unsubscribed
}
static void handleRequestFixed() throws Exception {
    RequestSession s = new RequestSession();
    try (AutoCloseable sub = EventBus.subscribe(s::onEvent)) {   // unsubscribed when the request ends
        // ... handle the request ...
    }
}
static long usedAfterGcKiB() {
    System.gc();                                         // only for measuring in a demo
    return ManagementFactory.getMemoryMXBean().getHeapMemoryUsage().getUsed() / 1024;
}
```

The method reference `s::onEvent` captures `s`, so the static list → lambda → session → 10 KiB buffer chain keeps every request's memory alive.

Output (KiB figures vary slightly between runs — machine-dependent):

```
mode = leak
  after  2,000 requests: listeners= 2,000  heap growth since start= 20,211 KiB
  after  4,000 requests: listeners= 4,000  heap growth since start= 40,537 KiB
  ...
  after 10,000 requests: listeners=10,000  heap growth since start=101,210 KiB
mode = fixed
  after  2,000 requests: listeners=     0  heap growth since start=      0 KiB
  ...
  after 10,000 requests: listeners=     0  heap growth since start=      0 KiB
```

The key measurement is **heap used *after* a full GC**: linear growth (~10 KiB per request) is a leak; flat is healthy. In production, read the same signal from the GC log's "after" numbers or `jstat`'s `O` column — no `System.gc()` needed.

Finding the culprit without a heap dump

Step 1 — class histogram (`jcmd <pid> GC.class_histogram`, real output, trimmed):

```
 num     #instances         #bytes  class name (module)
-------------------------------------------------------
   1:         20232      103004240  [B (java.base@26)
   ...
   6:         10000         160000  ch16.leak.ListenerLeak$$Lambda/0x000001fc01041800
   7:         10000         160000  ch16.leak.ListenerLeak$RequestSession
```

`[B` is `byte[]`. Exactly 10 000 sessions and 10 000 lambdas after 10 000 requests: suspicious. Take two histograms minutes apart; classes whose counts only rise are the suspects. (By default the histogram runs a full GC first, so it counts only live objects.)

Step 2 — JFR old-object sampling with paths to GC roots: `java -XX:StartFlightRecording=filename=leak.jfr,settings=profile,path-to-gc-roots=true … leak`, then `jfr print --events jdk.OldObjectSample --stack-depth 3 leak.jfr` (one of 178 samples):

```
jdk.OldObjectSample {
  ...
  objectSize = 10.0 kB
  object =  [
    byte[10240]
    buffer : ch16.leak.ListenerLeak$RequestSession
    arg$1 : ch16.leak.ListenerLeak$$Lambda.0x0000180001045400
    [9822] : java.lang.Object[10000]
    array : java.util.concurrent.CopyOnWriteArrayList
    LISTENERS : java.lang.Class Class Name: ch16.leak.ListenerLeak$EventBus
    ...
  ]
  ...
  stackTrace = [
    ch16.leak.ListenerLeak$RequestSession.<init>() line: 27
    ch16.leak.ListenerLeak.handleRequestLeaky() line: 32
    ...
  ]
}
```

Read the `object` chain bottom-up and you have the whole story: the `LISTENERS` static field of `EventBus` → its `CopyOnWriteArrayList` → element 9822 → the lambda (its captured `arg$1`) → the session's `buffer`. The stack trace shows **where** the leaked object was allocated. All of this without stopping the JVM to write a multi-gigabyte heap dump.

Step 3 (if needed) — heap dump: `jcmd <pid> GC.heap_dump /tmp/app.hprof` or `-XX:+HeapDumpOnOutOfMemoryError`, analysed in Eclipse MAT or VisualVM ("dominator tree", "path to GC roots"). Heap dumps pause the app and contain all data in memory (passwords, personal data) — handle them as secrets.

Common mistakes

- Treating a large heap as a leak. A cache that is big but bounded is fine; a leak is **growth without bound** in heap-after-GC.

- "Fixing" with `WeakReference` everywhere (a weak reference doesn't keep its object alive; the GC may clear it). Weak references are right for canonicalising maps (maps that ensure one shared instance per value) and listener registries you don't own; for caches you usually want size- or time-based eviction.

In the interview

Q: Can Java leak memory if it has a GC? Yes: the GC frees only *unreachable* objects, so anything still referenced but no longer needed — static maps, listeners never removed, thread-locals on pooled threads — leaks. Confirm with heap-after-GC growth, find the type with class histograms or JFR old-object samples, and follow the path to the GC root.

- *Leak vs high usage?* A leak keeps growing after GC; high usage plateaus.

Self-check

1. Why does `s::onEvent` keep the session alive? → A bound method reference captures its receiver `s`.
2. Which number proves a leak: heap before GC or after GC? → After GC.

TL;DR

Leak = still reachable, never used. Measure heap after GC → class histogram → JFR `OldObjectSample` path to root → fix the reference (unsubscribe, bound the cache, `remove()` the thread-local).

16.12 Modules (JPMS) basics

What it is

The **Java Platform Module System** (JPMS, Java 9, JEP 261) adds a level above packages: a **module** is a named group of packages whose `module-info.java` declares **what it needs** (`requires`) and **what it offers** (`exports`). The JDK itself is about 70 modules (`java.base`, `java.sql`, …). Analogy: a building with a front door for visitors (exported packages) and staff-only rooms (everything else).

The problem it solves

The flat **class path** made every `public` class usable by everyone (hence widespread use of internals like `sun.misc.Unsafe`, a JDK-private class for raw memory access), found missing or duplicate JARs only at run time ("JAR hell"), and couldn't produce a smaller JDK. Modules give **reliable configuration**, **strong encapsulation** (non-exported packages are inaccessible, even to reflection) and a **modular JDK** that `jlink` can trim.

The directives

Directive	Meaning
`requires M;`	This module reads module `M` (needs it at compile and run time)
`requires transitive M;`	…and anyone who requires me also reads `M` (for types that appear in my API)
`requires static M;`	Needed at compile time only; optional at run time
`exports p;`	Public types in package `p` are accessible to other modules
`exports p to M1, M2;`	Qualified export — only to the named modules
`opens p;` / `opens p to M;`	Allow deep reflection (private members) on `p` at run time — what frameworks like Jackson (JSON) or Hibernate (database mapping) need
`open module m { … }`	Opens every package of the module for reflection
`uses S;` / `provides S with Impl;`	Service consumer / provider for `ServiceLoader` (the JDK's built-in plug-in lookup mechanism)

`java.base` is required implicitly by every module.

The example: `jpms/`

```
jpms/src/
├── com.example.greet/                (library module)
│   ├── module-info.java
│   └── com/example/greet/Greeter.java, internal/Formatter.java
└── com.example.app/                  (application module)
    ├── module-info.java
    └── com/example/app/Main.java
```

```
// src/com.example.greet/module-info.java
module com.example.greet {
    exports com.example.greet;              // public types here are usable by other modules
    // com.example.greet.internal is NOT exported: public classes there are still inaccessible outside
}

// src/com.example.app/module-info.java
module com.example.app {
    requires com.example.greet;             // checked at compile AND launch time
    requires java.logging;                  // a JDK module; java.base is always required implicitly
}
```

```
// src/com.example.app/com/example/app/Main.java
public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println(Greeter.greet("modules"));
        System.out.println("my module      : " + Main.class.getModule().getName());
        System.out.println("Greeter module : " + Greeter.class.getModule().getName());
        System.out.println("Logger module  : " + Logger.class.getModule().getName());
        System.out.println("app reads greet: " + Main.class.getModule().canRead(Greeter.class.getModule()));
        Class<?> hidden = Class.forName("com.example.greet.internal.Formatter");
        try {
            hidden.getConstructor().newInstance();
        } catch (IllegalAccessException e) {
            System.out.println("reflection     : " + e.getMessage());
        }
    }
}
```

`Formatter` is a `public` class with a `public` constructor — yet outside its module it is off-limits.

Run with `bash jpms/run.sh` (the book's generic runner skips this folder: it compiles `code/` as one class-path unit). Real output:

```
$ javac -d mods --module-source-path src $(find src -name "*.java")
$ java --module-path mods -m com.example.app/com.example.app.Main
*** Hello, modules ***
my module      : com.example.app
Greeter module : com.example.greet
Logger module  : java.logging
app reads greet: true
reflection     : class com.example.app.Main (in module com.example.app) cannot access class
com.example.greet.internal.Formatter (in module com.example.greet) because module com.example.greet does not
export com.example.greet.internal to module com.example.app
$ jdeps --module-path mods -s mods/com.example.app
com.example.app -> com.example.greet
com.example.app -> java.base
com.example.app -> java.logging
$ jlink --module-path mods --add-modules com.example.app --output runtime --strip-debug --no-header-files --no-
man-pages
$ runtime/bin/java --list-modules
com.example.app
com.example.greet
java.base@26
java.logging@26
$ du -sh runtime
44M  runtime/
```

And if `Main` imports the internal package, compilation fails (real `javac` output):

```
Main.java:4: error: package com.example.greet.internal is not visible
import com.example.greet.internal.Formatter;
                        ^
  (package com.example.greet.internal is declared in module com.example.greet, which does not export it)
```

`--module-source-path` compiles several modules at once (one sub-folder per module); `-m module/mainclass` launches. `jdeps` shows dependencies (also useful on plain JARs to find use of JDK internals). `jlink` built a **44 MB** self-contained runtime with just four modules, compared with the full 377 MB JDK on this machine — ideal for **container images** (the packaged file system a container starts from).

Class path vs module path

Where the JAR is	It becomes	Can read	Exports
Class path	Part of the **unnamed module**	Every module	Everything (but named modules can't `require` it)
Module path, has `module-info.class`	**Named (explicit) module**	Only what it `requires`	Only what it `exports`
Module path, no `module-info`	**Automatic module** (name from `Automatic-Module-Name` in the **manifest** — the `META-INF/MANIFEST.MF` metadata file inside a JAR — else the file name)	Every module	Everything

This is why modules were adoptable gradually: most applications still run on the class path, while the **JDK's** internals are strongly encapsulated for everyone since Java 16–17 (JEPs 396 and 403). Code that reflects into JDK internals needs `--add-opens java.base/java.lang=ALL-UNNAMED` (a temporary workaround, not a fix).

When not to use it

Most applications stay on the class path — frameworks work fine there, and modularising adds `opens` friction with reflective libraries. Modules pay off for **libraries**, **large codebases** wanting enforced boundaries, and `jlink` **runtimes**.

Common mistakes

- Module name ≠ package name confusion: `requires` takes a *module* name, `exports` a *package* name.
- Split packages: the same package in two modules is an error on the module path.
- Forgetting `opens` for packages that frameworks access reflectively → `InaccessibleObjectException` at run time.
- Cyclic `requires` between modules — not allowed.

In the interview

Q: What does JPMS give you over the class path? Reliable configuration (dependencies checked at start-up), strong encapsulation (only exported packages are accessible; deep reflection needs `opens`), and a modular JDK that `jlink` can shrink.

- *`exports` vs `opens`?* Compile- and run-time access to public types vs run-time reflective access to all members, private included.

Self-check

1. Where does a class on the class path live? → The unnamed module.
2. A `public` class in a non-exported package: can another module call it? → No — neither directly nor through reflection.
3. Which tool builds a trimmed runtime image? → `jlink`.

TL;DR

`module-info.java`: `requires` what you need, `exports` your API, `opens` for reflection. Class path = unnamed module. `jdeps` to analyse, `jlink` to ship a small runtime. Exam-level detail: OCP Chapter 12.

16.13 Operating-system fundamentals for engineers

What it is

The JVM is a program like any other: the **operating system (OS)** gives it CPU time, memory and access to files and the network. Many "JVM" problems — a service stuck at 100% CPU, `Too many open files`, a container killed with no Java stack trace, a first request that is slow because the disk was cold — are really OS behaviour. This section covers the OS ideas every **backend** engineer (one who builds the server side of applications) is expected to know, each tied to what you see from Java. The Python handbook points here too (Python ch.17); the ideas are language-neutral.

Analogy: the OS **kernel** is a building's management. Tenants (**processes** — running programs) each get a private flat (address space); the lift schedule (**scheduler**) decides who moves when; tenants can't touch the wiring themselves and must file requests at the front desk (**system calls**); each tenant's key ring has numbered keys to doors they may open (**file descriptors**).

Processes vs threads

	Process	Thread
What	a running program with its **own virtual address space** (its private view of memory, explained below), file-descriptor table and resources	a path of execution **inside** a process: its own stack, registers and program counter
Shares memory with	nobody (only through explicit IPC)	every other thread in the same process — hence data races (14.5)
Create / switch cost	higher: new address space; switching may **flush** (empty) address-translation caches	lower, but a platform thread still has a kernel structure and ~1 MiB of reserved stack by default
Crash isolation	one process crashing leaves others alive	one thread corrupting memory can take down the whole process
Java	`ProcessBuilder`, `ProcessHandle` (each JVM is one process)	each **platform thread** is one OS thread (1:1). **Virtual threads** (15.4) are scheduled by the JVM onto a few OS carrier threads (M:N)

On Linux, `fork()` creates a child process as a copy of the parent (using **copy-on-write**: pages are shared until one side writes), and `exec()` replaces its program — that is what `ProcessBuilder.start()` does. Processes talk through **IPC** (inter-process communication): pipes (what `Process.getInputStream()` reads), sockets (including Unix domain sockets, supported by `SocketChannel` since Java 16), shared memory (a memory-mapped file), **signals** (small numbered notifications the OS delivers to a process; `kill -TERM` triggers the JVM's **shutdown hooks** — code registered to run when the JVM exits) and message queues.

CPU scheduling

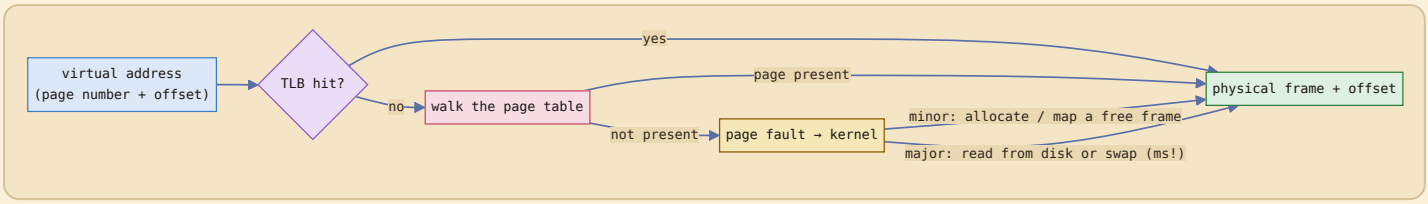
A CPU core runs one thread at a time. The kernel's **scheduler** gives each runnable thread a short **time slice**, then **preempts** it (a timer **interrupt** — a hardware signal that makes the CPU stop and run kernel code — forces a switch) and saves its registers — a **context switch**, costing on the order of a few microseconds plus cache warm-up. Linux's scheduler (CFS, the Completely Fair Scheduler, replaced by EEVDF in kernel 6.6) aims at fairness weighted by priority (`nice`, the Unix per-process priority setting). Threads waiting for I/O or a lock are **blocked** and use no CPU; **load average** counts threads running or waiting to run (on Linux it also counts uninterruptible disk waits).

Consequences for Java: a pool of 500 CPU-bound platform threads on 8 cores gets no faster than ~8 threads, just more switching; blocked threads are cheap in CPU but not in memory; and container CPU **limits** (cgroup quotas) can pause (**throttle**) a whole JVM for the rest of a 100 ms period once its quota is spent — the JVM reads the limit to size GC and fork-join threads (`-XX:ActiveProcessorCount` overrides it).

Diagnosing 100% CPU: `top -H -p <pid>` (Linux) lists threads by CPU; convert the hot thread id to hex; `jcmd <pid> Thread.print` shows each Java thread with its OS id as `nid=0x…`; match them to find the stack burning CPU. Take three dumps a few seconds apart — a frame that stays on top is the culprit. JFR's `ExecutionSample` events (16.10) do the same statistically.

Virtual memory and paging

Every process sees its own **virtual address space**; it never uses physical addresses. Memory is managed in fixed-size **pages** (4 KiB on most x86-64 Linux, 16 KiB on Apple Silicon — the demo prints it). A per-process **page table** maps virtual pages to physical **frames** (page-sized slots of real RAM), and the CPU's **TLB** (*translation lookaside buffer*, a small hardware cache) caches recent translations.



- Memory is **allocated lazily**: reserving address space (`-Xmx`, `mmap`) costs nothing until a page is touched; the first touch triggers a **page fault** and the kernel supplies a frame. That is why a process's **virtual size (VSZ)** is far larger than its **resident set size (RSS)**, the RAM actually used. `-XX:+AlwaysPreTouch` touches the whole heap at start-up to avoid faults later.
- When RAM runs short the kernel drops clean file pages, writes **dirty** ones (modified since they were read) back, and **swaps anonymous pages** (memory not backed by any file, such as the heap) out to a disk area called *swap*; a **major fault** then costs milliseconds instead of nanoseconds. A swapping JVM has terrible GC pauses, so servers usually run with little or no swap.
- Out of memory entirely, Linux's **OOM killer** (*out of memory*) picks a victim and sends `SIGKILL` (the signal that ends a process immediately and can't be caught) — the JVM dies with no `OutOfMemoryError` and no stack trace. In containers the cgroup memory limit applies to the *whole* process (heap + metaspace + thread stacks + direct buffers, 16.3), so size `-Xmx` (or `-XX:MaxRAMPercentage`) well below it.
- **Segmentation** (variable-size segments) is the older scheme; modern x86-64 and ARM OSes use flat paging, and "segmentation fault" today just means touching an unmapped or protected page.

Memory hierarchy — rough latencies (orders of magnitude; vary by hardware):

Level	Latency	Size
CPU register / L1 cache	~1 ns	tens of KiB per core
L2 / L3 cache	~4–40 ns	MiB
Main memory (DRAM)	~100 ns	GiB
NVMe SSD (fast flash storage) read	~20–100 µs	TiB
Spinning disk seek	~5–10 ms	TiB
Network round trip, same data centre / across a continent	~0.5 ms / ~50–150 ms	—

System calls

Your code runs in **user mode** and cannot touch hardware. To open a file, send bytes, create a thread or allocate pages it makes a **system call** (`open`, `read`, `write`, `mmap` — map a file or memory into the address space — `clone` — create a thread/process — `futex` — the kernel's wait/wake primitive behind locks — `epoll_wait` …): the CPU switches to **kernel mode**, the kernel does the work and checks

permissions, and control returns. A syscall costs roughly 100 ns to a few µs — cheap once, expensive per byte. That is why buffering matters (13.6): 1,000 one-byte `write` calls are 1,000 syscalls; one buffered 1,000-byte write is one. On Linux, `strace -f -c java …` (a tool that traces system calls) counts a program's syscalls (not run here: macOS needs `dtruss` with System Integrity Protection relaxed).

File descriptors

A **file descriptor (fd)** is a small integer index into a process's table of open kernel objects — files, pipes, sockets, the epoll instance itself ("everything is a file"). 0, 1 and 2 are stdin, stdout and stderr (standard input, output and error — §13.8). Every `FileInputStream`, `FileChannel`, socket and database connection holds one until closed. The table has a per-process limit (`ulimit -n`, often 1,024 by default for shells; servers raise it to tens of thousands); leaked streams or a connection storm (a sudden flood of new connections) end in `IOException: Too many open files`. Use try-with-resources (Chapter 8) and watch the count (`ls /proc/<pid>/fd | wc -l` on Linux, or the JMX `OpenFileDescriptorCount`). File **permissions** (`rwx` for owner, group, others) are checked when the fd is opened, not on each read.

I/O models: blocking, non-blocking, multiplexing, async

Model	How a thread waits	Java	Scales to
Blocking	`read()` sleeps in the kernel until data arrives	`java.io` streams, `Socket`	one thread per connection — fine for hundreds, costly for 100k **platform** threads
Non-blocking	`read()` returns 0 immediately if nothing is there; the app must retry	`configureBlocking(false)`	useless alone (busy-waiting)
Multiplexing	one call waits for **readiness** on many fds: `select`/`poll` (O(n) per call — cost grows with the number of fds), **`epoll`** (Linux) / **`kqueue`** (BSD, macOS) (O(ready))	NIO `Selector` — the basis of Netty, Tomcat NIO connector, Redis's (an in-memory key-value database) **event loop** — one thread looping over "what's ready next?" (SQL book §16.11)	one thread, tens of thousands of connections
Asynchronous (completion)	submit the operation; the kernel tells you when it is *done*	`AsynchronousFileChannel`; Linux `io_uring` (the kernel's modern batched async I/O interface) via libraries	high-throughput storage and networking
Virtual threads	write blocking code; the JVM parks the virtual thread and uses the OS readiness APIs underneath	`Thread.ofVirtual()`, `Executors.newVirtualThreadPerTaskExecutor()`	blocking style at event-loop scale

Page cache and zero-copy

The kernel keeps recently read and written file data in otherwise unused RAM, the **page cache**. A `read()` of cached data is a memory copy; a miss goes to disk. A `write()` normally only dirties cached pages and returns — the kernel flushes them later — so data survives a crash only after **`fsync`** (`FileChannel.force(true)`), which is what databases do at **commit** (when a transaction is made permanent) (SQL book's write-ahead log). This is also why "free" memory on a busy Linux server looks near zero (it is being used as cache and is released on demand) and why the second run of a file-heavy job is much faster. **Zero-copy** (`sendfile`, used by `FileChannel.transferTo`) sends file pages from the page cache straight to a socket without copying them through the JVM — one reason Kafka (a widely used distributed message log) is fast.

Seeing it: `code/ch16/os/OsBasics.java`

```
process  pid=94997 parent=94956 cpus=14
thread   wrote sharedCounter -> main thread reads 42
child    says: child pid=95011 parent=94997 | exit code 3 | it could not see sharedCounter (separate address
space)
fds      before=6  with 100 files open=108  after close=8
vm       page size=16384 B, mapped 1024 MiB, touched 1000 pages -> only those 15.6 MiB need physical RAM
nio      selector implementation: KQueueSelectorProvider
nio      one selector thread echoed to 200 of 200 clients
```

Reading it: a thread sees the variable another thread wrote; a child process (created by fork + exec) has its own pid, reports ours as its parent, and could not see our memory. Opening 100 channels added ~100 descriptors (the small leftovers are descriptors the JDK opened for its own use meanwhile, such as the pipes to the `getconf` child). Mapping 1 GiB was instant because pages are only backed when touched. On macOS the NIO selector is kqueue-based (it would print `EPollSelectorProvider` on Linux), and a single thread served 200 connections. Pids and counts change on every run.

How the demo's program itself ran — compiled vs interpreted vs JIT: C and Go are compiled ahead of time to machine code; CPython (the standard Python interpreter) interprets bytecode; the JVM starts interpreting bytecode and **JIT-compiles** hot methods to machine code (16.8). **Endianness** (byte order of multi-byte numbers) also comes from the CPU: x86 and ARM are little-endian, network protocols are big-endian, and Java's `ByteBuffer` defaults to big-endian (`ByteOrder.nativeOrder()` tells you the CPU's).

Common mistakes

- **Thread-per-request with thousands of platform threads** → memory for stacks and scheduler overhead; use a bounded pool, virtual threads, or an event loop.
- **Setting `-Xmx` equal to the container memory limit** → the OOM killer ends the JVM because non-heap memory has nowhere to go; leave 25% or more headroom.
- **Unbuffered small writes** → one syscall each; wrap streams in buffers.
- **Not closing streams, sockets or connections** → fd exhaustion under load.
- **Assuming a successful `write()` is on disk** → call `force()`/`fsync` when durability matters.
- **Treating `top`'s low "free" memory as a leak** → most of it is page cache; look at RSS and "available".

In the interview

30-second answer — process vs thread: "A process is a running program with its own virtual address space and file-descriptor table; a thread is an execution path inside a process that shares its memory. Processes are isolated and heavier to create and switch; threads are cheaper but need synchronisation. In Java each platform thread is an OS thread, while virtual threads are multiplexed by the JVM onto a few carrier threads."

- *What is a page fault?* → The CPU touched a virtual page with no valid mapping; the kernel maps a frame (minor) or loads it from disk (major), or kills the access (segfault).
- *epoll vs select?* → `select` re-scans every fd on each call (O(n), limited set size); `epoll` registers fds once and returns only the ready ones (O(ready)).
- *Why did my container JVM vanish without an `OutOfMemoryError`?* → The kernel's OOM killer sent `SIGKILL` because total process memory exceeded the cgroup limit.
- *What happens on a system call?* → Switch from user to kernel mode via a trap, the kernel validates and performs the operation, then returns — cheap but not free, so batch I/O.

Self-check

1. Two threads vs two processes: which share the heap? → Threads in the same process.
2. Why is VSZ much larger than RSS for a JVM? → Reserved address space (heap max, thread stacks, mappings) isn't backed by RAM until touched.
3. What error do you get when a process runs out of file descriptors? → `Too many open files` (`EMFILE`).

4. Which I/O model does NIO's `Selector` use? → Readiness multiplexing: epoll on Linux, kqueue on macOS.
5. When is written data safe from a power cut? → After `fsync` (`FileChannel.force`), not after `write`.

TL;DR

Processes own address spaces and fd tables; threads share them. The scheduler time-slices runnable threads (context switches aren't free; cgroup CPU limits throttle). Virtual memory = pages + page tables + TLB, allocated lazily on page faults; out of RAM → swap or OOM killer. Syscalls cross into the kernel (batch them); fds are per-process handles with a limit. I/O: blocking, non-blocking, multiplexed (epoll/kqueue — NIO, Netty, Redis), async (io_uring), or virtual threads. The page cache makes repeat reads fast, writes lazy (`fsync` for durability) and enables zero-copy.

Sources: Arpaci-Dusseau & Arpaci-Dusseau, *Operating Systems: Three Easy Pieces* (processes, scheduling, paging, TLBs, I/O); Linux man pages `fork(2)`, `mmap(2)`, `epoll(7)`, `select(2)`, `fsync(2)`, `proc(5)`, `getrlimit(2)`; Linux kernel docs (EEVDF scheduler, cgroup v2 memory/CPU); JDK 25 API docs (`ProcessHandle`, `Selector`, `FileChannel`, `ByteOrder`); JEP 444 (virtual threads).

Chapter TL;DR

- **Class loading**: lazy load → link → initialise; parent-first delegation; CDS/AOT caches speed start-up.
- **Memory areas**: heap, stacks, metaspace, code cache, native memory — each with its own `OutOfMemoryError`.
- **Objects**: 12-byte header (8 with compact headers), 4-byte references, 8-byte alignment: `new Object()` = 16 B, `Point(int,int)` = 24 B.
- **Collectors**: Serial, Parallel, **G1 (default)**, ZGC/Shenandoah (sub-ms pauses). Here: Serial 130 ms, G1 ~9 ms, ZGC ~0.01 ms pauses.
- **Tuning**: measure, size from the live set, set a goal, one change at a time.
- **JIT**: inlining unlocks escape analysis (30 M `Point`s, 0 bytes). **Benchmarks**: use JMH.
- **Tools**: `jcmd`, `jstat`, **JFR**; heap dumps last. **Leaks**: heap-after-GC growth → histogram → `OldObjectSample` path.
- **JPMS**: `requires`/`exports`/`opens`; `jdeps`; `jlink` (44 MB runtime vs 377 MB JDK).
- **OS fundamentals**: processes own address spaces, threads share them; time-sliced scheduling; virtual memory is lazy (page faults, RSS ≪ VSZ, OOM killer); syscalls and fds are per-process; NIO `Selector` = epoll/kqueue; page cache + `fsync` + zero-copy.
- Next: Chapter 17 — Modern Java 17 to 25.

Chapter 17 — Modern Java: 17 to 25

Java changes every six months, and a lot changed between the two **long-term-support** (LTS) releases — versions that vendors keep patching for years, so most companies standardise on them — most teams run today: **Java 17** (September 2021) and **Java 25** (September 2025). This chapter is a **concise tour**: one table of everything that became final in each release, one tiny runnable example per final feature that the rest of the book doesn't already teach, and an exact list of what is still *preview* or *incubating* — so you know what you can use in **production** code (the real system serving real users) today and what you can't.

Most of the big language features already have full sections elsewhere in the book; here they get a two-line reminder and a link. Read this chapter as the map, the linked sections as the territory.

What you'll learn

#	Topic
17.1	How to read feature status (final, preview, incubator, experimental, withdrawn)
17.2	The version table: every release 17 → 25 at a glance
17.3	The Java 17 baseline (features finalised in 14–17)
17.4	Java 18: UTF-8 by default, the simple web server, code snippets in Javadoc
17.5	Java 19 and 20: the preview years
17.6	Java 21 LTS: virtual threads, pattern matching for `switch`, record patterns, sequenced collections
17.7	Java 22: unnamed variables, the Foreign Function & Memory API, multi-file launch
17.8	Java 23: Markdown documentation comments; string templates withdrawn
17.9	Java 24: stream gatherers, the Class-File API, virtual threads without pinning
17.10	Java 25 LTS: compact source files, module imports, flexible constructors, scoped values, KDF API
17.11	Still preview or incubating in Java 25 — exact list, and what JDK 26/27 did next
17.12	Removals and deprecations that matter when upgrading 17 → 21 → 25

How the code was run. Programs are in `code/` (package `ch17.v17` … `ch17.v25`, one sub-package per release), compiled with `javac --release 25` (the Java compiler, targeting Java 25) **without** `--enable-preview`, and run on OpenJDK 26 (the open-source reference build of the Java Development Kit). Only **final** features appear in runnable code. Every JEP number and status in this chapter was checked against the release pages on openjdk.org (`openjdk.org/projects/jdk/<N>/`) and the individual JEP pages.

17.1 Reading a feature's status

Each change to the JDK is described in a **JEP** (JDK Enhancement Proposal) — see Chapter 1 §1.12 for the release model. A JEP title tells you the maturity:

Title contains	Status	Can you use it in production?
nothing (e.g. *JEP 444: Virtual Threads*)	**Final / permanent**	Yes
(Preview), *(Second Preview)* …	**Preview**: complete, but may still change	Only with `--enable-preview` on both `javac` and `java`, for experiments
(Incubator)	**Incubating API** in a `jdk.incubator.*` module	Only with `--add-modules jdk.incubator.…`; API will change
(Experimental)	**Experimental VM feature** (a JVM — Java Virtual Machine — feature still being tried out)	Needs `-XX:+UnlockExperimentalVMOptions`; not for production
status *Closed / Withdrawn*	**Withdrawn**	No — removed before finalisation

A feature usually goes preview → preview → … → final, and sometimes gets renamed along the way (e.g. *Statements before super(...)* became *Flexible Constructor Bodies*). This book marks every feature with the release that **finalised** it.

17.2 The version table

GA = general availability date. "Final" rows list what you can rely on; previews are in 17.11.

Release	GA	Language and library — final	JVM, GC and tools — final
17 LTS	2021-09-14	Sealed classes (409), enhanced pseudo-random generators (356), always-strict floating point (306), context-specific deserialization filters (415)	Strong encapsulation of JDK internals (403), macOS/AArch64 port (391; a *port* is a build for another OS/CPU), new macOS rendering pipeline (382); experimental AOT/Graal JIT removed (410); RMI activation removed (407)
18	2022-03-22	UTF-8 by default (400), code snippets in Javadoc `@snippet` (413), internet-address resolution SPI (418)	Simple web server `jwebserver` (408), core reflection reimplemented with method handles (416); finalization deprecated for removal (421)
19	2022-09-20	*(no final language/API features)*	Linux/RISC-V port (422)
20	2023-03-21	*(no final features — previews and incubators only)*	—
21 LTS	2023-09-19	Record patterns (440), pattern matching for `switch` (441), virtual threads (444), sequenced collections (431), key encapsulation mechanism API (452)	Generational ZGC (439); warnings for dynamically loaded agents (451); Windows 32-bit x86 port deprecated (449)
22	2024-03-19	Unnamed variables and patterns (456), Foreign Function & Memory API (454), launch multi-file source programs (458)	Region pinning for G1 (423)
23	2024-09-17	Markdown documentation comments (467)	ZGC generational mode by default (474); `sun.misc.Unsafe` memory-access methods deprecated for removal (471)
24	2025-03-18	Stream gatherers (485), Class-File API (484), quantum-resistant ML-KEM (496) and ML-DSA (497)	Virtual threads synchronize without pinning (491), ahead-of-time class loading and linking (483), ZGC non-generational mode removed (490), Security Manager permanently disabled (486), linking run-time images without JMODs (493), late barrier expansion for G1 (475), JNI restriction warnings (472), `Unsafe` memory-access warnings (498), Windows 32-bit port removed (479), 32-bit x86 port deprecated (501)
25 LTS	2025-09-16	Compact source files and instance `main` methods (512), module import declarations (511), flexible constructor bodies (513), scoped values (506), key derivation function API (510)	Compact object headers (519, opt-in), generational Shenandoah (521), AOT command-line ergonomics (514) and method profiling (515), JFR cooperative sampling (518) and method timing and tracing (520); 32-bit x86 port removed (503)

Numbers in parentheses are JEP numbers. Short glosses for the less obvious entries: **sealed** classes restrict which classes may extend them; **deserialization filters** reject unwanted classes when turning bytes back into objects; **SPI** = *service provider interface*, a plug-in point; **method handles** are fast, typed references to methods; **finalization** is the old `finalize()` clean-up hook; **RISC-V** is an open CPU architecture; **agents** are JARs that instrument (modify) classes as they load; **ZGC/G1/Shenandoah** are garbage collectors; **ML-KEM/ML-DSA** are *quantum-resistant* algorithms designed to stay secure even against future quantum computers; the **Security Manager** was an old in-process sandbox; **JMODs** are the JDK's module packaging files; **x86** is the Intel/AMD CPU family. In three words each: **21** = virtual threads + patterns, **22** = FFM + `_`, **24** = gatherers, **25** = simpler Java (compact source files, module imports, flexible constructors) + scoped values.

17.3 The Java 17 baseline

Teams moving from Java 11 got a whole set of features finalised in 14–17. They are taught in depth elsewhere:

Feature	Final in	JEP	Where in this book
Switch expressions (`->`, `yield`)	14	361	Chapter 3 §3.3
Helpful `NullPointerException` messages (on by default since 15)	14	358	below
Text blocks `"""`	15	378	Chapter 2 §2.12
Records	16	395	Chapter 6 §6.8
Pattern matching for `instanceof`	16	394	Chapter 3 §3.5
`Stream.toList()`	16	—	Chapter 12
`jpackage` (builds native installers such as `.msi`, `.dmg`, `.deb`)	16	392	—
Sealed classes and interfaces	17	409	Chapter 6 §6.7

All of them in one small program: `code/ch17/v17/Java17Baseline.java`

```
sealed interface Shape permits Circle, Square {}                // sealed classes: Java 17
record Circle(double r) implements Shape {}                     // records: Java 16
record Square(double side) implements Shape {}

static double area(Shape s) {
    if (s instanceof Circle c) return Math.PI * c.r() * c.r();   // pattern matching for instanceof: Java 16
    if (s instanceof Square q) return q.side() * q.side();
    throw new AssertionError();
}
static String kind(int day) {
    return switch (day) {                                         // switch expression: Java 14
        case 6, 7 -> "weekend";
        case 1, 2, 3, 4, 5 -> "weekday";
        default -> throw new IllegalArgumentException("day " + day);
    };
}
// main: a text block, then c.address().city().length() with a null address
```

Output:

```
text block : {"shape": "circle", "r": 1.5}
areas      : 7.07 4.00
switch     : weekend, weekday
record     : Circle[r=1.5]
NPE says   : Cannot invoke "ch17.v17.Java17Baseline$Address.city()" because the return value of
"ch17.v17.Java17Baseline$Customer.address()" is null
```

The **helpful NPE** (`NullPointerException`, thrown when you use a reference that points to no object) message names exactly which part of a chain was `null` — no more guessing which of three calls on one line failed.

Enhanced pseudo-random generators (JEP 356, Java 17)

What it is: a common interface `java.util.random.RandomGenerator` for every random source (old `Random`, `SplittableRandom`, `ThreadLocalRandom` and new *LXM* algorithms — a family that mixes a linear congruential generator with an XOR-based one), selectable by name. (*Pseudo-random* = generated by a deterministic formula that only looks random; a **seed** is its starting value, so the same seed gives the same

sequence.) **Why it matters:** code can accept "any generator", and you can pick algorithms with better statistical quality or splitting support for parallel work. `code/ch17/v17/RandomGenerators.java`:

```
RandomGenerator g = RandomGeneratorFactory.of("L64X128MixRandom").create(42); // seeded: repeatable
System.out.println("L64X128MixRandom dice: " + g.ints(8, 1, 7).boxed().toList());
RandomGenerator legacy = new java.util.Random(42);          // old classes implement the interface too
```

Output:

```
L64X128MixRandom dice: [1, 2, 1, 1, 4, 4, 6, 2]
java.util.Random dice: [3, 4, 1, 3, 1, 2, 6, 3]
default algorithm    : L32X64MixRandom
a few algorithms     : [L128X1024MixRandom, L128X128MixRandom, L128X256MixRandom, L32X64MixRandom,
L64X1024MixRandom, L64X128MixRandom]
```

For everyday use `ThreadLocalRandom.current()` or `RandomGenerator.getDefault()` is fine; for security use `SecureRandom` (unpredictable, suitable for passwords and keys; Chapter 7 §7.12).

Strong encapsulation (JEP 403, Java 17)

The JDK's internal packages (`sun.*`, `jdk.internal.*`) are closed to **reflection** (code inspecting or calling classes' private parts at run time) by default — the `--illegal-access` escape hatch was removed. Old libraries that poked at internals fail with `InaccessibleObjectException` unless you add `--add-opens`. This is the #1 upgrade issue from Java 8/11; see Chapter 16 §16.12.

17.4 Java 18

UTF-8 by default (JEP 400)

The problem it solved: before 18, APIs such as `FileReader`, `FileWriter`, `InputStreamReader`, `PrintStream` and `String.getBytes()` used the **platform** charset (the operating system's text encoding — the rule for turning characters into bytes) — UTF-8 on Linux/macOS but often Windows-1252 on Windows — so a file written on one machine could be garbled on another. Since 18 the default is **UTF-8 everywhere**; the OS encoding is still available as the `native.encoding` property. (Console output uses `stdout.encoding`, which still follows the terminal.)

`code/ch17/v18/Utf8Default.java`:

```
System.out.println("Charset.defaultCharset(): " + Charset.defaultCharset());
System.out.println("native.encoding (OS)    : " + System.getProperty("native.encoding"));
Files.writeString(tmp, "café ☕");                          // no charset argument needed
try (var reader = new java.io.FileReader(tmp.toFile())) { ... }   // FileReader also uses UTF-8 now
```

Output (macOS — on Windows `native.encoding` would typically be `Cp1252`, but the first two lines would still say UTF-8):

```
Charset.defaultCharset(): UTF-8
file.encoding           : UTF-8
native.encoding (OS)    : UTF-8
read back               : café ☕ (9 bytes)
```

9 bytes for 6 characters: `é` takes 2 bytes and `☕` 3 in UTF-8. Still best practice: pass a charset explicitly when reading data you did not write (Chapter 13).

Simple web server (JEP 408)

`jwebserver` serves the current directory as **static files** (files sent as-is, with no server-side code) over HTTP — handy for testing HTML or sharing files on a LAN (local network), **not** for production (HTTP/1.1 only, no HTTPS — the encrypted version of HTTP). `jwebserver -p 8000` (`-p` = port, the numbered door a server listens on) then open `http://localhost:8000/` (`localhost` = your own machine) (not run here

— the book's programs don't use the network). The same server is available as an API: `com.sun.net.httpserver.SimpleFileServer`.

Code snippets in Javadoc (JEP 413)

`{@snippet : …}` replaces `<pre>{@code …}</pre>` for examples in documentation (Javadoc — the API docs generated from `/** … */` comments); it supports highlighting, regions and snippets pulled from real, compiled source files — so documentation examples can be tested.

Finalization deprecated for removal (JEP 421)

`Object.finalize()` still works but is on its way out (*deprecated for removal* = officially discouraged and scheduled to be deleted); use try-with-resources (automatic closing with `try (…) { }`) and `java.lang.ref.Cleaner` (registers clean-up code to run after an object becomes unreachable) instead (Chapter 8).

17.5 Java 19 and 20: the preview years

Neither release finalised a language or library feature. They matter because the big Java 21 features were previewed here:

JEP	Feature in 19	JEP	Feature in 20
425	Virtual Threads (Preview)	436	Virtual Threads (Second Preview)
405	Record Patterns (Preview)	432	Record Patterns (Second Preview)
427	Pattern Matching for switch (Third Preview)	433	Pattern Matching for switch (Fourth Preview)
424	Foreign Function & Memory API (Preview)	434	Foreign Function & Memory API (Second Preview)
428	Structured Concurrency (Incubator)	437	Structured Concurrency (Second Incubator)
426	Vector API (Fourth Incubator)	438	Vector API (Fifth Incubator)
—	—	429	Scoped Values (Incubator)

Java 19 also added the Linux/RISC-V port (JEP 422). If you are on 17 and skipping to 21 or 25, you can ignore 19 and 20 entirely.

17.6 Java 21 LTS

Pattern matching for `switch` and record patterns (JEPs 441, 440)

`switch` can match on **types** and **deconstruct records** (pull a record's components out into variables), with `when` **guards** (extra boolean conditions on a case), explicit `case null`, and **exhaustiveness** checking (the compiler verifies every possible subtype is handled) over sealed hierarchies — no `default` needed. Full treatment: Chapter 3 §3.6 and §3.7. `code/ch17/v21/PatternsDemo.java` evaluates a small expression tree:

```java
sealed interface Expr permits Num, Add, Mul, Neg {}
record Num(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
record Mul(Expr left, Expr right) implements Expr {}
record Neg(Expr e) implements Expr {}

static int eval(Expr e) {
    return switch (e) {                                          // exhaustive: no default needed (sealed)
        case Num(int v) -> v;
        case Add(Expr l, Expr r) -> eval(l) + eval(r);
        case Mul(Num(int a), Num(int b)) when a == 0 || b == 0 -> 0;   // nested pattern + guard
        case Mul(Expr l, Expr r) -> eval(l) * eval(r);
        case Neg(Neg(Expr inner)) -> eval(inner);                // double negation
        case Neg(Expr inner) -> -eval(inner);
    };
}
static String describe(Object o) {
    return switch (o) {
        case null -> "null!";
        case Integer i when i > 100 -> "big int " + i;
        case Integer i -> "int " + i;
        case String s -> "string of length " + s.length();
        default -> "something else: " + o.getClass().getSimpleName();
    };
}
```

Output:

```
Add[left=Num[value=2], right=Mul[left=Num[value=3], right=Neg[e=Neg[e=Num[value=4]]]]] = 14
int 7
big int 700
string of length 3
something else: Double
null!
```

If you add a new `Expr` subtype, every `switch` like `eval` stops compiling until you handle it — the compiler finds the places to update. This "data-oriented" style (sealed types + records + pattern switch) is the modern alternative to the Visitor pattern (a classic design pattern for adding operations to a class hierarchy without editing it; LLD handbook, behavioural patterns).

Virtual threads (JEP 444)

Virtual threads are lightweight threads managed by the JVM rather than the OS: you can have millions, and a **blocking** call (one that waits — sleep, socket read, JDBC database query) unmounts the virtual thread from its **carrier** OS thread (the real operating-system thread it was running on) instead of blocking it. They make the simple **thread-per-request** style (one thread handles one incoming request from start to finish) scale for **I/O-bound** servers (ones that mostly wait on network or disk). Full coverage, including pinning and when they *don't* help: Chapter 15. `code/ch17/v21/VirtualThreadsDemo.java`:

```java
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {   // close() waits for all tasks
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> {
            Thread.sleep(Duration.ofMillis(200));            // simulated blocking I/O
            done.incrementAndGet();
            return null;
        });
    }
}
```

Output:

```
tasks finished: 10000
took < 2 s    : true   (10,000 x 200 ms sequentially would be 2,000 s)
is virtual    : true
```

(The program prints a yes/no instead of the elapsed time, which varies by machine; on the author's machine it was a few hundred milliseconds.)

Sequenced collections (JEP 431)

New interfaces `SequencedCollection`, `SequencedSet` and `SequencedMap` give every ordered collection `getFirst`, `getLast`, `addFirst`, `addLast`, `removeFirst/Last` and `reversed()` (maps: `firstEntry`, `lastEntry`, `pollFirstEntry`, `putFirst`, …). Full coverage: Chapter 10 §10.11. `code/ch17/v21/SequencedDemo.java`:

```
List<String> list = new ArrayList<>(List.of("b", "c"));
list.addFirst("a"); list.addLast("d");
LinkedHashSet<Integer> set = new LinkedHashSet<>(List.of(3, 1, 2));
set.addFirst(2);                                  // moves an existing element to the front
SequencedMap<String, Integer> map = new LinkedHashMap<>();
map.put("x", 1); map.put("y", 2); map.putFirst("w", 0);
```

Output:

```
[a, b, c, d] first=a last=d reversed=[d, c, b, a]
[2, 3, 1] reversed=[1, 3, 2]
{w=0, x=1, y=2} firstEntry=w=0 pollLast=y=2 -> {w=0, x=1}
```

Also final in 21

- **Generational ZGC** (JEP 439) — opt-in with `-XX:+UseZGC -XX:+ZGenerational` in 21; default mode from 23, the only mode from 24 (Chapter 16 §16.5).
- **Key Encapsulation Mechanism API** (JEP 452) — `javax.crypto.KEM`, the building block for hybrid and **post-quantum** key exchange (agreeing on a shared secret key in a way quantum computers can't break).
- **Warnings for dynamically loaded agents** (JEP 451) — attaching a Java agent to a running JVM now prints a warning; tools like Mockito's (the popular **mocking** library — mocks are fake objects used in tests, Chapter 18) inline mock maker should be loaded with `-javaagent` at start-up.

17.7 Java 22

Unnamed variables and patterns (JEP 456)

Use `_` for anything you must declare but never read: loop variables, `catch` parameters, lambda parameters, try-with-resources variables, and parts of record patterns. It documents intent and silences "unused" warnings. `code/ch17/v22/UnnamedDemo.java`:

```
for (var _ : List.of("a", "b", "c")) count++;                // loop variable not needed
catch (NumberFormatException _) { ... }                      // exception object not needed
m.forEach((k, _) -> System.out.print(k + " "));              // lambda parameter not needed
if (o instanceof Line(Point(var x1, _), Point(var x2, _)))   // ignore the y components
    System.out.println("horizontal distance = " + Math.abs(x2 - x1));
String kind = switch (o) {
    case Line(Point _, Point(int x, int y)) when x == y -> "ends on the diagonal";
    case Line _ -> "a line";                                 // unnamed pattern variable
    default -> "other";
};
```

Output:

```
count = 3
not a number
x y
horizontal distance = 3
a line
```

`_` has been a reserved keyword since Java 9, so no valid old code breaks. You can use several `_` in the same scope.

Foreign Function & Memory API (JEP 454)

What it is: a pure-Java way to call **native** (C) libraries and to work with memory outside the **heap** (the JVM-managed memory where objects live; memory outside it is called *off-heap*) — the safe, fast replacement for JNI (the Java Native Interface, which needed hand-written C glue code) and for `sun.misc.Unsafe` memory access. **Key types:** `Linker` (makes a Java `MethodHandle` — a callable reference to a function — for a C function), `FunctionDescriptor` (the C signature), `Arena` (owns off-heap memory and frees it when closed), `MemorySegment` (a bounded, checked region of memory), `ValueLayout` (C types like `int`, `long`, pointer).

`code/ch17/v22/ForeignStrlen.java` calls the C library's `strlen`:

```
Linker linker = Linker.nativeLinker();
MethodHandle strlen = linker.downcallHandle(
        linker.defaultLookup().find("strlen").orElseThrow(),
        FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));   // size_t strlen(const char*)

try (Arena arena = Arena.ofConfined()) {                    // off-heap memory freed at the end of the block
    MemorySegment cString = arena.allocateFrom("Hello, native world!");     // UTF-8, NUL-terminated
    long len = (long) strlen.invokeExact(cString);
    MemorySegment ints = arena.allocate(ValueLayout.JAVA_INT, 5);            // like int[5] in C
    for (int i = 0; i < 5; i++) ints.setAtIndex(ValueLayout.JAVA_INT, i, i * i);
}
```

Output of `java -cp out ch17.v22.ForeignStrlen` (warnings go to standard error; path shortened):

```
WARNING: A restricted method in java.lang.foreign.Linker has been called
WARNING: java.lang.foreign.Linker::downcallHandle has been called by ch17.v22.ForeignStrlen in an unnamed module
(file:.../out17/)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

strlen = 20
off-heap ints: [0, 1, 4, 9, 16], 20 bytes
```

Calling native code can crash the JVM, so such methods are **restricted**: grant access explicitly with `--enable-native-access=ALL-UNNAMED` (or a module name). Real-world users: Apache Lucene (the search library behind Elasticsearch; uses memory-mapped index files), database drivers, and **bindings** (ready-made Java wrappers for a C library) generated by the **jextract** tool from C header files (`.h` files that declare a C library's functions).

Launch multi-file source-code programs (JEP 458)

`java Main.java` now compiles and runs a program spread over several source files in a directory tree — no build step for small tools and learning. See Chapter 1 §1.10.

Region pinning for G1 (JEP 423)

Native code holding a **pointer** (a raw memory address) into a Java array (a JNI "critical region") no longer blocks G1 garbage collection; G1 just avoids moving that one region.

17.8 Java 23

Markdown documentation comments (JEP 467)

Javadoc comments can be written in **Markdown** (the lightweight `**bold**`, `- list` text format) using `///` lines instead of HTML inside `/** … */`. `code/ch17/v23/MarkdownDoc.java`:

```java
/// JEP 467 (Java 23): documentation comments written in **Markdown**.
///
/// Lines start with `///` instead of the `/** ... */` block, and use Markdown instead of HTML:
///
/// - lists like this one
/// - `inline code` and **bold**
/// - links to program elements: [String#strip()]
///
/// @param <T> the element type (block tags such as `@param` still work)
public class MarkdownDoc<T> {
```

`javadoc -d docs MarkdownDoc.java` turns it into this HTML (real output, the class description block):

```
<p>JEP 467 (Java 23): documentation comments written in <strong>Markdown</strong>.</p>
<p>Lines start with <code>///</code> instead of the <code>/** ... */</code> block, and use Markdown instead of
HTML:</p>
<ul>
<li>lists like this one</li>
<li><code>inline code</code> and <strong>bold</strong></li>
<li>links to program elements: <a
href="https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/lang/String.html#strip()" class="external-
link"><code>String.strip()</code></a></li>
</ul>
```

`[String#strip()]` became a link to the JDK's API page — reference links to program elements are the Markdown replacement for `{@link …}`.

Also in 23

- **ZGC generational mode by default** (JEP 474).
- **`sun.misc.Unsafe` memory-access methods deprecated for removal** (JEP 471; `Unsafe` is an internal JDK class for raw memory access) — the FFM API and `VarHandle` (a typed, safe handle for reading/writing a variable with chosen memory-ordering guarantees) are the replacements.
- **String templates withdrawn.** They were previewed in 21 (JEP 430) and 22 (JEP 459); a third preview (JEP 465) was proposed for 23 but **withdrawn**, and the feature was removed from JDK 23. The designers decided the syntax (`STR."Hello \{name}"`) and processor model needed rethinking. There is no replacement yet — use `String.formatted`, `+`, `StringBuilder` or `MessageFormat` (Chapter 7 §7.4). Don't trust tutorials from 2023–24 that show `STR.` templates: that code doesn't compile on any current JDK.

17.9 Java 24

Stream gatherers (JEP 485)

`Stream.gather(Gatherer)` adds **custom intermediate operations** (steps in the middle of a stream pipeline, like `map` or `filter`) — the missing counterpart to custom collectors. Built-ins in `java.util.stream.Gatherers`: `windowFixed`, `windowSliding`, `fold`, `scan`, `mapConcurrent`. Full coverage, including writing your own: Chapter 12 §12.9. `code/ch17/v24/GatherersDemo.java`:

```java
Stream.of(1, 2, 3, 4, 5, 6, 7).gather(Gatherers.windowFixed(3)).toList()
Stream.of(1, 2, 3, 4, 5).gather(Gatherers.windowSliding(2)).toList()
Stream.of(1, 2, 3, 4, 5).gather(Gatherers.scan(() -> 0, Integer::sum)).toList()
Stream.of("a", "b", "c").gather(Gatherers.fold(() -> "", String::concat)).toList()
```

Output:

```
[[1, 2, 3], [4, 5, 6], [7]]
[[1, 2], [2, 3], [3, 4], [4, 5]]
[1, 3, 6, 10, 15]
[abc]
```

Class-File API (JEP 484)

What it is: a standard API (`java.lang.classfile`) for reading, writing and transforming `.class` files. **Why:** frameworks and tools (Spring, Hibernate, Mockito, the JDK itself) generate or rewrite **bytecode** (the compiled instructions inside `.class` files), traditionally with the third-party ASM library, which always lags behind new class-file versions. The JDK's own API is updated in lock-step with each release. `code/ch17/v24/ClassFileDemo.java` parses its own class file:

```
ClassModel cm = ClassFile.of().parse(bytes);
for (MethodModel m : cm.methods()) {
    for (CodeElement e : m.code().get()) {                // walk the bytecode as typed objects
        if (e instanceof Instruction) instructions++;
        if (e instanceof InvokeInstruction inv) calls.add(inv.name().stringValue());
    }
}
```

Output:

```
class   : ch17/v24/ClassFileDemo  (class-file major version 69)
method  : <init>   ()V                        3 instructions, 1 calls [<init>]
method  : square   (I)I                       4 instructions, 0 calls []
method  : main     ([Ljava/lang/String;)V   133 instructions, 36 calls
square(7) = 49
```

Major version 69 = Java 25 (Chapter 1 §1.6). Note how pattern matching (`instanceof InvokeInstruction inv`) makes walking a model natural. Application developers rarely need this API directly.

Virtual threads without pinning (JEP 491)

Before 24, a virtual thread that blocked **inside a `synchronized` block** stayed *pinned* (stuck) to its carrier OS thread, which could starve the scheduler (leave it with no free carriers) — the standard advice was "replace `synchronized` with `ReentrantLock`". Since 24, virtual threads can release their carrier while holding or waiting for a **monitor** (the lock behind `synchronized`), so that advice is largely obsolete. (Pinning still happens in native frames.) Details: Chapter 15.

Also final in 24

- **Ahead-of-time class loading and linking** (JEP 483): record a **training run** (a sample run of the app) into an AOT (*ahead-of-time*) cache, start faster in production (Chapter 1 §1.7).
- **Security Manager permanently disabled** (JEP 486): `System.setSecurityManager` throws; `-Djava.security.manager` is an error.
- **Post-quantum cryptography**: ML-KEM key encapsulation (JEP 496) and ML-DSA signatures (JEP 497), standardised by NIST (the US National Institute of Standards and Technology) in FIPS 203/204 (US federal standards).
- **ZGC non-generational mode removed** (JEP 490), **late barrier expansion for G1** (JEP 475, a JIT-internal simplification — the JIT is the JVM's just-in-time compiler, and *barriers* are small bits of code it adds for the garbage collector), **linking run-time images without JMODs** (JEP 493, lets `jlink` work on JDKs shipped without the `jmods` folder).
- Integrity warnings: **JNI** use prints a warning unless enabled (JEP 472); `Unsafe` **memory access** prints a warning on first use (JEP 498).

17.10 Java 25 LTS

Compact source files and instance `main` methods (JEP 512)

A source file can now contain just methods and fields — the class around them is implicit — and `main` can be an instance method without `String[] args`. A new class `java.lang.IO` offers `IO.println` and `IO.readln`, and compact source files automatically import all of `java.base`. This is covered with runnable examples in Chapter 1 §1.10 and Chapter 4 §4.9:

```
void main() {
    IO.println("Hello, Java 25");
}
```

It was previewed four times under different names: *Unnamed Classes and Instance Main Methods* (21, JEP 445), *Implicitly Declared Classes…* (22 JEP 463, 23 JEP 477), *Simple Source Files…* (24, JEP 495).

Module import declarations (JEP 511)

`import module M;` imports every public top-level type in every package that module `M` exports (and that it transitively reads through `requires transitive`). It saves long import lists in scripts, prototypes and teaching code; ordinary code can use it too. `code/ch17/v25/ModuleImportDemo.java`:

```
package ch17.v25;

import module java.base;          // JEP 511 (Java 25): imports every package exported by java.base

public class ModuleImportDemo {
    public static void main(String[] args) {
        List<String> words = List.of("modules", "import", "java", "base");
        Map<Integer, List<String>> byLength = words.stream().collect(Collectors.groupingBy(String::length));
        IO.println("by length : " + new TreeMap<>(byLength));                // java.lang.IO: Java 25 (JEP 512)
        IO.println("a path    : " + Path.of("docs", "ch17.md"));
        IO.println("a date    : " + LocalDate.of(2025, 9, 16).getDayOfWeek()); // Java 25 GA day
    }
}
```

Output:

```
by length : {4=[java, base], 6=[import], 7=[modules]}
a path    : docs/ch17.md
a date    : TUESDAY
```

Trade-off: ambiguity. Add `import module java.desktop;` next to `java.base` and a plain `List` becomes ambiguous (`java.util.List` or `java.awt.List`?) — fix it with a normal single-type import (`import java.util.List;`), which always wins over module imports. The importing class does not have to be in a named module: this demo lives on the class path.

Flexible constructor bodies (JEP 513)

Statements may now appear **before** `super(…)` (the call to the parent class's constructor) or `this(…)`, as long as they don't use the object being constructed (except to assign its own fields). **The problem it solves:** validating or computing arguments used to need a static helper method or wasted superclass work; and a superclass constructor calling an overridden method saw the subclass's fields still at default values. `code/ch17/v25/FlexibleConstructors.java`:

```
static class Account {
    final String id;
    Account(String id) { this.id = id; describe(); }
    void describe() { System.out.println("  Account(" + id + ")"); }
}
static class SavingsAccount extends Account {
    final double rate;
    SavingsAccount(String id, double rate) {
        if (rate < 0 || rate > 0.2)                              // validate BEFORE the superclass runs
            throw new IllegalArgumentException("bad rate " + rate);
        this.rate = rate;                                        // allowed: initialise own field early
        super(id.strip().toUpperCase());                         // prepare the argument, then call super
    }
    @Override void describe() {                                 // called from Account's constructor:
        System.out.println("  SavingsAccount(" + id + ", rate=" + rate + ")");   // rate is already set
    }
}
```

Output:

```
  SavingsAccount(SA-1, rate=0.03)
  rejected before any superclass work: bad rate 5.0
```

Before Java 25 the overridden `describe()` would have printed `rate=0.0` (the field wasn't assigned yet). Still, calling overridable methods from constructors is a **design smell** (a sign of a likely design problem) (Chapter 6); this feature makes it safe where it can't be avoided. Not allowed in the **prologue** (the code before `super(…)`): reading fields, calling instance methods, using `this` as a value.

Scoped values (JEP 506)

A `ScopedValue` passes read-only context (the current user, a request ID, a **transaction** — a group of database operations that succeed or fail together) to code deep in a call chain **without** method parameters — like a `ThreadLocal` (a variable with a separate value per thread), but immutable, automatically unbound when the scope ends (no leaks on thread pools), and cheap to inherit into child threads in **structured concurrency** (a way of running subtasks as one unit; still preview, §17.11). Details and comparison with `ThreadLocal`: Chapter 15. `code/ch17/v25/ScopedValueDemo.java`:

```
static final ScopedValue<String> USER = ScopedValue.newInstance();

static void handle() { audit("read order 17"); }                 // no user parameter threaded through
static void audit(String action) {
    System.out.println("  audit: " + (USER.isBound() ? USER.get() : "<anonymous>") + " -> " + action);
}
// main:
ScopedValue.where(USER, "ana").run(ScopedValueDemo::handle); // bound only while run() executes
ScopedValue.where(USER, "bo").run(() -> {
    handle();
    ScopedValue.where(USER, "admin").run(ScopedValueDemo::handle);   // rebinding for a nested scope
    handle();                                                       // back to "bo"
});
handle();                                                            // unbound again
```

Output:

```
  audit: ana -> read order 17
  audit: bo -> read order 17
  audit: admin -> read order 17
  audit: bo -> read order 17
  audit: <anonymous> -> read order 17
bound outside? false
```

Key derivation function API (JEP 510)

`javax.crypto.KDF` derives **keys** (the secret values encryption algorithms use) from a secret, e.g. with **HKDF** (HMAC-based Key Derivation Function, used by TLS 1.3 — the encryption protocol behind HTTPS — and many protocols). `code/ch17/v25/HkdfDemo.java` checks the JDK against test case 1 of RFC 5869 (an RFC, *Request for Comments*, is an official internet standard document), the HKDF standard:

```
KDF hkdf = KDF.getInstance("HKDF-SHA256");
byte[] okm = hkdf.deriveData(HKDFParameterSpec.ofExtract()
        .addIKM(ikm).addSalt(salt).thenExpand(info, 42));    // extract-then-expand, 42 bytes out
```

Output:

```
OKM      = 3cb25f25faacd57a90434f64d0362f2a2d2d0a90cf1a5a4c5db02d56ecc4c5bf34007208d5b887185865
RFC 5869 = 3cb25f25faacd57a90434f64d0362f2a2d2d0a90cf1a5a4c5db02d56ecc4c5bf34007208d5b887185865
match    : true
```

Use `deriveKey("AES", spec)` (AES = the standard symmetric encryption algorithm) to get a `SecretKey` instead of raw bytes.

JVM and tooling in 25

JEP	Feature	One line
519	Compact object headers	8-byte object headers with `-XX:+UseCompactObjectHeaders`: smaller heaps (Chapter 16 §16.4)
521	Generational Shenandoah	`-XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational`, now a product feature
514	AOT command-line ergonomics	One-step AOT cache creation: `-XX:AOTCacheOutput=app.aot`
515	AOT method profiling	The AOT cache also stores method profiles, so the JIT warms up faster
518	JFR cooperative sampling	Safer, more robust stack sampling in Flight Recorder (JFR, the JVM's built-in low-overhead event recorder)
520	JFR method timing and tracing	Time or trace chosen methods via JFR without changing code
503	Remove the 32-bit x86 port	Only 64-bit x86 remains

17.11 Still preview or incubating in Java 25 — and what came next

These are **not** final in Java 25. Don't use them in production code; the book has no runnable examples of them. Each row shows every round up to JDK 27 (GA 15 September 2026).

Feature	History (JEP numbers)	Status in 25	After 25
Structured concurrency — treat a group of subtasks as one unit (fail together, cancel together)	Incubator 19 (428), 20 (437); preview 21 (453), 22 (462), 23 (480), 24 (499)	**Fifth preview** (505)	Sixth preview in 26 (525), seventh in 27 (533)
Primitive types in patterns, `instanceof` and `switch` — e.g. `case int i when i > 0`, `x instanceof byte b`	Preview 23 (455), 24 (488)	**Third preview** (507)	Fourth preview in 26 (530), fifth in 27 (532)
Stable values — lazily initialised constants the JIT can treat as `final`	—	**Preview** (502)	Renamed **Lazy Constants**: second preview in 26 (526), third in 27 (531)
PEM encodings of cryptographic objects — read/write keys and certificates (signed documents proving who owns a public key) as PEM text (the `-----BEGIN …-----` Base64 format)	—	**Preview** (470)	Second preview in 26 (524), third in 27 (538)
Vector API — portable SIMD computations (*single instruction, multiple data*: one CPU instruction works on several numbers at once) (`jdk.incubator.vector`)	Incubating since 16; 17 (414) … 24 (489)	**Tenth incubator** (508)	11th incubator in 26 (529), 12th in 27 (537) — waiting for Project Valhalla's value classes (objects without identity that can be stored as compactly as primitives)
JFR CPU-time profiling (Linux only)	—	**Experimental** (509)	—

To try a preview on JDK 25: `javac --release 25 --enable-preview Foo.java` and `java --enable-preview Foo`; single-file launch: `java --enable-preview --source 25 Foo.java`.

What JDK 26 (GA 17 March 2026) and JDK 27 (GA 15 September 2026) finalised — both non-LTS, noted briefly:

- **26**: HTTP/3 (the newest HTTP version, running over the UDP-based QUIC protocol) for the HTTP Client API (517), AOT object caching with any GC (516), G1 throughput improvement by reducing synchronization (522), *Prepare to Make Final Mean Final* (500 — warnings when deep reflection mutates `final` fields), Applet API removed (504).
- **27**: G1 the default collector in all environments (523), compact object headers on by default (534), post-quantum hybrid key exchange for TLS 1.3 (527), JFR in-process data redaction (536).

The next LTS is expected to be **Java 29** (September 2027).

17.12 Upgrading 17 → 21 → 25: removals and deprecations

Change	JEP, release	What breaks / what to do
JDK internals strongly encapsulated	403, 17	Reflection into `sun.*`/`java.*` private members fails → upgrade the library, or `--add-opens` temporarily
Security Manager deprecated → permanently disabled	411 (17) → 486 (24)	Code calling `System.setSecurityManager` fails; remove it
Finalization deprecated for removal	421, 18	Replace `finalize()` with try-with-resources / `Cleaner`
Applet API deprecated → removed	398 (17) → 504 (26)	Nothing for server code
Dynamic agent loading warns	451, 21	Load agents with `-javaagent` at start-up (e.g. Mockito in tests: add the agent to the test JVM)
`sun.misc.Unsafe` memory access deprecated → warns	471 (23) → 498 (24)	Upgrade libraries (Netty, older serializers) to versions using FFM/`VarHandle`
JNI use warns	472, 24	Add `--enable-native-access=…` for libraries that use native code
ZGC non-generational mode removed	490, 24	Drop `-XX:-ZGenerational`
32-bit ports removed	479 (24, Windows x86), 503 (25, x86)	Use 64-bit JDKs
String templates withdrawn	465, 23	Code written against the 21/22 preview must be rewritten

A practical upgrade checklist

1. Build and run the **test suite** (all the project's automated tests) on the new JDK with `--release` still at the old version, then raise `--release`.
2. Upgrade build plugins (add-ons for the build tools Maven/Gradle, Chapter 18) and bytecode-heavy libraries first (Maven/Gradle plugins, Lombok — generates boilerplate code at compile time — Mockito, Byte Buddy — a bytecode-generation library — Spring, Hibernate): they must understand the new class-file version.
3. Run `jdeps --jdk-internals your.jar` to find uses of JDK internals and `jdeprscan your.jar` to find deprecated APIs.
4. Watch the startup warnings (agents, JNI, `Unsafe`) — they become errors in future releases.
5. Re-check GC and memory settings; defaults changed (generational ZGC, compact headers in 27).

17.13 In the interview

Q: What changed in Java between 17 and 21, and between 21 and 25? 17 → 21: pattern matching for `switch` and record patterns (with sealed types, a new data-oriented style), **virtual threads**, sequenced collections, generational ZGC. 21 → 25: unnamed `_` variables, the Foreign Function & Memory API, stream gatherers, the Class-File API, virtual threads no longer pinned by `synchronized`, and in 25 itself compact source files with instance `main`, module import declarations, flexible constructor bodies, scoped values and AOT start-up improvements.

- *Is structured concurrency final?* No — it is still in preview (fifth preview in 25, seventh in 27).
- *What happened to string templates?* Previewed in 21 and 22, withdrawn in 23; no replacement yet.
- *Why upgrade from 21 to 25?* LTS support horizon, no `synchronized` pinning with virtual threads, faster start-up with AOT caches, smaller heaps with compact object headers, and the new language conveniences.
- *Can you use a preview feature in production?* Technically with `--enable-preview`, but it may change or disappear in the next release and class files compiled with previews only run on that exact release — so no.

Self-check

1. In which release did virtual threads become final, and which JEP? → Java 21, JEP 444.
2. Name two features finalised in Java 25. → Any of: compact source files and instance main (512), module import declarations (511), flexible constructor bodies (513), scoped values (506), KDF API (510), compact object headers (519), generational Shenandoah (521).
3. Did Java 19 or 20 finalise any language features? → No — only previews/incubators (and the RISC-V port in 19).
4. What does `import module java.base;` import? → All public top-level types of every package exported by `java.base`.
5. Which statements are allowed before `super(...)` since Java 25? → Ones that don't use `this` — validation, computing arguments, and assigning the class's own fields.

Chapter TL;DR

- Status words matter: **`final`** = use it; **`preview/incubator/experimental`** = experiments only; **`withdrawn`** = gone. Check the JEP title on openjdk.org.
- **17**: sealed classes, new random generators, strong encapsulation (plus records, text blocks, `switch` expressions, `instanceof` patterns from 14–16).
- **18**: UTF-8 by default, `jwebserver`, `@snippet`. **19–20**: previews only.
- **21 LTS**: pattern matching for `switch`, record patterns, **virtual threads**, sequenced collections, generational ZGC.
- **22**: `_` unnamed variables and patterns, **FFM API**, multi-file source launch. **23**: Markdown doc comments; string templates withdrawn.
- **24**: **stream gatherers**, Class-File API, virtual threads without `synchronized` pinning, AOT class loading, Security Manager disabled, ML-KEM/ML-DSA.
- **25 LTS**: compact source files + instance `main` + `IO`, `import module`, flexible constructor bodies, **scoped values**, KDF API, compact object headers (opt-in), generational Shenandoah.
- Still preview in 25: structured concurrency, primitive patterns, stable values (→ lazy constants), PEM; Vector API incubating. JDK 26/27 finalised HTTP/3, G1-everywhere and compact headers by default.
- Next: Chapter 18 — Testing, build and tooling.

Chapter 18 — Testing, Build and Tooling

Everything so far was compiled with a bare `javac` and run with `java`. Real projects add four things around the language: a **build tool** that downloads **libraries** (reusable code written by others, shipped as JAR files) and runs every step (compile, test, package) in the right order, **automated tests** that prove the code still works after each change, **logging** that tells you what a running program did, and a **package** (a JAR — *Java ARchive*, a zip file of compiled classes, §18.12) you can ship. This chapter teaches all four on one small shop application, plus the JDBC (Java Database Connectivity, Java's standard database API) basics you need to talk to a database, and ends with a map of where to go next (Spring Boot, the most popular Java framework for building services).

What you'll learn

#	Topic
18.1	The shape of a real Java project: layout, coordinates, repositories
18.2	Maven: `pom.xml`, dependencies and scopes, the lifecycle, a real offline build
18.3	Gradle: `build.gradle.kts`, configurations, tasks, Maven vs Gradle
18.4	JUnit (Jupiter): first tests, assertions, lifecycle, nested tests, tags, running from the command line
18.5	Parameterized, repeated and dynamic tests
18.6	Test doubles (stand-in objects used in tests) and Mockito
18.7	AssertJ fluent assertions (checks written as a readable chain of method calls)
18.8	Reading a failing test report
18.9	Test design: what makes a good test, coverage, flaky (randomly failing) tests, integration tests
18.10	Logging: `java.util.logging` (run) and SLF4J (Simple Logging Facade for Java; snippet)
18.11	JDBC basics: connections, prepared statements, **transactions** (groups of database changes that succeed or fail together), a repository
18.12	Packaging: JAR files, executable JARs, fat JARs (one JAR containing all dependencies), jlink/jpackage
18.13	Other everyday tools
18.14	Where next: Spring Boot and friends

How the code was run. All sources are in `code/`:

Folder	What	How it runs
`code/ch18/shop/`	the application: `Cart`, `Discounts`, `OrderService`, … and `ShopDemo` (has `main`)	the book's runner (`javac --release 25`, JDK 26)
`code/test/ch18/shop/`	JUnit tests (marked `// tools_run: skip` — they have no `main`)	`code/run_tests.sh`
`code/ch18/logging/`	`JulDemo`	the book's runner
`code/ch18/jdbc/`	`JdbcOrderRepository`, `JdbcDemo` (skip-marked: needs a database)	by hand against the SQL Handbook's private PostgreSQL (a popular open-source relational database) (18.11)
`code/build-files/`	`pom.xml`, `build.gradle.kts`, `settings.gradle.kts`	Maven 3.9.14 and Gradle 9.4.1 **offline**, from the local **artifact** cache (an artifact is a published file such as a library JAR; 18.2, 18.3)
`code/package_jar.sh`	builds and runs a JAR with the `jar` tool	by hand (18.12)

Test libraries were downloaded from Maven Central (`repo1.maven.org`, SHA-1 checked — a *checksum* comparison proving the file wasn't corrupted or altered) into `ROOT/.java-books/lib/`: **JUnit 6.1.3** (`junit-platform-console-standalone`, the latest release at the time of writing), **Mockito 5.24.0** (`mockito-core`, `mockito-junit-jupiter`) with **Byte Buddy 1.18.14** (`byte-buddy`, `byte-buddy-agent`) and **Objenesis 3.3**, and **AssertJ 3.27.7**. The Maven and Gradle builds used the versions already in the local cache (JUnit 6.0.3, Mockito 5.23.0) because they ran without network. Every output below is real.

18.1 The shape of a real Java project

What it is

A Java project is more than `.java` files. It has **dependencies** (other people's JARs that your code needs), **tests**, **resources** (config files, templates), and **build outputs** (compiled classes, the final JAR). Both mainstream build tools agree on one **standard directory layout**, so any Java developer can open any project:

```
shop/
├── pom.xml                  (Maven)  or  build.gradle.kts + settings.gradle.kts (Gradle)
├── mvnw / gradlew           the build-tool "wrapper" script (commit it)
└── src/
    ├── main/
    │   ├── java/ch18/shop/Cart.java        production code (package = folder path)
    │   └── resources/                       copied as-is onto the class path (e.g. logback.xml)
    └── test/
        ├── java/ch18/shop/CartTest.java     tests: same package, separate folder
        └── resources/                       test-only resources
target/  (Maven)  or  build/  (Gradle)        generated — never commit
```

(The **class path** is the list of folders and JARs where the JVM looks for classes.) Tests live in the **same package** as the code they test but in a **separate source folder**. That gives them access to **package-private** members (ones with no `public`/`private`/`protected` modifier, visible only inside the same package), while the build keeps test classes out of the final JAR. This chapter's `code/` folder mirrors it: `code/ch18/shop` is "main", `code/test/ch18/shop` is "test".

Coordinates and repositories

Every library is identified by three **coordinates**, written `groupId:artifactId:version` — for example `org.assertj:assertj-core:3.27.7`.

Term	Meaning
groupId	the organisation, reversed domain style (`org.junit.jupiter`)
artifactId	the library's name (`junit-jupiter-api`)
version	a release (`6.1.3`) or a **SNAPSHOT** (`1.0.0-SNAPSHOT`: work in progress, may change)
artifact	a file published under those coordinates — usually a JAR plus a `.pom` describing its own dependencies
repository	a server storing artifacts; the public one is **Maven Central** (`repo1.maven.org/maven2`, searchable at central.sonatype.com)
local repository	the cache on your disk (`~/.m2/repository` for Maven, `~/.gradle/caches` for Gradle) — downloaded once, reused by every project
transitive dependency	a dependency of a dependency, pulled in automatically (Mockito brings Byte Buddy and Objenesis)

A repository is laid out by coordinates: `org.assertj:assertj-core:3.27.7` lives at `repo1.maven.org/maven2/org/assertj/assertj-core/3.27.7/assertj-core-3.27.7.jar` — how this chapter fetched its jars by hand, and what a build tool does for every (transitive) dependency.

Why a build tool

Without one you'd download jars by hand, keep versions consistent, type long `-cp` lines and remember to test before packaging. A build tool turns that into one file and one command (`mvn verify`, `./gradlew build`) that behaves the same on every machine and on CI (*continuous integration*: a server that automatically builds and tests every pushed change).

TL;DR

Standard layout (`src/main/java`, `src/test/java`), libraries identified by `group:artifact:version`, downloaded from Maven Central into a local cache. The build tool does the rest.

18.2 Maven

What it is

Apache Maven (2004) is a build tool driven by a **declarative** (describes *what*, not step-by-step *how*) **XML** (a tag-based text format, `<tag>value</tag>`) file, `pom.xml` (**P**roject **O**bject **M**odel). You describe *what* the project is — coordinates, dependencies, **plugin** settings (a plugin is an add-on JAR that does one kind of build work) — and Maven's fixed **lifecycle** decides *how* and *in which order* things happen. Analogy: a set menu — the courses always come in the same order; you only choose the dishes. The current stable line is Maven 3.9 (3.9.16 at the time of writing); Maven 4.0 was still at **release-candidate** stage (a near-final version published for last testing).

The `pom.xml` — `code/build-files/maven/pom.xml`

Key parts (the full file adds pinned plugin versions):

```xml
<project ...>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>shop</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>25</maven.compiler.release>   <!-- same as javac option: release 25 -->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencyManagement>                        <!-- versions only; adds nothing by itself -->
    <dependencies>
      <dependency>                              <!-- a BOM pins every JUnit module's version -->
        <groupId>org.junit</groupId> <artifactId>junit-bom</artifactId> <version>6.0.3</version>
        <type>pom</type> <scope>import</scope>
      </dependency>
      <dependency>                              <!-- one Byte Buddy for Mockito AND AssertJ -->
        <groupId>net.bytebuddy</groupId> <artifactId>byte-buddy</artifactId> <version>1.18.11</version>
      </dependency>
      <!-- ... byte-buddy-agent 1.18.11 ... -->
    </dependencies>
  </dependencyManagement>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId> <artifactId>junit-jupiter</artifactId>
      <scope>test</scope>                       <!-- no version: taken from the BOM -->
    </dependency>
    <dependency>
      <groupId>org.mockito</groupId> <artifactId>mockito-junit-jupiter</artifactId>
      <version>5.23.0</version> <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.assertj</groupId> <artifactId>assertj-core</artifactId>
      <version>3.27.7</version> <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- maven-dependency-plugin 'properties' goal: exposes the mockito-core jar path as a property -->
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId> <version>3.5.4</version>
        <configuration>
          <argLine>-javaagent:${org.mockito:mockito-core:jar} -Xshare:off</argLine>
          <excludedGroups>demo-failure</excludedGroups>        <!-- JUnit tags to skip -->
        </configuration>
      </plugin>
      <plugin>
        <artifactId>maven-jar-plugin</artifactId> <version>3.5.1</version>
        <configuration>
          <archive><manifest><mainClass>ch18.shop.ShopDemo</mainClass></manifest></archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

What each part does:

- **Coordinates** of *this* project — what other projects would write to depend on it.
- `maven.compiler.release` — passes `--release 25` to `javac` (Chapter 1 §1.5).
- `dependencyManagement` — a table of versions; it *declares* nothing. A **BOM** (bill of materials) is a pom that is only such a table, imported with `<scope>import</scope>`, so all JUnit modules share one version.
- `dependencies` — what the project actually uses, each with a **scope**.

- `build/plugins` — configure the plugins that do the work (compile, test, jar).

Dependency scopes

Scope	On compile class path	On test class path	Packaged / at run time	Typical example
`compile` (default)	yes	yes	yes	Jackson (JSON library), Guava (Google's utility library)
`provided`	yes	yes	no — the runtime supplies it	Servlet API (Java's standard web-request API) inside an **app server** (a server like Tomcat that hosts deployed web applications)
`runtime`	no	yes	yes	JDBC **driver** (the vendor library that actually talks to a specific database; code uses only `java.sql`)
`test`	no (main code)	yes	no	JUnit, Mockito, AssertJ
`import`	—	—	—	only for a BOM inside `dependencyManagement`

`system` also exists but is **deprecated** (officially discouraged, may be removed) — don't use it.

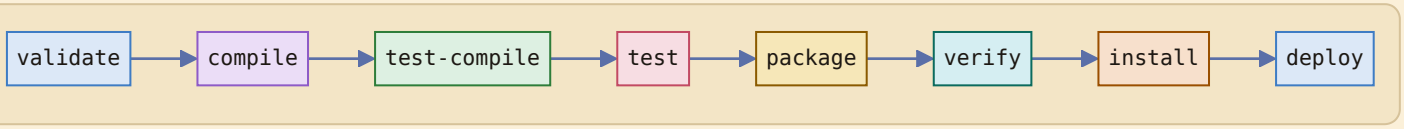
Transitive dependencies and version conflicts

Maven builds a dependency **tree**. When two paths bring *different versions* of one library, Maven picks the **nearest** one in the tree; on a tie, the one declared **first**. This build hit a real conflict: `mockito-core` wants Byte Buddy 1.17.7 at depth 3, `assertj-core` wants 1.18.3 at depth 2 — so "nearest wins" picked AssertJ's version, and Mockito's `byte-buddy-agent` would have stayed at a different version. The fix is to **pin** the version in `dependencyManagement`. The real tree after pinning (`mvn -o dependency:tree`):

```
[INFO] com.example:shop:jar:1.0.0-SNAPSHOT
[INFO] +- org.junit.jupiter:junit-jupiter:jar:6.0.3:test
[INFO] |  +- org.junit.jupiter:junit-jupiter-api:jar:6.0.3:test
[INFO] |  |  +- org.opentest4j:opentest4j:jar:1.3.0:test
[INFO] |  |  +- org.junit.platform:junit-platform-commons:jar:6.0.3:test
[INFO] |  |  +- org.apiguardian:apiguardian-api:jar:1.1.2:test
[INFO] |  |  \- org.jspecify:jspecify:jar:1.0.0:test
[INFO] |  +- org.junit.jupiter:junit-jupiter-params:jar:6.0.3:test
[INFO] |  \- org.junit.jupiter:junit-jupiter-engine:jar:6.0.3:test
[INFO] |     \- org.junit.platform:junit-platform-engine:jar:6.0.3:test
[INFO] +- org.mockito:mockito-junit-jupiter:jar:5.23.0:test
[INFO] |  \- org.mockito:mockito-core:jar:5.23.0:test
[INFO] |     +- net.bytebuddy:byte-buddy-agent:jar:1.18.11:test
[INFO] |     \- org.objenesis:objenesis:jar:3.3:test
[INFO] \- org.assertj:assertj-core:jar:3.27.7:test
[INFO]    \- net.bytebuddy:byte-buddy:jar:1.18.11:test
```

Other conflict tools: `<exclusions>` on a dependency removes one transitive artifact; the **Maven Enforcer** plugin can fail the build on version convergence problems (different versions of one library being requested).

The lifecycle: phases, plugins and goals



- A **phase** is a step in the lifecycle. Running a phase runs **every phase before it**: `mvn package` = validate, compile, test-compile, test, then package.
- A **plugin** is a JAR that does work; each unit of work is a **goal**, written `plugin:goal` (`compiler:compile`, `surefire:test`). The `jar` packaging binds default goals to phases:

Phase	Default goal bound (packaging `jar`)	Result
process-resources	`resources:resources`	copy `src/main/resources` into `target/classes`
compile	`compiler:compile`	`target/classes`
test-compile	`compiler:testCompile`	`target/test-classes`
test	`surefire:test`	runs unit tests; fails the build on a failure
package	`jar:jar`	`target/shop-1.0.0-SNAPSHOT.jar`
integration-test / verify	`failsafe:*` if configured	integration tests (`*IT.java`) and checks
install	`install:install`	copy the JAR into `~/.m2/repository` for other local projects
deploy	`deploy:deploy`	upload to a remote repository (company Nexus/Artifactory — self-hosted artifact repository servers — Maven Central)

Two other lifecycles exist: **clean** (`mvn clean` deletes `target/`) and **site** (project reports). Surefire runs test classes whose names match `Test*`, `*Test`, `*Tests` or `*TestCase` by default.

Everyday commands

Command	Does
`mvn clean verify`	full build from scratch with all checks — what CI usually runs
`mvn test`	compile and run unit tests
`mvn -Dtest=CartTest test`	run one test class (`CartTest#newCartIsEmpty` for one method)
`mvn package -DskipTests`	build the JAR without running tests (compiles them; `-Dname=value` sets a property)
`mvn dependency:tree`	print the resolved tree
`mvn -o …`	offline: use only the local repository
`./mvnw …`	the **Maven Wrapper**: downloads the exact Maven version the project pins (`mvn wrapper:wrapper` creates it)

A real build

The sources were copied into the standard layout in a scratch folder next to this `pom.xml` and built **offline** (`mvn -o -B package`; `-B` = batch mode, no interactive prompts or colours) from the local cache. Trimmed real output:

```
[INFO] --- dependency:3.10.0:properties (default) @ shop ---
[INFO] --- resources:3.3.1:resources (default-resources) @ shop ---
[INFO] --- compiler:3.15.0:compile (default-compile) @ shop ---
[INFO] Compiling 11 source files with javac [debug release 25] to target/classes
[INFO] --- resources:3.3.1:testResources (default-testResources) @ shop ---
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ shop ---
[INFO] Compiling 6 source files with javac [debug release 25] to target/test-classes
[INFO] --- surefire:3.5.4:test (default-test) @ shop ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] Running ch18.shop.DiscountsTest
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.105 s -- in ch18.shop.DiscountsTest
[INFO] Running ch18.shop.AssertjStyleTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.395 s -- in ch18.shop.AssertjStyleTest
[INFO] Running ch18.shop.OrderServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.166 s -- in ch18.shop.OrderServiceTest
...
[INFO] Results:
[WARNING] Tests run: 48, Failures: 0, Errors: 0, Skipped: 1
[INFO] --- jar:3.5.1:jar (default-jar) @ shop ---
[INFO] Building jar: .../java18_mvn/target/shop-1.0.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  2.645 s
```

That is the lifecycle table in action. The real run taught two lessons: the first attempt failed because Maven 3.9.14's *default* resources plugin (3.4.0) wasn't cached — **pin every plugin version** so a Maven upgrade can't silently change your build — and the Byte Buddy conflict above. `java -jar target/shop-1.0.0-SNAPSHOT.jar` runs `ShopDemo` (output in 18.12).

Multi-module builds (in brief)

Big projects split into **modules** (`shop-domain`, `shop-web`, …) under a parent pom with `<packaging>pom</packaging>` and a `<modules>` list. The parent holds shared `dependencyManagement` and plugin settings; `mvn verify` at the root builds modules in dependency order.

Common mistakes

- Omitting plugin versions (build changes when Maven is upgraded) or using version ranges.
- Test libraries without `<scope>test</scope>` — they end up in the product.
- Fighting a conflict with random exclusions instead of reading `dependency:tree` and pinning in `dependencyManagement`.
- Running `mvn install` just to build — `verify` is enough unless another local project needs the JAR.

In the interview

30-second answer: "Maven is a declarative build tool. The `pom.xml` gives coordinates, dependencies with scopes, and plugin configuration. It resolves transitive dependencies from Maven Central into `~/.m2`, choosing the nearest version on conflicts, and runs a fixed lifecycle — compile, test, package, verify, install, deploy — where running a phase runs all earlier ones. Plugins' goals are bound to phases."

- *`dependencies` vs `dependencyManagement`?* → The first adds dependencies; the second only fixes versions (and scopes) for when they are used, including transitive ones.
- *What is a BOM?* → A pom that only contains `dependencyManagement`; importing it aligns versions of a family of libraries (JUnit, Spring, Jackson).
- *`install` vs `deploy`?* → Local repository vs remote repository.

Self-check

1. What does `mvn package` run? → validate, compile, test-compile, test (and the resource phases) then package.
2. Which scope for a JDBC driver? → `runtime`: code compiles against `java.sql` only.

3. How does Maven choose between two versions of the same transitive dependency? → Nearest in the tree; first declared on a tie; `dependencyManagement` overrides both.
4. Why commit `mvnw`? → Everyone (and CI) uses the same Maven version without installing it.

TL;DR

`pom.xml` = coordinates + dependencies (with scopes) + plugins. Fixed lifecycle; a phase runs all earlier phases. Pin plugin and conflicting library versions; read `dependency:tree`.

18.3 Gradle

What it is

Gradle (2008) is a build tool whose build file is **code** — a Kotlin (`build.gradle.kts`, the default for new builds) or Groovy (`build.gradle`) script (Kotlin and Groovy are other languages that run on the JVM). Instead of a fixed lifecycle it builds a **graph of tasks** and runs only what's needed, skipping **up-to-date** tasks (**incremental builds**: only redo work whose inputs changed) and reusing outputs from a **build cache** (stored results of earlier task runs, possibly shared between machines). It uses the same Maven repositories and coordinates. Gradle 9 (the version used here is 9.4.1) needs Java 17+ to run the build itself.

The build file — `code/build-files/gradle/build.gradle.kts`

```
plugins {
    application                                  // adds 'java' + the run task
}

group = "com.example"
version = "1.0.0-SNAPSHOT"

java {
    toolchain { languageVersion = JavaLanguageVersion.of(25) }   // build with a JDK 25
}

repositories {
    mavenCentral()
}

val mockitoAgent = configurations.create("mockitoAgent")   // a separate list holding just the agent jar

dependencies {
    testImplementation(platform("org.junit:junit-bom:6.0.3"))
    testImplementation("org.junit.jupiter:junit-jupiter")
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")
    testImplementation("org.mockito:mockito-junit-jupiter:5.23.0")
    testImplementation("org.assertj:assertj-core:3.27.7")
    testImplementation("net.bytebuddy:byte-buddy:1.18.11")        // one Byte Buddy version for everybody
    testImplementation("net.bytebuddy:byte-buddy-agent:1.18.11")
    mockitoAgent("org.mockito:mockito-core:5.23.0") { isTransitive = false }
}

application { mainClass = "ch18.shop.ShopDemo" }

tasks.test {
    useJUnitPlatform { excludeTags("demo-failure") }
    jvmArgs("-javaagent:${mockitoAgent.asPath}", "-Xshare:off")
    testLogging { events("passed", "skipped", "failed") }
}

tasks.jar { manifest { attributes["Main-Class"] = "ch18.shop.ShopDemo" } }
```

`settings.gradle.kts` holds one line, `rootProject.name = "shop"` (multi-project builds list their subprojects there with `include(...)`).

- **Toolchain** — Gradle finds (or **provisions** — downloads and installs) a JDK 25 to compile and test with, independent of the JDK running Gradle.
- `platform(...)` — imports a BOM, like Maven's `import` scope.
- `testRuntimeOnly("org.junit.platform:junit-platform-launcher")` — required since Gradle 9 when running on the JUnit Platform.
- `useJUnitPlatform()` — without it Gradle looks for JUnit 4 tests and finds none.

Configurations (Gradle's scopes)

Gradle configuration	Closest Maven scope	Meaning
`implementation`	compile	needed to compile and run, **not** exposed to consumers' compile class path
`api` (plugin `java-library`)	compile	part of your public API: consumers compile against it too
`compileOnly`	provided	compile time only (annotations, provided APIs)
`runtimeOnly`	runtime	run time only (JDBC driver, logging backend)
`testImplementation` / `testRuntimeOnly`	test	the same, for tests

`implementation` vs `api` is a real improvement over Maven: hiding implementation dependencies means fewer recompilations and less accidental **coupling** (code depending on details it shouldn't know about).

A real build

The same sources, built offline with Gradle 9.4.1 (`gradle --offline clean build --console=plain`; for the offline run `mavenCentral()` was replaced by `mavenLocal()` so Gradle could read the `~/.m2` cache):

```
> Task :clean
> Task :compileJava
> Task :processResources NO-SOURCE
> Task :classes
> Task :jar
> Task :startScripts
> Task :distTar
> Task :distZip
> Task :assemble
> Task :compileTestJava
> Task :processTestResources NO-SOURCE
> Task :testClasses
> Task :test
Cart > itemsViewIsReadOnly() PASSED
Cart > shippingIsFreeAbove50() SKIPPED
Cart > adding the same SKU twice merges the quantities PASSED
...
> Task :check
> Task :build
BUILD SUCCESSFUL in 5s
8 actionable tasks: 8 executed
```

`build` depends on `assemble` (jar + distributions from the `application` plugin) and `check` (tests). `gradle run` then ran `ShopDemo`. Useful commands: `./gradlew test --tests 'ch18.shop.CartTest'`, `./gradlew dependencies --configuration testRuntimeClasspath`, `./gradlew tasks`, `./gradlew build --scan` (online build report).

The Gradle Wrapper. `gradle wrapper` generates `gradlew`, `gradlew.bat` and `gradle/wrapper/gradle-wrapper.properties` (holding `distributionUrl`). Commit them and always run `./gradlew`. The user's OCP study repo is a real example: its `build.gradle` (Groovy DSL — *domain-specific language*, a mini-language tailored to one job, here describing builds) pins a **Java 21 toolchain**, gives every chapter its own *source set* with its own compile task, and its wrapper pins Gradle 9.4.1.

Maven vs Gradle

	Maven	Gradle
Build file	XML, declarative	Kotlin/Groovy script
Model	fixed lifecycle of phases	task graph, incremental, build cache, daemon (a background process kept alive between builds so they start faster)
Speed on big builds	fine	usually faster (up-to-date checks, caching)
Flexibility	limited — write a plugin for custom logic	very high (and easy to over-engineer)
Learning curve	gentle, very uniform across projects	steeper; builds differ more
Where you'll meet it	most enterprise/back-end (server-side) Java, Spring examples	Android (mandatory), many newer projects, Kotlin

Both are good choices; follow the team. Spring Initializr (start.spring.io) generates either.

Common mistakes

- Using `gradle` from your PATH (the environment variable listing folders where the shell looks for commands) instead of `./gradlew`.
- Forgetting `useJUnitPlatform()` — "0 tests" and a green build.
- Putting everything in `api` "to be safe".
- Custom logic scattered in the build script; move it to a plugin in `buildSrc` or a convention plugin (a shared plugin that applies a team's standard build settings).

Self-check

1. `implementation` vs `api`? → `api` leaks to consumers' compile class path; `implementation` doesn't.
2. Why is a second `./gradlew build` with no changes fast? → Tasks are up-to-date; Gradle compares inputs/outputs and skips them.
3. What does a toolchain do? → Chooses the JDK for compiling/testing independently of the JDK running Gradle.

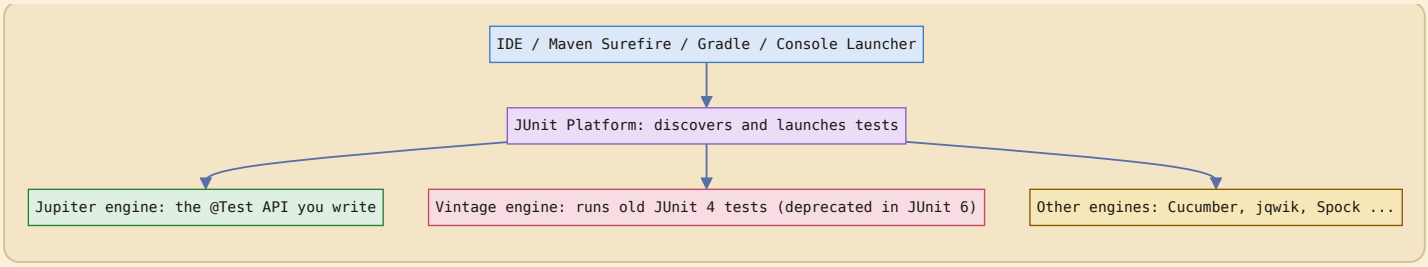
TL;DR

Gradle = scripted task graph with incremental builds; same repositories and coordinates as Maven. Use the wrapper, `implementation` by default, and remember `useJUnitPlatform()`.

18.4 JUnit: your first automated tests

What it is

An **automated test** is code that runs other code with known inputs and checks the result, reporting pass or fail without a human looking at output. A **unit test** checks one small piece (a class or method) in isolation, in milliseconds. **JUnit** is Java's standard test **framework** (a library that calls *your* code, here your test methods). Its modern generation (JUnit 5, 2017; **JUnit 6**, released 30 September 2025) has three parts:



- **JUnit Platform** — the launcher that tools talk to.
- **JUnit Jupiter** — the programming model (`org.junit.jupiter.api.*`) and its engine.
- **JUnit Vintage** — an engine for legacy JUnit 4 tests.

JUnit 6 kept the same Jupiter API (annotations and assertions in these examples are identical in JUnit 5), moved the minimum to **Java 17**, gave Platform, Jupiter and Vintage **one shared version number** (6.x instead of 1.x/5.x), added JSpecify nullness **annotations** (`@`-labels on code; these ones say whether a value may be `null`), a `--fail-fast` launcher option and test cancellation, and deprecated Vintage. Most JUnit 5 test code runs unchanged.

Why it matters

Tests let you **refactor** (restructure code without changing what it does), upgrade or fix without fear — hundreds of checks in a second. They document behaviour precisely ("SAVE25 needs a subtotal of at least 100"), catch **regressions** (things that used to work and broke) before users do, and push toward better design (hard-to-test code is usually too coupled).

The code under test

`Cart` holds `Item` records (Chapter 6 §6.8); adding the same SKU (*stock-keeping unit*, a product's id) twice merges quantities; money is `BigDecimal` (exact decimal numbers) (Chapter 7 §7.7).

```
public final class Cart {
    private final Map<String, Item> items = new LinkedHashMap<>();
    public void add(Item item) {
        items.merge(item.sku(), item,
                (old, extra) -> new Item(old.sku(), old.name(), old.unitPrice(), old.quantity() +
extra.quantity()));
    }
    public void remove(String sku) {
        if (items.remove(sku) == null) throw new NoSuchElementException("not in cart: " + sku);
    }
    public List<Item> items()   { return List.copyOf(items.values()); }
    public boolean isEmpty()    { return items.isEmpty(); }
    public int count()          { return items.values().stream().mapToInt(Item::quantity).sum(); }
    public BigDecimal subtotal() { /* sum of line totals, setScale(2, HALF_UP) */ }
}
```

The tests — `code/test/ch18/shop/CartTest.java`

```java
@Tag("fast")
@DisplayName("Cart")
class CartTest {

    private Cart cart;                                  // a NEW CartTest instance is created for every test

    @BeforeEach
    void setUp() { cart = new Cart(); }

    @Test
    void newCartIsEmpty() {
        assertTrue(cart.isEmpty());
        assertEquals(0, cart.count());
        assertEquals(new BigDecimal("0.00"), cart.subtotal());
    }

    @Test
    @DisplayName("adding the same SKU twice merges the quantities")
    void addSameSkuMerges() {
        cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 1));
        cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 2));

        assertAll("merged line",                        // reports EVERY failing check, not just the first
                () -> assertEquals(1, cart.items().size()),
                () -> assertEquals(3, cart.count()),
                () -> assertEquals(new BigDecimal("37.50"), cart.subtotal()));
    }

    @Test
    void removingUnknownSkuThrows() {
        NoSuchElementException e = assertThrows(NoSuchElementException.class, () -> cart.remove("NOPE"));
        assertEquals("not in cart: NOPE", e.getMessage());
    }

    @Test
    @Disabled("shipping costs not implemented yet")
    void shippingIsFreeAbove50() { fail("not written yet"); }

    @Nested
    @DisplayName("with a book and a mug")
    class WithTwoItems {
        @BeforeEach
        void fill() {                                   // runs AFTER the outer setUp()
            cart.add(new Item("BK-1", "Java book", new BigDecimal("45.00"), 2));
            cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 1));
        }
        @Test void subtotalAddsLineTotals() { assertEquals(new BigDecimal("102.50"), cart.subtotal()); }
        @Test void removeDropsTheWholeLine() { cart.remove("BK-1"); assertEquals(1, cart.count()); }
    }
    // ... itemsViewIsReadOnly()
}
```

How it works:

1. `@Test` marks a test method. Test classes and methods don't need to be `public` (package-private is the convention); they must not be `private`.
2. The static import `org.junit.jupiter.api.Assertions.*` gives the `assertXxx` methods (an **assertion** is a check that fails the test if it isn't true). Argument order is **(expected, actual)**, with an optional message **last**.
3. A failed assertion throws `AssertionFailedError`; any other uncaught exception also fails the test. A test that finishes normally passes.
4. `assertThrows` runs the lambda, checks the exception type and **returns** the exception so you can inspect it.
5. `@Nested` inner classes group tests sharing a situation; the outer `@BeforeEach` runs first.

The assertions you'll use

Assertion	Checks
`assertEquals(exp, act)` / `assertNotEquals`	`equals` (with a `delta` overload for doubles)
`assertTrue` / `assertFalse`	a condition
`assertNull` / `assertNotNull`	null-ness
`assertSame` / `assertNotSame`	same reference (`==`)
`assertArrayEquals`, `assertIterableEquals`	element by element
`assertLinesMatch`	lists of strings, with regex (regular-expression pattern) lines allowed
`assertInstanceOf(Type.class, obj)`	type, returns the cast value
`assertThrows` / `assertThrowsExactly`	an exception of that type (or subtype / exactly that type)
`assertDoesNotThrow`	no exception; returns the result
`assertAll(...)`	runs every check, reports all failures together
`assertTimeout(Duration, ...)`	finishes in time (`assertTimeoutPreemptively` aborts it)
`fail("msg")`	fails immediately

Messages can be a `Supplier<String>` (`() -> "got " + r`), built only on failure.

Lifecycle — `LifecycleTest.java`

```
class LifecycleTest {
    static int instances;
    LifecycleTest()            { System.out.println("  constructor  (instance #" + ++instances + ")"); }

    @BeforeAll static void beforeAll() { System.out.println("@BeforeAll   (static, once)"); }
    @BeforeEach void beforeEach()      { System.out.println("  @BeforeEach"); }
    @Test void first()                 { System.out.println("    test: first"); }
    @Test void second()                { System.out.println("    test: second"); }
    @AfterEach void afterEach()        { System.out.println("  @AfterEach"); }
    @AfterAll static void afterAll()   { System.out.println("@AfterAll    (static, once)"); }
}
```

Output (from the run below):

```
@BeforeAll   (static, once)
  constructor  (instance #1)
  @BeforeEach
    test: second
  @AfterEach
  constructor  (instance #2)
  @BeforeEach
    test: first
  @AfterEach
@AfterAll    (static, once)
```

Three facts to remember:

- **A fresh instance per test method** (default `@TestInstance(Lifecycle.PER_METHOD)`), so fields can't leak state between tests. That is also why `@BeforeAll`/`@AfterAll` must be `static` — unless you annotate the class `@TestInstance(Lifecycle.PER_CLASS)`.
- **Method order is deterministic but deliberately not obvious** — here `second` ran before `first`. Tests must not depend on each other. If you truly need an order: `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` + `@Order(n)`.
- `@AfterEach` runs even when the test fails — use it for cleanup.

More annotations

Annotation	Purpose
`@DisplayName("…")`	readable name in reports
`@Disabled("reason")`	skip, with a reason shown in the report (`↷` below)
`@Tag("fast")`	label for filtering (`--include-tag`, Maven `<groups>`, Gradle `includeTags`)
`@Timeout(2)`	fail if slower than 2 s
`@TempDir Path dir`	a fresh temporary directory, deleted afterwards (18.7)
`@ExtendWith(X.class)`	plug in an **extension** (a plug-in that hooks into the test lifecycle, e.g. Mockito, Spring) — replaces JUnit 4 runners and rules
`@EnabledOnOs`, `@EnabledIfEnvironmentVariable`, `@EnabledForJreRange` …	conditional execution

Assumptions (`assumeTrue(cond)`, `assumingThat`) stop a test as *aborted* rather than failed when a precondition doesn't hold (e.g. "only on CI").

Running the tests from the command line

IDEs (Integrated Development Environments such as IntelliJ IDEA) and build tools run JUnit for you. Without them, the **Console Launcher** (`junit-platform-console-standalone`, one JAR containing the Platform, Jupiter, Vintage and Suite engines) does it. The plain form — compile, then point the launcher at the classes:

```
$ javac --release 25 -cp '.java-books/lib/*' -d /tmp/java18t $(find ch18/shop test -name '*.java')
$ java -jar junit-platform-console-standalone-6.1.3.jar execute \
      --class-path /tmp/java18t:assertj-core-3.27.7.jar --scan-class-path \
      --include-classname '.*(Cart|Discounts)Test' --details=summary --disable-banner

Test run finished after 101 ms
[        13 containers found      ]
[         0 containers skipped    ]
[        13 containers started    ]
[         0 containers aborted    ]
[        13 containers successful ]
[         0 containers failed     ]
[        34 tests found           ]
[         1 tests skipped         ]
[        33 tests started         ]
[         0 tests aborted         ]
[        33 tests successful      ]
[         0 tests failed          ]
```

`--scan-class-path` discovers every class whose name matches the default pattern (`Test*`, `*Test`, `*Tests`). The **exit code** (the number a program returns to the shell when it ends; 0 means success) is 0 only if nothing failed — that's what CI checks.

For the Mockito tests the launcher must be started differently, because Mockito wants to be attached as a **Java agent** (a JAR that the JVM loads before your code so it can modify classes as they load; 18.6) and the agent must be the *same* JAR the tests load. So `code/run_tests.sh` puts everything on one class path and starts the launcher by its main class:

```
LIB="../../../../.java-books/lib"; CP="$LIB/*"; OUT=$(mktemp -d)
javac --release 25 -cp "$CP" -d "$OUT" $(find ch18/shop test -name '*.java') || exit 1
if [ "${1:-}" = fail ]; then TAGS=(--include-tag demo-failure); else TAGS=(--exclude-tag demo-failure); fi
java -Xshare:off -javaagent:"$(ls "$LIB"/mockito-core-*.jar)" \
     -Djava.util.logging.SimpleFormatter.format='%4$s: %5$s%n' \
     -cp "$OUT:$CP" org.junit.platform.console.ConsoleLauncher execute \
     --scan-class-path "$OUT" "${TAGS[@]}" \
     --disable-banner --disable-ansi-colors --details=tree
```

`bash code/run_tests.sh` output (tree trimmed; notes in parentheses added):

```
WARNING: declined: customer=bob total=102.50
INFO: order placed: id=ORD-1 total=102.50 ref=PAY-9
INFO: order placed: id=ORD-1 total=76.88 ref=PAY-9
INFO: order placed: id=ORD-7 total=92.25 ref=ok
@BeforeAll   (static, once)
  ... (lifecycle lines shown above)
╷
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  ├─ AssertjStyleTest ✔          (6 tests, 18.7)
│  ├─ DiscountsTest ✔             (27 tests, 18.5)
│  ├─ Cart ✔
│  │  ├─ itemsViewIsReadOnly() ✔
│  │  ├─ shippingIsFreeAbove50() ↷ shipping costs not implemented yet
│  │  ├─ adding the same SKU twice merges the quantities ✔
│  │  ├─ newCartIsEmpty() ✔
│  │  ├─ removingUnknownSkuThrows() ✔
│  │  └─ with a book and a mug ✔
│  │     ├─ removeDropsTheWholeLine() ✔
│  │     └─ subtotalAddsLineTotals() ✔
│  ├─ OrderServiceTest ✔          (6 tests, 18.6)
│  └─ LifecycleTest ✔
│     ├─ second() ✔
│     └─ first() ✔
└─ JUnit Vintage ✔

Test run finished after 566 ms
...
[        48 tests found           ]
[         1 tests skipped         ]
[        47 tests started         ]
[        47 tests successful      ]
[         0 tests failed          ]
```

The `INFO:` / `WARNING:` lines at the top are the application's own logging (18.10) printed while `OrderServiceTest` ran.

Coming from JUnit 4

JUnit 4	Jupiter (JUnit 5/6)
`org.junit.Test`, public classes/methods	`org.junit.jupiter.api.Test`, package-private OK
`@Before` / `@After`	`@BeforeEach` / `@AfterEach`
`@BeforeClass` / `@AfterClass`	`@BeforeAll` / `@AfterAll`
`@Ignore`	`@Disabled`
`@Test(expected = X.class)`	`assertThrows(X.class, ...)`
`@RunWith`, `@Rule`	`@ExtendWith` extensions
`@Category`	`@Tag`
`assertEquals("msg", exp, act)` — message first	`assertEquals(exp, act, "msg")` — message last

Common mistakes

- Swapping expected and actual — failure messages then read backwards.
- Forgetting `@Test` (the method never runs, silently).
- Mixing JUnit 4's `org.junit.Test` with Jupiter assertions — the Jupiter engine ignores JUnit 4 tests.
- Tests that depend on each other's order or on shared static state.
- Printing results instead of asserting them — a test that can't fail is not a test.

In the interview

> **30-second answer:** "JUnit 5 and 6 split into the Platform, which launches tests for IDEs and build tools, and Jupiter, the API: `@Test`, `@BeforeEach` / `@AfterEach`, static `@BeforeAll` / `@AfterAll`, assertions like `assertEquals`, `assertThrows` and `assertAll`, plus `@Nested`, `@Tag`, `@DisplayName`, `@Disabled` and extensions via `@ExtendWith`. A new test instance is created per method so tests stay independent."

- *Why is `@BeforeAll` static?* → One instance per test method exists, so a once-per-class hook can't belong to an instance — unless `PER_CLASS` lifecycle is used.
- *`assertAll` vs several asserts?* → Several asserts stop at the first failure; `assertAll` reports all of them.
- *What changed in JUnit 6?* → Java 17 baseline, a single version for all modules, JSpecify annotations, Vintage deprecated; the Jupiter API is essentially the same.

Self-check

1. What does `assertThrows` return? → The thrown exception, for further checks.
2. How many `LifecycleTest` instances were created for 2 tests? → Two.
3. How do you run only tests tagged `fast`? → `--include-tag fast` (console), `<groups>fast</groups>` (Surefire), `useJUnitPlatform { includeTags("fast") }` (Gradle).
4. Aborted vs failed? → Aborted = an assumption didn't hold; failed = an assertion or unexpected exception.

TL;DR

`@Test` + `Assertions` + lifecycle hooks, one fresh instance per test, no reliance on order. Run from IDE, build tool, or the Console Launcher.

18.5 Parameterized, repeated and dynamic tests

The problem it solves

Rules like "a code is 4+ capital letters followed by 2 digits" need many inputs. Copy-pasting a test per input is noisy; a loop inside one test stops at the first failure and reports one anonymous result. **Parameterized tests** run the same method once per input, each reported separately. (They live in `junit-jupiter-params`, included in the `junit-jupiter` aggregate artifact — one dependency that pulls in several others.)

The rules under test — `Discounts.java`

```
public static boolean isValidCode(String code) {
    return code != null && code.matches("[A-Z]{4,}\\d{2}");
}
public static int percentFor(String code, BigDecimal subtotal) {
    if (!isValidCode(code)) return 0;
    return switch (code) {
        case "SAVE10" -> 10;
        case "SAVE25" -> subtotal.compareTo(BIG_ORDER) >= 0 ? 25 : 0;   // BIG_ORDER = 100.00
        default -> 0;
    };
}
public static BigDecimal apply(BigDecimal amount, int percent) {
    if (percent < 0 || percent > 100) throw new IllegalArgumentException("percent out of range: " + percent);
    return amount.multiply(BigDecimal.valueOf(100 - percent)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
}
```

The tests — `DiscountsTest.java`

```
@ParameterizedTest(name = "{0} is a valid code")
@ValueSource(strings = {"SAVE10", "SAVE25", "WELCOME99"})
void validCodes(String code) { assertTrue(Discounts.isValidCode(code)); }

@ParameterizedTest(name = "[{index}] {0} is rejected")
@NullAndEmptySource                                  // adds null and ""
@ValueSource(strings = {"save10", "SAV10", "SAVE1", "SAVE100"})
void invalidCodes(String code) { assertFalse(Discounts.isValidCode(code)); }

@ParameterizedTest(name = "{0} on {1} -> {2}%")
@CsvSource({
        "SAVE10,  20.00, 10",
        "SAVE25,  99.99,  0",     // boundary: just below 100
        "SAVE25, 100.00, 25",     // boundary: exactly 100
        "SAVE99, 500.00,  0",     // well-formed but unknown
})
void percentForCode(String code, BigDecimal subtotal, int expected) {   // strings converted to BigDecimal/int
    assertEquals(expected, Discounts.percentFor(code, subtotal));
}

static Stream<Arguments> roundingCases() {
    return Stream.of(
            Arguments.of("10.00", 10, "9.00"),
            Arguments.of("0.05", 50, "0.03"),              // 0.025 rounds half-up to 0.03
            Arguments.of("102.50", 25, "76.88"),           // 76.875 -> 76.88
            Arguments.of("19.99", 100, "0.00"));
}

@ParameterizedTest
@MethodSource("roundingCases")
void applyRoundsHalfUpToCents(String amount, int percent, String expected) {
    assertEquals(new BigDecimal(expected), Discounts.apply(new BigDecimal(amount), percent));
}

@RepeatedTest(3)
void applyIsDeterministic() { assertEquals(new BigDecimal("9.00"), Discounts.apply(BigDecimal.TEN, 10)); }

@TestFactory
Stream<DynamicTest> everyPercentKeepsTheResultWithinRange() {
    return Stream.of(0, 33, 50, 99, 100).map(p -> dynamicTest("percent " + p, () -> {
        BigDecimal r = Discounts.apply(new BigDecimal("80.00"), p);
        assertTrue(r.signum() >= 0 && r.compareTo(new BigDecimal("80.00")) <= 0, () -> "got " + r);
    }));
}
```

Output (part of the same run; some rows collapsed, notes in parentheses added):

```
|  ├─ DiscountsTest ✔
|  |  ├─ validCodes(String) ✔
|  |  |  ├─ "SAVE10" is a valid code ✔
|  |  |  ├─ "SAVE25" is a valid code ✔
|  |  |  └─ "WELCOME99" is a valid code ✔
|  |  ├─ applyRoundsHalfUpToCents(String, int, String) ✔   (4 rows)
|  |  ├─ invalidCodes(String) ✔
|  |  |  ├─ [1] null is rejected ✔
|  |  |  ├─ [2] "" is rejected ✔
|  |  |  ├─ [3] "save10" is rejected ✔
|  |  |  ├─ [4] "SAV10" is rejected ✔
|  |  |  ├─ [5] "SAVE1" is rejected ✔
|  |  |  └─ [6] "SAVE100" is rejected ✔
|  |  ├─ applyRejectsBadPercent(int) ✔                      (2 rows)
|  |  ├─ percentForCode(String, BigDecimal, int) ✔
|  |  |  ├─ "SAVE10" on "20.00" -> "10"% ✔
|  |  |  ├─ "SAVE25" on "99.99" -> "0"% ✔
|  |  |  ├─ "SAVE25" on "100.00" -> "25"% ✔
|  |  |  └─ "SAVE99" on "500.00" -> "0"% ✔
|  |  ├─ everyPercentKeepsTheResultWithinRange() ✔
|  |  |  ├─ percent 0 ✔
|  |  |  ├─ ... percent 33, 50, 99 ✔
|  |  |  └─ percent 100 ✔
|  |  └─ applyIsDeterministic() ✔
|  |     ├─ repetition 1 of 3 ✔
|  |     ├─ repetition 2 of 3 ✔
|  |     └─ repetition 3 of 3 ✔
```

Notice: the name pattern placeholders are `{0}`, `{1}` … and `{index}`; this JUnit version shows text arguments **in quotes**, so `null` (no quotes) and `""` (the empty string) are easy to tell apart. `@CsvSource` values start as text and are **converted** to the parameter types (`BigDecimal`, `int`) automatically.

Argument sources

Source	Supplies	Example use
`@ValueSource(strings/ints/…)`	one literal argument per run	valid/invalid inputs
`@NullSource`, `@EmptySource`, `@NullAndEmptySource`	`null`, empty string/collection/array	edge cases (unusual inputs at the extremes)
`@EnumSource(Level.class)`	each enum constant (filterable with `names`)	behaviour per enum value
`@CsvSource({...})` / `@CsvFileSource(resources = "/x.csv")`	rows of several arguments	input → expected tables
`@MethodSource("factory")`	a `static` method returning `Stream<Arguments>` (or a collection)	objects that can't be written as text
`@FieldSource` (5.11+)	a static field holding the arguments	shared fixed data
`@ArgumentsSource(MyProvider.class)`	your own provider	reusable generators

Related features: `@RepeatedTest(n)` runs a test n times (useful for suspected flakiness); `@TestFactory` returns **dynamic tests** created at run time — the number of tests can depend on data. Since JUnit 5.13 a whole class can be parameterized with `@ParameterizedClass`.

Common mistakes

- Many inputs in one `@Test` with a loop — the first failure hides the rest.
- Hiding the *expected* value computation inside the test (re-implementing the logic). Expected values should be literal, worked out by hand — like `76.88` above.
- Forgetting boundaries: the `99.99` / `100.00` rows are the most valuable ones in that table.

Self-check

1. Where must a `@MethodSource` factory live? → A `static` method in the test class (or a fully qualified `Class#method` elsewhere), unless the class uses `PER_CLASS` lifecycle.
2. How do you include `null` as an input? → `@NullSource` or `@NullAndEmptySource`.
3. Parameterized vs dynamic tests? → Parameterized: fixed method, data from a source. Dynamic: the factory builds arbitrary tests at run time.

TL;DR

One method, many inputs, each reported separately: `@ParameterizedTest` + a source. Put boundary values in the table.

18.6 Test doubles and Mockito

The problem it solves

`OrderService.checkout` charges a card and saves an order. A unit test must not charge real cards or need a real database — it would be slow, flaky and expensive. So the service is written to **receive its collaborators** (the other objects it relies on) through the constructor (**dependency injection**: an object is handed its dependencies instead of creating them itself; the *Dependency Inversion Principle*, see LLD §4.10), and tests pass in stand-ins called **test doubles**.

```
public final class OrderService {
    private static final Logger LOG = Logger.getLogger(OrderService.class.getName());
    private final PaymentGateway gateway;          // interface: the card processor
    private final OrderRepository repository;      // interface: storage
    private final Clock clock;                     // time is a dependency too
    private final Supplier<String> ids;            // so is id generation

    public OrderService(PaymentGateway gateway, OrderRepository repository, Clock clock, Supplier<String> ids) {
... }

    public Order checkout(String customer, Cart cart, String discountCode) {
        if (cart.isEmpty()) throw new IllegalStateException("cart is empty");
        BigDecimal subtotal = cart.subtotal();
        int percent = Discounts.percentFor(discountCode, subtotal);
        BigDecimal total = Discounts.apply(subtotal, percent);

        PaymentGateway.PaymentResult result = gateway.charge(customer, total);
        if (!result.approved()) {
            LOG.warning(() -> "declined: customer=" + customer + " total=" + total);
            throw new PaymentDeclinedException(customer, total);
        }
        Order order = new Order(ids.get(), customer, total, clock.instant());
        repository.save(order);
        LOG.info(() -> "order placed: id=" + order.id() + " total=" + total + " ref=" + result.reference());
        return order;
    }
}
```

Injecting a `java.time.Clock` (Chapter 7 §7.10) and an id `Supplier` makes the result fully predictable: `Clock.fixed(...)` always returns the same instant.

The five kinds of test double

Double	What it is	In this chapter
Dummy	passed only to fill a parameter, never used	—
Stub	returns canned (pre-set) answers	the lambda `(c, amt) -> new PaymentResult(true, "ok")`
Fake	a real, working but simplified implementation	`InMemoryOrderRepository` (a `LinkedHashMap`)
Spy	a real object that also records calls	`spy(new InMemoryOrderRepository())`
Mock	programmed with expectations; you **verify** how it was called	`@Mock PaymentGateway gateway`

Rule of thumb: check **state** (the result, the fake's contents) when you can; **verify interactions** only when the interaction *is* the behaviour — "the card was charged the discounted amount", "nothing was saved after a decline". A longer discussion of doubles and test strategy is in LLD — Testing strategies.

Mockito in practice — `OrderServiceTest.java`

```java
@ExtendWith(MockitoExtension.class)                    // creates @Mock fields, checks for unused stubs
class OrderServiceTest {
    static final Instant NOW = Instant.parse("2025-09-16T10:00:00Z");

    @Mock PaymentGateway gateway;
    @Mock OrderRepository repository;
    @Captor ArgumentCaptor<Order> saved;
    OrderService service;
    Cart cart;

    @BeforeEach
    void setUp() {
        service = new OrderService(gateway, repository, Clock.fixed(NOW, ZoneOffset.UTC), () -> "ORD-1");
        cart = new Cart();
        cart.add(new Item("BK-1", "Java book", new BigDecimal("45.00"), 2));
        cart.add(new Item("MUG", "Mug", new BigDecimal("12.50"), 1));    // subtotal 102.50
    }

    @Test
    void approvedPaymentSavesTheOrder() {
        when(gateway.charge("ann", new BigDecimal("76.88"))).thenReturn(new PaymentResult(true, "PAY-9"));  // arrange

        Order order = service.checkout("ann", cart, "SAVE25");                                              // act

        assertEquals(new Order("ORD-1", "ann", new BigDecimal("76.88"), NOW), order);                       // assert
        verify(repository).save(saved.capture());         // called exactly once, capture the argument
        assertEquals("ORD-1", saved.getValue().id());
    }

    @Test
    void chargesBeforeSaving() {
        when(gateway.charge(anyString(), any())).thenReturn(new PaymentResult(true, "PAY-9"));
        service.checkout("ann", cart, null);
        InOrder inOrder = inOrder(gateway, repository);
        inOrder.verify(gateway).charge(eq("ann"), eq(new BigDecimal("102.50")));  // mixing matchers: all must be matchers
        inOrder.verify(repository).save(any(Order.class));
    }

    @Test
    void declinedPaymentThrowsAndSavesNothing() {
        when(gateway.charge(anyString(), any())).thenReturn(new PaymentResult(false, null));
        var e = assertThrows(PaymentDeclinedException.class, () -> service.checkout("bob", cart, null));
        assertEquals("payment of 102.50 declined for bob", e.getMessage());
        verifyNoInteractions(repository);
    }

    @Test
    void gatewayOutagePropagates() {
        when(gateway.charge(anyString(), any())).thenThrow(new IllegalStateException("gateway timeout"));
        assertThrows(IllegalStateException.class, () -> service.checkout("ann", cart, null));
        verify(repository, never()).save(any());
    }

    @Test
    void fakeRepositoryPlusSpy() {
        InMemoryOrderRepository fake = spy(new InMemoryOrderRepository());
        var svc = new OrderService((c, amt) -> new PaymentResult(true, "ok"),   // a stub written as a lambda
                fake, Clock.fixed(NOW, ZoneOffset.UTC), () -> "ORD-7");
        svc.checkout("cy", cart, "SAVE10");
        assertEquals(new BigDecimal("92.25"), fake.findById("ORD-7").orElseThrow().total());
        verify(fake, times(1)).save(any());
    }
    // ... emptyCartNeverReachesTheGateway()
}
```

All six passed (tree in 18.4). How Mockito works, step by step:

1. `@Mock` (or `mock(PaymentGateway.class)`) creates a generated subclass/**instrumented** class (one whose bytecode was rewritten at run time) whose methods record each call and return **defaults**: `null`, `0`, `false`, empty collections, `Optional.empty()`.
2. `when(mock.method(args)).thenReturn(x)` **stubs** a call: when those arguments arrive, return `x`. Also `thenThrow(...)`, `thenAnswer(inv -> ...)`, and several values in a row (`thenReturn(a, b)`).
3. The code under test runs.
4. `verify(mock).method(args)` checks a call happened (exactly once by default). Variants: `times(n)`, `never()`, `atLeastOnce()`, `atMost(n)`, `verifyNoInteractions(...)`, `verifyNoMoreInteractions(...)`; `inOrder(...)` checks ordering.
5. **Argument matchers** — `any()`, `anyString()`, `eq(x)`, `argThat(pred)` — loosen matching. Rule: if one argument uses a matcher, *all* must (hence `eq("ann")` above).
6. An `ArgumentCaptor` grabs what was passed so you can assert on it afterwards.

More Mockito you'll meet

Need	API
Stub a `void` method to throw	`doThrow(new X()).when(mock).voidMethod()`
Stub a spy without calling the real method	`doReturn(x).when(spy).method()`
Given/when/then wording (BDD — *behaviour-driven development* — style)	`BDDMockito.given(...).willReturn(...)`, `then(mock).should().method()`
Mock a `static` method	`try (MockedStatic<Util> m = mockStatic(Util.class)) { m.when(Util::now).thenReturn(...); }`
Allow an unused stub	`lenient().when(...)` or `@MockitoSettings(strictness = LENIENT)`
Mock without the extension	`mock(X.class)` or `MockitoAnnotations.openMocks(this)`

Mockito 5 notes. Mockito 5 needs Java 11+, and its default *inline* mock maker instruments classes at run time, so it can mock `final` classes, records and static methods. It does that through a Java agent. On JDK 21+ loading an agent *dynamically* (into an already-running JVM) prints a warning (JEP 451 — a JEP, *JDK Enhancement Proposal*, is the numbered design document for a Java change), and a future JDK will block it by default — without `-javaagent` the first run of this chapter printed:

```
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases
of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: ...
WARNING: A Java agent has been loaded dynamically (.../byte-buddy-agent-1.18.14.jar)
```

Hence `-javaagent:mockito-core.jar` in `run_tests.sh`, Surefire's `argLine`, and Gradle's `jvmArgs`. (`-Xshare:off` only silences a harmless class-data-sharing notice caused by the agent — CDS is a JVM start-up speed-up, Chapter 16 §16.2.)

When not to mock

- **Value objects** (small objects defined only by their data, with no identity or behaviour worth faking) **and records** (`Item`, `Order`, `BigDecimal`) — just create real ones.
- **Types you don't own** (e.g. a JDBC `ResultSet`, an HTTP client): wrap them behind your own interface and mock that, or use an **integration test** (a test that runs your code against real infrastructure such as a database, §18.9).
- **Everything** — tests that mock every collaborator end up restating the implementation ("calls A then B then C") and break on every refactor while catching few bugs. Prefer fakes for storage-like collaborators.

Common mistakes

- Stubbing with the wrong arguments — the stub never matches and the mock returns `null`. **Strict stubs** (the `MockitoExtension` default) turn that into an `UnnecessaryStubbingException`, as 18.8 shows.

- Verifying calls that aren't part of the behaviour (logging, getters).
- `when(spy.method())` on a spy calls the real method during stubbing — use `doReturn`.
- Mixing raw values and matchers in one call.

In the interview

> **30-second answer:** "A test double replaces a real collaborator: dummy, stub, fake, spy or mock. Mockito creates mocks, stubs them with `when(...).thenReturn(...)`, and verifies interactions with `verify`, argument matchers and captors. I inject collaborators through the constructor so tests can swap them, use fakes for things like repositories, and mock only true boundaries like payment gateways — checking state first and interactions only when they are the behaviour."

- *Mock vs stub?* → A stub provides answers; a mock is also verified for how it was called.
- *Mock vs spy?* → A mock is a hollow object with defaults; a spy wraps a real object and calls real methods unless stubbed.
- *How do you test code that uses `LocalDateTime.now()`?* → Inject a `Clock`, pass `Clock.fixed(...)` in tests.

Self-check

1. What does an unstubbed `List<String> find()` on a mock return? → An empty list.
2. You write `verify(gateway).charge("ann", any())`. What happens? → Mockito throws `InvalidUseOfMatchersException`: once one argument is a matcher, all must be — write `eq("ann")`.
3. What does `verifyNoInteractions(repository)` prove in the decline test? → No order was saved (or even looked up) after a decline.
4. Which double is `InMemoryOrderRepository`? → A fake.

TL;DR

Design for injection, then choose the lightest double: real object → fake → stub → mock. Mockito: `@Mock`, `when/thenReturn`, `verify`, matchers, captors; run it with `-javaagent`.

18.7 AssertJ fluent assertions

What it is

AssertJ is an assertion library: one entry point, `assertThat(actual)`, returns a type-specific object whose methods you chain — your IDE's auto-complete shows only assertions that make sense for that type. Failure messages are detailed (you'll see one in 18.8). It works with any test framework; Spring Boot's test **starter** (a single dependency bundling a set of related libraries) includes it. (Hamcrest — `assertThat(x, is(y))` — is the older matcher-style alternative.)

Code — `AssertjStyleTest.java`

```java
import static org.assertj.core.api.Assertions.*;

@Test
void collections() {
    assertThat(sampleCart().items())
            .hasSize(2)
            .extracting(Item::sku)
            .containsExactly("BK-1", "MUG");
}

@Test
void bigDecimalsCompareByValue() {
    assertThat(sampleCart().subtotal())
            .isEqualByComparingTo("102.5")              // ignores scale: 102.5 == 102.50
            .isGreaterThan(BigDecimal.ZERO);
}

@Test
void exceptions() {
    assertThatThrownBy(() -> new Item("X", "x", BigDecimal.ONE, 0))
            .isInstanceOf(IllegalArgumentException.class)
            .hasMessage("quantity must be positive: 0");
    assertThatIllegalArgumentException().isThrownBy(() -> Discounts.apply(BigDecimal.ONE, 200));
}

@Test
void recursiveComparisonIgnoringAField() {
    Order actual = new Order("ORD-42", "ann", new BigDecimal("9.99"), Instant.now());
    Order expected = new Order("ignored", "ann", new BigDecimal("9.99"), Instant.EPOCH);
    assertThat(actual).usingRecursiveComparison().ignoringFields("id", "placedAt").isEqualTo(expected);
}

@Test
void softAssertionsCollectAllFailures() {
    Cart cart = sampleCart();
    SoftAssertions.assertSoftly(soft -> {
        soft.assertThat(cart.count()).isEqualTo(3);
        soft.assertThat(cart.isEmpty()).isFalse();
        soft.assertThat(cart.items()).allSatisfy(i -> assertThat(i.quantity()).isPositive());
    });
}

@Test
void writesCsvIntoATemporaryDirectory(@TempDir Path dir) throws IOException {   // deleted after the test
    Order o = new Order("ORD-1", "ann", new BigDecimal("76.88"), Instant.parse("2025-09-16T10:00:00Z"));
    Path csv = OrderExporter.writeCsv(List.of(o), dir);
    assertThat(csv).exists().hasFileName("orders.csv");
    assertThat(Files.readAllLines(csv)).containsExactly(
            "id,customer,total,placed_at",
            "ORD-1,ann,76.88,2025-09-16T10:00:00Z");
}
```

All six passed (tree in 18.4). The last test uses JUnit's `@TempDir`: a real, empty directory per test, deleted afterwards — the right way to test file code (Chapter 13) without touching your disk.

JUnit assertions vs AssertJ

Check	JUnit Assertions	AssertJ
equality	`assertEquals(3, n)`	`assertThat(n).isEqualTo(3)`
list order and content	`assertEquals(List.of(...), list)`	`assertThat(list).containsExactly(...)` / `containsExactlyInAnyOrder` / `contains`
property of each element	loop	`extracting(Item::sku)`, `filteredOn(...)`, `allSatisfy(...)`
`BigDecimal` by value	`assertEquals(0, a.compareTo(b))`	`isEqualByComparingTo("102.5")`
exception and message	`assertThrows` then `assertEquals(msg, e.getMessage())`	`assertThatThrownBy(...).isInstanceOf(...).hasMessage(...)`
many checks, one report	`assertAll(...)`	`SoftAssertions.assertSoftly(...)`
object graph field by field	manual	`usingRecursiveComparison()`
strings	`assertTrue(s.startsWith("x"))`	`assertThat(s).startsWith("x").contains("y")`
files	manual	`assertThat(path).exists().hasContent(...)`
description	message argument	`.as("subtotal of %s", cart)` before the assertion

Common mistakes

- `assertThat(x);` with nothing chained — asserts nothing. (**Static analysis** tools — programs that inspect source code for bugs without running it — and AssertJ's own `@CheckReturnValue` catch this.)
- Calling `.as("…")` **after** the assertion — too late; it must come first.
- `isEqualTo` on `BigDecimal` when scale doesn't matter — use `isEqualByComparingTo`.

TL;DR

`assertThat(actual).<fluent checks>` — readable, type-aware, great messages. Especially strong for collections, exceptions, `BigDecimal`, and object graphs.

18.8 Reading a failing test report

A test suite is only useful if its failures are quick to understand. `FailingDemoTest` fails **on purpose** — it carries `@Tag("demo-failure")`, which every normal run excludes. `bash code/run_tests.sh fail` runs only it:

```java
@Tag("demo-failure")
@ExtendWith(MockitoExtension.class)
class FailingDemoTest {
    @Mock PaymentGateway gateway;

    @Test
    void bigDecimalEqualsChecksScale() {
        // BigDecimal.equals compares value AND scale: 102.5 is not equal to 102.50
        assertEquals(new BigDecimal("102.5"), AssertjStyleTest.sampleCart().subtotal());
    }

    @Test
    void assertjListDiff() {
        List<String> skus = AssertjStyleTest.sampleCart().items().stream().map(Item::sku).toList();
        Assertions.assertThat(skus).containsExactly("MUG", "BK-1", "PEN");
    }

    @Test
    void groupedAssertionsReportEveryFailure() {
        Cart cart = AssertjStyleTest.sampleCart();
        assertAll("cart",
                () -> assertEquals(2, cart.count(), "count"),
                () -> assertEquals(2, cart.items().size(), "lines"),
                () -> assertEquals("100.00", cart.subtotal().toPlainString(), "subtotal"));
    }

    @Test
    void unusedStubIsReported() {
        // Strict stubs: a stubbing the code under test never uses is a smell (wrong argument? dead setup?)
        Mockito.when(gateway.charge("ann", BigDecimal.TEN)).thenReturn(new PaymentGateway.PaymentResult(true,
"x"));
        Assertions.assertThat(AssertjStyleTest.sampleCart().count()).isEqualTo(3);
    }
}
```

Real output (exit code 1; stack frames trimmed where marked):

```
+
├─ JUnit Platform Suite ✔
├─ JUnit Jupiter ✔
│  └─ FailingDemoTest ✔
│     ├─ bigDecimalEqualsChecksScale() ✘ expected: <102.5> but was: <102.50>
│     ├─ groupedAssertionsReportEveryFailure() ✘ cart (2 failures)
│     │         org.opentest4j.AssertionFailedError: count ==> expected: <2> but was: <3>
│     │         org.opentest4j.AssertionFailedError: subtotal ==> expected: <100.00> but was: <102.50>
│     ├─ unusedStubIsReported() ✘
│     │        Unnecessary stubbings detected.
│     │        Clean & maintainable test code requires zero unnecessary code.
│     │        Following stubbings are unnecessary (click to navigate to relevant line of code):
│     │          1. -> at ch18.shop.FailingDemoTest.unusedStubIsReported(FailingDemoTest.java:47)
│     │        Please remove unnecessary stubbings or use 'lenient' strictness. More info: javadoc for
UnnecessaryStubbingException class.
│     └─ assertjListDiff() ✘
│              Expecting actual:
│                ["BK-1", "MUG"]
│              to contain exactly (and in same order):
│                ["MUG", "BK-1", "PEN"]
│              but could not find the following elements:
│                ["PEN"]
└─ JUnit Vintage ✔

Failures (4):
  JUnit Jupiter:FailingDemoTest:bigDecimalEqualsChecksScale()
    MethodSource [className = 'ch18.shop.FailingDemoTest', methodName = 'bigDecimalEqualsChecksScale',
methodParameterTypes = '']
    => org.opentest4j.AssertionFailedError: expected: <102.5> but was: <102.50>
       org.junit.jupiter.api.Assertions.assertEquals(Assertions.java:1199)
       ch18.shop.FailingDemoTest.bigDecimalEqualsChecksScale(FailingDemoTest.java:26)
  ... (the other three repeat the messages above, each with its stack frames)

Test run finished after 67 ms
...
[         4 tests found           ]
[         0 tests skipped         ]
[         4 tests started         ]
[         0 tests aborted         ]
[         0 tests successful      ]
[         4 tests failed          ]
```

How to read each one:

1. **`expected: <102.5> but was: <102.50>`** — looks equal, isn't: `BigDecimal.equals` compares value *and scale* (Chapter 7 §7.7). The last frame from *your* code (`FailingDemoTest.java:26`) is the line to open.
2. **`cart (2 failures)`** — `assertAll` ran all three checks and reports the two that failed (`count`, `subtotal`), each with its message label; the `lines` check passed.
3. **`Unnecessary stubbings`** — the test stubbed `charge("ann", 10)` but never called it. In real tests this usually means the stub's arguments don't match what the code passes, so the code got `null` instead. Mockito tells you the exact line.
4. **`AssertJ's diff`** prints actual, expected and exactly what's missing (and, when relevant, what's unexpected or out of order).

Build tools report the same information: Surefire writes XML/text reports under `target/surefire-reports/`, Gradle an HTML report at `build/reports/tests/test/index.html`; CI servers read the XML.

TL;DR

Read the first line (expected vs actual), then the first stack frame in your own code. Use `assertAll`/soft assertions and AssertJ to get complete, precise messages.

18.9 Test design

What makes a good unit test — FIRST

Letter	Property	How the tests here achieve it
Fast	milliseconds, so you run them constantly	no network, no DB, no sleeps: 47 tests in ~0.6 s
Independent	any order, any subset	fresh instance per test, `@BeforeEach` setup
Repeatable	same result everywhere, every time	fixed `Clock`, fixed ids, `@TempDir`
Self-validating	pass/fail without reading output	assertions, not `println`
Timely	written with (or before) the code	—

Structure: Arrange–Act–Assert

Every test has three blocks — **arrange** (inputs, doubles), **act** (one call to the code under test), **assert** (check outcome) — the same idea as BDD's (behaviour-driven development's) *given–when–then*. `approvedPaymentSavesTheOrder` above is commented that way. One *behaviour* per test; several assertions about that one behaviour are fine.

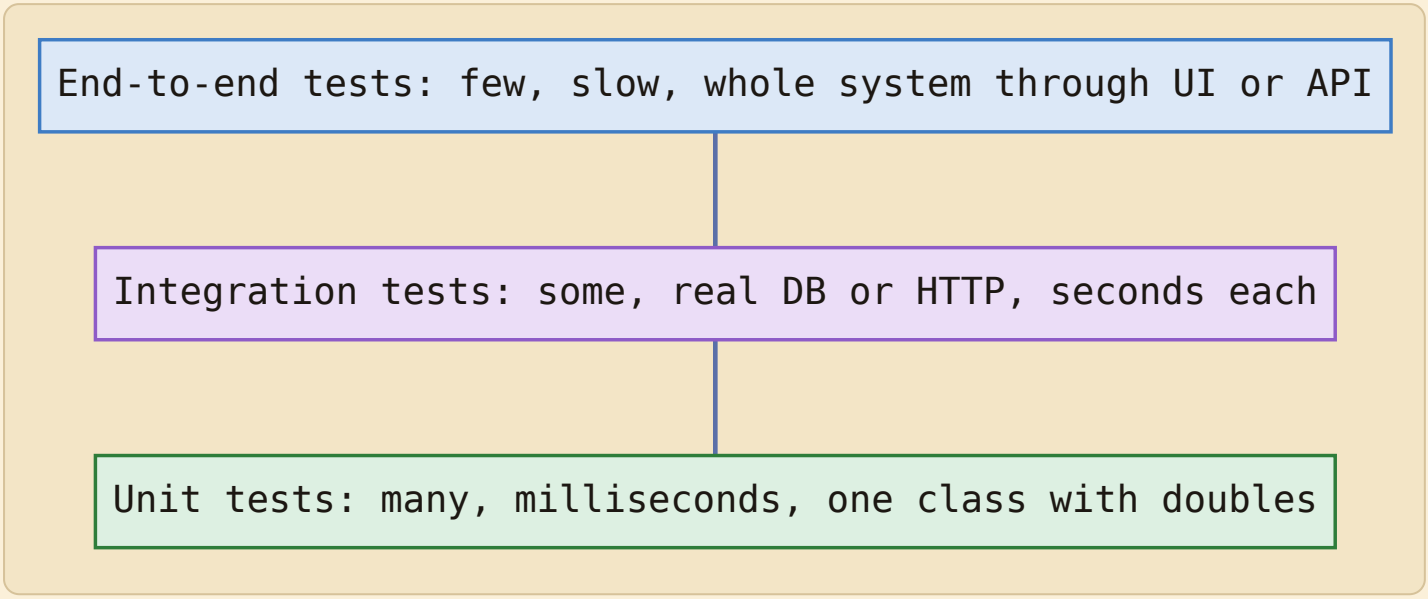
Naming

Name tests after behaviour, not methods: `declinedPaymentThrowsAndSavesNothing` beats `testCheckout2`. A failing test's name should already tell you what broke. `@DisplayName` allows full sentences.

Choosing inputs

- **Equivalence classes**: one representative per group that behaves the same (a valid code, a lowercase code, a too-short code).
- **Boundaries**: just below, at, just above each limit (`99.99`, `100.00` for SAVE25; `-1`, `0`, `100`, `101` for percentages).
- **Special values**: `null`, empty, zero, negative, very large, duplicates, Unicode (non-ASCII characters such as accents, emoji).
- **Errors**: each exception path, including exceptions thrown by collaborators (`gatewayOutagePropagates`).

The test pyramid



Most tests should be fast unit tests; fewer **integration tests** exercise real infrastructure (a real database, real HTTP); a handful of **end-to-end** tests cover whole user journeys. **Testcontainers** (a library, 2.x at the time of writing) starts real databases/**brokers** (message-queue servers such as Kafka

or RabbitMQ) in **Docker** (a tool that runs software in lightweight isolated *containers*) for integration tests — the modern replacement for in-memory lookalikes such as H2 (a small Java database that runs inside the test JVM), whose SQL **dialect** (its flavour of the SQL language) differs from production. Maven runs integration tests with the **Failsafe** plugin (`*IT.java`, `verify` phase); Gradle typically uses a separate test **source set** (a named group of source folders with its own compile task) or the JVM Test Suite plugin.

Coverage — useful, but not a goal

Code coverage measures which lines/**branches** (each possible path through an `if`/`switch`) the tests executed; **JaCoCo** is the standard Java tool (Maven plugin `jacoco-maven-plugin`, Gradle plugin `jacoco`), producing an HTML report. Low coverage reliably shows *untested* code; high coverage does **not** prove the tests check anything (a test without assertions still "covers" lines). **Mutation testing** (PIT/pitest) is stronger: it makes small changes to your code (flip `>=` to `>`) and checks that some test fails — our `99.99`/`100.00` rows would kill that **mutant** (the altered copy of the code).

Flaky tests

A **flaky** test sometimes passes and sometimes fails without code changes. Common causes and cures:

Cause	Cure
Current time, time zones	inject `Clock`; fixed zone
Randomness	inject `Random` with a seed, or a `Supplier`
Test order / shared static state	fresh state per test; no static mutables
`Thread.sleep` to "wait" for **async** work (work running in the background on another thread)	wait for a condition (Awaitility library, latches — Chapter 14)
Real network or external services	doubles in unit tests; Testcontainers/WireMock (a fake HTTP server for tests) in integration tests
Hash-order assumptions (`HashMap` iteration)	assert with `containsExactlyInAnyOrder` or sort first

TDD in one paragraph

Test-driven development repeats a short cycle: write a failing test (**red**), write the simplest code that passes (**green**), clean up (**refactor**) with the tests as a safety net. It keeps designs small and testable; many teams use it selectively (bug fixes: first write the test that reproduces the bug). See also LLD — refactoring with tests as a safety net.

In the interview

> **30-second answer:** "Good unit tests are fast, independent, repeatable and self-checking. I structure them arrange–act–assert, name them after behaviour, and pick inputs by equivalence classes and boundaries. I keep most tests at unit level with injected doubles, add integration tests against real infrastructure with Testcontainers, and treat coverage as a way to find untested code, not a target."

- *How do you test time-dependent code?* → Inject a `Clock`.
- *Is 100% coverage worth it?* → No; diminishing returns and it can't show assertion quality — mutation testing is a better signal.
- *What's a flaky test and how do you fix one?* → Nondeterministic outcome; remove the hidden input (time, randomness, order, network, sleeps).

Self-check

1. Two boundary inputs for "SAVE25 needs at least 100"? → 99.99 and 100.00.
2. What does 90% line coverage guarantee? → Only that 90% of lines ran during tests.

3. Why prefer Testcontainers over H2 for repository tests? → Same database engine and SQL dialect as production.

TL;DR

FIRST, AAA, behaviour names, boundary inputs, mostly unit tests, real infrastructure for integration tests, no hidden time/random/order dependencies.

18.10 Logging

What it is

Logging records what a running program does, each record with a **level**, a **logger name** (usually the class) and a timestamp, sent to destinations (console, file, **log collector** — a central service that gathers logs from many machines) chosen by **configuration**, not code. `System.out.println` has no levels, no way to turn it down in production, no timestamps and no routing.

How a logging system works

1. Code asks for a **logger** named after its class: `Logger.getLogger(OrderService.class.getName())`.
2. Loggers form a **hierarchy** by name: `ch18.shop.OrderService` → `ch18.shop` → `ch18` → root. A logger without its own level **inherits** the nearest ancestor's.
3. Each call has a **level**; if it is below the logger's effective level, the call is discarded cheaply.
4. Accepted records go to **handlers/appenders** (console, file…), which have their own level and a **formatter/layout**.

Levels across the libraries

`java.util.logging`	SLF4J / Logback / Log4j 2	`System.Logger`	Use for
`SEVERE`	`ERROR`	`ERROR`	failures needing attention
`WARNING`	`WARN`	`WARNING`	unexpected but handled
`INFO`	`INFO`	`INFO`	important business/lifecycle events
`CONFIG`	—	—	configuration details (JUL only)
`FINE`	`DEBUG`	`DEBUG`	diagnostic detail
`FINER`, `FINEST`	`TRACE`	`TRACE`	very fine detail

`java.util.logging` (JUL), run — `code/ch18/logging/JulDemo.java`

JUL is built into the JDK (module `java.logging`), so it needs no dependency. By default the root logger is at `INFO` and a `ConsoleHandler` writes to **System.err** (settings in `$JAVA_HOME/conf/logging.properties`).

```
// What you would normally put in a logging.properties file (-Djava.util.logging.config.file=...)
static final String CONFIG = """
        handlers = java.util.logging.ConsoleHandler
        .level = INFO
        java.util.logging.ConsoleHandler.level = ALL
        java.util.logging.SimpleFormatter.format = %4$-7s [%3$s] %5$s%6$s%n
        ch18.logging.level = FINE
        """;

public static void main(String[] args) throws IOException {
    System.setErr(System.out);                        // ConsoleHandler writes to System.err; merge for the demo
    LogManager.getLogManager().readConfiguration(
            new ByteArrayInputStream(CONFIG.getBytes(StandardCharsets.UTF_8)));
    // Normally a static final field; created AFTER the configuration here because the demo configures in code.
    Logger log = Logger.getLogger(JulDemo.class.getName());                    // "ch18.logging.JulDemo"

    System.out.println("-- 1. levels (ch18.logging is set to FINE, the root to INFO)");
    log.severe("disk full");
    log.warning("slow response: 1200 ms");
    log.info("server started on port 8080");
    log.config("pool size = 10");
    log.fine("cache miss for key 42");
    log.finer("not shown: FINER is below FINE");

    System.out.println("-- 2. another package inherits the root level INFO");
    Logger other = Logger.getLogger("com.acme.billing");
    other.info("billing ready");
    other.fine("not shown: com.acme.billing is at INFO");

    System.out.println("-- 3. parameters and lazy messages");
    log.log(Level.INFO, "user {0} bought {1} items", new Object[]{"ann", 3});   // MessageFormat style
    int[] calls = {0};
    Logger quiet = Logger.getLogger("com.acme.quiet");
    quiet.fine(() -> "expensive " + (++calls[0]));   // Supplier: not evaluated, FINE is disabled here
    quiet.fine("eager " + (++calls[0]));               // string built anyway, then thrown away
    System.out.println("message expressions evaluated: " + calls[0]);
    System.out.println("isLoggable(FINE) = " + quiet.isLoggable(Level.FINE));

    System.out.println("-- 4. logging an exception (message + stack trace)");
    try { Integer.parseInt("12x"); }
    catch (NumberFormatException e) { log.log(Level.WARNING, "bad quantity in request", e); }

    System.out.println("-- 5. a custom handler with its own format");
    Logger audit = Logger.getLogger("ch18.audit");
    audit.setUseParentHandlers(false);                // don't ALSO send to the root's ConsoleHandler
    Handler h = new StreamHandler(System.out, new java.util.logging.Formatter() {
        @Override public String format(LogRecord r) {
            return "AUDIT|" + r.getLevel() + "|" + r.getLoggerName() + "|" + formatMessage(r) + "\n";
        }
    });
    audit.addHandler(h);
    audit.info("ann changed her e-mail");
    h.flush();

    System.out.println("-- 6. System.Logger (Java 9): a facade that routes to JUL by default");
    System.Logger sys = System.getLogger("ch18.logging.system");
    sys.log(System.Logger.Level.INFO, "hello from System.Logger, {0}", "ann");
    sys.log(System.Logger.Level.DEBUG, "DEBUG maps to FINE -> shown, ch18.logging is FINE");
}
```

Output:

```
-- 1. levels (ch18.logging is set to FINE, the root to INFO)
SEVERE  [ch18.logging.JulDemo] disk full
WARNING [ch18.logging.JulDemo] slow response: 1200 ms
INFO    [ch18.logging.JulDemo] server started on port 8080
CONFIG  [ch18.logging.JulDemo] pool size = 10
FINE    [ch18.logging.JulDemo] cache miss for key 42
-- 2. another package inherits the root level INFO
INFO    [com.acme.billing] billing ready
-- 3. parameters and lazy messages
INFO    [ch18.logging.JulDemo] user ann bought 3 items
message expressions evaluated: 1
isLoggable(FINE) = false
-- 4. logging an exception (message + stack trace)
WARNING [ch18.logging.JulDemo] bad quantity in request
java.lang.NumberFormatException: For input string: "12x"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:527)
    at java.base/java.lang.Integer.parseInt(Integer.java:624)
    at ch18.logging.JulDemo.main(JulDemo.java:56)

-- 5. a custom handler with its own format
AUDIT|INFO|ch18.audit|ann changed her e-mail
-- 6. System.Logger (Java 9): a facade that routes to JUL by default
INFO    [ch18.logging.system] hello from System.Logger, ann
FINE    [ch18.logging.system] DEBUG maps to FINE -> shown, ch18.logging is FINE
```

What it shows:

- **Levels and inheritance**: `ch18.logging.level = FINE` applies to every logger below `ch18.logging`; `com.acme.billing` has no setting, so it inherits the root's `INFO`.
- **Two filters**: a record must pass the *logger's* level and then the *handler's* level (`ConsoleHandler.level = ALL` here; its default is `INFO`).
- **Formatter placeholders** in `SimpleFormatter.format` (a `String.format` pattern): `%1$` date/time, `%2$` source, `%3$` logger name, `%4$` level, `%5$` message, `%6$` exception with stack trace.
- **Lazy messages**: the `Supplier` overload skipped the string building; the plain overload built a string that was then discarded ("evaluated: 1"). Use suppliers (or `isLoggable` guards) for expensive messages.
- **Exceptions**: pass the exception as the last argument — you get the message *and* the stack trace (Chapter 8 §8.9).
- **Configuration timing**: levels from a configuration file are applied to loggers as they are created; configure logging at start-up (`-Djava.util.logging.config.file=logging.properties`) before your classes create loggers — that's why this demo, which configures in code, creates its logger afterwards.
- `System.Logger` (Java 9) is a tiny **facade** (a thin front-end API that forwards to whichever real implementation is installed) in `java.base`; without other setup it forwards to JUL.

SLF4J + Logback: what most applications use (not run)

Real applications usually code against **SLF4J** (Simple Logging Facade for Java, 2.0.x) — an API only — and pick a **backend** (the library that actually writes the logs) at **deployment** (installing and starting the app in its target environment): **Logback** (Spring Boot's default) or **Log4j 2**. Libraries log through SLF4J so the *application* decides where output goes. **Bridges** (adapter JARs) route JUL, Log4j 1, and Commons Logging calls into SLF4J so one configuration controls everything.

```java
// (SLF4J — not run: needs slf4j-api plus a backend such as logback-classic on the class path)
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.slf4j.MDC;

public final class OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderService.class);

    public Order checkout(String customer, Cart cart, String code) {
        MDC.put("customer", customer);                                  // added to every line on this thread
        try {
            log.debug("checkout started, {} items", cart.count());      // {} placeholders: no string built if
DEBUG is off
            // ...
            log.info("order placed id={} total={}", order.id(), total);
            log.atInfo().addKeyValue("orderId", order.id()).log("order placed");  // SLF4J 2 fluent API:
structured field
            return order;
        } catch (RuntimeException e) {
            log.error("checkout failed for {}", customer, e);           // exception LAST, no placeholder for it
            throw e;
        } finally {
            MDC.remove("customer");
        }
    }
}
```

```xml
<!-- src/main/resources/logback.xml (not run) -->
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder><pattern>%d{HH:mm:ss.SSS} %-5level [%thread] %logger{36} %X{customer} - %msg%n</pattern></encoder>
  </appender>
  <logger name="com.example.shop" level="DEBUG"/>
  <root level="INFO"><appender-ref ref="CONSOLE"/></root>
</configuration>
```

- `{}` **placeholders** replace string concatenation and are cheap when the level is off.
- **MDC** (Mapped Diagnostic Context) attaches key/values (request id, user) to every log line of the current thread — `%X{customer}` prints it. (With virtual threads and thread pools, always clear it in `finally`.)
- **Structured logging** (JSON lines with fields — JSON being the `{"key": value}` text format) makes logs searchable in tools such as Elasticsearch/OpenSearch, Loki or Splunk (log search and analytics systems).

Common mistakes

- Logging and rethrowing the same exception at every layer — one failure, five stack traces. Log once, where it's handled.
- Logging secrets or personal data (passwords, tokens, card numbers).
- String concatenation in hot debug logs (`log.debug("x=" + expensive())`).
- Several backends on the class path — SLF4J warns and picks one; keep exactly one.
- Treating the logging library as harmless: **Log4Shell** (CVE-2021-44228 — a CVE id is the public catalogue number of a security vulnerability — December 2021) let attackers run code through message lookups in Log4j 2 versions before 2.15.0 — keep logging libraries patched like any other dependency.

In the interview

30-second answer: "I code against SLF4J with a backend like Logback, one logger per class, levels ERROR to TRACE, parameterized `{}` messages, exceptions passed as the last argument, MDC for request ids, configuration outside the code. The JDK has java.util.logging and the `System.Logger` facade, fine for small tools."

- *Why a facade?* → Libraries shouldn't force a backend; the application chooses one, and bridges unify everything.
- *Why `{}` instead of concatenation?* → No string is built when the level is disabled.

Self-check

1. Which logger level applies to `com.acme.billing` in the demo? → Inherited root `INFO`.
2. What two levels must a JUL record pass? → The logger's and the handler's.
3. Where does JUL's default `ConsoleHandler` write? → `System.err`.

TL;DR

Loggers per class, levels, hierarchy, handlers/appenders, config outside code. JUL is built in; SLF4J + Logback/Log4j 2 is the industry default.

18.11 JDBC basics

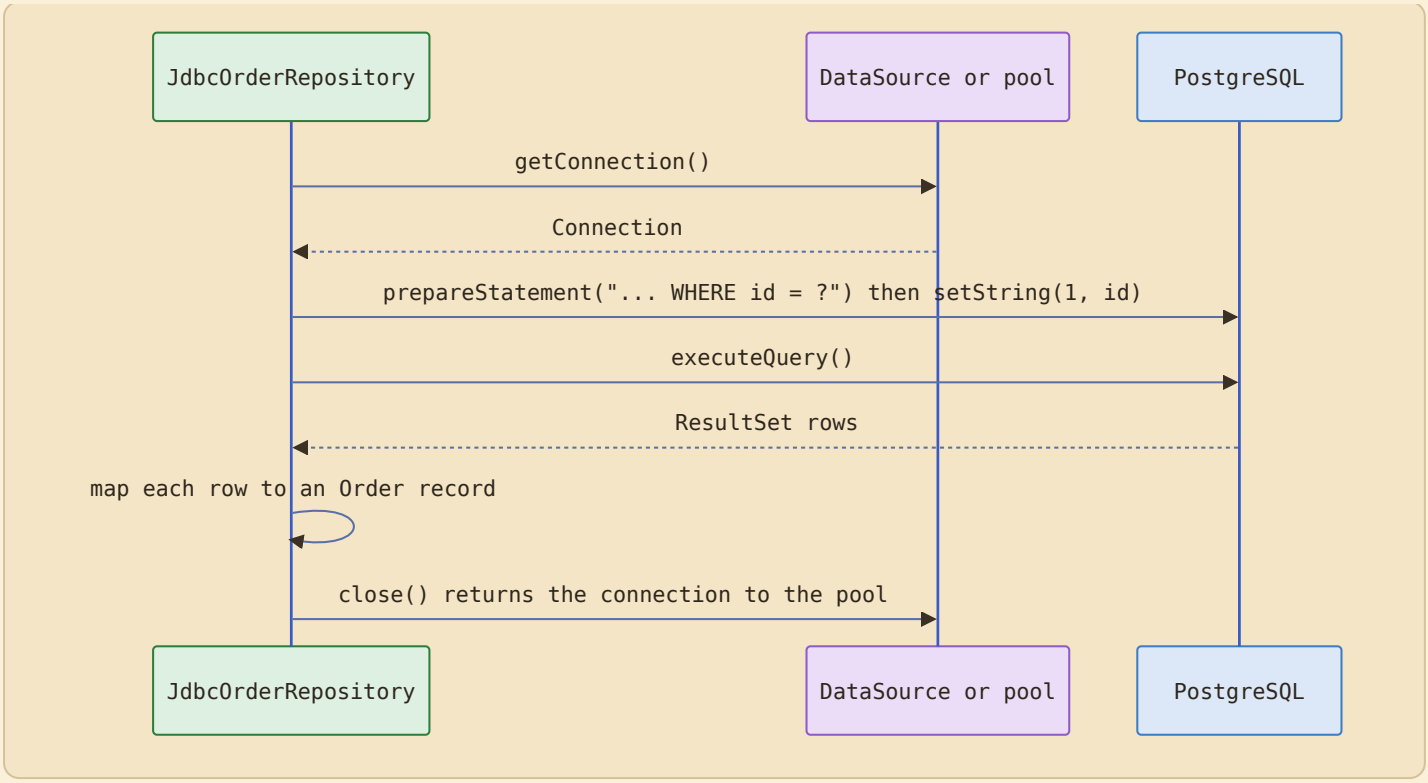
What it is

JDBC (Java Database Connectivity, packages `java.sql` and `javax.sql`) is the standard API for talking to **relational databases** (databases that store data in tables of rows and columns and are queried with SQL). Your code uses JDBC **interfaces**; a vendor's **driver** JAR (here PostgreSQL's `postgresql-42.7.13.jar`) implements them. Since JDBC 4.0 (Java 6) drivers register themselves through the **service loader** (the JDK's built-in mechanism for discovering plug-in implementations on the class path) — the old `Class.forName("org.postgresql.Driver")` line is unnecessary. Every higher-level tool (Spring's `JdbcTemplate`, jOOQ, Hibernate/JPA — the Java Persistence API) is built on JDBC.

The SQL Handbook's Chapter 14 runs a longer JDBC program against the same private PostgreSQL (SQL injection demonstrated, batches, **isolation levels** — how strictly concurrent transactions are kept from seeing each other's changes); this section focuses on the Java side: resource handling, mapping rows to records, transactions, and hiding JDBC behind the `OrderRepository` interface so the rest of the app — and its tests — don't care.

The pieces

Type	Role
`DriverManager`	gets a `Connection` from a URL — simple, for scripts and tools
`DataSource`	a connection **factory** (an object that creates other objects) configured once — what applications inject; a **connection pool** (a set of reusable open connections — HikariCP, Spring Boot's default) implements it
`Connection`	one session with the database; owns transactions
`PreparedStatement`	SQL with `?` placeholders, values bound by index (from **1**) with typed setters
`ResultSet`	a **cursor** (a pointer that moves through query results one row at a time) over result rows: `next()`, then `getString("col")`, `getBigDecimal`, `getObject(col, LocalDate.class)`
`SQLException`	**checked** exception (one the compiler forces you to catch or declare); `getSQLState()` gives a standard 5-character code (`23505` = unique violation)


JdbcOrderRepository
DataSource or pool
PostgreSQL
getConnection()
Connection
prepareStatement("... WHERE id = ?") then setString(1, id)
executeQuery()
ResultSet rows
map each row to an Order record
close() returns the connection to the pool
JdbcOrderRepository
DataSource or pool
PostgreSQL

The repository — `code/ch18/jdbc/JdbcOrderRepository.java`

```
public final class JdbcOrderRepository implements OrderRepository {
    private final DataSource dataSource;
    public JdbcOrderRepository(DataSource dataSource) { this.dataSource = dataSource; }

    @Override
    public void save(Order o) {
        String sql = "INSERT INTO orders (id, customer, total, placed_at) VALUES (?, ?, ?, ?)";
        try (Connection c = dataSource.getConnection();
             PreparedStatement ps = c.prepareStatement(sql)) {
            bind(ps, o);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RepositoryException("save failed for " + o.id(), e);
        }
    }

    /** All or nothing: one transaction for the whole list. */
    public void saveAll(List<Order> orders) {
        String sql = "INSERT INTO orders (id, customer, total, placed_at) VALUES (?, ?, ?, ?)";
        try (Connection c = dataSource.getConnection()) {
            c.setAutoCommit(false);                       // start a transaction
            try (PreparedStatement ps = c.prepareStatement(sql)) {
                for (Order o : orders) { bind(ps, o); ps.addBatch(); }
                ps.executeBatch();
                c.commit();
            } catch (SQLException e) {
                c.rollback();                             // undo every insert of this call
                throw e;
            }
        } catch (SQLException e) {
            throw new RepositoryException("saveAll failed", e);
        }
    }

    private List<Order> query(String sql, String param) {
        try (Connection c = dataSource.getConnection();
             PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, param);                       // bound parameter: never concatenate user input
            try (ResultSet rs = ps.executeQuery()) {
                List<Order> out = new ArrayList<>();
                while (rs.next()) out.add(map(rs));
                return out;
            }
        } catch (SQLException e) {
            throw new RepositoryException("query failed: " + sql, e);
        }
    }

    private static Order map(ResultSet rs) throws SQLException {       // row -> record
        return new Order(rs.getString("id"), rs.getString("customer"),
                rs.getBigDecimal("total"), rs.getTimestamp("placed_at").toInstant());
    }
    // ... findById / findByCustomer call query(...); bind(...) sets the four parameters;
    //     RepositoryException is an unchecked wrapper for SQLException
}
```

What to notice:

1. **try-with-resources** closes `ResultSet`, `PreparedStatement`, then `Connection`, even on exceptions (Chapter 8 §8.6). With a pool, `close()` returns the connection instead of disconnecting.
2. **Always `PreparedStatement` with `?`** for values — the driver sends them separately from the SQL text, which prevents **SQL injection** (an attack where user input is crafted to change the SQL command itself; demonstrated in the SQL book) and lets the database reuse the **plan** (its worked-out strategy for executing a query).
3. **Transactions**: a connection starts in **auto-commit** mode (each statement **commits** — makes its changes permanent — on its own). For several statements that must succeed together, `setAutoCommit(false)`,

then `commit()` or `rollback()` (undo everything since the transaction began).

4. **Exception translation** (Chapter 8 §8.8): the checked `SQLException` is wrapped, with the cause kept, so the `OrderRepository` interface stays storage-neutral.
5. The class names only `java.sql`/`javax.sql` types — the driver is a **runtime** dependency.

Running it — `code/ch18/jdbc/JdbcDemo.java`

The demo creates a database `ch_18_jdbc` on the SQL Handbook's private server (port 5544, reachable only through a **Unix socket** (a local inter-process connection that looks like a file, instead of a network port), hence the `junixsocket` socket-factory parameters in the URL; against a normal server the URL is just `jdbc:postgresql://localhost:5432/shop`), creates the table, and uses the repository through a `PGSimpleDataSource`:

```
PGSimpleDataSource ds = new PGSimpleDataSource();          // the driver's simple, non-pooled DataSource
ds.setURL(BASE + "ch_18_jdbc" + SOCKET);
ds.setUser("postgres");
JdbcOrderRepository repo = new JdbcOrderRepository(ds);
repo.save(new Order("ORD-1", "ann", new BigDecimal("76.88"), t));
repo.saveAll(List.of(/* ORD-2 for bob, ORD-3 for ann */));
System.out.println("findById(ORD-1)   = " + repo.findById("ORD-1").orElseThrow());
// ... findById("ORD-404"), findByCustomer("ann"), then saveAll(ORD-4, duplicate ORD-1) inside try/catch,
//     printing the SQLState of the cause, and finally whether ORD-4 exists
```

Commands (from the repository root) and real output:

```
$ bash tools_pg.sh start
postgres ready on port 5544 (socket /tmp)
$ javac --release 25 -cp '.java-books/lib/*' -d /tmp/java18o $(find java/chapters/18-testing-build-and-
tooling/code/ch18 -name '*.java')
$ java --enable-native-access=ALL-UNNAMED -cp "/tmp/java18o:.java-books/lib/*" ch18.jdbc.JdbcDemo
driver: PostgreSQL JDBC Driver 42.7.13
findById(ORD-1)   = Order[id=ORD-1, customer=ann, total=76.88, placedAt=2025-09-16T10:00:00Z]
findById(ORD-404) = Optional.empty
ann's orders      = [ORD-1, ORD-3]
saveAll failed: SQLState 23505 (BatchUpdateException)
ORD-4 present after rollback? false
```

(`--enable-native-access=ALL-UNNAMED` only silences the JDK's warning about the socket library loading native code — without it the program prints four `WARNING: A restricted method…` lines first.) The rollback worked: the valid `ORD-4` was undone together with the failing duplicate. `BatchUpdateException` is the `SQLException` subclass thrown by `executeBatch`.

Java ↔ SQL types

SQL (PostgreSQL)	Java	Getter / setter
`text`, `varchar`	`String`	`getString` / `setString`
`integer`, `bigint`	`int`, `long` (use `wasNull()` for SQL NULL) or `Integer` / `Long` via `getObject(col, Integer.class)`	`getInt`, `getLong`
`numeric`	`BigDecimal` — never `double` for money	`getBigDecimal` / `setBigDecimal`
`boolean`	`boolean`	`getBoolean`
`date`, `time`	`LocalDate`, `LocalTime` (JDBC 4.2)	`getObject(col, LocalDate.class)` / `setObject`
`timestamp`	`LocalDateTime`	`getObject(col, LocalDateTime.class)`
`timestamptz`	`OffsetDateTime` (pgjdbc does not accept `Instant` in `setObject`); `Timestamp.from(instant)` also works, as above	`getObject(col, OffsetDateTime.class)`

Testing database code

- **Unit tests** of services use the fake `InMemoryOrderRepository` — the reason the interface exists.
- **Integration tests** of `JdbcOrderRepository` itself need a real database: Testcontainers starts PostgreSQL in Docker, the test creates the **schema** (the table definitions), runs the repository, and checks rows. Mocking `Connection` / `ResultSet` tests nothing useful.

When not to write raw JDBC

For a handful of queries, raw JDBC is fine. For a real application, use a helper that removes the **boilerplate** (repetitive code that's the same every time): **Spring** `JdbcClient` / `JdbcTemplate` (SQL stays visible, mapping is a lambda), **jOOQ** (type-safe SQL builder), or **JPA/Hibernate** (maps objects to tables and generates SQL — powerful, but hides queries; watch for the **N+1 problem**: loading a list with one query, then firing one extra query per item). The SQL book compares **ORMs** (*object-relational mappers*, tools like Hibernate that map classes to tables) vs SQL in its Chapter 14.

Common mistakes

- Concatenating user input into SQL.
- Not closing connections — the pool runs dry and the app hangs.
- `getInt` on a NULL column returns `0`; check `wasNull()` or use `getObject(col, Integer.class)`.
- Using `double` for money columns.
- Forgetting `rollback()` (or `setAutoCommit(true)` before returning a connection to a pool that doesn't reset it).

In the interview

> **30-second answer:** "JDBC is the standard API; a driver implements it. Apps get connections from a pooled `DataSource`, use `PreparedStatement` with bound parameters to avoid SQL injection, map `ResultSet` rows to objects, close everything with try-with-resources, and control transactions with `setAutoCommit(false)`, `commit` and `rollback`. I hide it behind a repository interface and test it against a real database with Testcontainers."

- *`Statement` vs `PreparedStatement`?* → Prepared: parameters bound separately (safe, typed, plan reuse).
- *Why a connection pool?* → Opening a connection costs a network **handshake** (the opening exchange of messages that sets up a connection), authentication and a server process; a pool reuses a bounded number of them.

Self-check

1. First parameter index in JDBC? → 1.
2. What does auto-commit mean? → Each statement is its own committed transaction.
3. Which SQLState is a unique-key violation? → 23505.
4. Which scope/configuration for the driver? → `runtime` / `runtimeOnly`.

TL;DR

`DataSource` → `Connection` → `PreparedStatement` → `ResultSet`, all in try-with-resources; bind parameters; explicit transactions for multi-statement work; hide JDBC behind an interface; test it against a real DB.

18.12 Packaging: JAR files

What it is

A **JAR** (Java ARchive) is a ZIP file of `.class` files and resources plus a **manifest**, `META-INF/MANIFEST.MF`, holding **metadata** (data about the archive itself, such as which class to start). A manifest with a `Main-Class` makes it an **executable JAR**, started with `java -jar`.

Building one by hand — `code/package_jar.sh`

This is what `jar:jar` (Maven) and the `jar` task (Gradle) do for you. Real run:

```
$ javac --release 25 -d classes $(find src -name '*.java')
$ jar --create --file shop.jar --main-class ch18.shop.ShopDemo -C classes .
$ jar --list --file shop.jar | head -5
META-INF/
META-INF/MANIFEST.MF
ch18/
ch18/shop/
ch18/shop/Cart.class
$ unzip -p shop.jar META-INF/MANIFEST.MF
Manifest-Version: 1.0
Created-By: 26 (Oracle Corporation)
Main-Class: ch18.shop.ShopDemo

$ java -jar shop.jar
items=3 subtotal=102.50
Order[id=ORD-1, customer=ann, total=76.88, placedAt=2025-09-16T10:00:00Z]
Sep 28, 2026 3:37:08 PM ch18.shop.OrderService checkout
WARNING: declined: customer=bob total=899.00
error: payment of 899.00 declined for bob
$ ls -l shop.jar
13629 bytes
```

- `-C classes .` means "change into `classes/` and add everything" — package folders become ZIP paths.
- `--main-class` (short form `-e`) writes the `Main-Class` manifest line; `Created-By` records the JDK that built it (26 here, compiled for 25).
- The `WARNING` lines come from JUL's *default* format (timestamp, source, level) because no custom configuration was loaded — compare 18.10.
- Short, older option spellings still work: `jar cfe shop.jar ch18.shop.ShopDemo -C classes .`, `jar tf shop.jar`.

Dependencies at run time

`java -jar` **ignores** `-cp`; a JAR that needs libraries must say where they are, or contain them:

Approach	How	Notes
Plain JAR + class path	`java -cp "shop.jar:lib/*" ch18.shop.ShopDemo`	simplest; ship a `lib/` folder (Gradle's `application` plugin builds exactly this into `distZip` with start scripts)
`Class-Path` manifest entry	`Class-Path: lib/postgresql-42.7.13.jar`	relative paths, space-separated
Fat / uber JAR	Maven Shade plugin, Gradle Shadow plugin	one file with all classes merged; watch for clashing `META-INF/services` files and signatures (cryptographic proofs of who built a JAR)
Spring Boot JAR	`spring-boot-maven-plugin` / Gradle plugin	dependencies kept as nested JARs, loaded by Boot's own launcher
Custom runtime	`jlink` (Java 9)	a trimmed JDK containing only needed modules — see Chapter 16 §16.12
Native installer	`jpackage` (final in Java 16)	`.dmg` / `.pkg`, `.msi` / `.exe`, `.deb` / `.rpm` bundling a runtime
Container image (a packaged file system plus start command that a container runtime like Docker runs)	Dockerfile, Jib, Cloud Native Buildpacks	the common deployment unit for services
Native executable	GraalVM Native Image	**ahead-of-time** compiled (to machine code before running, not by the JIT), fast start-up; restrictions on **reflection** (inspecting and calling classes by name at run time)

Other JAR facts: a **multi-release JAR** (Java 9) holds version-specific classes under `META-INF/versions/N/`; a **modular JAR** contains `module-info.class` (Chapter 16, and the OCP modules chapter for exam depth); JARs can be **signed** with `jarsigner`. Libraries are published as a JAR + `.pom` (+ `-sources.jar` and `-javadoc.jar`) to a repository with `mvn deploy` or Gradle's `maven-publish` plugin.

Self-check

1. What makes a JAR executable? → A `Main-Class` entry in its manifest.
2. Why does `java -cp lib/* -jar app.jar` not find the libraries? → With `-jar`, the class path comes only from the JAR and its manifest.
3. Fat JAR vs `jlink` image? → Fat JAR bundles libraries but still needs an installed JRE (Java Runtime Environment: a JVM plus libraries, enough to run but not compile); `jlink` bundles a trimmed runtime.

TL;DR

JAR = ZIP + manifest. `jar --create --main-class`, run with `java -jar`. For dependencies: class path folder, fat JAR, Boot JAR, or container; `jlink` / `jpackage` to ship a runtime.

18.13 Other everyday tools

Need	Tool(s)	One-line note
IDE	IntelliJ IDEA, Eclipse, VS Code + Java extensions	import the Maven/Gradle build; don't hand-configure class paths (Chapter 1 §1.11)
Debugger	IDE debugger (**breakpoints** — lines where execution pauses so you can inspect variables — conditional breakpoints, watches, step into/over, evaluate expression); `jdb` on the command line	start the JVM with `-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005` to attach remotely
Quick experiments	`jshell` (Java's REPL — an interactive prompt that runs each line as you type it), single-file launch `java Foo.java`	Chapter 1 §1.9–1.10
Formatting	google-java-format, Palantir Java Format, Spotless (build plugin)	format automatically; stop debating style in reviews
Static analysis	compiler warnings (`-Xlint:all`), Error Prone, SpotBugs, PMD, Checkstyle, SonarQube	catch bugs and **smells** (patterns that suggest a design problem) before tests run
Null-safety	JSpecify annotations + NullAway	JUnit 6 and Spring Boot 4 both adopted JSpecify
Coverage / mutation	JaCoCo / PIT	18.9
Dependency hygiene	`dependency:tree`, `gradle dependencies`, Dependabot/Renovate (bots that open pull requests to update dependencies), OWASP Dependency-Check	keep libraries current and free of known vulnerabilities
Documentation	`javadoc` (Markdown doc comments since Java 23 — Chapter 17)	generated HTML API docs
Profiling / monitoring	JFR, JMC, VisualVM, async-profiler	Chapter 16 §16.10
Benchmarks	JMH	Chapter 16 §16.9
Continuous integration	GitHub Actions, GitLab CI, Jenkins	run `./mvnw verify` / `./gradlew build` on every push

A typical CI job is only a few lines — check out (fetch the code from the **repository**, the version-controlled project history), set up a JDK, run the wrapper:

```yaml
# .github/workflows/build.yml (GitHub Actions — not run)
name: build
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-java@v5
        with: { distribution: temurin, java-version: '25', cache: maven }
      - run: ./mvnw -B verify
```

18.14 Where next: Spring Boot and friends

What it is

Most server-side Java uses a **framework** for dependency injection, configuration, web **endpoints** (URLs your service answers), data access, security, **observability** (seeing what a running system does through logs, metrics and traces) and testing. The most widely used is **Spring Boot** (on the Spring Framework). **Spring Boot 4.0** was released on 20 November 2025 on Spring Framework 7 (4.1 followed in 2026); it runs on Java 17+ and supports Java 25.

What Spring Boot gives you

- **Dependency injection container**: you write classes like our `OrderService` with constructor parameters; Spring creates them (*beans*) and passes the right collaborators — exactly the design that made 18.6 testable, without the hand wiring in `ShopDemo`.
- **Starters**: one dependency per capability (`spring-boot-starter-webmvc`, `-jdbc`, `-data-jpa`, `-security`, `-actuator`) with compatible versions managed by Boot's BOM.
- **Auto-configuration**: sees a JDBC driver and a URL in `application.properties` → creates a HikariCP `DataSource`; sees the web starter → starts an **embedded** Tomcat (a web server running inside your own application rather than as a separate server).
- **Executable JAR**: `./mvnw spring-boot:run` in development, `java -jar app.jar` in production.
- **Testing**: the test starter bundles JUnit Jupiter, Mockito and AssertJ (this chapter's tools); `@SpringBootTest` for full context tests, *slice* tests like `@WebMvcTest` and `@DataJdbcTest`, `@MockitoBean` to replace a bean with a mock, and Testcontainers integration.
- **Actuator**: **health checks** (an endpoint reporting whether the service is working), **metrics** (numbers the service records about itself, collected via Micrometer), info endpoints for operations.

```java
// (Spring Boot — not run: needs the Spring Boot starters; shows the shape of an app)
@SpringBootApplication                          // component scan + auto-configuration
public class ShopApplication {
    public static void main(String[] args) { SpringApplication.run(ShopApplication.class, args); }
}

@RestController
@RequestMapping("/orders")
class OrderController {
    private final OrderService service;             // injected by Spring through the constructor
    OrderController(OrderService service) { this.service = service; }

    @GetMapping("/{id}")
    Order byId(@PathVariable String id) { return service.find(id).orElseThrow(); }
}
```

```properties
# src/main/resources/application.properties (not run)
spring.datasource.url=jdbc:postgresql://localhost:5432/shop
spring.datasource.username=shop
logging.level.com.example.shop=DEBUG
```

Generate a project at **start.spring.io** (Spring Initializr): choose Maven or Gradle, Java 25, and the starters you need.

Alternatives worth knowing

Framework	In one line
Quarkus	Kubernetes-oriented (Kubernetes runs and scales containers across many machines; see System Design Fundamentals), fast start-up, strong GraalVM native-image support; Jakarta EE / MicroProfile APIs
Micronaut	compile-time dependency injection (no run-time reflection), small **footprint** (memory use)
Jakarta EE (formerly Java EE)	the standard specifications (Servlet, JPA, CDI, REST) implemented by application servers and runtimes such as WildFly, Open Liberty, Payara
Helidon, **Javalin**, **Vert.x**	lighter or **reactive** options for services (reactive = non-blocking, event-driven code)

A learning path from here

1. Build a small **REST API** (a web interface where URLs name resources and HTTP verbs like GET/POST act on them) with Spring Boot (web + JDBC or JPA + validation), tested with this chapter's tools and Testcontainers.
2. Learn the design side: LLD Handbook — SOLID, patterns, and the case studies (parking lot, booking systems) — all in Java.
3. Learn the data side: SQL Handbook and *Designing Data-Intensive Applications* notes in `ddia/`.
4. Prepare for interviews: Chapter 19 — Java for coding interviews and the OCP Java 21 study repo for certification-level detail.

TL;DR

Spring Boot = DI container + starters + auto-configuration + executable JAR + first-class testing. Everything in this chapter — constructor injection, JUnit/Mockito/AssertJ, SLF4J logging, JDBC/DataSource, Maven/Gradle, JARs — is what Boot builds on.

Chapter TL;DR

- **Layout & coordinates**: `src/main/java`, `src/test/java`; libraries are `group:artifact:version` from Maven Central, cached locally.
- **Maven**: `pom.xml`; scopes; nearest-wins conflicts fixed by `dependencyManagement`/BOMs; lifecycle validate → … → deploy. Pin plugin versions; use `mvnw`.
- **Gradle**: scripted incremental task graph; `implementation`/`api`/`runtimeOnly`/`testImplementation`; `useJUnitPlatform()`; `gradlew`, toolchains.
- **JUnit (5/6 Jupiter)**: `@Test`, `assertEquals(expected, actual)`, `assertThrows`, `assertAll`; new instance per test; `@Nested`, `@Tag`, `@TempDir`; parameterized tests. JUnit 6 = Java 17 baseline, unified versions, same API.
- **Mockito**: inject collaborators, mock boundaries: `when/thenReturn`, `verify`, matchers (all or none), captors, strict stubs; prefer fakes; attach with `-javaagent`.
- **AssertJ**: `assertThat(x).…` fluent, type-aware, precise failure messages; soft assertions; recursive comparison.
- **Test design**: FIRST, arrange–act–assert, behaviour names, boundaries, pyramid, Testcontainers for integration, coverage ≠ quality, remove time/random/order/network nondeterminism.
- **Logging**: logger per class, levels, hierarchy, handlers; JUL built in; SLF4J + Logback/Log4j 2 in apps; `{}` placeholders, exception last, MDC, no secrets.
- **JDBC**: `DataSource` → `Connection` → `PreparedStatement` → `ResultSet` in try-with-resources; bind parameters; explicit transactions; wrap behind a repository.

- **Packaging**: JAR = ZIP + manifest; `jar --create --main-class`, `java -jar`; fat JARs, Boot JARs, `jlink`, `jpackage`, containers.
- **Next**: Spring Boot builds on all of the above.

Next: Chapter 19 — Java for coding interviews.

Chapter 19 — Java for Coding Interviews

A coding interview (the "LeetCode-style" round, named after LeetCode, a popular website of practice problems) checks two things: do you recognise the classic **data-structure and algorithm (DSA) patterns** (a *data structure* is a way of organising data in memory, such as an array, a list or a tree; an *algorithm* is a step-by-step recipe that solves a problem), and can you write them correctly in your language under time pressure? Java is a strong interview language — fast, predictable collections and a compiler that catches type slips — but it is verbose, and a few Java-specific traps (integer overflow, `Integer` compared with `==`, `Stack` instead of `ArrayDeque`, an `a - b` comparator) quietly fail hidden test cases (extra inputs the judge runs but never shows you). *Integer overflow* is a result too big for its type silently wrapping round to a wrong value; a *comparator* is the function that decides which of two values sorts first.

This chapter has three parts:

1. **The Java DSA toolkit** (19.1–19.7): the exact classes and idioms you reach for, the pitfalls, and a complexity table.
2. **Twenty patterns** (19.8–19.27): each with how to recognise it, the steps, runnable Java with checks, complexity and the common mistakes — from two pointers to tries (19.8–19.21), then linked lists, tree recursion, greedy, sort and search, bit manipulation, and math and geometry (19.22–19.27) — and how to talk through a solution (19.28).
3. **Java interview Q&A** (19.29): 42 short, correct answers to the language questions interviewers ask, each linked to the section of this book that teaches it properly.

It assumes you know Java basics (Chapters 2–6), collections (Chapter 10) and lambdas (Chapter 11). The same patterns in Python are in Python Handbook, Chapter 19 — reading both side by side is a good way to separate the *algorithm* from the *language*.

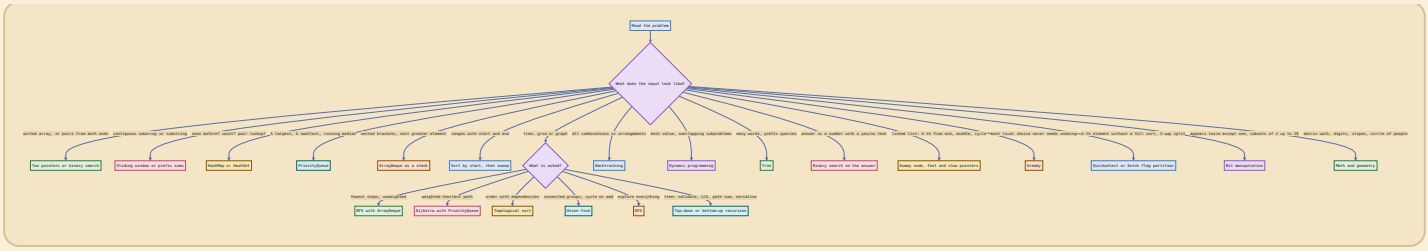
> **Key term — complexity (Big-O):** interviewers ask "what is the complexity?" meaning how the running time (and extra memory) grows as the input size *n* grows. O(1) means constant — the same cost for any n; O(log n) grows very slowly (halving each step); O(n) grows in step with n; O(n log n) is the cost of a good sort; O(n²) means a loop inside a loop. Constants are ignored, so O(2n) is just O(n). §19.7 collects the numbers for every Java collection.

What you'll learn

#	Topic	Program in `code/ch19/`
19.1	Why `assert` is off by default, and the tiny `Check` helper	`Check.java`, `toolkit/AssertKeywordDemo.java`
19.2	Arrays, `Arrays`, `String`, `StringBuilder`, char arithmetic	`toolkit/ArraysAndStrings.java`
19.3	`HashMap` / `HashSet` idioms: `merge`, `getOrDefault`, `computeIfAbsent`	`toolkit/MapSetIdioms.java`
19.4	`ArrayDeque` (stack and queue), `PriorityQueue`, `TreeMap` / `TreeSet`	`toolkit/DequesHeapsTrees.java`
19.5	Sorting with comparators	`toolkit/SortingDemo.java`
19.6	Overflow, `Math.*Exact`, `Integer` caching and other traps	`toolkit/NumericPitfalls.java`
19.7	Complexity cheat table	—
19.8–19.21	Two pointers, sliding window, hashing, prefix sums, stacks, heaps (incl. two heaps), binary search, intervals, BFS/DFS, topological sort, union-find, DP, backtracking, tries	`patterns/*.java`
19.22	Linked lists and fast/slow pointers: k-th from end, intersection, palindrome, cycle start, happy number	`patterns/LinkedLists.java`
19.23	Tree recursion templates: validate a BST, lowest common ancestor, max path sum, serialise/deserialise	`patterns/Trees.java`
19.24	Greedy: jump game, gas station, candies	`patterns/Greedy.java`
19.25	Sort and search: quicksort, quickselect, Dutch national flag, merge sort on a list	`patterns/SortSearch.java`
19.26	Bit manipulation: single number, counting bits, swapping bits, subsets by mask	`patterns/BitManipulation.java`
19.27	Math and geometry: spiral matrix, reverse an int with overflow check, points on a line, Josephus	`patterns/MathGeometry.java`
19.28	Talking through a solution	—
19.29	42 Java interview questions	`qa/InterviewFacts.java`

Key terms — the pattern names in the table and diagram: each is taught in its own section, but in one line: a **graph** is a set of *nodes* (points) joined by *edges* (links), and a **tree** is a graph with one *root* and no loops, where each node has children below it. **BFS** (breadth-first search) explores a graph ring by ring outward; **DFS** (depth-first search) follows one path as deep as possible before backing up. **Dijkstra** finds cheapest routes when edges have costs. **Topological sort** orders tasks so every prerequisite comes first. **Union-find** tracks which items are in the same group. **DP** (dynamic programming) reuses answers to smaller sub-problems; **backtracking** tries choices and undoes them; **greedy** always takes the best-looking step now. A **trie** is a tree of letters for prefix lookups; a **heap** keeps the smallest (or largest) item instantly available; a **BST** (binary search tree) keeps smaller keys to the left and larger to the right; the **LCA** (lowest common ancestor) is the deepest node above two given nodes. **Quickselect** finds the k-th smallest item without sorting everything.

How the code was run. Every program is in `code/`, compiled with `javac --release 25` (the Java compiler, told to produce code for Java 25) and run on OpenJDK 26, the free, open-source build of Java (`bash tools_run_java.sh java` from the repository root). Every pattern program ends with a line such as `10 checks passed`; the output shown is real.



Part 1 — The Java DSA toolkit

19.1 Testing your solutions: `assert` is off, use a helper

What it is

Interview practice means writing a function and checking it against a few inputs. Java has an `assert` statement, but the JVM (Java Virtual Machine — the program that runs compiled Java) **ignores it unless you start it with** `-ea` (enable assertions). Run a program normally and a failing `assert` does nothing at all, so a "passing" run proves nothing. This chapter therefore uses a ten-line helper, `ch19.Check`, whose `eq(actual, expected, label)` always runs, compares arrays by content, prints `ok …`, and throws `AssertionError` (an error object that stops the program and reports the message) on a mismatch.

```
// code/ch19/toolkit/AssertKeywordDemo.java (key part)
boolean enabled = false;
assert enabled = true;                  // the side effect runs ONLY when assertions are on (java -ea)
System.out.println("assert keyword enabled? " + enabled);
assert 1 + 1 == 3 : "never reported without -ea";
System.out.println("assert 1 + 1 == 3 did not stop the program");
eq(1 + 1, 2, "Check.eq always runs");
try { eq(1 + 1, 3, "deliberately wrong"); }
catch (AssertionError e) { System.out.println("Check.eq failed loudly: " + e.getMessage()); }
```

Output (plain `java`, no `-ea`):

```
assert keyword enabled? false
assert 1 + 1 == 3 did not stop the program
ok  Check.eq always runs -> 2
Check.eq failed loudly: deliberately wrong: expected 3 but got 2
```

With `java -ea` the first line becomes `true` and the second `assert` throws `AssertionError: never reported without -ea`.

In a real project you'd write JUnit tests (JUnit is Java's standard library for automated tests) instead (Chapter 18); in an interview you'd just run `main` with a few calls. Use `assert` in production code (the code that ships to real users) only for internal invariants (conditions your own code guarantees are always true) you're happy to have switched off — never for argument validation.

19.2 Arrays, strings and char arithmetic

Arrays are fixed-size and zero-based (the first element is index 0); `java.util.Arrays` is the toolbox of static helpers (methods called on the class itself, like `Arrays.sort(a)`) (§4.6, §4.8). `String` is immutable — it can never change after it is created (§7.1); build strings with `StringBuilder` (§7.3). A `char` is an unsigned 16-bit number (a whole number from 0 to 65,535 that stands for one character), so `c - 'a'` turns a lowercase letter into 0–25 — the basis of the `int[26]` frequency table used in dozens of string problems.

```java
// code/ch19/toolkit/ArraysAndStrings.java (key parts)
int[] a = {5, 2, 9, 1};
Arrays.sort(a);                                     // in place, ascending
int[] part = Arrays.copyOfRange(a, 1, 3);           // [from, to) -> indexes 1..2
int[] dist = new int[4];
Arrays.fill(dist, Integer.MAX_VALUE);               // "infinity" for shortest paths
int[][] grid = new int[2][3];                       // 2 rows x 3 cols, all 0
System.out.println(Arrays.equals(new int[]{1, 2}, new int[]{1, 2}) + " " + (new int[]{1, 2} == new int[]{1, 2}));

int[] freq = new int[26];
for (char c : "banana".toCharArray()) freq[c - 'a']++;  // 'a'->0, 'b'->1, ...
char next = (char) ('a' + 2);                       // int arithmetic, cast back to char
int digit = '7' - '0';                              // 7

char[] cs = "dcba".toCharArray();
Arrays.sort(cs);                                    // sort a string's letters
String sorted = new String(cs);

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) sb.append(i).append(',');
sb.setLength(sb.length() - 1);                      // drop the trailing comma
```

Output:

```
sorted      [1, 2, 5, 9]
copyOfRange [2, 5]
fill        [2147483647, 2147483647, 2147483647, 2147483647]
grid        [[0, 0, 0], [0, 0, 7]]
equals      true  ==  false
sum/max     17 9
freq a,b,n  3,1,2
'a'+2       99 as char c
digit value 7  Character.getNumericValue 7
helpers     false true q
trim/strip  [Hello, World] lower=hello, world
charAt/sub  e el 2
split       [a, b, , c]  words=[two, spaces]
sorted chars abcd  valueOf=abcd
builder     0,1,2,3,4  reversed=cba
edited      [X,1,2,3,4]  deleteCharAt(1)=ac
join/repeat a-b-c ababab
parse       -41 1010 10
```

Need	Java	Note
Length	`arr.length`, `s.length()`, `list.size()`	Field, method, method — a classic compile error
Print an array	`Arrays.toString(a)`, `Arrays.deepToString(grid)`	`println(a)` prints `[I@1b6d3586`
Compare arrays	`Arrays.equals(a, b)`	`a == b` and `a.equals(b)` compare references (same object?), not contents
Character at i	`s.charAt(i)`	O(1); `s.toCharArray()` copies (O(n)) — fine once, not in a loop
Substring	`s.substring(from, to)`	End exclusive; copies, O(to − from)
Split on whitespace	`s.trim().split("\\s+")`	`split` takes a **regex** (a text-pattern language); trailing empty strings are dropped
Letter test	`Character.isLetterOrDigit`, `isDigit`, `isLetter`, `toLowerCase`	
Build a string	`StringBuilder.append`, `insert`, `reverse`, `setCharAt`, `deleteCharAt`, `setLength`	`s += x` in a loop is O(n²)
Number ↔ text	`Integer.parseInt(s)`, `Integer.parseInt(s, 2)`, `String.valueOf(n)`, `Integer.toBinaryString(n)`	
2D grid	`new int[rows][cols]`, `grid[r][c]`	Rows first. `char[][] g` from `String[]` via `toCharArray()`

Common mistakes. Comparing strings with `==` (§19.6); forgetting that `'a' + 1` is the `int` 98, not `"b"`; using `split(".")` (the dot is a regex meaning "any character" — use `split("\\.")`); mutating an array you were given when the caller still needs it (clone first).

19.3 `HashMap` and `HashSet` idioms

A `HashMap` gives O(1) average `get`/`put`/`containsKey`; a `HashSet` is a `HashMap` with only keys (§10.4, internals in §10.5). Since Java 8 the `Map` interface (a contract listing the methods every map class provides) has default methods (methods that come with a ready-made body inside the interface) that turn the old "if key present … else …" into one line:

Idiom	One-liner	Replaces
Count	`count.merge(key, 1, Integer::sum)`	`if (!m.containsKey(k)) m.put(k, 1); else m.put(k, m.get(k) + 1);`
Count (alternative)	`count.put(k, count.getOrDefault(k, 0) + 1)`	same
Group	`groups.computeIfAbsent(key, k -> new ArrayList<>()).add(v)`	get, null check, put, add
Put once	`m.putIfAbsent(k, v)`	`if (!m.containsKey(k)) m.put(k, v)`
Seen before?	`if (!seen.add(x)) …`	`if (seen.contains(x)) … else seen.add(x)`
Iterate	`for (var e : m.entrySet()) e.getKey() / e.getValue()`	`for (k : keySet()) m.get(k)` (a second lookup per key)

```java
// code/ch19/toolkit/MapSetIdioms.java (key parts)
for (String w : words) count.merge(w, 1, Integer::sum);
for (String w : words) byLength.computeIfAbsent(w.length(), k -> new ArrayList<>()).add(w);
int n = count.get("missing");                  // null unboxed -> NullPointerException
if (!seen.add(x)) { firstDup = x; break; }     // add returns false for a duplicate
Set<int[]> arrays = ...;                       // arrays use identity equals/hashCode
record Cell(int r, int c) {}                   // records get value-based equals/hashCode
```

Output:

```
merge        {be=2, not=1, or=1, to=2}  same=true
group        {2=[to, be, or, to, be], 3=[not]}
be=2 to=2
unboxing null -> NullPointerException
first dup    1
int[] key    contains? false
List key     contains? true
record key   contains? true
linked order [to, be, or, not]
```

Keys. A key must have `equals` and `hashCode` based on its *value* (`hashCode` turns an object into an `int` that picks its bucket; `equals` then decides whether two objects count as the same) (§6.13). `int[]` doesn't, so `Set<int[]>` can't find an equal array. Use `List.of(r, c)`, a `record Cell(int r, int c)` (a *record* is a compact class for plain data whose `equals`/`hashCode` compare the field values), a `String` like `r + "," + c`, or encode a grid cell as the `int` `r * cols + c` (fastest). Never mutate an object while it is a key.

Order. `HashMap` iteration order is unspecified and can change between runs or JDK versions (the JDK, Java Development Kit, is the compiler, tools and standard library) — don't let an answer depend on it. Use `LinkedHashMap` (insertion order) or `TreeMap` (sorted) when order matters, as in `Hashing.firstUnique` (§19.10).

Arrays beat maps for small dense keys. When keys are lowercase letters or values 0–1000, an `int[26]` or `int[1001]` is faster than a `HashMap<Character, Integer>` (no boxing — wrapping each `int` in an `Integer` object — and no hashing, i.e. computing a hash code and finding its bucket).

19.4 `ArrayDeque`, `PriorityQueue` and `TreeMap`

`ArrayDeque` for stacks and queues — and why not `Stack`

`ArrayDeque` is a resizable circular array (an array whose end wraps round to its start, so both ends can grow cheaply) with amortised O(1) operations (O(1) on average over many calls — see §19.7) at both ends (§10.6). Use it for **both** stacks and queues:

Role	Add	Remove	Look	Empty remove
Stack (LIFO: last in, first out)	`push(x)`	`pop()`	`peek()`	`pop()` throws `NoSuchElementException`
Queue (FIFO: first in, first out)	`offer(x)`	`poll()`	`peek()`	`poll()` returns `null`

Declare the variable as `Deque<Integer>` (a stack) or `Queue<Integer>`/`Deque<Integer>` (a queue). `ArrayDeque` rejects `null` elements.

Why not `java.util.Stack`? It dates from Java 1.0, extends `Vector` (every method is `synchronized` — locked so only one thread can run it at a time — which costs time for nothing in single-threaded code; a *thread* is one independent path of execution inside a program), exposes index methods like `get(0)` that break the stack abstraction, and iterates **bottom-to-top** — the opposite of `ArrayDeque`. Its own Javadoc (the official API documentation) recommends `Deque` instead. `LinkedList` also implements `Deque` but allocates a node (a small object holding one value plus links to its neighbours) per element; `ArrayDeque` is faster.

`PriorityQueue` — a binary heap

`PriorityQueue` is a **min-heap** by default (a tree-shaped structure, stored in an array, that always keeps the smallest element at the top — the *root*): `peek()` is O(1), `offer`/`poll` are O(log n). For a max-heap pass `Comparator.reverseOrder()`. For arrays like `{distance, node}` use `Comparator.comparingInt(x -> x[0])`. Printing or iterating a `PriorityQueue` shows **heap order, not sorted order**; only repeated `poll()` gives sorted output. `contains` and `remove(Object)` are O(n).

`TreeMap` / `TreeSet` — sorted with navigation

A red-black tree (a self-balancing binary search tree, so it never becomes lopsided): O(log n) `get`/`put`/`remove`, and navigation methods a hash map can't offer. Remember the four: `floor` (≤), `ceiling` (≥), `lower` (<), ** `higher` ** (>). On a `TreeMap` they are `floorKey`/`floorEntry` and so on; all return `null` when nothing qualifies. Typical uses: finding the nearest booking before a given time, detecting clashes in a calendar, ranking players on a leaderboard, and medians over a moving window.

```
// code/ch19/toolkit/DequesHeapsTrees.java (key parts)
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2); stack.push(3);
Deque<String> queue = new ArrayDeque<>();
queue.offer("a"); queue.offer("b"); queue.offer("c");
PriorityQueue<Integer> max = new PriorityQueue<>(Comparator.reverseOrder());
PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(x -> x[0]));
TreeMap<Integer, String> prices = new TreeMap<>(Map.of(10, "ten", 20, "twenty", 30, "thirty"));
prices.floorKey(25); prices.ceilingKey(25); prices.lowerKey(20); prices.higherKey(30);
```

Output:

```
stack peek=3 pop=3 iterates [2, 1]
legacy Stack iterates [1, 2, 3] and allows get(0)=1
queue poll=a peek=b left [b, c]
empty poll -> null (pop() would throw)
min.peek=1 max.peek=8  toString is heap order, not sorted: [1, 3, 8, 5]
poll order 1 3 5 8
int[] heap first [2, 9]
floorKey(25)=20 ceilingKey(25)=30 lowerKey(20)=10 higherKey(30)=null
first=10 last=30=thirty headMap(20)={10=ten} tailMap(20)={20=twenty, 30=thirty}
TreeSet floor(10)=9 ceiling(10)=16 pollFirst=1 descending=[16, 9, 4]
```

19.5 Sorting with comparators

What you sort	Call	Algorithm and guarantees
`int[]`, `long[]`, `char[]` …	`Arrays.sort(a)`	Dual-Pivot Quicksort (quicksort splitting around two pivot values, §19.25); the Javadoc promises O(n log n) on all inputs. Ascending only — there is no comparator overload for primitives
`Integer[]`, `String[]`, `int[][]`, objects	`Arrays.sort(a, comparator)`	Stable (equal elements keep their order), TimSort-based merge sort (a merge sort tuned for partly sorted data), O(n log n)
`List`	`list.sort(cmp)` or `Collections.sort(list, cmp)`	Same stable sort; `Collections.sort` just calls `List.sort`

Build comparators from parts (`Person::age` is a *method reference*, shorthand for the lambda `p -> p.age()`) instead of writing `compare` by hand (§10.8): `Comparator.comparingInt(Person::age).reversed().thenComparing(Person::name)`. For `int[]` rows the lambda (a short anonymous function written `x -> …`) needs a type witness (the type argument written explicitly in angle brackets before the method name): `Comparator.<int[]>comparingInt(x -> x[0]).thenComparingInt(x -> x[1])` — otherwise the compiler infers `Object` once you chain.

```java
// code/ch19/toolkit/SortingDemo.java (key parts)
Integer[] boxed = {3, -1, 2};
Arrays.sort(boxed, Collections.reverseOrder());                    // descending needs boxes
int[][] intervals = {{5, 6}, {1, 9}, {1, 3}};
Arrays.sort(intervals, Comparator.<int[]>comparingInt(x -> x[0]).thenComparingInt(x -> x[1]));
people.sort(Comparator.comparingInt(Person::age).reversed().thenComparing(Person::name));
Integer[] big = {Integer.MIN_VALUE, 1, Integer.MAX_VALUE};
Arrays.sort(big, (x, y) -> x - y);                                 // WRONG: overflows
Arrays.sort(big, Integer::compare);                                // right
```

Output:

```
int[] asc        [-1, 2, 3]
Integer[] desc   [3, 2, -1]
stream desc      [3, 2, -1]
intervals        [[1, 3], [1, 9], [5, 6]]
age desc, name   [Person[name=Bo, age=30], Person[name=Ravi, age=30], Person[name=Ana, age=25]]
by length        [fig, pear, kiwi, apple]
(a-b) comparator [1, 2147483647, -2147483648]
Integer::compare [-2147483648, 1, 2147483647]
```

The `(a, b) -> a - b` comparator is the most common silent bug: `1 - Integer.MIN_VALUE` overflows to a negative number, so the sort puts `MIN_VALUE` last. Always use `Integer.compare(a, b)` or `Comparator.comparingInt`. Note also "by length" — `pear` stays before `kiwi` because the sort is stable.

19.6 Numeric and equality pitfalls

These are the bugs that pass the sample tests and fail the hidden ones. Background: overflow in §2.10, wrappers (object versions of primitives, such as `Integer` for `int`) and caching in §7.6.

```java
// code/ch19/toolkit/NumericPitfalls.java (key parts)
int lo = 2_000_000_000, hi = 2_100_000_000;
int bad  = (lo + hi) / 2;  // overflows
int mid  = lo + (hi - lo) / 2;  // safe midpoint
int mid2 = (lo + hi) >>> 1;     // also safe for non-negative lo, hi (unsigned shift)
long wrong = n * n;        // int * int overflows BEFORE widening
long right = (long) n * n;
Math.multiplyExact(n, n);  // throws ArithmeticException instead of wrapping
Integer c = 128, d = 128;
boolean same = c == d;             // false: compares references
list.remove(1);                     // removes INDEX 1
list.remove(Integer.valueOf(10));    // removes VALUE 10
```

Output:

```
MAX + 1            -2147483648
(lo+hi)/2          -97483648  lo+(hi-lo)/2=2050000000  (lo+hi)>>>1=2050000000
n*n wrong/right    1410065408 / 10000000000
multiplyExact      ArithmeticException: integer overflow
toIntExact ok      42
abs(MIN_VALUE)     -2147483648
7/2, -7/2          3, -3   7/2.0=3.5
-7 % 3, floorMod   -1, 2
127 == 127         true
128 == 128         false   equals=true
map.get == get     false   intValue== true
remove idx/value   [30]
String ==, equals  false, true
'a' + 1, "" + 'a' + 1  98, a1
for-each remove    ConcurrentModificationException
removeIf           [1, 3]
```

Trap	Why	Fix
`int` overflow	`int` wraps silently past ±2.1 billion	Use `long` for sums, products, and "infinity + edge"; `Math.addExact` / `multiplyExact` / `toIntExact` throw instead
Midpoint	`lo + hi` can overflow	`lo + (hi - lo) / 2`
`Math.abs(Integer.MIN_VALUE)`	$+2^{31}$ doesn't fit in `int`	Work in `long`, or `Math.absExact` (Java 15) to get an exception
`Integer.MAX_VALUE` as "infinity"	`INF + 1` wraps to a negative number	Guard before adding (see coin change, §19.19) or use `Integer.MAX_VALUE / 2`
`-7 % 3 == -1`	`%` keeps the dividend's (the number being divided) sign	`Math.floorMod(x, m)` for a non-negative index (circular arrays, hashing)
`7 / 2 == 3`	Integer division truncates toward zero	Cast one side: `7 / 2.0`, or `(double) a / b`
`Integer == Integer`	Compares references; only −128…127 are guaranteed cached by `Integer.valueOf`	`.equals`, `Integer.compare`, or unbox to `int`. Especially `map.get(a) == map.get(b)`
Unboxing `null`	`int x = map.get(missing)`	`getOrDefault`, or keep it as `Integer` and check
`list.remove(int)`	`List<Integer>` has `remove(int index)` and `remove(Object)`	`remove(Integer.valueOf(v))` to remove a value
Removing in for-each	Fail-fast iterator (it checks for changes) throws `ConcurrentModificationException`	`removeIf`, an explicit `Iterator.remove()`, or build a new list
Deep recursion	Default thread stack overflows around 10^4–10^5 frames (one frame per active call; depends on frame size)	Iterate with an explicit `ArrayDeque` stack for big grids/lists (see islands, §19.16)

19.7 Complexity cheat table

Big-O describes how cost grows with input size n; "amortised" means averaged over many calls (an occasional resize is paid for by many cheap operations). Details and a measured demo: §10.12.

Structure	Access / search	Insert / add	Remove	Notes
array `int[]`	O(1) index, O(n) search	— (fixed size)	—	`Arrays.binarySearch` O(log n) if sorted
`ArrayList`	O(1) `get(i)`, O(n) `contains`	O(1) amortised at end, O(n) in middle	O(n) (shifts), O(1) at end	Default list
`LinkedList`	O(n) `get(i)`	O(1) at ends	O(1) at ends	Rarely the right choice
`ArrayDeque`	O(1) `peek` at ends	O(1) amortised `push` / `offer`	O(1) `pop` / `poll`	Stack and queue
`HashMap` / `HashSet`	O(1) average	O(1) average	O(1) average	Worst case O(log n) with treeified (crowded bucket turned into a tree) buckets of `Comparable` keys
`LinkedHashMap`	O(1) average	O(1)	O(1)	Keeps insertion (or access) order — LRU (least-recently-used) cache
`TreeMap` / `TreeSet`	O(log n)	O(log n)	O(log n)	`floor` / `ceiling` / `first` / `last` also O(log n)
`PriorityQueue`	O(1) `peek`	O(log n) `offer`	O(log n) `poll`, O(n) `remove(obj)`	Build from a collection in O(n)
`String`	O(1) `charAt`	`+` copies: O(n)	`substring` copies	`StringBuilder.append` amortised O(1)

Algorithm	Time	Extra space
`Arrays.sort` / `List.sort`	O(n log n)	Small for primitives; up to O(n) buffer for objects (merge sort)
Binary search	O(log n)	O(1)
BFS / DFS on V vertices, E edges	O(V + E)	O(V)
Dijkstra with a binary heap	O((V + E) log V)	O(V + E)
Topological sort (Kahn)	O(V + E)	O(V)
Union-find with path compression + union by size	≈ O(1) per op (inverse Ackermann, §19.18)	O(n)
Subsets / permutations	$O(2^n \cdot n)$ / $O(n! \cdot n)$	O(n) recursion depth

Rough budget: about 10^8 simple operations per second in Java once the JIT (just-in-time compiler, which turns frequently run code into fast machine code while the program runs) has warmed up, so $n = 10^5$ needs O(n log n) or better and $n \le 20$ allows $O(2^n)$.

Part 2 — Twenty patterns

Each pattern follows the same shape: **what it is**, **recognise it when** (the wording in a problem that should trigger it), **how it works**, the code, its **complexity**, and **common mistakes**. Full programs are in `code/ch19/patterns/`; the README shows the key methods. Python versions of the same ideas: Python §19.2–19.11.

19.8 Two pointers

What it is

Two indexes move through the data in a coordinated way so that one pass replaces a nested loop. Three flavours: **opposite ends** moving inward (sorted arrays, palindromes — text that reads the same backwards), **read/write** pointers moving the same way (in-place filtering, i.e. changing the input array itself rather than building a new one), and **fast/slow** pointers moving at different speeds (linked lists — chains of nodes, each pointing to the next; §19.22). Analogy: two people searching a sorted bookshelf from both ends and meeting in the middle.

Recognise it when: the input is sorted; you need a pair/triple with a target sum; "in place, O(1) extra space"; palindrome; linked-list middle, cycle, or k-th from the end.

How it works (pair sum in a sorted array)

1. Put `left` at index 0 and `right` at the last index.
2. If the sum is too small, only moving `left` right can increase it; if too big, move `right` left.
3. Stop when they meet — each step discards one element for good, so the loop runs at most n times.

```
// code/ch19/patterns/TwoPointers.java (key parts)
static int[] pairWithSum(int[] sorted, int target) {
    int left = 0, right = sorted.length - 1;
    while (left < right) {
        int sum = sorted[left] + sorted[right];
        if (sum == target) return new int[]{left, right};
        if (sum < target) left++;          // need a bigger sum
        else right--;                      // need a smaller sum
    }
    return new int[]{-1, -1};
}

static int dedupe(int[] sorted) {          // read/write pointers
    if (sorted.length == 0) return 0;
    int write = 1;                         // sorted[0..write) holds the unique values
    for (int read = 1; read < sorted.length; read++)
        if (sorted[read] != sorted[write - 1]) sorted[write++] = sorted[read];
    return write;
}

static boolean hasCycle(ListNode head) {   // fast/slow (Floyd)
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) return true;     // reference comparison on purpose
    }
    return false;
}

static ListNode reverse(ListNode head) {   // prev / cur / next
    ListNode prev = null, cur = head;
    while (cur != null) { ListNode next = cur.next; cur.next = prev; prev = cur; cur = next; }
    return prev;
}
```

`ListNode` is a tiny class with `int val` and `ListNode next` — the shape interview platforms use. The file also has `isPalindrome` (skip non-alphanumerics, compare lower-cased) and `middle`.

Output:

```
ok  pairWithSum 15 -> [2, 4]
ok  pairWithSum none -> [-1, -1]
ok  dedupe -> [1, 2, 3, 4]
ok  palindrome yes -> true
ok  palindrome no -> true
ok  middle odd -> 3
ok  middle even -> 3
ok  cycle yes -> true
ok  cycle no -> true
ok  reverse -> 3->2->1
10 checks passed
```

Complexity: O(n) time, O(1) space (a sort first makes it O(n log n)).

Common mistakes

- Using opposite-end pointers on **unsorted** input — use hashing (§19.10) instead.
- `while (left <= right)` for pairs — an element would pair with itself.
- In fast/slow, checking `fast.next.next` without first checking `fast != null && fast.next != null`.
- For 3Sum (find all triples that add up to zero), forgetting to skip duplicate values after sorting.

19.9 Sliding window

What it is

A special two-pointer pattern for **contiguous** ranges (elements next to each other, with no gaps): keep a window `[left, right]` and a running summary of it (sum, counts, distinct characters). Moving `right` adds one element; moving `left` removes one — each element enters and leaves once, so the work is O(n) instead of O(n²) for re-examining every subarray (a contiguous slice of an array).

Recognise it when: "longest/shortest substring or subarray such that…", "at most k distinct", "window of size k", and the condition changes monotonically (only ever in one direction) as the window grows (for sums, this needs non-negative numbers — otherwise use prefix sums, §19.11).

How it works (variable-size window)

1. For each `right`, add `a[right]` to the window state.
2. **While** the window is invalid (or, for "shortest", while it is valid), update the answer as appropriate and remove `a[left++]`.
3. Record the answer for the current window.

```java
// code/ch19/patterns/SlidingWindow.java (key parts)
static int shortestWithSumAtLeast(int[] a, int target) {
    int left = 0, sum = 0, best = Integer.MAX_VALUE;
    for (int right = 0; right < a.length; right++) {
        sum += a[right];                          // grow
        while (sum >= target) {                   // valid: record, then shrink
            best = Math.min(best, right - left + 1);
            sum -= a[left++];
        }
    }
    return best == Integer.MAX_VALUE ? 0 : best;
}

static int longestUniqueSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        Integer prev = lastSeen.put(s.charAt(right), right);    // put returns the old value
        if (prev != null && prev >= left) left = prev + 1;      // jump past the duplicate
        best = Math.max(best, right - left + 1);
    }
    return best;
}

static int longestWithKDistinct(String s, int k) {
    int[] count = new int[128];                   // ASCII counts: faster than a HashMap
    int distinct = 0, left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        if (count[s.charAt(right)]++ == 0) distinct++;
        while (distinct > k) if (--count[s.charAt(left++)] == 0) distinct--;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

The file also has the fixed-size version, `maxSumOfK`: add `a[right]`, subtract `a[right - k]`.

Output:

```
ok  max sum of 3 -> 9
ok  shortest sum>=7 -> 2
ok  shortest none -> 0
ok  unique abcabcbb -> 3
ok  unique abba -> 2
ok  2 distinct eceba -> 3
6 checks passed
```

Complexity: O(n) time; O(1) or O(alphabet) space.

Common mistakes

- The `prev >= left` check in `longestUniqueSubstring`: without it, `"abba"` jumps `left` **backwards** at the last `a` and returns 3 instead of 2.
- Using `if` instead of `while` to shrink.
- Applying "shrink while sum ≥ target" to arrays with **negative** numbers — shrinking no longer reliably lowers the sum.

19.10 Hashing

What it is

Store what you've seen in a `HashMap`/`HashSet` so the question "have I seen X?" or "how many X so far?" costs O(1) instead of a scan. You trade O(n) memory for a factor of n in time.

Recognise it when: pairs with a target sum in **unsorted** data, duplicates, counting frequencies, grouping by a signature (anagrams — words with the same letters, like *tea* and *eat*), "first unique", "longest consecutive run".

How it works (Two Sum)

1. Walk the array once.
2. For each value x, look up `target - x` among the values already seen.
3. If found, you have the pair; otherwise record x → its index and continue. Looking only backwards also stops an element pairing with itself.

```
// code/ch19/patterns/Hashing.java (key parts)
static int[] twoSum(int[] a, int target) {
    Map<Integer, Integer> indexOf = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        Integer j = indexOf.get(target - a[i]);
        if (j != null) return new int[]{j, i};
        indexOf.put(a[i], i);
    }
    return new int[0];
}

static List<List<String>> groupAnagrams(String[] words) {
    Map<String, List<String>> groups = new LinkedHashMap<>();   // keeps first-seen order
    for (String w : words) {
        char[] cs = w.toCharArray();
        Arrays.sort(cs);                                        // "tea" -> "aet"
        groups.computeIfAbsent(new String(cs), k -> new ArrayList<>()).add(w);
    }
    return new ArrayList<>(groups.values());
}

static int longestConsecutive(int[] a) {
    Set<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    int best = 0;
    for (int x : set) {
        if (set.contains(x - 1)) continue;        // only count from the START of a run
        int len = 1;
        while (set.contains(x + len)) len++;
        best = Math.max(best, len);
    }
    return best;
}
```

Output:

```
ok  twoSum -> [2, 3]
ok  groupAnagrams -> [[eat, tea, ate], [tan, nat], [bat]]
ok  longestConsecutive -> 4
ok  firstUnique -> w
4 checks passed
```

Complexity: O(n) average time (`groupAnagrams` is O(n · k log k) for words of length k), O(n) space. `longestConsecutive` is O(n) because each run is walked only from its first element.

Common mistakes

- `Integer j = …; if (j != 0)` style checks on a missing key (it's `null`, not 0).
- Comparing two `Integer` values from a map with `==` (§19.6).
- Using `int[]` as a map key (identity `hashCode`) — use a `String`, `List`, or record.

19.11 Prefix sums

What it is

Precompute `prefix[i]` = sum of the first i elements. Then the sum of any range `a[l..r]` is `prefix[r + 1] - prefix[l]` — one subtraction instead of a loop. Combined with a hash map, it counts subarrays with a given sum even when numbers are **negative** (where a sliding window fails).

Recognise it when: many range-sum queries; "number of subarrays whose sum equals k"; 2D sub-rectangle sums; many "add v to every element in [l, r]" updates (the inverse trick, a **difference array**).

How it works (subarrays summing to k)

1. Keep a running sum and a map from "prefix sum value" → "how many times seen", starting with `{0: 1}` (the empty prefix).
2. A subarray ending here sums to k exactly when an earlier prefix equals `sum - k`. Add that count.
3. Record the current sum.

```
// code/ch19/patterns/PrefixSums.java (key parts)
static long[] prefix(int[] a) {                 // length n+1, prefix[0] = 0; long avoids overflow
    long[] p = new long[a.length + 1];
    for (int i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
    return p;
}
static long rangeSum(long[] p, int from, int to) { return p[to + 1] - p[from]; }

static int subarraysSumToK(int[] a, int k) {
    Map<Integer, Integer> seen = new HashMap<>();
    seen.put(0, 1);                             // empty prefix
    int sum = 0, count = 0;
    for (int x : a) {
        sum += x;
        count += seen.getOrDefault(sum - k, 0);
        seen.merge(sum, 1, Integer::sum);
    }
    return count;
}

// 2D: p[r+1][c+1] = g[r][c] + above + left - overlap (inclusion-exclusion)
static long rect(long[][] p, int r1, int c1, int r2, int c2) {
    return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
}
```

The difference array records `+v` at `l` and `-v` at `r + 1` for each update, then one running-sum pass produces the final array.

Output:

```
ok  rangeSum 1..3 -> 6
ok  rangeSum all -> 23
ok  subarrays sum 3 -> 4
ok  2D rect -> 28
ok  difference array -> [0, 2, 5, 5, 3]
5 checks passed
```

Complexity: O(n) to build, O(1) per query; 2D is O(rows · cols) to build.

Common mistakes

- Off-by-one (an index one too high or too low): the `n + 1` length and the `prefix[0] = 0` sentinel (a dummy value placed to remove a special case) exist precisely so that `l = 0` needs no special case.
- Forgetting `seen.put(0, 1)` — misses subarrays that start at index 0.
- `int` prefix sums of large inputs overflowing — use `long`.

19.12 Stacks and monotonic stacks

What it is

A **stack** (last in, first out) matches nested structure: brackets, expression evaluation, undo, DFS. A **monotonic stack** keeps its contents in increasing or decreasing order by popping everything that breaks the order before pushing; each popped element has just met its "next greater" (or smaller) element. A **monotonic deque** does the same at both ends to give the max of a sliding window in O(n).

Recognise it when: balanced brackets; "next greater/smaller element", "days until warmer", "stock span", "largest rectangle in histogram"; "maximum of every window of size k".

How it works (next warmer day)

1. Keep a stack of **indexes** whose temperatures are decreasing from bottom to top.
2. For each new day i, pop every index j whose temperature is lower — day i is j's answer, `i - j`.
3. Push i. Indexes left at the end never found a warmer day (answer 0).

```
// code/ch19/patterns/Stacks.java (key parts)
static boolean validBrackets(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' -> stack.push(')');          // push the EXPECTED closer
            case '[' -> stack.push(']');
            case '{' -> stack.push('}');
            default -> { if (stack.isEmpty() || stack.pop() != c) return false; }
        }
    }
    return stack.isEmpty();
}

static int[] daysUntilWarmer(int[] t) {
    int[] ans = new int[t.length];
    Deque<Integer> stack = new ArrayDeque<>();
    for (int i = 0; i < t.length; i++) {
        while (!stack.isEmpty() && t[stack.peek()] < t[i]) {
            int j = stack.pop();
            ans[j] = i - j;                       // i is j's "next greater"
        }
        stack.push(i);
    }
    return ans;
}

static int[] windowMax(int[] a, int k) {
    int[] out = new int[a.length - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();       // indexes, values decreasing front to back
    for (int i = 0; i < a.length; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();      // fell out of window
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i]) dq.pollLast();   // can never be max again
        dq.offerLast(i);
        if (i >= k - 1) out[i - k + 1] = a[dq.peekFirst()];
    }
    return out;
}
```

The file also evaluates Reverse Polish Notation (postfix notation, where each operator follows its operands: `"4 13 5 / +"` = 6) with an arrow-label `switch` (§3.3).

Output:

```
ok  brackets ok -> true
ok  brackets crossed -> true
ok  brackets unclosed -> true
ok  daysUntilWarmer -> [1, 1, 4, 2, 1, 1, 0, 0]
ok  windowMax -> [3, 3, 5, 5, 6, 7]
ok  RPN -> 6
6 checks passed
```

Complexity: O(n) — every index is pushed once and popped at most once, even though there's a loop inside a loop.

Common mistakes

- Storing values instead of indexes when you need distances.
- Forgetting the final `stack.isEmpty()` check (`"(("` would pass).
- Popping in RPN in the wrong order: the **second** pop is the left operand (the value on the left of the operator).
- Using `java.util.Stack` (§19.4).

19.13 Heaps and top-k

What it is

A heap (`PriorityQueue`) always hands you the smallest (or largest) element in O(log n) per change. The key trick for "top k" is counter-intuitive: to keep the k **largest**, use a **min**-heap of size k — its root is the weakest of the current top k, the one to evict when something bigger arrives.

Recognise it when: "k-th largest/smallest", "k most frequent", "k closest points", "merge k sorted lists", "running median" (the median of everything seen so far, updated as each value arrives) and "maximise capital" (two heaps, below), scheduling by earliest end time, Dijkstra.

How it works (k-th largest)

1. Offer each element into a min-heap.
2. If the heap grows past k, poll (remove the smallest).
3. After all elements, the root is the k-th largest. Cost O(n log k) — better than sorting when k ≪ n.

```java
// code/ch19/patterns/HeapTopK.java (key parts)
static int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();          // evict the smallest
    }
    return heap.peek();
}

static List<Integer> mergeSorted(int[][] arrays) {   // heap holds {value, arrayIndex, elementIndex}
    PriorityQueue<int[]> heap = new PriorityQueue<>(Comparator.comparingInt(x -> x[0]));
    for (int i = 0; i < arrays.length; i++) if (arrays[i].length > 0) heap.offer(new int[]{arrays[i][0], i, 0});
    List<Integer> out = new ArrayList<>();
    while (!heap.isEmpty()) {
        int[] top = heap.poll();
        out.add(top[0]);
        int arr = top[1], next = top[2] + 1;
        if (next < arrays[arr].length) heap.offer(new int[]{arrays[arr][next], arr, next});
    }
    return out;
}

static final class MedianFinder {                    // two heaps
    private final PriorityQueue<Integer> low = new PriorityQueue<>(Comparator.reverseOrder()); // max-heap
    private final PriorityQueue<Integer> high = new PriorityQueue<>();                         // min-heap
    void add(int x) {
        low.offer(x);
        high.offer(low.poll());                                  // largest of low moves up
        if (high.size() > low.size()) low.offer(high.poll());    // keep low >= high in size
    }
    double median() {
        return low.size() > high.size() ? low.peek() : (low.peek() + (long) high.peek()) / 2.0;
    }
}
```

`topKFrequent` counts with `merge`, then keeps a size-k min-heap of map entries ordered by `Map.Entry.comparingByValue()`, breaking ties by key.

Output:

```
ok  kthLargest -> 5
ok  topKFrequent -> [1, 2]
ok  mergeSorted -> [0, 1, 2, 4, 5, 7, 9]
ok  median of 5,15 -> 10.0
ok  median of 5,15,1 -> 5.0
ok  maximizeCapital k=2 -> 4
ok  maximizeCapital k=3 -> 6
ok  maximizeCapital nothing affordable -> 0
8 checks passed
```

Complexity: top-k O(n log k); merging k lists with N total elements O(N log k); median O(log n) per add, O(1) per query; maximise capital O(n log n) (each project enters and leaves each heap at most once).

Two heaps

Some problems need the best element from **two different groups** at once. Keep one heap per group and move elements across the boundary as the situation changes. Two shapes come up:

- **Split one stream into halves** (running median). A max-heap `low` holds the smaller half, a min-heap `high` the larger half, and the invariant (the rule kept true after every step) is `low.size() == high.size()` or one more. The median is then `low.peek()` or the average of both roots — O(1) — and each `add` costs O(log n). Every new value passes through `low` into `high` and, if `high` got too big, one value comes back; that keeps every element of `low` ≤ every element of `high` without a special case (`MedianFinder` above).

- **Locked vs available** (maximise capital, scheduling). A min-heap orders items by the threshold that **unlocks** them; a max-heap holds the unlocked ones by **value**. Each round: move everything whose threshold is now met from the first heap to the second, then take the best from the second. Example: you may run at most k projects; project i needs `capital[i]` to start and pays `profit[i]`; start with `start` money.

```java
// code/ch19/patterns/HeapTopK.java (key part)
static long maximizeCapital(int k, long start, int[] profit, int[] capital) {
    PriorityQueue<Integer> locked = new PriorityQueue<>(Comparator.comparingInt(i -> capital[i]));
    PriorityQueue<Integer> affordable = new PriorityQueue<>(Comparator.comparingInt((Integer i) ->
profit[i]).reversed());
    for (int i = 0; i < profit.length; i++) locked.offer(i);
    long money = start;
    for (int round = 0; round < k; round++) {
        while (!locked.isEmpty() && capital[locked.peek()] <= money) affordable.offer(locked.poll()); // unlock
        if (affordable.isEmpty()) break;                  // nothing we can start
        money += profit[affordable.poll()];               // greedy: the most profitable affordable project
    }
    return money;
}
```

Heaps store **indexes** here, and the comparators look the values up — cheaper than creating `int[]` pairs. The `(Integer i) ->` type on the lambda is needed because `.reversed()` stops Java inferring it. Taking the most profitable affordable project is a greedy choice (§19.24): money only grows, so nothing unlocked now can become locked later, and a bigger profit never unlocks fewer projects.

Recognise it when: "median of a stream / sliding window median", "maximise capital with at most k projects", "schedule tasks as they become available", anything that says "the best among those that currently qualify".

Common mistakes

- A max-heap of everything for top-k: O(n log n), and uses O(n) memory.
- Writing the max-heap comparator as `(a, b) -> b - a` (overflow, §19.5) — use `Comparator.reverseOrder()`.
- Expecting `heap.toString()` or a for-each loop to be sorted.
- Changing an object's priority field while it sits in the heap — the heap won't re-order; remove and re-insert (or insert a new entry and skip stale ones, as Dijkstra does in §19.16).

19.14 Binary search (and binary search on the answer)

What it is

Halve a sorted search space each step: O(log n). Beyond "find x in a sorted array", the powerful version is **binary search on the answer**: when the answer is a number and a yes/no test `feasible(x)` flips from false to true exactly once as x grows, binary-search x itself.

Recognise it when: sorted input; "first/last position of", "insert position"; rotated sorted array; "minimum capacity/speed/time such that…" or "maximise the minimum…" (search on the answer).

How it works

1. Decide the range and write down the **invariant** (what's true about `lo` and `hi` at every step).
2. Compute `mid = lo + (hi - lo) / 2` (never `(lo + hi) / 2`, §19.6).
3. Discard the half that cannot contain the answer, keeping the invariant.
4. Two templates cover almost everything: **closed** `[lo, hi]` with `while (lo <= hi)` for "find exactly", and **half-open / first-true** (the range includes `lo` but not `hi`) with `while (lo < hi)`, `hi = mid` for "first index where the condition holds".

```java
// code/ch19/patterns/BinarySearch.java (key parts)
static int lowerBound(int[] a, int target) {       // first index with a[i] >= target
    int lo = 0, hi = a.length;                      // answer is in [lo, hi]
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] < target) lo = mid + 1; else hi = mid;
    }
    return lo;
}

static int searchRotated(int[] a, int target) {    // e.g. [4,5,6,7,0,1,2]
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[lo] <= a[mid]) {                      // left half is sorted
            if (a[lo] <= target && target < a[mid]) hi = mid - 1; else lo = mid + 1;
        } else {                                    // right half is sorted
            if (a[mid] < target && target <= a[hi]) lo = mid + 1; else hi = mid - 1;
        }
    }
    return -1;
}

// Binary search on the answer: smallest ship capacity that ships everything in `days` days.
static int minCapacity(int[] weights, int days) {
    int lo = Arrays.stream(weights).max().getAsInt();   // must fit the heaviest package
    int hi = Arrays.stream(weights).sum();              // everything in one day
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (daysNeeded(weights, mid) <= days) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

`daysNeeded` greedily fills each day up to the capacity. Library versions: `Arrays.binarySearch` and `Collections.binarySearch` return the index if found, otherwise `-(insertionPoint) - 1`; with duplicates, they don't promise *which* equal element they return — write `lowerBound` yourself when you need the first. `TreeSet.ceiling` is the other ready-made lower bound.

Output:

```
ok  search hit -> 3
ok  search miss -> -1
ok  lowerBound first 2 -> 1
ok  lowerBound past end -> 5
ok  rotated -> 4
ok  ship capacity -> 15
ok  Arrays.binarySearch miss -> -3
ok  insertion point -> 2
ok  Collections.binarySearch -> 2
9 checks passed
```

Complexity: O(log n); on the answer, O(n · log(range)).

Common mistakes

- Mixing templates: `while (lo < hi)` with `hi = mid - 1`, or `while (lo <= hi)` with `hi = mid` (infinite loop).
- `lo = mid` with a downward-rounded mid: when `hi = lo + 1` the loop never moves.
- Search-on-answer with the wrong bounds (capacity below the heaviest item is never feasible) or a non-monotonic test.

19.15 Intervals

What it is

Problems about ranges `[start, end]`: merge overlaps, insert a new range, count simultaneous meetings. Almost all start the same way: **sort by start**, then sweep left to right, comparing each interval with the last one you kept.

Recognise it when: meetings, bookings, time ranges, "overlapping", "minimum rooms/platforms", "free time".

How it works (merge)

1. Sort by start.
2. For each interval: if it starts at or before the end of the last merged interval, they overlap — extend that end to the max of the two ends.
3. Otherwise start a new merged interval.

```java
// code/ch19/patterns/Intervals.java (key parts)
static int[][] merge(int[][] intervals) {
    int[][] sorted = intervals.clone();                          // don't reorder the caller's array
    Arrays.sort(sorted, Comparator.comparingInt(x -> x[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] cur : sorted) {
        if (!out.isEmpty() && cur[0] <= out.getLast()[1]) {      // overlaps the last merged one
            out.getLast()[1] = Math.max(out.getLast()[1], cur[1]);
        } else {
            out.add(new int[]{cur[0], cur[1]});
        }
    }
    return out.toArray(new int[0][]);
}

static int minRooms(int[][] meetings) {                          // [start, end) semantics
    int[][] sorted = meetings.clone();
    Arrays.sort(sorted, Comparator.comparingInt(x -> x[0]));
    PriorityQueue<Integer> ends = new PriorityQueue<>();         // end times of rooms in use
    for (int[] m : sorted) {
        if (!ends.isEmpty() && ends.peek() <= m[0]) ends.poll(); // reuse the room that frees first
        ends.offer(m[1]);
    }
    return ends.size();
}
```

`List.getLast()` is from sequenced collections (Java 21, §10.11); on older Java write `out.get(out.size() - 1)`. `insert` copies intervals that end before the new one, merges all that overlap it, then copies the rest — O(n) with no sort.

Output:

```
ok  merge -> [[1, 6], [8, 10], [15, 18]]
ok  merge touching -> [[1, 5]]
ok  insert -> [[1, 2], [3, 10]]
ok  minRooms -> 2
ok  minRooms back-to-back -> 1
5 checks passed
```

Complexity: O(n log n) for the sort; the sweep is O(n) (O(n log n) with the heap).

Common mistakes

- Not agreeing whether `[1, 4]` and `[4, 5]` overlap: for merging closed ranges, `<=` says yes; for meetings `[start, end)`, back-to-back meetings share a room. Ask the interviewer.
- Keeping `max(end)` out: `[1, 10]` then `[2, 3]` must stay `[1, 10]`.

- `clone()` on `int[][]` is **shallow** — rows are shared, so writing into `out` rows copied by reference would modify the input. That's why `merge` adds `new int[]{…}`.

19.16 BFS and DFS: trees, grids and graphs

What it is

Two ways to visit every node reachable from a start. **Breadth-first search (BFS)** uses a **queue** and visits nodes in rings of increasing distance — so in an unweighted graph (one whose edges all cost the same) the first time it reaches a node is via a shortest path. **Depth-first search (DFS)** uses a **stack** (or recursion, which uses the call stack) and goes as deep as possible before backing up — good for "explore everything", connected components (groups of nodes that can all reach each other), cycle detection and path enumeration. **Dijkstra's algorithm** is BFS generalised to non-negative edge weights (a cost on each edge, such as a distance), with a priority queue instead of a plain queue.

A graph is usually given as an edge list; build an **adjacency list** `List<List<Integer>>` (for each node, the list of nodes it links to). A grid is an implicit graph: each cell's neighbours are the four cells reached by the `DIRS` offsets.

Recognise it when: tree depth or level order; "number of islands/regions"; "fewest moves/steps" (BFS); "can you reach"; weighted "cheapest route" (Dijkstra).

How it works (BFS)

1. Put the start in the queue and mark it seen.
2. Poll a node; for each unseen neighbour, mark it seen **when you enqueue it** and record its distance.
3. For level-by-level output, read `q.size()` at the start of each level and poll exactly that many.

```java
// code/ch19/patterns/Graphs.java (key parts)
record TreeNode(int val, TreeNode left, TreeNode right) {}

static int depth(TreeNode n) {                       // DFS, recursive
    return n == null ? 0 : 1 + Math.max(depth(n.left()), depth(n.right()));
}

static List<List<Integer>> levels(TreeNode root) {  // BFS, level by level
    List<List<Integer>> out = new ArrayList<>();
    Deque<TreeNode> q = new ArrayDeque<>();
    if (root != null) q.offer(root);
    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int size = q.size(); size > 0; size--) {
            TreeNode n = q.poll();
            level.add(n.val());
            if (n.left() != null) q.offer(n.left());
            if (n.right() != null) q.offer(n.right());
        }
        out.add(level);
    }
    return out;
}

static final int[][] DIRS = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

static int shortestPath(String[] maze) {            // BFS on a grid, '#' = wall
    int rows = maze.length, cols = maze[0].length();
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, -1);    // -1 = not seen
    Deque<int[]> q = new ArrayDeque<>();
    q.offer(new int[]{0, 0});
    dist[0][0] = 0;
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        if (cur[0] == rows - 1 && cur[1] == cols - 1) return dist[cur[0]][cur[1]];
        for (int[] d : DIRS) {
            int nr = cur[0] + d[0], nc = cur[1] + d[1];
            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) continue;
            if (maze[nr].charAt(nc) == '#' || dist[nr][nc] != -1) continue;
            dist[nr][nc] = dist[cur[0]][cur[1]] + 1;
            q.offer(new int[]{nr, nc});
        }
    }
    return -1;
}

static long[] dijkstra(int n, int[][] edges, int src) {   // edges = {from, to, weight}
    // ... build List<List<int[]>> g
    long[] dist = new long[n];
    Arrays.fill(dist, Long.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<long[]> pq = new PriorityQueue<>(Comparator.comparingLong(x -> x[0]));
    pq.offer(new long[]{0, src});
    while (!pq.isEmpty()) {
        long[] top = pq.poll();
        int u = (int) top[1];
        if (top[0] > dist[u]) continue;             // stale entry: a shorter path was found already
        for (int[] e : g.get(u)) {
            long nd = dist[u] + e[1];
            if (nd < dist[e[0]]) { dist[e[0]] = nd; pq.offer(new long[]{nd, e[0]}); }
        }
    }
    return dist;
}
```

`islands` in the same file counts connected groups of `'1'` cells with an **explicit** `ArrayDeque` **stack** instead of recursion, so a 1000 × 1000 grid can't cause `StackOverflowError` (thrown when the call stack, the JVM's record of active method calls, runs out of space); `dfs` shows the recursive adjacency-list

version. Java's `PriorityQueue` has no "decrease key", so Dijkstra re-inserts a node with its new distance and skips stale entries on the way out ("lazy deletion").

Output:

```
ok  tree depth -> 3
ok  level order -> [[3], [9, 20], [15, 7]]
ok  islands -> 3
ok  maze BFS -> 4
ok  maze blocked -> -1
ok  graph DFS component -> [0, 1, 2, 3]
ok  dijkstra -> [0, 3, 1, 4]
7 checks passed
```

Complexity: BFS/DFS O(V + E) (a grid: O(rows · cols)); Dijkstra O((V + E) log V).

Common mistakes

- Marking a cell seen when it's **polled** rather than when it's **offered** — the same cell gets queued many times.
- `new boolean[rows][cols]` vs `[cols][rows]` swapped; `r < rows` vs `c < cols` bound mix-ups.
- Recursive DFS on huge inputs → `StackOverflowError`.
- Dijkstra with negative edge weights (wrong answers — use Bellman-Ford, a slower shortest-path algorithm that allows them); forgetting the stale-entry check (still correct, but slower).
- Building an undirected graph (edges work in both directions) with only one direction of each edge.

19.17 Topological sort

What it is

An ordering of a **directed acyclic graph** (DAG) in which every edge `u → v` puts u before v — a valid order to take courses, run build steps or apply database migrations (ordered, scripted changes to a database's structure). It exists only if there's no cycle, so the same algorithm doubles as cycle detection.

Recognise it when: prerequisites, dependencies, "order of tasks", "can all courses be finished", build systems, alien-dictionary letter order.

How it works (Kahn's algorithm)

1. Count each node's **in-degree** (number of incoming edges).
2. Queue every node with in-degree 0 — nothing has to come before them.
3. Poll a node, append it to the order, and decrement its neighbours' in-degrees; enqueue any that reach 0.
4. If the order has fewer than n nodes, the rest are stuck in a cycle.

```java
// code/ch19/patterns/TopoSort.java (key parts)
static List<Integer> kahn(int n, int[][] prereqs) {        // {a, b}: b before a
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < n; i++) g.add(new ArrayList<>());
    int[] indegree = new int[n];
    for (int[] p : prereqs) { g.get(p[1]).add(p[0]); indegree[p[0]]++; }

    Deque<Integer> ready = new ArrayDeque<>();
    for (int i = 0; i < n; i++) if (indegree[i] == 0) ready.offer(i);
    List<Integer> order = new ArrayList<>();
    while (!ready.isEmpty()) {
        int u = ready.poll();
        order.add(u);
        for (int v : g.get(u)) if (--indegree[v] == 0) ready.offer(v);
    }
    return order.size() == n ? order : List.of();          // leftovers are in a cycle
}

// DFS version: colour 0 = unvisited, 1 = on the current path, 2 = finished.
private static boolean visit(List<List<Integer>> g, int u, int[] colour, Deque<Integer> finished) {
    colour[u] = 1;
    for (int v : g.get(u)) {
        if (colour[v] == 1) return false;                  // back edge -> cycle
        if (colour[v] == 0 && !visit(g, v, colour, finished)) return false;
    }
    colour[u] = 2;
    finished.push(u);                                      // reverse post-order
    return true;
}
```

Output:

```
ok  kahn order -> [0, 1, 2, 3]
ok  kahn cycle -> []
ok  dfs order -> [0, 2, 1, 3]
ok  dfs cycle -> []
4 checks passed
```

Both `[0, 1, 2, 3]` and `[0, 2, 1, 3]` are valid — a DAG usually has many topological orders. If the problem wants the *smallest* order lexicographically (in dictionary order), swap the queue for a `PriorityQueue`.

Complexity: O(V + E) time and space.

Common mistakes

- Reversing the edge direction (read `{a, b}` carefully).
- In DFS, using a single `visited` flag: it can't tell "on the current path" (a cycle) from "finished earlier" (fine) — that's why there are three colours.

19.18 Union-find (disjoint set union)

What it is

A structure that tracks which elements belong to the same group and merges groups quickly. Each group is a tree; its root is the group's representative. `find(x)` walks to the root; `union(a, b)` links one root under the other. With **path compression** (make nodes point closer to the root while finding) and **union by size** (hang the smaller tree under the larger), each operation costs effectively O(1) — formally O(α(n)), the inverse Ackermann function, which is below 5 for any realistic n.

Recognise it when: "number of connected components" with edges arriving over time; "does adding this edge create a cycle?"; grouping accounts/emails that share an attribute; Kruskal's minimum spanning tree (the cheapest set of edges that connects every node without forming a cycle).

How it works

1. Start with every element as its own root (`parent[i] = i`).
2. `find`: follow `parent` pointers to the root, halving the path as you go.
3. `union`: find both roots; if equal they were already connected (return false); otherwise attach the smaller tree to the larger and decrement the component count.

```
// code/ch19/patterns/UnionFind.java (key parts)
int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];        // path halving: point to grandparent
        x = parent[x];
    }
    return x;
}

boolean union(int a, int b) {
    int ra = find(a), rb = find(b);
    if (ra == rb) return false;               // already connected: this edge closes a cycle
    if (size[ra] < size[rb]) { int t = ra; ra = rb; rb = t; }
    parent[rb] = ra;                          // smaller tree under the larger
    size[ra] += size[rb];
    components--;
    return true;
}

static int kruskal(int n, int[][] edges) {    // minimum spanning tree weight
    int[][] sorted = edges.clone();
    Arrays.sort(sorted, Comparator.comparingInt(e -> e[2]));
    UnionFind uf = new UnionFind(n);
    int total = 0;
    for (int[] e : sorted) if (uf.union(e[0], e[1])) total += e[2];
    return total;
}
```

Output:

```
ok  components -> 3
ok  0 and 1 connected -> true
ok  1 and 3 separate -> true
ok  redundant edge -> [2, 3]
ok  kruskal MST -> 19
5 checks passed
```

Complexity: O(α(n)) amortised per operation; O(n) space. Kruskal is O(E log E) for the sort.

Common mistakes

- Comparing `parent[a] == parent[b]` instead of `find(a) == find(b)`.
- Linking `a` to `b` instead of **root** to root.
- Using union-find for **directed** reachability — it only models undirected connectivity.

19.19 Dynamic programming

What it is

Dynamic programming (DP) solves a problem by combining answers to smaller versions of the same problem, computing each smaller answer **once**. It applies when a problem has **optimal substructure** (the best answer is built from best answers to subproblems) and **overlapping subproblems** (plain recursion would recompute the same subproblem many times — e.g. naive Fibonacci — the sequence 1, 1, 2, 3, 5, … where each number is the sum of the two before — is exponential, roughly doubling in cost with each step of n).

Two ways to write it:

	Memoisation (top-down)	Tabulation (bottom-up)
Shape	Recursive function + cache	Loops filling an array
Computes	Only the states actually reached	Every state in the table
Order	Natural — recursion finds it	You must choose a loop order where dependencies come first
Risk in Java	`StackOverflowError` on deep recursion; `HashMap` memo boxes keys	Off-by-one indexes
Space optimisation	Hard	Easy — keep only the rows you need

Java has no `@cache` decorator (unlike Python's `functools.cache`, a one-line wrapper that remembers a function's results); use an array (fast; pick a sentinel such as 0 or -1 that can't be a real answer) or a `HashMap` keyed by a record for multi-part states. Don't use `computeIfAbsent` with a recursive mapping function on a `HashMap` — modifying the map inside its own mapping function throws `ConcurrentModificationException` since Java 9 (memoisation note in §11.8); use get-then-put as in `editDistance`.

Recognise it when: "number of ways", "minimum/maximum cost", "longest/shortest … subsequence" (elements kept in their original order but not necessarily adjacent), "can you reach/make", choices at each step whose effects add up, and a brute force (try-every-possibility solution) that branches.

How to design a DP (the five questions)

1. **State**: what does `dp[i]` (or `dp[i][j]`) mean, in words?
2. **Transition**: how is a state computed from smaller ones?
3. **Base cases**.
4. **Order**: which loop direction guarantees dependencies are ready?
5. **Answer**: which state holds it? Then shrink space if only the last row/two values are used.

```java
// code/ch19/patterns/DynamicProgramming.java (key parts)
// 1D, top-down: ways to climb n stairs taking 1 or 2 steps
static long climbMemo(int n, long[] memo) {
    if (n <= 1) return 1;
    if (memo[n] != 0) return memo[n];
    return memo[n] = climbMemo(n - 1, memo) + climbMemo(n - 2, memo);
}
// 1D, bottom-up in O(1) space
static long climbTab(int n) {
    long prev2 = 1, prev1 = 1;
    for (int i = 2; i <= n; i++) { long cur = prev1 + prev2; prev2 = prev1; prev1 = cur; }
    return prev1;
}

// Coin change: dp[x] = fewest coins that make x
static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++)
        for (int c : coins)
            if (c <= x && dp[x - c] != Integer.MAX_VALUE)   // guard: MAX_VALUE + 1 would overflow
                dp[x] = Math.min(dp[x], dp[x - c] + 1);
    return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
}

// 2D: longest common subsequence, dp[i][j] = LCS of a[0..i) and b[0..j)
static int lcs(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    for (int i = 1; i <= a.length(); i++)
        for (int j = 1; j <= b.length(); j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1)
                    ? dp[i - 1][j - 1] + 1
                    : Math.max(dp[i - 1][j], dp[i][j - 1]);
    return dp[a.length()][b.length()];
}

// 0/1 knapsack in one row: iterate capacity DOWNWARDS so each item is used at most once
static int knapsack(int[] weight, int[] value, int capacity) {
    int[] dp = new int[capacity + 1];
    for (int item = 0; item < weight.length; item++)
        for (int c = capacity; c >= weight[item]; c--)
            dp[c] = Math.max(dp[c], dp[c - weight[item]] + value[item]);
    return dp[capacity];
}
```

The file also has house robber (`dp[i] = max(dp[i-1], dp[i-2] + a[i])` in two variables), longest increasing subsequence in O(n log n) with `Arrays.binarySearch` on a `tails` array, edit distance top-down with a `HashMap<Key, Integer>` memo (`record Key(int i, int j)`), and grid paths with a single rolling row.

Output:

```
ok  climb memo n=50 -> 20365011074
ok  climb tab n=50 -> 20365011074
ok  house robber -> 12
ok  coin change 15 -> 3
ok  coin change impossible -> -1
ok  LIS -> 4
ok  LCS -> 3
ok  edit distance -> 3
ok  0/1 knapsack -> 9
ok  grid paths -> 28
10 checks passed
```

Note `coinChange({1, 5, 11}, 15) = 3` (5 + 5 + 5): a greedy "largest coin first" would take 11 + 1 + 1 + 1 + 1 = 5 coins — the classic reason this problem needs DP. And `climb(50)` is 20,365,011,074, beyond `int` range — hence `long`.

Complexity: states × work per state. Climbing O(n); coin change O(amount · coins); LCS and edit distance O(m · n); knapsack O(items · capacity).

Common mistakes

- Undefined state ("dp[i] is… something about i") — write the sentence first.
- 0/1 knapsack with the capacity loop going **up** (items get reused — that's the unbounded version).
- `Integer.MAX_VALUE + 1` as "impossible + one coin" wrapping to negative.
- `int` answers for counting problems that exceed 2^{31} — use `long` or the requested modulus (report the remainder after dividing by a big prime, `% 1_000_000_007`).

19.20 Backtracking

What it is

A systematic search that builds a candidate one choice at a time, recurses, then **undoes** the choice to try the next one. Think of walking a maze with chalk: mark a turn, explore, and rub the mark out when you come back. Pruning (abandoning a branch as soon as it can't succeed) is what makes it fast enough.

Recognise it when: "all subsets / permutations / combinations", "all valid arrangements", N-Queens (place N chess queens so that none attacks another), Sudoku, word search in a grid, partitioning a string; small n (≤ ~20).

How it works (template)

1. If the current path is a complete answer, record a **copy** of it.
2. For each available choice: **choose** (add to path, mark used) → **explore** (recurse) → **un-choose** (remove, unmark).
3. Skip choices that already violate a constraint (prune).

```java
// code/ch19/patterns/Backtracking.java (key parts)
private static void subsets(int[] nums, int start, List<Integer> path, List<List<Integer>> out) {
    out.add(new ArrayList<>(path));                  // COPY: path keeps changing
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]);                           // choose
        subsets(nums, i + 1, path, out);             // explore
        path.removeLast();                           // un-choose (Java 21; or remove(size()-1))
    }
}

private static void combo(int[] c, int remaining, int start, List<Integer> path, List<List<Integer>> out) {
    if (remaining == 0) { out.add(List.copyOf(path)); return; }
    for (int i = start; i < c.length && c[i] <= remaining; i++) {   // sorted input: prune
        path.add(c[i]);
        combo(c, remaining - c[i], i, path, out);                    // i, not i+1: reuse allowed
        path.removeLast();
    }
}

private static int place(int n, int row, boolean[] cols, boolean[] diag, boolean[] anti) {  // N-Queens
    if (row == n) return 1;
    int count = 0;
    for (int c = 0; c < n; c++) {
        int d = row - c + n, a = row + c;            // each diagonal has a unique index
        if (cols[c] || diag[d] || anti[a]) continue;
        cols[c] = diag[d] = anti[a] = true;
        count += place(n, row + 1, cols, diag, anti);
        cols[c] = diag[d] = anti[a] = false;
    }
    return count;
}
```

`permutations` uses a `boolean[] used` instead of a start index, since every element can appear at any position.

Output:

```
ok  subsets -> [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
ok  permutations count -> 6
ok  permutations of 2 -> [[1, 2], [2, 1]]
ok  combination sum -> [[2, 2, 3], [7]]
ok  8 queens -> 92
5 checks passed
```

Complexity: subsets O(2ⁿ · n), permutations O(n! · n); pruning improves the practical cost, not the worst case.

Common mistakes

- `out.add(path)` without copying: every stored answer is the same list object, empty at the end.
- `path.remove(x)` on a `List<Integer>` with an `int` x removes by **index** (§19.6). `removeLast()` (Java 21) or `remove(path.size() - 1)` is unambiguous.
- Forgetting to un-mark `used[i]` or the diagonal flags.
- Duplicates in the input: sort, then skip `nums[i] == nums[i - 1]` at the same depth.

19.21 Tries

What it is

A **trie** (prefix tree, pronounced "try") stores strings character by character: each node has one child per possible next character and a flag saying "a word ends here". Words sharing a prefix share the path, so "does any word start with `car`?" costs O(length of `car`) no matter how many words are stored. Real uses: autocomplete, spell-check, IP routing tables (longest-prefix match: a router picks the most specific matching network address), word games.

Recognise it when: many prefix queries; autocomplete; "word search II" (many words in one grid); replacing words by their shortest root.

How it works

1. Start at the root. For each character, follow (or create, when inserting) the child for that character.
2. At the end of an insert, set `isWord = true`.
3. `contains` needs the path **and** `isWord`; `startsWith` only needs the path.

```java
// code/ch19/patterns/Trie.java (key parts)
private static final class Node {
    final Node[] next = new Node[26];     // lowercase a-z; HashMap<Character, Node> for big alphabets
    boolean isWord;
    int prefixCount;                      // how many inserted words pass through this node
}
private final Node root = new Node();

void insert(String word) {
    Node n = root;
    for (char ch : word.toCharArray()) {
        int i = ch - 'a';
        if (n.next[i] == null) n.next[i] = new Node();
        n = n.next[i];
        n.prefixCount++;
    }
    n.isWord = true;
}

private Node walk(String s) {
    Node n = root;
    for (char ch : s.toCharArray()) {
        n = n.next[ch - 'a'];
        if (n == null) return null;
    }
    return n;
}

boolean contains(String word)     { Node n = walk(word); return n != null && n.isWord; }
boolean startsWith(String prefix) { return walk(prefix) != null; }
```

`complete(prefix)` walks to the prefix node and then does a DFS over children `a`…`z` with a shared `StringBuilder` (append, recurse, `deleteCharAt` — backtracking again), so results come out in alphabetical order.

Output:

```
ok  contains car -> true
ok  ca is only a prefix -> true
ok  startsWith ca -> true
ok  count prefix car -> 3
ok  autocomplete car -> [car, card, care]
ok  autocomplete none -> []
6 checks passed
```

Complexity: insert and lookup O(L) for a word of length L; space O(total characters × alphabet) with arrays — a `HashMap` per node uses less memory for sparse alphabets but is slower.

Common mistakes

- Treating "path exists" as "word exists" (`ca` is only a prefix).
- `ch - 'a'` on uppercase or non-letters → negative index `ArrayIndexOutOfBoundsException`. Normalise input or use a map.

19.22 Linked lists and fast/slow pointers

What it is

A **singly linked list** is a chain of nodes, each holding a value and a reference `next` to the following node; the last node's `next` is `null`. There is no index: reaching the k-th node means walking k steps. Interview list problems are therefore about **pointer bookkeeping** — which references to keep so that you never lose the rest of the chain. Two tools do most of the work:

- A **dummy (sentinel) node** placed in front of the head, so "delete or insert at the head" is not a special case.

- **Fast and slow pointers**: two references walking at different speeds (usually 1 and 2 steps) or with a fixed gap. When fast reaches the end, slow is at the middle; if the list loops, fast eventually laps slow. This is **Floyd's cycle-finding algorithm** ("tortoise and hare"). Analogy: two runners on a track — on a straight road the fast one simply finishes, on a loop it comes round and catches the slow one.

§19.8 already shows `reverse`, `middle` and `hasCycle`; this section adds the other classic list problems.

Recognise it when: "linked list" in the signature; k-th node from the end; middle; cycle or where it starts; two lists that merge; palindrome list in O(1) space; any sequence `x → f(x)` that might repeat (happy number, "find the duplicate number").

How it works

1. **k-th from the end in one pass**: move `lead` k steps ahead of `trail`, then move both until `lead` is at the last node; `trail` is now just **before** the node to remove.
2. **Intersection**: pointer `p` walks list A then list B, `q` walks B then A. Both travel `lenA + lenB` steps, so they arrive at the first shared node together (or both reach `null`).
3. **Palindrome in O(1) space**: find the middle with fast/slow, reverse the second half, compare the halves, then reverse it back so the caller's list is unchanged.
4. **Where does the cycle start?** After slow and fast meet, start a new pointer at the head and step it and slow one node at a time; they meet at the cycle's first node. Why: if the tail before the loop has length a and they met b steps into the loop of length L, slow walked a + b and fast 2(a + b), and the difference a + b is a multiple of L — so a steps from the meeting point also lands on the loop's start.

```java
// code/ch19/patterns/LinkedLists.java (key parts)
static Node removeKthFromEnd(Node head, int k) {
    Node dummy = new Node(0);                  // a dummy in front makes "remove the head" a normal case
    dummy.next = head;
    Node lead = dummy, trail = dummy;
    for (int i = 0; i < k; i++) lead = lead.next;          // open a gap of k
    while (lead.next != null) { lead = lead.next; trail = trail.next; }
    trail.next = trail.next.next;              // trail stops just BEFORE the node to delete
    return dummy.next;
}

static Node intersection(Node a, Node b) {
    Node p = a, q = b;
    while (p != q) {                           // reference comparison: same node object, not same value
        p = (p == null) ? b : p.next;
        q = (q == null) ? a : q.next;
    }
    return p;
}

static boolean isPalindrome(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    Node secondHalf = reverse(slow);
    boolean ok = true;
    for (Node p = head, q = secondHalf; q != null; p = p.next, q = q.next)
        if (p.val != q.val) { ok = false; break; }
    reverse(secondHalf);                       // put the caller's list back the way it was
    return ok;
}

static Node cycleStart(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) {
            Node p = head;
            while (p != slow) { p = p.next; slow = slow.next; }
            return p;
        }
    }
    return null;
}
```

The file also has `isHappy(n)`: a number is "happy" if repeatedly replacing it by the sum of the squares of its digits reaches 1. The sequence either hits 1 or repeats forever, so fast/slow on **numbers** (`slow = next(slow)`, `fast = next(next(fast))`) detects the loop without a `HashSet`.

Output:

```
ok  remove 2nd from end -> 1->2->3->5
ok  remove the head -> 2
ok  intersection node -> 8
ok  no intersection -> null
ok  palindrome odd -> true
ok  list restored after palindrome check -> 1->2->3->2->1
ok  palindrome even -> true
ok  not a palindrome -> false
ok  cycle starts at 2 -> 2
ok  no cycle -> null
ok  19 is happy -> true
ok  2 is not happy -> false
12 checks passed
```

Complexity: all O(n) time, O(1) extra space (intersection O(m + n)).

Common mistakes

- Losing the rest of the list: save `next` **before** you overwrite `cur.next`.

- No dummy node, so removing the head needs its own `if` (and is usually the bug).
- Comparing nodes with `equals` or by `val` when the question is "the same node" — use `==` on references.
- Checking `fast.next.next` without `fast != null && fast.next != null` first → `NullPointerException`.
- Leaving the input modified (a reversed half) when the caller still uses the list.

19.23 Trees: recursion templates, LCA and serialisation

What it is

A **binary tree** node has a value and up to two children. Almost every tree problem is solved by recursion in one of two shapes:

- **Top-down**: pass information **down** as parameters (the allowed range, the depth so far, the path so far). Used for "validate a binary search tree", "root-to-leaf path sum".
- **Bottom-up**: each call **returns** a summary of its subtree (height, "did I find p?", best downward path) and the parent combines its children's answers. Used for height, diameter, balance check, lowest common ancestor, maximum path sum. When the answer is not the same thing a parent needs, return one value and record the overall best in a side variable.

A **binary search tree (BST)** keeps every key in the left subtree smaller and every key in the right subtree larger than the node — for the **whole** subtree, not just the direct children. The **lowest common ancestor (LCA)** of p and q is the deepest node that has both below it (a node counts as its own descendant). **Serialising** a tree means turning it into a string that can be turned back into the identical tree. §19.16 has depth and level-order BFS; this section adds the recursion templates.

Recognise it when: "tree" plus height, depth, balanced, diameter, path sum, "is it a valid BST", "lowest common ancestor", "serialize / deserialize", "build the tree from traversals".

How it works

1. **Validate a BST (top-down)**: each node must lie strictly inside a range `(low, high)`. Going left, the node's value becomes the new `high`; going right, the new `low`. Use `long` bounds so a key equal to `Integer.MIN_VALUE` or `Integer.MAX_VALUE` is still allowed.
2. **LCA (bottom-up)**: return the node itself if it is p or q; otherwise ask both children. If both sides found something, this node is where p and q split — it is the LCA. Otherwise pass up whichever side found something.
3. **Maximum path sum (bottom-up with a side result)**: a call returns the best path going **down** from its node (a parent can extend only one branch); at each node also record the best path that **bends** through it (left + node + right). Negative branches are replaced by 0 — better not to take them.
4. **Serialise**: preorder (visit the node, then its left subtree, then its right) and write `#` for every `null`. The null markers make the shape unambiguous, so one traversal is enough. **Deserialise** reads tokens (the separate pieces of the string) in the same order with a queue.

```java
// code/ch19/patterns/Trees.java (key parts)
record TreeNode(int val, TreeNode left, TreeNode right) {}

private static boolean valid(TreeNode n, long low, long high) {        // top-down
    if (n == null) return true;
    if (n.val() <= low || n.val() >= high) return false;
    return valid(n.left(), low, n.val()) && valid(n.right(), n.val(), high);
}

static TreeNode lca(TreeNode root, int p, int q) {                      // bottom-up
    if (root == null || root.val() == p || root.val() == q) return root;
    TreeNode left = lca(root.left(), p, q), right = lca(root.right(), p, q);
    if (left != null && right != null) return root;       // p and q split here
    return left != null ? left : right;
}

private static int gain(TreeNode n, int[] best) {                       // bottom-up + side result
    if (n == null) return 0;
    int left = Math.max(0, gain(n.left(), best));          // a negative branch is better skipped
    int right = Math.max(0, gain(n.right(), best));
    best[0] = Math.max(best[0], n.val() + left + right);   // path that bends at n
    return n.val() + Math.max(left, right);                // a parent can extend only one side
}

private static void write(TreeNode n, StringJoiner out) {               // serialise: preorder with "#"
    if (n == null) { out.add("#"); return; }
    out.add(String.valueOf(n.val()));
    write(n.left(), out);
    write(n.right(), out);
}

private static TreeNode read(Deque<String> tokens) {                    // deserialise: same order
    String t = tokens.poll();
    if (t.equals("#")) return null;
    TreeNode left = read(tokens);
    TreeNode right = read(tokens);
    return new TreeNode(Integer.parseInt(t), left, right);
}
```

The node is a `record`, so it is immutable (children are built first, then the parent) and two trees with the same shape and values are `equals` — which makes the round-trip check a one-liner. `int[] best` is the usual Java trick for a mutable "side result" inside a recursive helper (a field works too).

Output:

```
ok  valid BST -> true
ok  grandchild breaks the BST rule -> false
ok  MAX_VALUE key is legal -> true
ok  lca(5, 1) -> 3
ok  lca(7, 4) -> 2
ok  lca where one is the ancestor -> 5
ok  max path 15-20-7 -> 42
ok  all negative -> -3
ok  serialize -> 3,5,6,#,#,2,7,#,#,4,#,#,1,0,#,#,8,#,#
ok  deserialize round trip (records compare by value) -> true
ok  empty tree -> null
11 checks passed
```

Complexity: each is O(n) time — every node visited once — and O(h) stack space, where the height h is O(log n) for a balanced tree (every node's left and right sides have similar heights) and O(n) for a degenerate one (a "linked list" tree of 10^5 nodes can throw `StackOverflowError`; then use an explicit `ArrayDeque`, §19.16).

Common mistakes

- Validating a BST by comparing each node only with its direct children — a grandchild can break the rule (the second check above).

- `int` bounds with `Integer.MIN_VALUE`/`MAX_VALUE` as "no limit" — a legal key equal to them is rejected; use `long` or `Integer` with `null`.
- Starting `best` at 0 in max path sum — an all-negative tree must return its largest (negative) value.
- Serialising without null markers, or with preorder for write and a different order for read.
- Forgetting that a node can be its own ancestor in LCA.

19.24 Greedy

What it is

A **greedy algorithm** builds the answer one step at a time and at each step takes the choice that looks best **right now**, never undoing it. It is fast (often one pass after a sort) but only correct when the problem has the **greedy-choice property**: some optimal answer starts with the locally best choice. Unlike DP (§19.19), which tries all choices for each subproblem, greedy commits to one. Analogy: paying an amount with the fewest coins by always taking the largest coin that fits — right for euro coins, wrong for coins {1, 3, 4} and amount 6 (greedy gives 4 + 1 + 1, the best is 3 + 3), which is exactly why coin change is a DP problem.

Proving a greedy choice is safe usually takes one of two arguments: **exchange** ("take any optimal answer that didn't make my choice; swapping my choice in doesn't make it worse") or **stays ahead** ("after every step my partial answer is at least as good as any other").

Recognise it when: "minimum number of …", "maximum number of non-overlapping …", "can you reach …", jump game, gas station, candies, assigning cookies/boats/meeting rooms, intervals sorted by end time (§19.15), and when constraints ($n = 10^5$) rule out DP over pairs.

How it works

1. **Jump game** (each value is the maximum jump from that index): sweep left to right keeping `farthest`, the furthest index reachable so far. If you ever stand on an index beyond `farthest`, you're stuck. For the **fewest jumps**, treat it as BFS without a queue: the indexes reachable with j jumps form a range; when you reach its end, take another jump and the new range ends at `farthest`.
2. **Gas station** (circular route; `gas[i]` fuel at station i, `cost[i]` to reach the next): if total gas < total cost, no start works. Otherwise sweep with a running tank; whenever it drops below 0 at station i, no station from the current start to i can be the start (each would arrive at i with even less), so restart at i + 1.
3. **Candies** (every child gets ≥ 1; a child with a higher rating than a neighbour gets more than that neighbour): one pass left→right satisfies the left-neighbour rule, one pass right→left satisfies the right-neighbour rule, taking the `max` so the first pass's constraint is kept.

```java
// code/ch19/patterns/Greedy.java (key parts)
static boolean canJump(int[] jumps) {
    int farthest = 0;
    for (int i = 0; i < jumps.length; i++) {
        if (i > farthest) return false;                    // stuck: i is beyond every reachable index
        farthest = Math.max(farthest, i + jumps[i]);
    }
    return true;
}

static int minJumps(int[] jumps) {
    int count = 0, end = 0, farthest = 0;
    for (int i = 0; i < jumps.length - 1; i++) {
        farthest = Math.max(farthest, i + jumps[i]);
        if (i == end) { count++; end = farthest; }         // the current range is used up: jump
    }
    return count;
}

static int gasStation(int[] gas, int[] cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        int diff = gas[i] - cost[i];
        total += diff;
        tank += diff;
        if (tank < 0) { start = i + 1; tank = 0; }        // no station in [start..i] can work
    }
    return total >= 0 ? start : -1;
}

static int candies(int[] ratings) {
    int n = ratings.length;
    int[] c = new int[n];
    Arrays.fill(c, 1);
    for (int i = 1; i < n; i++)
        if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
    for (int i = n - 2; i >= 0; i--)
        if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
    int sum = 0;
    for (int x : c) sum += x;
    return sum;
}
```

Output:

```
ok  canJump yes -> true
ok  canJump blocked by 0 -> false
ok  minJumps -> 2
ok  minJumps already there -> 0
ok  gas station start -> 3
ok  gas station impossible -> -1
ok  candies 2,1,2 -> 5
ok  candies equal neighbours -> 4
ok  candies 1,2,3,4,1 -> 11
9 checks passed
```

Complexity: jump game and gas station O(n) time, O(1) space; candies O(n) time, O(n) space. Greedy problems that sort first are O(n log n).

Common mistakes

- Using greedy without a reason it is safe — try a small counter-example (like coins {1, 3, 4}) before coding; if one exists, it's DP.
- In `minJumps`, looping to the last index too — you'd count one extra jump when a range ends exactly there.
- Candies in one pass, or overwriting the first pass's value instead of taking `max`.
- Gas station with a nested loop trying every start: O(n²) and times out.

19.25 Sort and search: quicksort, quickselect, Dutch flag, merge sort on a list

What it is

You normally call `Arrays.sort` (§19.5), but interviews ask you to **write** the pieces of sorting algorithms because they solve other problems:

- **Partition**: rearrange an array around a **pivot** value so that everything smaller is on its left. It is the core of **quicksort** (partition, then sort both sides) and of **quickselect** (partition, then continue on the **one** side that holds the k-th element — average O(n), no full sort). The **Lomuto** partition used here keeps a `store` index: everything left of it is `< pivot`.
- **Three-way partition** (the **Dutch national flag** problem, named by Edsger Dijkstra after the flag's three stripes): sort an array of only 0s, 1s and 2s in one pass.
- **Merge sort** splits in half, sorts each half and **merges** two sorted runs. It needs no random access (jumping straight to any index in O(1)), so it is the natural way to sort a **linked list** in O(n log n).

Recognise it when: "k-th largest/smallest" without needing the rest sorted; "sort colors", "move all x to the end", "partition around a value"; "sort a linked list"; "median without sorting"; "count inversions" (a merge-sort variant).

How it works

1. **Partition**: swap a **random** element to the end as the pivot (random pivots make the O(n²) worst case, e.g. already-sorted input, practically impossible). Walk `i` over the rest; each value smaller than the pivot is swapped to `store++`. Finally swap the pivot into `store` — its final sorted position.
2. **Quickselect for the k-th largest**: the target index in ascending order is `n - k`. Partition; if the pivot landed on the target you're done, otherwise narrow `lo`/`hi` to the side containing it.
3. **Dutch flag**: three regions `[0, low)` = 0s, `[low, mid)` = 1s, `(high, end]` = 2s, and `[mid, high]` still unknown. Look at `a[mid]`: 0 → swap to `low`, advance both; 1 → advance `mid`; 2 → swap with `high`, shrink `high`, and **don't** advance `mid` (the value just swapped in hasn't been looked at).
4. **Merge sort on a list**: find the middle with fast/slow (§19.22), cut the list there, sort each half recursively, and merge by relinking nodes behind a dummy head.

```java
// code/ch19/patterns/SortSearch.java (key parts)
static int partition(int[] a, int lo, int hi) {
    swap(a, lo + RNG.nextInt(hi - lo + 1), hi);          // random pivot defeats sorted-input worst case
    int pivot = a[hi], store = lo;
    for (int i = lo; i < hi; i++) if (a[i] < pivot) swap(a, i, store++);
    swap(a, store, hi);
    return store;
}

static int kthLargest(int[] input, int k) {               // quickselect
    int[] a = input.clone();                              // don't reorder the caller's array
    int target = a.length - k;
    int lo = 0, hi = a.length - 1;
    while (true) {
        int p = partition(a, lo, hi);
        if (p == target) return a[p];
        if (p < target) lo = p + 1; else hi = p - 1;
    }
}

static void sortColors(int[] a) {                         // Dutch national flag
    int low = 0, mid = 0, high = a.length - 1;
    while (mid <= high) {
        if (a[mid] == 0) swap(a, low++, mid++);
        else if (a[mid] == 1) mid++;
        else swap(a, mid, high--);                        // don't advance mid
    }
}

static Node mergeSort(Node head) {                        // merge sort on a linked list
    if (head == null || head.next == null) return head;
    Node slow = head, fast = head.next;                   // slow stops at the END of the left half
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    Node right = slow.next;
    slow.next = null;                                     // cut the list in two
    return merge(mergeSort(head), mergeSort(right));
}
```

`quickSort` is the three-line recursion around `partition`, and `merge` is the classic two-pointer merge with `<=` so equal values keep their order (stable). `RNG` is a `Random` with a fixed seed (the starting value of the random-number generator — the same seed gives the same sequence) so runs are repeatable.

Output:

```
ok  quickSort -> [-3, 0, 1, 2, 5, 5, 6, 9]
ok  quickselect 2nd largest -> 5
ok  quickselect with duplicates -> 4
ok  input left untouched -> [3, 2, 1, 5, 6, 4]
ok  Dutch national flag -> [0, 0, 1, 1, 2, 2]
ok  merge sort list -> [1, 2, 3, 4]
ok  merge sort odd length -> [-1, 0, 3, 4, 5]
7 checks passed
```

Complexity: quicksort O(n log n) average, O(n²) worst, O(log n) stack on average; quickselect O(n) average, O(n²) worst; Dutch flag O(n), O(1) space; merge sort on a list O(n log n) time, O(log n) recursion stack. Compare quickselect with the size-k heap of §19.13 (O(n log k), streams well, never quadratic) — say both in the interview.

Common mistakes

- A fixed pivot (first or last element) on sorted or all-equal input → O(n²) and deep recursion. Randomise; for many duplicates a three-way partition helps.
- Mixing up k-th **largest** (index `n - k`) with k-th smallest (index `k - 1`).
- Advancing `mid` after swapping with `high` in the Dutch flag.
- Finding the middle with `fast = head` in list merge sort — for two nodes the split never shrinks and recursion never ends.

- Quickselect reordering the caller's array without saying so — clone, or state it.

19.26 Bit manipulation

What it is

An `int` is 32 bits in **two's complement** (the top bit is the sign; `-x == ~x + 1`). Bitwise operators work on all 32 bits at once in one CPU instruction (a single basic step of the processor): `&` (and), `|` (or), `^` (xor: 1 when the bits differ), `~` (not), `<<` (shift left, × 2 per step), `>>` (arithmetic shift right, copies the sign bit) and `>>>` (logical shift right, fills with 0) — see §2.8. Interview problems use a handful of identities:

Identity	Meaning
`x ^ x == 0`, `x ^ 0 == x`	equal values cancel under xor
`x & (x - 1)`	x with its lowest 1-bit cleared
`x & -x`	only the lowest 1-bit of x
`(x >> i) & 1`	bit i of x
`x | (1 << i)`, `x & ~(1 << i)`, `x ^ (1 << i)`	set, clear, toggle bit i
`1 << n`	2^n, and the number of subsets of n items

Java also has the library calls `Integer.bitCount`, `Integer.highestOneBit`, `Integer.numberOfTrailingZeros` and `Integer.toBinaryString` — say you'd use them in real code.

Recognise it when: "every element appears twice except one"; "without extra space" with integers; count 1-bits; powers of two; "swap without a temp"; subsets of up to ~20 items (a bitmask — an integer whose bits act as on/off flags — per subset); flags packed into one number.

How it works

1. **Single number**: xor everything together — pairs cancel and the lone value remains. O(n), O(1) space, no `HashSet`.
2. **Counting bits for 0..n**: a tiny DP — `i >> 1` is i without its last bit, so `bits[i] = bits[i >> 1] + (i & 1)`.
3. **Swap odd and even bits**: mask the even positions with `0x55555555` (binary 0101…) and the odd with `0xAAAAAAAA` (1010…), shift each one place toward the other, and `|` them. Use `>>>` so a set sign bit doesn't copy itself in.
4. **Power of two**: exactly one 1-bit, so `n > 0 && (n & (n - 1)) == 0`.
5. **Subsets by mask**: loop `mask` from 0 to $2^n - 1$; bit i of `mask` says whether item i is in this subset — an iterative alternative to backtracking (§19.20).

```java
// code/ch19/patterns/BitManipulation.java (key parts)
static int singleNumber(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}

static int[] countBits(int n) {
    int[] bits = new int[n + 1];
    for (int i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);
    return bits;
}

static int swapOddEvenBits(int x) {
    int even = x & 0x55555555;                // bits 0, 2, 4, ...
    int odd  = x & 0xAAAAAAAA;                // bits 1, 3, 5, ...
    return (even << 1) | (odd >>> 1);
}

static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

static List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> out = new ArrayList<>();
    for (int mask = 0; mask < (1 << nums.length); mask++) {
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < nums.length; i++) if ((mask >> i & 1) == 1) s.add(nums[i]);
        out.add(s);
    }
    return out;
}
```

Output:

```
ok  singleNumber -> 4
ok  singleNumber negative -> -7
ok  countBits 0..5 -> [0, 1, 1, 2, 1, 2]
ok  matches Integer.bitCount -> 2
ok  swap odd/even bits of 184 -> 116
ok  sign bit moves right, no smear -> 1073741824
ok  64 is a power of two -> true
ok  0 is not -> false
ok  MIN_VALUE is not (negative) -> false
ok  2^3 subsets -> 8
ok  subsets by mask -> [[], [1], [2], [1, 2]]
11 checks passed
```

184 is `1011 1000` in binary; swapping each pair of bits gives `0111 0100` = 116.

Complexity: single number and counting bits O(n); the bit tricks O(1); subsets O($2^n \cdot n$).

Common mistakes

- Operator precedence: `x & 1 == 0` parses as `x & (1 == 0)` and doesn't compile for `int`; write `(x & 1) == 0`.
- `>>` on negative numbers when you meant `>>>` — the sign bit smears in.
- `1 << 32` is **1**, not 0 (the shift count is taken mod 32); use `1L << k` for 64-bit masks.
- Forgetting `n > 0` in the power-of-two test: `0 & -1 == 0` would say 0 is a power of two, and `Integer.MIN_VALUE` also has a single 1-bit.

19.27 Math and geometry

What it is

A small family of problems where the trick is arithmetic or coordinates rather than a data structure: walking a matrix in a spiral, reversing an integer's digits **without overflow**, counting points on one straight line, and closed-form (a direct formula) or recurrence (each answer computed from the previous one) answers such as the **Josephus problem** (n people in a circle, every k-th is removed; who survives?). The Java-specific part is almost always **overflow** (§19.6) and **exact arithmetic** — avoid `double` for anything you compare for equality.

Recognise it when: "spiral order", "rotate the matrix"; "reverse the digits", "string to integer", "return 0 if it overflows"; "points on the same line", slopes; "circle of people, every k-th"; GCD/LCM (greatest common divisor / least common multiple), primes (sieve — crossing out multiples to leave the primes), modular arithmetic (working with remainders) `mod 1_000_000_007`.

How it works

1. **Spiral order**: keep four boundaries `top`, `bottom`, `left`, `right`. Walk the top row left→right, then the right column down, then (if a row remains) the bottom row right→left, then (if a column remains) the left column up — shrinking the boundary after each side. The two `if`s stop a single remaining row or column being printed twice.
2. **Reverse an int**: pop the last digit with `% 10`, push it with `result * 10 + digit`. Before multiplying, check that `result` is within `Integer.MAX_VALUE / 10` and `Integer.MIN_VALUE / 10`; outside that range the next step overflows, so return 0. Java's `%` keeps the sign of the dividend, so negatives work without special-casing.
3. **Max points on a line**: fix each point as an anchor and group every later point by the **slope** to it. Represent the slope exactly as the reduced fraction `(dx/g, dy/g)` with g = gcd(|dx|, |dy|), normalised so dx ≥ 0 (and dy > 0 when dx = 0 — vertical lines). Pack the pair into a `long` key. The best group + 1 (the anchor) is the answer for that anchor.
4. **Josephus**: number the people 0…n−1. After the first removal the circle is a Josephus problem of size n − 1 whose numbering is shifted by k, so `J(1) = 0` and `J(n) = (J(n−1) + k) mod n` — an O(n) loop instead of simulating a list (O(n·k)).

```java
// code/ch19/patterns/MathGeometry.java (key parts)
static List<Integer> spiral(int[][] m) {
    List<Integer> out = new ArrayList<>();
    if (m.length == 0) return out;
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int c = left; c <= right; c++) out.add(m[top][c]);
        top++;
        for (int r = top; r <= bottom; r++) out.add(m[r][right]);
        right--;
        if (top <= bottom) { for (int c = right; c >= left; c--) out.add(m[bottom][c]); bottom--; }
        if (left <= right) { for (int r = bottom; r >= top; r--) out.add(m[r][left]); left++; }
    }
    return out;
}

static int reverse(int x) {
    int result = 0;
    while (x != 0) {
        int digit = x % 10;                    // negative for negative x
        x /= 10;
        if (result > Integer.MAX_VALUE / 10 || result < Integer.MIN_VALUE / 10) return 0;
        result = result * 10 + digit;
    }
    return result;
}

static int maxPointsOnLine(int[][] pts) {
    int best = Math.min(pts.length, 1);
    for (int i = 0; i < pts.length; i++) {
        Map<Long, Integer> slopes = new HashMap<>();
        for (int j = i + 1; j < pts.length; j++) {
            int dx = pts[j][0] - pts[i][0], dy = pts[j][1] - pts[i][1];
            int g = gcd(Math.abs(dx), Math.abs(dy));
            dx /= g; dy /= g;
            if (dx < 0 || (dx == 0 && dy < 0)) { dx = -dx; dy = -dy; }   // (1,2) and (-1,-2) are the same line
            long key = ((long) dx << 32) ^ (dy & 0xFFFFFFFFL);          // exact key, no double rounding
            best = Math.max(best, 1 + slopes.merge(key, 1, Integer::sum));
        }
    }
    return best;
}

static int josephus(int n, int k) {
    int survivor = 0;
    for (int size = 2; size <= n; size++) survivor = (survivor + k) % size;
    return survivor;
}
```

Why is the overflow check enough in `reverse` when `result` equals `MAX_VALUE / 10` exactly (214748364)? That can only happen on the last digit of a 10-digit input, and a 10-digit `int` starts with 1 or 2, so the digit added is at most 2 and `2147483642` still fits.

Output:

```
ok  spiral 3x3 -> [1, 2, 3, 6, 9, 8, 7, 4, 5]
ok  spiral 3x4 -> [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
ok  spiral one column -> [1, 2, 3]
ok  reverse 123 -> 321
ok  reverse -120 -> -21
ok  reverse 1534236469 overflows -> 0
ok  reverse near MIN_VALUE fits -> -2143847412
ok  three collinear -> 3
ok  four on one line -> 4
ok  vertical line -> 3
ok  Josephus n=7 k=3 (0-based) -> 3
ok  Josephus single person -> 0
12 checks passed
```

With 7 people and k = 3 the survivor is index 3 — person 4 when counting from 1.

Complexity: spiral O(rows · cols); reverse O(number of digits); max points O(n²) time, O(n) space (the gcd adds a log factor); Josephus O(n) time, O(1) space.

Common mistakes

- Slopes as `double dy / dx`: division by zero for vertical lines, `0.0` vs `-0.0`, and rounding that makes different slopes equal. Use reduced integer pairs.
- Checking overflow **after** `result * 10` — the value has already wrapped.
- `Math.abs(Integer.MIN_VALUE)` is still negative (§19.6) — an input like that breaks "take abs, reverse, restore sign" solutions.
- Spiral without the two `if`s: a single middle row or column comes out twice.
- Simulating Josephus by removing from an `ArrayList` — O(n²) — when the recurrence is O(n).

19.28 Talking through a solution in Java

Interviewers grade communication as much as code. A dependable rhythm:

1. **Clarify** (2 min): input sizes, value ranges (overflow?), duplicates, empty input, sorted?, what to return when there's no answer.
2. **Examples**: work one small example by hand, plus one edge case.
3. **Brute force first**, state its complexity, then name the pattern that improves it ("sorted → two pointers", "subarray sum with negatives → prefix sums + map").
4. **Code** with clear names and a signature (the method's name and parameter list) like the platform's (`public int[] twoSum(int[] nums, int target)`). Say out loud why you pick `ArrayDeque`, `long`, or `Integer.compare`.
5. **Test** by tracing your example through the code, then the edge cases: empty, one element, all equal, negatives, `Integer.MAX_VALUE`.
6. **State time and space** complexity, including the sort and the recursion stack.

Java-specific polish: import `java.util.*`; declare interface types (`Map<String, Integer> m = new HashMap<>()`); prefer `var` only where the type is obvious (§2.6); return `new int[0]` or `List.of()` rather than `null`; and don't reach for streams (Java's pipeline API for processing collections, Chapter 12) in tight loops — a plain loop is clearer to trace and avoids boxing (see Q28).

Part 3 — Java interview questions

19.29 Java interview Q&A

Short, correct answers — say the first sentence, then add detail if asked. Each "→" link goes to the section of this book that teaches the topic in full; for certification-level traps see the OCP (Oracle Certified Professional exam) Java 21 study notes (e.g. collections and generics), and for design-level questions (LLD, low-level design: how to shape classes and objects) the LLD handbook. The claims marked ✔ are demonstrated in `code/ch19/qa/InterviewFacts.java`, whose output is at the end of this section.

Strings, objects and the language

Q1. Why is `String` immutable? A `String`'s characters can never change after construction; "modifying" methods return new strings. This makes strings safe to share between threads without locks (mechanisms that let only one thread use something at a time), safe as `HashMap` keys (the hash can be cached in the object and never goes stale), safe to pool (many variables can point at one literal), and secure for things like file paths and class names that are checked and then used. → §7.1

Q2. What is the string pool, and how many objects does `new String("hi")` create? The pool (intern pool) is a JVM table, on the heap (the memory area where all objects live) since Java 7, holding one shared

`String` per distinct literal (a value written directly in the code, like `"hi"`) and compile-time constant. `new String("hi")` always creates a **new** heap object, plus the pooled `"hi"` literal if the class hadn't already caused it to be created — so "one or two". `==` compares references, so `"hi" == new String("hi")` is false; `intern()` returns the pooled copy. Concatenation at run time produces a new object; `"h" + "i"` with literals is folded at compile time and pooled. ✔ → §7.1

Q3. `String` vs `StringBuilder` vs `StringBuffer`? `String` is immutable; `StringBuilder` is a mutable, unsynchronised buffer (use it to build strings in loops — `+=` in a loop is O(n²)); `StringBuffer` is the old synchronised version, rarely useful because a single string is rarely built by several threads. → §7.3

Q4. What is the `equals`/`hashCode` contract, and what breaks if you override only `equals`? Equal objects **must** have equal hash codes (unequal objects may collide); `equals` must be reflexive, symmetric, transitive, consistent and `false` for `null` (x equals x; if x equals y then y equals x; if x equals y and y equals z then x equals z; and the answer doesn't change while the objects don't). Override only `equals` and two "equal" objects get different identity hash codes, land in different buckets (slots of the hash table's internal array), and `HashSet.contains`/`HashMap.get` fail to find them. ✔ → §6.13

Q5. `==` vs `equals`? `==` compares primitive values or, for objects, **references** (same object?); `equals` compares logical content if the class overrides it (`String`, wrappers, collections, records do; arrays don't). With wrappers, `Integer a = 127, b = 127; a == b` is true only because `Integer.valueOf` caches −128…127; with 128 it's false. → §7.6, §19.6

Q6. Is Java pass-by-value or pass-by-reference? Always pass-by-value. For objects the *value* passed is a copy of the reference: the method can mutate the object the caller sees, but reassigning the parameter doesn't affect the caller's variable. ✔ → §4.2

Q7. `final` vs `finally` vs `finalize`? `final` is a modifier: a final variable can't be reassigned, a final method can't be overridden, a final class can't be extended. `finally` is the block after `try` that runs whether or not an exception was thrown (a `return` inside it overrides the try's return — never do that ✔). `finalize()` was a GC callback (a method the garbage collector called before freeing an object) on `Object`; it's unpredictable and has been deprecated for removal since Java 18 (JEP 421 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java change) — use try-with-resources or `Cleaner`. → §5.8, §8.4

Q8. How do you make a class immutable? Make the class `final` (or give it private constructors), make all fields `private final`, set everything in the constructor, provide no setters, **defensively copy** mutable inputs and outputs (keep your own copy so the caller can't change it later) (e.g. `List.copyOf`), and don't let `this` escape during construction. A `record` gives you most of this, but its fields are only *shallowly* immutable. → §5.9, §6.8

Q9. Interface vs abstract class? An interface is a contract a class can implement *many* of; it has no instance state (only constants), and can have `default`, `static` (Java 8) and `private` (Java 9) methods. An abstract class can have fields, constructors and any access level, but a class can extend only one. Use an interface for a capability ("is `Comparable`"), an abstract class for sharing state and code among closely related classes. → §6.5, §6.6, §6.12

Q10. What is a record (Java 16), and what are its limits? A concise, transparent data carrier: `record Point(int x, int y) {}` generates private final fields, a canonical constructor (one taking every component), accessors `x()`/`y()`, and `equals`/`hashCode`/`toString` based on the components. Records are implicitly `final`, can't extend another class (they extend `java.lang.Record`) and can't declare extra instance fields, but they can implement interfaces, have static members, extra methods and a compact constructor (written without a parameter list) for validation. → §6.8

Q11. What are sealed classes (Java 17) for? `sealed interface Shape permits Circle, Square` restricts which classes may extend or implement a type. Each permitted subclass must be `final`, `sealed` or `non-sealed`. The benefit: the compiler knows every subtype, so a pattern-matching `switch` over a sealed type can be **exhaustive** (handle every possible case) without a `default`, and adding a new subtype makes non-exhaustive switches fail to compile. → §6.7, §3.6

Q12. Overloading vs overriding? Overloading = same name, different parameter lists, chosen at **compile time** from the static argument types. Overriding = a subclass redefines an inherited instance method with the same signature, chosen at **run time** from the object's actual class (dynamic dispatch). Static

methods can't be overridden — a same-signature static method in a subclass *hides* the parent's. → §4.3, §6.3

Collections

Q13. How does `HashMap` work internally? An array of buckets (default capacity 16, load factor 0.75 — how full the table may get before it doubles). `put` computes `hashCode()`, mixes the high bits into the low bits, and uses `hash & (capacity − 1)` as the bucket index. Colliding entries form a linked list; since Java 8, a bucket that grows past 8 entries becomes a red-black tree if the table has at least 64 buckets (otherwise the table resizes instead). When size exceeds capacity × load factor, the table doubles and entries are redistributed. Lookup compares the hash, then `equals`. Average O(1); worst case O(log n) for a tree bucket. → §10.5

Q14. What happens if you mutate a key after putting it in a `HashMap`? Its hash code changes but the entry stays in the bucket computed from the old hash, so `get` with the same object (or an equal one) usually returns `null` and the entry is effectively lost until you iterate. Use immutable keys (`String`, records of immutable values). → §10.5

Q15. `HashMap` vs `Hashtable` vs `Collections.synchronizedMap` vs `ConcurrentHashMap`? `HashMap`: not thread-safe (unsafe to change from several threads at once), allows one `null` key and `null` values. `Hashtable`: legacy, every method synchronized, no nulls. `synchronizedMap`: a wrapper with one lock around every call — you must still lock manually while iterating or doing check-then-act (test a condition, then act on it — another thread can change things in between). `ConcurrentHashMap`: designed for concurrency, scales with many threads, offers atomic (all-in-one, indivisible) compound operations (`merge`, `compute`, `putIfAbsent`), no nulls. → §14.12

Q16. How is `ConcurrentHashMap` thread-safe? Since Java 8: reads are lock-free (they read volatile fields); an insert into an empty bucket uses a CAS (compare-and-set) instruction; an update to a non-empty bucket locks only that bucket's first node with `synchronized`; resizing is done cooperatively by the threads that notice it. So writers to different buckets don't block each other. (Java 7 and earlier used a fixed number of locked "segments".) Iterators are **weakly consistent** — they never throw `ConcurrentModificationException` and may or may not reflect concurrent updates; `size()` is an estimate under concurrent change. Nulls are forbidden because `get` returning `null` must unambiguously mean "absent". → §14.12, §14.10

Q17. `ArrayList` vs `LinkedList`? `ArrayList` is a growable array: O(1) random access, amortised O(1) append, O(n) insert/remove in the middle, cache-friendly (elements sit side by side in memory, which the CPU cache loads quickly). `LinkedList` is doubly linked (each node points to both neighbours): O(1) at the ends or at an iterator's position, O(n) to reach an index, more memory per element. Default to `ArrayList`; for a queue or stack use `ArrayDeque`. → §10.2

Q18. `HashMap` vs `LinkedHashMap` vs `TreeMap`? How would you build an LRU cache? Unordered O(1); insertion- or access-ordered O(1); sorted O(log n) with navigation (`floorKey`, `ceilingKey`). LRU cache: `new LinkedHashMap<>(16, 0.75f, true)` (access order) and override `removeEldestEntry` to return `size() > capacity`. → §10.4

Q19. `Comparable` vs `Comparator`? `Comparable<T>` is implemented by the class itself (`compareTo`) and defines its single *natural* order; `Comparator<T>` is a separate strategy object (`compare`) so you can have many orders, including for classes you don't own. Build comparators with `comparing`/`thenComparing`/`reversed`, and never compare by subtraction (overflow). → §10.8, §19.5

Q20. Fail-fast vs fail-safe (weakly consistent) iterators? Fail-fast iterators (`ArrayList`, `HashMap`) track a modification count and throw `ConcurrentModificationException` on a best-effort basis if the collection is structurally changed except through the iterator itself. Concurrent collections' iterators are weakly consistent (`ConcurrentHashMap`) or snapshot-based (`CopyOnWriteArrayList`) and never throw it. Remove safely with `Iterator.remove()` or `removeIf`. ✔ → §10.7

Q21. Why prefer `ArrayDeque` to `Stack`? `Stack` extends `Vector`: every call is synchronised (cost for nothing), it exposes list methods such as `get(i)` that break LIFO, and it iterates bottom-to-top. `ArrayDeque` is an unsynchronised circular array with O(1) `push`/`pop`, and the `Stack` Javadoc itself recommends `Deque`. → §19.4, §10.6

Q22. `List.of` vs `Arrays.asList` vs `Collections.unmodifiableList`? `List.of` (Java 9) is truly unmodifiable and rejects nulls. `Arrays.asList` is a fixed-size **view** of an array: `set` works (and writes through to

the array), `add`/`remove` throw. `unmodifiableList` is a read-only **view**: you can't change it through the view, but changes to the backing list (the original it reads from) show through. `List.copyOf` (Java 10) takes an unmodifiable snapshot. ✔ → §10.10

Exceptions

Q23. Checked vs unchecked exceptions? Checked exceptions (subclasses of `Exception` but not `RuntimeException`, e.g. `IOException`) must be caught or declared with `throws` — the compiler enforces it; use them for recoverable conditions the caller should handle. Unchecked exceptions (`RuntimeException` and subclasses, e.g. `IllegalArgumentException`, `NullPointerException`) signal programming errors and needn't be declared. `Error`s (`OutOfMemoryError`) are serious JVM problems you normally don't catch. → §8.2, §8.3

Q24. How does try-with-resources work? Any `AutoCloseable` (an object with a `close()` method) declared in `try (…)` is closed automatically when the block exits, normally or by exception, in **reverse** order of declaration ✔. If the body throws and `close()` also throws, the body's exception is propagated and the close exception is attached as a *suppressed* exception (`getSuppressed()`), instead of replacing it as a hand-written `finally` would. → §8.6

Q25. Can a `finally` block not run? Only if the JVM stops or the thread never leaves the `try`: `System.exit`, a JVM crash or kill, an infinite loop or deadlock inside the `try`, or a daemon thread (a background thread that doesn't keep the JVM alive) killed at JVM shutdown. An exception or `return` in the `try` doesn't skip it. → §8.4

Generics, lambdas and streams

Q26. What is type erasure, and what can't you do because of it? The compiler checks generic types, then erases them to their bounds (usually `Object`) and inserts casts; at run time `ArrayList<String>` and `ArrayList<Integer>` are the same class ✔. Consequences: no `new T()`, no `new T[n]`, no `instanceof List<String>`, no primitive type arguments (`List<int>`), and two overloads that differ only in type arguments clash. → §9.7, §9.8

Q27. What does `? extends` vs `? super` mean (PECS)? "Producer `extends`, consumer `super`": a `List<? extends Number>` can be read as `Number`s but you can't add to it; a `List<? super Integer>` accepts `Integer`s but reads come back as `Object`. Example: `Collections.copy(List<? super T> dest, List<? extends T> src)`. → §9.6

Q28. Streams vs loops — which is faster, which is better? For simple work on small collections a loop is usually as fast or faster (no pipeline objects, no boxing if you use primitives). Streams win on readability for declarative (say what you want, not how to loop) transforms (filter–map–group) and make parallelism a one-word change — though parallel streams only pay off for large, CPU-bound (limited by computation rather than by waiting), easily split work. Loops are better for early exits with complex state, checked exceptions, and index arithmetic (most DSA code). → §12.11, §12.8

Q29. How do you misuse `Optional`? Calling `get()` without checking (it throws `NoSuchElementException` ✔ — use `orElse`, `orElseGet`, `orElseThrow`, `map`, `ifPresent`); using it for fields, method parameters or collection elements (it's designed as a *return type* for "maybe no result"); returning `Optional<List<…>>` instead of an empty list; returning `null` from a method declared to return `Optional`; and `orElse(expensiveCall())` when `orElseGet(() -> expensiveCall())` would avoid evaluating it every time. → §11.7

Q30. What is a functional interface, and why must captured variables be effectively final? An interface with exactly one abstract method (e.g. `Runnable`, `Function<T,R>`), which a lambda or method reference can implement; `@FunctionalInterface` makes the compiler check it. Lambdas capture the *value* of local variables, so Java requires those locals to be final or never reassigned — otherwise the lambda's copy and the variable could silently disagree (and lambdas may run on other threads). → §11.2, §11.5

Concurrency

Q31. `volatile` vs `synchronized`? `volatile` on a field guarantees **visibility and ordering**: every read sees the latest write, and writes before a volatile write are visible to a thread that later reads it. It does **not** make compound actions like `count++` atomic. `synchronized` gives **mutual exclusion** (one thread in the block per lock) *and* visibility on lock release/acquire. Use `volatile` for a flag or a safely published reference (one other threads are guaranteed to see fully built), locks or atomics (`AtomicInteger`) for read-modify-write. → §14.6, §14.7

Q32. What is "happens-before"? The Java Memory Model's rule for when one thread's write is guaranteed visible to another thread's read. Key edges: program order within a thread; unlocking a monitor (the lock built into every object) → later locking of the same monitor; a volatile write → later read of that variable; `Thread.start()` → the started thread's actions; a thread's actions → another thread's successful `join()`. Without such an edge, the JIT and CPU may reorder or cache writes and the other thread may never see them. → §15.8

Q33. What is a deadlock and how do you prevent it? Two or more threads each hold a lock and wait for a lock another holds, forever. Prevent it by acquiring locks in one global order, holding them briefly, using `tryLock` with a timeout, or avoiding shared locks (immutable data, message passing). Diagnose with a thread dump (a snapshot of what every thread is doing; `jstack`), which reports found deadlocks. → §15.9

Q34. What are virtual threads (Java 21), and when do they help? Lightweight threads scheduled by the JVM onto a small pool of OS (operating-system, "carrier") threads. When a virtual thread blocks on I/O (input/output — waiting for a disk, network or database) it unmounts, freeing the carrier, so you can run hundreds of thousands of simple blocking-style tasks (`Executors.newVirtualThreadPerTaskExecutor()`). They help **I/O-bound, high-concurrency** servers; they don't make CPU-bound work faster. Don't pool them; limit access to scarce resources with a `Semaphore` (a counter of permits that caps how many threads may enter). Before Java 24 a virtual thread blocking inside `synchronized` pinned its carrier; JEP 491 (Java 24) removed that case, though native calls (calls into non-Java code such as C) back into Java and class initialisation still pin. → §15.4, §15.5

Q35. `Runnable` vs `Callable`; `Future` vs `CompletableFuture`? `Runnable.run()` returns nothing and can't throw checked exceptions; `Callable.call()` returns a value and can. A `Future` is a handle whose only way to get the result is a blocking `get()`. `CompletableFuture` (Java 8) can be completed explicitly and composed without blocking — `thenApply`, `thenCompose`, `thenCombine`, `allOf`, `exceptionally`/`handle`. → §15.1, §15.2

Q36. Why use an `ExecutorService` instead of `new Thread()`? It reuses a bounded number of platform threads (ordinary threads backed one-to-one by OS threads) (creation is expensive and unbounded threads exhaust memory), queues tasks, returns `Future`s, and gives one place to shut everything down. Size CPU-bound pools near the core count; for blocking I/O prefer virtual threads. Always shut executors down (they're `AutoCloseable` since Java 19). → §14.11

The JVM

Q37. What are the JVM's memory areas? The **heap** (all objects and arrays, shared, garbage-collected, usually split into young and old generations); one **stack per thread** (frames with local variables and partial results — `StackOverflowError` when too deep); **metaspace** (class metadata in native memory, replaced PermGen in Java 8); the **code cache** (JIT-compiled machine code); plus direct buffers and other native memory (memory outside the Java heap, managed by the OS). → §16.3, §1.8

Q38. How does garbage collection work, and which collector is the default? An object is garbage when it's no longer **reachable** from GC roots (thread stacks, static fields, JNI references — JNI is the Java Native Interface for calling C code). Collectors are **generational**: most objects die young, so the young generation is collected often and cheaply by copying survivors, and long-lived objects are promoted to the old generation. G1 is the default since Java 9 (Serial on very small machines until JDK 27 made G1 the default everywhere); Parallel maximises throughput (total work done per second); ZGC (generational-only since Java 24) and Shenandoah keep pauses at or below about a millisecond by working concurrently. → §16.5, §16.7

Q39. Can Java leak memory? Yes — the GC only frees *unreachable* objects, so anything you still reference stays: ever-growing static maps and caches without eviction (removing old entries), listeners never unregistered, `ThreadLocal` values (per-thread variables) on pooled threads, mutated `HashMap` keys, and class-loader leaks in app servers. Confirm with heap-after-GC growth; find them with class histograms, JFR (JDK Flight Recorder, a built-in low-overhead profiler) old-object samples, or a heap dump (a file listing every object in memory; `jcmd <pid> GC.heap_dump`) in Eclipse MAT (a heap-dump analyser). → §16.11

Q40. How does the JVM run bytecode — interpreter, JIT, class loading? Bytecode is the portable instruction set inside `.class` files. Class loaders load classes lazily (only when first needed) (delegating to their parent first), then the JVM verifies, links and initialises them. Bytecode starts in the interpreter; methods that become hot are compiled to machine code by the tiered JIT compilers

(C1 quickly, C2 with heavy optimisations like inlining — copying a called method's body into the caller — and escape analysis — proving an object never leaves a method so it needn't be allocated on the heap), which is why Java "warms up". → §1.7, §16.2, §16.8

Q41. What's `static`, and when is a static initialiser run? A `static` member belongs to the class, not to instances — one copy shared by all. Static initialisers and static field initialisers run once, when the class is first initialised (first instance created, static method called or non-constant static field used). → §5.5, §5.10

Q42. What changed in modern Java that you'd use in an interview? `var` (10), switch expressions (14), text blocks (multi-line string literals, 15), records and `instanceof` patterns (16), sealed types (17), pattern matching for `switch`, record patterns, virtual threads and sequenced collections with `getFirst`/`getLast`/`removeLast` (21), unnamed variables `_` (22), stream gatherers (custom stream steps, 24), and compact source files with instance `main` methods (25). → §17.2

Proof of the ✔ claims

```
ok  literals share one pooled object -> true
ok  new String makes a separate object -> true
ok  intern() returns the pooled copy -> true
ok  runtime concatenation creates a new object -> true
ok  compile-time constant concatenation is pooled -> true
ok  BadPoint equals says equal -> true
ok  but HashSet cannot find it (different hashCode) -> true
ok  record works as a key -> true
ok  return in finally overrides try -> 2
ok  try-with-resources order -> [open A, open B, body, close B, close A]
ok  same runtime class -> true
ok  adding during for-each throws ConcurrentModificationException -> true
ok  callee can mutate the object but not rebind the caller's variable -> x!
ok  view sees later changes, copy does not -> 2,1
ok  orElse instead of get() -> default
ok  get() on empty throws NoSuchElementException -> true
16 checks passed
```

`BadPoint` overrides `equals` but not `hashCode`; `javac -Xlint` (the compiler with extra warnings switched on) even warns about it (`[overrides] Class BadPoint overrides equals, but neither it nor any superclass overrides hashCode method`).

Self-check

1. Why does `new PriorityQueue<Integer>((a, b) -> b - a)` risk wrong answers? → `b - a` overflows for values far apart; use `Comparator.reverseOrder()`.
2. You need "the largest key ≤ x" repeatedly while inserting keys. Which structure? → `TreeMap.floorKey(x)` / `TreeSet.floor(x)`, O(log n).
3. A sliding-window solution fails when the array contains negative numbers. What pattern replaces it? → Prefix sums with a `HashMap` of prefix counts.
4. Why does the islands solution use an explicit `ArrayDeque` instead of recursion? → A large grid can recurse ~10^6 deep and throw `StackOverflowError`.
5. `Map<String, Integer> m`; `m.get("a") == m.get("b")` when both are 1000 — true or false? → Usually false: two different `Integer` objects outside the cache; compare with `equals`.
6. Removing the k-th node from the end fails only when k equals the list length. What's missing? → A dummy node in front of the head, so `trail` can stop before the head (§19.22).
7. A BST check compares each node only with its two children and passes a broken tree. Why? → The rule is for whole subtrees; pass a `(low, high)` range down instead (§19.23).
8. Coins {1, 3, 4}, amount 6: does "take the largest coin that fits" work? → No — it gives 4 + 1 + 1 (3 coins), the best is 3 + 3. No greedy-choice property, so use DP (§19.24, §19.19).
9. Why does `maxPointsOnLine` key slopes by a reduced `(dx, dy)` pair instead of `dy / (double) dx`? → Exact: no division by zero for vertical lines, no rounding collisions, no `-0.0` (§19.27).

Chapter TL;DR

- **Test with a helper, not** `assert` — the keyword is disabled unless you run `java -ea`.
- **Toolkit**: `int[]` + `Arrays` (sort, fill, copyOfRange, toString); `c - 'a'` for letter indexes; `StringBuilder` for building; `HashMap.merge` / `getOrDefault` / `computeIfAbsent`; `ArrayDeque` for stacks **and** queues (never `Stack`); `PriorityQueue` (min-heap; `Comparator.reverseOrder()` for max); `TreeMap` / `TreeSet` for `floor` / `ceiling`; comparators from `comparingInt` / `thenComparing`, never `a - b`.
- **Traps**: `int` overflow (use `long`, `Math.*Exact`, `lo + (hi - lo) / 2`); `Integer == Integer` outside −128…127; unboxing `null`; `list.remove(int)`; `%` with negatives (`Math.floorMod`); removing inside for-each; deep recursion.
- **Patterns**: two pointers (sorted/in-place/linked lists) · sliding window (contiguous, non-negative) · hashing (seen/count/group) · prefix sums (range sums, subarray sum = k) · stack/monotonic stack (nesting, next greater) · heap (top-k, merge k, median) · binary search (sorted, first-true, on the answer) · intervals (sort by start, sweep) · BFS (fewest steps) / DFS (explore) / Dijkstra (weights) · topological sort (dependencies, cycle check) · union-find (dynamic connectivity) · DP (state, transition, base, order, answer) · backtracking (choose, explore, un-choose, copy the path) · tries (prefix queries) · linked lists (dummy node; fast/slow for middle, cycle start, k-th from end) · trees (top-down ranges, bottom-up summaries; LCA, max path, preorder + `#` serialisation) · greedy (only with an exchange argument; jump game, gas station, candies) · sort & search (random-pivot partition, quickselect, Dutch flag, merge sort on lists) · bits (`^` cancels pairs, `x & (x - 1)`, masks, `>>>`) · math & geometry (spiral boundaries, overflow check before `* 10`, gcd-reduced slopes, Josephus recurrence). Two heaps handle running medians and "best among the currently eligible".
- **Interview talk**: clarify → example → brute force → pattern → code → trace tests → complexity.
- **Language Q&A** centres on a few ideas: immutability and the string pool, the `equals` / `hashCode` contract, `HashMap` / `ConcurrentHashMap` internals, checked vs unchecked, erasure, `volatile` vs `synchronized` and happens-before, virtual threads, JVM memory and GC — each linked above to its full chapter.
- Same patterns in Python: Python Chapter 19.
- Next: Chapter 20 — Cheat sheet and glossary for one-page reminders of the whole book.

Chapter 20 — Cheat Sheets and Glossary

This last chapter compresses Chapters 1–19 into pages you can scan in a minute. There is one **cheat sheet** per area of the language and platform, a **feature-by-version table** for Java 17 → 25, the build and test **commands** you type every day, an **index of the interview questions** answered in Chapter 19, a short **"where next"** guide, and an alphabetical **glossary** of 700+ terms, each linked to the section that explains it properly.

Nothing here is new. If a line doesn't make sense, follow its link back to the full explanation — and every technical term used on these sheets has a one-sentence definition in the glossary at the end (§20.19). References look like **§10.5**, meaning Chapter 10, section 5.

What you'll learn: how to use this chapter as a revision tool. Read a sheet, cover the right-hand column, and try to say the answer before you look. The glossary is a good last check before an interview: if you can't define a term in one sentence, follow its link.

Every concrete value on the sheets is tested. Primitive ranges, the `Integer` cache, the string pool, collection behaviours, stream results, `BigDecimal` and `java.time` edge cases, concurrency results and more are asserted by `code/ch20/CheatsheetChecks.java` (package `ch20` — a package is a named group of classes, like a folder), compiled with `javac --release 25` and run on JDK 26 by the book's runner (`bash tools_run_java.sh java`). A failed check throws `AssertionError`. Real output:

```
OK   20.1 syntax, types, primitives           29 checks
OK   20.2 operators & precedence              16 checks
OK   20.3 control flow & patterns             12 checks
OK   20.4 methods & arrays                    16 checks
OK   20.5 OOP                                 13 checks
OK   20.6 core APIs                           45 checks
OK   20.7 exceptions                          12 checks
OK   20.8 generics                             7 checks
OK   20.9 collections                         28 checks
OK   20.10 functional interfaces & Optional   12 checks
OK   20.11 streams & collectors               18 checks
OK   20.12 I/O & NIO.2                        11 checks
OK   20.13 concurrency                        13 checks
OK   20.14 JVM                                9 checks
OK   20.15 modern Java 17-25                  9 checks
all 250 checks passed on Java 26
```

The helper is three small methods — `eq(actual, expected, label)`, `yes(condition, label)` and `throwsA(ExceptionType.class, code, label)` — because Java's own `assert` keyword is switched off unless you run `java -ea` (§19.1). A few lines as a taste:

```
Integer i1 = 127, i2 = 127, i3 = 128, i4 = 128;
eq(i1 == i2, true, "Integer cache -128..127");
eq(i3 == i4, false, "outside the cache: different objects");
throwsA(IllegalStateException.class, () -> emps.stream().collect(Collectors.toMap(Emp::dept, Emp::name)),
        "toMap duplicate key throws");
eq(LocalDate.of(2024, 1, 31).plusMonths(1), LocalDate.of(2024, 2, 29), "plusMonths clamps to month end");
```

What's inside

§	Sheet	Built from
20.1	Syntax, types and primitive ranges	Ch 1, 2
20.2	Operators and precedence	Ch 2
20.3	Control flow, `switch` and patterns	Ch 3
20.4	Methods and arrays	Ch 4
20.5	OOP: classes, interfaces, records, enums, sealed types	Ch 5, 6
20.6	Core APIs: strings, numbers, dates	Ch 7
20.7	Exceptions	Ch 8
20.8	Generics and PECS	Ch 9
20.9	Collections: big-O and which to choose	Ch 10
20.10	Functional interfaces, lambdas and `Optional`	Ch 11
20.11	Streams and collectors	Ch 12
20.12	I/O and NIO.2	Ch 13
20.13	Concurrency	Ch 14, 15
20.14	The JVM: memory, GC, flags, tools	Ch 1, 16
20.15	Modern Java by version, 17 → 25	Ch 17
20.16	Testing, build and tooling commands	Ch 1, 18
20.17	Interview question index	Ch 19
20.18	Where next	Ch 18
20.19	Glossary A–Z	all

20.1 Syntax, types and primitive ranges

Program shapes (§1.5, §1.10, §4.9):

```
// Classic: file Hello.java, class name = file name
public class Hello {
    public static void main(String[] args) { System.out.println("Hi " + args.length); }
}

// Java 25 compact source file (JEP 512): no class declaration, instance main, java.lang.IO
void main() { IO.println("Hi"); }
```

Command	Does	Since
`javac Hello.java` → `java Hello`	compile to `Hello.class`, then run it on the JVM	1.0
`javac -d out $(find src -name '*.java')` → `java -cp out pkg.Main`	compile a tree into `out/`, run with a class path (the folders and JARs searched for classes)	—
`java Hello.java`	compile in memory and run a single source file	11
`java Main.java` (other files next to it)	launch a multi-file source program	22
`jshell`	interactive REPL (read-eval-print loop): type expressions, see results	9
`javap -c Hello`	disassemble bytecode (list the JVM instructions in a `.class` file)	—

Basic rules (§2.1): statements end with `;`; blocks use `{}`; Java is case-sensitive; identifiers (the names you give variables, methods and classes) start with a letter, `_` or `$` (a lone `_` is not a name — since 22 it means "unnamed"); comments are `//`, `/* */` and `/** Javadoc */` (documentation comments turned into HTML API docs) (Markdown `///` doc comments since 23). Naming: `ClassName`, `methodAndVariable`, `CONSTANT_NAME`, `com.example.pkg`. Contextual keywords — `var`, `record`, `sealed`, `non-sealed`, `permits`, `yield`, `when`, `module` — are keywords only in their positions.

The eight primitives (§2.3) — everything else is a reference type (§2.5):

Type	Bits	Range	Default (fields)	Literal
`byte`	8	−128 … 127	`0`	`(byte) 1`
`short`	16	−32 768 … 32 767	`0`	`(short) 1`
`int`	32	−2 147 483 648 … 2 147 483 647 (±2.1 × 10^9)	`0`	`42`, `0x2A`, `0b101010`, `052`, `1_000_000`
`long`	64	−9 223 372 036 854 775 808 … 9 223 372 036 854 775 807 (±9.2 × 10^{18})	`0L`	`42L`
`float`	32	±3.4028235 × 10^{38}, ~7 significant digits	`0.0f`	`3.14f`
`double`	64	±1.7976931348623157 × 10^{308}, ~15–16 digits	`0.0`	`3.14`, `1e-9`
`char`	16	0 … 65 535 (unsigned UTF-16 code unit: one 16-bit piece of text)	`'\u0000'`	`'A'`, `'\n'`, `'\u0041'`
`boolean`	—	`true`, `false`	`false`	`true`

- `Double.MIN_VALUE` (4.9E-324) is the smallest **positive** double; the most negative is `-Double.MAX_VALUE`.
- Fields get defaults (`0`, `false`, `null`); **local variables don't** — reading one before assigning it is a compile error (§2.2).
- Money: `long` cents or `BigDecimal` (an exact decimal-number class), never `double` (§2.11).

Conversions (§2.7):

Rule	Example	Result
Widening is automatic: `byte → short → int → long → float → double`, `char → int`	`long l = 42;`	ok
Narrowing needs a cast and keeps the low bits	`(byte) 200`	`-56`
Casting a floating value to an integer truncates toward zero	`(int) 3.99`	`3`
Arithmetic promotes `byte`/`short`/`char` to `int`	`byte b = 10; b = b + 1;`	compile error
Compound assignment hides a cast	`byte b = 10; b += 300;`	`54`
`char` is a number	`(int) 'A'`, `(char) ('a' + 1)`	`65`, `'b'`

Integer and floating-point facts (§2.10, §2.11):

Expression	Value	Why
`Integer.MAX_VALUE + 1`	`-2147483648`	silent wrap-around; use `Math.addExact` to get an `ArithmeticException`
`7 / 2`, `-7 / 2`	`3`, `-3`	integer division truncates toward zero
`-7 % 3` / `Math.floorMod(-7, 3)`	`-1` / `2`	`%` takes the dividend's sign
`7 / 2.0`	`3.5`	one `double` operand → `double` arithmetic
`0.1 + 0.2`	`0.30000000000000004`	binary fractions can't represent 0.1 exactly
`1.0 / 0`, `0.0 / 0`, `1 / 0`	`Infinity`, `NaN` (Not a Number), `ArithmeticException`	floats have special values; ints throw
`Double.NaN == Double.NaN`	`false`	test with `Double.isNaN`

`var` (Java 10, §2.6): local variables with an initializer only (also in `for` loops and lambda parameters); not for fields, parameters or return types; `var x = null;` and `var a = {1, 2};` don't compile. The type is fixed at compile time — `var` is not dynamic typing (types decided while the program runs, as in Python).

Text blocks (Java 15, §2.12): `"""` + newline opens; common indentation is stripped (the closing `"""` position counts); a closing delimiter on its own line adds a trailing `\n`; `\` at line end joins lines; `\s` keeps a trailing space.

Console I/O (§2.13, §13.8): `System.out.println/print/printf`; input with `Scanner(System.in)` (`nextInt`, `nextLine` — beware the leftover newline), `BufferedReader`, or `IO.readln("prompt")` in Java 25. `System.console()` returns `null` without an interactive terminal (IDEs, pipes) — always check it (Java 22–24 briefly returned a JLine-based console (JLine is a library for rich terminal input) even then; 25 reverted).

20.2 Operators and precedence

From highest to lowest (§2.8, §2.9). Operands are always evaluated **left to right**; associativity only decides grouping.

Level	Operators	Associativity
1 Postfix	`x++` `x--`	left
2 Unary	`++x` `--x` `+` `-` `~` `!` `(type)`	right
3 Multiplicative	`*` `/` `%`	left
4 Additive	`+` `-` (and `String` `+`)	left
5 Shift	`<<` `>>` `>>>`	left
6 Relational	`<` `>` `<=` `>=` `instanceof`	left
7 Equality	`==` `!=`	left
8–10 Bitwise	`&` then `^` then `|`	left
11–12 Logical	`&&` then `||` (short-circuit: stop once the answer is known)	left
13 Ternary	`? :`	right
14 Assignment, lambda	`=` `+=` `-=` `*=` `/=` `%=` `&=` `|=` `^=` `<<=` `>>=` `>>>=` `->`	right

Memory hook: **unary → multiply → add → shift → compare → equal → bitwise → logical → ternary → assign.**

Expression	Value	Lesson
`2 + 3 * 4`	`14`	`*` before `+`
`10 - 4 - 3`	`3`	left-associative
`1 + 2 + "3" + 4 + 5`	`"3345"`	numbers add until the first `String`, then everything concatenates
`true || false && false`	`true`	`&&` binds tighter than `||`
`x & 1 == 0`	compile error for `int x`	`==` binds tighter than `&` — write `(x & 1) == 0`
`i = 5; i++ + ++i`	`12`	`5 + 7`; never write this in real code
`-8 >> 1`, `-8 >>> 28`	`-4`, `15`	`>>` keeps the sign, `>>>` fills with zeros
`5 ^ 3`, `~5`, `1 << 10`	`6`, `-6`, `1024`	XOR, complement (−(n+1)), shift = ×2ⁿ
`a = b = c = 7`	all `7`	assignment is right-associative
`s != null && s.length() > 0` with `s = null`	`false`, no exception	`&&` stops at the first `false`
`o instanceof String str && str.length() == 4`	`true` for `"text"`	pattern variable in scope after `&&` (Java 16)

`==` on objects compares **references**; use `equals` for content (§6.13, §7.6).

20.3 Control flow, `switch` and patterns

Branches and loops (§3.1, §3.8, §3.9):

Construct	Shape	Notes
`if` / `else if` / `else`	`if (cond) { … } else { … }`	condition must be `boolean` — `if (1)` doesn't compile
Ternary	`cond ? a : b`	an expression; keep it short
`for`	`for (int i = 0; i < n; i++)`	index loops
Enhanced `for`	`for (String s : list)`	arrays and any `Iterable`; can't remove safely inside
`while` / `do … while`	test first / body runs at least once	
`break` / `continue`	leave / skip to next iteration of the innermost loop	`break outer;` / `continue outer;` with a label

The four `switch` shapes (§3.2, §3.3, §3.4):

Shape	Falls through (runs on into the next case)?	Produces a value?	Must be exhaustive?
Statement, `case X:`	**yes** — until `break`	no	no (unless it uses patterns or `null`)
Statement, `case X ->` (14)	no	no	no (same exception)
Expression, `case X ->` (14)	no	yes; blocks use `yield`	**yes**
Expression, `case X:` + `yield`	yes	yes	yes

```
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "weekday";
    case 6, 7 -> "weekend";
    default -> { yield "invalid"; }
};
```

Selector types (the type of the value being switched on): `int` and smaller (`char`, `byte`, `short`) and their wrappers, `String`, enums — and, with patterns (21), **any reference type**. `long`, `float`, `double` and `boolean` selectors need *primitive patterns*, which are still **preview** (shipped but switched off by default, and may still change) in Java 25.

Pattern matching (§3.5, §3.6, §3.7):

Feature	Syntax	Final in
Type pattern in `instanceof`	`if (o instanceof String s && !s.isEmpty())`	16
Type pattern in `switch`	`case Integer i ->`	21
Guard	`case Integer i when i < 0 ->`	21
`null` label	`case null ->` or `case null, default ->`	21
Record pattern (nestable)	`case Point(int x, int y) when x == y ->`	21
`var` in a record pattern	`case Point(var x, var y) ->`	21
Unnamed pattern / variable	`case Point(var x, _) ->`, `catch (Exception _)`, `for (var _ : list)`	22

Rules to remember: without `case null`, a `null` selector throws `NullPointerException`; the first matching label wins, and a label that can never match (dominated, e.g. `CharSequence` before `String`) is a compile error; a pattern `switch` must be exhaustive — add `default` or cover every permitted subtype of a **sealed** type:

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double side) implements Shape {}

static double area(Shape s) {
    return switch (s) {                // no default needed: the compiler knows all subtypes
        case Circle c -> Math.PI * c.r() * c.r();
        case Square q -> q.side() * q.side();
    };
}
```

Loop pitfalls (§3.10): off-by-one (`<=` vs `<`), modifying a collection inside for-each (→ `ConcurrentModificationException`), `float` loop counters, `==` on strings in conditions, and infinite `while` loops whose variable never changes.

20.4 Methods and arrays

Methods (§4.1–§4.5):

Topic	Rule
Signature	name + parameter types (not the return type, not parameter names)
Passing	**always by value**; for objects the value is a copy of the reference — the callee can mutate the object but not rebind the caller's variable
Overload resolution	phase 1: exact match or widening; phase 2: boxing/unboxing; phase 3: varargs. `over(5)` with `over(long)`, `over(Integer)`, `over(int...)` picks `long`
Varargs	`int... nums` is an `int[]`; must be the last parameter; one per method; zero arguments allowed
`return`	a non-`void` method must return on every path
Recursion	needs a base case; each call adds a stack frame → `StackOverflowError` if too deep (Java has no tail-call elimination — reusing the current frame when a method's last act is to call itself)
`static`	called on the class, has no `this` (§5.5)

Arrays (§4.6, §4.7):

```
int[] a = new int[3];             // [0, 0, 0] — elements get default values
int[] b = {3, 1, 2};              // initializer
String[] names = new String[2];   // [null, null]
int[][] grid = new int[3][4];     // rectangular
int[][] jagged = {{1}, {2, 3}, {}};  // rows of different lengths
a.length                           // a field, not a method; fixed forever
```

Index `0 … length − 1`; anything else → `ArrayIndexOutOfBoundsException`. Arrays are objects: `a.equals(b)` and `==` compare identity; `a.toString()` prints something like `[I@1b6d3586`.

`java.util.Arrays` (§4.8):

Method	Does	Example result
`toString(a)` / `deepToString(m)`	print 1D / nested	`[1, 2, 3]`, `[[1], [2, 3], []]`
`equals(a, b)` / `deepEquals`	compare contents	`true` for `{3,1,2}` vs `{3,1,2}`
`sort(a)`, `sort(a, from, to)`, `sort(T[], cmp)`	sort in place (dual-pivot quicksort for primitives, stable merge sort — TimSort — for objects)	
`binarySearch(a, key)`	on a **sorted** array; not found → `-(insertionPoint) − 1`	`binarySearch([1,2,3], 5)` = `-4`
`copyOf(a, n)`, `copyOfRange(a, from, to)`	new array, padded with defaults	`copyOf([1,2,3], 5)` = `[1, 2, 3, 0, 0]`
`fill(a, v)`, `setAll(a, i -> …)`	set every element	
`asList(…)`	fixed-size `List` **view** of the array — `add` throws, `set` writes through	
`stream(a)`	`IntStream` / `Stream<T>`	
`mismatch(a, b)`, `compare(a, b)`	first differing index / lexicographic (dictionary) order (9)	

Command-line arguments: `args` is never `null` (empty array when none); all elements are `String`s; the program name is not included (§4.9).

20.5 OOP: classes, interfaces, records, enums and sealed types

Access modifiers (§5.6):

Modifier	Same class	Same package	Subclass (other package)	Everywhere
`private`	✓			
(none) package-private	✓	✓		
`protected`	✓	✓	✓ (through inheritance)	
`public`	✓	✓	✓	✓

Top-level classes can only be `public` or package-private; one `public` top-level class per file, named like the file.

Constructors and initialisation (§5.3, §5.10):

- If you write no constructor, the compiler adds a no-argument one; write any constructor and it doesn't.
- `this(...)` calls another constructor of the same class, `super(...)` the parent's; if you write neither, `super()` is inserted. Since **Java 25** (JEP 513 — a JEP, JDK Enhancement Proposal, is the numbered design document for a Java feature) statements that don't use `this` — validation, computing arguments, assigning the class's own fields — may appear **before** `super(...)`/`this(...)`.
- Order: static fields and `static {}` blocks run **once**, in source order, when the class is initialised → for each `new`: parent constructor → instance field initialisers and `{}` blocks in source order → the rest of the constructor body.

`static` and `final` (§5.5, §5.8): `static` = one per class, shared; a static method has no `this`. `final` variable = assigned once; `final` method = can't be overridden; `final` class = can't be extended. `final` on a reference doesn't freeze the object.

Inheritance and overriding (§6.1–§6.4):

Overriding rule	Detail
Same signature	otherwise it's an overload; use `@Override` to let the compiler check
Return type	same or a subtype (covariant)
Access	same or wider (`protected` → `public` ok, not the reverse)
Checked exceptions	same, narrower or none — never new or broader
Not overridable	`private`, `static` (hidden instead), `final` methods; constructors
Dispatch	instance methods use the object's run-time class; fields and static methods use the compile-time type

```
class Base    { static String who() { return "Base"; }    String me() { return "Base"; }
                String call() { return me() + "/" + who(); } }
class Derived extends Base { static String who() { return "Derived"; }
                @Override String me() { return "Derived"; } }
new Derived().call()   // "Derived/Base"
```

Choosing a type (§6.12):

Need	Use	Key facts
Mutable object with identity and lifecycle	**`class`** (`final` unless designed for extension)	full control of encapsulation (hiding state behind methods)
Shared state/code for a family of related classes	**`abstract class`**	fields, constructors, any access; single inheritance; can't be instantiated (§6.5)
A capability many unrelated types can have	**`interface`**	many per class; `abstract`, `default` (8), `static` (8), `private` (9) methods; fields are `public static final` constants; no instance state (§6.6)
Immutable data where equal values are interchangeable	**`record`** (16)	private final fields, canonical constructor, accessors `x()`, `equals`/`hashCode`/`toString`; implicitly `final`; extends `Record`; no extra instance fields; compact constructor for validation (§6.8)
A fixed set of instances known at compile time	**`enum`**	fields, constructors (private), methods, per-constant bodies; `values()`, `valueOf`, `ordinal()`; `EnumSet`/`EnumMap` (§6.9)
A closed set of variants	**`sealed`** interface/class (17) + records	`permits` lists subtypes; each is `final`, `sealed` or `non-sealed`; exhaustive `switch` without `default` (§6.7)

```
record Range(int lo, int hi) {
    Range {                                          // compact constructor
        if (lo > hi) throw new IllegalArgumentException(lo + " > " + hi);
    }
}
new Point(1, 2).equals(new Point(1, 2))          // record Point(int x, int y): true;  toString() = "Point[x=1, y=2]"
```

Nested types (§6.10, §6.11):

Kind	Declared	Has an outer instance?	Typical use
Static nested class	`static class Node` inside a class	no	helpers, builders, `Map.Entry`-like types — the default choice
Inner class	`class Itr` inside a class	yes (`Outer.this`)	iterators that read the outer object's fields
Local class	inside a method body	yes if in an instance method	one-off helper with a name
Anonymous class	`new Interface() { … }`	yes if in an instance method	one-off implementation with state or several methods
Lambda	`x -> …`	captures effectively final locals	single-method (functional) interfaces

Nested records, enums and interfaces are implicitly `static`.

`Object` methods contract (§6.13): `equals` must be reflexive, symmetric, transitive, consistent, and `false` for `null`; equal objects **must** have equal `hashCode`s; override both or neither (`Objects.equals`, `Objects.hash` help). `toString` defaults to `ClassName@hexHash`. Immutability recipe (§5.9): `final` class, `private final` fields, no setters, defensive copies in and out (`List.copyOf`), don't leak `this` from the constructor. Design-level treatment: LLD handbook.

20.6 Core APIs: String, numbers, dates

Strings (§7.1, §7.2) are immutable; every "modifying" method returns a new `String`.

Fact	Value
`"java" == "java"` (two literals)	`true` — both are the pooled object
`"java" == new String("java")`	`false` — `new` always makes a new object
`"java" == new String("java").intern()`	`true`
`"java" == part + "va"` (run time, `part = "ja"`)	`false`; `"ja" + "va"` (constants) is folded → `true`
`"Java".substring(1, 3)`	`"av"` — end index exclusive
`"a,b,,".split(",").length` / `split(",", -1)`	`2` / `4` — trailing empties dropped unless limit is negative
`"a.b".replace(".", "-")` / `replaceAll(".", "-")`	`"a-b"` / `"---"` — `replaceAll` takes a **regex**
`" ".isBlank()` / `" ".isEmpty()`	`true` / `false`
`compareTo`	negative, zero or positive — not necessarily −1/1

Everyday methods: `length`, `charAt`, `indexOf` (−1 if absent), `contains`, `startsWith`/`endsWith`, `substring`, `strip` (Unicode-aware; prefer to `trim`), `toLowerCase(Locale.ROOT)` (a `Locale` is a language/region setting), `equalsIgnoreCase`, `split`, `String.join`, `repeat` (11), `lines` (11), `formatted` (15), `chars`, `codePoints`.

Builders (§7.3): `StringBuilder` (mutable, not synchronised — use it for loops: `append`, `insert`, `reverse`, `deleteCharAt`, `setLength(0)`); `StringBuffer` (synchronised, legacy); `StringJoiner` / `String.join` / `Collectors.joining` for delimited lists. `s += x` in a loop is O(n²).

Formatting (§7.4): `String.format(Locale.US, "%5.2f|%-4s|%04d|%,d", 3.14159, "ab", 42, 1234567)` → `" 3.14|ab  |0042|1,234,567"`. Conversions `%s %d %f %e %x %c %b %n %%`; flags `-` left, `0` pad, `,` grouping, `+` sign; `%2$s` argument index. Always pass a `Locale` when the output is parsed by machines.

`Math` (§7.5):

Call	Result	Note
`Math.round(2.5)`, `Math.round(-2.5)`	`3`, `-2`	ties toward +∞; returns `long` for `double`
`Math.rint(2.5)`	`2.0`	ties to even
`Math.floorDiv(-7, 2)`, `floorMod(-7, 3)`	`-4`, `2`	round toward −∞
`Math.abs(Integer.MIN_VALUE)`	`-2147483648`	overflow! `absExact` throws
`Math.clamp(15, 0, 10)`	`10`	Java 21
`addExact`, `multiplyExact`, `toIntExact`, `powExact` (25)	throw `ArithmeticException` on overflow	
`Math.random()`	`double` in [0.0, 1.0)	prefer `RandomGenerator`

Wrappers and autoboxing (§7.6):

Fact	Detail
`Integer a = 127, b = 127; a == b`	`true` — `Integer.valueOf` caches **`−128 … 127`** (also `Short`, `Byte`, `Long`, `Character` ≤ 127, both `Boolean`s)
`Integer c = 128, d = 128; c == d`	`false` — always compare wrappers with `equals`
`int x = (Integer) null`	`NullPointerException` on unboxing
`list.remove(1)` vs `list.remove(Integer.valueOf(1))`	removes **index** 1 vs the **value** 1
`Integer.parseInt("ff", 16)`	`255`; bad input → `NumberFormatException`
`Boolean.parseBoolean("YES")`	`false` — only `"true"` (any case) is true

`BigInteger` / **`BigDecimal`** (§7.7): immutable, arbitrary precision (as many digits as needed).

Rule	Example
Build from a `String` (or `BigDecimal.valueOf(double)`)	`new BigDecimal("0.1").add(new BigDecimal("0.2"))` = `0.3`; `new BigDecimal(0.1)` = `0.1000000000000000055511…`
`equals` checks value **and scale** (digits after the point); `compareTo` checks value only	`2.0` vs `2.00`: `equals` false, `compareTo` 0 — so `HashSet<BigDecimal>` can hold both
Non-terminating division needs a scale or `MathContext`	`ONE.divide(valueOf(3))` → `ArithmeticException`; `divide(valueOf(3), 4, HALF_UP)` = `0.3333`
Rounding modes	`2.345` → `HALF_UP` `2.35`, `HALF_EVEN` (banker's) `2.34`
`BigInteger`	`BigInteger.TWO.pow(100)` has 31 digits

`java.time` (§7.8–§7.11) — immutable, thread-safe (safe to share between threads); months are 1–12:

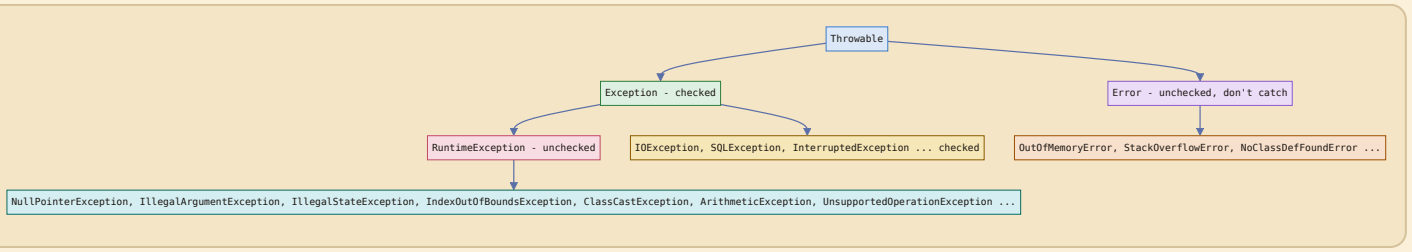
Class	Represents	Example
`LocalDate` / `LocalTime` / `LocalDateTime`	wall-calendar date/time, **no zone**	birthdays, opening hours
`ZonedDateTime`	date-time in a region (`ZoneId.of("Europe/Paris")`), DST-aware (handles daylight-saving clock changes)	meetings
`OffsetDateTime`	date-time with a fixed offset (`+02:00`)	API timestamps
`Instant`	a point on the UTC (world reference time) time-line	logs, DB timestamps
`Duration`	seconds + nanos (machine time)	`Duration.ofMinutes(90)` → `PT1H30M`
`Period`	years, months, days (calendar)	2024-01-15 → 2025-03-20 = `P1Y2M5D`
`DateTimeFormatter`	format/parse, thread-safe	`ofPattern("yyyy-MM-dd HH:mm")`

Edge cases (all checked): `LocalDate.of(2024, 1, 31).plusMonths(1)` → `2024-02-29` (clamped); in `America/New_York`, 2025-03-09 01:30 plus 1 hour → 03:30, and that day lasts 23 hours; `yyyy` vs `YYYY` for 2025-12-29 → `2025` vs `2026` (week-based year — a classic bug); `MM` month vs `mm` minute; `HH` 0–23 vs `hh` 1–12; `LocalDate.of(2025, 2, 29)` → `DateTimeException`. Legacy `Date`/`Calendar`/`SimpleDateFormat` are mutable and not thread-safe — convert at the edges.

Random numbers (§7.12): `new Random(42)` is reproducible (same seed → same sequence); `ThreadLocalRandom.current()` for concurrent code; `SecureRandom` for tokens and passwords; `RandomGenerator.of("L64X128MixRandom")` (17) for choosing an algorithm; `random.nextInt(lo, hi)` (17) excludes `hi`.

20.7 Exceptions

The hierarchy (§8.2):



Kind	Which classes	Compiler rule	Meaning
Checked	`Exception` and subclasses **except** the `RuntimeException` branch	must be caught or declared with `throws` (§8.3)	a recoverable problem outside the program's control (file, network, DB)
Unchecked	`RuntimeException` and subclasses	no rule	a programming error (bad argument, bad state, `null`)
Error	`Error` and subclasses (also unchecked)	no rule	the JVM or environment is in trouble — let it propagate

Common exception	Thrown when	Kind
`NullPointerException`	dereferencing `null` (using it to reach a field or method; helpful messages name the `null` part: Java 14, on by default since 15)	unchecked
`IllegalArgumentException` / `NumberFormatException`	bad argument / unparsable number	unchecked
`IllegalStateException`	object not in the right state (stream reused, `toMap` duplicate key)	unchecked
`ArrayIndexOutOfBoundsException`, `StringIndexOutOfBoundsException`	bad index	unchecked
`ClassCastException`, `ArithmeticException`	bad cast / integer `/ 0`, overflow in `*Exact`	unchecked
`UnsupportedOperationException`	mutating an unmodifiable collection	unchecked
`ConcurrentModificationException`	structural change during iteration	unchecked
`IOException`, `FileNotFoundException`, `NoSuchFileException`	I/O failure	**checked**
`InterruptedException`	a blocked thread was interrupted	**checked**
`SQLException`	JDBC (Java's database API) failure	**checked**
`UncheckedIOException`	an `IOException` wrapped for lambdas/streams	unchecked

Syntax rules (§8.4–§8.6):

Construct	Rule
`try { } catch (A e) { } finally { }`	`catch` or `finally` (or both) required; `finally` runs on normal exit, exception, `return`, `break`
Catch order	subclasses before superclasses, else "exception has already been caught" compile error
Multi-catch `catch (A | B e)`	`A` and `B` must not be subtype-related; `e` is effectively final
`return` in `finally`	overrides the `try`'s return **and swallows exceptions** — never do it (`try { return 1; } finally { return 2; }` returns `2`)
try-with-resources `try (var r = open())`	any `AutoCloseable`; closed automatically in **reverse** order; a close failure is added as **suppressed** to the body's exception (`getSuppressed()`)
`finally` doesn't run	only if the JVM exits (`System.exit`, crash) or the thread never leaves `try`
Rethrow	`catch (Exception e) { throw e; }` — precise rethrow keeps the narrower checked types
Chaining	`throw new ServiceException("load failed", e)` — keep the cause (§8.8)

```
try (Res a = new Res("A", log); Res b = new Res("B", log)) {
    throw new RuntimeException("body failed");
}   // log: open A, open B, body, close B, close A — both close() failures become suppressed
```

Custom exceptions (§8.7): extend `Exception` (checked) or `RuntimeException` (unchecked); provide `(String message)` and `(String message, Throwable cause)` constructors; add fields for context.

Best practices (§8.10): throw early (validate arguments with `Objects.requireNonNull`, `IllegalArgumentException`), catch late (where you can really handle it); never swallow (`catch (Exception e) {}`); don't use exceptions for control flow; catch the most specific type; translate low-level exceptions at layer boundaries (where, say, database code hands over to business code) and keep the cause; log **or** rethrow, not both; clean up with try-with-resources. Reading traces: the first line is the type and message, then frames newest-first; read `Caused by:` sections bottom-up to find the root (§8.9).

20.8 Generics and PECS

Syntax (§9.1–§9.5):

Form	Example
Generic class / record	`class Box<T> { T value; }`, `record Pair<A, B>(A first, B second) {}`
Generic interface	`interface Repository<T, ID> { Optional<T> findById(ID id); }`
Generic method	`static <T> List<T> repeat(T x, int n)` — type parameter before the return type
Explicit type argument	`Collections.<String>emptyList()` (rarely needed — inference)
Diamond	`Map<String, List<Integer>> m = new HashMap<>();`
Bounded type	`<T extends Number>`, several bounds: `<T extends Number & Comparable<T>>` (class first)
Recursive bound	`<T extends Comparable<? super T>> T max(List<? extends T> xs)`
Wildcards	`List<?>`, `List<? extends Number>`, `List<? super Integer>`

Naming convention: `T` type, `E` element, `K`/`V` key/value, `R` result, `ID` identifier.

Invariance and PECS (§9.6): `List<Integer>` is **not** a `List<Number>` (arrays are covariant — a `String[]` counts as an `Object[]` — and fail at run time with `ArrayStoreException`; generics refuse at compile time).

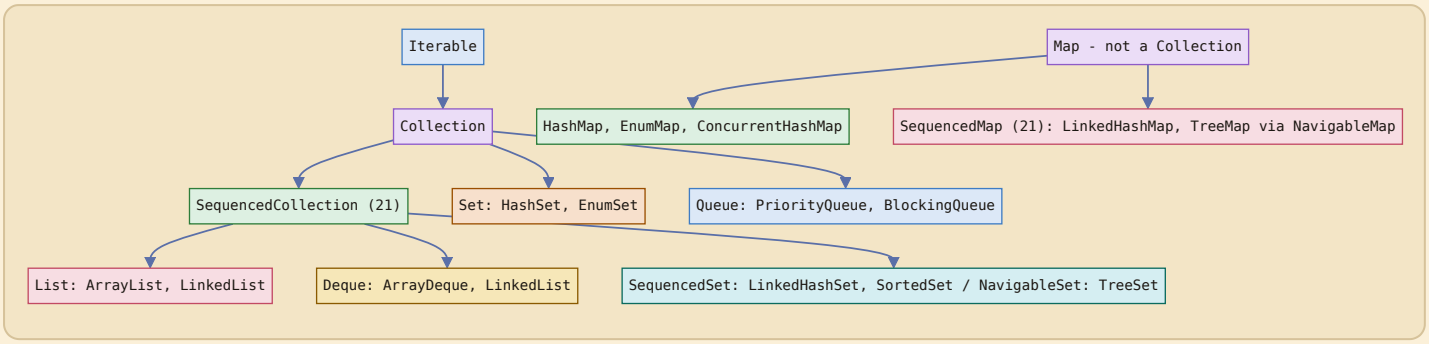
Wildcard	You can read as	You can add	Use when the parameter…	Example
`List<? extends Number>`	`Number`	nothing (except `null`)	**produces** values for you	`double sumAll(List<? extends Number> xs)`
`List<? super Integer>`	`Object`	`Integer`	**consumes** values you give it	`void fill(List<? super Integer> sink)` accepts `List<Number>`, `List<Object>`
`List<?>`	`Object`	nothing	you only use `Object` methods / size	`printAll(List<?> xs)`
`List<T>`	`T`	`T`	you both read and write	`swap(List<T> xs, int i, int j)`

Producer **E**xtends, **C**onsumer **S**uper — e.g. `Collections.copy(List<? super T> dest, List<? extends T> src)`. Don't use wildcards in return types.

Type erasure (§9.7, §9.8): generics are checked by the compiler, then erased to their bounds (`Object` if unbounded) with casts inserted. `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()` is `true`. So you **can't**: use primitives (`List<int>`), `new T()`, `new T[n]`, `instanceof List<String>`, use `T` in static members, overload `f(List<String>)` with `f(List<Integer>)` (same erasure), or make a generic `Throwable`. Workarounds: `Supplier<T>` factories, `Class<T>` tokens, `List<T>` instead of `T[]`. Raw types (`List raw`) switch checking off — they exist only for pre-Java-5 code. Eliminate unchecked warnings; suppress only on the smallest scope with a comment; mark safe generic varargs methods `@SafeVarargs`.

20.9 Collections: big-O and which to choose

The hierarchy (§10.1):



Big-O (§10.12) — `*` amortised (averaged over many calls), `†` average (worst case O(log n) in a tree-ified bucket):

Collection	add at end	add/remove at front	get(i)	contains / get(key)	remove element	Order	null ?
`ArrayList`	O(1)*	O(n)	O(1)	O(n)	O(n)	index	yes
`LinkedList`	O(1)	O(1)	O(n)	O(n)	O(n)	index	yes
`ArrayDeque`	O(1)*	O(1)*	—	O(n)	O(n)	head → tail	**no**
`PriorityQueue`	O(log n)	`poll` O(log n), `peek` O(1)	—	O(n)	O(n)	heap (iteration unsorted)	no
`HashSet` / `HashMap`	O(1)†	—	—	O(1)†	O(1)†	none	one `null` key
`LinkedHashSet` / `LinkedHashMap`	O(1)†	—	—	O(1)†	O(1)†	insertion (or access)	one `null` key
`TreeSet` / `TreeMap`	O(log n)	—	—	O(log n)	O(log n)	sorted	no `null` keys (natural order)
`EnumSet` / `EnumMap`	O(1)	—	—	O(1)	O(1)	enum order	no
`CopyOnWriteArrayList`	O(n)	O(n)	O(1)	O(n)	O(n)	index	yes
`ConcurrentHashMap`	O(1)†	—	—	O(1)†	O(1)†	none	**no**

Which one? (§10.13) Key → value: `HashMap`; insertion order or LRU (least-recently-used eviction): `LinkedHashMap`; sorted or range queries: `TreeMap`. Unique elements: `HashSet` / `LinkedHashSet` / `TreeSet` by the same rule. Indexed or append: `ArrayList` (the default list). Stack or queue: `ArrayDeque` (not `Stack`, not `LinkedList`). "Smallest next": `PriorityQueue`. Enum keys: `EnumMap` / `EnumSet`. Fixed data: `List.of` / `Set.of` / `Map.of`. Shared between threads: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`.

`HashMap` internals (§10.5): table of buckets, default capacity 16, load factor 0.75 (resize — double — when size > capacity × 0.75); index = `(h ^ (h >>> 16)) & (capacity − 1)`; collisions chain in a list that turns into a red-black tree (a self-balancing sorted tree) past 8 entries once the table has ≥ 64 buckets; lookup = hash, then `equals`. Keys must have consistent `equals` / `hashCode` and must not change while in the map.

Unmodifiable vs views (§10.10):

Factory	Changes through it?	Sees later changes to the source?	null s
`List.of`, `Set.of`, `Map.of` (9)	no — `UnsupportedOperationException`	— (no source)	**rejected** (NPE); `Set.of("a","a")` → `IllegalArgumentException`
`List.copyOf` (10), `stream.toList()` (16)	no	no — a snapshot	`copyOf` rejects; `toList()` **allows** `null` (unlike `Collectors.toUnmodifiableList()`)
`Collections.unmodifiableList(src)`	no	**yes** — a read-only view	allowed
`Arrays.asList(array)`	`set` yes (writes through to the array); `add` / `remove` no	yes	allowed

API you use daily (§10.2–§10.6, §10.9, §10.11):

Want	Call	Result in the checks
Count words	`map.merge(w, 1, Integer::sum)`	`{a=1, b=3, c=1}` in a `TreeMap`
Group into lists	`map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)`	
Default	`map.getOrDefault(k, -1)`	`-1`
Previous value	`map.put("a", 2)`	returns the old value (`null` if none)
Nearest keys	`treeMap.floorKey(25)`, `ceilingKey(25)` on {10, 20, 30}	`20`, `30`; `floorKey(5)` = `null`
Sub-ranges	`headMap(20)` / `tailMap(20)`	`[10]` / `[20, 30]` (head exclusive, tail inclusive)
Stack	`deque.push(x)`, `pop()`, `peek()`	`push` = `addFirst`
Queue	`deque.offer(x)`, `poll()`, `peek()`	`offer` = `addLast`; `poll` returns `null` when empty, `remove` throws
Min-heap / max-heap	`new PriorityQueue<>()` / `new PriorityQueue<>(Comparator.reverseOrder())`	`poll()` of {5, 1, 4}: `1` / `5`
LRU cache	`new LinkedHashMap<>(16, 0.75f, true)` + override `removeEldestEntry` → `size() > cap`	
Ends of an ordered collection (21)	`getFirst()`, `getLast()`, `addFirst`, `removeLast`, `reversed()`	`[x, y, z]` → `x`, `z`, `[z, y, x]`
Sort with several keys	`list.sort(comparing(String::length).thenComparing(reverseOrder()))`	`[fig, pear, kiwi, apple]`
Remove while iterating	`removeIf(pred)` or `iterator.remove()`	for-each + `list.remove` → `ConcurrentModificationException`
Utilities	`Collections.sort`, `reverse`, `shuffle`, `frequency`, `max`, `nCopies`, `emptyList`	

Ordering (§10.8): `Comparable<T>.compareTo` = the class's one natural order; `Comparator<T>` = any number of external orders (`comparing`, `thenComparing`, `reversed`, `nullsFirst`). Never compare by subtraction (`a - b` overflows) — use `Integer.compare`. `TreeSet`/`TreeMap` treat `compareTo == 0` as "equal", so keep it consistent with `equals`.

20.10 Functional interfaces, lambdas and `Optional`

`java.util.function` (§11.2, §11.3) — a *functional interface* has exactly one abstract method:

Interface	Shape	Method	Typical use
`Supplier<T>`	`() → T`	`get`	factories, lazy defaults, `orElseGet`
`Consumer<T>` / `BiConsumer<T,U>`	`T → void` / `(T,U) → void`	`accept`	`forEach`, `Map.forEach`
`Predicate<T>` / `BiPredicate<T,U>`	`T → boolean`	`test`	`filter`, `removeIf`
`Function<T,R>` / `BiFunction<T,U,R>`	`T → R` / `(T,U) → R`	`apply`	`map`, `computeIfAbsent`, `merge`
`UnaryOperator<T>` / `BinaryOperator<T>`	`T → T` / `(T,T) → T`	`apply`	`replaceAll`, `reduce`
`IntPredicate`, `IntFunction<R>`, `ToIntFunction<T>`, `IntUnaryOperator`, `IntBinaryOperator`, `IntSupplier`, `ObjIntConsumer<T>` …	primitive versions (also `Long…`, `Double…`)	`test`, `apply`, `applyAsInt`, `getAsInt` …	avoid boxing in hot code (code that runs very often)
`Runnable` / `Callable<V>`	`() → void` / `() → V`, may throw	`run` / `call`	tasks
`Comparator<T>`	`(T,T) → int`	`compare`	sorting

Lambda syntax: `() -> 42`, `x -> x * 2`, `(a, b) -> a + b`, `(String s) -> s.length()`, `(var x, var y) -> …`, block form `x -> { …; return r; }`. Captured local variables must be **effectively final** (assigned once) (§11.5); fields can change. `this` in a lambda is the enclosing object (in an anonymous class it's the anonymous object).

Method references (§11.4):

Kind	Syntax	Equivalent lambda
Static method	`Integer::sum`	`(a, b) -> Integer.sum(a, b)`
Bound instance (specific object)	`System.out::println`, `"*"::repeat`	`x -> System.out.println(x)`
Unbound instance (first argument is the receiver)	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`, `int[]::new`	`() -> new ArrayList<>()`

Composition (§11.6): `f.andThen(g)` runs `f` then `g`; `f.compose(g)` runs `g` then `f` (with `plus1`, `times2` on 3: `8` vs `7`); `Predicate.and/or/negate`, `Predicate.not(String::isBlank)` (11); `Function.identity()`; `Comparator.comparing(...).thenComparing(...)`.

`Optional<T>` (§11.7) — a return type for "maybe no result":

Do	Don't
`opt.map(f).filter(p).orElse(default)`	`opt.get()` without checking (→ `NoSuchElementException`)
`orElseGet(() -> expensive())` for costly defaults	`orElse(expensive())` — the argument is **always** evaluated
`orElseThrow()` (10) when absence is a bug	`Optional` fields, parameters, collection elements
`ifPresentOrElse`, `or`, `stream()` (9)	returning `null` from a method that returns `Optional`
return an empty collection for "no results"	`Optional<List<…>>`
`Optional.ofNullable(x)` for maybe-null values	`Optional.of(null)` (→ `NullPointerException`)
`OptionalInt`/`OptionalDouble` for primitives	`Optional<Integer>` in hot numeric code

20.11 Streams and collectors

A **stream** is a one-shot, lazy pipeline over a source: **source → zero or more intermediate operations → one terminal operation** (§12.1, §12.2). Nothing runs until the terminal operation; elements flow one at a time, and short-circuiting operations stop early:

```
Stream.of(1, 2, 3, 4).peek(x -> trace.add("saw " + x)).filter(x -> x > 1).findFirst();
// result 2, trace [saw 1, saw 2] — elements 3 and 4 are never touched
```

Stage	Operations
Sources	`coll.stream()`, `Stream.of(…)`, `Arrays.stream(a)`, `IntStream.range(0, n)` / `rangeClosed`, `Stream.iterate(seed, hasNext, next)` (9), `Stream.generate(s)`, `Files.lines(p)`, `str.chars()`, `map.entrySet().stream()`
Intermediate, stateless	`filter`, `map`, `mapToInt/Long/Double`, `mapToObj`, `boxed`, `flatMap`, `mapMulti` (16), `peek` (debugging only)
Intermediate, stateful	`distinct`, `sorted`, `limit`, `skip`, `takeWhile` / `dropWhile` (9), `gather(…)` (24)
Terminal	`forEach`, `collect`, `toList()` (16), `toArray`, `reduce`, `count`, `min` / `max`, `sum` / `average` (primitive streams), `findFirst` / `findAny`, `anyMatch` / `allMatch` / `noneMatch`

Terminal facts (§12.4): on an empty stream `anyMatch` → `false`, `allMatch` and `noneMatch` → `true`; `reduce(identity, op)` returns `T`, `reduce(op)` returns `Optional<T>`; `IntStream.average()` returns `OptionalDouble`; `Stream.toList()` is unmodifiable but accepts `null` elements; reusing a consumed stream → `IllegalStateException`.

Collectors (§12.5) — the examples use five employees (eng: Ann 120, Bob 100; ops: Cid 90, Dee 95; hr: Eve 80):

Collector	Result in the checks	Watch out
`groupingBy(Emp::dept, TreeMap::new, counting())`	`{eng=2, hr=1, ops=2}`	default map is a `HashMap` (no order)
`partitioningBy(e -> e.salary() >= 100, mapping(Emp::name, toList()))`	`{false=[Cid, Dee, Eve], true=[Ann, Bob]}`	always has both keys, even for an empty stream
`toMap(Emp::dept, Emp::name)`	`IllegalStateException`	duplicate keys throw; `null` values → NPE
`toMap(Emp::dept, Emp::salary, Math::max, TreeMap::new)`	`{eng=120, hr=80, ops=95}`	merge function resolves duplicates
`joining(", ", "[", "]")`	`[Ann, Bob, Cid, Dee, Eve]`	elements must be `CharSequence`
`counting()`, `summingInt`, `averagingDouble`, `minBy`, `maxBy`, `summarizingInt`	numbers, `Optional`, `IntSummaryStatistics`	usually as downstream collectors
`mapping`, `filtering` (9), `flatMapping` (9), `collectingAndThen`, `teeing` (12)	adapted results	
`toList()`, `toSet()`, `toUnmodifiableList()` (10), `toCollection(TreeSet::new)`	collections	`Collectors.toList()` mutability is unspecified

Primitive streams (§12.6): `IntStream`, `LongStream`, `DoubleStream` have `sum`, `average`, `summaryStatistics`, `range`; convert with `mapToInt` / `boxed` / `mapToObj`. `IntStream.rangeClosed(1, 5).reduce(1, (a, b) -> a * b)` = `120`.

Gatherers (Java 24, §12.9): `stream.gather(g)` is a custom intermediate operation. Built-ins in `Gatherers`: `windowFixed(2)` on 1..5 → `[[1, 2], [3, 4], [5]]`; `windowSliding(n)`; `scan(() -> 0, Integer::sum)` on 1, 2, 3 → `[1, 3, 6]`; `fold`; `mapConcurrent(n, f)` (runs `f` on virtual threads, keeps order).

Rules and mistakes (§12.7, §12.8, §12.10, §12.11): lambdas in a pipeline should be stateless (not depend on anything that changes while the stream runs) and must not modify the source; don't use `peek`/`forEach` to fill an outside list — collect instead; `sorted`/`distinct` on infinite streams never finish; close streams from `Files.lines`/`list`/`walk`. Parallel streams pay off only for large, CPU-bound, easily split, stateless work (they use the common `ForkJoinPool`, the JVM's shared work-stealing thread pool); never for I/O. Loops remain better for early exits with complex state, checked exceptions and index arithmetic.

20.12 I/O and NIO.2

Which API? (§13.1):

Need	Use	Since
Name a file or directory	`Path.of("dir", "a.txt")` (was `Paths.get`)	7 (`Path.of` 11)
Small text file in one call	`Files.readString(p)`, `Files.writeString(p, s)`	11
Lines of a small file	`Files.readAllLines(p)`, `Files.write(p, lines)`	7
Huge text file	`try (Stream<String> s = Files.lines(p))` or `Files.newBufferedReader(p)`	8 / 7
Raw bytes from anywhere	`InputStream` / `OutputStream` (+ `transferTo`, `readAllBytes`)	1.0 (9)
Text from anywhere	`Reader` / `Writer` with an explicit charset	1.1
Big files, random access, memory mapping	`FileChannel` + `ByteBuffer`	1.4
Legacy code that wants `File`	`path.toFile()`, `file.toPath()`	7

`Path` (§13.2) never touches the disk: `Path.of("/a/b/../c/./d.txt").normalize()` → `/a/c/d.txt`; `Path.of("/a/b").relativize(Path.of("/a/c/d"))` → `../c/d`; `Path.of("/a").resolve("b.txt")` → `/a/b.txt`; `getFileName`, `getParent`, `getRoot`, `toAbsolutePath`, `startsWith`.

`Files` (§13.3, §13.4): `exists`, `isDirectory`, `createDirectories` (no error if present), `createFile` (fails if the file exists — `FileAlreadyExistsException`), `createTempFile`/`createTempDirectory`, `copy`/`move` (add `REPLACE_EXISTING`, `ATOMIC_MOVE`), `delete` (throws if missing or a non-empty directory) / `deleteIfExists`, `size`, `mismatch` (12), `list` / `walk` / `find` (lazy `Stream<Path>` — **close them**). `Files.size` counts **bytes**: `"üni\n"` is 6 bytes in UTF-8.

Streams and decorators (wrappers that add a feature, such as buffering, around another stream) (§13.5, §13.6): byte streams (`InputStream`/`OutputStream`) vs character streams (`Reader`/`Writer`); bridge with `InputStreamReader`/`OutputStreamWriter` and a charset; wrap in `BufferedReader`/`BufferedWriter`/`BufferedInputStream` (8 KB buffer by default) — unbuffered byte-at-a-time I/O is very slow, and a writer you forget to `flush`/`close` loses data. Always use try-with-resources.

Charsets (§13.7): the default charset is **UTF-8** since Java 18 (`Charset.defaultCharset()`); still pass `StandardCharsets.UTF_8` explicitly for files other systems produce. Mojibake = bytes decoded with the wrong charset.

Binary data (§13.9): `DataInputStream`/`DataOutputStream` for primitives; `FileChannel`, `ByteBuffer` (`flip` before reading what you wrote), memory-mapped files; `WatchService` for directory change events.

Serialization (§13.10): `implements Serializable`, `transient` fields are skipped, declare `serialVersionUID`. Deserialising untrusted data is a **security risk** (gadget chains: existing classes an attacker strings together to run code) — prefer JSON (a common text format for data) or protobuf (Protocol Buffers, Google's compact binary format); if you must, use deserialization filters (`ObjectInputFilter`, 9; context-specific filters, 17).

Also (§13.8, §13.11–§13.13): `Properties` for `key=value` config (`load`, `getProperty(k, default)`); JSON needs a library (Jackson, Gson) — the JDK has none; `System.in`/`out`/`err`, `Console`, `Scanner`, `IO` (25).

20.13 Concurrency

Threads (§14.1–§14.4):

```
Thread t = Thread.ofPlatform().name("worker").start(task);      // or new Thread(task).start()
Thread v = Thread.ofVirtual().start(task);                      // Java 21
t.join();                                                       // wait for it to finish
```

Call `start()`, not `run()` (which runs on the *current* thread). States: `NEW` → `RUNNABLE` → (`BLOCKED` on a monitor — the lock built into every object · `WAITING` on `wait`/`join`/`park` · `TIMED_WAITING` on `sleep`/timeouts) → `TERMINATED`. Interruption is cooperative: `t.interrupt()` sets a flag; blocking methods throw `InterruptedException` — rethrow it or restore the flag with `Thread.currentThread().interrupt()`, never swallow it. There is no safe way to kill a thread: `Thread.stop()` threw `UnsupportedOperationException` from Java 20 and was removed in JDK 26. The JVM exits when only daemon threads (background threads) remain.

The primitives (§14.5–§14.13):

Tool	Gives you	Use for	Remember
`synchronized` (method or block)	mutual exclusion **and** visibility; reentrant (the holder may lock again)	protecting shared mutable state, multi-variable invariants	instance method locks `this`, static method locks the `Class` object; released automatically
`volatile` field	visibility and ordering only	a stop flag, safely publishing a reference	`count++` on a volatile is **still a race**
`AtomicInteger`, `AtomicLong`, `AtomicReference`	lock-free single-variable updates via CAS (compare-and-set)	counters, sequence numbers	`compareAndSet(expected, new)` → `true` only if the value matched; `LongAdder` for very hot counters
`ReentrantLock`	`tryLock`, timeouts, interruptible, fair mode, several `Condition`s	when `synchronized` isn't flexible enough	**`unlock()` in `finally`**
`ReadWriteLock` / `StampedLock`	many readers or one writer / optimistic reads	read-mostly data	`StampedLock` is not reentrant
`wait` / `notifyAll`	guarded blocks	legacy code — prefer `java.util.concurrent`	call in a `while` loop, holding the lock
`ConcurrentHashMap`	thread-safe map with atomic `merge`, `compute`, `computeIfAbsent`, `putIfAbsent`	shared caches, counters	no `null` keys/values; iterators weakly consistent
`CopyOnWriteArrayList`	snapshot iteration, never throws CME	listeners, read-mostly lists	every write copies the array
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)	`put` waits when full, `take` waits when empty	producer–consumer (one thread makes items, another uses them)	prefer bounded queues
`CountDownLatch(n)`	wait until n `countDown()` calls	start/finish gates	one-shot
`CyclicBarrier(n)`	n threads wait for each other, repeatedly	phased algorithms	reusable
`Semaphore(k)`	at most k holders at a time	limit concurrent DB/API calls (also with virtual threads)	`release` in `finally`
`ThreadLocal` / `ScopedValue` (25)	per-thread value / immutable per-call context	request context	remove `ThreadLocal`s on pooled threads

Executors (§14.11): `try (ExecutorService pool = Executors.newFixedThreadPool(n)) { Future<T> f = pool.submit(callable); }` — `ExecutorService` is `AutoCloseable` since Java 19, and `close()` waits for submitted tasks. Factories: `newFixedThreadPool(n)` (unbounded queue!), `newCachedThreadPool()` (unbounded threads!), `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newWorkStealingPool()`, `newVirtualThreadPerTaskExecutor()` (21). For full control use `new ThreadPoolExecutor(core, max, keepAlive, unit, boundedQueue, factory, rejectionPolicy)`. Size CPU-bound pools (work limited by computation, not waiting) ≈ number of cores.

`CompletableFuture` (§15.1, §15.2):

Method	Does	Checked result
`supplyAsync(supplier[, executor])` / `runAsync`	start async work (default: common `ForkJoinPool`)	
`thenApply(fn)`	transform (like `map`)	
`thenCompose(fn)`	chain another async step (like `flatMap`)	`5` → `6`
`thenCombine(other, fn)`	combine two independent results	price 20 × qty 3 = `60`
`allOf(…)` / `anyOf(…)`	wait for all / the first	
`exceptionally(fn)`	recover on failure	`-1`
`handle((v, ex) -> …)` / `whenComplete`	always runs; can change result / can't	
`orTimeout(d, unit)` / `completeOnTimeout(v, d, unit)` (9)	time limits	
`join()` / `get()`	block for the result	`join` wraps failures in `CompletionException` (unchecked); `get` in `ExecutionException` (checked)

Virtual threads (Java 21, §15.4, §15.5): cheap threads scheduled by the JVM onto a few carrier (platform) threads; blocking I/O unmounts them. Use one per task (`newVirtualThreadPerTaskExecutor()`) for **I/O-bound**, high-concurrency work; they don't speed up CPU-bound work. Don't pool them; cap scarce resources with a `Semaphore`; avoid big `ThreadLocal` caches. They are always daemon threads. Since **Java 24** (JEP 491) blocking inside `synchronized` no longer pins the carrier; native calls that call back into blocking Java code, and class initialisation, still do.

Also modern (§15.3, §15.6, §15.7): Fork/Join (`RecursiveTask`, work stealing — idle threads take queued tasks from busy ones — used by parallel streams); **scoped values** are final in Java 25 (`ScopedValue.where(USER, "alice").call(() -> USER.get())` → `"alice"`, and unbound again afterwards); **structured concurrency** (`StructuredTaskScope.open()` plus a `Joiner` policy) is still **preview** in 25 (JEP 505) and kept changing in 26 (JEP 525) and 27 (JEP 533: built-in joiners make `join()` throw `ExecutionException`, `awaitAll()` removed) — don't write production code against it yet.

Java Memory Model (§15.8): without a *happens-before* edge another thread may never see your write. Edges: program order in one thread; unlock → later lock of the same monitor; volatile write → later read of that field; `Thread.start()` → actions in the thread; actions in a thread → a successful `join()`; `java.util.concurrent` hand-offs (queue put → take, `countDown` → `await`, task submit → run, task → `Future.get`); transitivity. Safe publication (making a new object visible to other threads only once fully built): `final` fields, volatile, locks, concurrent collections, static initialisers.

Liveness (§15.9): **deadlock** (threads wait for each other's locks forever — fix with one global lock order, `tryLock` with timeout, fewer locks; diagnose with `jstack` / `jcmd Thread.print`), **livelock** (threads keep reacting to each other without progress — add randomised back-off), **starvation** (a thread never gets CPU or a lock — fair locks, bounded work).

Choosing a model (§15.10): blocking I/O at scale → virtual threads; fan-out/fan-in with composition → `CompletableFuture` (structured concurrency once final); CPU-heavy data work → parallel streams / Fork/Join / fixed pool of ≈ cores; periodic jobs → `ScheduledExecutorService`; shared counters/caches → atomics, `LongAdder`, `ConcurrentHashMap`. For design-level concurrency case studies see the LLD handbook.

20.14 The JVM: memory, GC, flags and tools

How code runs (§1.6, §1.7, §16.1, §16.2, §16.8): `javac` → bytecode in `.class` files → class loaders load classes lazily (bootstrap → platform → application, **parent-first delegation**) → verify, prepare, resolve, initialise (static initialisers run once) → the interpreter runs bytecode; hot methods are

compiled by the tiered JIT (**C1** fast, then **C2** heavily optimising: inlining (copying a called method into its caller), devirtualisation (calling the one known implementation directly), escape analysis, loop optimisations, deoptimisation when assumptions break). Hence "warm-up". Class loaders in the checks: `String.class.getClassLoader()` is `null` (bootstrap), `java.sql` classes use the `platform` loader, your classes the `app` loader.

Memory areas (§16.3, §1.8):

Area	Per thread?	Holds	Size flag	When it runs out
Heap (young: eden, where new objects start, + survivors; old)	shared	all objects and arrays	`-Xms`, `-Xmx`, `-XX:MaxRAMPercentage`	`OutOfMemoryError: Java heap space`
Java stack	per thread	frames: locals, operand stack	`-Xss`	`StackOverflowError`
Metaspace (native memory; replaced PermGen in 8)	shared	class metadata	`-XX:MaxMetaspaceSize` (unlimited by default)	`OutOfMemoryError: Metaspace`
Code cache	shared	JIT-compiled code	`-XX:ReservedCodeCacheSize`	JIT stops compiling (slow, not a crash)
Direct / native memory	shared	direct `ByteBuffer`s, thread stacks, GC structures	`-XX:MaxDirectMemorySize`	`OutOfMemoryError: Direct buffer memory` / `unable to create native thread`

Object size basics (§16.4): 12-byte header (8 with `-XX:+UseCompactObjectHeaders`, Java 25; the default from JDK 27), 4-byte compressed references, padded to 8 bytes — `new Object()` is 16 bytes; `record Point(int x, int y)` 24.

Garbage collectors (§16.5–§16.7) — an object is garbage when it isn't reachable from GC roots (thread stacks, static fields, JNI handles):

Collector	Flag	Pauses	Choose it for
G1 (default since 9 on machines with ≥ 2 CPUs and ≥ 1792 MB; in all environments from JDK 27, JEP 523)	`-XX:+UseG1GC`	goal `-XX:MaxGCPauseMillis` (200 ms default); typically a few–tens of ms	most services — the default for a reason
Parallel	`-XX:+UseParallelGC`	longer, grows with live data	batch jobs: maximum throughput
Serial	`-XX:+UseSerialGC`	stop-the-world (all app threads paused), single thread	tiny heaps, 1 CPU, CLI tools (before JDK 27 the JVM picked it on machines with < 2 CPUs or < 1792 MB)
ZGC (generational; generational-only since 24)	`-XX:+UseZGC`	typically < 1 ms, independent of heap size	latency-sensitive services, huge heaps
Shenandoah (generational mode product in 25)	`-XX:+UseShenandoahGC`	sub-ms to a few ms	low latency; in most OpenJDK builds, not Oracle's
Epsilon	`-XX:+UnlockExperimentalVMOptions -XX:+UseEpsilonGC`	none — never collects	benchmarks, ultra-short jobs

Key flags (§16.7):

Flag	Meaning
`-Xms512m -Xmx2g`	initial / maximum heap (set them equal for servers to avoid resizing)
`-XX:MaxRAMPercentage=75`	heap as % of container memory (default 25)
`-Xss1m`	thread stack size (default platform-dependent: about 1 MB on 64-bit Linux, 2 MB on macOS/AArch64)
`-Xlog:gc*` (or `-Xlog:gc:file=gc.log`)	unified GC logging (9+)
`-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps`	heap dump on OOM (`OutOfMemoryError`)
`-XX:+PrintFlagsFinal`	print every flag's final value
`-XX:StartFlightRecording=duration=60s,filename=app.jfr`	record with JDK Flight Recorder at start-up
`-XX:TieredStopAtLevel=1`	C1 only: faster start-up, lower peak speed
`-ea`	enable `assert` statements
`--enable-preview`	allow preview features (needs `--release N` at compile time)
`-XX:+UseCompactObjectHeaders`	8-byte object headers (25, opt-in; on by default from JDK 27, JEP 534)

Diagnostic tools (§16.10, §16.11):

Tool	Use
`jps -l`	list running JVMs and PIDs (process IDs)
`jcmd <pid> help`	the Swiss-army knife: `VM.flags`, `GC.heap_info`, `GC.class_histogram`, `Thread.print`, `GC.heap_dump file.hprof`, `JFR.start` / `JFR.dump`, `VM.native_memory`
`jstat -gcutil <pid> 1000`	heap occupancy and GC counts every second
`jstack <pid>`	thread dump — look for `BLOCKED` and "Found one Java-level deadlock"
JFR + `jfr view hot-methods app.jfr` / JDK Mission Control	low-overhead production profiling
VisualVM, Eclipse MAT, async-profiler	GUI monitoring, heap-dump analysis (dominator tree), flame graphs (charts of where CPU time goes)
JMH	micro-benchmarks that avoid warm-up and dead-code traps (§16.9)
`jdeps`, `jlink`	module dependencies / build a trimmed runtime (§16.12)

Symptom → first tool: high CPU → JFR hot methods; latency spikes → GC log pauses; hangs → two or three thread dumps; memory growth → `jstat` old-gen trend + class histogram twice, then a heap dump; RSS (resident set size — the memory the OS says the process uses) far above heap → native memory tracking. Classic leaks: unbounded static caches, unregistered listeners, `ThreadLocal`s on pooled threads, mutated `HashMap` keys, class-loader leaks.

Modules (JPMS) (§16.12): `module-info.java` with `requires`, `exports`, `opens` (for reflection — code inspecting classes at run time), `uses`/`provides` (services); the class path is the *unnamed module*; `import module java.base;` (25) imports every exported package of a module.

20.15 Modern Java by version: 17 to 25

Final features only — the version that **finalised** each one (§17.2; JEP numbers in parentheses). The book's baseline before 17: `var` (10), `HttpClient` and `java Hello.java` (11), switch expressions and helpful NPE messages (14; the messages on by default since 15), text blocks (15), records, `instanceof` patterns and `Stream.toList()` (16).

Release	GA (general availability)	Language and libraries	JVM and tools
17 LTS (long-term support)	Sep 2021	sealed classes (409), enhanced pseudo-random generators (356), always-strict floating point (306), context-specific deserialization filters (415)	strong encapsulation of JDK internals (403), macOS/AArch64 port (391)
18	Mar 2022	UTF-8 by default (400), `@snippet` in Javadoc (413)	simple web server `jwebserver` (408); finalization deprecated for removal (421)
19, 20	Sep 2022, Mar 2023	previews and incubators (experimental API modules) only	Linux/RISC-V port (422, in 19)
21 LTS	Sep 2023	record patterns (440), pattern matching for `switch` (441), **virtual threads** (444), sequenced collections (431), key encapsulation mechanism API (452)	generational ZGC (439), warnings for dynamically loaded agents (451)
22	Mar 2024	unnamed variables and patterns `_` (456), Foreign Function & Memory API (454), multi-file source launch (458)	region pinning for G1 (423)
23	Sep 2024	Markdown documentation comments `///` (467)	ZGC generational by default (474); `sun.misc.Unsafe` memory access deprecated for removal (471)
24	Mar 2025	**stream gatherers** (485), Class-File API (484), ML-KEM (496) and ML-DSA (497) post-quantum crypto	virtual threads without `synchronized` pinning (491), AOT class loading and linking (483), non-generational ZGC removed (490), Security Manager permanently disabled (486)
25 LTS	Sep 2025	compact source files and instance `main` (512), module import declarations (511), flexible constructor bodies (513), **scoped values** (506), key derivation function API (510)	compact object headers (519, opt-in), generational Shenandoah (521), AOT ergonomics (514) and method profiling (515), JFR cooperative sampling (518) and method timing (520)

Still not final in Java 25 (§17.11): structured concurrency (fifth preview, 505; sixth in 26, seventh in 27), primitive types in patterns (third preview, 507), stable values (preview, 502; renamed *lazy constants* in 26), PEM encodings (preview, 470), Vector API (tenth incubator, 508), JFR CPU-time profiling (experimental, 509). **Withdrawn**: string templates (previewed in 21 and 22, removed in 23). Try a preview with `javac --release 25 --enable-preview` and `java --enable-preview`. After 25 (§17.11): JDK 26 (Mar 2026) finalised HTTP/3 in `HttpClient` (517) and removed the Applet API (504) and `Thread.stop`; JDK 27 (Sep 2026) made G1 the default everywhere (523) and compact object headers the default (534). LTS releases: 8, 11, 17, 21, 25 (every two years); the next is expected to be 29 in September 2027 (§1.12).

Upgrading 17 → 25 (§17.12): raise the JDK first and `--release` second; upgrade build plugins and bytecode libraries (Lombok, Mockito, Byte Buddy, Spring); run `jdeps --jdk-internals` and `jdeprscan`; read the start-up warnings (agents, JNI, `Unsafe`); drop removed flags such as `-XX:-ZGenerational`.

20.16 Testing, build and tooling commands

JDK tools (§1.4–§1.11, §18.12, §18.13):

Command	Does
`java -version`, `javac -version`	check the installed JDK
`javac --release 25 -d out $(find src -name '*.java')`	compile for a target Java version
`java -cp out:lib/* com.example.Main`	run with a class path (`;` on Windows)
`jar --create --file app.jar --main-class com.example.Main -C out .`	build an executable JAR; run with `java -jar app.jar`
`javadoc -d docs …`, `javap -c`, `jshell`	docs, bytecode, REPL
`jlink`, `jpackage`	custom runtime image / native installer
`bash tools_run_java.sh java`	this book's runner: compiles each chapter's `code/` with `--release 25` and runs every `main`

Maven (§18.2; this book: Maven 3.9.14, Gradle 9.4.1) — `pom.xml`; coordinates `groupId:artifactId:version`; scopes `compile`, `test`, `provided`, `runtime`; "nearest wins" for version conflicts (pin with `dependencyManagement`); pin every plugin version too. Lifecycle: `validate → compile → test-compile → test → package → verify → install → deploy` — running a phase runs every phase before it.

Command	Does
`mvn clean verify`	full build with all checks (what CI — continuous integration, the server that builds every change — runs)
`mvn test`, `mvn -Dtest=CartTest test`	run all / one test class (`CartTest#method` for one method)
`mvn package -DskipTests`	build the JAR without running tests
`mvn dependency:tree`	print the resolved dependency tree
`./mvnw …`	the Maven Wrapper: pinned Maven version, committed to the repo

Gradle (§18.3) — `build.gradle(.kts)`; configurations `implementation`, `testImplementation`, `runtimeOnly`; incremental and cached tasks.

Command	Does
`./gradlew build`	compile, test, assemble
`./gradlew test --tests 'ch18.shop.CartTest'`	run matching tests
`./gradlew run`	run the `application` plugin's main class
`./gradlew dependencies --configuration testRuntimeClasspath`, `./gradlew tasks`	inspect

JUnit Jupiter (§18.4, §18.5, §18.8) — the same API in JUnit 5 and JUnit 6 (6.0, 2025: Java 17 baseline, one version number for Platform/Jupiter/Vintage, JSpecify annotations, Vintage deprecated); this book ran JUnit 6.1.3:

Annotation	Meaning
`@Test`	a test method (package-private is fine)
`@BeforeEach` / `@AfterEach`	run around every test (a new test-class instance per test by default)
`@BeforeAll` / `@AfterAll`	once per class (`static`)
`@DisplayName`, `@Disabled`, `@Tag`, `@Timeout`, `@Nested`	naming, skipping, filtering, time limit, grouping
`@ParameterizedTest` + `@ValueSource`, `@CsvSource`, `@MethodSource`, `@EnumSource`, `@NullAndEmptySource`	one test, many inputs
`@TempDir Path dir`	fresh temporary directory
`@ExtendWith(MockitoExtension.class)`	plug in an extension

Assertions: `assertEquals(expected, actual)` (expected first), `assertTrue`, `assertNull`, `assertThrows(X.class, () -> …)` (returns the exception), `assertAll(…)` (report every failure), `assertIterableEquals`, `assertTimeout`. Console launcher: `java -jar junit-platform-console-standalone-<version>.jar execute --class-path out:libs --scan-class-path`.

Mockito (§18.6): `mock(Repo.class)` or `@Mock`; stub with `when(repo.find("a")).thenReturn(Optional.of(x))` / `thenThrow`; `verify(repo).save(any())`, `verify(repo, never()).delete(…)`, `times(2)`; `ArgumentCaptor`; `@InjectMocks`; stub only what the test needs (strict stubs flag the rest). Fakes beat mocks for simple collaborators. Mockito 5 (this book: 5.24.0) mocks `final` classes through an agent — on JDK 21+ add `-javaagent:mockito-core.jar` to the test JVM (Surefire `argLine`, Gradle `jvmArgs`) instead of letting it self-attach.

AssertJ (§18.7): `assertThat(list).hasSize(3).containsExactly("a", "b", "c")`; `assertThat(s).startsWith("J").contains("av")`; `assertThatThrownBy(() -> …).isInstanceOf(IllegalArgumentException.class).hasMessageContaining("negative")`; `extracting(Order::id)`; `usingRecursiveComparison()`.

Test design (§18.9): Arrange–Act–Assert; one behaviour per test; name tests after behaviour; FIRST (fast, independent, repeatable, self-validating, timely); inject clocks and random seeds; test pyramid (many unit, some integration — Testcontainers — few end-to-end — whole-system tests). Testcontainers starts throwaway real services (such as a database) in Docker containers for tests.

Logging (§18.10): code against SLF4J (`private static final Logger log = LoggerFactory.getLogger(Foo.class); log.info("order {} placed", id);` — placeholders, not concatenation), with Logback or Log4j 2 as the backend; levels `TRACE < DEBUG < INFO < WARN < ERROR`; `java.util.logging` is built in (`FINE`, `INFO`, `WARNING`, `SEVERE`). Pass the exception as the last argument to keep the stack trace.

JDBC (§18.11): `try (Connection c = dataSource.getConnection(); PreparedStatement ps = c.prepareStatement("SELECT … WHERE id = ?")) { ps.setLong(1, id); try (ResultSet rs = ps.executeQuery()) { while (rs.next()) … } }` — always `PreparedStatement` (prevents SQL injection — attackers smuggling SQL commands inside input values), a connection pool (HikariCP; a set of open database connections reused across requests) in real apps, `setAutoCommit(false)` + `commit`/`rollback` for transactions (groups of changes that succeed or fail together). SQL itself: the SQL handbook.

20.17 Interview question index

All 42 questions are answered in Chapter 19 §19.29, each with links back to the teaching section; claims marked ✔ there are proven by a runnable program. The coding side — the Java DSA toolkit (§19.1–§19.7) and twenty patterns from two pointers to math and geometry (§19.8–§19.27) — is in the same chapter. Pattern finder:

Problem says…	Pattern	Section
sorted input, pair with target sum, in place	two pointers	§19.8
longest/shortest contiguous substring or subarray	sliding window	§19.9
seen before, count, group, pair in unsorted data	hashing	§19.10
range sums, subarrays summing to k (negatives allowed)	prefix sums	§19.11
brackets, next greater element	stack / monotonic stack	§19.12
k largest, merge k lists, running median, best among currently eligible	heap, two heaps	§19.13
sorted, first true, minimum capacity such that	binary search (on the answer)	§19.14
meetings, overlaps, rooms	intervals	§19.15
fewest steps, islands, weighted route	BFS / DFS / Dijkstra	§19.16
prerequisites, order with dependencies	topological sort	§19.17
connected groups as edges arrive	union-find	§19.18
number of ways, min cost, longest subsequence	dynamic programming	§19.19
all subsets, permutations, N-Queens	backtracking	§19.20
many prefix queries, autocomplete	trie	§19.21
k-th from end, middle, cycle start, intersection, palindrome list	linked lists, fast/slow pointers	§19.22
validate BST, lowest common ancestor, path sum, serialise	tree recursion (top-down / bottom-up)	§19.23
jump game, gas station, candies, local best is safe	greedy	§19.24
k-th element without full sort, sort colours, sort a list	quickselect, Dutch flag, merge sort	§19.25
appears twice except one, count 1-bits, subsets of ≤ 20	bit manipulation	§19.26
spiral matrix, reverse digits, points on a line, Josephus	math and geometry	§19.27

Strings, objects and the language → §19.29

Q	Topic	Q	Topic
1	why `String` is immutable	7	`final` vs `finally` vs `finalize`
2	the string pool; objects made by `new String("hi")`	8	making a class immutable
3	`String` vs `StringBuilder` vs `StringBuffer`	9	interface vs abstract class
4	the `equals` / `hashCode` contract	10	records and their limits
5	`==` vs `equals`, the `Integer` cache	11	sealed classes
6	pass-by-value	12	overloading vs overriding

Collections → §19.29: Q13 `HashMap` internals · Q14 mutating a key · Q15 `HashMap` vs `Hashtable` vs `synchronizedMap` vs `ConcurrentHashMap` · Q16 how `ConcurrentHashMap` is thread-safe · Q17 `ArrayList` vs `LinkedList` · Q18 map orderings and an LRU cache · Q19 `Comparable` vs `Comparator` · Q20 fail-fast vs weakly consistent iterators · Q21 `ArrayDeque` vs `Stack` · Q22 `List.of` vs `Arrays.asList` vs `unmodifiableList`.

Exceptions → §19.29: Q23 checked vs unchecked · Q24 how try-with-resources works · Q25 when `finally` doesn't run.

Generics, lambdas and streams → **§19.29**: Q26 type erasure and its limits · Q27 PECS · Q28 streams vs loops · Q29 misusing `Optional` · Q30 functional interfaces and effectively final.

Concurrency → **§19.29**: Q31 `volatile` vs `synchronized` · Q32 happens-before · Q33 deadlock · Q34 virtual threads · Q35 `Runnable`/`Callable`, `Future`/`CompletableFuture` · Q36 why executors.

The JVM → **§19.29**: Q37 memory areas · Q38 GC and the default collector · Q39 memory leaks · Q40 class loading, interpreter and JIT · Q41 `static` and static initialisers · Q42 modern Java features to use in interviews.

More interview material: each chapter's **In the interview** sections, the modern-Java round in §17.13, and exam-level traps in the OCP Java 21 study notes.

20.18 Where next

You now know the language, the core library, the JVM and the tools. What to learn next depends on the job you want:

Direction	What it is	First step
Spring Boot	the dominant framework for Java back-ends (Boot 4.0, Nov 2025, on Spring Framework 7; 4.1 in 2026; Java 17+): dependency injection, auto-configuration, starters for web, data, security, and Actuator for operations (§18.14)	generate a project at start.spring.io, build a small REST API (a web API of URLs answered over HTTP) with JDBC or JPA (Jakarta Persistence, which maps objects to database tables), test it with JUnit, Mockito and Testcontainers
Jakarta EE (formerly Java EE)	the standard enterprise specifications — Servlet, CDI, JPA, JAX-RS (Jakarta REST), Bean Validation — implemented by WildFly, Open Liberty, Payara	learn JPA and CDI; much of Spring uses the same specifications (e.g. JPA via Hibernate)
Quarkus, Micronaut, Helidon	cloud-native frameworks with fast start-up and small memory, strong GraalVM native-image support (compiling Java ahead of time into a stand-alone executable)	port your Spring REST API to compare
Kotlin	a JetBrains language on the JVM, fully interoperable with Java: null-safe types, data classes, coroutines; Google's preferred language for Android since 2019	rewrite one small program; call it from Java
Other JVM languages	Scala (functional + OO, Spark), Groovy (Gradle scripts, scripting), Clojure (Lisp)	only if a job needs them
Android	mobile apps with the Android SDK (Kotlin-first today, Java supported)	Android Developers' basic course
Data and messaging	JDBC/JPA, Kafka (a distributed message log) clients, Spark, Flink (big-data processing engines)	the SQL handbook and the DDIA notes
Design	SOLID (five class-design principles), patterns, low-level design case studies in Java	the LLD handbook
Certification	Oracle Certified Professional: Java SE 21 Developer (1Z0-830)	the OCP Java 21 study notes
Performance	JFR, JMH, GC tuning, async-profiler	revisit Chapter 16 with a real service

Keep up to date by reading the JEP list for each release on openjdk.org and the dev.java articles; a new feature release ships every March and September.

20.19 Glossary (A–Z)

Each term has a short definition and a link to the section that teaches it. Backticks and symbols are ignored when sorting; *italic* names point to another entry. 706 terms.

A

- **Abstract class** — A class declared `abstract` that can't be instantiated; it may hold state, constructors and abstract methods for subclasses to implement. §6.5
- **Abstract method** — A method with no body that a concrete subclass (or implementing class) must implement. §6.5
- **Access modifier** — `private`, package-private (none), `protected` or `public` — controls where a member can be used. §5.6
- **Accessor (record)** — The generated method named like a record component, e.g. `x()` for `record Point(int x, int y)`. §6.8
- **Actuator** — The Spring Boot module that exposes health, metrics and info endpoints for operations. §18.14
- **Adjacency list** — A graph stored as, for each node, the list of nodes it has an edge to. §19.16
- **Algebraic data type** — A type defined as "exactly one of these variants, each with these fields" — in Java, a sealed interface plus records. §3.7, §6.7
- **Algorithm** — A precise, step-by-step procedure that solves a problem; its cost is described with *Big-O notation*. Ch 19
- **Allocation rate** — How fast a program creates new objects (MB per second); a high rate means more frequent young collections. §16.10
- **Amortised O(1)** — Constant cost on average over many calls, even if an occasional call (such as an `ArrayList` resize) is O(n). §10.12
- **Anagram** — A word made by rearranging another word's letters (*tea*, *eat*); grouped by sorting each word's letters. §19.10
- **Annotation** — Metadata written `@Name` on code (`@Override`, `@FunctionalInterface`, `@Test`) that the compiler, tools or frameworks read. §6.3, §18.4
- **Anonymous class** — A class declared and instantiated in one expression, `new Interface() { … }`, with no name. §6.11
- **AOT cache** — Ahead-of-time class loading and linking data (Java 24–25) that the JVM reuses to start faster. §17.9, §17.10
- **API** — Application Programming Interface: the classes and methods a library, the JDK or a service offers for other code to call. §1.1
- **`ArithmeticException`** — Unchecked exception for integer division by zero or overflow detected by `Math.*Exact` methods. §2.10, §7.5
- **Arrange–Act–Assert** — The three-part shape of a clear test: set up the inputs, run the code under test, then check the result. §18.9
- **Array** — A fixed-length, indexed, zero-based container of one element type; an object with a `length` field. §4.6
- **`ArrayDeque`** — A resizable circular-array `Deque`; the recommended stack and queue; rejects `null`. §10.6
- **`ArrayIndexOutOfBoundsException`** — Thrown when an array index is negative or ≥ `length`. §4.6
- **`ArrayList`** — A growable-array `List` with O(1) indexed access and amortised O(1) append; the default list. §10.2
- **`Arrays.asList`** — Returns a fixed-size `List` view backed by an array: `set` writes through, `add`/`remove` throw. §4.8, §10.10
- **Arrow label** — `case X ->` in a `switch` (Java 14): runs only that case, with no fall-through. §3.3
- **ASCII** — The original 7-bit character set of 128 English letters, digits and symbols; the first 128 Unicode characters are the same. §7.1, §13.7
- **`assert`** — A statement that checks a condition only when the JVM runs with `-ea`; disabled by default. §19.1
- **AssertJ** — A fluent assertion library: `assertThat(x).isEqualTo(y)`. §18.7

- **Associativity** — The grouping order for operators of equal precedence — left for most, right for assignment and the ternary. §2.9
- **Asynchronous** — Started now and finished later without the caller waiting; the result arrives through a `Future`, a callback or a `CompletableFuture`. §15.1, §15.2
- **Atomic variable** — A class such as `AtomicInteger` whose single-variable updates are atomic without locks, using CAS. §14.10
- **Atomicity** — An operation is atomic when other threads can see it only as not started or fully done, never half-done. §14.5
- **Autoboxing** — Automatic conversion from a primitive to its wrapper (`int` → `Integer`); unboxing is the reverse. §7.6
- `AutoCloseable` — The interface with `close()` that try-with-resources calls automatically. §8.6

B

- **Backing array** — The array behind an `ArrayList` or an `Arrays.asList` view. §10.2, §10.10
- **Backtracking** — Searching every candidate solution by making a choice, exploring further, then undoing the choice to try the next one. §19.20
- **Banker's rounding** — `RoundingMode.HALF_EVEN`: a tie rounds to the even neighbour (2.345 → 2.34), which avoids drift in large sums. §7.7
- **Behaviour parameterisation** — Passing behaviour (today, a lambda) as an argument so one method can serve many cases. §11.1
- **Bellman-Ford** — A shortest-path algorithm that, unlike Dijkstra's, allows negative edge weights; slower at O(V · E). §19.16
- **Benchmark** — A measured timing of code; a fair Java microbenchmark needs warm-up and a harness such as *JMH*. §16.9
- **BFS (breadth-first search)** — Visiting a graph level by level from a start node using a queue; it finds shortest paths when every edge costs the same. §19.16
- `BigDecimal` — Immutable arbitrary-precision decimal number with an explicit scale; used for money. §7.7
- `BigInteger` — Immutable arbitrary-size integer. §7.7
- **Big-O notation** — How an operation's cost grows with input size n: O(1), O(log n), O(n), O(n log n), O(n²). §10.12, §19.7
- **Binary search** — Finding a key in sorted data by halving the range each step, O(log n). §4.8, §19.14
- **Binary search tree (BST)** — A binary tree where every key in a node's left subtree is smaller, and every key in its right subtree larger, than the node's key. §19.23
- **Binary tree** — A tree in which every node has at most two children, called left and right. §19.23
- **Bit manipulation** — Solving problems with bitwise operators on the binary form of integers — masks, shifts and xor tricks. §19.26
- **Bitmask** — An integer whose individual bits act as on/off flags, for example one bit per item of a subset. §19.26
- **Bitwise operators** — `&`, `\|`, `^`, `~`, `<<`, `>>`, `>>>` — operate on the individual bits of integers. §2.8
- `BLOCKED` — The thread state of a thread waiting to acquire a monitor held by another thread. §14.3
- `BlockingQueue` — A queue whose `put` waits when full and `take` waits when empty; the core of producer–consumer. §14.12
- **BOM** — (1) Byte order mark: optional bytes at the start of a text file that announce its encoding. (2) In Maven, a "bill of materials" POM that pins versions for a family of libraries. §13.7, §18.2
- **Bootstrap class loader** — The JVM's built-in loader for core modules such as `java.base`; shown as `null` by `getClassLoader()`. §16.2
- **Bounded type parameter** — A type parameter with a limit, e.g. `<T extends Number>`. §9.5
- **Boxing** — Wrapping a primitive value in its wrapper object. §7.6
- `break` — Leaves the innermost loop or `switch` (or a labelled statement with `break label`). §3.9
- **Bridge method** — A hidden method `javac` generates so that overriding still works after generic types are erased. §9.7

- **Brute force** — Solving a problem by trying every possibility; usually too slow, but a good first answer to improve on. §19.28
- **Bucket** — One slot of a hash table's array, holding the entries whose hashes map to that index. §10.5
- **Buffer** — A block of memory that collects data so it can be read or written in large chunks rather than byte by byte. §13.6, §13.9
- **Buffering** — Reading or writing data in blocks through an in-memory buffer to avoid many slow system calls. §13.6
- **Build tool** — Software (Maven, Gradle) that resolves dependencies, compiles, tests and packages a project. §18.2, §18.3
- **Byte Buddy** — A library for generating and changing classes at run time; Mockito uses it to build mocks. §17.12
- **Byte stream** — `InputStream` / `OutputStream`: I/O in raw 8-bit bytes. §13.5
- `ByteBuffer` — A NIO container of bytes with position, limit and capacity, used with channels. §13.9
- **Bytecode** — The platform-independent instruction set in `.class` files that the JVM executes. §1.6

C

- **C1 / C2** — HotSpot's two JIT compilers: C1 compiles quickly with light optimisation, C2 slowly with heavy optimisation. §1.7, §16.8
- **Cache** — A fast store of recently used or precomputed results kept so the next request is cheaper; it needs a size limit or eviction rule, or it becomes a memory leak. §10.4, §16.11, System Design Fundamentals
- **Call stack** — The stack of frames for the method calls a thread currently has in progress: a call pushes a frame, a return pops it. §4.5, §16.3
- `Callable` — A task interface `V call() throws Exception` that returns a value; see `Runnable`. §14.11
- **Callback** — A function handed to other code so that code can call it later, for example the lambda given to `thenApply`. §15.1, §15.2
- **Canonical constructor** — A record's constructor that takes every component in declaration order; generated unless you write it. §6.8
- **Card table / remembered set** — GC bookkeeping that records which old-generation areas point into the young generation, so a young collection needn't scan the whole heap. §16.5
- **Carrier thread** — A platform thread on which the JVM mounts virtual threads to run them. §15.4
- **CAS (compare-and-set)** — An atomic CPU instruction that updates a value only if it still equals an expected value. §14.10
- **Casting** — Explicit conversion `(Type) value` — narrowing a primitive, or checking an object's run-time type. §2.7, §6.4
- **CDI** — Jakarta Contexts and Dependency Injection: the Jakarta EE standard for dependency injection. §1.2
- **CDS (Class Data Sharing)** — Saving already-parsed classes in an archive that later JVM runs map in, so they start faster; the *AOT cache* builds on it. §16.2
- **Channel** — An NIO connection to a file or socket (such as `FileChannel`) that reads and writes through `ByteBuffer`s. §13.9
- `char` — The 16-bit unsigned primitive holding one UTF-16 code unit (0 … 65 535). §2.3
- **Character stream** — `Reader` / `Writer`: I/O in 16-bit characters, decoded with a charset. §13.5
- **Charset** — A mapping between characters and bytes, such as UTF-8; the JDK default since Java 18 is UTF-8. §13.7
- **Checked exception** — An `Exception` outside the `RuntimeException` branch; must be caught or declared with `throws`. §8.2, §8.3
- **Check-then-act** — Testing a condition and then acting on it as two separate steps; another thread can change things in between, so the pair must be made atomic. §14.5
- **CI (continuous integration)** — A server that automatically builds and tests every change pushed to a shared repository. §18.1, §18.2
- **Circular array** — An array used as a ring whose indexes wrap from the end back to the start, so a queue can grow at both ends cheaply (`ArrayDeque`). §10.6

- **Class** — A blueprint that defines fields, constructors and methods; objects are its instances. §5.1
- **Class file** — The `.class` file `javac` produces, containing bytecode and a constant pool. §1.6
- **Class initialisation** — Running a class's static field initialisers and `static {}` blocks, once, on first active use. §5.10, §16.2
- **Class loader** — The object that finds and loads class bytes; loaders delegate to their parent first. §16.2
- **Class path** — The list of directories and JARs where the JVM looks for classes (`-cp`). §1.5
- **Class-File API** — The `java.lang.classfile` API (final in Java 24) for reading and writing class files. §17.9
- `Clock` — `java.time.Clock`: a replaceable source of "now"; inject it so tests can fix the time. §7.8, §18.9
- **Closure** — A function together with the variables it captured from its surrounding scope; lambdas capture effectively final locals. §11.5
- **CME** — Short for `ConcurrentModificationException`. §10.7
- **Code cache** — Memory where the JIT stores compiled machine code. §16.3
- **Code coverage** — The share of lines or branches that the tests execute, measured with tools such as JaCoCo; high coverage doesn't prove good tests. §18.9
- **Code point** — The number Unicode gives one character; a code point above U+FFFF needs two `char`s (a *surrogate pair*) in Java. §7.2
- **Code unit** — One 16-bit `char` of UTF-16 text; most characters take one code unit, some take two. §2.3, §7.2
- **Coffman conditions** — The four conditions that must all hold for a deadlock: mutual exclusion, hold and wait, no preemption, circular wait. Break any one to prevent it. §15.9
- **Collection** — The root interface of lists, sets and queues (maps are separate). §10.1
- `Collections` — Utility class with `sort`, `reverse`, `shuffle`, `frequency`, `unmodifiableList` and more. §10.9
- **Collector** — A recipe for accumulating stream elements into a result, e.g. `groupingBy`, `toMap`, `joining`. §12.5
- **Coloured pointers** — ZGC's trick of storing a few GC status bits inside object references themselves. §16.5
- **Compact constructor** — A record constructor without a parameter list, used to validate or normalise components. §6.8
- **Compact object headers** — Java 25 option (`-XX:+UseCompactObjectHeaders`) that shrinks object headers from 12 to 8 bytes; on by default from JDK 27. §16.4, §17.10
- **Compact source file** — A Java 25 source file with top-level methods and no class declaration; run with `java File.java`. §4.9, §17.10
- **Compaction** — Moving live objects next to each other so free memory forms one large block. §16.5
- `Comparable` — Interface whose `compareTo` defines a class's natural order. §10.8
- `Comparator` — A separate object whose `compare` defines an ordering; built with `comparing`, `thenComparing`, `reversed`. §10.8
- **Compiler** — A program that translates source code into another form: `javac` turns Java into bytecode, the JIT turns bytecode into machine code. §1.5, §1.7
- **Compile-time constant** — A `final` primitive or `String` initialised with a constant expression; folded by `javac`. §7.1
- `CompletableFuture` — A `Future` you can complete manually and compose asynchronously (`thenApply`, `thenCompose`, `allOf`). §15.2
- `CompletionException` — The unchecked wrapper `CompletableFuture.join()` throws when the computation failed. §15.2
- **Composition** — Building a class from other objects held in fields and delegating work to them — often preferred to inheritance. §6.1
- **Compound assignment** — Operators such as `+=`, which include an implicit cast back to the variable's type. §2.8

- **Compressed oops** — Storing object references in 32 bits on a 64-bit JVM (heaps up to about 32 GB) to save memory. §16.4
- **Concurrency** — Several tasks in progress over the same period, possibly interleaved on one core. §14.1
- **Concurrent collection** — A collection from `java.util.concurrent`, such as `ConcurrentHashMap` or `BlockingQueue`, designed for safe use by many threads at once. §14.12
- `ConcurrentHashMap` — A thread-safe, highly scalable hash map with atomic `merge`/`compute`; no `null` keys or values. §14.12
- `ConcurrentModificationException` — Thrown by fail-fast iterators when a collection is structurally changed during iteration. §10.7
- `Condition` — A waiting queue tied to an explicit `Lock` (`await`, `signal`); one lock can have several conditions. §14.9
- **Connected component** — A group of graph nodes that can all reach each other; counted with DFS, BFS or union-find. §19.16, §19.18
- **Connection pool** — A set of open database connections reused across requests, because opening one is slow (e.g. HikariCP). §18.11
- **Constant folding** — The compiler working out a constant expression such as `"ja" + "va"` at compile time. §7.1
- **Constant pool** — The table inside a class file that holds the literals and symbolic names its bytecode refers to. §1.6
- **Constructor** — A special method with the class's name that initialises a new object. §5.3
- **Constructor chaining** — Calling another constructor with `this(...)` or the parent's with `super(...)`. §5.3, §6.1
- **Container** — A lightweight, isolated package of an application with its own CPU and memory limits (e.g. Docker); the JVM reads those limits to size itself. §16.3, §16.7
- **Context switch** — The OS pausing one thread and running another on the same CPU core. §14.1
- **Contiguous** — Stored or taken next to each other with no gaps, like the elements of an array or a subarray. §4.6, §19.9
- `continue` — Skips to the next iteration of the innermost (or labelled) loop. §3.9
- `CopyOnWriteArrayList` — A thread-safe list that copies its array on every write; iterators see a snapshot. §14.12
- `CountDownLatch` — A one-shot synchronizer: threads `await()` until `countDown()` has been called n times. §14.13
- **Covariant return type** — An overriding method may return a subtype of the overridden method's return type. §6.3
- **CPU** — Central processing unit: the processor that executes instructions; its cores each run one thread at a time. §14.1
- **CPU-bound** — Work limited by computation rather than by waiting for I/O; more threads than cores won't speed it up. §12.8, §15.4
- **Critical section** — Code that touches shared mutable state and must not run in two threads at once. §14.1
- **Currying** — Turning a function of several arguments into a chain of one-argument functions. §11.8
- `CyclicBarrier` — A reusable synchronizer where n threads wait for each other before all continue. §14.13

D

- **Daemon thread** — A background thread that doesn't keep the JVM alive; virtual threads are always daemon. §14.2
- **DAG (directed acyclic graph)** — A graph whose edges have a direction and that contains no cycle; it always has a topological order. §19.17
- **Dangling else** — The question of which `if` an `else` belongs to when braces are missing; Java pairs it with the nearest `if`. §3.1
- **Data race** — Two threads access the same variable without a happens-before order and at least one writes. §15.8

- **Data structure** — A way of organising data in memory — array, list, hash table, tree, heap — chosen for the operations you need to be fast. §10.1, Ch 19
- `DateTimeFormatter` — Immutable, thread-safe formatter and parser for `java.time` values. §7.11
- **Daylight saving time (DST)** — The seasonal shift of clocks in many regions, which makes some local times skip or repeat. §7.10
- **Dead-code elimination** — The JIT removing computation whose result is never used — the classic trap in hand-written benchmarks. §16.9
- **Deadlock** — Threads wait forever for locks each other holds. §15.9
- **Declarative style** — Describing what result you want (filter, map, group) instead of writing the loop that produces it. §11.8, §12.11
- **Decorator** — A wrapper that adds behaviour to an object while offering the same interface, e.g. `BufferedReader` around a `Reader`. §13.5
- **Default constructor** — The no-argument constructor the compiler adds when a class declares no constructor. §5.3
- **Default method** — An interface method with a body (Java 8), inherited by implementing classes unless overridden. §6.6
- **Default value** — The initial value of a field or array element: `0`, `false` or `null`; local variables have none. §2.2, §2.3
- **Defensive copy** — Copying a mutable input or output so outside code can't change an object's internal state. §5.9
- **Definite assignment** — The compiler rule that a local variable must certainly have been assigned before it is read. §2.2
- **Deoptimisation** — The JIT discarding compiled code whose speculative assumptions stopped holding. §16.8
- **Dependency** — A library your project uses; build tools download it together with the libraries it needs in turn. §18.1, §18.2
- **Dependency injection** — Supplying an object's collaborators from outside (usually through the constructor) instead of creating them inside. §18.6, §18.14
- `Deque` — A double-ended queue: insert and remove at both ends; used as stack or queue. §10.6
- **Deserialisation** — Rebuilding objects from bytes; dangerous with untrusted input. §13.10
- **Devirtualisation** — The JIT replacing an overridable (virtual) call with a direct call when only one implementation has been seen. §16.8
- **DFS (depth-first search)** — Visiting a graph by following each path as deep as possible before backing up, using recursion or an explicit stack. §19.16
- **Diamond operator** — `<>` in `new HashMap<>()` — lets the compiler infer the type arguments. §9.2
- **Difference array** — An array of "change starts here" marks: add v at l and −v at r + 1, then one running sum applies every range update. §19.11
- **Dijkstra's algorithm** — Finds the cheapest paths from one node in a graph with non-negative edge weights, using a priority queue. §19.16
- **Direct memory** — Native memory outside the heap used by direct `ByteBuffer`s. §16.3
- **Divide and conquer** — Splitting a problem into smaller parts, solving each (often recursively or in parallel) and combining the results. §4.5, §15.3
- **Docker** — The most widely used tool for building and running containers. §16.3, §18.9
- `double` — The 64-bit IEEE 754 floating-point primitive; the default for decimals. §2.3, §2.11
- `do-while` — A loop that runs its body once before testing the condition. §3.8
- **Downcasting / upcasting** — Upcasting treats an object as one of its supertypes (automatic); downcasting casts back to a subtype and can throw `ClassCastException`. §6.4
- **Downstream collector** — A collector applied to each group inside `groupingBy` or `partitioningBy`, such as `counting()`. §12.5
- **DSA** — Data structures and algorithms — the subject of coding interviews. Ch 19
- **DSL** — Domain-specific language: a small language for one job, such as Gradle's Kotlin build scripts. §18.3

- **DTO** — Data transfer object: a plain data carrier passed between layers or over the network; a record is a natural fit. §6.8
- **Dummy (sentinel) node** — A placeholder node in front of a linked list's head, so changing the head needs no special case. §19.22
- `Duration` — A `java.time` amount of time in seconds and nanoseconds (e.g. `PT1H30M`). §7.10
- **Dutch national flag** — Splitting an array into three regions (less than, equal to, greater than a value) in one pass. §19.25
- **Dynamic dispatch** — Choosing which overridden method to run from the object's run-time class. §6.4
- **Dynamic programming (DP)** — Solving a problem from the answers to overlapping smaller subproblems, computing each answer once (memoisation or tabulation). §19.19

E

- **Eclipse Temurin** — The Adoptium project's free, widely used OpenJDK build. §1.2
- **Eden** — The part of the young generation where new objects are first allocated. §16.5
- **Effectively final** — A local variable that is never reassigned after initialisation; lambdas may capture only these. §11.5
- **Encapsulation** — Hiding an object's state behind methods so its invariants can't be broken from outside. §5.7
- **Encoding** — The rules that turn characters into bytes and back; see *charset*. §13.7
- **Encounter order** — The order in which a stream meets its elements: a `List` has one, a `HashSet` does not. §12.4
- **Endianness** — The byte order of multi-byte numbers: big-endian (Java's default) stores the most significant byte first. §13.9
- **End-to-end test** — A test of the whole running system through its real entry points, as a user would use it. §18.9
- **Enhanced `for`** — The for-each loop, `for (T x : iterable)`. §3.8
- **Enum** — A type with a fixed set of named instances; may have fields, constructors and methods. §6.9
- `EnumMap` / `EnumSet` — Very compact, fast map/set implementations for enum keys/elements. §10.4, §10.3
- **Environment variable** — A named setting the operating system passes to every program it starts, such as `JAVA_HOME` or `PATH`. §1.4
- **Epoch** — The fixed instant 1970-01-01T00:00:00Z from which `Instant` and `System.currentTimeMillis()` count. §7.10
- **Epsilon GC** — A no-op collector that never frees memory; for testing. §16.5
- `equals` — The method that defines logical equality; must be consistent with `hashCode`. §6.13
- **Equivalence class** — A group of inputs that the code treats the same way, so one test (plus the boundaries) covers it. §18.9
- **Erasure** — See *type erasure*. §9.7
- `Error` — The `Throwable` branch for serious JVM problems such as `OutOfMemoryError`; unchecked, normally not caught. §8.2
- **Escape analysis** — A JIT analysis proving an object doesn't leave a method, enabling scalar replacement and lock elision. §16.8
- **Escape sequence** — A backslash code for a special character in a literal: `\n`, `\t`, `\"`, `\\`, `A`. §2.4
- **Exception** — An object signalling an abnormal condition, thrown with `throw` and caught with `catch`. §8.1
- **Exception chaining** — Wrapping a low-level exception as the cause of a higher-level one. §8.8
- **Exception translation** — Catching an exception at a layer boundary and throwing one that fits the higher layer's abstraction. §8.8
- **Executable JAR** — A JAR whose manifest names a `Main-Class`, so it runs with `java -jar app.jar`. §18.12
- `ExecutorService` — A thread-pool abstraction that accepts tasks and returns `Future`s; `AutoCloseable` since Java 19. §14.11

- **Exhaustive switch** — A `switch` whose cases cover every possible value (all enum constants, all permitted subtypes, or `default`). §3.3, §3.6
- **Explicit lock** — A `java.util.concurrent.locks.Lock` such as `ReentrantLock`, acquired and released in code. §14.9
- **Expression** — Code that produces a value, such as `a + b`, `x > 0` or a method call; a statement is a complete instruction. §2.1

F

- **Factory method** — A static method that creates instances (`List.of`, `Integer.valueOf`); unlike a constructor it has a name and may cache or return a subtype. §5.3
- **Fail-fast iterator** — An iterator that throws `ConcurrentModificationException` on a best-effort basis when its collection changes structurally. §10.7
- **Failure atomicity** — A method that fails leaves its object exactly as it was before the call. §8.7, §8.10
- **Fair lock** — A lock that grants access in waiting order, avoiding starvation at some cost in speed. §14.9
- **Fake** — A working but simplified test double, such as an in-memory repository. §18.6
- **Fall-through** — In a classic `case X:` switch, execution running on into the next case when there is no `break`. §3.2
- **Fast and slow pointers** — Two references moving through a list at different speeds, used to find the middle or detect a cycle. §19.22
- **Fat JAR** — A single JAR containing the application and all of its dependencies (also "uber JAR"). §18.12
- **Feature release** — One of the two Java versions released every year, in March and September. §1.12
- **Field** — A variable declared in a class that holds an object's state (or, if `static`, the class's). §5.1
- **FIFO / LIFO** — First in, first out (a queue) / last in, first out (a stack). §10.1, §10.6
- **File descriptor** — The operating system's small-integer handle for an open file or socket; each process may hold only a limited number. §16.13
- **`final`** — Modifier: a final variable is assigned once, a final method can't be overridden, a final class can't be extended. §5.8
- **Finalization** — The deprecated-for-removal `finalize()` mechanism (Java 18, JEP 421); use try-with-resources or `Cleaner`. §17.12
- **`finally`** — A block after `try` that always runs when control leaves the `try`. §8.4
- **Flaky test** — A test that sometimes passes and sometimes fails with no code change, usually because of timing, randomness or shared state. §18.9
- **Flexible constructor bodies** — Java 25 feature allowing statements that don't use `this` before `super(...)` or `this(...)`. §17.10
- **`float`** — The 32-bit IEEE 754 floating-point primitive (~7 significant digits). §2.3
- **Floating-point error** — Rounding error because binary fractions can't represent most decimals exactly (`0.1 + 0.2`). §2.11
- **Floyd's cycle-finding algorithm** — Detecting a loop with a slow pointer (1 step) and a fast pointer (2 steps): they meet only if there is a cycle. §19.22
- **Foreign Function & Memory API** — The `java.lang.foreign` API (final in Java 22) to call native code and manage off-heap memory, replacing JNI. §17.7
- **Fork/Join** — A framework for recursive divide-and-conquer tasks run by a work-stealing `ForkJoinPool`. §15.3
- **Formatter specifier** — A pattern such as `%5.2f` or `%-10s` in `String.format`/`printf`. §7.4
- **Fragile base class** — A subclass breaking because its superclass's internals changed — a risk of inheritance across teams or libraries. §6.1
- **Frame (stack frame)** — The record one method call keeps on its thread's stack: parameters, local variables and where to return. §4.5, §16.3
- **`fsync`** — An operating-system call that forces written data out of memory caches onto the disk. §16.13

- **Full GC** — A collection of the whole heap, usually with a long stop-the-world pause. §16.5
- **Functional interface** — An interface with exactly one abstract method; a lambda or method reference can implement it. §11.2
- **Functional programming** — A style built from pure functions, immutable data and functions passed around as values. §11.8
- `Future` — A handle to an asynchronous result, obtained with a blocking `get()`. §15.1

G

- **GA (general availability)** — The official release of a Java version for production use. §20.15
- **Garbage collection (GC)** — Automatic reclamation of memory used by objects that are no longer reachable. §1.8, §16.5
- **Garbage-First (G1)** — The default collector since Java 9 (in all environments from JDK 27); region-based, with a pause-time goal. §16.5
- **Gatherer** — A custom intermediate stream operation used with `stream.gather(...)` (final in Java 24). §12.9
- **GC roots** — Starting points for reachability: thread stacks, static fields, JNI references. §16.5
- **GCD / LCM** — Greatest common divisor (the largest number dividing both) / least common multiple (the smallest number both divide). §19.27
- **Generational hypothesis** — Most objects die young, so collectors split the heap into young and old generations. §16.5
- **Generic method** — A method that declares its own type parameters: `static <T> T first(List<T> xs)`. §9.4
- **Generics** — Type parameters on classes, interfaces and methods for compile-time type safety without casts. §9.1
- **Getter / setter** — Methods that read (`getName()`) or change (`setName(...)`) a private field. §5.7
- **GIL** — Python's global interpreter lock, which lets one thread run Python code at a time; Java has none, so its threads truly run in parallel. §14.14
- **GraalVM** — Oracle's JDK distribution with an advanced JIT and *Native Image*. §1.2, §18.14
- **Gradle** — A build tool using a Groovy or Kotlin DSL with incremental, cached tasks. §18.3
- **Graph** — A set of nodes joined by edges, which may have a direction and a weight (cost); trees, grids and road maps are graphs. §19.16
- **Greedy algorithm** — Building an answer by always taking the choice that looks best right now and never undoing it; correct only when the greedy-choice property holds. §19.24
- **Greedy-choice property** — Some optimal answer starts with the locally best choice — the condition that makes a greedy algorithm correct. §19.24
- **Guard clause** — An early `return` (or `throw`) at the top of a method for special cases, keeping the main logic unindented. §3.1
- **Guard (pattern)** — A `when` condition attached to a `case` pattern. §3.6
- **GUI** — Graphical user interface: windows, buttons and menus. §16.10

H

- **Handle-or-declare rule** — A checked exception must be caught or listed in the method's `throws` clause. §8.3
- **Happens-before** — The Java Memory Model relation guaranteeing one action's effects are visible to another. §15.8
- **Hash table** — An array of buckets indexed by each key's hash code, giving O(1) average lookups; `HashMap` is one. §10.5
- `hashCode` — Returns an `int` used by hash-based collections; equal objects must return equal hash codes. §6.13
- `HashMap` — An unordered hash-table map with average O(1) operations; allows one `null` key. §10.4, §10.5
- `HashSet` — A set backed by a `HashMap`; unordered, average O(1). §10.3
- **Heap** — The JVM memory area where all objects and arrays live, shared by all threads and garbage-collected. §16.3

- **Heap (data structure)** — A tree kept in an array where each parent is ≤ its children (a min-heap), so the smallest item is always at the root; O(log n) insert and remove. Unrelated to the JVM *heap*. §10.6, §19.13
- **Heap dump** — A snapshot of every object in the heap, analysed with Eclipse MAT or VisualVM. §16.10, §16.11
- **Heap pollution** — A variable of a parameterized type referring to an object of the wrong type, caused by unchecked operations. §9.8
- **Helpful NullPointerException** — NPE messages that name exactly which expression was `null` (Java 14, on by default since 15). §8.1
- **Hexadecimal** — Base-16 notation with digits 0–9 and a–f, written `0x2A` in Java. §2.4
- **Hibernate** — The most popular JPA implementation, which maps Java objects to database tables. §18.11, §18.14
- **Hiding** — A static method or field in a subclass with the same name as the parent's; chosen by compile-time type, not overridden. §6.2
- **Higher-order function** — A function that takes or returns another function, such as `map` or `Comparator.comparing`. §11.8
- **HikariCP** — A fast JDBC connection pool; Spring Boot's default. §18.11
- **HotSpot** — The JVM inside OpenJDK and most Java distributions, named after its compilation of "hot" code. §1.3
- **HTTP** — Hypertext Transfer Protocol: the request/response protocol of the web, spoken by browsers, REST APIs and Java's `HttpClient`. §18.14
- **Humongous object** — In G1, an object at least half a region in size, which gets regions of its own. §16.5

I

- **IDE** — Integrated development environment, such as IntelliJ IDEA, Eclipse or VS Code: editor, compiler, debugger and tools in one. §1.11
- **Identity** — Whether two references point to the very same object (`==`), as opposed to being equal in value (`equals`). §6.13
- **`IdentityHashMap` / `WeakHashMap`** — Special maps: `IdentityHashMap` compares keys with `==`; `WeakHashMap` drops an entry once nothing else references its key. §10.4
- **IEEE 754** — The international standard for binary floating-point numbers that `float` and `double` follow. §2.11
- **`IllegalArgumentException`** — Unchecked exception for a method argument that violates a precondition. §8.7
- **`IllegalStateException`** — Unchecked exception for a call made when the object is in the wrong state. §8.7
- **Immutable object** — An object whose state can't change after construction, such as `String` or a record of immutable values. §5.9
- **Incidental indentation** — The common leading whitespace that a text block strips from every line. §2.12
- **Incubator module** — An experimental API in a `jdk.incubator.*` module, e.g. the Vector API. §17.1
- **In-degree** — The number of edges pointing into a node; Kahn's algorithm starts from the nodes with in-degree 0. §19.17
- **Inheritance** — A subclass acquiring a superclass's members with `extends`; Java has single class inheritance. §6.1
- **Initialisation order** — Static initialisers once per class, then for each object: parent constructor, field initialisers and instance blocks, constructor body. §5.10
- **Inlining** — The JIT copying a callee's body into its caller to remove call overhead and enable further optimisation. §16.8
- **Inner class** — A non-static nested class; each instance holds a reference to an enclosing instance. §6.10
- **In-place** — Changing the input directly with O(1) extra memory instead of building a new copy. §19.8

- **Instance** — An object created from a class; instance members belong to each object, `static` ones to the class. §5.1
- **Instance initialiser block** — A `{ … }` block in a class body that runs for every new object, before the rest of the constructor body. §5.10
- **Instance main method** — A Java 25 `main` that is not `static` and may take no parameters: `void main()`. §4.9, §17.10
- `instanceof` — Operator testing an object's run-time type; with a pattern it also binds a variable (Java 16). §3.5
- `Instant` — A point on the UTC time-line, stored as seconds and nanoseconds since the epoch; use it for timestamps. §7.10
- `int` — The 32-bit signed integer primitive, range -2^{31} … $2^{31}-1$. §2.3
- `Integer` **cache** — `Integer.valueOf` returns cached objects for −128 … 127, so `==` happens to work only in that range. §7.6
- **Integer division** — Dividing two integers drops the fraction, rounding toward zero: `7 / 2` is 3 and `-7 / 2` is −3. §2.8, §19.6
- **Integer overflow** — A result outside a type's range wraps around silently (`Integer.MAX_VALUE + 1` is `MIN_VALUE`). §2.10
- **Integration test** — A test that runs several real parts together (for example code plus a real database) instead of one class in isolation. §18.9
- **IntelliJ IDEA** — JetBrains' widely used Java IDE. §1.11
- **Interface** — A contract type: abstract, default, static and private methods plus constants; a class can implement many. §6.6
- **Intermediate operation** — A lazy stream operation such as `filter` or `map` that returns a new stream. §12.3
- **Interning** — Obtaining the pooled copy of a string with `intern()`. §7.1
- **Interpreter** — The JVM component that executes bytecode instruction by instruction before the JIT compiles hot code. §1.7
- **Interruption** — Cooperative cancellation: `interrupt()` sets a flag that blocking methods answer with `InterruptedException`. §14.4
- **Interval** — A range with a start and an end, such as a meeting `[9, 11)`; interval problems usually sort by start and sweep. §19.15
- **Intrinsic lock** — Another name for an object's *monitor*. §14.6
- **Invariance (generics)** — `List<Integer>` is not a `List<Number>` even though `Integer` is a `Number`; wildcards add the flexibility back. §9.6
- **Invariant** — A condition that must always hold — for an object after every public method, or for a loop on every iteration. §5.7
- **Inverse Ackermann function** — α(n), a function that grows so slowly it is below 5 for any realistic n; union-find costs O(α(n)) per operation. §19.18
- `invokedynamic` — A bytecode instruction whose call target is linked at run time; Java uses it for lambdas and string concatenation. §1.6
- **I/O** — Input/output: reading and writing files, network sockets, the console and databases. §13.1
- `IO` **class** — `java.lang.IO` (Java 25): `println`, `print` and `readln` for simple console programs. §4.9, §17.10
- `Iterable` — Anything with an `iterator()`; usable in an enhanced `for`. §10.1, §10.7
- `Iterator` — An object with `hasNext`, `next` and optional `remove` for walking a collection. §10.7

J

- **Jackson / Gson** — Popular third-party libraries for reading and writing JSON in Java. §13.12
- **Jakarta EE** — The Eclipse Foundation's enterprise Java specifications (formerly Java EE): Servlet, JPA, CDI, REST. §1.2, §18.14
- **JAR** — A ZIP archive of class files and resources with a manifest; executable with `java -jar` when it names a `Main-Class`. §18.12
- **Java agent** — A JAR loaded with `-javaagent` that can change classes as they load (used by profilers and Mockito); loading one into a running JVM now prints a warning. §17.6, §18.6

- **Java Memory Model (JMM)** — The rules defining when writes by one thread become visible to others. §15.8
- **Java SE** — Java Standard Edition: the core language, JVM and standard library this book covers. §1.2
- `javac` — The Java compiler; turns `.java` source into `.class` bytecode. §1.5
- **Javadoc** — Documentation comments `/** … */` and the tool that turns them into HTML API pages. §1.5, §2.1
- `JAVA_HOME` — The environment variable naming the JDK directory that build tools should use. §1.4
- `java.time` — The modern date-time API (Java 8): immutable, thread-safe classes such as `LocalDate` and `Instant`. §7.8
- **JCIP** — *Java Concurrency in Practice*, the classic book on Java threads and locking. §14.4
- `jcmd` — The command-line tool that sends diagnostic commands to a running JVM. §16.10
- **JDBC** — The standard API for talking to relational databases: `Connection`, `PreparedStatement`, `ResultSet`. §18.11
- **JDK** — Java Development Kit: the JVM, the class library and tools such as `javac`, `jar`, `jshell`, `jcmd`. §1.3
- **JDK Mission Control (JMC)** — A desktop application for analysing JFR recordings. §16.10
- **JEP** — JDK Enhancement Proposal: the numbered document describing a new feature. §1.12, §17.1
- **JFR (JDK Flight Recorder)** — Low-overhead event recording built into the JVM, for production profiling. §16.10
- **JIT compiler** — Just-in-time compiler that turns hot bytecode into optimised machine code while the program runs. §1.7, §16.8
- `jlink` — Builds a custom runtime image containing only the modules an application needs. §16.12
- **JLS** — The Java Language Specification: the official definition of the language's rules. §4.3
- **JMH** — The Java Microbenchmark Harness, which handles warm-up, forks and dead-code elimination for benchmarks. §16.9
- **JMX** — Java Management Extensions: the JVM's API for publishing and reading monitoring data and controls. §16.1
- **JNI** — Java Native Interface: the older way to call C or C++ code from Java, being replaced by the *Foreign Function & Memory API*. §16.1
- **Josephus problem** — n people stand in a circle and every k-th is removed; the survivor's position follows J(n) = (J(n − 1) + k) mod n. §19.27
- **JPA** — Jakarta Persistence: the standard API for mapping objects to database tables, usually through Hibernate. §1.2, §18.14
- **JPMS** — The Java Platform Module System (Java 9): `module-info.java` with `requires`, `exports`, `opens`. §16.12
- **JRE** — Java Runtime Environment: the JVM plus class library, enough to run but not compile programs; since Java 11 Oracle ships only the JDK (trimmed runtimes are built with `jlink`). §1.3
- `jshell` — The Java REPL (Java 9) for trying expressions interactively. §1.9
- **JSON** — JavaScript Object Notation: a common text format of objects, arrays, strings and numbers for exchanging data. §13.12
- `jstack` — Prints a thread dump of a running JVM. §16.10
- **JUL** — Short for `java.util.logging`, the JDK's built-in logging API. §18.10
- **JUnit** — The standard Java testing framework; tests are methods annotated `@Test`. §18.4
- **JUnit Platform / Jupiter / Vintage** — The parts of JUnit: the Platform discovers and launches tests, Jupiter is the modern test API, Vintage runs old JUnit 4 tests. §18.4
- **JVM** — Java Virtual Machine: loads, verifies and runs bytecode, manages memory and threads. §1.3, §16.1

K

- **Kafka** — A distributed, durable message log used to stream events between services. §20.18
- **Kahn's algorithm** — Topological sort by repeatedly taking a node with in-degree 0 and removing its outgoing edges. §19.17

- **Kernel** — The core of the operating system that manages memory, devices and scheduling; programs reach it through system calls. §16.13
- **Key derivation function API** — `javax.crypto.KDF` (Java 25) for deriving keys, e.g. with HKDF. §17.10
- **Keyword** — A word reserved by the language (`class`, `if`, `int`); contextual keywords such as `var` and `record` are special only in certain positions. §2.1
- **Kotlin** — A JetBrains language for the JVM that is fully interoperable with Java. §20.18
- **Kruskal's algorithm** — Building a minimum spanning tree by taking edges cheapest first and skipping any that would close a cycle (checked with union-find). §19.18

L

- **Label** — A name before a loop (`outer:`) that `break` or `continue` can target. §3.9
- **Lambda expression** — An anonymous function, `(params) -> body`, implementing a functional interface. §11.2
- **Latency** — The time one operation takes from start to finish — a request, a disk read, a GC pause. §16.5, §16.6, System Design Fundamentals
- **Lazy evaluation** — Deferring work until a result is needed; streams do nothing until a terminal operation runs. §12.2
- **LeetCode** — A popular website of coding-interview practice problems; "LeetCode-style" means a timed algorithm question. Ch 19
- **Lexicographic order** — Dictionary order: compare the first elements and look at the next ones only on a tie. §4.8
- **Lifecycle (Maven)** — Maven's fixed sequence of build phases (`validate` … `compile` … `test` … `package` … `deploy`); running a phase runs every earlier one. §18.2
- **Linked list** — A chain of nodes, each holding a value and a reference to the next node (and, if doubly linked, the previous one). §10.2, §19.22
- `LinkedHashMap` — A hash map that keeps insertion (or access) order; the basis of a simple LRU cache. §10.4
- `LinkedHashSet` — A hash set that remembers insertion order. §10.3
- `LinkedList` — A doubly linked `List` and `Deque`; O(1) at the ends, O(n) indexing. §10.2
- **Liskov substitution principle** — A subclass must work anywhere its parent is expected, without surprising the caller. §6.1
- `ListIterator` — An iterator over a `List` that can move in both directions and `set` or `add` elements as it goes. §10.2
- **Literal** — A value written directly in source code: `42`, `3.14`, `'a'`, `"hi"`, `true`. §2.4
- **Little's law** — Requests in flight = arrival rate × time per request; e.g. 1,000 requests/s each taking 0.1 s keep 100 busy at once. §15.4
- **Livelock** — Threads keep changing state in response to each other but make no progress. §15.9
- **Liveness** — A program's ability to keep making progress; deadlock, livelock and starvation are liveness failures. §15.9
- **Load average** — On Linux, the average number of threads running or waiting to run over the last 1, 5 and 15 minutes. §16.13
- **Load barrier** — A tiny piece of code ZGC adds to every reference read so it can move objects while the application keeps running. §16.5
- **Load factor** — The fullness ratio (default 0.75) at which a `HashMap` doubles its table. §10.5
- **Local class** — A class declared inside a method body. §6.11
- **Local variable type inference** — `var` (Java 10): the compiler infers a local variable's type from its initializer. §2.6
- `LocalDate` — A date without time or zone, e.g. `2025-03-09`. §7.9
- `LocalDateTime` — A date and time without a zone. §7.9
- **Locale** — A language-and-region setting (`Locale.US`, `Locale.ROOT`) that controls number, date and case formatting. §7.2, §7.4
- `LocalTime` / `OffsetDateTime` — `LocalTime` is a time of day with no date or zone; `OffsetDateTime` is a date-time with a fixed offset from UTC. §7.8

- **Lock elision** — The JIT removing locking on objects proved not to escape their thread. §16.8
- **Lock ordering** — Always acquiring several locks in one agreed global order, which makes deadlock impossible. §15.9
- **Logback / Log4j 2** — Logging back-ends that write SLF4J log calls to the console, files or other destinations. §18.10
- **Logging** — Recording events with a level and logger name to configurable destinations (SLF4J, Logback, `java.util.logging`). §18.10
- **Lombok** — A library that generates boilerplate such as getters at compile time by hooking into `javac`. §17.12
- **Lomuto partition** — A simple partition scheme that keeps one `store` index; everything left of it is smaller than the pivot. §19.25
- `long` — The 64-bit signed integer primitive, about ±9.2 × 10^{18}. §2.3
- `LongAdder` — A counter spread over several cells to avoid contention; faster than `AtomicLong` when many threads increment it. §14.10
- **Loop unrolling** — The JIT repeating a loop body several times per iteration to cut loop overhead. §16.8
- **Lowest common ancestor (LCA)** — The deepest tree node that has both given nodes beneath it (a node counts as its own descendant). §19.23
- **LRU cache** — A cache that evicts the least-recently-used entry; built with an access-ordered `LinkedHashMap`. §10.4
- **LTS release** — A long-term-support Java release (8, 11, 17, 21, 25) that vendors patch for years. §1.12

M

- `main` **method** — The program entry point: classically `public static void main(String[] args)`. §4.9
- **Manifest** — `META-INF/MANIFEST.MF` inside a JAR: metadata such as the `Main-Class` to run. §18.12
- `Map` — A key → value collection with unique keys; not a subtype of `Collection`. §10.4
- **Mark word** — The part of an object header that holds the identity hash code, lock state and GC age. §16.4
- **Mark-and-sweep** — GC that first marks every reachable object, then frees the rest. §16.5
- **Marker interface** — An interface with no methods that just tags a class, such as `Serializable`. §6.2
- `Math` — Utility class for arithmetic: `abs`, `pow`, `sqrt`, `round`, `floorMod`, `clamp`, `*Exact`. §7.5
- **Maven** — A build tool driven by `pom.xml`, with a fixed lifecycle of phases and a central repository. §18.2
- **Maven Central** — The main public repository from which Maven and Gradle download libraries. §18.1, §18.2
- **MDC (mapped diagnostic context)** — Per-thread key-value data, such as a request id, that the logging framework adds to every log line. §18.10
- **Megamorphic call** — A call site that has seen many different receiver classes, which the JIT can't inline well. §11.8, §16.8
- **Memoisation** — Caching a function's results by its arguments so each is computed only once — top-down dynamic programming. §19.19
- **Memory leak (Java)** — Objects that are no longer needed but stay reachable, so the GC can't free them. §16.11
- **Memory-mapped file** — A file mapped into the process's address space, so reading or writing memory reads or writes the file. §13.9
- **Merge sort** — Sorting by splitting the input in half, sorting each half and merging them; O(n log n) and stable. §19.25
- **Metaspace** — Native memory holding class metadata (replaced PermGen in Java 8). §16.3
- **Method** — A named block of code in a class that can take parameters and return a value. §4.1
- **Method overloading** — Several methods with the same name but different parameter lists, chosen at compile time. §4.3
- **Method overriding** — A subclass providing its own implementation of an inherited instance method, chosen at run time. §6.3

- **Method reference** — Shorthand for a lambda that calls one method: `String::length`, `System.out::println`, `ArrayList::new`. §11.4
- **Method signature** — A method's name plus its parameter types. §4.1
- **Microbenchmark** — A timing of a tiny piece of code; easy to get wrong without a harness such as JMH. §16.9
- **Min-heap / max-heap** — A heap whose root is always the smallest / largest element; `PriorityQueue` is a min-heap by default. §19.4, §19.13
- **Minimum spanning tree** — The cheapest set of edges that connects every node of a weighted graph without a cycle (Kruskal's algorithm). §19.18
- **Mock** — A test double that records calls and returns programmed answers (e.g. with Mockito). §18.6
- **Mockito** — The most widely used Java mocking library: `mock`, `when(...).thenReturn`, `verify`. §18.6
- **Modular arithmetic** — Working with remainders after division by a fixed number m; answers are often reported "mod 1,000,000,007" to keep them small. §19.6, §19.27
- **Modular JAR** — A JAR that contains a `module-info.class`. §18.12
- **Module** — A named, self-describing group of packages declared in `module-info.java`. §16.12
- **Module import declaration** — `import module java.base;` (Java 25): imports every package a module exports. §17.10
- **Module path** — Where the JVM looks for modules (`--module-path`), as opposed to the class path. §16.12
- **Mojibake** — Garbled text produced by decoding bytes with the wrong charset. §13.7
- **Monitor** — The lock every Java object has, used by `synchronized`, `wait` and `notify`. §14.6
- **Monotonic stack** — A stack kept in increasing or decreasing order by popping out-of-order items first; finds each "next greater element" in O(n). §19.12
- **Mounting / unmounting** — A virtual thread being attached to, or detached from, a carrier thread; blocking I/O unmounts it. §15.4
- **Multi-catch** — `catch (A \| B e)` — one handler for several unrelated exception types. §8.5
- **Multi-release JAR** — A JAR that holds different versions of some classes for different Java releases. §18.12
- **Mutable** — Able to change state after creation. §5.9
- **Mutable reduction** — Accumulating stream elements into a mutable container such as a list or map — what `collect` does. §12.5
- **Mutation testing** — Judging tests by planting small bugs ("mutants") and checking that the tests fail. §18.9
- **Mutual exclusion** — Letting only one thread at a time into a critical section. §14.6

N

- **NaN** — "Not a Number": the `double` result of undefined operations such as `0.0 / 0`; it is not even equal to itself. §2.11
- **Narrowing conversion** — Converting to a type with a smaller range (`long` → `int`); requires a cast and may lose information. §2.7
- **Native Image** — GraalVM's ahead-of-time compiler that turns a Java application into a fast-starting stand-alone executable. §18.12, §18.14
- **Native memory** — Memory the JVM uses outside the Java heap: metaspace, thread stacks, direct buffers, GC data. §16.3
- **Native Memory Tracking (NMT)** — A JVM option (`-XX:NativeMemoryTracking=summary`) that reports native memory use by area. §16.3
- **Natural order** — The order a class defines for itself by implementing `Comparable` (numbers ascending, strings alphabetically). §10.8
- **Nested class** — A class declared inside another; static nested or inner. §6.10
- **Netty** — A high-performance asynchronous networking framework for the JVM. §13.9
- **NIO.2** — The `java.nio.file` API (Java 7): `Path`, `Files`, `FileSystem`, `WatchService`. §13.1
- **Node** — One element of a linked structure (list, tree or graph) holding a value and references to its neighbours. §19.16, §19.22

- **Non-blocking** — An operation that returns immediately instead of making the thread wait, such as a CAS update. §14.10
- **NPE** — Short for `NullPointerException`. §8.1
- `null` — The reference value meaning "no object". §2.5
- `NullPointerException` — Thrown when code uses a `null` reference as if it pointed to an object. §8.1

O

- **Object** — An instance of a class, with state (fields) and behaviour (methods), living on the heap. §5.1
- `Object` **class** — The root of every class hierarchy; defines `equals`, `hashCode`, `toString`, `getClass`, `wait`, `notify`. §6.2, §6.13
- **Object header** — The bookkeeping at the start of every object — mark word and class pointer: 12 bytes, or 8 with compact headers. §16.4
- **Observer / listener** — An object registered to be told about events; forgetting to unregister listeners is a classic memory leak. §16.11
- **OCP** — Oracle Certified Professional: the Java SE 21 Developer certification exam (1Z0-830). §20.18
- **Off-by-one error** — A loop or index that goes one step too far or stops one too short (`<=` instead of `<`). §3.10
- **Old generation** — The heap area for objects that survived several young collections. §16.5
- **On-stack replacement (OSR)** — Switching a long-running loop from interpreted to compiled code while it is still running. §16.8
- **OOM** — Short for `OutOfMemoryError`. §16.3
- **OOP** — Object-oriented programming: organising code as objects that combine state and behaviour, using encapsulation, inheritance and polymorphism. §5.1
- **OpenJDK** — The open-source project and code base from which every mainstream Java distribution is built. §1.2
- **Operating system (OS)** — The software (Linux, Windows, macOS) that manages hardware, processes, memory and files for programs. §16.13
- **Operator** — A symbol that computes a value from operands, such as `+`, `==`, `&&` or `?:`. §2.8
- **Optimal substructure** — The best answer is built from the best answers to subproblems — one of the two conditions for dynamic programming. §19.19
- `Optional` — A container holding one value or none; a return type for "maybe no result". §11.7
- **Ordinal** — An enum constant's zero-based position in its declaration (`ordinal()`). §6.9
- `OutOfMemoryError` — Thrown when the JVM can't allocate memory in the heap, metaspace or native memory. §16.3
- **Overlapping subproblems** — Plain recursion would solve the same subproblem many times — the other condition for dynamic programming. §19.19

P

- **P99** — The 99th-percentile latency: 99 % of operations finished at least this fast. §16.6
- **Package** — A namespace grouping related types, declared with `package`; mirrors the directory structure. §5.4
- **Package-private** — The default access (no modifier): visible within the same package only. §5.6
- **Page cache** — Operating-system memory holding recently read or written file data, so repeated reads skip the disk. §16.13
- **Page fault** — A CPU trap when a program touches memory that isn't currently mapped to RAM; the OS must load or allocate the page. §16.13
- **Palindrome** — Text (or a list) that reads the same forwards and backwards. §19.8
- **Parallel GC** — A throughput collector that uses many threads in stop-the-world pauses. §16.5
- **Parallel stream** — A stream whose work is split across the common `ForkJoinPool`; only worth it for large CPU-bound tasks. §12.8
- **Parallelism** — Tasks literally running at the same instant on different CPU cores; concurrency only requires them to overlap in time. §14.1
- **Parameterized test** — A test run once per set of arguments (`@ParameterizedTest` with a source). §18.5

- **Partition** — Rearranging an array around a pivot so smaller values end up on its left; the core step of quicksort and quickselect. §19.25
- **Pass-by-value** — Java always copies the argument value into the parameter; for objects, the value copied is the reference. §4.2
- `Path` — An NIO.2 object naming a file-system location; doesn't touch the disk by itself. §13.2
- **Path compression** — In union-find, pointing nodes at (or nearer to) the root during `find`, so later finds are almost O(1). §19.18
- **Path traversal** — An attack that uses `../` in a user-supplied file name to escape the intended directory. §13.2
- **Pattern matching** — Testing a value's shape and extracting parts in one step: type, record and unnamed patterns. §3.5, §3.6, §3.7
- **Pattern variable** — The variable a pattern binds when it matches, such as `s` in `o instanceof String s`. §3.5
- **PECS** — "Producer `extends`, consumer `super`": the wildcard rule for flexible generic APIs. §9.6
- `Period` — A `java.time` amount in years, months and days (e.g. `P1Y2M5D`). §7.9
- **Permitted subclass** — A class listed in a sealed type's `permits` clause. §6.7
- `Phaser` — A reusable barrier-like synchronizer whose number of parties can change between phases. §14.13
- **Pinning** — A virtual thread blocking without unmounting, holding its carrier; `synchronized` no longer pins since Java 24. §15.5
- **Pivot** — The value a partition splits around; picking it at random avoids quicksort's O(n²) worst case. §19.25
- **Platform thread** — A Java thread backed one-to-one by an OS thread. §15.4
- **Plugin (build)** — An add-on that gives Maven or Gradle a job to do, such as compiling, running tests or packaging. §18.2
- **Pointer (in algorithms)** — An index or reference that marks a position while an algorithm walks the data, such as `left` and `right`. §19.8
- **Poison pill** — A special item put on a queue to tell its consumer to stop. §14.8, §14.12
- **Polymorphism** — One reference type, many run-time behaviours: a call runs the implementation of the object's actual class. §6.4
- **Precedence** — The order in which operators bind (`*` before `+`, `&&` before `\|\|`). §2.9
- **Precise rethrow** — `catch (Exception e) { throw e; }` keeps the narrower checked types the `try` can actually throw. §8.5
- **Prefix sum** — Running totals where `prefix[i]` is the sum of the first i elements, so any range sum costs one subtraction. §19.11
- `PreparedStatement` — A pre-compiled JDBC statement with `?` parameters; prevents SQL injection. §18.11
- **Preview feature** — A complete feature that may still change; needs `--enable-preview` on `javac` and `java`. §1.12, §17.1
- **Primitive stream** — `IntStream`, `LongStream` or `DoubleStream`, which avoid boxing and add `sum`/`average`. §12.6
- **Primitive type** — One of the eight built-in value types: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`. §2.3
- `PriorityQueue` — A binary-heap queue that always removes the smallest element (by natural order or a comparator). §10.6
- `private` — Access modifier: visible only inside the declaring top-level class. §5.6
- **Process** — A running program with its own memory; the threads inside it share that memory. §14.1, §16.13
- **Producer–consumer** — One set of threads creating work items and another processing them, joined by a shared (usually blocking) queue. §14.12
- **Profiling** — Measuring where a running program spends CPU time or allocates memory (JFR, async-profiler). §16.10
- **Promotion (numeric)** — Automatic widening of `byte`, `short` and `char` to `int` (or to the wider operand type) in arithmetic. §2.7

- **`Properties`** — A key=value configuration map loaded from `.properties` files. §13.11
- **`protected`** — Access modifier: visible in the package and to subclasses. §5.6
- **Pseudo-random number generator** — An algorithm whose output looks random but is fully determined by its seed. §7.12
- **Pure function** — A function whose result depends only on its arguments and that has no side effects. §11.8

Q

- **Quarkus / Micronaut** — Cloud-native Java frameworks designed for fast start-up and low memory. §18.14, §20.18
- **Queue** — A collection that hands elements out in arrival order (FIFO): `offer` adds at the tail, `poll` removes from the head. §10.6
- **Quickselect** — Finding the k-th smallest element by partitioning and continuing on one side only; O(n) on average. §19.25
- **Quicksort** — Sorting by partitioning around a pivot and recursing on both sides; O(n log n) on average. §19.25

R

- **Race condition** — A result that depends on the timing of threads, e.g. lost updates from `count++`. §14.5
- **Raw type** — A generic type used without type arguments (`List`); disables generic type checking. §9.7
- **Reachability** — An object is live if a chain of references leads to it from a GC root. §16.5
- **Read-modify-write** — Reading a value, computing a new one and writing it back (`count++`); not atomic unless protected. §14.5
- **`ReadWriteLock`** — A lock pair allowing many concurrent readers or one writer. §14.9
- **Record** — A concise immutable data carrier (Java 16) with generated constructor, accessors, `equals`, `hashCode`, `toString`. §6.8
- **Record pattern** — A pattern that deconstructs a record: `case Point(int x, int y)` (Java 21). §3.7
- **Recurrence** — A formula that defines each answer from earlier ones, such as J(n) = (J(n − 1) + k) mod n. §19.19, §19.27
- **Recursion** — A method calling itself; needs a base case to stop. §4.5
- **Red-black tree** — A self-balancing binary search tree that keeps every operation O(log n); `TreeMap` and `TreeSet` use one. §10.4
- **Reduction** — Combining stream elements into one result with `reduce` or a collector. §12.4
- **Reentrant** — A lock its holder can acquire again without blocking itself, as `synchronized` and `ReentrantLock` allow. §14.6
- **`ReentrantLock`** — An explicit reentrant lock with `tryLock`, timeouts, fairness and conditions. §14.9
- **Refactoring** — Restructuring code to make it clearer without changing what it does; tests make it safe. §18.4, §18.9
- **Reference type** — Any non-primitive type; variables hold references to objects (or `null`). §2.5
- **Reflection** — Inspecting and using classes, methods and fields at run time (`getClass()`, `java.lang.reflect`). §16.2
- **Region (G1)** — One of the equal-sized blocks G1 divides the heap into; each region is eden, survivor, old or humongous at any time. §16.5
- **Regular expression** — A pattern language for matching text, used by `split`, `replaceAll`, `matches`, `Pattern`. §7.2
- **Reifiable type** — A type fully known at run time (`String`, `List<?>`, `int[]`); `List<String>` is not, because of erasure. §9.8
- **Rejection policy** — What a `ThreadPoolExecutor` does with a task when its queue is full and no thread is free — throw, run it in the caller, or drop it. §14.11
- **Relative / absolute path** — An absolute path starts at the file-system root; a relative path is resolved against the current directory. §13.2
- **REPL** — Read-eval-print loop: an interactive prompt such as `jshell`. §1.9

- **Resident set size (RSS)** — The physical memory a process actually occupies, as reported by the operating system. §16.10, §16.13
- **Resource leak** — Failing to close a file, socket or connection; prevented by try-with-resources. §8.6, §13.13
- **REST API** — A web API in which URLs name resources and HTTP methods (GET, POST …) act on them, usually exchanging JSON. §18.14
- **Reverse Polish Notation** — Postfix notation in which each operator follows its operands (`3 4 +`), evaluated with a stack. §19.12
- **RMI** — Remote Method Invocation: an old Java API for calling methods in another JVM, built on serialization. §13.10
- `Runnable` — A task interface with `void run()` and no result. §14.2
- `RuntimeException` — The superclass of unchecked exceptions. §8.2

S

- **Safepoint** — A point at which all Java threads can be paused safely, for example for parts of a garbage collection. §16.5
- **Scalar replacement** — Replacing a non-escaping object's fields with local variables so it's never allocated. §16.8
- **Scale (`BigDecimal`)** — The number of digits after the decimal point; `2.0` and `2.00` have different scales. §7.7
- `Scanner` — A `java.util` class that reads and parses console or file input token by token. §2.13
- `ScheduledExecutorService` — An executor that runs tasks after a delay or repeatedly at a fixed rate. §14.11
- **Scheduler** — The part of the OS (or, for virtual threads, of the JVM) that decides which thread runs on which core next. §15.4, §16.13
- **Scoped value** — An immutable value bound for the duration of a call and readable by code it invokes (final in Java 25). §15.6
- **SDKMAN!** — A command-line tool for installing and switching between JDK versions. §1.4
- **Sealed class** — A class or interface (Java 17) that lists its permitted subclasses with `permits`. §6.7
- `SecureRandom` — A random-number generator strong enough for passwords, keys and tokens. §7.12
- **Seed** — The starting value of a pseudo-random generator; the same seed reproduces the same sequence. §7.12
- **Selector** — The value a `switch` examines. §3.2
- `Semaphore` — A synchronizer handing out a fixed number of permits to limit concurrent access. §14.13
- **Sentinel** — A special marker value (such as −1 for "not seen") or dummy element that removes a special case from the code. §19.11, §19.22
- **Sequenced collection** — Java 21 interfaces (`SequencedCollection`, `SequencedSet`, `SequencedMap`) with `getFirst`, `getLast`, `reversed`. §10.11
- **Serial GC** — A single-threaded, stop-the-world collector for small heaps. §16.5
- **Serialization** — Converting objects to bytes with `ObjectOutputStream`; risky with untrusted data. §13.10
- `serialVersionUID` — The version number checked when deserialising a `Serializable` class. §13.10
- `Set` — A collection with no duplicate elements. §10.3
- **Shadowing** — A local variable or parameter hiding a field with the same name; use `this.name` to reach the field. §5.2
- **Shallow immutability** — Fields can't be reassigned, but the objects they refer to may still be mutable (true of records). §5.9, §6.8
- **Shebang** — A first line such as `#!/usr/bin/java --source 25` that lets a Java source file run as a Unix script. §1.10
- **Shenandoah** — A low-pause OpenJDK collector that compacts concurrently. §16.5
- **Short-circuit evaluation** — `&&` and `\|\|` skip the right operand when the left decides the result; also short-circuiting stream operations. §2.8, §12.4

- **Side effect** — Anything a function does besides returning a value: changing a field, printing, writing a file. §11.8
- **Sieve of Eratosthenes** — Finding every prime up to n by crossing out the multiples of each prime in turn. §19.27
- **SIGKILL** — The Unix signal that ends a process at once, with no chance to clean up (sent, for example, by the OOM killer). §16.13
- **SIMD** — Single instruction, multiple data: one CPU instruction that works on several numbers at once. §16.8
- **Single-file source launch** — Running `java Hello.java` without a separate compile step (Java 11). §1.10
- **Singleton** — A class with exactly one instance, often written as an `enum` with one constant. §5.3
- **SLF4J** — Simple Logging Facade for Java: a logging API with pluggable backends such as Logback. §18.10
- **Sliding window** — Keeping a running summary of a contiguous range while moving its ends forward, so subarray problems take O(n). §19.9
- **SNAPSHOT version** — A Maven version ending in `-SNAPSHOT`, meaning "work in progress — may change". §18.1
- **SOLID** — Five object-oriented design principles: single responsibility, open/closed, Liskov substitution, interface segregation, dependency inversion. §20.18
- **Space complexity** — How much extra memory an algorithm needs as its input grows, in Big-O terms. §19.7
- `Spliterator` — The splittable iterator behind every stream, which divides the work for parallel streams. §12.2
- **Spring Boot** — The most widely used Java framework for back-end services, with auto-configuration and starters. §18.14
- **Spy** — A test double that wraps a real object and records calls, letting you stub only some methods. §18.6
- **SQL** — Structured Query Language: the language for querying and changing relational databases. §18.11
- **SQL injection** — An attack in which user input is pasted into SQL text and runs as commands; `PreparedStatement` parameters prevent it. §18.11
- **Stable sort** — A sort that keeps equal elements in their original relative order. §10.8, §19.5
- **Stack (data structure)** — A LIFO collection: `push` adds on top and `pop` removes from the top; use `ArrayDeque`, not `Stack`. §10.6, §19.4
- **Stack (JVM)** — Per-thread memory of frames for active method calls, holding locals and partial results. §16.3
- **Stack trace** — The list of method frames active when an exception was created, newest first. §8.9
- **Stack unwinding** — An exception travelling up the call stack, abandoning each frame until a matching `catch` is found. §8.1
- `StackOverflowError` — Thrown when a thread's stack is exhausted, usually by runaway recursion. §4.5, §16.3
- `StampedLock` — A lock with an optimistic-read mode for read-mostly data; not reentrant. §14.9
- **Standard streams** — `System.in`, `System.out` and `System.err`: the input, output and error channels every program starts with. §13.8
- **Starvation** — A thread never gets the CPU or lock it needs because others always win. §15.9
- **Stateless / stateful operation** — A stateless stream step (`map`, `filter`) looks at one element at a time; a stateful one (`sorted`, `distinct`) must remember others. §12.3
- `static` — Belonging to the class rather than to an instance; one copy shared by all. §5.5
- **Static nested class** — A nested class declared `static`; it has no enclosing instance. §6.10
- **Stop-the-world pause** — A GC phase during which all application threads are paused. §16.5
- **Stream** — A lazy, one-shot pipeline of operations over a source of elements (`java.util.stream`). §12.1
- **Stream gatherers** — See *gatherer*. §12.9
- `String` — An immutable sequence of UTF-16 characters. §2.12, §7.1
- **String pool** — The JVM table of shared `String` objects for literals and interned strings. §7.1

- **String templates** — A preview feature in Java 21 and 22 that was withdrawn in Java 23. §17.8
- `StringBuilder` — A mutable, unsynchronised character buffer for building strings efficiently. §7.3
- **Strong encapsulation** — Since Java 17 JDK internals are inaccessible by default; `--add-opens` is required to reach them. §17.3
- **Structured concurrency** — Treating a group of concurrent subtasks as one unit (`StructuredTaskScope`); still preview in Java 25. §15.7
- **Structured logging** — Logging events as named fields (often JSON) that machines can search, rather than only free text. §18.10
- **Stub** — A test double that returns canned answers. §18.6
- **STW** — Short for *stop-the-world pause*. §16.5
- **Subarray / subsequence** — A subarray (or substring) is a contiguous slice; a subsequence keeps the original order but may skip elements. §19.9, §19.19
- `super` — Refers to the parent class: `super.method()` calls the parent's version, `super(...)` its constructor. §6.1
- **Suppressed exception** — An exception from `close()` attached to the primary exception in try-with-resources. §8.6
- **Surefire** — The Maven plugin that runs unit tests (its sibling Failsafe runs integration tests). §18.2
- **Surrogate pair** — Two `char`s encoding one Unicode character outside the Basic Multilingual Plane (many emoji). §7.2
- **Survivor space** — Young-generation areas to which objects that survived a collection are copied before promotion to the old generation. §16.5
- `switch` **expression** — A `switch` that yields a value (Java 14), using arrow labels or `yield`. §3.3
- `synchronized` — Keyword that acquires an object's monitor for a method or block, giving mutual exclusion and visibility. §14.6
- **Synchronizer** — A coordination object such as a latch, barrier, semaphore or phaser. §14.13
- **System call** — A request from a program to the operating-system kernel, such as reading a file. §16.13

T

- **Tabulation** — Bottom-up dynamic programming: filling a table in an order where every entry's dependencies are already computed. §19.19
- **Template Method pattern** — A base class fixes the steps of an algorithm and lets subclasses fill in some of them. §6.5
- `TemporalAdjusters` — Ready-made date adjustments such as "last day of the month" or "next Monday". §7.9
- **Terminal operation** — The stream operation that produces a result and triggers execution (`collect`, `count`, `forEach`). §12.4
- **Ternary operator** — `cond ? a : b` — the conditional expression. §2.8
- **Test double** — Any stand-in for a real collaborator in a test: dummy, stub, fake, spy or mock. §18.6
- **Testcontainers** — A library that starts real services (databases, message brokers) in throwaway Docker containers for tests. §18.9
- **Test-driven development (TDD)** — Writing a failing test first, then the code to make it pass, then tidying up. §18.9
- **Text block** — A multi-line string literal delimited by `"""` (Java 15). §2.12
- `this` — A reference to the current object; `this(...)` calls another constructor. §5.2, §5.3
- **Thread** — An independent path of execution within a process, sharing the heap with other threads. §14.1
- **Thread confinement** — Keeping data reachable by only one thread, so it needs no locking. §14.14
- **Thread dump** — A snapshot of every thread's state and stack; shows deadlocks. §16.10
- **Thread pool** — A set of reusable worker threads that execute submitted tasks. §14.11
- **Thread priority** — A hint from 1 to 10 for the scheduler; the operating system may ignore it. §15.9
- `ThreadLocal` — A variable with an independent value per thread. §15.6

- **Thread-per-request** — A server model that gives every request its own thread; affordable at scale with virtual threads. §15.4
- `ThreadPoolExecutor` — The configurable thread-pool class behind most `Executors` factory methods. §14.11
- **Thread-safe** — Behaving correctly when used by several threads at once without extra coordination by callers. §14.1, §14.14
- **Throughput** — Work completed per unit of time — requests per second, or for a GC the share of time not spent collecting. §16.5, System Design Fundamentals
- `throw` / `throws` — `throw` raises an exception; `throws` in a method signature declares checked exceptions it may raise. §8.3, §8.7
- `Throwable` — The root class of everything that can be thrown: `Error` and `Exception`. §8.2
- **Tiered compilation** — Running code in the interpreter, then C1, then C2 as it gets hotter. §1.7
- **Time slice** — The short period a thread runs before the scheduler may switch the core to another thread. §16.13
- **Time zone** — A region's rules for its offset from UTC, including daylight saving, such as `Europe/Paris`. §7.10
- **TimSort** — The stable merge sort used by `Arrays.sort` and `List.sort` for objects; very fast on partly sorted data. §19.5
- **TLAB** — Thread-local allocation buffer: a private chunk of eden from which one thread allocates without locking. §16.5
- **TLB** — Translation lookaside buffer: the CPU's cache of virtual-to-physical address translations. §16.13
- **TLS** — Transport Layer Security: the encryption behind HTTPS. §17.10
- **Toolchain** — A build-tool setting that picks the exact JDK used to compile and test, independent of the JDK running the build. §18.3
- **Topological sort** — Ordering a DAG's nodes so every edge points forward — prerequisites before the tasks that need them. §19.17
- `toString` — Returns a text representation; `Object`'s default is `ClassName@hash`. §6.13
- **Transaction** — A group of database changes that succeed or fail together (`commit` or `rollback`). §18.11
- **Transitive dependency** — A library you get indirectly because one of your dependencies needs it. §18.1, §18.2
- **Treeification** — `HashMap` turning a bucket of more than 8 entries into a small red-black tree (Java 8+, once the table has ≥ 64 buckets). §10.5
- `TreeMap` / `TreeSet` — Red-black-tree map/set kept in sorted order, O(log n), with navigation methods such as `floorKey`. §10.4, §10.3
- **Trie** — A prefix tree that stores strings character by character, so prefix lookups cost O(length of the prefix). §19.21
- **try-with-resources** — `try (Resource r = …)`, which closes every declared `AutoCloseable` automatically, in reverse order. §8.6
- **Two pointers** — Two indexes moving through data in a coordinated way, so one pass replaces a nested loop. §19.8
- **Two's complement** — How Java stores signed integers: the top bit is the sign and −x equals `~x + 1`. §2.10, §19.26
- **Type erasure** — Compiling generics by checking types and then removing type arguments, inserting casts. §9.7
- **Type inference** — The compiler deducing types (`var`, diamond, generic method arguments, lambda parameters). §2.6, §9.4
- **Type parameter** — A placeholder type such as `T` in `class Box<T>`. §9.2
- **Type pattern** — A pattern that tests a type and binds a variable: `instanceof String s`, `case Integer i`. §3.5
- **Type witness** — An explicit type argument written before a generic method's name, e.g. `Comparator.<int[]>comparingInt(...)`. §9.4, §19.5

U

- **Unboxing** — Converting a wrapper object back to its primitive; unboxing `null` throws `NullPointerException`. §7.6
- **Unchecked exception** — `RuntimeException`, `Error` and their subclasses — not checked by the compiler. §8.2
- **Unchecked warning** — A compiler warning that a generic operation can't be type-checked (raw types, unchecked casts). §9.8
- **`UncheckedIOException`** — An unchecked wrapper for `IOException`, used inside lambdas and streams. §13.4
- **Unicode** — The universal character set; Java strings use the UTF-16 encoding of it. §7.2
- **Union-find (disjoint set union)** — A structure that tracks which elements belong to the same group and merges groups in nearly O(1) per operation. §19.18
- **Unit test** — A fast test of one small piece of code — a method or a class — in isolation. §18.4, §18.9
- **Unmodifiable collection** — A collection whose mutators throw `UnsupportedOperationException`, such as `List.of`. §10.10
- **Unnamed module** — The module that holds everything loaded from the class path. §16.12
- **Unnamed variable `_`** — An underscore for a variable or pattern you don't use (Java 22). §3.7, §17.7
- **`UnsupportedOperationException`** — Thrown by an operation a collection or object doesn't support. §10.10
- **Upcasting** — See *downcasting / upcasting*. §6.4
- **UTC** — Coordinated Universal Time: the world reference time, with no time-zone or daylight-saving shifts. §7.10
- **UTF-16** — The encoding Java strings use: each character is one or two 16-bit `char`s. §2.3, §7.2
- **UTF-8** — A variable-length encoding using 1–4 bytes per character; the Java default charset since 18. §13.7

V

- **Value object** — A small immutable object defined only by its values (an amount of money, a date range); records fit well. §6.8, §18.6
- **`var`** — Local variable type inference (Java 10). §2.6
- **Varargs** — A parameter `T... values` accepting zero or more arguments as an array. §4.4
- **Vector API** — An incubating API for SIMD calculations — one instruction applied to many numbers at once. §17.11
- **Virtual memory** — Each process sees its own large address space, which the OS maps onto physical RAM page by page. §16.13
- **Virtual thread** — A lightweight thread (Java 21) scheduled by the JVM onto carrier threads; cheap enough to create one per task. §15.4
- **Visibility (memory)** — Whether one thread's writes are guaranteed to be seen by another. §14.7, §15.8
- **VisualVM / Eclipse MAT / async-profiler** — Free diagnostic tools: VisualVM monitors a running JVM, Eclipse MAT analyses heap dumps, async-profiler samples CPU and allocations into flame graphs. §16.10
- **`volatile`** — A field modifier guaranteeing visibility and ordering of reads and writes, but not atomic compound actions. §14.7

W

- **`wait` / `notify`** — `Object` methods for guarded blocks; must be called holding the object's monitor. §14.8
- **Wait set** — The threads waiting in `wait()` on an object's monitor until they are notified. §14.8
- **Warm-up** — The start-up period while the JVM loads classes and the JIT compiles hot code; timings taken then are misleadingly slow. §16.9
- **`WatchService`** — The NIO.2 API that reports files being created, changed or deleted in a directory. §13.9
- **Weakly consistent iterator** — A concurrent-collection iterator that never throws CME and may or may not show concurrent updates. §14.12

- **Week-based year** — `YYYY` in a date pattern: the year a week belongs to, which differs from the calendar year `yyyy` around New Year. §7.11
- **Widening conversion** — Automatic conversion to a type with a larger range (`int` → `long` → `double`). §2.7
- **Wildcard** — `?` in a generic type: `List<?>`, `List<? extends T>`, `List<? super T>`. §9.6
- **Work stealing** — Idle threads taking tasks from the back of busy threads' queues, as `ForkJoinPool` does. §15.3
- **Wrapper class** — The object form of a primitive: `Integer`, `Long`, `Double`, `Character`, `Boolean`, … §7.6

X

- **XML** — Extensible Markup Language: a tag-based text format, used for example by Maven's `pom.xml`. §18.2

Y

- `yield` — Returns a value from a block inside a `switch` expression. §3.3
- **Young generation** — The heap area (eden plus survivor spaces) where new objects are allocated; collected often and cheaply. §16.5

Z

- **Zero-copy** — Moving data between files and sockets without copying it through the Java heap (e.g. `transferTo`). §13.9
- **ZGC** — A scalable low-latency collector with sub-millisecond pauses; generational-only since Java 24. §16.5
- `ZonedDateTime` — A date-time with a time-zone region, aware of daylight-saving rules. §7.10

Chapter TL;DR

- **Types**: eight primitives with fixed sizes (`int` ±2.1 × 10^9, `long` ±9.2 × 10^{18}, `char` 0–65 535); integer overflow wraps silently; `0.1 + 0.2 != 0.3`; money in `BigDecimal` built from strings.
- **Equality**: `==` compares references for objects; use `equals` — the `Integer` cache (−128…127) and the string pool only make `==` *look* right.
- **Modern syntax**: switch expressions, pattern matching with guards, record patterns and `_`, sealed types for exhaustive switches, records for data, `var` for locals, text blocks.
- **Collections**: `ArrayList` and `HashMap` by default, `ArrayDeque` for stacks and queues, `TreeMap` for order and ranges, `LinkedHashMap` for LRU; `List.of` is unmodifiable and rejects `null`.
- **Functional and streams**: `Supplier`/`Consumer`/`Predicate`/`Function`; streams are lazy and one-shot; `groupingBy`, `partitioningBy`, `toMap` (duplicates throw); gatherers in 24; `Optional` only as a return type.
- **Exceptions**: checked = recoverable and declared, unchecked = bugs; try-with-resources closes in reverse and keeps close failures as suppressed; never `return` from `finally`.
- **Concurrency**: `synchronized`/locks for mutual exclusion, `volatile` for visibility only, atomics and `ConcurrentHashMap` for counters and caches, `CompletableFuture` for composition, virtual threads for blocking I/O at scale; happens-before decides visibility.
- **JVM**: heap, stacks, metaspace, code cache; G1 by default, ZGC for low latency; `-Xmx`, `-Xlog:gc*`, heap dumps on OOM; `jcmd`, JFR, `jstack` to diagnose.
- **Versions**: 17 sealed · 21 virtual threads, pattern `switch`, record patterns, sequenced collections · 22 `_`, FFM · 24 gatherers · 25 compact source files, module imports, flexible constructors, scoped values. Structured concurrency is still preview.
- **Tools**: `mvn clean verify` / `./gradlew build`, JUnit + Mockito + AssertJ, SLF4J for logging, `PreparedStatement` for SQL.
- **Next**: Spring Boot (or Jakarta EE, Quarkus), Kotlin if you like, and the LLD, SQL and DDIA notes — then build something real.

Chapter 21 — Interview Question Bank

How to use this bank

This chapter collects the Java questions that come up again and again in real interviews — gathered from big public question lists (GitHub collections, LeetCode Discuss interview-experience posts, Java 8 stream-coding lists) — and worded the way interviewers usually say them.

- ⋆ marks a question that appears in many sources: expect it.
- Each answer is short enough to **say out loud** in under a minute. Where this book teaches the topic in full, a → **More** link takes you there. Chapter 19 §19.29 already answers 42 core questions in more depth; those entries here are deliberately brief and link to it.
- Questions the rest of the book does not cover still get a complete short answer here, so nothing is missing.
- Every "what does this print" output and every coding answer was run on a real JDK; the printed results are real.
- **How to drill:** cover the answer, say yours out loud, then uncover and compare. Mark the ones you missed and repeat them the next day.

Versions: the book targets **Java 25 LTS** (LTS = long-term support release). Features are tagged with the Java version that made them final.

1. Platform and language basics

Q1. ⋆ What is the difference between JDK, JRE and JVM? The **JVM** (Java Virtual Machine) runs bytecode. The **JRE** (Java Runtime Environment) was JVM + standard libraries, enough to run programs. The **JDK** (Java Development Kit) adds the compiler `javac` and tools (`jshell`, `jcmd`, `jlink`…). Since Java 11 Oracle no longer ships a separate JRE (some vendors, such as Eclipse Temurin, still offer JRE builds); you can build a trimmed runtime for your app with `jlink`. → *More:* §1.3 JDK, JRE and JVM

Q2. Why is Java platform independent? Is the JVM platform independent? `javac` compiles source to **bytecode** (`.class` files), a portable instruction set; any JVM on any OS runs the same bytecode ("write once, run anywhere"). The JVM itself is *not* platform independent — each OS/CPU pair has its own JVM build. → *More:* §1.6 Bytecode and class files

Q3. Is Java a pure object-oriented language? No. It has eight **primitive** types (`int`, `double`…) that are not objects, and `static` members that belong to no object. Everything else is an object, and wrappers (`Integer`) let primitives act as objects.

Q4. ⋆ What are the primitive types and their sizes? `byte` 8 bits, `short` 16, `char` 16 (unsigned UTF-16 code unit), `int` 32, `long` 64, `float` 32, `double` 64, `boolean` (true/false, size not specified). Integer types are signed two's complement. Default field values are `0`, `0.0`, `'\u0000'`, `false`; local variables have no default and must be assigned before use. → *More:* §2.3 Primitive types

Q5. Primitive vs reference types? A primitive variable holds the value itself; a reference variable holds a pointer to an object on the heap (or `null`). Copying a reference copies the pointer, so both variables see the same object. Primitives can't be `null` and can't be used as generic type arguments. → *More:* §2.5 Reference types and null

Q6. ⋆ What are wrapper classes and autoboxing? What can go wrong? Each primitive has a wrapper object (`Integer`, `Long`, `Character`…). **Autoboxing** converts `int → Integer` automatically; **unboxing** does the reverse. Pitfalls: unboxing `null` throws `NullPointerException`; `==` on wrappers compares references (works by luck inside the −128…127 cache); boxing in hot loops allocates objects. → *More:* §7.6 Wrapper classes and autoboxing

Q7. ⋆ Is Java pass-by-value or pass-by-reference? Always pass-by-value; for objects the value copied is the reference, so a method can mutate the caller's object but not make the caller's variable point elsewhere. → *More:* §4.2 · §19.29 Q6

Q8. Widening vs narrowing conversion; what is casting? Widening (`int → long → double`) is automatic and safe (except precision loss for big `long → float/double`). **Narrowing** (`double → int`, `int → byte`) needs an

explicit **cast** `(int) 3.99` and may truncate or overflow: `(byte) 200` is `-56`. In expressions, `byte`/`short`/`char` are promoted to `int`, which is why `short s = 1; s = s + 1;` doesn't compile but `s += 1` does (compound assignment casts implicitly). → *More:* §2.7 Type conversion

Q9. What does `var` do, and where can't you use it? `var` (Java 10) lets the compiler infer a **local** variable's type from its initializer; the variable is still statically typed. Not allowed for fields, method parameters, return types, without an initializer, or with a bare `null` initializer. Use it when the type is obvious from the right-hand side. → *More:* §2.6 var

Q10. ⋆ What does `static` mean? Can a static method use instance members? A static member belongs to the class, one copy shared by all instances. A static method has no `this`, so it can't use instance fields or methods directly (it needs an object reference). Static blocks run once when the class is initialised. → *More:* §5.5 Static members · §19.29 Q41

Q11. ⋆ Explain `final` vs `finally` vs `finalize`. `final` = can't reassign / override / extend; `finally` = block that always runs after `try`; `finalize()` = deprecated-for-removal GC callback (JEP 421, Java 18) — use try-with-resources or `Cleaner`. → *More:* §5.8 final · §19.29 Q7

Q12. What are the access modifiers? `private` (same class only), *package-private* (no keyword: same package), `protected` (same package plus subclasses in other packages), `public` (everyone). Top-level classes can only be `public` or package-private. Since Java 9, modules can additionally hide whole packages. → *More:* §5.6 Access modifiers

Q13. Why is `main` `public static void main(String[] args)`? What changed in Java 25? The launcher must call it without creating an object (`static`), from outside the package (`public`), and expects no result (`void`). Java 25 (JEP 512) allows **instance main methods** and **compact source files**: a file containing just `void main() { IO.println("Hi"); }` runs with `java Hello.java`. → *More:* §4.9 main and simpler main

Q14. What is the difference between `this` and `super`? `this` refers to the current object (disambiguate fields, call another constructor with `this(...)`, pass yourself). `super` reaches the parent's version of a member (`super.toString()`) or calls a parent constructor with `super(...)`. Before Java 25, `this(...)`/`super(...)` had to be the first statement; Java 25's flexible constructor bodies (JEP 513) allow statements before it (validation, computing arguments) as long as they don't read fields or call instance methods; the only use of `this` allowed there is assigning the class's own fields that have no initializer. → *More:* §6.1 Inheritance and super

Q15. What happens on integer overflow? How do you detect it? It silently wraps around: `Integer.MAX_VALUE + 1 == Integer.MIN_VALUE`. Use `Math.addExact`/`multiplyExact` (throw `ArithmeticException`), a wider type (`long`), or `BigInteger`. → *More:* §2.10 Integer overflow

Q16. Why shouldn't you use `double` for money? What do you use instead? Binary floating point can't represent most decimal fractions exactly (`0.1 + 0.2` prints `0.30000000000000004`). Use `BigDecimal` created from a **string** or `BigDecimal.valueOf(double)` (not `new BigDecimal(0.1)`, which captures the binary error), with an explicit `RoundingMode`, or store amounts as `long` cents. → *More:* §7.7 BigInteger and BigDecimal

2. OOP: classes, inheritance and polymorphism

Q17. ⋆ What are the four principles of OOP? Encapsulation — hide state behind methods so invariants hold. **Abstraction** — expose *what* an object does, not *how* (interfaces, abstract classes). **Inheritance** — a subclass reuses and specialises a parent. **Polymorphism** — one call, different behaviour depending on the object's run-time class. → *More:* §5.7 Encapsulation · LLD Handbook

Q18. ⋆ Abstraction vs encapsulation? Abstraction is a *design* idea: model only the essential behaviour and hide complexity behind a simple contract. Encapsulation is the *mechanism*: private fields, controlled access through methods. Abstraction decides what to show; encapsulation enforces what to hide.

Q19. ⋆ What is polymorphism? Compile-time vs run-time? Compile-time (static) polymorphism is **overloading**, resolved by the compiler from argument types. Run-time (dynamic) polymorphism is **overriding**: `Animal a = new Dog(); a.speak()` runs `Dog.speak()` because the JVM dispatches on the actual object's class. → *More:* §6.4 Polymorphism and casting

Q20. ⋆ Overloading vs overriding? Overloading: same name, different parameters, chosen at compile time. Overriding: same signature in a subclass, chosen at run time; can't reduce visibility, can't throw

broader checked exceptions, may return a subtype (**covariant return**). → *More:* §6.3 Overriding · §19.29 Q12

Q21. ⋆ Can you override a static method? A private method? Can you overload `main`? Static: no — a same-signature static method in a subclass **hides** the parent's; which runs depends on the reference's declared type. Private: no — it isn't inherited, so a same-named subclass method is unrelated. `main`: yes, you can overload it, but the launcher only calls the standard signature. → *More:* §6.2 Members, hiding and Object

Q22. ⋆ Why doesn't Java support multiple inheritance of classes? How do you get it? Inheriting state and implementations from two parents creates the "diamond problem" (which parent's field or method wins?). Java allows one superclass but many interfaces. If two interfaces provide the same `default` method, the class must override it and may pick one with `A.super.method()`. → *More:* §6.6 Interfaces

Q23. ⋆ Interface vs abstract class (and what changed in Java 8/9)? Interfaces: multiple implementation, no instance state, `default`/`static` methods (8) and `private` methods (9). Abstract class: single inheritance, fields, constructors. Interface for a capability, abstract class for shared state among related classes. → *More:* §6.5 · §19.29 Q9

Q24. Can an abstract class have a constructor? Can it have no abstract methods? Yes to both. The constructor runs when a subclass is created (via `super(...)`), initialising shared state. A class can be `abstract` with zero abstract methods simply to forbid direct instantiation.

Q25. ⋆ Composition vs inheritance — which do you prefer? Inheritance ("is-a") couples the subclass to the parent's implementation details; a change in the parent can break it (the "fragile base class" problem). Composition ("has-a") holds another object in a field and delegates to it, so you can swap implementations and expose only what you choose. Prefer composition; use inheritance only for a true is-a relation designed for extension. → *More:* LLD Handbook

Q26. Association vs aggregation vs composition? **Association**: objects know each other. **Aggregation**: a "has-a" where the part can live without the whole (a team has players). **Composition**: the part's lifetime is owned by the whole (an order owns its order lines).

Q27. What is a constructor? Default constructor? Constructor chaining? A constructor initialises a new object; it has the class name and no return type. If you write none, the compiler adds a no-arg **default constructor**; if you write any, it doesn't. Chaining: `this(...)` calls another constructor of the same class, `super(...)` a parent one; if neither is written, the compiler inserts `super()`, so a parent without a no-arg constructor causes a compile error. → *More:* §5.3 Constructors

Q28. What is a private constructor used for? To stop outside instantiation: utility classes with only static methods, singletons, and classes that hand out instances through static factory methods (`List.of`, `Integer.valueOf`).

Q29. What is the initialisation order of a class? Once per class: static field initialisers and static blocks, top to bottom (parent class first). Per object: parent constructor chain first, then the child's instance field initialisers and instance blocks in order, then the rest of the child constructor body. So a parent constructor that calls an overridden method sees the child's fields still at default values (see Q218). → *More:* §5.10 Initialisation order

Q30. What methods does `Object` have? `equals`, `hashCode`, `toString`, `getClass`, `clone` (protected), `wait`/`notify`/`notifyAll` (thread coordination on the object's monitor), and the deprecated `finalize`. Every class inherits them. → *More:* §6.13 equals, hashCode and toString

Q31. What is an enum, and why is it better than int constants? An `enum` is a class with a fixed set of instances: type-safe (you can't pass 7 where a `Day` is expected), can have fields, constructors, methods and per-constant behaviour, works in `switch`, and comes with `values()`, `valueOf()`, `EnumMap`, `EnumSet`. Enum constants are singletons, safe even across serialization. → *More:* §6.9 Enums

Q32. Static nested vs inner class vs anonymous class? A **static nested** class has no link to an outer instance (prefer it). An **inner** (non-static) class holds a hidden reference to its outer object — it can use the outer fields but can leak memory if it outlives the outer. An **anonymous** class is an unnamed one-off implementation, mostly replaced by lambdas for single-method interfaces. → *More:* §6.10 Nested classes

Q33. What is a marker interface? Give examples. An interface with no methods that tags a class for special treatment: `Serializable`, `Cloneable`, `RandomAccess`. Today annotations usually do this job, but a

marker interface is a real type, so the compiler can check it.

Q34. What is a covariant return type? An overriding method may declare a *subtype* of the parent's return type, e.g. `Animal reproduce()` overridden as `Dog reproduce()`, which saves callers a cast.

Q35. What is upcasting and downcasting? When do you get `ClassCastException`? Upcasting (`Animal a = dog`) is implicit and always safe. Downcasting (`(Dog) a`) is explicit and checked at run time; if the object isn't really a `Dog` it throws `ClassCastException`. Guard with pattern matching: `if (a instanceof Dog d) d.bark();`. → *More:* §3.5 Pattern matching for instanceof

Q36. What are coupling and cohesion? Coupling is how much one class depends on another's details — aim low. **Cohesion** is how focused a class is on one responsibility — aim high. Low coupling plus high cohesion makes code easier to change and test.

Q37. ⋆ What are the SOLID principles? Single responsibility (one reason to change), **O**pen/closed (extend without editing), **L**iskov substitution (subtypes must work wherever the parent is expected), **I**nterface segregation (small focused interfaces), **D**ependency inversion (depend on abstractions, inject concrete classes). → *More:* LLD Handbook

Q38. ⋆ How do you write a thread-safe singleton? Best: an `enum` with one constant (serialization- and reflection-safe), or the **holder idiom** (a nested static class holds the instance; the JVM's class initialisation makes it lazy and thread-safe). Double-checked locking also works but the field **must** be `volatile`. Code in Q248.

3. Strings

Q39. ⋆ Why is `String` immutable? Safe sharing between threads, safe as a hash key (hash cached), poolable, and secure for values that are checked then used (paths, class names). → *More:* §7.1 · §19.29 Q1

Q40. ⋆ What is the String pool? How many objects does `new String("hi")` create? A JVM table (on the heap since Java 7) with one shared instance per literal; `new String("hi")` makes one new object plus the pooled literal if not already there — "one or two". `intern()` returns the pooled copy. → *More:* §7.1 · §19.29 Q2

Q41. ⋆ `String` vs `StringBuilder` vs `StringBuffer`? Immutable vs mutable-unsynchronised vs mutable-synchronised (legacy). Build strings in loops with `StringBuilder`. → *More:* §7.3 · §19.29 Q3

Q42. ⋆ `==` vs `equals()` for strings? `==` asks "same object?", `equals` asks "same characters?". Literals happen to share a pooled object, so `==` sometimes "works" — a classic trap; always use `equals` (or `"x".equals(var)` to be null-safe). See Q210. → *More:* §7.1 Strings and the pool

Q43. Why is `+=` on a `String` in a loop slow? Each `+=` copies the whole string into a new object, so n appends cost $O(n^2)$ character copies. Outside loops, `+` is fine: since Java 9 the compiler emits an `invokedynamic` call to `StringConcatFactory`, which builds the result efficiently. → *More:* §7.3 StringBuilder

Q44. How is a `String` stored internally? Since Java 9 (**compact strings**, JEP 254), as a `byte[]` plus a coder flag: Latin-1 strings use one byte per character, others UTF-16 (two bytes). The hash code is computed lazily and cached in a field. → *More:* §7.1 Strings and the pool

Q45. Why prefer `char[]` over `String` for passwords? A `String` can't be wiped and may linger in memory (and in the pool) until GC, where heap dumps can expose it; a `char[]` can be overwritten with `Arrays.fill(pw, '\0')` after use. That's why `Console.readPassword()` returns `char[]`.

Q46. `trim()` vs `strip()`; what else did Java 11 add? `trim()` removes characters ≤ U+0020 (ASCII spaces and control chars); `strip()` (11) removes all Unicode whitespace. Java 11 also added `isBlank()`, `lines()`, `repeat(n)`, `stripLeading/Trailing()`. → *More:* §7.2 The String API

Q47. What does `intern()` do, and when would you use it? Returns the canonical pooled instance with the same content, adding it to the pool if absent. Rarely needed: it can save memory when you hold millions of duplicate strings, but `-XX:+UseStringDeduplication` (G1 since Java 8u20, the other collectors since Java 18) usually does that for you.

Q48. What are text blocks? Multi-line string literals (Java 15) delimited by `"""`; incidental indentation is stripped, so JSON/SQL/HTML can be pasted readably. They are ordinary `String`s. → *More:*

§2.12 Strings and text blocks

Q49. How do you split, join and format strings? `"a,b".split(",")` (regex; trailing empty strings dropped), `String.join(",", list)`, `Collectors.joining(", ", "[", "]")`, `StringJoiner`, and `"%s=%d".formatted(k, v)` (Java 15) or `String.format`. → *More:* §7.4 Formatting

Q50. Is `String` thread-safe? Is `StringBuilder`? `String` yes (immutable). `StringBuilder` no — don't share one between threads; give each thread its own, or use `StringBuffer` in the rare case one buffer really is shared.

4. equals, hashCode, immutability and cloning

Q51. ⋆ What is the `equals`/`hashCode` contract? What breaks if you override only `equals`? Equal objects must have equal hash codes; override only `equals` and `HashSet`/`HashMap` stop finding "equal" objects because they hash to different buckets (Q224 shows it). → *More:* §6.13 · §19.29 Q4

Q52. How do you write a correct `equals`? Check identity (`this == o`), then type (`o instanceof Point p`, or `getClass() == o.getClass()` if subclasses must never equal the parent), then compare significant fields (`Objects.equals` for references, `==` for primitives except `Double.compare`/`Float.compare`). Make the class final or use `getClass()` to stay symmetric with subclasses. Build `hashCode` from the same fields: `Objects.hash(x, y)`. Records generate both. → *More:* §6.13 equals and hashCode

Q53. Can two unequal objects have the same hash code? Yes — that's a **collision** and is legal (`"Aa"` and `"BB"` both hash to 2112). Collisions only cost performance; the map falls back to `equals`. → *More:* §6.13 equals and hashCode

Q54. ⋆ How do you make a class immutable? `final` class, `private final` fields, no setters, defensive copies in and out (`List.copyOf`), no `this` escaping. Records cover most of it, shallowly. → *More:* §5.9 Immutability basics · §19.29 Q8

Q55. What are the benefits of immutable objects? Thread-safe without locks, safe as map keys and set elements, can be cached and shared freely, easier to reason about (no state changes after construction), and failure-atomic (an exception can't leave them half-updated). The cost is allocating a new object for every "change".

Q56. Shallow copy vs deep copy? A shallow copy duplicates the top object but shares the objects its fields point to; a deep copy recursively copies them too. `Object.clone()` and `new ArrayList<>(list)` are shallow.

Q57. Why is `Cloneable`/`clone()` considered broken? What do you use instead? `Cloneable` has no `clone` method (it just switches on `Object.clone`'s field-by-field copy), `clone` is protected, returns `Object`, bypasses constructors, is shallow, and clashes with `final` fields. Prefer a **copy constructor** (`new Point(other)`) or a static factory (`Point.copyOf(p)`).

Q58. `Comparable` vs `Comparator`? `compareTo` defines one natural order inside the class; `Comparator` objects define any number of external orders. Never compare by subtraction (overflow); use `Integer.compare` or `Comparator.comparing`. → *More:* §10.8 · §19.29 Q19

5. Exceptions

Q59. ⋆ Explain the exception hierarchy. `Throwable` has two branches: `Error` (serious JVM problems: `OutOfMemoryError`, `StackOverflowError` — don't catch) and `Exception`. Under `Exception`, `RuntimeException` and its subclasses are **unchecked**; all other exceptions are **checked**. → *More:* §8.2 The Throwable hierarchy

Q60. ⋆ Checked vs unchecked exceptions? Checked must be caught or declared (recoverable conditions like `IOException`); unchecked signal bugs (`NullPointerException`, `IllegalArgumentException`) and needn't be declared. → *More:* §8.3 · §19.29 Q23

Q61. Error vs Exception? An `Error` means the JVM or environment is in trouble (out of memory, stack overflow, class-linkage failure); the application usually can't recover, so you let it propagate. An `Exception` is a condition the program can reasonably handle. → *More:* §8.2 The Throwable hierarchy

Q62. `throw` vs `throws`? `throw` is a statement that throws one exception object now: `throw new IllegalStateException("closed")`. `throws` is part of a method signature listing checked exceptions the

method may pass to its caller. → *More:* §8.7 Throwing exceptions

Q63. ⋆ How does try-with-resources work? Closes every `AutoCloseable` declared in the header in reverse order; a close failure after a body failure is attached as a suppressed exception (Q228 shows the order). → *More:* §8.6 · §19.29 Q24

Q64. ⋆ When does `finally` not run? What if both `try` and `finally` return? Only on `System.exit`, JVM crash/kill, or a thread that never leaves the `try`. A `return` in `finally` overrides the `try`'s return and swallows any exception — never do it (Q212). → *More:* §8.4 · §19.29 Q25

Q65. Is `try` without `catch` allowed? Yes: `try`–`finally`, or try-with-resources with no `catch` at all. A `try` alone, with neither `catch` nor `finally` nor resources, doesn't compile. → *More:* §8.4 try/catch/finally

Q66. Why must `catch` blocks go from specific to general? What is multi-catch? A `catch (Exception e)` before `catch (IOException e)` makes the second unreachable — a compile error. Multi-catch `catch (IOException | SQLException e)` handles unrelated types in one block; the variable is implicitly final, and the alternatives can't be subclasses of each other. → *More:* §8.5 Multi-catch

Q67. How do you create a custom exception? Checked or unchecked? Extend `RuntimeException` (unchecked, the common modern choice) or `Exception` (checked, when callers must handle it), and provide constructors that take a message and a **cause** so the original exception is kept (`super(msg, cause)`). → *More:* §8.7 Custom exceptions

Q68. Can an overriding method throw a broader checked exception? No. It may throw the same checked exceptions, narrower ones, or none, and any unchecked exceptions — otherwise callers using the parent type would face exceptions they didn't expect. → *More:* §6.3 Overriding

Q69. `ClassNotFoundException` vs `NoClassDefFoundError`? `ClassNotFoundException` is a checked exception from code that loads a class *by name* (`Class.forName`, a class loader) and can't find it. `NoClassDefFoundError` is an `Error`: a class that was present at compile time is missing at run time, or its static initialiser failed earlier.

Q70. What is exception chaining, and why does it matter? Wrapping a low-level exception in a higher-level one while keeping it as the **cause**: `throw new OrderException("save failed", e)`. The stack trace then shows "Caused by: …", so you don't lose the root cause. → *More:* §8.8 Chaining

Q71. What are exception-handling best practices? Don't swallow exceptions (empty `catch`); catch only what you can handle; keep the cause; log *or* rethrow, not both; prefer specific exceptions; validate arguments early (`Objects.requireNonNull`); don't use exceptions for normal control flow; clean up with try-with-resources. → *More:* §8.10 Best practices

6. Generics

Q72. Why use generics? Compile-time type safety (a `List<String>` can't receive an `Integer`), no casts on read, and reusable algorithms that work for many types. → *More:* §9.1 Why generics

Q73. ⋆ What is type erasure? Generic types are checked by the compiler, then erased to their bounds with casts inserted; at run time `List<String>` and `List<Integer>` are the same class. So no `new T()`, no `new T[]`, no `instanceof List<String>`, no `List<int>`. → *More:* §9.7 · §19.29 Q26

Q74. ⋆ Explain `? extends T` vs `? super T` (PECS). Producer `extends` (read as `T`, can't add), consumer `super` (can add `T`, reads give `Object`). → *More:* §9.6 · §19.29 Q27

Q75. Is `List<Integer>` a subtype of `List<Number>`? No — generics are **invariant**. Otherwise you could add a `Double` to a list of integers through the `List<Number>` reference. Use `List<? extends Number>` to accept both read-only. Arrays, by contrast, are covariant and fail at run time with `ArrayStoreException`. → *More:* §9.6 Invariance and wildcards

Q76. What is a bounded type parameter? `<T extends Comparable<T>> T max(List<T> xs)` restricts `T` to types that have `compareTo`, so the method can call it. Multiple bounds: `<T extends Number & Comparable<T>>` (class first). → *More:* §9.5 Bounded type parameters

Q77. What is a raw type, and why avoid it? Using a generic class without type arguments (`List list`) — kept for pre-Java-5 compatibility. It turns off type checks, so a wrong element blows up later with a `ClassCastException` far from the bug. → *More:* §9.8 Erasure restrictions

Q78. Can you overload `void f(List<String>)` and `void f(List<Integer>)`? No. After erasure both become `f(List)` — same signature, compile error ("name clash"). → *More:* §9.8 Erasure restrictions

7. Collections

Q79. Explain the Collections Framework hierarchy. `Iterable` → `Collection` → `List` (ordered, indexed: `ArrayList`, `LinkedList`), `Set` (no duplicates: `HashSet`, `LinkedHashSet`, `TreeSet`), `Queue`/`Deque` (`ArrayDeque`, `PriorityQueue`). `Map` (`HashMap`, `LinkedHashMap`, `TreeMap`) is separate because it stores key–value pairs, not single elements. Java 21 added `SequencedCollection`/`SequencedMap` for types with a defined order. → *More:* §10.1 The hierarchy

Q80. ⋆ How does `HashMap` work internally? Bucket array (16, load factor 0.75), index = spread hash `&(n−1)`, collisions chained, bucket becomes a red-black tree beyond 8 entries (when the table has ≥ 64 buckets), table doubles when size > capacity × load factor; lookup compares hash then `equals`. → *More:* §10.5 HashMap internals · §19.29 Q13

Q81. Why is `HashMap` capacity a power of two? What does "spreading" the hash do? With n a power of two, `hash & (n − 1)` is a cheap replacement for `hash % n`. Because that only uses the low bits, `HashMap` XORs the high 16 bits into the low 16 (`h ^ (h >>> 16)`) so keys whose hash codes differ only in high bits still land in different buckets. On resize each entry either stays at index *i* or moves to *i* + *oldCapacity*. → *More:* §10.5 HashMap internals

Q82. What happens when two keys have the same hash code? They go to the same bucket; `put` walks the bucket and uses `equals` to decide "same key → replace value" or "different key → add a node". `get` does the same. Many collisions degrade a bucket from O(1) to O(n) (O(log n) once treeified). → *More:* §10.5 HashMap internals

Q83. Can `HashMap` have a null key? `Hashtable`? `ConcurrentHashMap`? `TreeMap`? `HashMap`: one null key (stored in bucket 0) and null values. `Hashtable` and `ConcurrentHashMap`: no nulls at all. `TreeMap`: no null key with natural ordering (it calls `compareTo` → NPE), null values are fine. → *More:* §10.5 HashMap internals

Q84. Why must map keys be immutable? What if you mutate a key? The entry stays in the bucket of the old hash, so lookups miss it. → *More:* §10.5 · §19.29 Q14

Q85. ⋆ `HashMap` vs `Hashtable` vs `synchronizedMap` vs `ConcurrentHashMap`? Unsynchronised with nulls / legacy fully synchronised / one lock wrapper / fine-grained concurrent with atomic compound operations. → *More:* §14.12 Concurrent collections · §19.29 Q15

Q86. ⋆ How does `ConcurrentHashMap` achieve thread safety? How did Java 7 differ? Java 8+: lock-free reads, CAS into empty buckets, `synchronized` on a bucket's head node for updates, cooperative resizing; Java 7 used 16 lock "segments". Iterators are weakly consistent. → *More:* §14.12 · §19.29 Q16

Q87. ⋆ Is `map.get(k) == null ? map.put(k, v) : …` safe on a `ConcurrentHashMap`? No — it's a check-then-act race: two threads can both see `null` and both put. Use the atomic methods: `putIfAbsent`, `computeIfAbsent(k, key -> new ArrayList<>())`, `merge(word, 1, Integer::sum)`. Keep the functions short — they run while the bucket is locked. → *More:* §14.12 Concurrent collections

Q88. ⋆ `ArrayList` vs `LinkedList`? Array-backed, O(1) index access, cache-friendly vs doubly linked, O(1) only at ends or an iterator. Default to `ArrayList`. → *More:* §10.2 · §19.29 Q17

Q89. How does `ArrayList` grow? `new ArrayList<>()` starts with a shared empty array and allocates 10 slots on the first `add`. When full it grows to about 1.5× (`old + old/2`) and copies the elements. Each copy is O(n) but rare, so `add` is **amortised O(1)**. Pass an initial capacity if you know the size. → *More:* §10.2 Lists

Q90. `ArrayList` vs `Vector`? `Vector` is the legacy synchronised version (every method locked, grows 2×). Don't use it; use `ArrayList`, or `CopyOnWriteArrayList`/`Collections.synchronizedList` when shared. → *More:* §10.2 Lists

Q91. `HashSet` vs `LinkedHashSet` vs `TreeSet`? How does `HashSet` work internally? `HashSet` is unordered O(1); `LinkedHashSet` keeps insertion order; `TreeSet` keeps elements sorted, O(log n), and supports `floor`/`ceiling`/ranges. Internally a `HashSet` *is* a `HashMap` whose keys are the elements and whose values are one shared dummy object; `add` returns `false` if the key was already there. → *More:* §10.3 Sets

Q92. ⋆ `HashMap` vs `LinkedHashMap` vs `TreeMap`? How is `TreeMap` implemented? Unordered / insertion- or access-ordered / sorted. `TreeMap` is a **red-black tree** (a self-balancing binary search tree), so `get` / `put` / `remove` are O(log n) and it offers `floorKey`, `ceilingKey`, `headMap`, `tailMap`, `firstKey`. It uses `compareTo` / `Comparator`, not `equals`, to detect duplicates. → *More:* §10.4 Maps · §19.29 Q18

Q93. ⋆ Fail-fast vs fail-safe iterators? How do you remove while iterating? Fail-fast iterators throw `ConcurrentModificationException` on structural change through another path (tracked by `modCount`); concurrent collections' iterators are weakly consistent or snapshot-based. Remove with `Iterator.remove()` or `removeIf`. → *More:* §10.7 · §19.29 Q20

Q94. `Comparable` vs `Comparator` — how do you sort a list of employees by salary, then name? `list.sort(Comparator.comparingDouble(Employee::salary).thenComparing(Employee::name))`; add `.reversed()` for descending. `Collections.sort` and `List.sort` use a stable merge sort (TimSort) for objects. → *More:* §10.8

Q95. `List.of` vs `Arrays.asList` vs `Collections.unmodifiableList`? Truly unmodifiable and null-hostile / fixed-size view that writes through / read-only view of a list that may still change. → *More:* §10.10 · §19.29 Q22

Q96. What is `CopyOnWriteArrayList` and when do you use it? A thread-safe list that copies the whole underlying array on every write; reads and iteration take no lock and see a snapshot (no `ConcurrentModificationException`). Great for read-mostly data such as listener lists; terrible for frequent writes. → *More:* §14.12 Concurrent collections

Q97. `Queue`, `Deque`, `PriorityQueue` — and why prefer `ArrayDeque` to `Stack`? `Queue` is FIFO (`offer` / `poll` / `peek`), `Deque` adds both ends (stack or queue), `PriorityQueue` is a binary heap that always yields the smallest element (O(log n) `offer` / `poll`). `Stack` is synchronised legacy extending `Vector` (so it also exposes index access that breaks the stack abstraction); `ArrayDeque` is unsynchronised, faster, and gives `push` / `pop` / `peek` through the `Deque` interface. → *More:* §10.6 · §19.29 Q21

Q98. What are `EnumMap`, `WeakHashMap` and `IdentityHashMap`? `EnumMap`: a compact array indexed by enum ordinal — fastest map for enum keys. `WeakHashMap`: keys held by weak references, so an entry disappears once nothing else references the key (used for metadata caches). `IdentityHashMap`: compares keys with `==` instead of `equals` (graph traversal, serialization).

Q99. What Java 8 methods did `Map` gain? `getOrDefault`, `putIfAbsent`, `computeIfAbsent`, `computeIfPresent`, `compute`, `merge`, `forEach`, `replaceAll`, `remove(k, v)`. Word count in one line: `counts.merge(word, 1, Integer::sum)`. → *More:* §10.4 Maps

Q100. What are sequenced collections (Java 21)? New interfaces `SequencedCollection`, `SequencedSet`, `SequencedMap` give every ordered collection uniform `getFirst()`, `getLast()`, `addFirst/Last`, `removeFirst/Last` and `reversed()` — so `list.getLast()` replaces `list.get(list.size() - 1)`. → *More:* §10.11 Sequenced collections

Q101. What is the time complexity of common collection operations? `ArrayList`: get O(1), add-at-end amortised O(1), insert/remove middle O(n), `contains` O(n). `HashMap` / `HashSet`: O(1) average. `TreeMap` / `TreeSet`: O(log n). `PriorityQueue`: offer/poll O(log n), peek O(1). `ArrayDeque`: O(1) at both ends. → *More:* §10.12 Big-O cheat sheet · §19.7

Q102. ⋆ How would you implement an LRU cache? `LinkedHashMap` in access order with `removeEldestEntry` returning `size() > capacity` (Q249). Without it: a `HashMap` from key to node plus a doubly linked list — move a node to the front on access, evict from the tail; both O(1). Make it thread-safe with a lock, or use a library like Caffeine in production. → *More:* §10.4

8. Lambdas, functional interfaces and Optional

Q103. ⋆ What are the main features of Java 8? Lambdas and method references, functional interfaces (`java.util.function`), the Stream API, `Optional`, `default` and `static` interface methods, the `java.time` date/time API, `CompletableFuture`, new `Map` methods, and Metaspace replacing PermGen.

Q104. What is a lambda expression? A compact way to write an implementation of a functional interface: `(a, b) -> a + b`. It's not an anonymous class: it has no own `this` (it uses the enclosing one), and the JVM creates it through `invokedynamic`, usually without a new class file. → *More:* §11.2 Lambda syntax

Q105. ⋆ What is a functional interface? Is `@FunctionalInterface` required? An interface with exactly one abstract method (`default`/`static` methods and `Object`'s public methods don't count). The annotation is optional; it makes the compiler reject a second abstract method. → *More:* §11.2 · §19.29 Q30

Q106. Name the core functional interfaces. `Function<T,R>` (T → R), `BiFunction<T,U,R>`, `Supplier<T>` (() → T), `Consumer<T>` (T → nothing), `Predicate<T>` (T → boolean), `UnaryOperator<T>` (T → T), `BinaryOperator<T>` ((T,T) → T), plus primitive versions like `IntPredicate`, `ToIntFunction` that avoid boxing. → *More:* §11.3

Q107. Supplier vs Consumer? A `Supplier<T>` takes nothing and produces a value (`get()`), e.g. a lazy default in `orElseGet`. A `Consumer<T>` takes a value and returns nothing (`accept()`), e.g. the action in `forEach`. → *More:* §11.3 Built-in functional interfaces

Q108. What are method references? Which kinds exist? Shorthand for a lambda that only calls a method: static (`Integer::parseInt`), bound instance (`System.out::println`, `str::startsWith`), unbound instance (`String::length` — the first argument becomes the receiver), and constructor (`ArrayList::new`). → *More:* §11.4 Method references

Q109. Why must variables used in a lambda be effectively final? The lambda captures a copy of the value; allowing later reassignment would make the copy and the variable silently disagree, and lambdas may run on other threads. → *More:* §11.5

Q110. ⋆ What is `Optional`, and how should you use it? A container that holds one value or none, used as a **return type** for "maybe no result" instead of returning `null`. Consume it with `map`, `filter`, `orElse`, `orElseGet`, `orElseThrow`, `ifPresentOrElse`. `Optional.of(null)` throws NPE; use `ofNullable`. → *More:* §11.7 Optional

Q111. ⋆ `orElse` vs `orElseGet`? Common misuses of `Optional`? `orElse(x)` always evaluates `x`; `orElseGet(() -> x)` only when empty — matters for expensive defaults. Misuses: `get()` without a check, `Optional` fields/parameters, `Optional<List>`, returning `null` from an `Optional` method. → *More:* §11.7 · §19.29 Q29

Q112. What are `default` methods for? Can a default method be overridden? They let an interface gain new methods without breaking existing implementations — that's how `Collection.stream()` and `Iterable.forEach` were added in Java 8. Implementations can override them; static interface methods can't be overridden and are called as `Interface.method()`. → *More:* §6.6 Interfaces

9. Streams

Q113. What is a stream? Collection vs Stream? A stream is a one-shot pipeline of operations over a data source: it stores nothing, is lazy, doesn't modify its source, and can be consumed only once (a second terminal operation throws `IllegalStateException`). A collection stores elements and can be iterated many times. → *More:* §12.1 What a stream is

Q114. ⋆ Intermediate vs terminal operations? What does "lazy" mean? Intermediate operations (`filter`, `map`, `sorted`, `distinct`, `limit`) return a new stream and do nothing yet. A terminal operation (`collect`, `forEach`, `reduce`, `count`, `findFirst`) starts the work. Elements flow one at a time through the whole pipeline, and short-circuiting operations stop early (Q223). → *More:* §12.2 Laziness

Q115. ⋆ `map` vs `flatMap`? `map` turns each element into exactly one element (`Stream<List<Integer>>` stays a stream of lists). `flatMap` turns each element into a stream and concatenates them (`lists.stream().flatMap(List::stream)` → `Stream<Integer>`). → *More:* §12.3 Intermediate operations

Q116. How do you create a stream? `collection.stream()`, `Stream.of(a, b)`, `Arrays.stream(arr)`, `IntStream.range(0, n)`, `Stream.iterate(seed, next)`, `Stream.generate(supplier)`, `Files.lines(path)`, `string.chars()`, `Pattern.splitAsStream`. → *More:* §12.1 What a stream is

Q117. What does `collect` do? Name the useful collectors. `collect` is a mutable reduction into a container. Collectors: `toList()`, `toSet()`, `toMap(k, v, merge)`, `joining(sep)`, `groupingBy(classifier, downstream)`, `partitioningBy(predicate)`, `counting()`, `summingInt`, `averagingDouble`, `mapping`, `collectingAndThen`, `teeing` (Java 12). → *More:* §12.5 Collectors

Q118. ⋆ `groupingBy` vs `partitioningBy`? `groupingBy` splits into any number of groups by a key (`Map<K, List<T>>`); `partitioningBy` splits into exactly two by a predicate (`Map<Boolean, List<T>>`, both keys always present). → *More:* §12.5 Collectors

Q119. `Collectors.toList()` vs `Stream.toList()`? `stream.toList()` (Java 16) returns an **unmodifiable** list and allows nulls. `Collectors.toList()` currently returns an `ArrayList` but promises nothing about mutability.

Use `Collectors.toCollection(ArrayList::new)` when you need to modify the result. → *More:* §12.5 Collectors

Q120. ⋆ What happens with duplicate keys in `Collectors.toMap`? It throws `IllegalStateException` ("Duplicate key"). Pass a merge function: `toMap(Emp::dept, Emp::salary, Double::sum)`; add a map supplier such as `LinkedHashMap::new` to control the map type. → *More:* §12.5 Collectors

Q121. What does `reduce` do? Combines all elements into one value with an associative function: `nums.stream().reduce(0, Integer::sum)`. Without an identity it returns `Optional<T>`. For sums prefer `mapToInt(...).sum()` (no boxing). → *More:* §12.4 Terminal operations and reduce

Q122. `findFirst` vs `findAny`; `anyMatch`/`allMatch`/`noneMatch`? `findFirst` respects encounter order; `findAny` may return any element (cheaper in parallel). The match operations short-circuit and return `boolean`; `allMatch` on an empty stream is `true`, `anyMatch` is `false`. → *More:* §12.4 Terminal operations

Q123. ⋆ Are parallel streams always faster? When do you use them? No. They split work over the common `ForkJoinPool`; worth it only for large, CPU-bound, easily splittable sources (arrays, `ArrayList`, ranges) with independent, stateless operations. Bad for I/O, small data, `LinkedList`/`iterate` sources, ordered operations like `limit`, and shared mutable state (adding to a shared `ArrayList` from `forEach` loses elements or throws). → *More:* §12.8 Parallel streams

Q124. Why have `IntStream`, `LongStream`, `DoubleStream`? To avoid boxing every number into an object. They add `sum`, `average`, `max`, `summaryStatistics`, `range`. Convert with `mapToInt`, back with `boxed()`. → *More:* §12.6 Primitive streams

Q125. `forEach` vs `forEachOrdered`; `peek`? In parallel streams `forEach` may process in any order; `forEachOrdered` keeps encounter order (slower). `peek` is for debugging only — it runs only if a terminal operation pulls elements through it. → *More:* §12.8 Parallel streams

Q126. What are stream gatherers (Java 24)? `Stream.gather(Gatherer)` lets you write custom intermediate operations, the way collectors are custom terminal ones. Built-ins in `Gatherers`: `windowFixed(3)`, `windowSliding(3)`, `fold`, `scan`, `mapConcurrent`. → *More:* §12.9 Stream gatherers

10. Modern Java (9 → 25)

Q127. ⋆ What are the main features from Java 9 to 25? 9: modules, `List.of`, JShell. 10: `var`. 11: `String` helpers, `HttpClient`, single-file launch. 14: switch expressions. 15: text blocks. 16: records, `instanceof` patterns, `Stream.toList`. 17: sealed classes. 21: virtual threads, pattern matching for `switch`, record patterns, sequenced collections, generational ZGC. 22: unnamed `_`, Foreign Function & Memory API. 24: gatherers, no `synchronized` pinning. 25: compact source files, module imports, flexible constructor bodies, scoped values. → *More:* §17.2 The version table · §19.29 Q42

Q128. ⋆ What is a record? Can it have extra fields or extend a class? A class whose whole job is to hold a fixed set of values: the compiler writes the constructor, the accessors (`name()`, not `getName()`), and `equals`/`hashCode`/`toString`. Implicitly final, extends `Record` (so it can't extend another class), fields are `private final` (shallowly immutable), no extra instance fields (static ones are fine); can implement interfaces and validate in a compact constructor. → *More:* §6.8 Records · §19.29 Q10

Q129. Record vs Lombok `@Data` vs a normal class? A record is a language feature: final, immutable, fixed shape, no setters, works with record patterns. Lombok `@Data` generates getters, setters, `equals`/`hashCode` for a mutable class through an annotation processor. Use a normal class (or Lombok) when you need mutability, inheritance or hidden state (e.g. JPA entities). → *More:* §6.8 Records

Q130. ⋆ What are sealed classes and why use them? `sealed ... permits A, B` limits subclasses; each must be `final`, `sealed` or `non-sealed`. The compiler then knows all cases, so `switch` over the type can be exhaustive without `default`. → *More:* §6.7 · §19.29 Q11

Q131. ⋆ What is pattern matching for `instanceof` and `switch`? `if (obj instanceof String s)` tests and binds in one step (Java 16). Switch patterns (Java 21) match on type, with guards and null handling:

```java
String describe(Object o) {
    return switch (o) {
        case null -> "null";
        case Integer i when i > 0 -> "positive int";
        case Integer i -> "int";
        case String s -> "string of " + s.length();
        default -> "other";
    };
}
```

→ *More:* §3.6 Pattern matching for switch

Q132. What are record patterns? Patterns that deconstruct a record in place (Java 21): `if (p instanceof Point(int x, int y))` or `case Circle(Point(var x, var y), double r) ->`. With sealed types they give algebraic-data-type style code. Java 22's unnamed pattern `_` skips components you don't need. → *More:* §3.7 Record patterns

Q133. Switch statement vs switch expression? The arrow form (`case A -> ...`) has no fall-through. A switch **expression** yields a value (`int n = switch (d) { ... };`), must be exhaustive, and uses `yield` inside a block. The old colon form falls through without `break`. → *More:* §3.3 switch expressions

Q134. ⋆ What are virtual threads? JVM-managed lightweight threads mounted on a few carrier threads; blocking I/O unmounts them, so millions of blocking tasks are cheap. Help I/O-bound concurrency, not CPU-bound speed. Don't pool them. → *More:* §15.4 Virtual threads · §19.29 Q34

Q135. What is pinning, and is it still a problem? A virtual thread is **pinned** when it can't unmount from its carrier while blocked, so it ties up an OS thread. Before Java 24, blocking inside `synchronized` pinned; JEP 491 (Java 24) fixed that. It still pins when blocking under a native frame (a `native` method or FFM call that calls back into Java) and when blocking during class loading or class initialisation. Diagnose with the JFR event `jdk.VirtualThreadPinned`. → *More:* §15.5 Pinning and JEP 491

Q136. Virtual threads vs reactive programming (WebFlux, RxJava)? Both serve many concurrent I/O operations with few OS threads. Reactive code does it with non-blocking callbacks and operator chains — efficient, but hard to read and debug. Virtual threads keep simple blocking code and normal stack traces. Reactive still fits streaming and back-pressure use cases.

Q137. What are scoped values (Java 25)? `ScopedValue` passes immutable context (user, request ID) down a call chain for a bounded scope — `ScopedValue.where(USER, u).run(task)` — without parameters. Unlike `ThreadLocal` it can't be changed, is unbound automatically when `run` / `call` returns, and child threads forked in a `StructuredTaskScope` inherit it without copying. Final in Java 25 (JEP 506). → *More:* §15.6 ThreadLocal and scoped values

Q138. What is structured concurrency? Is it final? It treats a group of subtasks as one unit: fork them in a scope, join, and if one fails the others are cancelled — no leaked threads. Still a **preview** in Java 25 (fifth preview, JEP 505), so it needs `--enable-preview`. → *More:* §15.7 Structured concurrency

Q139. What are Java modules (JPMS)? Java 9's module system: a `module-info.java` declares what a module `requires` and which packages it `exports`. It gives strong encapsulation (non-exported packages are inaccessible, even by reflection unless `opens`), reliable configuration, and `jlink` custom runtimes. Java 25 (JEP 511) adds `import module java.base;` to import all packages a module exports. → *More:* §16.12 Modules

Q140. Why upgrade from Java 8/11/17 to 21 or 25? Language productivity (records, patterns, text blocks), virtual threads, better GCs (generational ZGC, G1 improvements), lower memory (compact object headers, a product option in 25 enabled with `-XX:+UseCompactObjectHeaders`), faster start-up (the AOT cache, Java 24–25), security and support lifetime (LTS), and frameworks such as Spring Boot 3+ requiring Java 17+. → *More:* §17.12 Upgrading

11. Concurrency

Q141. Process vs thread? A **process** is a running program with its own memory space; a **thread** is a path of execution inside a process, sharing the heap with other threads but having its own stack. Threads are cheaper to create and communicate through shared memory — which is why they need synchronisation. → *More:* §14.1 Concurrency basics

Q142. ⋆ How do you create a thread? `Runnable` vs extending `Thread`? Implement `Runnable`/`Callable` and hand it to a `Thread` or, better, an `ExecutorService`; Java 21 adds `Thread.ofPlatform()`/`Thread.ofVirtual()` builders. Prefer `Runnable` over extending `Thread`: it separates the task from the mechanism and leaves your class free to extend something else. → *More:* §14.2 Creating threads

Q143. ⋆ `start()` vs `run()`? `start()` creates a new thread that then calls `run()`. Calling `run()` directly just executes the method on the current thread — no concurrency (Q232). Calling `start()` twice throws `IllegalThreadStateException`. → *More:* §14.2 Creating threads

Q144. What are the thread states? `NEW`, `RUNNABLE` (running or ready), `BLOCKED` (waiting for a monitor lock), `WAITING` (`wait()`, `join()`, `park` with no timeout), `TIMED_WAITING` (`sleep`, timed `wait`/`join`), `TERMINATED`. → *More:* §14.3 Thread lifecycle

Q145. ⋆ `sleep()` vs `wait()`? `Thread.sleep` pauses the current thread and keeps any locks it holds. `obj.wait()` must be called while holding `obj`'s monitor (else `IllegalMonitorStateException`), **releases** it, and waits until `notify`/`notifyAll`, a timeout, or interruption. Always call `wait` in a `while` loop that re-checks the condition (spurious wake-ups). → *More:* §14.8 wait/notify

Q146. `notify()` vs `notifyAll()`? `notify` wakes one arbitrary waiter; `notifyAll` wakes all, and each re-checks its condition. Prefer `notifyAll` unless all waiters wait for the same condition and any one can proceed — a wrong `notify` can wake the "wrong" thread and stall everyone. → *More:* §14.8 wait/notify

Q147. Why are `wait`/`notify` in `Object` and not in `Thread`? Because every object has a monitor and the waiting is on a *condition protected by that object's lock*, not on a particular thread. → *More:* §14.8 wait/notify

Q148. What is `join()`? What does `yield()` do? `t.join()` blocks the caller until thread `t` finishes (and guarantees its writes are visible). `Thread.yield()` hints that the scheduler may run another thread — rarely useful. → *More:* §14.3 Thread lifecycle

Q149. How do you stop a thread? Cooperatively: call `t.interrupt()`, and have the task check `Thread.currentThread().isInterrupted()` or catch `InterruptedException` from blocking calls and exit. If you catch `InterruptedException` and can't exit, restore the flag with `Thread.currentThread().interrupt()`. `Thread.stop()` is gone (throws `UnsupportedOperationException` since Java 20). → *More:* §14.4 Interruption

Q150. ⋆ What is a race condition? Why isn't `count++` thread-safe? A race condition is when correctness depends on the timing of threads. `count++` is three steps — read, add, write — so two threads can read the same value and one increment is lost. Fix with `synchronized`, `AtomicInteger.incrementAndGet()`, or `LongAdder`. → *More:* §14.5 Race conditions

Q151. ⋆ How does `synchronized` work? Method vs block; static synchronized? Every object has an intrinsic lock (**monitor**). A synchronized instance method locks `this`; a static synchronized method locks the `Class` object; a block locks the object you name (prefer a private lock object and the smallest block). The lock is **reentrant** (the owning thread can re-acquire it) and gives visibility on release/acquire. → *More:* §14.6 synchronized

Q152. ⋆ What does `volatile` do? Is it enough for a counter? Visibility and ordering of reads/writes of that field; no atomicity, so not enough for `count++`. Fine for a stop flag or a published reference. → *More:* §14.7 volatile · §19.29 Q31

Q153. What is the Java Memory Model / happens-before? The rules that say when one thread's writes must be visible to another: unlock → lock, volatile write → read, `start()`, `join()`, program order. Without such an edge, reordering and caching can hide writes. → *More:* §15.8 · §19.29 Q32

Q154. ⋆ `synchronized` vs `ReentrantLock`? `ReentrantLock` does the same mutual exclusion but adds `tryLock()` (with timeout), interruptible locking, optional fairness, and several `Condition`s per lock. You must `unlock()` in `finally`. Use `synchronized` by default (simpler, can't forget to release); use `ReentrantLock` for those extras. → *More:* §14.9 Explicit locks

Q155. What are `ReadWriteLock` and `StampedLock`? `ReentrantReadWriteLock` lets many readers in at once but only one writer — useful when reads dominate. `StampedLock` (Java 8) adds **optimistic reads**: read without locking, then `validate(stamp)` and retry under a real lock if a write happened. `StampedLock` isn't reentrant. → *More:* §14.9 Explicit locks

Q156. What are atomic classes and CAS? `AtomicInteger`, `AtomicLong`, `AtomicReference` update a single variable without locks using **CAS** (compare-and-set: "set to new value only if it still equals the

expected value", one CPU instruction, retried on failure). Under heavy contention on a counter, `LongAdder` scales better by spreading updates over cells. → *More:* §14.10 Atomic variables

Q157. What is the ABA problem? A CAS sees value A, another thread changes it to B and back to A, and the CAS succeeds even though the state changed in between. Fix with a version stamp: `AtomicStampedReference`.

Q158. ⋆ What is a deadlock? How do you prevent and detect it? Threads holding locks while waiting for each other's locks forever. Prevent with a global lock order, short critical sections, `tryLock` timeouts; detect with a thread dump (`jstack`, `jcmd <pid> Thread.print`), which reports "Found one Java-level deadlock". Code in Q252. → *More:* §15.9 Liveness · §19.29 Q33

Q159. Livelock vs starvation? Livelock: threads keep reacting to each other and changing state but make no progress (two people stepping aside in a corridor forever). **Starvation**: a thread never gets the CPU or lock because others always win; fair locks or bounded queues help. → *More:* §15.9 Liveness

Q160. ⋆ Why use `ExecutorService` instead of `new Thread()`? What types are there? Reuse, bounded threads, task queue, futures, one place to shut down. Factories: `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newWorkStealingPool()`, `newVirtualThreadPerTaskExecutor()` (21). → *More:* §14.11 Executors · §19.29 Q36

Q161. ⋆ Explain `ThreadPoolExecutor`'s parameters and what happens when a task is submitted. `corePoolSize`, `maximumPoolSize`, `keepAliveTime`, `workQueue`, `threadFactory`, `RejectedExecutionHandler`. On submit: start a new thread if fewer than *core* are running; else queue the task; if the queue is full, start threads up to *max*; if that's full too, **reject** (default `AbortPolicy` throws; alternatives `CallerRunsPolicy`, `DiscardPolicy`, `DiscardOldestPolicy`). Note `newFixedThreadPool` uses an **unbounded** queue — tasks can pile up until out of memory; in production build a `ThreadPoolExecutor` with a bounded queue. → *More:* §14.11 Executors

Q162. How big should a thread pool be? CPU-bound: about the number of cores (`Runtime.getRuntime().availableProcessors()`). I/O-bound: cores × (1 + wait time / compute time), or simply virtual threads with a `Semaphore` around the scarce resource. Measure, don't guess. → *More:* §14.11 Executors

Q163. `shutdown()` vs `shutdownNow()`? `shutdown()` stops accepting new tasks and lets queued ones finish; `shutdownNow()` also interrupts running tasks and returns the queued ones. Follow with `awaitTermination(timeout)`. Since Java 19, `ExecutorService` is `AutoCloseable`, so try-with-resources waits for all tasks. → *More:* §14.11 Executors

Q164. `Runnable` vs `Callable`; `Future` vs `CompletableFuture`? `Callable` returns a value and may throw checked exceptions. `Future.get()` blocks; `CompletableFuture` composes asynchronously. → *More:* §15.2 CompletableFuture · §19.29 Q35

Q165. ⋆ `thenApply` vs `thenCompose` vs `thenCombine`; how are errors handled in `CompletableFuture`? `thenApply` maps the result (like `map`); `thenCompose` chains another async call that returns a `CompletableFuture` (like `flatMap`); `thenCombine` joins two independent futures; `allOf`/`anyOf` wait for many. Errors: `exceptionally` (recover), `handle` (result or error), `whenComplete` (side effect). Without an executor argument, `*Async` methods run on the common `ForkJoinPool` — pass your own executor for blocking I/O. `orTimeout`/`completeOnTimeout` (Java 9) add timeouts. Code in Q255. → *More:* §15.2 CompletableFuture

Q166. What is the Fork/Join framework? A pool for divide-and-conquer tasks (`RecursiveTask`/`RecursiveAction`): split a task, `fork()` one half, compute the other, `join()`. Idle workers **steal** tasks from busy workers' queues. Parallel streams and default `CompletableFuture` async methods use the common pool. → *More:* §15.3 Fork/Join

Q167. ⋆ `CountDownLatch` vs `CyclicBarrier` vs `Semaphore`? `CountDownLatch`: threads wait until a count reaches zero; one-shot (e.g. wait for 3 services to start). `CyclicBarrier`: N threads wait for each other at a point, then all continue; reusable. `Semaphore`: N permits limit how many threads use a resource at once (e.g. at most 10 DB calls). → *More:* §14.13 Synchronizers

Q168. What is a `BlockingQueue`? Which implementations exist? A thread-safe queue where `put` blocks when full and `take` blocks when empty — the standard producer–consumer tool (Q253). `ArrayBlockingQueue` (bounded array), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `DelayQueue`, `SynchronousQueue` (no capacity: each put waits for a take). → *More:* §14.12 Concurrent collections

Q169. ⋆ What is `ThreadLocal`, and what is its danger? A variable with a separate value per thread (e.g. a `SimpleDateFormat` or the current request's user). In thread pools threads live forever, so values leak or bleed into the next task unless you `remove()` in `finally`. With virtual threads prefer scoped values (Q137). → *More:* §15.6

Q170. What is a daemon thread? A background thread that doesn't keep the JVM alive: when only daemon threads remain, the JVM exits without running their `finally` blocks. Set with `setDaemon(true)` before `start()`. Virtual threads are always daemon threads. → *More:* §14.2 Creating threads

Q171. How do you design a thread-safe class? Prefer immutability; otherwise confine state (one thread owns it), guard every access to shared mutable state with the same lock, use thread-safe building blocks (`ConcurrentHashMap`, atomics), don't let `this` escape the constructor, and document the policy (`@GuardedBy`). Compound actions (check-then-act) need one atomic step. → *More:* §14.14 Designing thread-safe classes

12. JVM, memory and garbage collection

Q172. ⋆ What are the JVM memory areas? Heap vs stack? Heap (objects, shared, GC'd), one stack per thread (frames, locals), metaspace (class metadata, native memory), code cache, direct/native memory. The stack is per thread, LIFO, freed automatically on return, and holds primitives and references; objects themselves live on the heap (unless escape analysis removes the allocation). → *More:* §16.3 · §19.29 Q37

Q173. ⋆ What replaced PermGen, and why? Java 8 replaced PermGen (a fixed-size heap region for class metadata, prone to `OutOfMemoryError: PermGen space`) with **Metaspace** in native memory, which grows as needed (cap it with `-XX:MaxMetaspaceSize`). Interned strings and static fields had already moved to the heap in Java 7. → *More:* §16.3 Memory areas

Q174. ⋆ How does garbage collection work? Which collector is the default? Objects not reachable from GC roots are reclaimed; generational collectors collect the young generation often and the old one rarely. G1 is the default on typical servers since Java 9; on a machine or container with one CPU or under about 1.8 GB of memory the JVM picks Serial instead (still true in 25; JEP 523 makes G1 the default everywhere in a later release). Parallel suits throughput; ZGC (generational only since Java 24) and Shenandoah target sub-millisecond pauses. → *More:* §16.5 GC algorithms · §19.29 Q38

Q175. What are GC roots, and how does an object become eligible for GC? Roots are references the JVM always considers live: local variables on thread stacks, static fields, active threads, JNI references. An object is eligible once no chain of references from a root reaches it — even if objects reference each other in a cycle. → *More:* §16.5 GC algorithms

Q176. Explain young generation, old generation, minor and major GC. New objects go to **Eden**; a **minor GC** copies survivors to a **survivor** space, and objects that survive several collections are **promoted** to the **old generation**. A **major/full GC** collects the old generation (and usually everything), typically with longer pauses. The design relies on the observation that most objects die young. → *More:* §16.5 GC algorithms

Q177. Can you force garbage collection? No. `System.gc()` is only a request (and can be disabled with `-XX:+DisableExplicitGC`). Calling it in application code is almost always wrong. → *More:* §16.5 GC algorithms

Q178. Strong, soft, weak and phantom references? Strong: normal; never collected while reachable. **Soft**: cleared only when memory is short (memory-sensitive caches). **Weak**: cleared at the next GC once only weakly reachable (`WeakHashMap`). **Phantom**: only tells you (through a `ReferenceQueue`) that the object was collected; used for clean-up, as `Cleaner` does.

Q179. Name the types of `OutOfMemoryError` and `StackOverflowError` causes. `Java heap space` (too many live objects or a leak), `Metaspace` (too many classes, class-loader leak), `GC overhead limit exceeded` (GC running nearly all the time), `unable to create native thread`, `Direct buffer memory`. `StackOverflowError` comes from too-deep recursion (usually infinite recursion); `-Xss` sets the stack size. → *More:* §16.3 Memory areas

Q180. ⋆ Can Java have memory leaks? How do you find one? Yes — anything still referenced: unbounded caches, static collections, listeners, `ThreadLocal`s, class-loader leaks. Watch heap-after-GC grow, then

use `jcmd GC.class_histogram`, JFR, or a heap dump in Eclipse MAT. → *More:* §16.11 Memory leaks · §19.29 Q39

Q181. ⋆ What is a class loader? Explain parent delegation. A class loader turns `.class` bytes into `Class` objects. Built in: **bootstrap** (core `java.*`), **platform** (other JDK modules; replaced the "extension" loader in Java 9), **application/system** (your class path). A loader first asks its parent; only if the parent can't find the class does it load it itself — so nobody can replace `java.lang.String`. A class is identified by its name **and** its loader. → *More:* §16.2 Class loading · §19.29 Q40

Q182. What are the phases of class loading? Loading (find bytes, create `Class`), **linking** — *verification* (bytecode is safe), *preparation* (static fields get default values), *resolution* (symbolic references → real ones) — and **initialisation** (static initialisers run), which happens lazily on first active use. → *More:* §16.2 Class loading

Q183. What is the JIT compiler? The just-in-time compiler turns frequently executed ("hot") bytecode into native machine code at run time. HotSpot uses tiered compilation: C1 compiles quickly with light optimisation, C2 later with heavy optimisation (inlining, escape analysis, loop unrolling), using profiling data gathered while interpreting. → *More:* §16.8 JIT compilation

Q184. Which JVM flags do you know? `-Xms` / `-Xmx` (initial/max heap), `-Xss` (thread stack), `-XX:+UseG1GC` / `UseZGC` / `UseParallelGC`, `-XX:MaxGCPauseMillis`, `-XX:+HeapDumpOnOutOfMemoryError`, `-Xlog:gc*` (GC logging), `-XX:MaxRAMPercentage` (heap as % of container memory), `-XX:+UseCompactObjectHeaders` (Java 25). → *More:* §16.7 GC tuning basics

Q185. How do you choose a garbage collector? G1 (default) balances throughput and pauses for most services. Parallel GC for batch jobs where total throughput matters and pauses don't. ZGC (generational) or Shenandoah for large heaps and latency-sensitive services needing pauses around a millisecond. Serial for tiny heaps or single-CPU containers. → *More:* §16.6 Collectors compared

Q186. What tools do you use to diagnose a running JVM? `jcmd` (thread dumps, heap dumps, GC info, JFR control), `jstack` (thread dump), `jmap` / `jcmd GC.heap_dump`, `jstat` (GC statistics), JDK Flight Recorder + JDK Mission Control (low-overhead profiling), VisualVM, async-profiler, and Eclipse MAT for heap dumps. → *More:* §16.10 Profiling and diagnostic tools

Q187. What is escape analysis? A JIT analysis that proves an object never leaves a method or thread; the JIT can then skip the heap allocation (scalar replacement — keep the fields in registers) and remove locks on it. → *More:* §16.8 JIT and escape analysis

13. I/O, serialization, reflection and annotations

Q188. Byte streams vs character streams? Why buffer? `InputStream` / `OutputStream` move raw bytes (images, binary); `Reader` / `Writer` move characters and need a **charset** to convert (always specify UTF-8). `BufferedReader` / `BufferedOutputStream` batch many small reads/writes into few system calls — often 10–100× faster. → *More:* §13.5 Byte and character streams · §13.6 Buffering

Q189. `java.io` vs NIO / NIO.2? `java.io` is stream-based and blocking (`File`, `FileInputStream`). NIO (Java 4) added buffers, channels and selectors for non-blocking I/O. NIO.2 (Java 7) added `Path` and `Files` — the modern API: `Files.readString`, `Files.lines`, `Files.walk`, better errors, symbolic links, file attributes. → *More:* §13.1 java.io vs NIO.2

Q190. How do you read a large file line by line efficiently? `try (Stream<String> lines = Files.lines(path, UTF_8)) { ... }` or `BufferedReader.readLine()` in a loop — both stream the file rather than loading it all. Use `Files.readString` / `readAllLines` only for small files. → *More:* §13.4 Reading and writing whole files

Q191. ⋆ What is serialization? What is `serialVersionUID`? Converting an object graph to bytes (`ObjectOutputStream`) and back (`ObjectInputStream`); the class must implement `Serializable`. `serialVersionUID` is the class's version number: if the stream's value differs from the class's, deserialization fails with `InvalidClassException`. Declare it explicitly, otherwise the compiler-computed default changes with almost any class edit. → *More:* §13.10 Serialization

Q192. What does `transient` do? Are static fields serialized? A `transient` field is skipped and comes back as its default value (`null`, 0). Static fields belong to the class, not the object, so they're never serialized. → *More:* §13.10 Serialization

Q193. `Serializable` vs `Externalizable`? What if a parent isn't serializable? `Serializable` is automatic (reflection-based); `Externalizable` makes you write `writeExternal`/`readExternal` yourself and requires a public no-arg constructor. If a superclass isn't serializable, its fields aren't saved and its no-arg constructor runs during deserialization (it must exist).

Q194. Why is Java serialization considered dangerous? How do singletons survive it? Deserializing untrusted bytes can execute "gadget chains" of existing classes — a classic remote-code-execution hole; it also bypasses constructors and ties you to class internals. Prefer JSON/Protobuf, and if you must, use deserialization filters (`ObjectInputFilter`). A singleton needs `readResolve()` returning the instance, or simply use an enum. → *More:* §13.10 Serialization

Q195. What is reflection? Pros and cons? The ability to inspect and use classes at run time: `Class.forName`, `getDeclaredMethods`, `Field.setAccessible(true)`, `Method.invoke`. Frameworks (Spring, Hibernate, JUnit, Jackson) depend on it. Cons: slower, no compile-time checks, breaks encapsulation, and module boundaries restrict it.

Q196. How do annotations work? What is retention? Annotations are metadata on code elements. `@Retention` says how long they live: `SOURCE` (dropped by compiler, e.g. `@Override`), `CLASS` (in the class file, not at run time — the default), `RUNTIME` (readable by reflection, e.g. `@Test`, Spring's annotations). `@Target` limits where they may appear. A custom one: `@Retention(RUNTIME) @Target(METHOD) @interface Timed {}`.

Q197. How many ways can you create an object? `new`, reflection (`clazz.getDeclaredConstructor().newInstance()`), `clone()`, deserialization, and factory methods / builders (which use `new` underneath). Records and enums add their own restrictions (enums can't be instantiated reflectively).

14. Testing and tooling

Q198. ⋆ How do you write a unit test in JUnit (5/6)? Annotate a method with `@Test`, arrange inputs, act, assert with `assertEquals`/`assertTrue` or AssertJ's `assertThat(x).isEqualTo(y)`. Lifecycle: `@BeforeEach`/`@AfterEach` around each test, `@BeforeAll`/`@AfterAll` (static) once per class; JUnit 4 called these `@Before`/`@BeforeClass`. The book uses JUnit 6, which keeps the JUnit 5 ("Jupiter") API. → *More:* §18.4 JUnit

Q199. How do you test that a method throws an exception? `var ex = assertThrows(IllegalArgumentException.class, () -> account.withdraw(-1)); assertEquals("negative amount", ex.getMessage());` or AssertJ `assertThatThrownBy(...).isInstanceOf(...).hasMessage(...)`. → *More:* §18.4 JUnit

Q200. ⋆ What is mocking? Mock vs stub vs spy? A **test double** replaces a real dependency. A **stub** returns canned answers; a **mock** also lets you verify interactions (`verify(repo).save(order)`); a **spy** wraps a real object and calls real methods unless stubbed. With Mockito: `@Mock`, `@InjectMocks`, `when(x.f()).thenReturn(v)`, `ArgumentCaptor`. Static methods can be mocked with `mockStatic` (Mockito 5's default inline mock maker), though needing it is often a design smell. → *More:* §18.6 Test doubles and Mockito

Q201. What are parameterized tests? One test method run with many inputs: `@ParameterizedTest` with `@ValueSource`, `@CsvSource`, `@MethodSource` or `@EnumSource`. → *More:* §18.5 Parameterized tests

Q202. Unit vs integration vs end-to-end tests; what is TDD? Unit: one class in isolation, fast. Integration: several real components together (database via Testcontainers, HTTP layer). End-to-end: the whole system as a user sees it. Keep many unit tests, fewer integration, few E2E (the test pyramid). **TDD** (test-driven development): write a failing test, make it pass, refactor. → *More:* §18.9 Test design

Q203. Maven vs Gradle? What are the Maven lifecycle phases? Both resolve dependencies and build. Maven: XML `pom.xml`, fixed conventions, lifecycle `validate → compile → test → package → verify → install → deploy`. Gradle: Kotlin/Groovy build scripts, incremental builds and build cache, more flexible and usually faster. Dependency scopes like `test` or `provided` keep libraries out of the runtime. → *More:* §18.2 Maven · §18.3 Gradle

Q204. How should you log in Java? Through the SLF4J API with Logback or Log4j 2 behind it; use parameterised messages (`log.info("user {} logged in", id)`) so strings are built only when the level is enabled; never log secrets; log an exception once, with its stack trace. → *More:* §18.10 Logging

15. Scenario and senior-level questions

Q205. ⋆ Your service's CPU is at 100%. How do you investigate? Find the hot threads (`top -H -p <pid>`, convert the thread ID to hex) and match them in a thread dump (`jcmd <pid> Thread.print`), taking several dumps a few seconds apart. Check GC logs — constant GC from a nearly full heap looks like high CPU. Profile with JFR or async-profiler for a flame graph. Typical culprits: infinite loops, regex backtracking, busy-wait spin loops, and GC thrashing. → *More:* §16.10 Diagnostic tools

Q206. ⋆ The application throws `OutOfMemoryError` in production. What do you do? Read the message (heap, Metaspace, native thread…). Make sure `-XX:+HeapDumpOnOutOfMemoryError` is on, open the dump in Eclipse MAT and look at the dominator tree / biggest retained sizes. Check whether heap-after-GC grows steadily (leak) or the heap is simply too small for the load. Fix the leak or bound the cache before raising `-Xmx`. → *More:* §16.11 Memory leaks

Q207. Requests hang and threads pile up. What would you look at? Thread dumps: are many threads `BLOCKED` on one lock, `WAITING` on a connection pool, or stuck in socket reads with no timeout? Typical fixes: set timeouts on every remote call, size or bound pools, remove the lock contention, and look for a deadlock reported in the dump. → *More:* §16.10 Diagnostic tools

Q208. How would you make a `HashMap`-based cache thread-safe and bounded? Use `ConcurrentHashMap` with `computeIfAbsent` for atomic load-on-miss, and add eviction (size or time based) — a hand-rolled version gets hard quickly, so in production use Caffeine. For an LRU cache with a lock, wrap a `LinkedHashMap` (Q249) with `synchronized` methods.

Q209. Why can a `parallelStream()` make a web service slower? All parallel streams share the JVM-wide common `ForkJoinPool` (sized cores − 1). Under many concurrent requests they compete for the same few threads, and blocking calls inside them stall the pool for everyone. In servers, prefer sequential streams, or an explicit executor for parallel work. → *More:* §12.8 Parallel streams

16. "What does this print?" output puzzles

All outputs below were produced by running the code.

Q210. ⋆ String pool and `==`.

```
String s1 = "Java", s2 = "Ja" + "va", s3 = new String("Java");
String part = "Ja", s4 = part + "va";
final String fp = "Ja";
System.out.println((s1 == s2) + " " + (s1 == s3) + " " + (s1 == s4) + " "
        + (s1 == s4.intern()) + " " + (s1 == fp + "va"));
```

Prints `true false false true true`. Literal-only concatenation (and a `final` constant) is folded at compile time into the pooled `"Java"`; `new` and run-time concatenation create new objects; `intern()` returns the pooled one. → *More:* §7.1 Strings and the pool

Q211. ⋆ The `Integer` cache.

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));
```

Prints `true false true`. Autoboxing calls `Integer.valueOf`, which caches −128…127; 128 gives two different objects. Compare wrappers with `equals`. → *More:* §7.6 Wrappers and autoboxing

Q212. ⋆ `return` in `try` and `finally`.

```
static int f1() { try { return 1; } finally { return 2; } }
static int f2() { int x = 1; try { return x; } finally { x = 2; } }
```

`f1()` returns `2` — the `finally` return replaces the first one. `f2()` returns `1` — the return value was already evaluated before `finally` changed `x`. → *More:* §8.4 try/catch/finally

Q213. String concatenation order. `System.out.println(1 + 2 + "3" + 4 + 5);` prints `3345`. `+` is left-associative: `1 + 2` is int addition (3), then everything after the first string is concatenation. → *More:* §7.1 Strings and the pool

Q214. `char` arithmetic. `System.out.println('a' + 'b');` prints `195` (both promoted to `int`: 97 + 98), but `"" + 'a' + 'b'` prints `ab`. And `char ch = 'a'; ch += 1;` gives `b` (compound assignment casts back), while `System.out.println(ch + 1)` with the original `ch = 'a'` prints `98` (an `int`).

Q215. Floating point and overflow. `0.1 + 0.2 == 0.3` is `false` (sum is `0.30000000000000004`). `Math.abs(Integer.MIN_VALUE)` is `-2147483648` (its positive value doesn't fit in an `int`), and `Integer.MAX_VALUE + 1` is `-2147483648`. → *More:* §2.11 Floating point · §2.10 Integer overflow

Q216. ⋆ Overloading with `null`.

```
static void m(Object o) { System.out.println("Object"); }
static void m(String s) { System.out.println("String"); }
m(null);
```

Prints `String`: the compiler picks the **most specific** applicable overload. Add `m(Integer)` too and the call becomes ambiguous — a compile error. → *More:* §4.3 Overloading

Q217. ⋆ Overriding vs hiding (fields and static methods).

```
class P { String name = "P"; static String s() { return "P.s"; } String who() { return "P.who"; } }
class C extends P { String name = "C"; static String s() { return "C.s"; } String who() { return "C.who"; } }
P p = new C();
System.out.println(p.name + " " + p.s() + " " + p.who());
```

Prints `P P.s C.who`. Fields and static methods are resolved from the **declared** type (`P`); only instance methods are dispatched on the run-time object. → *More:* §6.2 Hiding

Q218. ⋆ Calling an overridable method from a constructor.

```
class P { P() { init(); } void init() {} }
class C extends P { String v = "set"; void init() { System.out.println("init sees v=" + v); } }
new C();
```

Prints `init sees v=null`. `P`'s constructor runs before `C`'s field initialisers, but `init()` dispatches to `C`. Never call overridable methods from constructors. → *More:* §5.10 Initialisation order

Q219. `i = i++`. `int i = 0; i = i++;` leaves `i` at `0`: `i++` yields the old value (0), increments `i` to 1, then the assignment writes the old value 0 back. Similarly `int x = 5; x += x++ * 2;` gives `15` (5 + 5·2, with the left `x` read before the increment).

Q220. `List.remove(int)` vs `remove(Object)`.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.remove(1);                    // index 1 → [1, 3]
list.remove(Integer.valueOf(1));   // the value 1 → [3]
```

An `int` argument picks `remove(int index)` because exact match beats boxing.

Q221. `Arrays.asList` traps. `Arrays.asList(new int[]{1, 2, 3}).size()` is `1` — a primitive array is one object, giving a `List<int[]>`; `Arrays.asList(1, 2, 3).size()` is `3`. And `Arrays.asList(3, 1, 2).add(4)` throws `UnsupportedOperationException` (fixed-size view).

Q222. ⋆ Removing inside a for-each loop.

```
List<String> l = new ArrayList<>(List.of("a", "b", "c"));
for (String x : l) if (x.equals("b")) l.remove(x);
System.out.println(l);
```

Prints `[a, c]` — **no** exception, because removing the second-to-last element shrinks the list so `hasNext()` returns false before the check runs. Removing `"a"` instead throws `ConcurrentModificationException`. Either way, use `l.removeIf(...)`. → *More:* §10.7 Iterators

Q223. ⋆ Stream laziness and short-circuiting.

```
Stream.of(1, 2, 3).peek(x -> System.out.println("never")).map(x -> x * 2);
String r = Stream.of("a", "bb", "ccc")
        .filter(x -> { System.out.print(x + " "); return x.length() > 1; })
        .findFirst().get();
```

The first line prints nothing (no terminal operation). The second prints `a bb` and returns `bb`: `findFirst` stops the pipeline, so `"ccc"` is never examined. → *More:* §12.2 Laziness

Q224. ⋆ `equals` without `hashCode`.

```
class Key { int k; Key(int k) { this.k = k; }
    public boolean equals(Object o) { return o instanceof Key x && x.k == k; } }
Set<Key> set = new HashSet<>(); set.add(new Key(1)); set.add(new Key(1));
System.out.println(set.size());
```

Prints `2` even though the keys are equal: each has a different identity hash code, so they land in different buckets. → *More:* §6.13 equals and hashCode

Q225. Ternary unboxing NPE.

```
Integer n = null;
int y = flag ? n : 0;   // NullPointerException when flag is true
```

With one `Integer` and one `int` operand, the result type is `int`, so `n` is unboxed.

Q226. Narrowing casts. `(byte) 200` is `-56` (keeps the low 8 bits), `(int) 3.99` is `3` (truncates toward zero), `(char) 65` is `A`.

Q227. Strings are immutable. `String s = "hello"; s.toUpperCase(); System.out.println(s);` prints `hello`. The result was discarded — write `s = s.toUpperCase();`.

Q228. try-with-resources close order.

```
try (R r1 = new R("first"); R r2 = new R("second")) { System.out.println("body"); }
```

Prints `body`, `close second`, `close first` — resources close in reverse order of declaration. → *More:* §8.6 try-with-resources

Q229. Division edge cases. `10 / 0` throws `ArithmeticException: / by zero`; `10.0 / 0` is `Infinity`; `0.0 / 0` is `NaN`; `Double.NaN == Double.NaN` is `false` (use `Double.isNaN`). `Math.round(-2.5)` is `-2` and `Math.round(2.5)` is `3` (it rounds half up: `floor(x + 0.5)`). → *More:* §2.11 Floating point

Q230. Overload resolution order: widening, boxing, varargs. With `m2(long)` and `m2(Integer)`, `m2(5)` prints `long`. With `m3(long)` and `m3(int...)`, `m3(5)` prints `long3`. The compiler tries, in phases: exact/widening first, then boxing, then varargs. → *More:* §4.3 Overloading

Q231. Immutable factory traps. `List.of("a", null)` throws `NullPointerException`; `Map.of("k", 1, "k", 2)` throws `IllegalArgumentException` (duplicate key).

Q232. `run()` instead of `start()`. `new Thread(() -> System.out.println(Thread.currentThread().getName())).run();` prints `main` — no new thread was started.

Q233. `HashMap` and the null key. `map.put(null, 1); map.put(null, 2);` leaves `{null=2}`. `put` returns the previous value, so `map.put("z", 3)` on an absent key returns `null`.

Q234. `StringBuilder` equality. `new StringBuilder("x").equals(new StringBuilder("x"))` is `false` — `StringBuilder` doesn't override `equals`. Compare `sb1.toString().equals(sb2.toString())` or use `sb1.compareTo(sb2) == 0` (Java 11).

Q235. `BigDecimal` in `HashSet` vs `TreeSet`. Adding `new BigDecimal("1.0")` and `new BigDecimal("1.00")`: a `HashSet` has size `2` (`equals` compares scale too), a `TreeSet` has size `1` (`compareTo` says they're equal). Also `new BigDecimal(0.1)` prints `0.1000000000000000055511151231257827021181583404541015625`, while `BigDecimal.valueOf(0.1)` prints `0.1`.

Q236. `switch` on a null string. `String s = null; switch (s) { case "a" -> ...; default -> ...; }` throws `NullPointerException` — `default` doesn't match `null`. Since Java 21 you can add `case null ->`.

Q237. Overflow before widening. `long big = Integer.MAX_VALUE * 2;` is `-2`: the multiplication happens in `int` and overflows *before* the assignment widens it. `Integer.MAX_VALUE * 2L` gives `4294967294`. → *More:*

§2.10 Integer overflow

Q238. Integer literal prefixes. `System.out.println(010 + 0x10 + 0b10);` prints `26`: octal 8 + hex 16 + binary 2.

Q239. `split` drops trailing empty strings. `"a,b,,".split(",").length` is `2`. Use `split(",", -1)` to keep them (length 4).

17. Coding tasks asked in Java rounds

These are the "write it in Java" tasks that keep appearing in Java-developer interviews (especially Java 8 stream tasks). Each solution was run; the result is shown in the comment.

Q240. ⋆ Reverse a string.

```
String s = "programming";
String r1 = new StringBuilder(s).reverse().toString();   // gnimmargorp
String r2 = "";
for (int i = s.length() - 1; i >= 0; i--) r2 += s.charAt(i);        // O(n²): mention it
```

Interviewers often forbid `reverse()`: swap `char`s in a `char[]` with two pointers instead (O(n)). → *More:* §19.2

Q241. Check whether a string is a palindrome.

```
boolean pal = IntStream.range(0, w.length() / 2)
        .allMatch(i -> w.charAt(i) == w.charAt(w.length() - 1 - i));  // "racecar" → true
```

For "ignore case and punctuation" variants, skip non-letters with two pointers and compare `Character.toLowerCase`.

Q242. Check whether two strings are anagrams.

```
char[] a = "listen".toCharArray(), b = "silent".toCharArray();
Arrays.sort(a); Arrays.sort(b);
boolean anagram = Arrays.equals(a, b);   // true
```

O(n log n). O(n): count letters in an `int[26]`, increment for one string, decrement for the other, check all zeros.

Q243. ⋆ Count character frequency and find the first non-repeated character (streams).

```
Map<Character, Long> freq = "programming".chars().mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new, Collectors.counting()));
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}
char first = freq.entrySet().stream().filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey).findFirst().orElseThrow();   // p
```

`LinkedHashMap::new` keeps first-appearance order — that's what makes "first" correct. → *More:* §19.3 HashMap idioms

Q244. Find duplicate elements.

```
Set<Character> seen = new HashSet<>();
Set<Character> dups = "programming".chars().mapToObj(c -> (char) c)
        .filter(c -> !seen.add(c))
        .collect(Collectors.toCollection(LinkedHashSet::new));   // [r, m, g]
```

`Set.add` returns `false` for an element already present. Alternative: `groupingBy` + `counting()`, keep counts > 1. (The stateful `filter` is fine sequentially, not in a parallel stream.) → *More:* §19.3 HashMap idioms

Q245. ⋆ Find the second-highest number.

```
int second = List.of(5, 9, 2, 9, 7, 2, 4).stream().distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst().orElseThrow();   // 7
```

`distinct()` matters: without it the answer would be the duplicate 9. In a loop, track `max` and `second` in one O(n) pass.

Q246. ⋆ Employee stream tasks: count per department, average salary, top earner per department.

```
record Employee(String name, String dept, double salary, int age) {}
var emps = List.of(new Employee("Ana", "Eng", 120, 28), new Employee("Bo", "Eng", 100, 35),
        new Employee("Cy", "Ops", 90, 40), new Employee("Di", "Ops", 95, 25), new Employee("Ed", "HR", 70, 45));
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new, counting()));
// {Eng=2, HR=1, Ops=2}
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new, averagingDouble(Employee::salary)));
// {Eng=110.0, HR=70.0, Ops=92.5}
emps.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        collectingAndThen(maxBy(comparingDouble(Employee::salary)), o -> o.orElseThrow().name())));
// {Eng=Ana, HR=Ed, Ops=Di}
emps.stream().filter(e -> e.age() > 30).sorted(comparingDouble(Employee::salary).reversed())
        .map(Employee::name).collect(joining(", "));   // Bo, Cy, Ed
```

(Static imports from `Collectors` and `Comparator`.) Second-highest salary: `map(Employee::salary).distinct().sorted(reverseOrder()).skip(1).findFirst()`. → *More:* §12.5 Collectors

Q247. ⋆ Sort a map by its values.

```
Map<String, Integer> scores = Map.of("a", 3, "b", 1, "c", 2);
Map<String, Integer> sorted = scores.entrySet().stream()
        .sorted(Map.Entry.comparingByValue())
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));   // {b=1, c=2, a=3}
```

The `LinkedHashMap` supplier is essential — a plain `toMap` returns a `HashMap` and loses the order.

Q248. ⋆ Write a thread-safe singleton.

```
final class Config {                          // holder idiom: lazy, thread-safe, no locking
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    static Config get() { return Holder.INSTANCE; }
}
final class Dcl {                             // double-checked locking
    private static volatile Dcl instance;     // volatile is mandatory
    static Dcl get() {
        Dcl r = instance;
        if (r == null) synchronized (Dcl.class) { r = instance; if (r == null) instance = r = new Dcl(); }
        return r;
    }
}
```

Without `volatile`, another thread could see a non-null reference to a not-yet-fully-constructed object. Simplest of all: `enum Config { INSTANCE; }`.

Q249. ⋆ Implement an LRU cache.

```
class LRU<K, V> extends LinkedHashMap<K, V> {
    private final int cap;
    LRU(int cap) { super(16, 0.75f, true); this.cap = cap; }   // true = access order
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > cap; }
}
// cap 2: put 1, put 2, get 1, put 3 → keys [1, 3]
```

If asked to do it "from scratch", describe `HashMap<K, Node>` + doubly linked list (Q102). → *More:* §10.4 Maps

Q250. ⋆ Write an immutable class.

```java
final class Money {
    private final String currency;
    private final List<String> tags;
    Money(String currency, List<String> tags) {
        this.currency = Objects.requireNonNull(currency);
        this.tags = List.copyOf(tags);          // defensive, unmodifiable copy
    }
    String currency() { return currency; }
    List<String> tags() { return tags; }       // safe to return: can't be modified
}
```

Then point out a `record Money(String currency, List<String> tags)` with `tags = List.copyOf(tags)` in a compact constructor does the same. → *More:* §5.9 Immutability

Q251. Implement a simple `HashMap`.

```java
class SimpleHashMap<K, V> {
    private static class Node<K, V> { final K key; V value; Node<K, V> next;
        Node(K k, V v, Node<K, V> n) { key = k; value = v; next = n; } }
    private Node<K, V>[] table = newTable(16);
    private int size;
    private static int index(Object key, int n) { int h = Objects.hashCode(key); return (h ^ (h >>> 16)) & (n -
1); }
    public V get(K key) {
        for (Node<K, V> e = table[index(key, table.length)]; e != null; e = e.next)
            if (Objects.equals(e.key, key)) return e.value;
        return null;
    }
    public void put(K key, V value) {
        int i = index(key, table.length);
        for (Node<K, V> e = table[i]; e != null; e = e.next)
            if (Objects.equals(e.key, key)) { e.value = value; return; }
        table[i] = new Node<>(key, value, table[i]);
        if (++size > table.length * 3 / 4) resize();   // rehash every node into a 2× table
    }
    // newTable: @SuppressWarnings("unchecked") cast of new Node[n]; resize: loop over old buckets
}
```

Talk through: power-of-two capacity, hash spreading, chaining, load factor 0.75, `equals` for key match, null key via `Objects.hashCode`. (The full version with `resize` was run: 100 keys in, updates and lookups correct.) → *More:* §10.5 HashMap internals

Q252. ⋆ Write a program that deadlocks — then fix it.

```java
Object a = new Object(), b = new Object();
new Thread(() -> { synchronized (a) { sleep(100); synchronized (b) { } } }).start();
new Thread(() -> { synchronized (b) { sleep(100); synchronized (a) { } } }).start();
// both threads end up BLOCKED forever
```

Fix: make both threads take `a` then `b` (one global lock order), or use `ReentrantLock.tryLock(timeout)` and back off. → *More:* §15.9 Liveness

Q253. ⋆ Producer–consumer with a `BlockingQueue`.

```java
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(2);
Thread producer = Thread.ofPlatform().start(() -> {
    try { for (int v = 1; v <= 5; v++) q.put(v); q.put(-1); }        // -1 = "poison pill" end marker
    catch (InterruptedException e) { Thread.currentThread().interrupt(); } });
Thread consumer = Thread.ofPlatform().start(() -> {
    try { for (int v; (v = q.take()) != -1; ) got.add(v); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); } });
// got = [1, 2, 3, 4, 5]
```

`put` blocks when full, `take` when empty, so no explicit locking is needed. → *More:* §14.12 Concurrent collections

Q254. Producer–consumer with `wait`/`notify` (a bounded buffer).

```
final class BoundedQueue<T> {
    private final Deque<T> items = new ArrayDeque<>();
    private final int cap;
    BoundedQueue(int cap) { this.cap = cap; }
    synchronized void put(T t) throws InterruptedException {
        while (items.size() == cap) wait();      // while, not if: re-check after waking
        items.addLast(t); notifyAll();
    }
    synchronized T take() throws InterruptedException {
        while (items.isEmpty()) wait();
        T t = items.removeFirst(); notifyAll(); return t;
    }
}
```

→ *More:* §14.8 wait/notify

Q255. ⋆ Run two async calls and combine them with `CompletableFuture`.

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    var price = CompletableFuture.supplyAsync(() -> 100, ex);
    var rate  = CompletableFuture.supplyAsync(() -> 2, ex);
    int total = price.thenCombine(rate, (p, r) -> p * r)
            .orTimeout(1, TimeUnit.SECONDS)
            .exceptionally(t -> -1)       // fallback on failure or timeout
            .join();                      // 200
}
```

For N calls: `CompletableFuture.allOf(fs).thenApply(v -> fs.stream().map(CompletableFuture::join).toList())`.

Q256. ⋆ Print odd and even numbers alternately with two threads.

```
static void printTurn(Object lock, int[] counter, int parity, int max) {
    synchronized (lock) {
        while (counter[0] <= max) {
            if (counter[0] % 2 == parity) { System.out.print(counter[0]++ + " "); lock.notifyAll(); }
            else try { lock.wait(); } catch (InterruptedException e) { Thread.currentThread().interrupt(); return;
}
        }
        lock.notifyAll();   // release the other thread so it can exit
    }
}
// thread A: printTurn(lock, counter, 1, 10); thread B: printTurn(lock, counter, 0, 10)
// 1 2 3 4 5 6 7 8 9 10
```

Alternatives: two `Semaphore`s handing a permit back and forth, or a `ReentrantLock` with two `Condition`s.
→ *More:* §14.8 wait/notify

Q257. Count words; flatten a list of lists.

```
Map<String, Long> wc = Arrays.stream("to be or not to be".split(" "))
        .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()));
// {be=2, not=1, or=1, to=2}
List<Integer> flat = List.of(List.of(1, 2), List.of(3), List.<Integer>of())
        .stream().flatMap(List::stream).toList();   // [1, 2, 3]
```

Loop version of word count: `counts.merge(word, 1, Integer::sum)`. → *More:* §12.5 Collectors

Q258. Fibonacci and primes.

```
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
        .limit(10).map(f -> f[0]).toList();         // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
IntStream.rangeClosed(2, 30)
        .filter(n -> IntStream.rangeClosed(2, (int) Math.sqrt(n)).noneMatch(d -> n % d == 0))
        .boxed().toList();                          // [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Mention: recursive Fibonacci without memoisation is $O(2^n)$; the iterative one is $O(n)$. For factorials beyond 20!, `long` overflows — use `BigInteger` (25! = 15511210043330985984000000).

Q259. Merge two sorted arrays; find the missing number in 1..n.

```java
int i = 0, j = 0, k = 0;
while (i < a.length && j < b.length) m[k++] = a[i] <= b[j] ? a[i++] : b[j++];
while (i < a.length) m[k++] = a[i++];
while (j < b.length) m[k++] = b[j++];      // {1,3,5} + {2,4,6,8} → [1, 2, 3, 4, 5, 6, 8]
int missing = n * (n + 1) / 2 - Arrays.stream(arr).sum();   // {1,2,4,5}, n=5 → 3
```

For large n the sum can overflow — use `long` or XOR all values and indices.

Q260. String compression (`aaabccdddd` → `a3b1c2d4`).

```java
StringBuilder sb = new StringBuilder();
for (int i = 0; i < s.length(); ) {
    int j = i;
    while (j < s.length() && s.charAt(j) == s.charAt(i)) j++;
    sb.append(s.charAt(i)).append(j - i);
    i = j;
}
return sb.length() < s.length() ? sb.toString() : s;   // a3b1c2d4
```

Q261. Reverse the words of a sentence; count vowels.

```java
String.join(" ", Arrays.asList("the quick brown fox".split("\\s+")).reversed());  // fox brown quick the
"Interview".toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();    // 4
```

`List.reversed()` is a Java 21 sequenced-collection view; before 21 use `Collections.reverse`.

Q262. Swap two numbers without a temporary variable. `a = a ^ b; b = a ^ b; a = a ^ b;` (3, 7 → 7, 3). The arithmetic version `a = a + b; b = a - b; a = a - b;` can overflow. In real code just use a temp variable.

Q263. Count events from many threads correctly.

```java
var counter = new AtomicInteger();
try (var ex = Executors.newFixedThreadPool(4)) {
    for (int t = 0; t < 1000; t++) ex.submit(counter::incrementAndGet);
}                                       // close() waits for all tasks
System.out.println(counter.get());      // 1000 (a plain int++ would usually lose updates)
```

→ *More:* §14.10 Atomic variables

18. DSA problems in Java rounds

Java rounds usually include one or two classic LeetCode problems. Chapter 19 solves each pattern in idiomatic Java; here is the problem, the one-line approach, and where the full solution lives.

Q264. ⋆ Two Sum. One pass with a `HashMap<value, index>`: for each `x`, check whether `target − x` is already in the map. O(n). → §19.10 Hashing

Q265. ⋆ Longest substring without repeating characters. Sliding window: move `right`, and when a character repeats jump `left` past its previous position (stored in a map or `int[128]`). O(n). → §19.9 Sliding window

Q266. Valid parentheses. Push opening brackets on an `ArrayDeque`; on a closing bracket pop and check it matches; valid if the stack ends empty. → §19.12 Stacks

Q267. Reverse a linked list; detect a cycle. Reverse with three pointers `prev`/`cur`/`next`. Cycle: Floyd's fast/slow pointers meet if there is a cycle. → §19.22 Linked lists

Q268. ⋆ K-th largest element / top-k frequent. Keep a min-heap (`PriorityQueue`) of size k; the root is the answer. O(n log k). Quickselect gives O(n) average. → §19.13 Heaps

Q269. Binary search (and its overflow bug). Compute the middle as `lo + (hi - lo) / 2` (or `(lo + hi) >>> 1`); `(lo + hi) / 2` overflows for large indices. → §19.14 Binary search

Q270. ⋆ Merge overlapping intervals. Sort by start (`Comparator.comparingInt(a -> a[0])`), then extend the last merged interval or start a new one. O(n log n). → §19.15 Intervals

Q271. Group anagrams. Key each word by its sorted characters; `computeIfAbsent(key, k -> new ArrayList<>()).add(word)`. → §19.10 Hashing

Q272. Level-order traversal; number of islands. BFS with an `ArrayDeque` queue (process one level per `size()` snapshot); islands = count BFS/DFS starts over unvisited land cells. → §19.16 BFS and DFS

Q273. Maximum subarray, climbing stairs, coin change. Dynamic programming: Kadane's algorithm keeps the best sum ending here; stairs is Fibonacci; coin change fills `dp[amount]` bottom-up. → §19.19 Dynamic programming

Q274. Which Java-specific pitfalls cost points in coding rounds? `==` on `Integer`/`String`, `int` overflow in sums and midpoints, `Arrays.sort` on `int[]` with a comparator (needs `Integer[]`), comparators written as `a - b`, using `Stack`/`LinkedList` instead of `ArrayDeque`, string `+=` in loops, and forgetting `assert` is disabled by default. → §19.6 Pitfalls

Sources of the question lists

Question wording and frequency were taken from these public lists and reports (answers are this book's own):

- teamlead/java-interview-questions (GitHub) — questions ranked by how often they were asked in real interviews (high/moderate/low occurrence)
- learning-zone/java-basics (GitHub) — "Java Basics" interview question collection
- in28minutes/JavaInterviewQuestionsAndAnswers (GitHub) — 200+ Java interview questions list
- Devinterview-io/java-interview-questions (GitHub) — top Java interview questions
- DopplerHQ/awesome-interview-questions (GitHub) — Java section and the lists it links (Toptal "Essential Java Interview Questions", Java Code Geeks "115 Java Interview Questions")
- sats17/Java-8-Interview-Sample-Coding-Questions (GitHub) and the widely shared "commonly asked Java 8 stream coding questions" articles (Medium)
- LeetCode Discuss interview-experience posts for Java back-end roles (e.g. "Senior Software Engineer | Java tech stack": Java 8 features, `HashMap` and `ConcurrentHashMap` internals)
- Glassdoor interview reports for Java developer roles (`HashMap` vs `ConcurrentHashMap`, stream tasks on employee lists)
- LeetCode "Top Interview 150" problem list (for the DSA problems in group 18)